BẮT ĐẦU VỚI SPRING

Học đúng những gì bạn cần và làm chủ nền tảng

(Spring Start Here: Learn what you need and learn it well)

Tác giả: Laurențiu Spilcă

Bản dịch tiếng Việt xuất bản phục vụ mục đích học tập & nghiên cứu

Năm 2026

Mục lục

I Nền tảng cơ bản (Fundamentals) 1

1 Spring trong thế giới thực 2

1.1 Tại sao chúng ta nên sử dụng framework? . . . 3

1.2 Hệ sinh thái Spring . . . 7

1.2.1 Khám phá Spring Core: Nền tảng của Spring . . . 9

1.2.2 Sử dụng tính năng Spring Data Access để triển khai tầng lưu trữ dữ liệu của ứng dụng . . . 10

1.2.3 Năng lực của Spring MVC trong phát triển ứng dụng web . . . 10

1.2.4 Tính năng kiểm thử của Spring . . . 11

1.2.5 Các dự án trong hệ sinh thái Spring . . . 11

1.3 Spring trong các kịch bản thực tế . . . 13

1.3.1 Sử dụng Spring trong phát triển ứng dụng backend . . . 14

1.3.2 Sử dụng Spring trong ứng dụng kiểm thử tự động . . . 16

1.3.3 Sử dụng Spring cho phát triển ứng dụng máy tính để bàn (Desktop app) 18

1.3.4 Sử dụng Spring trong ứng dụng di động (Mobile apps) . . . 18

1.4 Khi nào không nên dùng framework . . . 19

1.4.1 Bạn cần mức chiếm dụng tài nguyên nhỏ (Small footprint) . . . 19

1.4.2 Yêu cầu bảo mật buộc phải dùng mã nguồn tự viết (Custom code) . 20

1.4.3 Tùy biến quá nhiều khiến việc dùng framework trở nên bất khả thi . 21

1.4.4 Bạn không thu được lợi ích gì từ việc chuyển đổi sang framework . 21

1.5 Bạn sẽ học được gì trong cuốn sách này . . . 24

Tóm tắt chương . . . 24

2 Ngữ cảnh Spring: Định nghĩa các bean 26

2.1 Tạo một dự án Maven 27

2.2 Thêm các bean mới vào ngữ cảnh Spring 33

2.2.1 Sử dụng chú thích @Bean để thêm bean vào ngữ cảnh Spring . . . 36

2.2.2 Sử dụng các chú thích định hình (Stereotype annotations) để thêm bean 43

2.2.3 Thêm bean vào ngữ cảnh Spring bằng phương thức lập trình 46

Tóm tắt chương 48

3 Ngữ cảnh Spring: Nối dây các bean 49

3.1 Thiết lập mối quan hệ giữa các bean trong tệp cấu hình 50

3.1.1 Nối dây các bean bằng cách gọi trực tiếp phương thức @Bean . . . 54

3.1.2 Nối dây các bean thông qua tham số của phương thức @Bean . . . 56

3.2 Sử dụng chú thích @Autowired để tiêm các bean 58

3.2.1 Tiêm giá trị qua trường của lớp (Field Injection) 58

3.2.2 Tiêm giá trị qua hàm khởi tạo (Constructor Injection) 60

3.2.3 Tiêm phụ thuộc qua phương thức setter (Setter Injection) 61

3.3 Xử lý phụ thuộc vòng (Circular Dependencies) 61

3.4 Lựa chọn trong số nhiều bean cùng kiểu trong ngữ cảnh 62

Tóm tắt chương 65

4 Ngữ cảnh Spring: Sử dụng các tầng trừu tượng 67

4.1 Sử dụng interface để định nghĩa các hợp đồng 68

4.1.1 Sử dụng interface để tách rời các cài đặt cụ thể 68

4.1.2 Yêu cầu của kịch bản nghiệp vụ 72

4.1.3 Hiện thực yêu cầu mà không sử dụng framework 73

4.2 Sử dụng tiêm phụ thuộc với các tầng trừu tượng 79

4.2.1 Quyết định đối tượng nào nên thuộc ngữ cảnh Spring 80

4.2.2 Lựa chọn bean cần tự động liên kết khi có nhiều cài đặt của một tầng trừu tượng 87

4.3 Làm nổi bật trách nhiệm của đối tượng bằng các chú thích định hình 92

4.4 Tóm tắt chương 95

5 Ngữ cảnh Spring: Phạm vi và vòng đời của bean 96

5.1 Sử dụng phạm vi bean singleton 97

5.1.1 Cách thức hoạt động của singleton bean 97

5.1.2 Singleton bean trong các kịch bản thực tế 105

5.1.3 Sử dụng cơ chế khởi tạo tức thì (Eager) và khởi tạo trễ (Lazy) . . . 107

5.2 Sử dụng phạm vi bean prototype 110

5.2.1 Cách thức hoạt động của prototype bean 110

5.2.2 Prototype bean trong các kịch bản thực tế 116

5.3 Tóm tắt chương 121

6 Sử dụng các khía cạnh với Spring AOP 123

6.1 Cách thức hoạt động của các khía cạnh trong Spring 125

6.2 Triển khai các khía cạnh với Spring AOP 128

6.2.1 Triển khai một khía cạnh đơn giản 129

6.2.2 Thay đổi tham số và giá trị trả về của phương thức bị chặn 137

6.2.3 Đan dệt khía cạnh vào các phương thức được gắn chú thích 141

6.2.4 Các chú thích can thiệp (Advice) khác trong Spring 143

6.3 Chuỗi thực thi các khía cạnh (Aspect Execution Chain) 145

6.4 Tóm tắt chương 150

II Triển khai thực tế (Implementation) 151

7 Tìm hiểu Spring Boot và Spring MVC 152

7.1 Ứng dụng web là gì? 153

7.1.1 Bức tranh tổng quan về ứng dụng web 153

7.1.2 Các cách tiếp cận xây dựng ứng dụng web với Spring 155

7.1.3 Sử dụng Servlet Container trong phát triển ứng dụng web Java . . . 157

7.2 Phép màu của Spring Boot 160

7.2.1 Khởi tạo dự án Spring Boot bằng dịch vụ Spring Initializr 160

7.2.2 Sử dụng Dependency Starters để đơn giản hóa quản lý phụ thuộc . . 164

7.2.3 Cơ chế tự động cấu hình theo quy ước (Convention-over-Configuration) 164
7.3 Triển khai ứng dụng web với Spring MVC 165
7.3.1 Bước 1: Viết tài liệu HTML 166
7.3.2 Bước 2: Định nghĩa lớp Controller 167
7.3.3 Chạy và kiểm thử ứng dụng trên trình duyệt 167
7.3.4 Cơ chế kiến trúc Spring MVC vận hành đằng sau hậu trường 169
7.4 Tóm tắt chương 170

8 Triển khai ứng dụng web với Spring Boot và Spring MVC 172
8.1 Triển khai ứng dụng web với khung nhìn động 174
8.1.1 Tiếp nhận dữ liệu từ yêu cầu HTTP 177
8.1.2 Sử dụng tham số yêu cầu (Request Parameters) để truyền dữ liệu . . 178
8.1.3 Sử dụng biến đường dẫn (Path Variables) để truyền dữ liệu 182
8.2 Sử dụng các phương thức HTTP GET và POST 183
8.2.1 Xây dựng ứng dụng quản lý sản phẩm với GET và POST 185
8.2.2 Xử lý thêm sản phẩm với phương thức HTTP POST 188
8.2.3 Rút gọn mã nguồn bằng cách liên kết trực tiếp đối tượng Model . . 191
8.3 Tóm tắt chương 193

9 Sử dụng các phạm vi web của Spring 195
9.1 Sử dụng phạm vi yêu cầu (Request scope) trong ứng dụng Spring 197
9.1.1 Hiện thực hóa chức năng đăng nhập với Request Scope 198
9.1.2 Cơ chế Scoped Proxy của Spring đằng sau Request Scope 202
9.2 Sử dụng phạm vi phiên (Session scope) trong ứng dụng Spring 205
9.2.1 Xây dựng dịch vụ quản lý người dùng đăng nhập với Session Scope 207
9.2.2 Bảo vệ trang chủ và điều hướng người dùng 210
9.2.3 Tính năng đăng xuất (Logout) và kỹ thuật chuyển hướng (Redirection) 212
9.3 Sử dụng phạm vi ứng dụng (Application scope) trong ứng dụng Spring . . . 214
9.3.1 Đếm tổng số lượt đăng nhập thành công với Application Scope . . 215
9.4 Tóm tắt chương 218

10 Xây dựng các dịch vụ REST **220**

10.1 Dịch vụ REST là gì? . . . 221

10.2 Triển khai các điểm cuối REST trong Spring . . . 222

10.2.1 Điểm cuối REST đầu tiên với chú thích @ResponseBody . . . 223

10.2.2 Tối ưu hóa mã nguồn với chú thích @RestController . . . 225

10.2.3 Trả về đối tượng Java và cơ chế chuyển đổi JSON tự động . . . 226

10.3 Tùy biến phản hồi HTTP với ResponseEntity . . . 229

10.4 Quản lý và xử lý ngoại lệ trong dịch vụ REST . . . 230

10.4.1 Xử lý ngoại lệ cục bộ bên trong Controller . . . 230

10.4.2 Tách biệt xử lý ngoại lệ tập trung với @RestControllerAdvice . . . 232

10.5 Nhận dữ liệu từ Client qua phần thân yêu cầu (Request Body) . . . 234

10.6 Tóm tắt chương . . . 236

11 Tiêu thụ các điểm cuối REST **238**

11.1 Gọi các điểm cuối REST bằng Spring Cloud OpenFeign . . . 239

11.1.1 Sức mạnh của OpenFeign: Lập trình client dạng khai báo . . . 241

11.2 Gọi các điểm cuối REST bằng RestTemplate . . . 244

11.3 Gọi các điểm cuối REST bằng WebClient (Mô hình phản ứng) . . . 246

11.3.1 Hạn chế của mô hình ngăn chặn và sức mạnh của lập trình phản ứng 246

11.3.2 Sử dụng WebClient trong dự án Spring WebFlux . . . 249

11.4 Tóm tắt chương . . . 251

12 Sử dụng nguồn dữ liệu trong ứng dụng Spring **253**

12.1 Nguồn dữ liệu (Data Source) là gì? . . . 254

12.1.1 Cơ chế Bể chứa kết nối (Connection Pool) . . . 256

12.2 Sử dụng JdbcTemplate để thao tác với dữ liệu . . . 257

12.2.1 Xây dựng ứng dụng quản lý đơn hàng với JdbcTemplate . . . 257

12.2.2 Thêm mới dữ liệu với phương thức update() . . . 260

12.2.3 Truy vấn dữ liệu và ánh xạ bằng RowMapper . . . 260

12.3 Tùy biến cấu hình DataSource . . . 263

12.4 Tóm tắt chương . . . 264

13 Sử dụng giao dịch trong ứng dụng Spring **266**

13.1 Giao dịch (Transactions) là gì? 267

13.2 Cơ chế hoạt động của giao dịch trong Spring 269

13.3 Triển khai giao dịch trong ứng dụng Spring 270

13.4 Kiểm thử cơ chế Rollback khi xảy ra lỗi 276

13.5 Tóm tắt chương 278

14 Triển khai lưu trữ dữ liệu với Spring Data (Implementing data persistence with Spring Data) **279**

14.1 Spring Data là gì? (What Spring Data is) 280

14.2 Cơ chế hoạt động của Spring Data (How Spring Data works) 282

14.3 Sử dụng Spring Data JDBC trong ứng dụng thực tế 287

14.3.1 Tự động sinh câu lệnh truy vấn từ tên phương thức 291

14.3.2 Khai báo tường minh truy vấn với chú thích @Query 292

14.3.3 Triển khai tầng dịch vụ (Service) và quản lý giao dịch 294

14.3.4 Phơi bày các điểm cuối REST Controller và kiểm thử 296

14.4 Tóm tắt nội dung chương (Summary) 300

15 Kiểm thử ứng dụng Spring (Testing your Spring app) **302**

15.1 Vị trí đúng đắn của kiểm thử trong thiết kế phần mềm 303

15.2 Triển khai kiểm thử trong ứng dụng Spring 305

15.2.1 Triển khai kiểm thử đơn vị (Unit tests) 306

15.2.2 Tối ưu hóa mã kiểm thử bằng các chú thích Mockito 311

15.2.3 Kiểm thử luồng phát sinh ngoại lệ (Exception Flows) 313

15.2.4 Kiểm thử giá trị trả về của phương thức Controller 315

15.2.5 Triển khai kiểm thử tích hợp (Spring Integration Tests) 316

15.3 Tóm tắt nội dung chương (Summary) 319

A Các phương pháp tiếp cận kiến trúc (Architectural approaches) **320**

A.1 Phương pháp tiếp cận khối thống nhất (The monolithic approach) 320

A.2 Kiến trúc hướng dịch vụ (Service-Oriented Architecture - SOA) 322

A.3 Từ Microservices đến Serverless 324
A.4 Tài liệu đọc thêm (Further reading) 327

B Cấu hình Spring Context bằng XML (Using XML for the context configuration) 328

C Giới thiệu nhanh về giao thức HTTP (A quick introduction to HTTP) 330
C.1 HTTP là gì? (What is HTTP?) 330
C.2 Yêu cầu HTTP (The HTTP request) 331
C.3 Phản hồi HTTP (The HTTP response) 331
C.4 Phiên làm việc HTTP (The HTTP session) 333

D Định dạng dữ liệu JSON (Using JSON formatting) 335

E Cài đặt MySQL và tạo cơ sở dữ liệu (Installing MySQL and creating a database) 338
E.1 Các bước thực hiện 338

F Các công cụ khuyên dùng (Recommended tools) 343
F.1 Môi trường phát triển tích hợp (IDEs) 343
F.2 Công cụ kiểm thử RESTful API 343
F.3 Hệ quản trị cơ sở dữ liệu và công cụ quản trị 344

G Tài liệu học tập khuyên đọc để nghiên cứu sâu hơn (Recommended learning materials for further study) 345

Phần I

Nền tảng cơ bản (Fundamentals)

Chương 1

Spring trong thế giới thực

Khung phát triển Spring (gọi tắt là **Spring**) là một khung phát triển ứng dụng (*application framework*) thuộc hệ sinh thái Java. Một khung phát triển ứng dụng là một tập hợp các chức năng phần mềm dùng chung nhằm cung cấp cấu trúc nền tảng cho việc phát triển một ứng dụng. Khung phát triển ứng dụng giúp giảm thiểu công sức xây dựng ứng dụng bằng cách loại bỏ việc phải tự viết toàn bộ mã nguồn chương trình từ con số không.

Ngày nay, chúng ta sử dụng Spring trong việc phát triển rất nhiều loại ứng dụng khác nhau, từ các giải pháp backend quy mô lớn cho đến các ứng dụng kiểm thử tự động. Theo nhiều báo cáo khảo sát về các công nghệ Java (chẳng hạn khảo sát của JRebel năm 2020 [2]: `http://mng.bz/N4V7`; hay khảo sát từ JAXEnter [1]: `http://mng.bz/DK9a`), Spring là framework Java được sử dụng phổ biến nhất hiện nay.

Spring rất phổ biến, và các lập trình viên cũng bắt đầu sử dụng nó thường xuyên hơn với các ngôn ngữ máy ảo JVM khác ngoài Java. Trong vài năm qua, chúng ta đã chứng kiến sự gia tăng ấn tượng của các nhà phát triển sử dụng Spring kết hợp với Kotlin (một ngôn ngữ được đánh giá cao khác thuộc gia đình JVM). Trong cuốn sách này, chúng ta sẽ tập trung vào các nền tảng cốt lõi của Spring, và tôi sẽ trang bị cho bạn những kỹ năng thiết yếu để áp dụng Spring vào các ví dụ thực tế. Nhằm giúp chủ đề này trở nên dễ tiếp cận nhất và cho phép bạn tập trung trọn vẹn vào Spring, chúng ta sẽ chỉ sử dụng các ví dụ bằng Java. Xuyên suốt cuốn sách, chúng ta sẽ cùng thảo luận và thực hành, thông qua các ví dụ, những kỹ năng thiết yếu như kết nối cơ sở dữ liệu, thiết lập giao tiếp giữa các ứng dụng, bảo mật và kiểm thử một ứng dụng.

Trước khi đi sâu vào các chi tiết kỹ thuật hơn trong những chương kế tiếp, hãy cùng bàn luận về khung phát triển Spring và nơi bạn sẽ thực sự sử dụng nó. Tại sao Spring lại được trân trọng đến vậy, và khi nào thì bạn nên sử dụng nó?

Trong chương này, chúng ta sẽ tập trung vào bản chất của một framework, đặc biệt liên hệ tới khung phát triển Spring. Trong mục 1.1, chúng ta sẽ thảo luận về những lợi thế của việc sử dụng framework. Trong mục 1.2, chúng ta tìm hiểu hệ sinh thái Spring cùng các thành phần

bạn cần nắm vững để bắt đầu với Spring. Sau đó, tôi sẽ đưa bạn qua các trường hợp sử dụng khả dĩ của Spring — cụ thể là các kịch bản thực tế trong mục 1.3. Trong mục 1.4, chúng ta sẽ phân tích khi nào việc sử dụng framework có thể không phải là một giải pháp đúng đắn. Bạn cần thấu hiểu tất cả những điều này về Spring trước khi bắt tay vào sử dụng nó. Nếu không, bạn có thể sẽ rơi vào tình huống dùng búa để đào đất trong vườn nhà mình.

Tùy thuộc vào xuất phát điểm của bạn, bạn có thể cảm thấy chương này có phần trừu tượng. Tôi có thể sẽ đề cập đến một số khái niệm mà bạn chưa từng nghe qua, và điều này có thể khiến bạn đôi chút băn khoăn. Nhưng xin đừng lo lắng; ngay cả khi bạn chưa hiểu hết một số điều ngay lúc này, chúng sẽ được làm sáng tỏ dần trong các phần tiếp theo của cuốn sách. Đôi khi, xuyên suốt cuốn sách, tôi sẽ liên hệ lại với những điều đã được đề cập ở các chương trước. Tôi lựa chọn cách tiếp cận này bởi vì việc học một framework như Spring không phải lúc nào cũng mang lại một lộ trình học tập tuyến tính; đôi khi bạn cần phải kiên nhẫn chờ đợi cho đến khi thu thập đủ các mảnh ghép của trò chơi xếp hình trước khi nhìn thấy toàn bộ bức tranh hoàn chỉnh. Nhưng cuối cùng, bạn chắc chắn sẽ có được một hình dung rõ ràng, cùng những kỹ năng quý báu để phát triển ứng dụng một cách chuyên nghiệp.

Nội dung chính của chương này

- Khung phát triển (framework) là gì.
- Khi nào nên sử dụng và khi nào nên tránh sử dụng framework.
- Bản chất của khung phát triển Spring.
- Ứng dụng Spring trong các kịch bản thực tế.

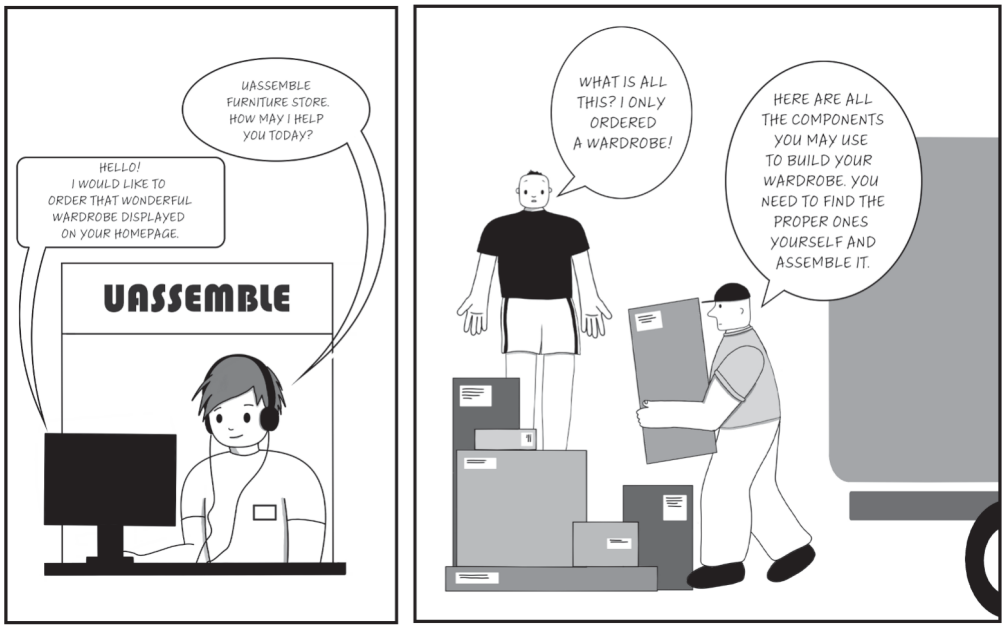
1.1 Tại sao chúng ta nên sử dụng framework?

Trong mục này, chúng ta sẽ cùng thảo luận về framework. Chúng là gì? Khái niệm này xuất hiện như thế nào và tại sao? Để có động lực sử dụng một công cụ nào đó, bạn cần hiểu rõ công cụ đó mang lại giá trị gì cho mình. Và điều này hoàn toàn đúng với Spring. Tôi sẽ truyền đạt cho bạn những chi tiết thiết yếu này bằng cách chia sẻ những hiểu biết tích lũy từ trải nghiệm thực tế của chính bản thân mình, thông qua quá trình nghiên cứu và áp dụng nhiều framework khác nhau trong các bài toán thực tế, bao gồm cả Spring.

Một khung phát triển ứng dụng (*application framework*) là một tập hợp các chức năng mà dựa trên đó chúng ta xây dựng nên các ứng dụng. Khung phát triển ứng dụng cung cấp cho chúng ta một tập hợp phong phú các công cụ và chức năng mà bạn có thể khai thác để phát triển phần mềm. Bạn không nhất thiết phải sử dụng toàn bộ các tính năng mà framework cung cấp. Tùy thuộc vào yêu cầu của ứng dụng bạn đang xây dựng, bạn sẽ chọn lọc đúng những phần cần thiết của framework để sử dụng.

Sau đây là một hình ảnh so sánh mà tôi rất tâm đắc về khung phát triển ứng dụng. Bạn đã

bao giờ mua một món đồ nội thất từ một cửa hàng tự lắp ráp (DIY) như Ikea chưa? Giả sử bạn mua một chiếc tủ quần áo — bạn sẽ không nhận được một chiếc tủ đã lắp ráp sẵn nguyên vẹn, mà là đúng các bộ phận linh kiện cần thiết để dựng nó lên, cùng một cuốn sổ hướng dẫn cách lắp ráp món đồ nội thất đó. Bây giờ hãy tưởng tượng bạn đặt mua một chiếc tủ, nhưng thay vì chỉ nhận được các linh kiện của chiếc tủ đó, bạn lại nhận được tất cả các linh kiện khả dĩ có thể dùng để lắp ráp bất kỳ món đồ nội thất nào: bàn, ghế, tủ quần áo, v.v. Nếu muốn có một chiếc tủ quần áo, bạn phải tự tìm đúng các bộ phận tương ứng và ráp chúng lại với nhau. Một khung phát triển ứng dụng cũng hoạt động y hệt như vậy. Khung phát triển ứng dụng cung cấp cho bạn vô số mảnh ghép phần mềm mà bạn cần để xây dựng ứng dụng. Bạn cần phải biết mình nên chọn tính năng nào và lắp ghép chúng ra sao để thu được kết quả mong muốn (Hình 1.1).



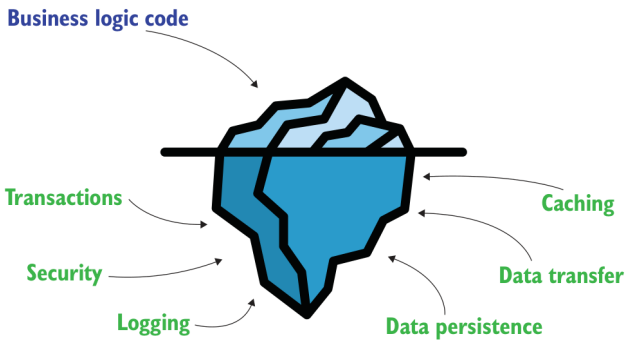
Hình 1.1: David đặt mua một chiếc tủ quần áo từ cửa hàng UAssemble. Cửa hàng (đóng vai trò framework) không chỉ giao cho David (lập trình viên) đúng các linh kiện (năng lực phần mềm) cần thiết để lắp tủ quần áo mới (ứng dụng). Cửa hàng gửi cho anh tất cả các bộ phận có thể cần để dựng tủ. Quyết định lựa chọn thành phần nào và lắp ráp chúng ra sao để tạo ra kết quả chuẩn xác hoàn toàn thuộc về David.

Ý tưởng về một framework hoàn toàn không mới mẻ. Xuyên suốt lịch sử phát triển phần mềm, các lập trình viên đã sớm nhận ra rằng họ có thể tái sử dụng lại các đoạn mã đã viết cho nhiều ứng dụng khác nhau. Ban đầu, khi số lượng ứng dụng được triển khai còn ít, mỗi ứng dụng đều mang tính độc bản và được viết hoàn toàn từ đầu bằng một ngôn ngữ lập trình cụ thể. Khi lĩnh vực phát triển phần mềm ngày càng mở rộng và ngày càng có nhiều ứng dụng xuất hiện trên thị trường, người ta dễ dàng nhận thấy rằng phần lớn các ứng dụng này đều có những yêu cầu chung tương tự nhau. Hãy cùng điểm qua một vài yêu cầu điển hình:

- Việc ghi nhật ký (logging) các thông báo lỗi (*error*), cảnh báo (*warning*) và thông tin (*info*) xuất hiện trong mọi ứng dụng.
- Hầu hết các ứng dụng đều sử dụng các giao dịch (*transactions*) để xử lý các thay đổi dữ liệu. Giao dịch đại diện cho một cơ chế trọng yếu nhằm đảm bảo tính nhất quán của dữ liệu. Chúng ta sẽ thảo luận chi tiết chủ đề này trong Chương 13.
- Hầu hết các ứng dụng đều áp dụng các cơ chế bảo vệ để chống lại những lỗ hổng bảo mật phổ biến giống nhau.
- Hầu hết các ứng dụng đều sử dụng những phương thức tương đồng để giao tiếp qua lại với nhau.
- Hầu hết các ứng dụng đều dùng các kỹ thuật tương tự để nâng cao hiệu năng, chẳng hạn như bộ nhớ đệm (*caching*) hay nén dữ liệu.

Và danh sách này vẫn còn kéo dài nữa. Thực tế chứng minh rằng phần mã logic nghiệp vụ (*business logic*) được triển khai trong một ứng dụng thường có khối lượng nhỏ hơn đáng kể so với những bánh răng và trục xích tạo nên động cơ vận hành của ứng dụng (phần này thường được giới chuyên môn ví von là "hệ thống ống dẫn" - *plumbing*).

Khi tôi nói "mã logic nghiệp vụ", tôi muốn chỉ tới phần mã nguồn hiện thực hóa các yêu cầu nghiệp vụ thực tế của ứng dụng. Phần mã này chính là thứ đáp ứng những kỳ vọng của người dùng đối với một ứng dụng. Ví dụ, "khi nhấn vào một liên kết nhất định thì hệ thống sẽ xuất ra hóa đơn thanh toán" là một điều mà người dùng mong đợi sẽ xảy ra. Một đoạn mã cụ thể trong ứng dụng của bạn sẽ đảm nhiệm chức năng này, và phần mã đó chính là thứ mà các nhà phát triển gọi là mã logic nghiệp vụ. Tuy nhiên, bất kỳ ứng dụng nào cũng đều phải chăm lo thêm rất nhiều khía cạnh kỹ thuật khác: bảo mật, ghi log, tính nhất quán của dữ liệu, và nhiều yếu tố phụ trợ khác (Hình 1.2).



Hình 1.2: Góc nhìn của người dùng giống như một tảng băng trôi. Người dùng chủ yếu chỉ quan sát thấy kết quả của mã logic nghiệp vụ, nhưng đây chỉ là một phần rất nhỏ cấu thành nên chức năng hoàn chỉnh của ứng dụng. Giống như phần chìm khổng lồ của tảng băng nằm dưới mặt nước, trong một ứng dụng doanh nghiệp, chúng ta không nhìn thấy phần lớn mã nguồn vì nó đã được cung cấp sẵn bởi các thư viện phụ thuộc (dependencies).

Hơn thế nữa, mã logic nghiệp vụ chính là yếu tố tạo nên sự khác biệt về mặt chức năng giữa

ứng dụng này với ứng dụng khác. Nếu bạn xem xét hai ứng dụng hoàn toàn khác nhau, chẳng hạn như một hệ thống gọi xe công nghệ và một ứng dụng mạng xã hội, chúng phục vụ những trường hợp sử dụng (*use case*) rất khác biệt.

Khái niệm Ca sử dụng (Use Case)

Một ca sử dụng (*use case*) đại diện cho lý do hoặc mục đích khiến một người sử dụng ứng dụng. Ví dụ, trong một ứng dụng gọi xe, một ca sử dụng điển hình là "yêu cầu đặt xe". Đối với một ứng dụng quản lý giao đồ ăn, một ca sử dụng là "đặt mua một chiếc pizza".

Bạn thực hiện các hành động khác nhau, nhưng cả hai ứng dụng đều cần tới khả năng lưu trữ dữ liệu, truyền tải dữ liệu, ghi log, thiết lập cấu hình bảo mật, cơ chế lưu đệm (cache), v.v. Các ứng dụng khác nhau hoàn toàn có thể tái sử dụng lại những triển khai phi nghiệp vụ này. Vậy thì liệu việc viết lại từ đầu những chức năng dùng chung này cho mỗi lần làm dự án mới có hiệu quả hay không? Chắc chắn là không:

- Bạn tiết kiệm được rất nhiều thời gian và tiền bạc bằng cách tái sử dụng một giải pháp có sẵn thay vì tự mình phát triển lại từ đầu.
- Một bản triển khai sẵn có đã được rất nhiều ứng dụng kiểm chứng trong thực tế sẽ có xác suất phát sinh lỗi (bug) thấp hơn nhiều, bởi vì nó đã được cộng đồng thử nghiệm kỹ lưỡng.
- Bạn được hưởng lợi từ sự tư vấn và hỗ trợ của cả một cộng đồng rộng lớn, bởi vì có rất nhiều nhà phát triển cùng am hiểu sâu sắc về chức năng đó. Nếu bạn tự viết mã nguồn riêng cho mình, chỉ có rất ít người trong nhóm của bạn hiểu được nó.

Mối liên hệ giữa Framework và Kiến trúc

Việc lựa chọn và sử dụng một framework có mối liên hệ mật thiết với thiết kế và kiến trúc tổng thể của ứng dụng. Bạn sẽ thấy rất hữu ích khi tìm hiểu thêm về các chủ đề này song song với quá trình học khung phát triển Spring. Trong Phụ lục A, bạn sẽ tìm thấy phần thảo luận chuyên sâu về các kiến trúc phần mềm cùng các tài liệu tham khảo tuyệt vời nếu bạn muốn đào sâu chi tiết.

Một câu chuyện chuyển đổi (A story of transition)

Một trong những ứng dụng đầu tiên mà tôi tham gia phát triển là một hệ thống khổng lồ được xây dựng bằng Java. Hệ thống này bao gồm nhiều ứng dụng được thiết kế xoay quanh kiến trúc máy chủ kiểu cũ, tất cả đều được viết từ đầu bằng Java SE thuần. Quá trình phát triển ứng dụng này đã khởi đầu cùng với sự ra đời của ngôn ngữ Java cách đây khoảng 25 năm. Đó là lý do chính định hình nên hình hài ban đầu của nó, và hầu như không ai có thể ngờ rằng hệ thống ấy lại phình to đến mức độ như vậy. Vào thời điểm đó, các khái niệm nâng cao hơn về kiến trúc hệ thống vẫn chưa xuất hiện, và nhìn chung mọi thứ vận hành rất khác biệt so với các hệ thống đơn lẻ ngày nay do tốc độ kết nối internet còn hạn chế.

Nhưng thời gian thoi đưa, nhiều năm trôi qua, ứng dụng dần biến thành một "mớ bòng bong" khổng lồ (*big ball of mud*). Vì những lý do hoàn toàn xác đáng mà tôi sẽ không đi sâu ở đây, đội ngũ phát triển quyết định phải chuyển đổi sang một kiến trúc hiện đại hơn. Sự thay đổi này trước hết đòi hỏi phải dọn dẹp mã nguồn, và một trong những bước đi then chốt là đưa một framework vào sử dụng. Chúng tôi đã quyết định chọn Spring. Vào thời điểm đó, chúng tôi có một lựa chọn thay thế là Java EE (nay đổi tên thành Jakarta EE), nhưng phần lớn các thành viên trong nhóm đều nhận định rằng tốt hơn hết là đi cùng Spring — một giải pháp nhẹ nhàng hơn, dễ triển khai hơn và cũng dễ bảo trì hơn theo đánh giá của chúng tôi.

Cuộc chuyển đổi này không hề đơn giản. Cùng với một vài đồng nghiệp là những chuyên gia xuất sắc trong lĩnh vực của họ và nắm rất rõ bản thân ứng dụng, chúng tôi đã đầu tư rất nhiều công sức vào cuộc cải tổ này.

Và kết quả thu được thật đáng kinh ngạc! Chúng tôi đã loại bỏ được hơn 40% tổng số dòng mã nguồn thừa thãi. Bước chuyển mình ấy chính là khoảnh khắc đầu tiên giúp tôi thấu hiểu sâu sắc tác động to lớn mà việc sử dụng một framework có thể mang lại.

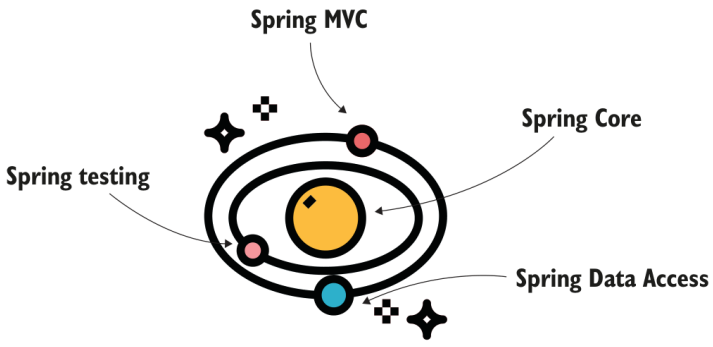
1.2 Hệ sinh thái Spring

Trong mục này, chúng ta sẽ cùng thảo luận về Spring và các dự án liên quan mật thiết như Spring Boot hay Spring Data. Bạn sẽ học tất cả những điều này trong cuốn sách, cũng như mối liên kết chặt chẽ giữa chúng. Trong các tình huống thực tế, việc kết hợp nhiều framework khác nhau cùng hoạt động là rất phổ biến, trong đó mỗi framework được thiết kế chuyên biệt để giúp bạn hiện thực hóa một phần cụ thể của ứng dụng nhanh chóng hơn.

Chúng ta thường gọi Spring là một framework, nhưng thực tế nó phức tạp và toàn diện hơn thế rất nhiều. Spring là một **hệ sinh thái các khung phát triển** (*an ecosystem of frameworks*). Thông thường, khi các lập trình viên nhắc tới cụm từ "khung phát triển Spring", họ thường ám chỉ một phần năng lực phần mềm bao gồm các trụ cột sau:

1. **Spring Core:** Một trong những phần nền tảng cốt lõi của Spring, chứa đựng các năng lực cơ sở. Một trong những tính năng này là ngữ cảnh Spring (*Spring context*). Như bạn sẽ được học chi tiết trong Chương 2, Spring context là một năng lực căn bản cho phép Spring quản lý các thể hiện đối tượng (instances) trong ứng dụng của bạn. Ngoài ra, thuộc Spring Core còn có tính năng khía cạnh (*aspects*). Các khía cạnh giúp Spring đánh chặn và can thiệp vào các phương thức do bạn định nghĩa trong ứng dụng. Chúng ta sẽ bàn kỹ hơn về aspects trong Chương 6. Ngôn ngữ biểu thức Spring (*Spring Expression Language - SpEL*) cũng là một năng lực nằm trong Spring Core, cho phép bạn mô tả cấu hình cho Spring bằng một cú pháp biểu thức chuyên biệt. Tất cả những điều này đều là các khái niệm mới, và tôi chưa kỳ vọng bạn phải biết ngay lúc này. Nhưng chẳng mấy chốc bạn sẽ hiểu rằng Spring Core nắm giữ toàn bộ cơ chế mà Spring sử dụng để tích hợp vào ứng dụng của bạn.
2. **Spring Model-View-Controller (MVC):** Phần của Spring cho phép bạn phát triển các ứng dụng web phục vụ các yêu cầu HTTP. Chúng ta sẽ bắt đầu sử dụng Spring MVC từ Chương 7.
3. **Spring Data Access:** Cũng là một phần nền tảng của Spring. Module này cung cấp các công cụ cơ bản để bạn kết nối với các cơ sở dữ liệu SQL nhằm triển khai tầng lưu trữ dữ liệu bền vững (*persistence layer*) của ứng dụng. Chúng ta sẽ bắt đầu khai thác Spring Data Access từ Chương 12.
4. **Spring testing:** Phần nắm giữ các công cụ bạn cần để viết kiểm thử cho ứng dụng Spring của mình. Chúng ta sẽ bàn luận về chủ đề này trong Chương 15.

Ban đầu, bạn có thể hình dung khung phát triển Spring giống như một hệ mặt trời, trong đó Spring Core đóng vai trò là ngôi sao trung tâm, giữ cho toàn bộ hệ thống framework liên kết chặt chẽ với nhau (Hình 1.3).



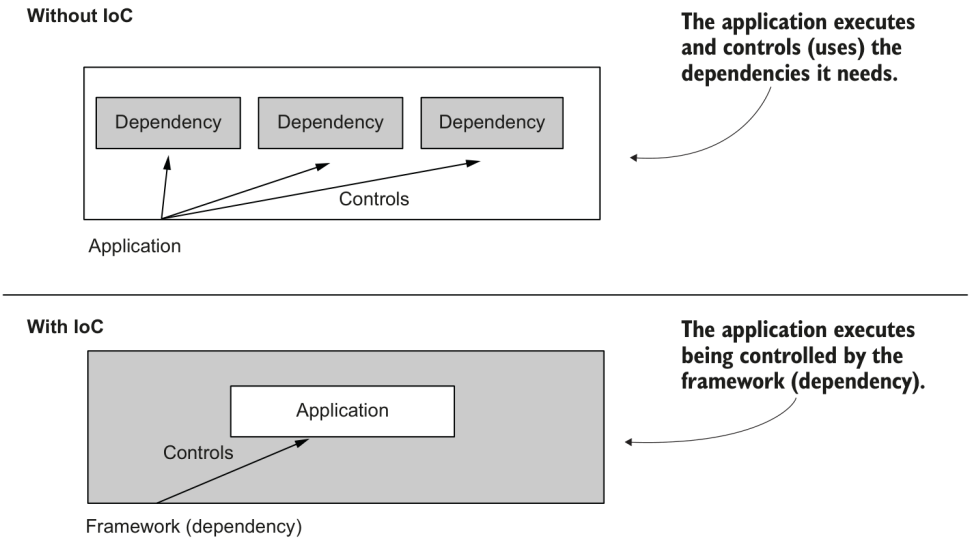
Hình 1.3: Bạn có thể hình dung khung phát triển Spring như một hệ mặt trời với Spring Core nằm ở tâm điểm. Các năng lực phần mềm khác là các hành tinh quay quanh Spring Core và được giữ chặt nhờ trường trọng lực của nó.

1.2.1 Khám phá Spring Core: Nền tảng của Spring

Spring Core là phần của khung phát triển Spring cung cấp các cơ chế nền tảng để tích hợp vào các ứng dụng. Spring vận hành dựa trên nguyên lý **Nghịch đảo quyền điều khiển** (*Inversion of Control - IoC*). Khi áp dụng nguyên lý này, thay vì để ứng dụng tự kiểm soát luồng thực thi, chúng ta trao quyền điều khiển cho một phần mềm khác — trong trường hợp của chúng ta chính là khung phát triển Spring. Thông qua các cấu hình, chúng ta chỉ dẫn cho framework cách thức quản lý phần mã mà chúng ta viết ra, nơi định nghĩa logic của ứng dụng. Đó chính là nguồn gốc của từ "nghịch đảo" trong IoC: bạn không để ứng dụng kiểm soát việc thực thi bằng mã riêng của nó và tự gọi các thư viện phụ thuộc. Thay vào đó, chúng ta cho phép framework (đóng vai trò phụ thuộc) kiểm soát ứng dụng và mã nguồn của nó (Hình 1.4).

Ý nghĩa của khái niệm "Điều khiển"

Trong ngữ cảnh này, thuật ngữ "điều khiển" (controls) ám chỉ các hành động như "tạo một thể hiện đối tượng" hoặc "gọi một phương thức". Một framework có thể tự tạo ra các đối tượng từ những lớp mà bạn định nghĩa trong ứng dụng. Dựa trên các cấu hình mà bạn thiết lập, Spring sẽ đánh chặn phương thức để bổ sung thêm nhiều tính năng phong phú. Ví dụ, Spring có thể chặn một phương thức cụ thể để ghi log lại bất kỳ lỗi nào có thể phát sinh trong quá trình phương thức đó thực thi.



Hình 1.4: Nghịch đảo quyền điều khiển (Inversion of Control - IoC). Thay vì ứng dụng tự thực thi mã của mình và trực tiếp gọi các thư viện phụ thuộc, trong kịch bản IoC, việc thực thi của ứng dụng được kiểm soát bởi thư viện/framework. Khung phát triển Spring kiểm soát ứng dụng trong suốt quá trình chạy, do đó nó triển khai một kịch bản thực thi theo mô thức IoC.

Bạn sẽ bắt đầu hành trình học tập Spring với Spring Core bằng việc tìm hiểu chức năng Spring

IoC trong các Chương 2 đến Chương 5. Bộ chứa IoC (*IoC container*) gắn kết các thành phần của Spring và các thành phần trong ứng dụng của bạn lại với nhau. Thông qua việc sử dụng IoC container — thứ mà chúng ta thường gọi là **ngữ cảnh Spring (Spring context)** — bạn làm cho các đối tượng nhất định được Spring nhận biết, từ đó cho phép framework sử dụng chúng theo đúng cách mà bạn đã cấu hình.

Trong Chương 6, chúng ta sẽ tiếp tục thảo luận về **Lập trình hướng khía cạnh trong Spring (Spring AOP)**. Spring có khả năng kiểm soát các thể hiện được đưa vào bộ chứa IoC của nó, và một trong những điều nó có thể làm là đánh chặn các phương thức đại diện cho hành vi của những thể hiện này. Năng lực này được gọi là "áp khía cạnh cho phương thức" (*aspecting the method*). Spring AOP là một trong những cách phổ biến nhất mà framework tương tác với những gì ứng dụng của bạn làm. Đặc tính này biến Spring AOP trở thành một phần kiến thức nền tảng bắt buộc phải nắm vững. Ngoài ra, thuộc Spring Core, chúng ta còn tìm thấy khả năng quản lý tài nguyên (*resource management*), quốc tế hóa (*internationalization - i18n*), chuyển đổi kiểu dữ liệu (*type conversion*), và SpEL. Chúng ta sẽ bắt gặp các khía cạnh của những tính năng này trong các ví dụ xuyên suốt cuốn sách.

1.2.2 Sử dụng tính năng Spring Data Access để triển khai tầng lưu trữ dữ liệu của ứng dụng

Đối với đại đa số các ứng dụng, việc lưu trữ bền vững (*persist*) một phần dữ liệu mà chúng xử lý là vô cùng trọng yếu. Làm việc với cơ sở dữ liệu là một chủ đề nền tảng, và trong Spring, module Data Access chính là công cụ bạn sẽ dùng để đảm trách việc lưu trữ dữ liệu trong rất nhiều trường hợp. Spring Data Access bao gồm việc sử dụng JDBC, tích hợp với các khung ánh xạ đối tượng - quan hệ (*Object-Relational Mapping - ORM*) như Hibernate (đừng lo lắng nếu bạn chưa biết ORM framework là gì hoặc chưa từng nghe tới Hibernate; chúng ta sẽ đề cập tới những khía cạnh này ở phần sau của sách), và quản lý giao dịch (*transactions*). Trong các Chương 12 đến 14, chúng ta sẽ bao quát mọi kiến thức cần thiết để giúp bạn bắt đầu với Spring Data Access.

1.2.3 Năng lực của Spring MVC trong phát triển ứng dụng web

Những ứng dụng phổ biến nhất được phát triển bằng Spring là các ứng dụng web, và bên trong hệ sinh thái Spring, bạn sẽ tìm thấy một tập hợp đồ sộ các công cụ cho phép bạn xây dựng ứng dụng web và các dịch vụ web theo nhiều phong cách khác nhau. Bạn có thể sử dụng Spring MVC để phát triển các ứng dụng theo phong cách servlet tiêu chuẩn, vốn đang được áp dụng trong một số lượng khổng lồ các ứng dụng hiện nay. Trong Chương 7, chúng ta sẽ đi sâu vào chi tiết cách sử dụng Spring MVC.

1.2.4 Tính năng kiểm thử của Spring

Module kiểm thử của Spring cung cấp cho chúng ta một bộ công cụ phong phú mà chúng ta sẽ dùng để viết các kiểm thử đơn vị (*unit tests*) và kiểm thử tích hợp (*integration tests*). Đã có vô số trang sách được viết về đề tài kiểm thử, nhưng chúng ta sẽ thảo luận tất cả những gì cốt lõi nhất để giúp bạn làm quen với Spring testing trong Chương 15. Tôi cũng sẽ gợi ý cho bạn một số tài liệu giá trị cần đọc để nắm bắt trọn vẹn chi tiết của chủ đề này. Quy tắc bất di bất dịch của tôi là: bạn chưa thể là một lập trình viên trưởng thành nếu bạn không thấu hiểu về kiểm thử, vì vậy đây là chủ đề bạn đặc biệt phải quan tâm.

1.2.5 Các dự án trong hệ sinh thái Spring

Hệ sinh thái Spring rộng lớn hơn rất nhiều so với những năng lực đã được thảo luận ở phần trên. Nó bao gồm một bộ sưu tập khổng lồ các framework khác có khả năng tích hợp trơn tru với nhau để tạo nên một vũ trụ phần mềm rộng lớn. Ở đây, chúng ta có các dự án tiêu biểu như Spring Data, Spring Security, Spring Cloud, Spring Batch, Spring Boot, v.v. Khi phát triển một ứng dụng, bạn có thể kết hợp nhiều dự án này lại với nhau. Ví dụ, bạn có thể xây dựng một ứng dụng tận dụng đồng thời cả Spring Boot, Spring Security và Spring Data. Trong các chương tiếp theo, chúng ta sẽ thực hành trên các dự án nhỏ có sử dụng các thành phần khác nhau của hệ sinh thái Spring.

Khi tôi nói "dự án" (*project*), tôi muốn ám chỉ một phân nhánh trong hệ sinh thái Spring được phát triển độc lập. Mỗi dự án này đều có một đội ngũ riêng chuyên trách việc mở rộng năng lực của nó. Ngoài ra, mỗi dự án đều được mô tả tách biệt và có tài liệu tham khảo chính thức riêng trên trang web của Spring: `https://spring.io/projects`.

Bên cạnh vũ trụ rộng lớn được tạo ra bởi Spring, chúng ta cũng sẽ đặc biệt đề cập tới hai dự án là Spring Data và Spring Boot. Đây là những dự án xuất hiện với tần suất rất cao trong các ứng dụng thực tế, vì vậy việc làm quen với chúng ngay từ đầu là hết sức quan trọng.

Mở rộng năng lực lưu trữ dữ liệu với Spring Data Dự án Spring Data hiện thực hóa một phần của hệ sinh thái Spring nhằm giúp bạn kết nối dễ dàng tới các cơ sở dữ liệu và khai thác tầng lưu trữ dữ liệu với số lượng dòng mã viết ra là ít nhất. Dự án này hỗ trợ cả công nghệ SQL lẫn NoSQL và tạo ra một tầng trừu tượng cấp cao, giúp đơn giản hóa đáng kể cách thức bạn thao tác với việc lưu trữ dữ liệu bền vững.

Phân biệt Spring Data Access và Spring Data

Chúng ta có Spring Data Access là một module thuộc Spring Core, và chúng ta cũng có một dự án độc lập trong hệ sinh thái Spring mang tên Spring Data. Spring Data Access chứa các triển khai truy cập dữ liệu nền tảng như cơ chế giao dịch và các công cụ JDBC. Trong khi đó, Spring Data nâng tầm khả năng truy cập cơ sở dữ liệu và cung cấp bộ công cụ toàn diện hơn, giúp việc phát triển trở nên dễ tiếp cận hơn và cho phép ứng dụng kết nối tới nhiều loại nguồn dữ liệu khác nhau. Chúng ta sẽ thảo luận chủ đề này trong Chương 14.

Spring Boot Spring Boot là một dự án thuộc hệ sinh thái Spring nhằm giới thiệu triết lý "quy ước hơn cấu hình" (*convention over configuration*). Tư tưởng chủ đạo của khái niệm này là: thay vì bạn phải tự tay thiết lập toàn bộ cấu hình cho một framework, Spring Boot cung cấp sẵn cho bạn một cấu hình mặc định hợp lý mà bạn có thể tùy biến thêm khi có nhu cầu. Kết quả chung là bạn viết ít mã hơn vì bạn tuân theo các quy ước phổ biến, và ứng dụng của bạn thường chỉ khác biệt so với các ứng dụng khác ở một vài điểm nhỏ. Do đó, thay vì phải cặm cụi viết đi viết lại cấu hình cho từng ứng dụng một, việc bắt đầu bằng cấu hình mặc định và chỉ thay đổi những gì khác biệt so với quy ước sẽ mang lại hiệu quả vượt trội. Chúng ta sẽ thảo luận sâu hơn về Spring Boot từ Chương 7.

Hệ sinh thái Spring vô cùng bao la và chứa đựng rất nhiều dự án. Một số dự án bạn sẽ gặp thường xuyên hơn những dự án khác, và có những dự án bạn thậm chí có thể không bao giờ dùng tới nếu ứng dụng của bạn không có nhu cầu đặc thù. Trong cuốn sách này, chúng ta chỉ tập trung vào những dự án thiết yếu nhất để bạn khởi đầu vững chắc: Spring Core, Spring Data và Spring Boot. Bạn có thể tra cứu danh sách đầy đủ các dự án thuộc hệ sinh thái Spring trên trang chủ: `https://spring.io/projects/`.

Các giải pháp thay thế cho Spring Chúng ta thực sự không thể bàn về các giải pháp thay thế cho toàn bộ Spring, bởi vì ai đó có thể hiểu nhầm là thay thế cho cả hệ sinh thái rộng lớn của nó. Nhưng đối với nhiều thành phần và dự án đơn lẻ tạo nên hệ sinh thái Spring, bạn hoàn toàn có thể tìm thấy các lựa chọn khác từ các thư viện hoặc framework thương mại hay mã nguồn mở.

Ví dụ, hãy xem xét bộ chứa Spring IoC container. Nhiều năm trước, đặc tả Java EE là một giải pháp rất được giới lập trình viên ưa chuộng. Với một triết lý hơi khác biệt, Java EE (đã được chuyển thành mã nguồn mở vào năm 2017 và đổi tên thành Jakarta EE, `https://jakarta.ee/`) cung cấp các đặc tả như Context and Dependency Injection (CDI) hay Enterprise Java Beans (EJB). Bạn có thể dùng CDI hoặc EJB để quản lý một ngữ cảnh chứa các thể hiện đối tượng và triển khai các khía cạnh (được gọi là "interceptors" theo thuật ngữ của Java EE). Ngoài ra, trong lịch sử, Google Guice (`https://github.com/google/guice`) cũng từng là

một framework được đánh giá cao trong việc quản lý các thể hiện đối tượng bên trong một bộ chứa container.

Đối với một số dự án xét riêng lẻ, bạn có thể tìm thấy một hoặc nhiều phương án thay thế. Ví dụ, bạn có thể chọn dùng Apache Shiro (`https://shiro.apache.org/`) thay vì Spring Security. Hoặc bạn có thể quyết định xây dựng ứng dụng web của mình bằng Play framework (`https://www.playframework.com/`) thay cho Spring MVC và các công nghệ liên quan của Spring.

Một dự án gần đây đầy hứa hẹn là Red Hat Quarkus. Quarkus được thiết kế chuyên biệt cho các triển khai hướng đám mây nguyên bản (*cloud-native*) và đang ngày càng trưởng thành với những bước tiến nhanh chóng. Tôi sẽ không ngạc nhiên nếu thấy nó trở thành một trong những dự án hàng đầu trong việc phát triển ứng dụng doanh nghiệp thuộc hệ sinh thái Java trong tương lai (`https://quarkus.io/`).

Lời khuyên của tôi dành cho bạn là hãy luôn cân nhắc tới các phương án thay thế. Trong phát triển phần mềm, bạn cần phải cởi mở và không bao giờ được tuyệt đối hóa một giải pháp duy nhất là "chân ái". Bạn sẽ luôn tìm thấy những kịch bản mà một công nghệ cụ thể này hoạt động tốt hơn hẳn một công nghệ khác.

1.3 Spring trong các kịch bản thực tế

Giờ đây khi bạn đã có cái nhìn tổng quan về Spring, bạn đã nhận thức được khi nào và tại sao chúng ta nên sử dụng một framework. Trong mục này, tôi sẽ đưa ra một số kịch bản ứng dụng mà việc sử dụng khung phát triển Spring có thể là một sự kết hợp hoàn hảo. Đã quá nhiều lần tôi chứng kiến các lập trình viên chỉ liên hệ tới các ứng dụng backend khi nói về việc sử dụng một framework như Spring. Thậm chí tôi còn thấy xu hướng thu hẹp kịch bản hơn nữa, chỉ giới hạn trong các ứng dụng web backend. Mặc dù đúng là trong phần lớn trường hợp chúng ta thấy Spring được dùng theo cách này, nhưng điều quan trọng cần nhớ là framework không hề bị giới hạn trong kịch bản đó. Tôi đã từng chứng kiến nhiều đội ngũ áp dụng thành công Spring vào các thể loại ứng dụng khác nhau, chẳng hạn như phát triển một ứng dụng kiểm thử tự động hay thậm chí trong các ứng dụng máy tính độc lập (*desktop applications*).

Tiếp theo, tôi sẽ mô tả chi tiết cho bạn một số kịch bản thực tế phổ biến mà tôi từng chứng kiến Spring được ứng dụng thành công rực rỡ. Đây không phải là những kịch bản khả dĩ duy nhất, và Spring cũng không phải lúc nào cũng phù hợp trong mọi tình huống của các trường hợp này. Hãy nhớ lại những gì chúng ta đã thảo luận trong mục 1.2: một framework không phải lúc nào cũng là sự lựa chọn đúng đắn. Nhưng đây là những trường hợp điển hình mà nhìn chung Spring mang lại sự tương thích tuyệt vời:

1. Phát triển một ứng dụng backend.
2. Phát triển một khung kiểm thử tự động (*automation testing framework*).

3. Phát triển một ứng dụng chạy trên máy tính cá nhân (*desktop app*).
4. Phát triển một ứng dụng di động (*mobile app*).

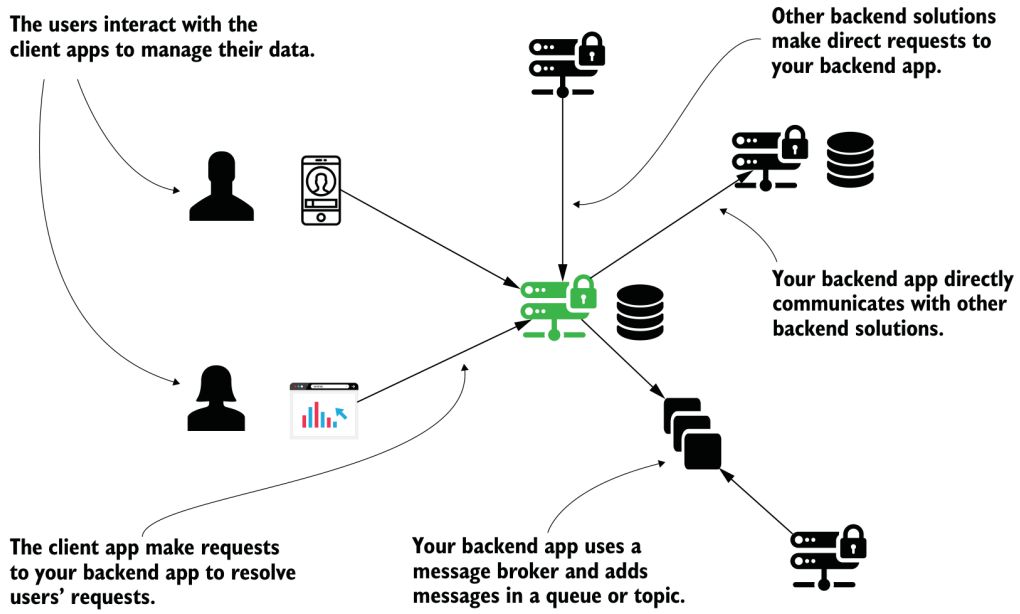
1.3.1 Sử dụng Spring trong phát triển ứng dụng backend

Ứng dụng backend là phần của một hệ thống thực thi ở phía máy chủ (*server side*), nắm giữ trách nhiệm quản lý dữ liệu và phục vụ các yêu cầu từ các ứng dụng phía máy khách (*client applications*). Người dùng tiếp cận các chức năng bằng cách thao tác trực tiếp trên các ứng dụng client. Tiếp đó, các ứng dụng client sẽ gửi yêu cầu tới ứng dụng backend để làm việc với dữ liệu của người dùng. Ứng dụng backend có thể sử dụng các cơ sở dữ liệu để lưu trữ dữ liệu hoặc giao tiếp với các ứng dụng backend khác theo nhiều phong cách khác nhau.

Bạn có thể hình dung, trong một kịch bản thực tế, ứng dụng đó chính là hệ thống backend quản lý các giao dịch trong tài khoản ngân hàng của bạn. Người dùng có thể truy cập tài khoản và quản lý chúng thông qua một ứng dụng web (ngân hàng trực tuyến) hoặc ứng dụng di động. Cả ứng dụng di động lẫn ứng dụng web đều đóng vai trò là client đối với ứng dụng backend. Để quản lý các giao dịch của người dùng, ứng dụng backend cần liên lạc với các giải pháp backend khác, và một phần dữ liệu mà nó xử lý cần được lưu trữ bền vững vào cơ sở dữ liệu. Trong Hình 1.5, bạn có thể hình dung kiến trúc tổng thể của một hệ thống như vậy.

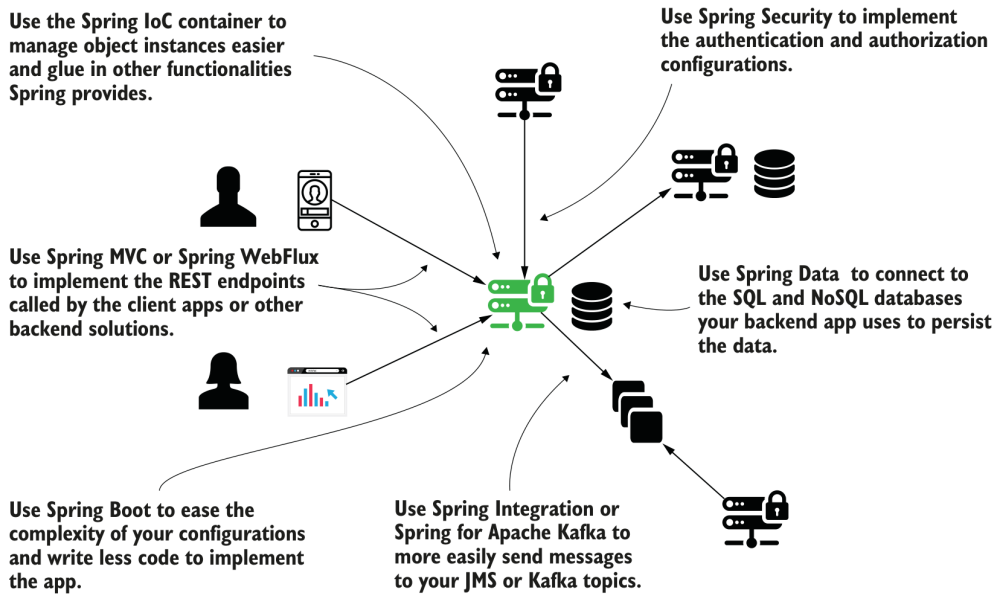
Ghi chú về tính phức tạp của hệ thống

Đừng lo lắng nếu bạn chưa nắm bắt được hết tất cả các chi tiết trong Hình 1.5. Tôi chưa kỳ vọng bạn phải biết bộ điều phối thông điệp (*message broker*) là gì và thậm chí cả cách thiết lập trao đổi dữ liệu giữa các thành phần. Điều tôi muốn bạn nhận ra là một hệ thống như vậy trong thực tế có thể trở nên rất phức tạp, và sau đó hiểu rằng các dự án trong hệ sinh thái Spring được xây dựng chính là để giúp bạn triệt tiêu độ phức tạp đó càng nhiều càng tốt.



Hình 1.5: Ứng dụng backend tương tác theo nhiều cách thức khác nhau với các ứng dụng khác và sử dụng cơ sở dữ liệu để quản lý dữ liệu. Thông thường, một ứng dụng backend có độ phức tạp cao và có thể đòi hỏi kết hợp nhiều công nghệ khác nhau. Các framework giúp đơn giản hóa quá trình triển khai bằng cách cung cấp các công cụ giúp bạn xây dựng giải pháp backend nhanh hơn.

Spring cung cấp một bộ công cụ xuất sắc cho việc xây dựng các ứng dụng backend. Nó làm cho cuộc sống của bạn trở nên dễ dàng hơn nhiều với các chức năng đa dạng thường thấy trong một giải pháp backend, từ việc tích hợp với các ứng dụng khác cho đến lưu trữ dữ liệu trên nhiều công nghệ cơ sở dữ liệu khác nhau. Không có gì ngạc nhiên khi các nhà phát triển thường xuyên lựa chọn Spring cho các ứng dụng loại này. Framework về cơ bản cung cấp cho bạn mọi thứ bạn cần trong các triển khai như vậy và là một mảnh ghép hoàn hảo cho bất kỳ phong cách kiến trúc nào. Hình 1.6 minh họa những khả năng ứng dụng Spring cho một ứng dụng backend.



Hình 1.6: Khả năng ứng dụng Spring trong một ứng dụng backend là vô tận, từ việc công khai các chức năng để các ứng dụng khác gọi tới cho đến quản lý truy cập cơ sở dữ liệu, và từ bảo mật ứng dụng cho tới quản lý tích hợp qua các hệ thống điều phối thông điệp của bên thứ ba.

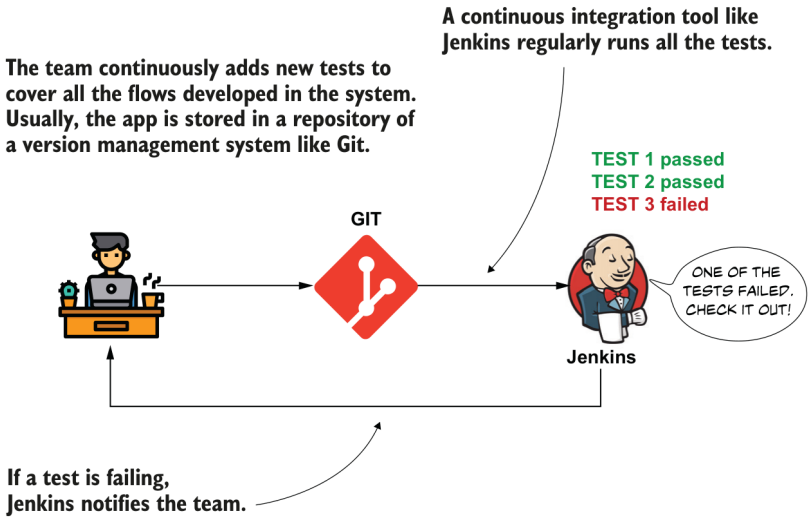
1.3.2 Sử dụng Spring trong ứng dụng kiểm thử tự động

Ngày nay, chúng ta thường xuyên sử dụng kiểm thử tự động (*automation testing*) để kiểm thử đầu-cuối (*end-to-end*) cho các hệ thống mà chúng ta phát triển. Kiểm thử tự động đề cập đến việc xây dựng phần mềm mà đội ngũ phát triển sử dụng để đảm bảo ứng dụng vận hành đúng như kỳ vọng. Nhóm phát triển có thể lên lịch cho ứng dụng kiểm thử tự động chạy định kỳ để kiểm tra ứng dụng và cảnh báo cho các lập trình viên nếu có điều gì bất thường. Việc có một chức năng như vậy mang lại cho lập trình viên sự tự tin vững chắc, bởi vì họ biết mình sẽ được thông báo ngay lập tức nếu vô tình làm hỏng bất kỳ chức năng hiện có nào của ứng dụng trong quá trình phát triển tính năng mới.

Mặc dù với các hệ thống nhỏ bạn có thể thực hiện kiểm thử thủ công, nhưng việc tự động hóa các trường hợp kiểm thử luôn là một ý tưởng sáng suốt. Đối với các hệ thống phức tạp hơn, kiểm thử thủ công toàn bộ các luồng thậm chí không còn là một lựa chọn khả thi. Bởi vì số lượng luồng nghiệp vụ là quá nhiều, nó sẽ đòi hỏi một lượng giờ công khổng lồ cùng quá nhiều sức lực để có thể bao quát trọn vẹn.

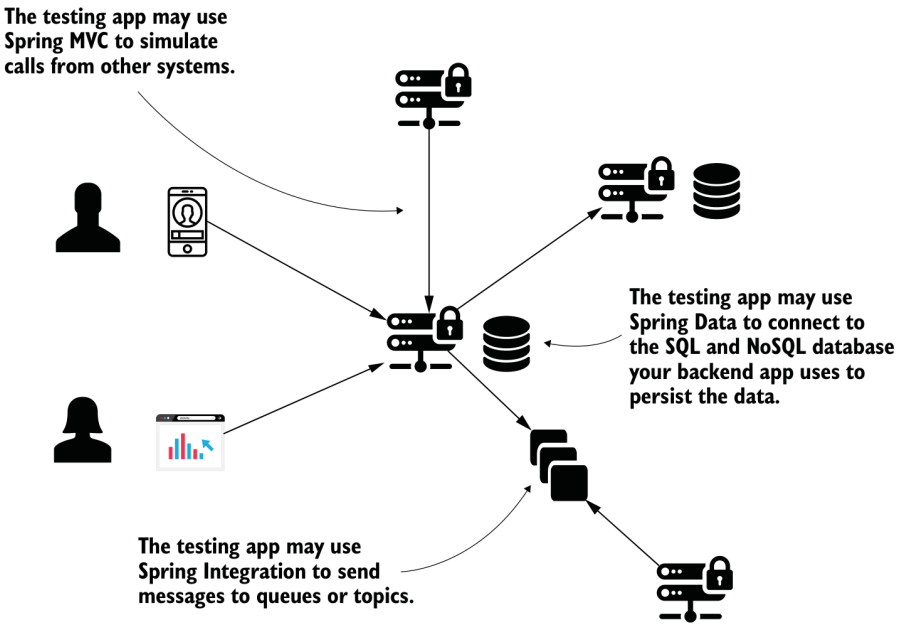
Thực tế cho thấy giải pháp hiệu quả nhất là để một nhóm riêng biệt triển khai một ứng dụng chuyên trách việc xác thực toàn bộ các luồng của hệ thống được kiểm thử. Khi các lập trình viên bổ sung thêm chức năng mới vào hệ thống, ứng dụng kiểm thử này cũng được nâng cấp tương ứng để bao quát những điểm mới, và các đội ngũ sử dụng nó để xác nhận rằng mọi thứ vẫn vận hành đúng như ý muốn. Các nhà phát triển sau cùng sẽ sử dụng một công cụ tích

hợp liên tục (*Continuous Integration - CI*) và lên lịch cho ứng dụng chạy đều đặn nhằm nhận phản hồi sớm nhất có thể cho những thay đổi của họ (Hình 1.7).



Hình 1.7: Đội ngũ phát triển triển khai ứng dụng kiểm thử trong một môi trường kiểm thử riêng. Một công cụ tích hợp liên tục như Jenkins sẽ thực thi ứng dụng định kỳ và gửi phản hồi về cho nhóm. Bằng cách này, nhóm luôn nắm bắt được trạng thái của hệ thống và biết ngay nếu họ làm hỏng điều gì đó trong quá trình phát triển.

Một ứng dụng như vậy có thể trở nên phức tạp không kém gì một ứng dụng backend thực thụ. Để xác thực các luồng, ứng dụng kiểm thử cần phải giao tiếp với các thành phần của hệ thống và thậm chí kết nối trực tiếp vào cơ sở dữ liệu. Đôi khi ứng dụng phải giả lập (*mock*) các phụ thuộc bên ngoài để mô phỏng các kịch bản thực thi khác nhau. Để viết các kịch bản kiểm thử, các lập trình viên thường sử dụng các framework như Selenium, Cucumber, Gauge và nhiều công cụ khác. Tuy nhiên, song hành cùng các framework này, ứng dụng kiểm thử vẫn có thể hưởng lợi theo nhiều cách từ các công cụ của Spring. Ví dụ, ứng dụng có thể quản lý các thể hiện đối tượng để mã nguồn dễ bảo trì hơn thông qua bộ chứa Spring IoC. Nó có thể dùng Spring Data để kết nối tới các cơ sở dữ liệu nơi nó cần xác minh tính chính xác của dữ liệu. Nó có thể bắn thông điệp vào các hàng đợi (*queues*) hoặc chủ đề (*topics*) của một hệ thống broker để mô phỏng các tình huống cụ thể, hoặc đơn giản là dùng Spring để gọi tới một số điểm cuối REST (Hình 1.8). (Hãy nhớ rằng, nếu những điều này có vẻ quá nâng cao lúc này thì cũng hoàn toàn bình thường; ý nghĩa của chúng sẽ được làm sáng tỏ dần khi bạn tiến bước qua từng chương sách).



Hình 1.8: Ứng dụng kiểm thử có thể cần kết nối tới các cơ sở dữ liệu hoặc giao tiếp với các hệ thống khác hay chính hệ thống đang được kiểm thử. Các lập trình viên có thể tận dụng các thành phần của hệ sinh thái Spring để đơn giản hóa việc hiện thực hóa các chức năng này.

1.3.3 Sử dụng Spring cho phát triển ứng dụng máy tính để bàn (Desktop app)

Ngày nay, các ứng dụng máy tính để bàn không còn được phát triển thường xuyên như trước, khi mà các ứng dụng web và di động đã đảm nhận phần lớn vai trò tương tác trực tiếp với người dùng. Tuy nhiên, vẫn còn một số lượng nhất định các ứng dụng desktop, và các thành phần trong hệ sinh thái Spring hoàn toàn có thể là một lựa chọn tuyệt vời trong việc phát triển các tính năng của chúng. Một ứng dụng desktop có thể tận dụng thành công bộ chứa Spring IoC để quản lý các thể hiện đối tượng. Nhờ đó, việc triển khai ứng dụng sẽ trở nên gọn gàng hơn và cải thiện đáng kể khả năng bảo trì. Ngoài ra, ứng dụng có tiềm năng sử dụng các công cụ của Spring để hiện thực hóa những chức năng khác nhau, chẳng hạn như giao tiếp với hệ thống backend hoặc các thành phần khác (thông qua việc gọi web service hoặc các kỹ thuật gọi từ xa) hay xây dựng một giải pháp lưu đệm (*caching*).

1.3.4 Sử dụng Spring trong ứng dụng di động (Mobile apps)

Với dự án Spring for Android (`https://spring.io/projects/spring-android`), cộng đồng Spring đã nỗ lực hỗ trợ việc phát triển các ứng dụng di động. Mặc dù bạn có thể sẽ hiếm khi bắt gặp tình huống này trong thực tế, nhưng cũng rất đáng để nhắc tới rằng bạn có thể sử dụng các công cụ của Spring để phát triển ứng dụng Android. Dự án Spring này cung cấp

một client REST cho Android cùng cơ chế hỗ trợ xác thực để truy cập vào các API được bảo mật.

1.4 Khi nào không nên dùng framework

Trong mục này, chúng ta sẽ thảo luận về lý do tại sao đôi khi bạn nên tránh sử dụng framework. Việc nắm rõ khi nào nên dùng và khi nào nên tránh dùng framework là điều tối quan trọng. Đôi khi, việc sử dụng một công cụ quá cồng kềnh cho một công việc đơn giản có thể tiêu tốn nhiều năng lượng hơn mà lại thu về kết quả tệ hơn. Hãy tưởng tượng bạn dùng một chiếc cưa máy chỉ để cắt một ổ bánh mì. Mặc dù bạn hoàn toàn có thể thử và thậm chí cũng đạt được kết quả cuối cùng, nhưng việc đó sẽ khó khăn hơn rất nhiều, tốn công sức hơn nhiều so với việc dùng một con dao bình thường (và bạn có thể kết thúc với một đống vụn bánh mì thay vì những lát bánh mì cắt gọn ghẽ). Chúng ta sẽ thảo luận một vài kịch bản mà việc sử dụng một framework không phải là một ý tưởng hay, và sau đó tôi sẽ kể cho bạn nghe câu chuyện về một đội ngũ mà tôi từng tham gia đã thất bại trong việc triển khai ứng dụng chỉ vì lạm dụng framework.

Hóa ra, cũng giống như mọi thứ khác trong ngành phát triển phần mềm, bạn không nên áp dụng framework cho mọi trường hợp. Bạn sẽ gặp những tình huống mà framework không phải là giải pháp phù hợp — hoặc có thể framework là giải pháp tốt, nhưng không phải là Spring framework. Trong những kịch bản nào dưới đây bạn nên cân nhắc không sử dụng framework?

1. Bạn cần triển khai một chức năng cụ thể với mức chiếm dụng tài nguyên (*footprint*) càng nhỏ càng tốt. Bằng từ "footprint", tôi muốn nói tới dung lượng bộ nhớ lưu trữ mà các tệp của ứng dụng chiếm dụng.
2. Các yêu cầu bảo mật đặc thù buộc bạn chỉ được phép triển khai mã nguồn tùy biến riêng trong ứng dụng mà không được sử dụng bất kỳ framework mã nguồn mở nào.
3. Bạn sẽ phải tùy biến lại các thành phần của framework nhiều đến mức cuối cùng bạn phải viết nhiều mã nguồn hơn so với việc bạn hoàn toàn không dùng framework.
4. Bạn đã có sẵn một ứng dụng đang hoạt động tốt, và việc thay đổi nó để sử dụng một framework không mang lại bất kỳ lợi ích thiết thực nào.

Hãy cùng phân tích chi tiết từng điểm nêu trên.

1.4.1 Bạn cần mức chiếm dụng tài nguyên nhỏ (Small footprint)

Đối với điểm thứ nhất, tôi muốn đề cập đến những tình huống mà bạn cần giữ cho ứng dụng của mình thật nhỏ gọn. Trong các hệ thống ngày nay, chúng ta thấy ngày càng nhiều trường hợp các dịch vụ được phân phối dưới dạng các bộ chứa (*containers*). Bạn hẳn đã nghe nói

về container, chẳng hạn như Docker, Kubernetes hay các thuật ngữ liên quan đến chủ đề này (nếu chưa nghe tới thì cũng không sao cả).

Chủ đề trọn vẹn về container nằm ngoài phạm vi của cuốn sách này, do đó hiện tại điều duy nhất tôi cần bạn biết là: khi bạn sử dụng phong cách triển khai như vậy, bạn luôn mong muốn ứng dụng của mình càng nhỏ càng tốt. Một container giống như một chiếc hộp chứa đựng ứng dụng bên trong. Một nguyên tắc tối quan trọng liên quan đến việc triển khai ứng dụng trong container là các container cần phải dễ dàng thay thế và loại bỏ: chúng có thể bị hủy và tái tạo lại càng nhanh càng tốt. Kích thước của ứng dụng (*footprint*) đóng vai trò vô cùng quyết định ở đây. Bạn có thể tiết kiệm được hàng giây quý giá trong quá trình khởi tạo ứng dụng bằng cách làm cho nó nhỏ hơn. Điều đó không có nghĩa là bạn sẽ không dùng framework cho toàn bộ các ứng dụng được triển khai trong container.

Tuy nhiên, đối với một số ứng dụng vốn dĩ đã khá nhỏ, việc tối ưu hóa thời gian khởi tạo và thu nhỏ dung lượng chiếm dụng của chúng sẽ hợp lý hơn nhiều so với việc nhồi nhét thêm các phụ thuộc vào các framework khác nhau. Một ví dụ điển hình là loại ứng dụng được gọi là **hàm không máy chủ** (*serverless function*). Các serverless function này là những ứng dụng siêu nhỏ được đóng gói triển khai trong container. Bởi vì bạn không có nhiều quyền can thiệp vào cách thức chúng được triển khai trên hạ tầng, chúng tạo cảm giác như đang thực thi mà không cần máy chủ (do đó mới có tên gọi như vậy). Những ứng dụng này bắt buộc phải nhỏ gọn, và đó là lý do tại sao đối với trường hợp cụ thể này, bạn sẽ muốn hạn chế thêm vào một framework càng nhiều càng tốt. Do kích thước vốn đã rất khiêm tốn của nó, nhiều khả năng bạn cũng sẽ chẳng cần tới một framework để làm gì.

1.4.2 Yêu cầu bảo mật buộc phải dùng mã nguồn tự viết (Custom code)

Tôi đã nêu ở điểm thứ hai rằng trong những tình huống đặc thù, các ứng dụng không thể sử dụng framework do những đòi hỏi khắt khe về bảo mật. Kịch bản này thường xảy ra với các ứng dụng trong lĩnh vực quốc phòng hoặc các tổ chức chính phủ. Xin nhắc lại, điều này không có nghĩa là tất cả các ứng dụng trong cơ quan nhà nước đều bị cấm dùng framework, nhưng đối với một số ứng dụng nhất định, các giới hạn nghiêm ngặt sẽ được áp dụng. Bạn có thể tự hỏi tại sao lại như vậy? Hãy giả định một framework mã nguồn mở như Spring được sử dụng. Nếu ai đó phát hiện ra một lỗ hổng bảo mật cụ thể, thông tin đó sẽ trở nên công khai, và một hacker có thể lợi dụng sự hiểu biết này để khai thác tấn công. Đôi khi, các bên liên quan của những ứng dụng như vậy muốn đảm bảo rằng xác suất ai đó xâm nhập được vào hệ thống của họ phải tiệm cận mức số 0 tuyệt đối. Điều này có thể dẫn tới quyết định tự xây dựng lại toàn bộ chức năng từ đầu thay vì sử dụng từ một nguồn thứ ba.

Nghịch lý về an toàn mã nguồn mở

Khoan đã! Ở phần trước tôi vừa nói rằng việc sử dụng một framework mã nguồn mở sẽ an toàn hơn vì nếu có lỗ hổng, ai đó sẽ nhanh chóng phát hiện ra nó. Đúng vậy, nếu bạn đầu tư đủ thời gian và tiền bạc, bạn có lẽ cũng có thể tự mình đạt được mức độ an toàn đó. Nhìn chung, việc sử dụng framework rõ ràng là rẻ hơn rất nhiều. Và nếu bạn không cần phải quá mức cẩn trọng phi thường, việc sử dụng framework là hoàn toàn hợp lý. Nhưng trong một số dự án đặc biệt, các bên hữu quan thực sự muốn đảm bảo tuyệt đối rằng không có bất kỳ thông tin nào về hệ thống bị lộ ra ngoài công chúng.

1.4.3 Tùy biến quá nhiều khiến việc dùng framework trở nên bất khả thi

Một trường hợp khác (điểm thứ ba) mà bạn có thể muốn tránh sử dụng framework là khi bạn phải tùy biến các thành phần của nó nhiều đến mức bạn kết thúc bằng việc viết nhiều mã nguồn hơn so với khi không dùng nó. Như tôi đã chỉ ra trong mục 1.1, một framework cung cấp cho bạn những bộ phận để bạn lắp ghép cùng với mã nghiệp vụ của mình nhằm tạo nên một ứng dụng. Những thành phần do framework cung cấp này không phải lúc nào cũng khớp hoàn hảo với bài toán của bạn, và bạn cần phải tinh chỉnh chúng theo các cách khác nhau. Việc tùy biến các thành phần của framework và cách thức lắp ráp chúng là hoàn toàn bình thường, nhưng nếu số lượng tùy biến đó lại đòi hỏi nhiều công sức hơn cả việc tự xây dựng chức năng từ đầu, thì bạn đang gặp vấn đề. Nếu rơi vào hoàn cảnh như vậy, nhiều khả năng bạn đã chọn nhầm framework (hãy tìm kiếm các giải pháp thay thế) hoặc bạn hoàn toàn không nên sử dụng framework cho trường hợp đó.

1.4.4 Bạn không thu được lợi ích gì từ việc chuyển đổi sang framework

Ở điểm thứ tư, tôi đã đề cập rằng một sai lầm tiềm ẩn là cố gắng dùng một framework để thay thế một thứ đã tồn tại và đang vận hành ổn định trong ứng dụng. Đôi khi chúng ta bị cám dỗ muốn thay thế một kiến trúc hiện có bằng một thứ gì đó mới mẻ. Một framework mới xuất hiện, nó đang rất thịnh hành, mọi người đều đổ xô đi dùng nó, vậy tại sao chúng ta lại không thay đổi ứng dụng của mình để sử dụng framework này? Bạn hoàn toàn có thể làm điều đó, nhưng bạn cần phải phân tích thật cẩn trọng xem mình muốn đạt được điều gì bằng cách thay đổi một thứ vốn dĩ đang hoạt động tốt. Trong một số trường hợp, giống như câu chuyện của tôi trong mục 1.1, việc thay đổi ứng dụng để dựa vào một framework cụ thể có thể mang lại lợi ích to lớn. Miễn là sự thay đổi đó đem lại giá trị thiết thực, hãy cứ làm! Một lý do xác đáng có thể là bạn muốn làm cho ứng dụng dễ bảo trì hơn, đạt hiệu năng cao hơn, hoặc bảo mật tốt hơn. Nhưng nếu sự thay đổi đó không đem lại bất kỳ lợi ích nào, và đôi khi thậm chí

còn mang lại sự bấp bênh, thì rốt cuộc bạn có thể sẽ nhận ra mình đã ném tiền bạc và thời gian qua cửa sổ chỉ để nhận về một kết quả tồi tệ hơn. Hãy để tôi kể cho bạn nghe một câu chuyện từ chính trải nghiệm xương máu của mình.

Một sai lầm có thể tránh được (An avoidable mistake)

Sử dụng framework không phải lúc nào cũng là sự lựa chọn sáng suốt nhất, và tôi đã phải học bài học đó qua một con đường đầy chông gai. Nhiều năm trước, chúng tôi đang phát triển phần backend cho một ứng dụng web. Thời thế tác động lên nhiều thứ, bao gồm cả kiến trúc phần mềm. Ứng dụng khi đó đang sử dụng JDBC thuần để kết nối trực tiếp tới cơ sở dữ liệu Oracle. Mã nguồn trông khá xấu xí. Bất cứ nơi nào ứng dụng cần thực thi một câu truy vấn trên cơ sở dữ liệu, nó lại mở một đối tượng statement rồi gửi một câu lệnh truy vấn đôi khi kéo dài qua nhiều dòng. Bạn có thể còn đủ trẻ để chưa từng bắt gặp việc sử dụng trực tiếp JDBC trong các ứng dụng, nhưng hãy tin tôi đi, đó là một đoạn mã dài dòng và luộm thuộm.

Vào thời điểm đó, một số framework áp dụng phương pháp luận khác để làm việc với cơ sở dữ liệu đang ngày càng trở nên thịnh hành. Tôi còn nhớ lần đầu tiên mình tiếp cận Hibernate. Đây là một ORM framework cho phép bạn đối xử với các bảng và mối quan hệ giữa chúng trong cơ sở dữ liệu như những đối tượng và mối quan hệ giữa các đối tượng trong Java. Khi được sử dụng đúng cách, nó cho phép bạn viết ít mã hơn và tạo ra các chức năng trực quan hơn. Nhưng khi bị lạm dụng sai cách, nó có thể làm chậm ứng dụng của bạn, khiến mã nguồn trở nên khó hiểu hơn, và thậm chí còn đưa thêm lỗi vào hệ thống.

Ứng dụng mà chúng tôi đang phát triển thực sự cần một sự cải tổ. Chúng tôi biết mình có thể cải thiện đoạn mã JDBC xấu xí đó. Trong suy nghĩ của tôi, ít nhất chúng tôi có thể giảm thiểu số lượng dòng mã. Sự thay đổi này sẽ mang lại lợi ích to lớn cho khả năng bảo trì. Cùng với một số lập trình viên khác, chúng tôi đề xuất sử dụng một công cụ do Spring cung cấp có tên là `JdbcTemplate` (bạn sẽ được học công cụ này trong Chương 12). Tuy nhiên, một số người khác lại thúc đẩy mạnh mẽ quyết định sử dụng Hibernate. Nó đang rất nổi tiếng, vậy tại sao lại không dùng? (Thực tế hiện nay nó vẫn là một trong những framework phổ biến nhất trong thể loại của mình, và bạn sẽ học cách tích hợp nó với Spring trong Chương 13). Tôi có thể nhìn thấy trước rằng việc chuyển đổi đoạn mã đó sang một phương pháp luận hoàn toàn mới sẽ là một thử thách gian nan. Hơn nữa, tôi không thấy được lợi ích rõ rệt nào. Sự thay đổi này cũng tiềm ẩn nguy cơ rất lớn làm phát sinh thêm lỗi.

May mắn thay, cuộc thay đổi được bắt đầu bằng một bản thử nghiệm chứng minh khái niệm (*Proof of Concept - PoC*). Sau một vài tháng trời, tiêu tốn biết bao công sức và sự căng thẳng tột độ, cả nhóm cuối cùng đã quyết định từ bỏ.

Sau khi phân tích kỹ lưỡng các phương án, chúng tôi đã hoàn tất việc triển khai bằng cách sử dụng `JdbcTemplate`. Chúng tôi đã viết được mã nguồn sạch hơn nhờ loại bỏ một số lượng khổng lồ các dòng mã thừa, mà lại không cần phải đưa thêm bất kỳ một framework mới toanh nào vào hệ thống cho sự thay đổi này.

1.5 Bạn sẽ học được gì trong cuốn sách này

Kể từ khi bạn mở cuốn sách này ra, tôi giả định bạn có thể là một nhà phát triển phần mềm trong hệ sinh thái Java — người đã nhận thấy rằng việc học Spring là vô cùng hữu ích. Mục tiêu của cuốn sách này là trang bị cho bạn những nền tảng vững chắc nhất về Spring, với giả định rằng bạn chưa biết bất cứ điều gì về các framework nói chung và tất nhiên là cả về Spring. Khi tôi nói Spring, tôi muốn ám chỉ toàn bộ hệ sinh thái Spring, chứ không chỉ riêng phần lõi của framework.

Khi bạn đọc xong cuốn sách này, bạn sẽ nắm vững cách thức thực hiện những điều sau:

- Sử dụng ngữ cảnh Spring (*Spring context*) và triển khai các khía cạnh (*aspects*) xung quanh các đối tượng do framework quản lý.
- Triển khai cơ chế giúp một ứng dụng Spring kết nối với cơ sở dữ liệu và làm việc với dữ liệu được lưu trữ bền vững.
- Thiết lập trao đổi dữ liệu giữa các ứng dụng bằng cách sử dụng các API REST được xây dựng với Spring.
- Xây dựng các ứng dụng cơ bản áp dụng phương pháp tiếp cận quy ước hơn cấu hình (*convention-over-configuration*).
- Vận dụng các thực hành tốt nhất (*best practices*) trong thiết kế lớp tiêu chuẩn của một ứng dụng Spring.
- Kiểm thử chuẩn chỉ các triển khai ứng dụng Spring của bạn.

Tóm tắt chương

- Một khung phát triển ứng dụng (*application framework*) là một tập hợp các chức năng phần mềm dùng chung nhằm cung cấp cấu trúc nền tảng cho việc phát triển một ứng dụng. Framework đóng vai trò như một bộ khung xương vững chắc để nâng đỡ và xây dựng nên ứng dụng.
- Framework giúp bạn xây dựng ứng dụng hiệu quả hơn bằng cách cung cấp sẵn các chức năng mà bạn chỉ việc lắp ráp vào bản triển khai của mình thay vì phải tự tay phát triển từ đầu. Việc sử dụng framework giúp bạn tiết kiệm thời gian và đảm bảo giảm thiểu nguy cơ mắc phải các lỗi phần mềm.
- Việc sử dụng một framework được biết đến rộng rãi như Spring mở ra cánh cửa bước vào một cộng đồng rộng lớn, nơi có khả năng cao là những người khác cũng đã từng đối mặt với những vấn đề tương tự như bạn. Nhờ đó, bạn có cơ hội tuyệt vời để học hỏi cách người khác giải quyết một vấn đề tương đồng, giúp bạn tiết kiệm thời gian tự mình mò mẫm nghiên cứu.
- Khi triển khai một ứng dụng, hãy luôn cân nhắc tới mọi khả năng, bao gồm cả phương

án không sử dụng framework. Nếu bạn quyết định dùng một hoặc nhiều framework, hãy xem xét tất cả các giải pháp thay thế cho chúng. Bạn nên suy ngẫm về mục đích của framework, những ai đang sử dụng nó (quy mô cộng đồng lớn đến đâu), và nó đã tồn tại trên thị trường được bao lâu (độ chín muồi).

- Spring không đơn thuần là một framework đơn lẻ. Chúng ta thường gọi là "khung phát triển Spring" để chỉ các chức năng cốt lõi, nhưng Spring cung cấp cả một hệ sinh thái trọn vẹn được tạo nên từ nhiều dự án phục vụ cho việc phát triển ứng dụng. Mỗi dự án được dành riêng cho một mảng chuyên biệt, và khi xây dựng ứng dụng, bạn có thể kết hợp nhiều dự án này để hiện thực hóa chức năng mong muốn. Các dự án trong hệ sinh thái Spring mà chúng ta sẽ sử dụng trong cuốn sách này bao gồm:
  - **Spring Core:** Tạo dựng nền tảng cho Spring và cung cấp các tính năng như ngữ cảnh (*context*), khía cạnh (*aspects*), và truy cập dữ liệu cơ bản.
  - **Spring Data:** Cung cấp một bộ công cụ cấp cao, tiện lợi để triển khai tầng lưu trữ dữ liệu cho ứng dụng. Bạn sẽ thấy việc sử dụng Spring Data để làm việc với cả cơ sở dữ liệu SQL lẫn NoSQL dễ dàng đến mức nào.
  - **Spring Boot:** Một dự án thuộc hệ sinh thái Spring giúp bạn áp dụng phương pháp tiếp cận "quy ước hơn cấu hình" (*convention-over-configuration*).
- Rất thường xuyên, các tài liệu học tập (như sách, bài viết hay video hướng dẫn) chỉ cung cấp các ví dụ dùng Spring cho các ứng dụng backend. Mặc dù đúng là việc dùng Spring với các ứng dụng backend là cực kỳ phổ biến, nhưng bạn hoàn toàn có thể sử dụng Spring với các thể loại ứng dụng khác, ngay cả trong các ứng dụng máy tính cá nhân và các ứng dụng kiểm thử tự động.

Chương 2

Ngữ cảnh Spring: Định nghĩa các bean

Trong chương này, bạn sẽ bắt đầu học cách làm việc với một phần tử trọng yếu bậc nhất của khung phát triển Spring: **ngữ cảnh** (*context*, hay còn gọi là *application context* trong một ứng dụng Spring). Hãy hình dung ngữ cảnh như một vùng không gian trong bộ nhớ ứng dụng của bạn, nơi chúng ta đưa vào toàn bộ các thể hiện đối tượng (*instances*) mà chúng ta muốn framework quản lý. Theo mặc định, Spring không hề biết tới bất kỳ đối tượng nào bạn định nghĩa trong ứng dụng. Để Spring có thể "nhìn thấy" các đối tượng của bạn, bạn cần phải thêm chúng vào ngữ cảnh.

Ở các phần sau của cuốn sách, chúng ta sẽ thảo luận về việc khai thác các năng lực đa dạng do Spring cung cấp trong ứng dụng. Bạn sẽ học được rằng việc gắn kết các tính năng đó được thực hiện thông qua ngữ cảnh bằng cách đưa các thể hiện đối tượng vào và thiết lập các mối quan hệ giữa chúng. Spring sử dụng các thể hiện trong ngữ cảnh để kết nối ứng dụng của bạn tới các chức năng phong phú mà nó cung cấp. Bạn sẽ nắm bắt được các kiến thức căn bản của những tính năng quan trọng nhất (chẳng hạn như giao dịch, kiểm thử, v.v.) xuyên suốt cuốn sách này.

Nội dung chính của chương này

- Thấu hiểu sự cần thiết của ngữ cảnh Spring (*Spring context*).
- Thêm các thể hiện đối tượng mới vào ngữ cảnh Spring.

Việc hiểu rõ ngữ cảnh Spring là gì và cách thức nó vận hành ra sao chính là bước đi đầu tiên trong hành trình làm chủ Spring, bởi vì nếu không biết cách quản lý ngữ cảnh Spring, hầu như mọi điều bạn học sau này đều sẽ không thể thực hiện được. Ngữ cảnh là một cơ chế phức hợp cho phép Spring kiểm soát các thể hiện do bạn định nghĩa, từ đó cho phép bạn khai thác các năng lực mà framework mang lại.

Chúng ta bắt đầu chương này bằng việc tìm hiểu cách thêm các thể hiện đối tượng vào ngữ cảnh Spring. Trong Chương 3, bạn sẽ học cách tham chiếu tới các thể hiện đã thêm và thiết

lập mối quan hệ liên kết giữa chúng.

Chúng ta sẽ gọi các thể hiện đối tượng này là các **"bean"**. Tất nhiên, đối với các cú pháp bạn cần học, chúng ta sẽ viết các đoạn mã nguồn minh họa, và bạn có thể tìm thấy toàn bộ các đoạn mã này trong những dự án mẫu đi kèm cuốn sách. Tôi sẽ làm phong phú thêm các ví dụ mã nguồn bằng hình ảnh trực quan cùng những giải thích cặn kẽ về từng phương pháp tiếp cận.

Vì tôi muốn quá trình bạn tiếp cận Spring diễn ra một cách tuần tự và từng bước vững chắc, trong chương này chúng ta sẽ tập trung vào các cú pháp cần thiết để thao tác với ngữ cảnh Spring. Sau này bạn sẽ nhận ra rằng không phải mọi đối tượng trong ứng dụng đều cần phải để Spring quản lý, do đó bạn không nhất thiết phải đưa toàn bộ các thể hiện đối tượng trong ứng dụng vào ngữ cảnh Spring. Tại thời điểm này, tôi mời bạn tập trung tối đa vào việc làm chủ các phương pháp đưa một thể hiện vào để Spring quản lý.

2.1 Tạo một dự án Maven

Trong mục này, chúng ta sẽ cùng thảo luận về việc tạo một dự án Maven. Maven không phải là một chủ đề liên quan trực tiếp đến Spring, nhưng nó là một công cụ giúp bạn dễ dàng quản lý toàn bộ quy trình xây dựng (*build process*) của ứng dụng, bất kể bạn sử dụng framework nào. Bạn cần nắm vững các kiến thức cơ bản về dự án Maven để có thể theo dõi và thực hành các ví dụ mã nguồn. Maven cũng là một trong những công cụ xây dựng dự án được sử dụng phổ biến nhất cho các dự án Spring trong thực tế (với Gradle là công cụ đứng thứ hai, nhưng chúng ta sẽ không bàn về nó trong cuốn sách này). Vì Maven là một công cụ rất đỗi quen thuộc, có thể bạn đã biết cách tạo dự án và khai báo các phụ thuộc thông qua cấu hình của nó. Nếu đã thành thạo, bạn hoàn toàn có thể bỏ qua mục này để đi thẳng tới mục 2.2.

Một công cụ xây dựng (*build tool*) là một phần mềm chúng ta sử dụng để xây dựng ứng dụng thuận tiện hơn. Bạn thiết lập cấu hình để công cụ xây dựng tự động thực thi các tác vụ cấu thành nên quá trình xây dựng ứng dụng thay vì phải thao tác thủ công. Dưới đây là một số ví dụ điển hình về các tác vụ thường có trong quá trình xây dựng một ứng dụng:

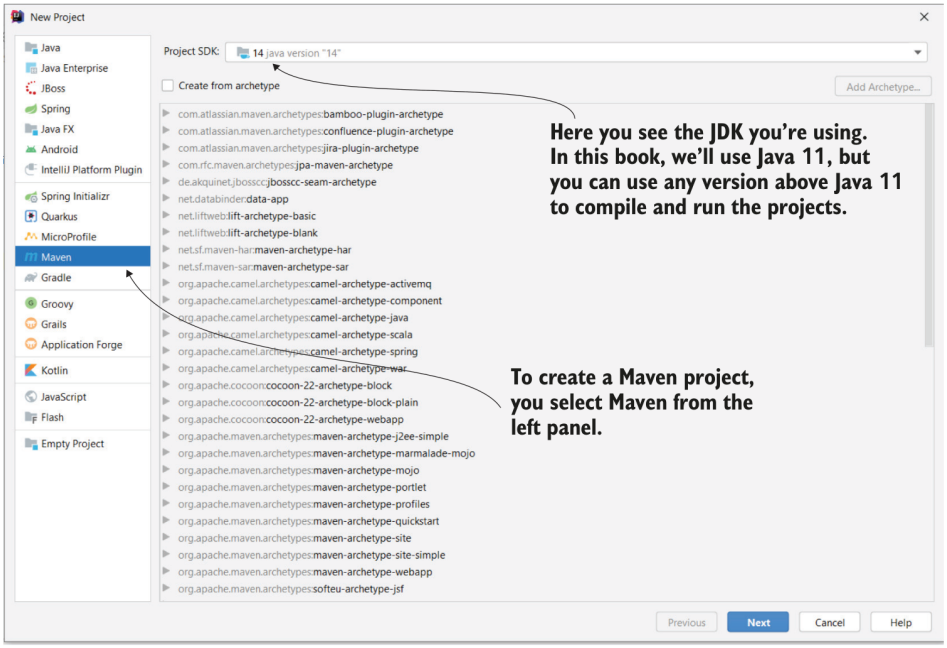
- Tải về các thư viện phụ thuộc (*dependencies*) mà ứng dụng của bạn cần.
- Thực thi các bài kiểm thử (*tests*).
- Xác thực mã nguồn tuân thủ các quy tắc định dạng đã đề ra.
- Quét và kiểm tra các lỗ hổng bảo mật.
- Biên dịch ứng dụng.
- Đóng gói ứng dụng thành một tệp lưu trữ thực thi được (chẳng hạn như tệp JAR).

Để các ví dụ của chúng ta có thể quản lý các thư viện phụ thuộc dễ dàng, chúng ta cần dùng một công cụ xây dựng cho các dự án sẽ phát triển. Mục này chỉ hướng dẫn đúng những gì

bạn cần biết để thực hiện các ví dụ trong sách; chúng ta sẽ đi từng bước qua quy trình tạo một dự án Maven, và tôi sẽ chỉ cho bạn những điểm cốt lõi nhất về cấu trúc của nó.

Hãy bắt đầu ngay từ những bước đầu tiên. Trước hết, cũng như khi phát triển bất kỳ ứng dụng nào khác, bạn cần có một môi trường phát triển tích hợp (*Integrated Development Environment - IDE*). Mọi IDE chuyên nghiệp ngày nay đều hỗ trợ hoàn hảo cho các dự án Maven, vì vậy bạn có thể tùy chọn công cụ mình yêu thích: IntelliJ IDEA, Eclipse, Spring STS, NetBeans, v.v. Trong cuốn sách này, tôi sử dụng IntelliJ IDEA — IDE mà tôi thường xuyên làm việc nhất. Đừng bận tâm — cấu trúc của một dự án Maven hoàn toàn đồng nhất bất kể bạn chọn IDE nào.

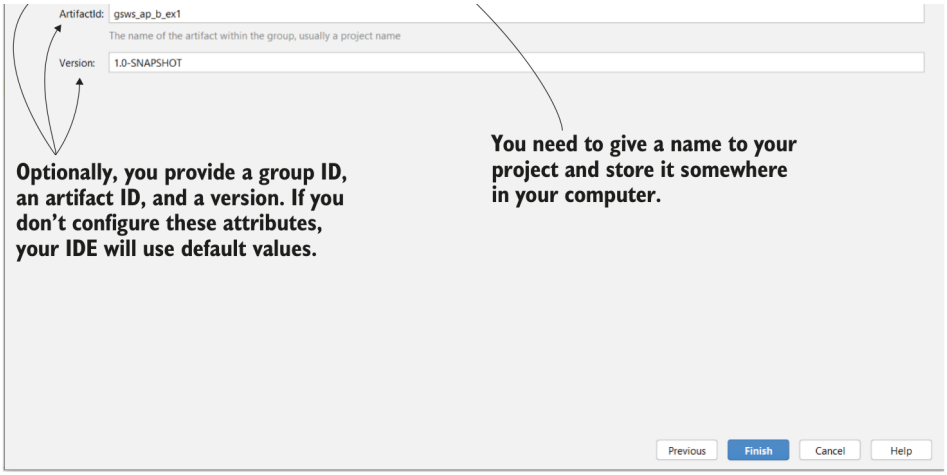
Hãy bắt đầu bằng việc tạo một dự án mới. Bạn tạo một dự án mới trong IntelliJ bằng cách vào menu **File > New > Project**. Thao tác này sẽ mở ra cửa sổ như trong Hình 2.1.



Hình 2.1: Tạo một dự án Maven mới. Sau khi chọn File > New > Project, bạn sẽ đến cửa sổ này, nơi bạn chọn loại dự án từ bảng bên trái. Trong trường hợp này, chúng ta chọn Maven. Ở phần trên của cửa sổ, bạn chọn phiên bản JDK muốn sử dụng để biên dịch và chạy dự án (trong sách dùng Java 11 trở lên).

Sau khi chọn loại dự án, ở cửa sổ kế tiếp (Hình 2.2), bạn cần đặt tên cho dự án. Bên cạnh tên dự án và vị trí lưu trữ tệp trên máy tính, đối với một dự án Maven bạn còn có thể chỉ định thêm:

- **Group ID:** Được dùng để nhóm nhiều dự án liên quan lại với nhau.
- **Artifact ID:** Tên định danh của ứng dụng hiện tại.
- **Version:** Mã định danh trạng thái phiên bản triển khai hiện thời.



Hình 2.2: Trước khi hoàn tất tạo dự án, bạn cần đặt tên cho nó và chỉ định vị trí lưu trữ trên máy tính. Tùy chọn, bạn có thể cung cấp Group ID, Artifact ID và Version. Nhấn Finish ở góc dưới bên phải để hoàn tất.

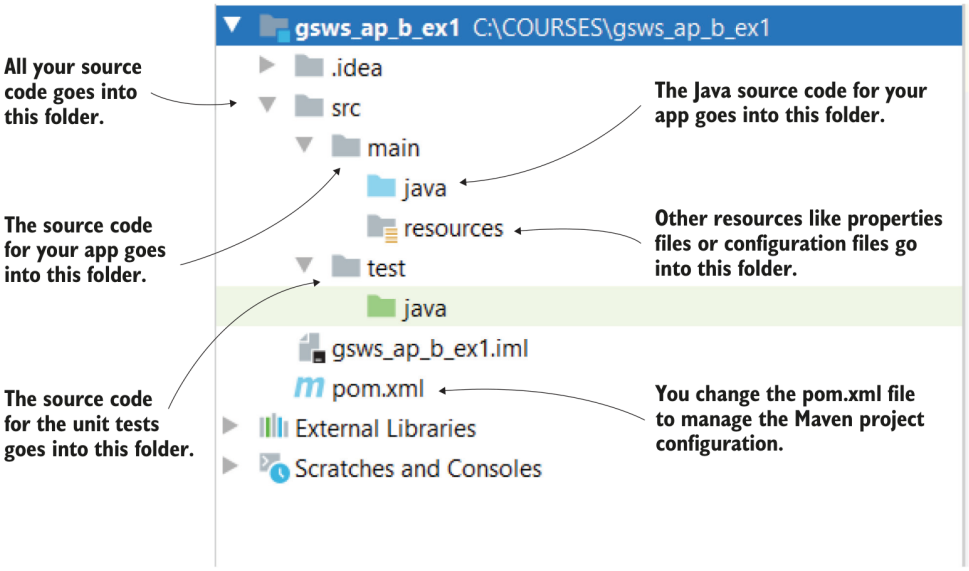
Trong một ứng dụng thực tế, ba thuộc tính này là những thông tin rất quan trọng. Nhưng trong phạm vi cuốn sách này, do chúng ta chỉ làm việc trên các ví dụ học tập, bạn có thể bỏ qua chúng và để IDE tự động điền các giá trị mặc định.

Khi dự án đã được tạo, bạn sẽ thấy cấu trúc của nó tương tự như minh họa trong Hình 2.3. Khi nhìn vào dự án lần đầu tiên, bạn sẽ nhận thấy hai thành phần trọng yếu:

- Thư mục `src` (thư mục nguồn): Nơi bạn đặt toàn bộ những gì thuộc về ứng dụng.
- Tệp `pom.xml`: Nơi bạn viết các cấu hình cho dự án Maven, chẳng hạn như khai báo các thư viện phụ thuộc mới.

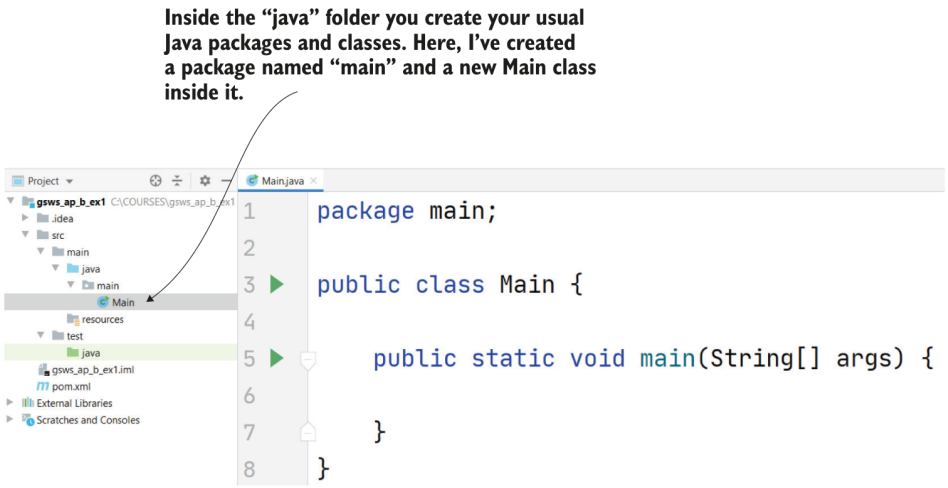
Maven tổ chức thư mục `src` thành các thư mục con sau:

- Thư mục `main`: Nơi lưu mã nguồn của ứng dụng. Thư mục này chia tách mã Java và các cấu hình riêng biệt thành hai thư mục con là `java` và `resources`.
- Thư mục `test`: Nơi chứa mã nguồn của các bài kiểm thử đơn vị (*unit tests*). Chúng ta sẽ bàn kỹ hơn về kiểm thử đơn vị trong Chương 15.



Hình 2.3: Cách tổ chức cấu trúc của một dự án Maven. Bên trong thư mục src, chúng ta chứa toàn bộ những gì thuộc về ứng dụng: mã nguồn ứng dụng nằm trong thư mục main, và mã kiểm thử đơn vị nằm trong thư mục test. Trong tệp pom.xml, chúng ta khai báo cấu hình dự án (chủ yếu dùng để định nghĩa các thư viện phụ thuộc).

Hình 2.4 minh họa cách thức bạn bổ sung mã nguồn mới vào thư mục `main/java` của dự án Maven. Toàn bộ các lớp mới của ứng dụng sẽ được đặt vào đây.



Hình 2.4: Bên trong thư mục java, bạn tạo các gói (packages) và lớp Java thông thường của ứng dụng. Đây là những lớp định nghĩa toàn bộ logic của ứng dụng và khai thác các thư viện phụ thuộc được cung cấp.

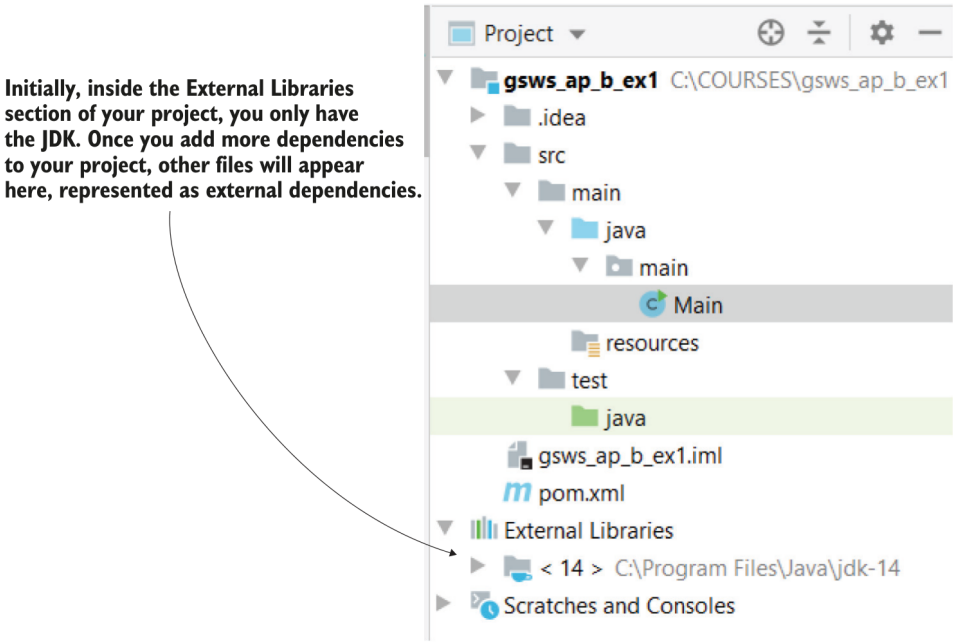
Trong các dự án chúng ta tạo ra ở cuốn sách này, chúng ta sử dụng rất nhiều thư viện phụ thuộc bên ngoài: các thư viện hoặc framework hỗ trợ hiện thực hóa chức năng của ví dụ. Để thêm các phụ thuộc này vào dự án Maven, chúng ta cần thay đổi nội dung của tệp `pom.xml`.

Trong Đoạn mã 2.1, bạn sẽ thấy nội dung mặc định của tệp `pom.xml` ngay sau khi vừa tạo dự án Maven.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>
  <groupId>org.example</groupId>
  <artifactId>sq-ch2-ex1</artifactId>
  <version>1.0-SNAPSHOT</version>
</project>
```

Listing 2.1: Nội dung mặc định của tệp pom.xml khi vừa khởi tạo

Với tệp `pom.xml` nguyên bản này, dự án hoàn toàn chưa sử dụng bất kỳ thư viện bên ngoài nào. Nếu kiểm tra phần thư viện ngoài trong IDE, bạn chỉ thấy duy nhất môi trường JDK (Hình 2.5).



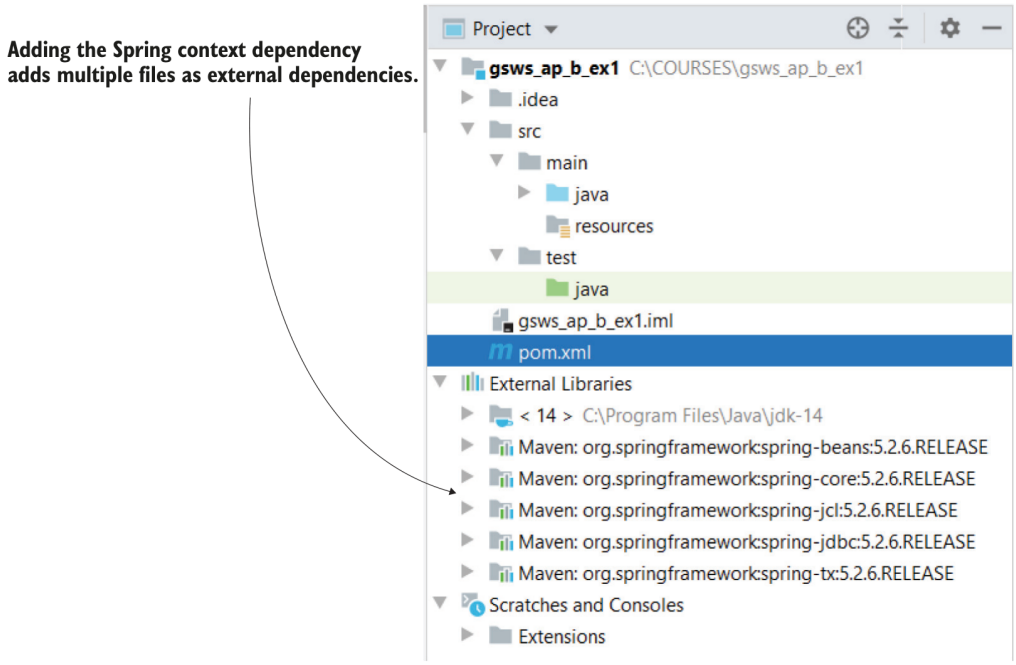
Hình 2.5: Với tệp pom.xml mặc định, dự án của bạn chỉ sử dụng JDK làm phụ thuộc ngoại vi. Một trong những lý do chính khiến bạn sửa đổi pom.xml là để khai báo thêm các thư viện phụ thuộc mới cho ứng dụng.

Đoạn mã 2.2 chỉ cho bạn cách thức thêm các thư viện phụ thuộc bên ngoài vào dự án. Bạn khai báo toàn bộ các phụ thuộc nằm giữa cặp thẻ `<dependencies> </dependencies>`. Mỗi

phụ thuộc được đại diện bởi một nhóm thẻ con <dependency> </dependency>, nơi bạn chỉ định các thuộc tính của phụ thuộc: Group ID, Artifact ID và Version. Maven sẽ tự động tìm kiếm phụ thuộc dựa trên ba giá trị này và tải các tệp JAR tương ứng về từ kho lưu trữ trung tâm Maven Central (Hình 2.6).

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>
  <groupId>org.example</groupId>
  <artifactId>sq_ch2_ex1</artifactId>
  <version>1.0-SNAPSHOT</version>
  <dependencies>
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-jdbc</artifactId>
      <version>5.2.6.RELEASE</version>
    </dependency>
  </dependencies>
</project>
```

Listing 2.2: Thêm một thư viện phụ thuộc mới vào tệp pom.xml



Hình 2.6: Khi bạn thêm một phụ thuộc mới vào pom.xml, Maven sẽ tự động tải các tệp jar đại diện cho phụ thuộc đó. Bạn sẽ thấy các tệp jar này xuất hiện trong mục External Libraries của dự án.

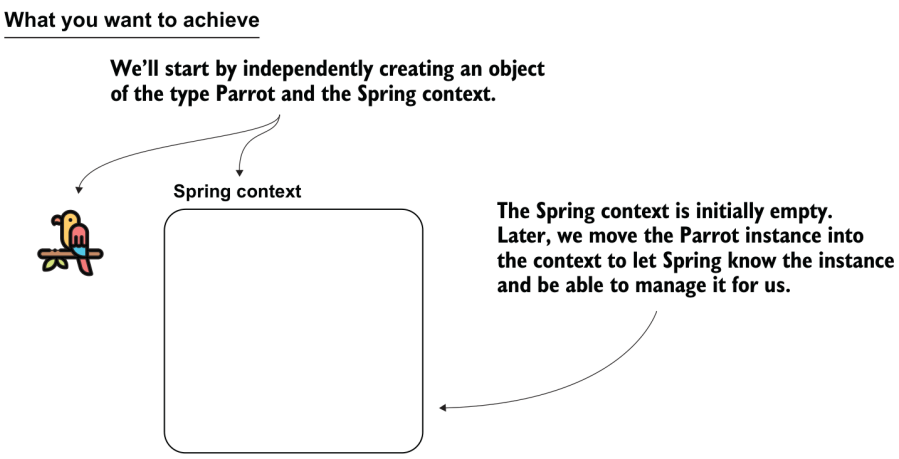
Giờ đây chúng ta đã sẵn sàng chuyển sang mục tiếp theo để tìm hiểu các khái niệm nền tảng về ngữ cảnh Spring và cách quản lý nó qua phụ thuộc `spring-context`.

2.2 Thêm các bean mới vào ngữ cảnh Spring

Trong mục này, bạn sẽ học cách thêm các thể hiện đối tượng mới (tức là các bean) vào ngữ cảnh Spring. Bạn sẽ khám phá ra rằng có nhiều cách tiếp cận khác nhau để đưa bean vào ngữ cảnh nhằm giúp Spring quản lý chúng và gắn kết các tính năng vào ứng dụng của bạn. Tùy thuộc vào từng trường hợp cụ thể, bạn sẽ chọn một cách làm phù hợp. Bạn có thể thêm bean vào ngữ cảnh theo 3 cách sau:

- Sử dụng chú thích `@Bean`.
- Sử dụng các chú thích định hình (*stereotype annotations*).
- Bằng phương thức lập trình (*programmatically*).

Trước tiên, hãy tạo một dự án Java cơ sở chưa liên kết với bất kỳ framework nào. Sau đó, chúng ta sẽ bổ sung các thư viện cần thiết để sử dụng ngữ cảnh Spring và khởi tạo nó (Hình 2.7). Ví dụ khởi đầu này sẽ đóng vai trò làm khung xương mẫu cho các thử nghiệm ở các tiểu mục tiếp theo.



Hình 2.7: Bước khởi đầu: Chúng ta tạo độc lập một thể hiện đối tượng kiểu Parrot và một ngữ cảnh Spring trống. Mục tiêu tiếp theo là đưa thể hiện Parrot vào ngữ cảnh để Spring nhận diện và quản lý nó.

Chúng ta tạo một dự án Maven và định nghĩa một lớp đơn giản. Để ví dụ trở nên sinh động và dễ hình dung, tôi sẽ định nghĩa một lớp mang tên `Parrot` (con vẹt) chỉ có một thuộc tính chuỗi `name` đại diện cho tên của chú vẹt (Đoạn mã 2.3).

```
public class Parrot {
  private String name;
  // Cac phuong thuc getter va setter duoc luoc bo
}
```

Listing 2.3: Lớp Parrot

Bây giờ bạn có thể định nghĩa một lớp chứa phương thức `main` và tạo ra một thể hiện của lớp `Parrot` như trong Đoạn mã 2.4. Tôi thường đặt tên cho lớp này là `Main`.

```
public class Main {
  public static void main(String[] args) {
    Parrot p = new Parrot();
  }
}
```

Listing 2.4: Tạo một thể hiện của lớp Parrot

Đã đến lúc thêm phụ thuộc cần thiết vào dự án. Chúng ta khai báo phụ thuộc `spring-context` trong tệp `pom.xml` như trong Đoạn mã 2.5.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
  http://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```
  <modelVersion>4.0.0</modelVersion>
  <groupId>org.example</groupId>
  <artifactId>sq-ch2-ex1</artifactId>
  <version>1.0-SNAPSHOT</version>
  <dependencies>
     <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-context</artifactId>
        <version>5.2.6.RELEASE</version>
     </dependency>
  </dependencies>
</project>
```

Listing 2.5: Thêm phụ thuộc cho ngữ cảnh Spring trong pom.xml

Một đặc tính quan trọng cần lưu ý là Spring được thiết kế theo cấu trúc mô-đun (*modular*). Điều này đồng nghĩa với việc bạn không cần phải nạp toàn bộ framework Spring vào ứng dụng khi chỉ sử dụng một vài năng lực nhất định. Bạn chỉ cần thêm đúng những thành phần mình thực sự dùng đến.

Cách tra cứu phụ thuộc Maven của Spring

Bạn có thể tự hỏi làm thế nào để biết chính xác phụ thuộc Maven nào cần thêm vào. Bạn không cần phải học thuộc lòng chúng. Bất cứ khi nào làm việc với một dự án Spring mới, bạn có thể tra cứu trực tiếp các phụ thuộc cần thiết trong tài liệu tham khảo chính thức của Spring (`https://docs.spring.io/spring-framework/docs/current/spring-framework-reference/core.html`). Nhìn chung, các phụ thuộc cốt lõi của Spring đều thuộc nhóm `org.springframework`.

Với phụ thuộc đã sẵn sàng, chúng ta có thể tạo ra một thể hiện của ngữ cảnh Spring. Trong Đoạn mã 2.6, bạn sẽ thấy cách tôi cập nhật phương thức `main` để khởi tạo ngữ cảnh Spring.

```
public class Main {
  public static void main(String[] args) {
    var context =
       new AnnotationConfigApplicationContext();
    Parrot p = new Parrot();
  }
}
```

Listing 2.6: Khởi tạo thể hiện của ngữ cảnh Spring

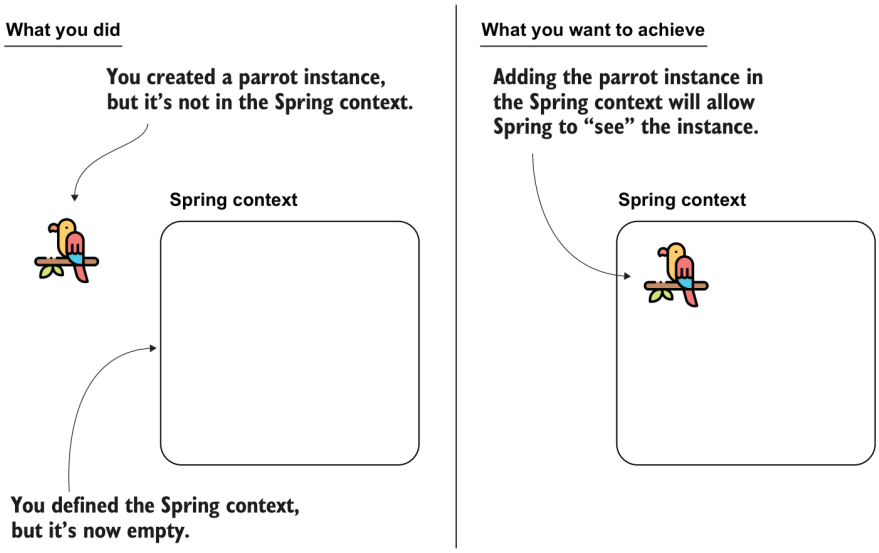
Lớp AnnotationConfigApplicationContext

Chúng ta sử dụng lớp `AnnotationConfigApplicationContext` để khởi tạo ngữ cảnh Spring. Spring cung cấp nhiều bản triển khai ngữ cảnh khác nhau, nhưng vì trong phần lớn các ứng dụng hiện đại ngày nay chúng ta đều sử dụng cách tiếp cận bằng chú thích (*annotations*), nên chúng ta sẽ tập trung vào lớp này trong suốt cuốn sách. Nếu bạn mới bắt đầu học Spring, lời khuyên chân thành của tôi là tránh sa đà vào chi tiết các chuỗi kế thừa phức tạp của các lớp ngữ cảnh để không bị lạc lối khỏi những kiến thức trọng tâm.

Như minh họa trong Hình 2.8, hiện tại bạn đã tạo ra một thể hiện của `Parrot`, thêm phụ thuộc Spring vào dự án, và khởi tạo một ngữ cảnh Spring trống. Mục tiêu tiếp theo của chúng ta là đưa đối tượng `Parrot` này vào bên trong ngữ cảnh để Spring nhận biết nó.

with unimportant details instead of focusing on the essential things.

As presented in figure 2.8, you created an instance of `Parrot`, added the Spring context dependencies to your project, and created an instance of the Spring context. Your objective is to add the `Parrot` object to the context, which is the next step.



Hình 2.8: Bạn đã tạo thể hiện ngữ cảnh Spring và thể hiện Parrot. Bây giờ, mục tiêu là đưa đối tượng Parrot vào trong ngữ cảnh Spring để Spring nhận biết và quản lý thể hiện này.

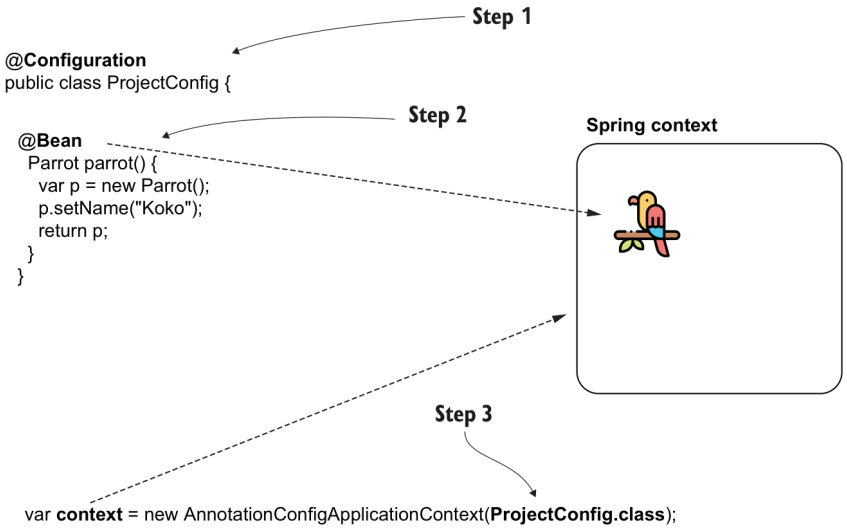
2.2.1 Sử dụng chú thích @Bean để thêm bean vào ngữ cảnh Spring

Trong tiểu mục này, chúng ta sẽ tìm hiểu cách thêm một thể hiện đối tượng vào ngữ cảnh Spring bằng cách sử dụng chú thích `@Bean`. Cách làm này cho phép bạn thêm các thể hiện của những lớp được định nghĩa ngay trong dự án của mình (như `Parrot`), cũng như các lớp mà bạn không tự tay tạo ra nhưng lại sử dụng trong ứng dụng (chẳng hạn từ các thư viện bên thứ ba). Đây là cách tiếp cận trực quan và dễ hiểu nhất khi mới bắt đầu.

Các bước bạn cần thực hiện để thêm một bean vào ngữ cảnh Spring bằng chú thích `@Bean` như sau (Hình 2.9):

1. Định nghĩa một lớp cấu hình (được đánh dấu bằng chú thích `@Configuration`) cho dự án của bạn.
2. Thêm một phương thức vào lớp cấu hình trả về thể hiện đối tượng mà bạn muốn đưa vào ngữ cảnh, và đánh dấu phương thức đó bằng chú thích `@Bean`.
3. Chỉ định cho Spring sử dụng lớp cấu hình đã định nghĩa ở bước 1 khi khởi tạo ngữ cảnh.

> **NOTE** Remember, you can find the book's projects in the "Resources" section of the live book.



Hình 2.9: Ba bước thêm bean vào ngữ cảnh sử dụng chú thích @Bean: Định nghĩa lớp @Configuration, viết phương thức @Bean trả về đối tượng, và truyền lớp cấu hình vào AnnotationConfigApplicationContext.

Khái niệm Lớp cấu hình (Configuration Class)

Lớp cấu hình là một lớp đặc biệt trong ứng dụng Spring mà chúng ta dùng để chỉ dẫn cho Spring thực hiện các hành động cụ thể, chẳng hạn như yêu cầu Spring tạo các bean hoặc kích hoạt những chức năng nhất định.

Bước 1: Định nghĩa một lớp cấu hình trong dự án Bước đầu tiên là tạo một lớp cấu hình trong dự án, được đánh dấu bằng chú thích `@Configuration`. Đoạn mã 2.7 minh họa cách định nghĩa lớp cấu hình mà tôi đặt tên là `ProjectConfig`.

```
@Configuration
public class ProjectConfig {
}
```

Listing 2.7: Định nghĩa lớp cấu hình cho dự án

Quy ước phân chia gói (Package Organization)

Tôi phân chia các lớp vào các gói riêng biệt để mã nguồn sáng sủa và dễ hiểu hơn. Ví dụ, tôi tạo lớp cấu hình trong gói `config`, và lớp `Main` trong gói `main`. Đây là một thực hành tốt mà bạn nên áp dụng trong các dự án thực tế.

Bước 2: Tạo phương thức trả về bean và đánh dấu bằng @Bean Chúng ta định nghĩa một phương thức trả về thể hiện đối tượng cần đưa vào ngữ cảnh và đánh dấu phương thức đó bằng chú thích `@Bean`. Chú thích này báo cho Spring biết rằng nó cần gọi phương thức này khi khởi tạo ngữ cảnh và đưa giá trị trả về vào bên trong ngữ cảnh (Đoạn mã 2.8).

```
@Configuration
public class ProjectConfig {

  @Bean
  Parrot parrot() {
    var p = new Parrot();
    p.setName("Koko");
    return p;
  }
}
```

Listing 2.8: Định nghĩa phương thức @Bean trong lớp cấu hình

Hãy quan sát thấy rằng tên tôi đặt cho phương thức không hề chứa động từ. Bạn có thể đã học được rằng quy ước chuẩn của Java là nên dùng động từ cho tên phương thức vì chúng đại diện cho các hành động. Nhưng đối với các phương thức dùng để thêm bean vào ngữ cảnh Spring, chúng ta không tuân theo quy ước này. Những phương thức như vậy đại diện cho chính các thể hiện đối tượng mà chúng trả về. Tên của phương thức cũng chính là tên định danh của bean trong ngữ cảnh (trong Đoạn mã 2.8, tên của bean sẽ là `"parrot"`). Theo quy ước, bạn có thể dùng danh từ, và thông thường nó trùng với tên của lớp.

Bước 3: Chỉ dẫn Spring khởi tạo ngữ cảnh sử dụng lớp cấu hình vừa tạo Bây giờ chúng ta cần đảm bảo Spring sử dụng lớp cấu hình này khi khởi tạo ngữ cảnh. Đoạn mã 2.9 chỉ ra cách truyền lớp cấu hình vào hàm khởi tạo của `AnnotationConfigApplicationContext`.

```
public class Main {
  public static void main(String[] args) {
    var context =
      new AnnotationConfigApplicationContext(
            ProjectConfig.class);
  }
```

```
}
```

Listing 2.9: Khởi tạo ngữ cảnh Spring dựa trên lớp cấu hình đã định nghĩa

Để kiểm chứng rằng thể hiện `Parrot` thực sự đã nằm trong ngữ cảnh, bạn có thể lấy tham chiếu của nó ra và in tên chú vẹt ra màn hình console như trong Đoạn mã 2.10.

```
public class Main {
  public static void main(String[] args) {
    var context =
      new AnnotationConfigApplicationContext(
        ProjectConfig.class);

    Parrot p = context.getBean(Parrot.class);
    System.out.println(p.getName());
  }
}
```

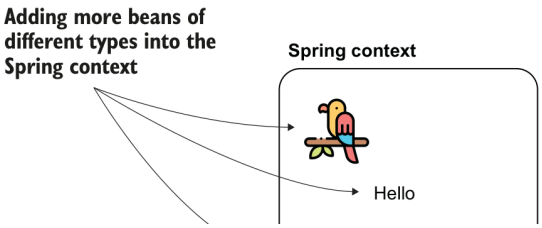
Listing 2.10: Lấy tham chiếu tới thể hiện Parrot từ ngữ cảnh

Khi chạy chương trình, bạn sẽ thấy tên chú vẹt bạn đã thêm vào ngữ cảnh hiển thị trên màn hình: `Koko`.

Giống như ví dụ trên, bạn có thể thêm bất kỳ loại đối tượng nào vào ngữ cảnh Spring (Hình 2.10). Hãy thử thêm một đối tượng kiểu `String` và một đối tượng kiểu `Integer` vào ngữ cảnh.

discussed subject.

As in the previous example, you can add any kind of object to the Spring context (figure 2.10). Let's also add a `String` and an `Integer` and see that it's working.



Hình 2.10: Bạn có thể thêm bất kỳ đối tượng nào vào ngữ cảnh Spring để Spring nhận biết và quản lý nó (ví dụ thêm đối tượng String và Integer bên cạnh Parrot).

Đoạn mã 2.11 chỉ cho bạn cách tôi thay đổi lớp cấu hình để thêm hai bean kiểu `String` và `Integer`.

```
@Configuration
public class ProjectConfig {

```

```
  @Bean
  Parrot parrot() {
    var p = new Parrot();
    p.setName("Koko");
    return p;
  }

  @Bean
  String hello() {
    return "Hello";
  }

  @Bean
  Integer ten() {
    return 10;
  }
}
```

Listing 2.11: Bổ sung thêm hai bean thuộc các kiểu dữ liệu khác vào ngữ cảnh

Bây giờ bạn có thể tham chiếu tới hai bean mới này theo cùng cách thức đã làm với chú vẹt. Đoạn mã 2.12 minh họa phương thức `main` in giá trị của các bean ra console.

```
public class Main {
  public static void main(String[] args) {
    var context = new AnnotationConfigApplicationContext(
                    ProjectConfig.class);

    Parrot p = context.getBean(Parrot.class);
    System.out.println(p.getName());

    String s = context.getBean(String.class);
    System.out.println(s);

    Integer n = context.getBean(Integer.class);
    System.out.println(n);
  }
}
```

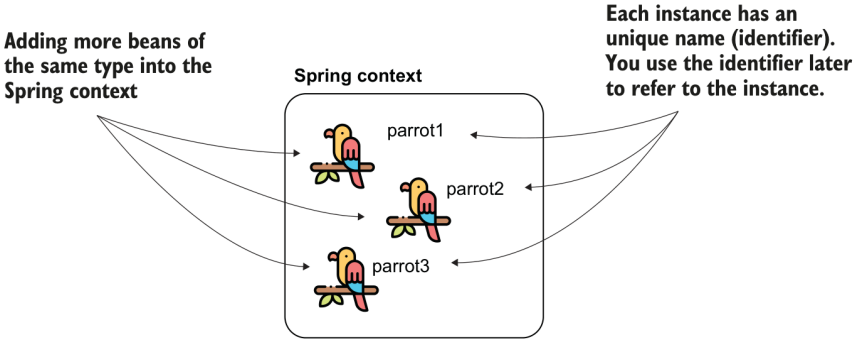
Listing 2.12: In giá trị của hai bean mới ra console

Kết quả hiển thị trên màn hình console sẽ lần lượt là:

```
Koko
```

```
Hello
10
```

Thêm nhiều bean cùng một kiểu dữ liệu vào ngữ cảnh Cho đến nay, chúng ta đã thêm các bean thuộc các kiểu dữ liệu khác nhau. Nhưng liệu chúng ta có thể thêm nhiều hơn một đối tượng có cùng kiểu dữ liệu vào ngữ cảnh hay không (Hình 2.11)? Câu trả lời là có!



Hình 2.11: Bạn có thể thêm nhiều bean cùng kiểu dữ liệu vào ngữ cảnh Spring bằng cách khai báo nhiều phương thức @Bean. Mỗi thể hiện sẽ có một tên định danh duy nhất (parrot1, parrot2, parrot3).

Đoạn mã 2.13 minh họa cách tôi khai báo ba bean cùng kiểu `Parrot` trong lớp cấu hình.

```
@Configuration
public class ProjectConfig {

  @Bean
  Parrot parrot1() {
    var p = new Parrot();
    p.setName("Koko");
    return p;
  }

  @Bean
  Parrot parrot2() {
    var p = new Parrot();
    p.setName("Miki");
    return p;
  }

  @Bean
  Parrot parrot3() {
    var p = new Parrot();
```

```
    p.setName("Riki");
    return p;
  }
}
```

Listing 2.13: Thêm nhiều bean cùng kiểu Parrot vào ngữ cảnh Spring

Lúc này, bạn không thể lấy bean từ ngữ cảnh chỉ bằng cách chỉ định kiểu dữ liệu `Parrot.class` được nữa (Đoạn mã 2.14). Nếu làm vậy, Spring sẽ ném ra ngoại lệ `NoUniqueBeanDefinitionException` vì không biết bạn muốn chọn chú vẹt nào trong số ba chú vẹt đã khai báo!

```
public class Main {
  public static void main(String[] args) {
    var context = new
      AnnotationConfigApplicationContext(ProjectConfig.class);

    // Dong nay se nem ra ngoai le NoUniqueBeanDefinitionException
    Parrot p = context.getBean(Parrot.class);
    System.out.println(p.getName());
  }
}
```

Listing 2.14: Lấy bean chỉ bằng kiểu dữ liệu khi có nhiều bean cùng kiểu (Gây lỗi)

Để giải quyết vấn đề nhập nhằng này, bạn cần chỉ định đích danh tên của bean khi gọi phương thức `getBean()` (Đoạn mã 2.15).

```
public class Main {
  public static void main(String[] args) {
    var context = new
      AnnotationConfigApplicationContext(ProjectConfig.class);

    Parrot p = context.getBean("parrot2", Parrot.class);
    System.out.println(p.getName());
  }
}
```

Listing 2.15: Tham chiếu tới bean bằng tên định danh cụ thể

Chạy lại chương trình, bạn sẽ thấy tên chú vẹt thứ hai hiển thị trên màn hình: `Miki`.

Nếu bạn muốn đặt một tên khác cho bean thay vì lấy theo tên phương thức, bạn có thể dùng thuộc tính `name` hoặc `value` của chú thích `@Bean`:

- `@Bean(name = "miki")`
- `@Bean(value = "miki")`

- `@Bean("miki")`

Định nghĩa một bean làm mặc định với chú thích @Primary Khi có nhiều bean cùng kiểu trong ngữ cảnh, bạn có thể chỉ định một bean làm lựa chọn mặc định bằng chú thích `@Primary`. Khi đó, nếu bạn gọi `getBean(Parrot.class)` mà không truyền tên bean, Spring sẽ tự động chọn bean được đánh dấu `@Primary`:

```
@Bean
@Primary
Parrot parrot2() {
  var p = new Parrot();
  p.setName("Miki");
  return p;
}
```

Listing 2.16: Đánh dấu một bean là ưu tiên chính với @Primary

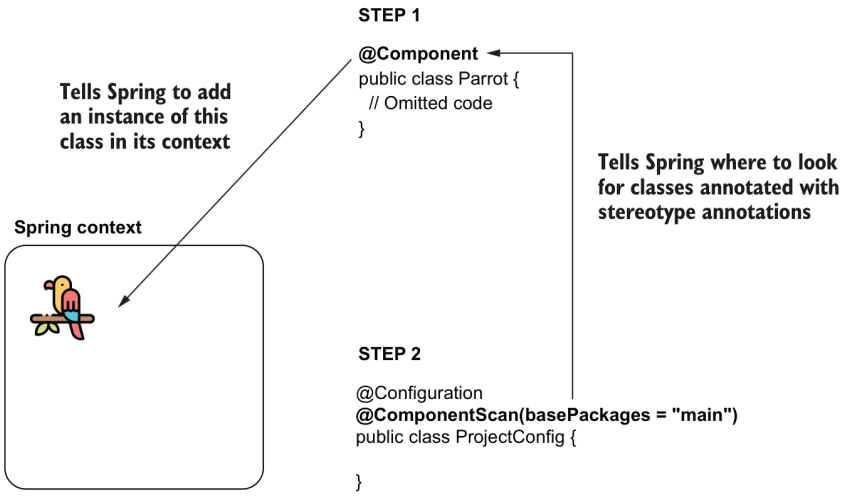
2.2.2 Sử dụng các chú thích định hình (Stereotype annotations) để thêm bean

Trong tiểu mục này, bạn sẽ học một cách tiếp cận khác để thêm bean vào ngữ cảnh Spring. Với các chú thích định hình (*stereotype annotations*), bạn sẽ thấy mình phải viết ít mã nguồn hơn đáng kể để yêu cầu Spring đưa một bean vào ngữ cảnh.

Spring cung cấp nhiều chú thích định hình khác nhau, nhưng phổ biến và căn bản nhất là `@Component`. Khi bạn đặt chú thích này lên trên một lớp, chúng ta nói rằng bạn đã đánh dấu lớp đó là một thành phần (*component*). Khi ứng dụng khởi tạo ngữ cảnh, Spring sẽ tự động tạo một thể hiện của lớp đó và đưa vào ngữ cảnh.

Các bước thực hiện phương pháp này như sau (Hình 2.12):

1. Đánh dấu lớp bạn muốn Spring quản lý bằng chú thích `@Component`.
2. Đặt chú thích `@ComponentScan` trên lớp cấu hình để chỉ dẫn cho Spring biết nơi cần quét tìm các lớp này.



Hình 2.12: Hai bước thêm bean bằng stereotype annotation: Đánh dấu @Component trên lớp Parrot và đặt @ComponentScan trên lớp cấu hình để chỉ định gói cần quét.

Đoạn mã 2.17 minh họa việc đặt chú thích `@Component` lên trên lớp `Parrot`.

```
@Component
public class Parrot {
  private String name;

  public String getName() {
    return name;
  }

  public void setName(String name) {
    this.name = name;
  }
}
```

Listing 2.17: Sử dụng chú thích @Component cho lớp Parrot

Để Spring biết nó cần quét tìm các lớp có đánh dấu chú thích này ở đâu, chúng ta sử dụng chú thích `@ComponentScan` trên lớp cấu hình (Đoạn mã 2.18).

```
@Configuration
@ComponentScan(basePackages = "main")
public class ProjectConfig {
}
```

Listing 2.18: Sử dụng @ComponentScan trên lớp cấu hình

Đoạn mã 2.19 minh họa phương thức `main` kiểm tra xem Spring đã tự động tạo và đưa bean vào ngữ cảnh hay chưa.

```
public class Main {
  public static void main(String[] args) {
    var context = new
      AnnotationConfigApplicationContext(ProjectConfig.class);

    Parrot p = context.getBean(Parrot.class);
    System.out.println(p);
    System.out.println(p.getName());
  }
}
```

Listing 2.19: Kiểm tra bean được tạo tự động bởi Spring

Khi chạy ứng dụng, dòng đầu tiên in ra biểu diễn chuỗi mặc định của thể hiện `Parrot` (chứng minh đối tượng đã tồn tại). Tuy nhiên, dòng thứ hai in ra giá trị `null` vì chúng ta chưa gán tên cho chú vẹt này. Spring chỉ đảm nhiệm việc tạo thể hiện của lớp, việc thiết lập giá trị cho đối tượng vẫn là trách nhiệm của chúng ta.

Bảng 2.1 tổng hợp ưu và nhược điểm của hai cách tiếp cận thêm bean vào ngữ cảnh Spring.

Bảng 2.1: So sánh ưu và nhược điểm giữa cách dùng @Bean và Stereotype annotations

Sử dụng chú thích @Bean	**Sử dụng chú thích định hình (@Component)**
1. Bạn có toàn quyền kiểm soát quá trình tạo thể hiện đối tượng. Việc khởi tạo và cấu hình diễn ra trong thân phương thức.	1. Bạn chỉ có quyền can thiệp vào thể hiện sau khi framework đã tạo ra nó.
2. Có thể thêm nhiều thể hiện cùng kiểu vào ngữ cảnh Spring (như đã thêm 3 chú vẹt ở ví dụ trước).	2. Chỉ có thể thêm duy nhất một thể hiện của lớp vào ngữ cảnh bằng cách này.
3. Có thể dùng để thêm bất kỳ đối tượng nào vào ngữ cảnh, ngay cả các lớp từ thư viện bên ngoài mà bạn không sở hữu mã nguồn.	3. Chỉ dùng được cho các lớp do chính bạn sở hữu và có thể can thiệp sửa đổi mã nguồn để thêm chú thích.
4. Phải viết phương thức riêng cho từng bean, làm tăng lượng mã rập khuôn (*boilerplate code*).	4. Viết ít mã hơn, giúp cấu hình dự án gọn gàng và dễ đọc hơn.

Sử dụng @PostConstruct để cấu hình thể hiện sau khi tạo Khi sử dụng `@Component`, làm thế nào để thực thi một số lệnh cấu hình ngay sau khi Spring gọi hàm khởi tạo của lớp?

Chúng ta sử dụng chú thích `@PostConstruct`.

Chúng ta định nghĩa một phương thức bên trong lớp thành phần và đánh dấu nó bằng `@PostConstruct`:

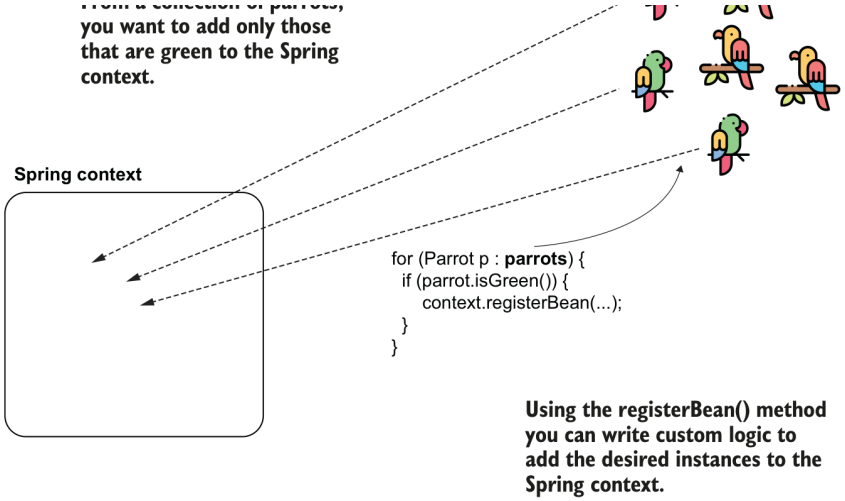
```
@Component
public class Parrot {
  private String name;

  @PostConstruct
  public void init() {
    this.name = "Kiki";
  }

  // Getter va Setter duoc luoc bo
}
```

Listing 2.20: Sử dụng chú thích @PostConstruct để khởi tạo giá trị

Khi ứng dụng khởi chạy, phương thức `init()` sẽ được tự động gọi ngay sau hàm khởi tạo, và thuộc tính `name` sẽ có giá trị là `"Kiki"`. Ngược lại, chú thích `@PreDestroy` cho phép định nghĩa phương thức chạy ngay trước khi Spring đóng và dọn dẹp ngữ cảnh.

2.2.3 Thêm bean vào ngữ cảnh Spring bằng phương thức lập trình

Kể từ Spring 5, chúng ta có thêm tùy chọn đưa bean vào ngữ cảnh theo phương thức lập trình (*programmatically*). Phương pháp này mang lại sự linh hoạt tuyệt đối khi bạn cần thêm bean một cách động theo các điều kiện logic phức tạp mà chú thích `@Bean` hay `@Component` không đáp ứng được (Hình 2.13).



Hình 2.13: Sử dụng phương thức registerBean() để thêm các đối tượng cụ thể vào ngữ cảnh Spring theo điều kiện tùy biến (ví dụ: chỉ thêm những chú vẹt có màu xanh lá cây vào ngữ cảnh).

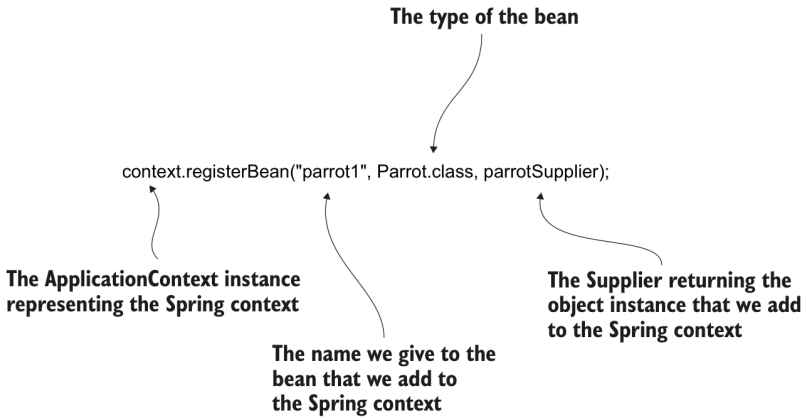
Để thêm bean bằng phương thức lập trình, bạn gọi phương thức `registerBean()` trên thể hiện của `ApplicationContext`:

```
<T> void registerBean(
  String beanName,
  Class<T> beanClass,
  Supplier<T> supplier,
  BeanDefinitionCustomizer... customizers);
```

Listing 2.21: Chữ ký phương thức registerBean()

Phương thức này tiếp nhận 4 tham số (Hình 2.14):

1. `beanName`: Tên định danh của bean trong ngữ cảnh (có thể truyền `null` nếu không cần đặt tên).
2. `beanClass`: Lớp định nghĩa bean (ví dụ `Parrot.class`).
3. `supplier`: Một thể hiện của giao diện hàm `Supplier<T>` trả về đối tượng cần đưa vào ngữ cảnh.
4. `customizers`: Danh sách tham số biến đổi (*varargs*) thuộc kiểu `BeanDefinitionCustomizer` để cấu hình thêm các thuộc tính đặc biệt (chẳng hạn như đánh dấu bean là ưu tiên chính: `bc -> bc.setPrimary(true)`).



Hình 2.14: Cú pháp gọi phương thức registerBean() để thêm bean vào ngữ cảnh Spring bằng lập trình.

Đoạn mã 2.22 minh họa cách thức sử dụng phương thức `registerBean()` trong phương thức `main`.

```
public class Main {
  public static void main(String[] args) {
    var context =
      new AnnotationConfigApplicationContext(ProjectConfig.class);

    Parrot x = new Parrot();
```

```
    x.setName("Kiki");

    Supplier<Parrot> parrotSupplier = () -> x;

    context.registerBean("parrot1",
      Parrot.class, parrotSupplier);

    Parrot p = context.getBean(Parrot.class);
    System.out.println(p.getName());
  }
}
```

Listing 2.22: Sử dụng phương thức registerBean() để thêm bean vào ngữ cảnh

Cấu hình ngữ cảnh bằng XML thời kỳ đầu

Trong cuốn sách này, chúng ta chỉ sử dụng các phương pháp cấu hình hiện đại. Tuy nhiên, việc nắm được cách các thế hệ lập trình viên thời kỳ đầu cấu hình Spring bằng XML cũng rất hữu ích. Trong Phụ lục B, bạn sẽ tìm thấy một ví dụ ngắn minh họa cách dùng XML để thêm bean vào ngữ cảnh Spring.

Tóm tắt chương

- Điều đầu tiên bạn cần học trong Spring là thêm các thể hiện đối tượng (gọi là các bean) vào ngữ cảnh Spring. Bạn có thể hình dung ngữ cảnh như một chiếc xô chứa những đối tượng mà bạn muốn Spring quản lý. Spring chỉ có thể nhìn thấy và tác động lên những đối tượng nằm bên trong ngữ cảnh của nó.
- Bạn có thể thêm bean vào ngữ cảnh Spring theo 3 cách:
  - **Sử dụng chú thích `@Bean`:** Cho phép bạn thêm bất kỳ đối tượng nào vào ngữ cảnh và thậm chí thêm nhiều đối tượng cùng kiểu. Đây là cách làm linh hoạt nhất nhưng đòi hỏi viết nhiều mã hơn vì mỗi bean cần một phương thức riêng biệt trong lớp cấu hình.
  - **Sử dụng các chú thích định hình (như `@Component`):** Cho phép tạo bean tự động cho các lớp thuộc quyền sở hữu của bạn có đánh dấu chú thích. Cách tiếp cận này giúp viết ít mã hơn và cấu hình gọn gàng hơn.
  - **Sử dụng phương thức `registerBean()`:** Cung cấp khả năng lập trình tùy biến logic để thêm bean vào ngữ cảnh theo điều kiện động (áp dụng từ Spring 5 trở lên).

Chương 3

Ngữ cảnh Spring: Nối dây các bean

Trong Chương 2, chúng ta đã thảo luận về ngữ cảnh Spring (*Spring context*): vùng không gian trong bộ nhớ của ứng dụng nơi chúng ta thêm vào các thể hiện đối tượng mà mình muốn Spring quản lý. Bởi vì Spring vận hành dựa trên nguyên lý Đảo ngược quyền điều khiển (*Inversion of Control - IoC*) như đã bàn ở Chương 1, chúng ta cần chỉ dẫn cho Spring biết những đối tượng nào trong ứng dụng mà nó cần kiểm soát. Spring cần quyền kiểm soát đối với một số đối tượng nhằm bồi đắp và mở rộng năng lực cho chúng bằng những tính năng phong phú mà framework cung cấp. Trong Chương 2, bạn đã học nhiều phương pháp khác nhau để đưa các thể hiện đối tượng vào ngữ cảnh Spring. Bạn cũng hiểu rằng chúng ta thêm các thể hiện này (gọi là các bean) vào ngữ cảnh để giúp Spring nhận biết được sự hiện diện của chúng.

Trong chương này, chúng ta sẽ bàn luận về cách thức truy cập các bean mà chúng ta đã đưa vào ngữ cảnh Spring. Ở Chương 2, chúng ta đã sử dụng trực tiếp phương thức `getBean()` của thể hiện ngữ cảnh để lấy ra các bean. Tuy nhiên, trong các ứng dụng thực tế, chúng ta cần tham chiếu từ bean này sang bean khác theo một cách thức trực tiếp và thuận tiện hơn — bằng cách chỉ dẫn cho Spring tự động cung cấp tham chiếu tới một thể hiện từ ngữ cảnh của nó tới chính xác nơi mà chúng ta cần. Bằng cách này, chúng ta thiết lập nên mối quan hệ giữa các bean (một bean sẽ nắm giữ tham chiếu tới một bean khác để ủy quyền thực thi lời gọi khi cần thiết). Như bạn có thể đã biết, trong bất kỳ ngôn ngữ lập trình hướng đối tượng nào, một đối tượng thường cần phải ủy thác các trách nhiệm cụ thể cho những đối tượng khác khi hiện thực hóa hành vi của mình, do đó bạn cũng cần phải biết cách thiết lập các mối quan hệ như vậy giữa các đối tượng khi sử dụng Spring làm framework nền tảng.

Nội dung chính của chương này

- Thiết lập các mối quan hệ liên kết giữa các bean.
- Ứng dụng kỹ thuật tiêm phụ thuộc (*Dependency Injection - DI*).
- Truy cập các bean từ ngữ cảnh Spring thông qua tiêm phụ thuộc.

Bạn sẽ học được rằng có nhiều cách để truy cập các đối tượng đã được đưa vào ngữ cảnh Spring, và chúng ta sẽ nghiên cứu từng cách thức thông qua các ví dụ, hình ảnh minh họa và mã nguồn cụ thể. Khi hoàn thành chương này, bạn sẽ sở hữu những kỹ năng cần thiết để sử dụng ngữ cảnh Spring cũng như cấu hình các bean và mối quan hệ giữa chúng. Kỹ năng này chính là viên gạch nền móng của việc lập trình với Spring; bạn sẽ không thể tìm thấy bất kỳ một ứng dụng Spring nào trong thực tế mà lại không áp dụng những phương pháp tiếp cận được thảo luận trong chương này. Vì lý do đó, mọi nội dung trong cuốn sách này (và bất kỳ điều gì bạn học được từ các cuốn sách, bài viết hay video hướng dẫn khác) đều dựa trên việc thấu hiểu thấu đáo những phương pháp mà chúng ta nghiên cứu từ Chương 2 đến Chương 5.

Ở Chương 2, bạn đã học cách dùng chú thích `@Bean` để thêm bean vào ngữ cảnh Spring. Trong mục 3.1, chúng ta bắt đầu bằng việc thiết lập mối quan hệ giữa hai bean được định nghĩa trong lớp cấu hình bằng chú thích `@Bean`. Tại đây, chúng ta sẽ thảo luận hai cách:

- Liên kết các bean bằng cách gọi trực tiếp các phương thức tạo ra chúng (chúng ta sẽ gọi là **nối dây** - *wiring*).
- Cho phép Spring tự động cung cấp một giá trị thông qua tham số của phương thức (chúng ta sẽ gọi là **nối dây tự động** - *auto-wiring*).

Tiếp đó, trong mục 3.2, chúng ta thảo luận cách tiếp cận thứ ba — một kỹ thuật bắt nguồn từ nguyên lý IoC: **tiêm phụ thuộc** (*Dependency Injection - DI*). Chúng ta sẽ tìm hiểu cách áp dụng DI trong Spring, sử dụng chú thích `@Autowired` để thiết lập mối quan hệ giữa hai bean (đây cũng là một dạng nối dây tự động). Bạn sẽ kết hợp cả hai cách tiếp cận này trong các dự án thực tế.

Lưu ý về các ví dụ học tập ban đầu

Bạn có thể nghĩ rằng các ví dụ trong Chương 2 và Chương 3 chưa đủ gần gũi với mã nguồn thực tế của môi trường sản xuất. Suy cho cùng, các ứng dụng thật đâu có quản lý vẹt và người! Tuy nhiên, tôi muốn bắt đầu một cách êm ái với các ví dụ trực quan, đơn giản nhất để đảm bảo bạn tập trung tối đa vào các cú pháp cốt lõi — những thứ bạn sẽ gặp trong hầu như mọi ứng dụng Spring. Bằng cách này, tôi đảm bảo bạn hiểu rõ bản chất cách thức vận hành của từng giải pháp. Bắt đầu từ Chương 4, thiết kế lớp của chúng ta sẽ tiến dần tới các bài toán thực tế trong doanh nghiệp.

3.1 Thiết lập mối quan hệ giữa các bean trong tệp cấu hình

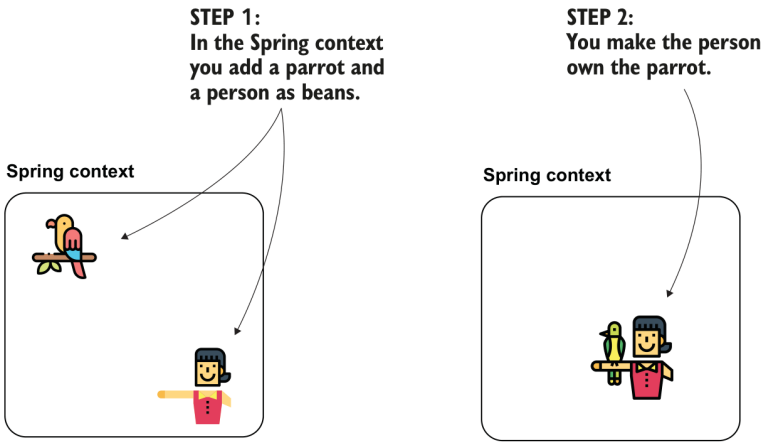
Trong mục này, bạn sẽ học cách thiết lập mối quan hệ giữa hai bean được định nghĩa trong lớp cấu hình thông qua các phương thức đánh dấu chú thích `@Bean`. Bạn sẽ thường xuyên bắt gặp phương pháp này khi cần thiết lập liên kết giữa các bean mà bạn không thể can thiệp vào mã nguồn gốc của lớp (chẳng hạn như các lớp thuộc JDK hoặc thư viện bên ngoài).

Giả sử chúng ta có hai thể hiện trong ngữ cảnh Spring: một chú vẹt (`Parrot`) và một con người (`Person`). Chúng ta muốn người sở hữu chú vẹt này. Nói cách khác, chúng ta cần liên kết hai thể hiện lại với nhau.

Đối với mỗi phương pháp (nối dây trực tiếp hoặc nối dây tự động), chúng ta đều trải qua hai bước (Hình 3.1):

1. Thêm các bean `Person` và `Parrot` vào ngữ cảnh Spring (như đã học ở Chương 2).
2. Thiết lập mối quan hệ giữa thể hiện `Person` và `Parrot`.

Hình 3.2 biểu diễn mối quan hệ phụ thuộc "has-A" (Người có một chú vẹt) theo phong cách kỹ thuật.



Hình 3.1: Có hai bean trong ngữ cảnh Spring, chúng ta muốn thiết lập mối quan hệ giữa chúng để đối tượng này có thể ủy thác công việc cho đối tượng kia. Bạn có thể làm điều này bằng cách nối dây trực tiếp (gọi hàm tạo bean) hoặc nối dây tự động thông qua tính năng tiêm phụ thuộc của framework.

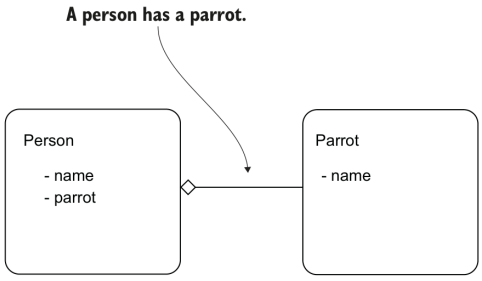


Figure 3.2 Implementing the relationship between the beans. This is a simplified diagram representing the "has-A" relationship between the `Person` and the `Parrot` objects. We'll implement this relationship through wiring and auto-wiring.

this project ready, we change it to establish the relationship between the two instances

Hình 3.2: Biểu đồ đơn giản hóa mô tả mối quan hệ "has-A" giữa đối tượng Person và Parrot. Một Person nắm giữ thuộc tính kiểu Parrot.

Trước tiên, hãy bắt đầu với dự án cơ sở (`sq-ch3-ex1`) để nhớ lại cách thêm bean bằng phương thức `@Bean` trong lớp cấu hình (Bước 1). Sau khi hoàn thành khung dự án này, chúng ta sẽ sửa đổi nó để thiết lập liên kết giữa hai thể hiện (Bước 2).

Trong tệp `pom.xml`, chúng ta thêm phụ thuộc cho `spring-context`:

```
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>5.2.7.RELEASE</version>
</dependency>
```

Listing 3.1: Khai báo phụ thuộc spring-context trong pom.xml

Sau đó, chúng ta định nghĩa lớp `Parrot` và lớp `Person`:

```
public class Parrot {
  private String name;

  // Cac getter va setter duoc luoc bo
  @Override
  public String toString() {
    return "Parrot : " + name;
  }
}
```

Listing 3.2: Định nghĩa lớp Parrot

```
public class Person {
  private String name;
  private Parrot parrot;

  // Cac getter va setter duoc luoc bo
}
```

Listing 3.3: Định nghĩa lớp Person

Đoạn mã 3.4 minh họa cách định nghĩa hai bean trong lớp cấu hình sử dụng chú thích `@Bean`.

```
@Configuration
public class ProjectConfig {

  @Bean
  public Parrot parrot() {
    Parrot p = new Parrot();
    p.setName("Koko");
    return p;
  }

  @Bean
```

```
  public Person person() {
    Person p = new Person();
    p.setName("Ella");
    return p;
  }
}
```

Listing 3.4: Định nghĩa các bean Person và Parrot trong lớp cấu hình

Bây giờ bạn có thể viết lớp `Main` như trong Đoạn mã 3.5 để kiểm tra xem hai thể hiện lúc này đã được liên kết với nhau hay chưa.

```
public class Main {
  public static void main(String[] args) {
    var context = new AnnotationConfigApplicationContext
      (ProjectConfig.class);

    Person person =
      context.getBean(Person.class);

    Parrot parrot =
      context.getBean(Parrot.class);

    System.out.println(
      "Person's name: " + person.getName());
    System.out.println(
      "Parrot's name: " + parrot.getName());
    System.out.println(
      "Person's parrot: " + person.getParrot());
  }
}
```

Listing 3.5: Lớp Main kiểm tra trạng thái khởi tạo ban đầu

Khi chạy ứng dụng, kết quả in ra màn hình console như sau:

```
Person's name: Ella
Parrot's name: Koko
Person's parrot: null
```

Điều quan trọng nhất cần nhận thấy là thuộc tính `parrot` của người (dòng thứ 3) đang có giá trị là `null`. Cả hai thể hiện `Person` và `Parrot` đều đã nằm trong ngữ cảnh, nhưng do chưa có mối quan hệ nào được thiết lập giữa chúng, nên giá trị liên kết vẫn là `null` (Hình 3.3).

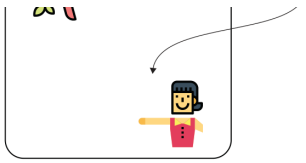


Figure 3.3 We added the two beans in the context to configure the relationship between them further.

Wiring the beans using a direct method call between the @Bean methods

In this section, we establish the relationship between the two instances of `Person` and `Parrot`. The first way (wiring) to achieve this is to call one method from another in the configuration class. You'll find this often used because it's a straightforward approach. In the next listing, you find the small change I had to make in my configuration class to establish a link between the person and the parrot (see figure 3.4). To keep all the steps separate and help you easier understand the code, I have also sepa-

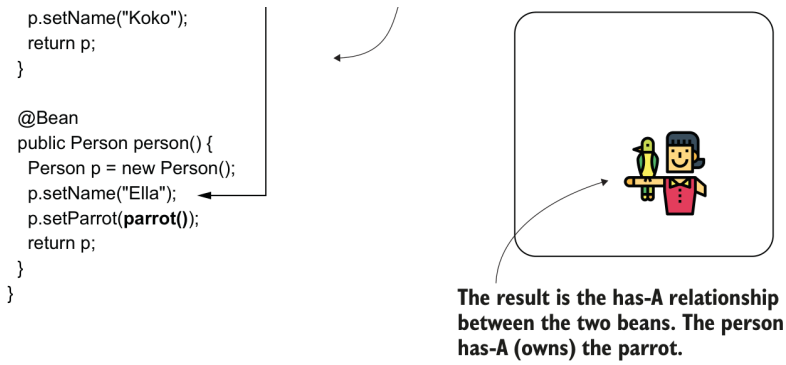
Hình 3.3: Hai bean Person và Parrot đều đã nằm trong ngữ cảnh Spring, nhưng chưa có liên kết nào được thiết lập giữa chúng.

3.1.1 Nối dây các bean bằng cách gọi trực tiếp phương thức @Bean

Trong tiểu mục này, chúng ta thiết lập mối quan hệ giữa hai thể hiện `Person` và `Parrot`. Cách thứ nhất (nối dây trực tiếp) là gọi trực tiếp phương thức tạo bean này từ phương thức tạo bean kia trong lớp cấu hình. Trong Đoạn mã 3.6, bạn sẽ thấy sự thay đổi nhỏ trong lớp cấu hình để gắn kết chú vẹt vào người (Hình 3.4).

```
@Configuration
public class ProjectConfig {

  @Bean
  public Parrot parrot() {
    Parrot p = new Parrot();
    p.setName("Koko");
    return p;
  }

  @Bean
  public Person person() {
    Person p = new Person();
    p.setName("Ella");
    p.setParrot(parrot()); // Goi truc tiep phuong thuc parrot()
    return p;
  }
}
```

Listing 3.6: Liên kết các bean bằng cách gọi trực tiếp phương thức tạo bean



Hình 3.4: Thiết lập mối quan hệ giữa các bean bằng cách nối dây trực tiếp: Phương thức person() gọi trực tiếp phương thức parrot() để lấy thể hiện và gán vào thuộc tính.

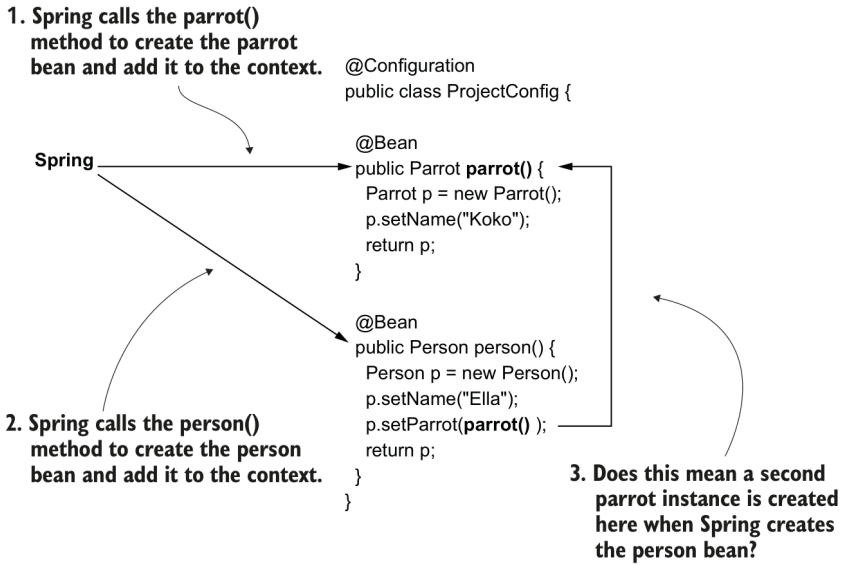
Chạy lại ứng dụng, kết quả trên màn hình console sẽ thay đổi:

```
Person's name: Ella
Person's parrot: Parrot : Koko
```

Dòng thứ hai chứng minh rằng Ella (thể hiện `Person`) hiện đã sở hữu Koko (thể hiện `Parrot`).

Khi giảng dạy phương pháp này, tôi thường nhận được câu hỏi: *Liệu cách làm này có tạo ra hai thể hiện riêng biệt của Parrot hay không (Hình 3.5) — một thể hiện do Spring gọi để đưa vào ngữ cảnh, và một thể hiện nữa khi phương thức `person()` gọi trực tiếp tới `parrot()`?*

Câu trả lời là: **Không, ứng dụng chỉ tạo duy nhất một thể hiện Parrot!**



Hình 3.5: Liệu việc person() gọi trực tiếp parrot() có dẫn tới việc hai thể hiện vẹt được tạo ra trong ứng dụng?

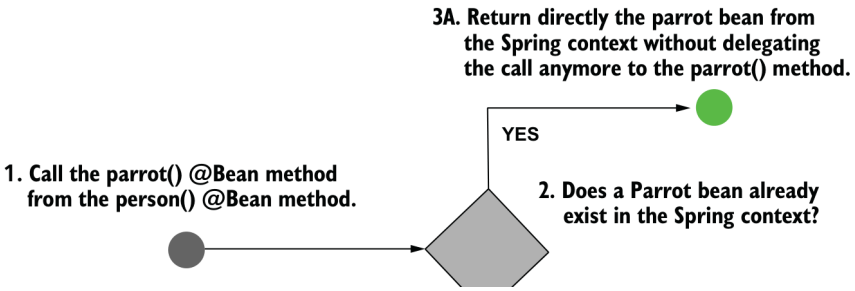
Thoạt nhìn có vẻ kỳ lạ, nhưng Spring đủ thông minh để hiểu rằng bằng việc gọi phương thức `parrot()`, ý định thực sự của bạn là muốn tham chiếu tới bean `parrot` đã tồn tại trong ngữ

cảnh của nó. Khi chúng ta sử dụng chú thích `@Bean` trong một lớp `@Configuration`, Spring sẽ kiểm soát cách thức các phương thức được gọi và áp dụng cơ chế đánh chặn (*intercepting*) lên trên các lời gọi phương thức này (chúng ta sẽ tìm hiểu sâu hơn về cơ chế proxy và đánh chặn trong Chương 6).

Cụ thể, logic vận hành của Spring diễn ra như sau (Hình 3.6):

- Nếu bean `Parrot` đã tồn tại trong ngữ cảnh, Spring sẽ trả về trực tiếp thể hiện từ ngữ cảnh mà không thực thi lại thân hàm của phương thức `parrot()`.
- Nếu bean `Parrot` chưa tồn tại trong ngữ cảnh, Spring sẽ thực thi phương thức `parrot()`, đưa giá trị trả về vào ngữ cảnh, rồi mới chuyển giao tham chiếu đó cho phương thức `person()`.

> If the parrot bean already exists in the context, then instead of calling the `parrot()` method, Spring will directly take the instance from its context. If the parrot bean does not yet exist in the context, Spring calls the `parrot()` method and returns the bean (figure 3.6).



Hình 3.6: Cơ chế đánh chặn của Spring: Nếu bean đã tồn tại trong ngữ cảnh (nhánh 3A), Spring trả về ngay thể hiện đó mà không chạy lại hàm. Nếu chưa tồn tại (nhánh 3B), Spring mới chạy hàm và đưa vào ngữ cảnh.

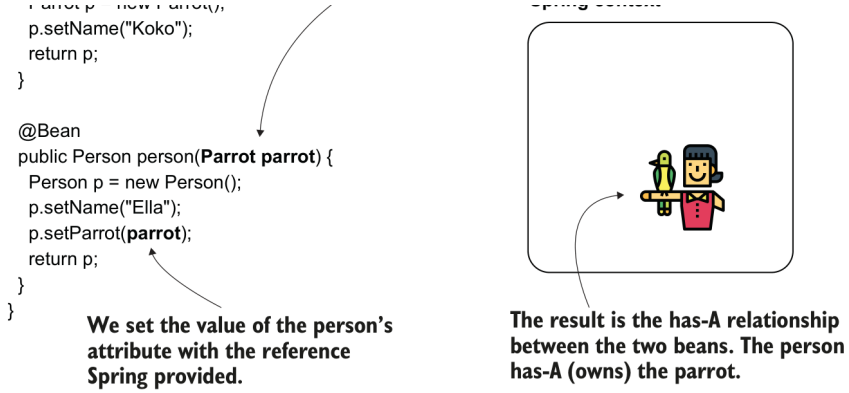
Bạn có thể dễ dàng kiểm chứng hành vi này bằng cách thêm một hàm khởi tạo không tham số vào lớp `Parrot` và in ra dòng chữ `"Parrot created"`. Khi chạy chương trình, thông báo này sẽ chỉ xuất hiện đúng một lần duy nhất:

```
Parrot created
Person's name: Ella
Person's parrot: Parrot : Koko
```

3.1.2 Nối dây các bean thông qua tham số của phương thức @Bean

Trong tiểu mục này, tôi sẽ chỉ cho bạn một phương pháp thay thế cho việc gọi hàm trực tiếp. Thay vì tự gọi phương thức định nghĩa bean, chúng ta thêm một tham số có kiểu đối tượng tương ứng vào phương thức `person()`, và để Spring tự động tiêm giá trị qua tham số đó (Hình 3.7).

Cách tiếp cận này linh hoạt hơn cách gọi hàm trực tiếp. Với phương pháp này, không quan trọng bean `Parrot` được tạo ra bằng phương thức `@Bean` hay thông qua chú thích định hình `@Component`, Spring đều tự động tìm kiếm trong ngữ cảnh và truyền vào tham số.



Hình 3.7: Khai báo tham số cho phương thức @Bean: Spring sẽ tự động tìm kiếm bean kiểu Parrot trong ngữ cảnh và tiêm vào tham số của phương thức person().

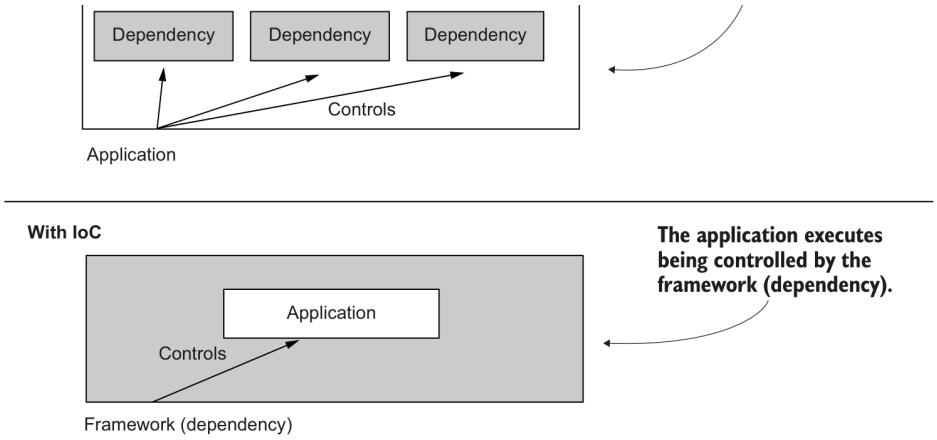
Đoạn mã 3.7 minh họa định nghĩa lớp cấu hình áp dụng phương thức này.

```
@Configuration
public class ProjectConfig {

  @Bean
  public Parrot parrot() {
    Parrot p = new Parrot();
    p.setName("Koko");
    return p;
  }

  @Bean
  public Person person(Parrot parrot) { // Spring tu dong tiem parrot
   vao day
    Person p = new Person();
    p.setName("Ella");
    p.setParrot(parrot);
    return p;
  }
}
```

Listing 3.7: Tiêm phụ thuộc thông qua tham số của phương thức @Bean

Ở đoạn trên, tôi đã sử dụng từ "tiêm" (*inject*). Từ thời điểm này trở đi, chúng ta sẽ gọi kỹ thuật này là **tiêm phụ thuộc** (*Dependency Injection - DI*). Đúng như tên gọi, DI là kỹ thuật

mà framework tự động thiết lập giá trị cho một trường hoặc tham số cụ thể, qua đó giải quyết sự phụ thuộc của phương thức hay đối tượng đó. DI là một hiện thực hóa cụ thể của nguyên lý IoC, nơi framework đóng vai trò kiểm soát luồng vận hành của ứng dụng (Hình 3.8).



Hình 3.8: Ứng dụng không dùng IoC sẽ tự điều khiển việc khởi tạo các phụ thuộc. Ứng dụng áp dụng IoC trao quyền cho framework (thư viện phụ thuộc) kiểm soát luồng thực thi và tiêm các phụ thuộc (DI) vào đối tượng.

3.2 Sử dụng chú thích @Autowired để tiêm các bean

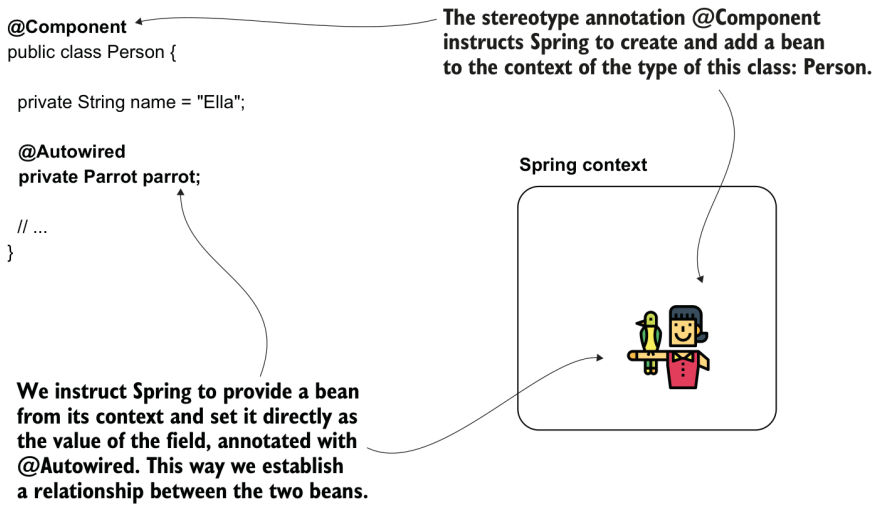
Trong mục này, chúng ta thảo luận về một kỹ thuật rất phổ biến để liên kết các bean trong ngữ cảnh Spring: sử dụng chú thích `@Autowired`. Phương pháp này áp dụng khi bạn nắm quyền kiểm soát mã nguồn của lớp. Bằng cách đặt chú thích `@Autowired`, chúng ta chỉ định vị trí mà Spring cần tiêm giá trị từ ngữ cảnh vào ngay bên trong chính lớp định nghĩa đối tượng.

Có 3 cách thức sử dụng chú thích `@Autowired`:

- **Tiêm qua trường của lớp (*field injection*):** Thường gặp trong các ví dụ nhanh hoặc kiểm thử.
- **Tiêm qua hàm khởi tạo (*constructor injection*):** Cách tiếp cận chuẩn tắc, được khuyến nghị sử dụng nhiều nhất trong môi trường thực tế.
- **Tiêm qua phương thức setter (*setter injection*):** Ít khi được áp dụng trong mã nguồn hiện đại.

3.2.1 Tiêm giá trị qua trường của lớp (Field Injection)

Đây là cách sử dụng đơn giản nhất của `@Autowired`: đặt chú thích ngay trên trường dữ liệu của lớp (Hình 3.9). Mặc dù cách làm này rất nhanh gọn, nhưng nó tiềm ẩn những nhược điểm khiến chúng ta tránh dùng trong mã nguồn sản xuất (*production*). Tuy nhiên, bạn sẽ thấy nó xuất hiện nhiều trong các bài kiểm thử hoặc mã thử nghiệm (PoC).



Hình 3.9: Tiêm qua trường: Đặt @Autowired trên trường parrot của lớp Person để yêu cầu Spring tự động tìm kiếm và gán giá trị từ ngữ cảnh.

Chúng ta sử dụng chú thích định hình `@Component` cho cả hai lớp `Parrot` và `Person`:

```
@Component
public class Parrot {
  private String name = "Koko";
  // Getters, setters va toString() duoc luoc bo
}
```

Listing 3.8: Lớp Parrot với chú thích @Component

```
@Component
public class Person {
  private String name = "Ella";

  @Autowired
  private Parrot parrot;

  // Getters va setters duoc luoc bo
}
```

Listing 3.9: Lớp Person sử dụng @Autowired trên trường dữ liệu

Lớp cấu hình chỉ cần khai báo quét gói với chú thích `@ComponentScan`:

```
@Configuration
@ComponentScan(basePackages = "beans")
public class ProjectConfig {
}
```

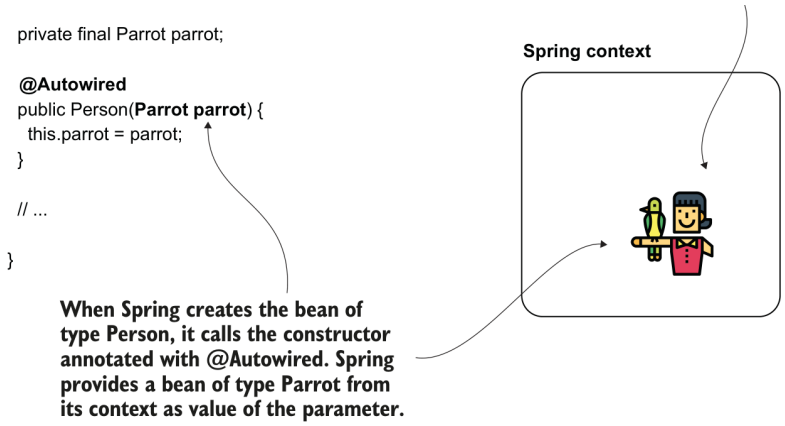
Listing 3.10: Lớp cấu hình quét các thành phần trong gói beans

Tại sao nên tránh tiêm qua trường trong môi trường sản xuất?

- Bạn không thể khai báo trường dữ liệu là `final`. Việc không thể dùng `final` đồng nghĩa với việc bạn không thể đảm bảo tính bất biến (*immutability*) của đối tượng sau khi khởi tạo, khiến trường đó có nguy cơ bị gán lại ngoài ý muốn.
- Gây khó khăn cho việc viết kiểm thử đơn vị (*unit testing*), bởi vì bạn không thể truyền trực tiếp đối tượng giả lập (*mock*) qua hàm khởi tạo mà phải dựa vào phản chiếu (*reflection*) hoặc nạp toàn bộ ngữ cảnh Spring.

3.2.2 Tiêm giá trị qua hàm khởi tạo (Constructor Injection)

Lựa chọn thứ hai là tiêm phụ thuộc thông qua hàm khởi tạo của lớp (Hình 3.10). Đây là phương pháp tối ưu nhất và được khuyến nghị sử dụng rộng rãi. Nó cho phép bạn khai báo các trường là `final`, đảm bảo an toàn tuyệt đối và tạo điều kiện thuận lợi nhất cho việc kiểm thử đơn vị.



Hình 3.10: Tiêm qua hàm khởi tạo: Khi tạo bean Person, Spring gọi hàm khởi tạo có đánh dấu @Autowired và truyền thể hiện Parrot từ ngữ cảnh vào tham số.

Đoạn mã 3.11 minh họa cách triển khai tiêm qua hàm khởi tạo trong lớp `Person`.

```
@Component
public class Person {
  private String name = "Ella";
  private final Parrot parrot; // Truong duoc khai bao final an toan

  @Autowired
  public Person(Parrot parrot) {
    this.parrot = parrot;
  }

  // Getters va setters duoc luoc bo
```

```
}
```

Listing 3.11: Tiêm giá trị phụ thuộc thông qua hàm khởi tạo

Quy tắc Spring 4.3 trở lên

Bắt đầu từ phiên bản Spring 4.3, nếu lớp của bạn chỉ có duy nhất một hàm khởi tạo, bạn hoàn toàn có thể lược bỏ chú thích `@Autowired`. Spring sẽ tự động hiểu và sử dụng hàm khởi tạo đó để tiêm phụ thuộc.

3.2.3 Tiêm phụ thuộc qua phương thức setter (Setter Injection)

Bạn cũng có thể đặt chú thích `@Autowired` trên phương thức setter của thuộc tính:

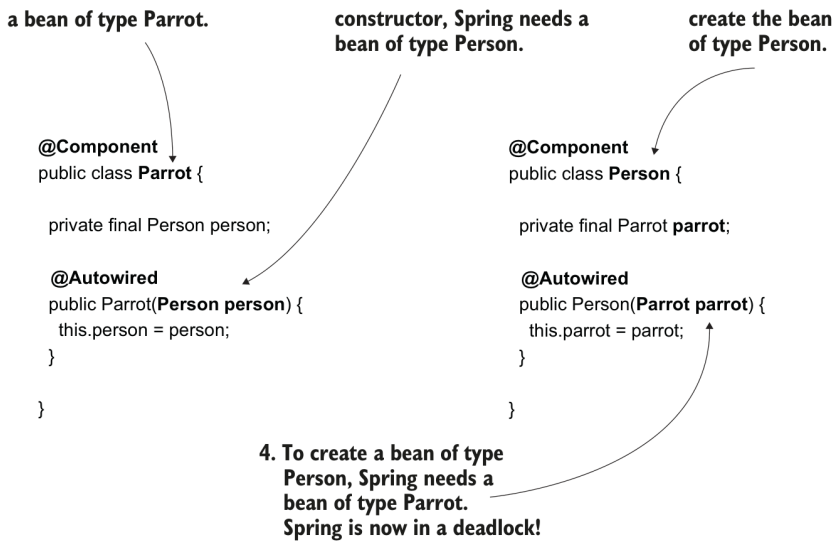
```
@Component
public class Person {
  private String name = "Ella";
  private Parrot parrot;

  @Autowired
  public void setParrot(Parrot parrot) {
    this.parrot = parrot;
  }
}
```

Listing 3.12: Tiêm phụ thuộc thông qua phương thức setter

Cách này ít khi được áp dụng vì nó không hỗ trợ trường `final` và làm mã nguồn trở nên rườm rà hơn.

3.3 Xử lý phụ thuộc vòng (Circular Dependencies)

Mặc dù việc để Spring tự động quản lý phụ thuộc mang lại sự tiện lợi to lớn, nhưng đôi khi bạn có thể vô tình tạo ra tình huống **phụ thuộc vòng** (*circular dependency*) (Hình 3.11).



Hình 3.11: Phụ thuộc vòng: Để tạo Parrot, Spring cần có Person. Nhưng để tạo Person, Spring lại cần có Parrot. Spring rơi vào tình trạng bế tắc (deadlock).

Tình huống này xảy ra khi để tạo bean A, Spring cần tiêm bean B; nhưng để tạo bean B, Spring lại cần tiêm bean A. Lúc này Spring rơi vào bế tắc và sẽ ném ra ngoại lệ:

```
org.springframework.beans.factory.BeanCurrentlyInCreationException:
Error creating bean with name 'parrot': Requested bean is currently in creation:
Is there an unresolvable circular reference?
```

Phụ thuộc vòng phản ánh một thiết kế lớp chưa hợp lý. Khi gặp lỗi này, giải pháp căn cơ nhất là bạn phải tái cấu trúc lại thiết kế để triệt tiêu sự phụ thuộc chéo giữa hai đối tượng.

3.4 Lựa chọn trong số nhiều bean cùng kiểu trong ngữ cảnh

Điều gì sẽ xảy ra nếu ngữ cảnh Spring có nhiều bean cùng một kiểu dữ liệu (chẳng hạn có 3 bean `Parrot`), và bạn yêu cầu Spring tiêm một thể hiện kiểu `Parrot` vào một tham số hoặc trường?

Spring sẽ xử lý theo các quy tắc sau:

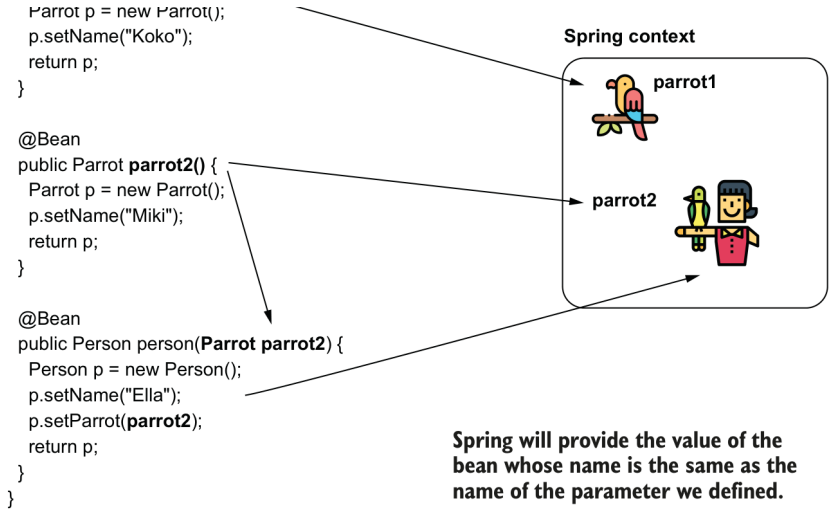
1. Nếu tên của tham số trùng khớp với tên định danh của một trong các bean trong ngữ cảnh, Spring sẽ chọn bean có tên tương ứng (Hình 3.12).
2. Nếu tên tham số không trùng với tên của bất kỳ bean nào:
    - Nếu có một bean được đánh dấu `@Primary`, Spring sẽ chọn bean đó.
    - Nếu bạn sử dụng chú thích `@Qualifier` để chỉ định rõ tên bean cần tiêm, Spring sẽ chọn bean được chỉ định.

- Nếu không thỏa mãn hai điều kiện trên, ứng dụng sẽ dừng lại và ném ra ngoại lệ thông báo không thể xác định bean cần dùng.

Đoạn mã 3.13 minh họa trường hợp tên tham số trùng với tên của bean `parrot2`.

```
@Configuration
public class ProjectConfig {

  @Bean
  public Parrot parrot1() {
    Parrot p = new Parrot();
    p.setName("Koko");
    return p;
  }

  @Bean
  public Parrot parrot2() {
    Parrot p = new Parrot();
    p.setName("Miki");
    return p;
  }

  @Bean
  public Person person(Parrot parrot2) { // Ten tham so trung voi bean
   parrot2
    Person p = new Person();
    p.setName("Ella");
    p.setParrot(parrot2);
    return p;
  }
}
```

Listing 3.13: Tiêm phụ thuộc khi tên tham số khớp với tên bean



Hình 3.12: Spring chọn bean có tên định danh trùng khớp với tên tham số khi có nhiều bean cùng kiểu trong ngữ cảnh.

Trong thực tế, việc dựa vào tên tham số có thể tiềm ẩn rủi ro nếu một lập trình viên khác vô tình đổi tên biến trong quá trình tái cấu trúc (*refactor*). Do đó, cách làm an toàn và minh bạch nhất là sử dụng chú thích `@Qualifier` (Đoạn mã 3.14).

```
@Configuration
public class ProjectConfig {

  @Bean
  public Parrot parrot1() {
    Parrot p = new Parrot();
    p.setName("Koko");
    return p;
  }

  @Bean
  public Parrot parrot2() {
    Parrot p = new Parrot();
    p.setName("Miki");
    return p;
  }

  @Bean
  public Person person(@Qualifier("parrot2") Parrot parrot) {
    Person p = new Person();
    p.setName("Ella");
    p.setParrot(parrot);
    return p;
```

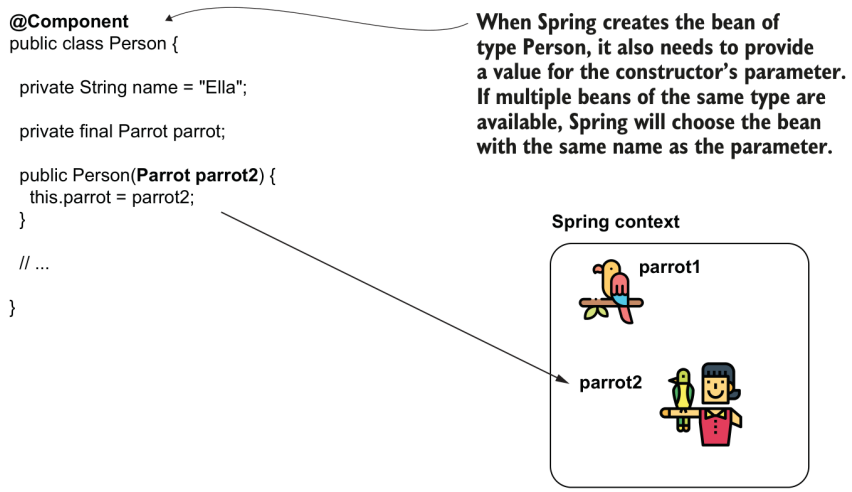
```
  }
}
```

Listing 3.14: Sử dụng chú thích @Qualifier để chỉ định đích danh bean cần tiêm

Bạn cũng có thể kết hợp `@Qualifier` với hàm khởi tạo khi sử dụng các chú thích định hình (Hình 3.13):

```
@Component
public class Person {
  private String name = "Ella";
  private final Parrot parrot;

  public Person(@Qualifier("parrot2") Parrot parrot) {
    this.parrot = parrot;
  }

  // Getters va setters duoc luoc bo
}
```

Listing 3.15: Kết hợp @Qualifier với hàm khởi tạo trong lớp Person



Hình 3.13: Khi có nhiều bean cùng kiểu trong ngữ cảnh, việc sử dụng @Qualifier giúp biểu đạt rõ ràng và an toàn ý định tiêm bean cụ thể.

Tóm tắt chương

- Ngữ cảnh Spring là nơi quản lý các đối tượng của ứng dụng. Để các đối tượng có thể cộng tác và ủy quyền thực thi cho nhau, chúng ta cần thiết lập các mối quan hệ liên kết giữa các bean trong ngữ cảnh.

- Bạn có thể thiết lập mối quan hệ giữa hai bean theo 3 cách:
  - **Gọi trực tiếp phương thức `@Bean`:** Spring sẽ chặn lời gọi và tái sử dụng thể hiện bean đã có trong ngữ cảnh thay vì tạo mới.
  - **Khai báo tham số trong phương thức `@Bean`:** Spring tự động tìm kiếm bean thích hợp trong ngữ cảnh để truyền vào tham số.
  - **Sử dụng chú thích `@Autowired`:** Áp dụng cho các lớp định nghĩa bằng chú thích định hình (`@Component`). Có thể tiêm qua trường, qua phương thức setter, hoặc tối ưu nhất là qua hàm khởi tạo (*constructor injection*).
- Bất cứ khi nào bạn để Spring cung cấp giá trị cho thuộc tính hoặc tham số, chúng ta nói rằng Spring đang sử dụng kỹ thuật **Tiêm phụ thuộc (DI)** — một ứng dụng cụ thể của nguyên lý IoC.
- Phụ thuộc vòng (*circular dependency*) xảy ra khi hai bean phụ thuộc lẫn nhau trong quá trình khởi tạo, dẫn tới bế tắc và ném ra ngoại lệ `BeanCurrentlyInCreationException`. Cần tránh điều này bằng cách thiết kế lại lớp.
- Khi có nhiều bean cùng kiểu trong ngữ cảnh, bạn có thể chỉ định bean cần tiêm bằng cách dùng chú thích `@Primary` hoặc dùng chú thích `@Qualifier` kèm tên định danh của bean.

Chương 4

Ngữ cảnh Spring: Sử dụng các tầng trừu tượng

Trong chương này, chúng ta sẽ thảo luận về việc sử dụng tính trừu tượng (*abstraction*) cùng với các bean của Spring. Chủ đề này mang tính sống còn bởi vì trong các dự án thực tế của doanh nghiệp, chúng ta thường xuyên sử dụng các tầng trừu tượng để tách rời (*decouple*) các cài đặt cụ thể. Như bạn sẽ học được trong chương này, việc tách rời các cài đặt sẽ đảm bảo ứng dụng của chúng ta luôn dễ dàng bảo trì và thuận tiện cho việc viết kiểm thử.

Chúng ta sẽ mở đầu bằng việc ôn lại cách sử dụng các giao diện (*interface*) để định nghĩa các bản hợp đồng (*contracts*) trong Mục 4.1. Để tiếp cận chủ đề này, trước tiên chúng ta bàn về trách nhiệm của các đối tượng và tìm hiểu xem chúng ăn khớp như thế nào trong một thiết kế lớp tiêu chuẩn của một ứng dụng. Chúng ta sẽ sử dụng các kỹ năng lập trình hướng đối tượng thuần túy để thiết kế một kịch bản thực tế đại diện cho một trường hợp sử dụng (*use case*) kinh điển trong phần mềm doanh nghiệp.

Nội dung trọng tâm của chương này

- Sử dụng các giao diện (*interface*) để phân tách sự phụ thuộc giữa các đối tượng.
- Ứng dụng kỹ thuật tiêm phụ thuộc (*Dependency Injection - DI*) trên các tầng trừu tượng.
- Đưa ra quyết định chính xác xem đối tượng nào nên và không nên đưa vào ngữ cảnh Spring.
- Xử lý linh hoạt tình huống có nhiều bean cùng hiện thực một tầng trừu tượng bằng các chú thích `@Primary` và `@Qualifier`.
- Làm nổi bật trách nhiệm và vai trò của các đối tượng thông qua các chú thích định hình (*stereotype annotations*): `@Service` và `@Repository`.

Tiếp đó, chúng ta sẽ đi sâu vào hành vi của Spring khi áp dụng kỹ thuật tiêm phụ thuộc (DI) với các tầng trừu tượng trong Mục 4.2. Chúng ta sẽ xuất phát từ chính mã nguồn thuần Java

đã xây dựng ở Mục 4.1, sau đó bổ sung Spring vào danh sách phụ thuộc của dự án. Kế đến, chúng ta dùng ngữ cảnh Spring để hiện thực hóa kỹ thuật tiêm phụ thuộc. Thông qua ví dụ này, bạn sẽ tiến gần hơn tới bức tranh thực tế của các sản phẩm triển khai trên môi trường sản xuất (*production*): những đối tượng mang trọng trách nghiệp vụ điển hình, kết hợp nhịp nhàng giữa tính trừu tượng, kỹ thuật DI và vùng chứa ngữ cảnh Spring.

Cuối cùng, trong Mục 4.3, chúng ta sẽ bàn luận về các chú thích định hình mang ngữ nghĩa chuyên biệt: `@Service` và `@Repository`. Bạn sẽ hiểu được lý do tại sao các lập trình viên chuyên nghiệp lại ưu tiên sử dụng các chú thích này thay vì chỉ dùng chú thích chung chung `@Component`.

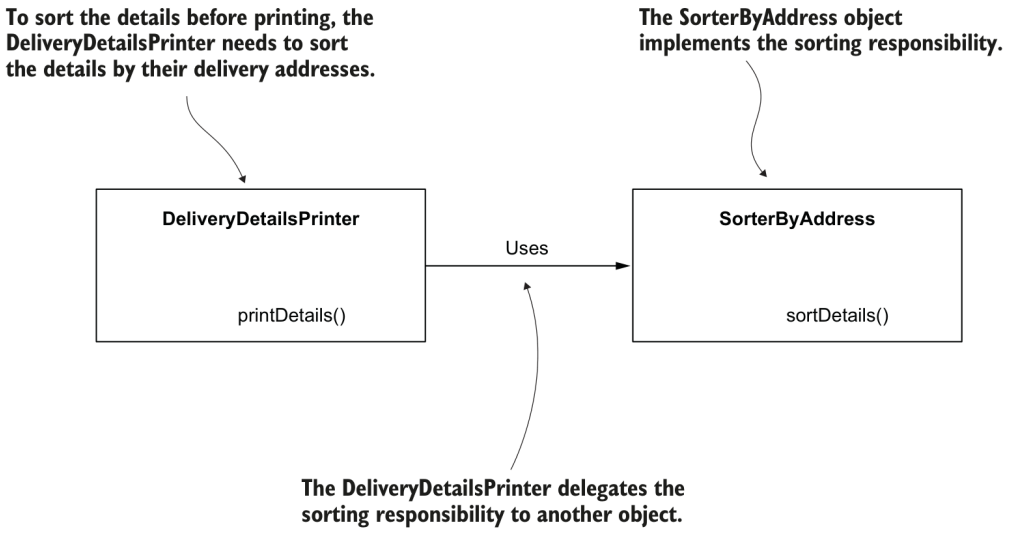
4.1 Sử dụng interface để định nghĩa các hợp đồng

Trong mục này, chúng ta sẽ thảo luận về việc sử dụng các interface để định nghĩa các bản hợp đồng hợp tác giữa các thành phần. Interface là một công cụ mạnh mẽ trong các ngôn ngữ hướng đối tượng như Java, cho phép chúng ta phân tách sự phụ thuộc giữa các lớp. Sau phần phân tích lý thuyết ngắn gọn, chúng ta sẽ cùng nhau hiện thực hóa một kịch bản ứng dụng hoàn chỉnh bằng Java thuần để bạn thấy rõ cách các đối tượng phối hợp với nhau thông qua interface trước khi đưa framework Spring vào quản lý.

4.1.1 Sử dụng interface để tách rời các cài đặt cụ thể

Khi thiết kế các lớp trong một ứng dụng, một nguyên tắc vàng là: **đối tượng sử dụng chỉ nên biết đối tượng được sử dụng LÀM CÁI GÌ (what), chứ không nên bị ràng buộc vào việc nó LÀM NHƯ THẾ NÀO (how)**.

Hãy xem xét một ví dụ minh họa: Giả sử bạn có một đối tượng phụ trách in ấn thông tin chi tiết của các đơn hàng vận chuyển (`DeliveryDetailsPrinter`). Các đơn hàng này cần được sắp xếp theo địa chỉ nơi giao trước khi in ra. Đối tượng phụ trách in ấn sẽ ủy thác công việc sắp xếp đó cho một đối tượng khác có tên là `SorterByAddress` (Hình 4.1).



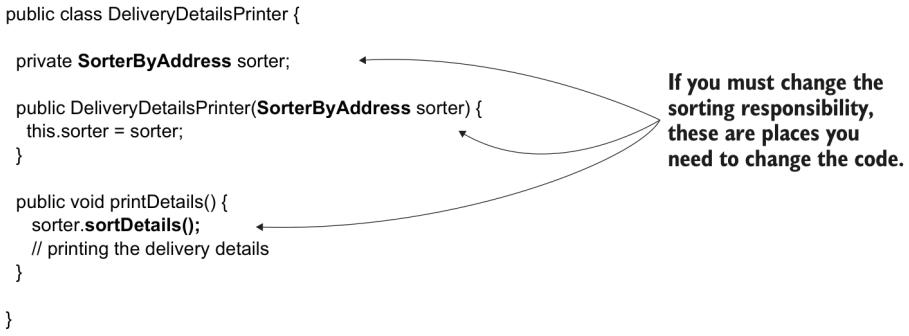
Hình 4.1: Đối tượng `DeliveryDetailsPrinter` ủy thác trách nhiệm sắp xếp các chi tiết giao hàng theo địa chỉ cho đối tượng `SorterByAddress`.

Như được thể hiện trên Hình 4.1, lớp `DeliveryDetailsPrinter` trực tiếp ủy thác trách nhiệm sắp xếp cho đối tượng `SorterByAddress`. Nếu chúng ta giữ nguyên thiết kế lớp này, chúng ta sẽ phải đối mặt với khó khăn lớn sau này mỗi khi có yêu cầu thay đổi chức năng. Hãy tưởng tượng rằng một ngày nào đó, khách hàng yêu cầu thay đổi thứ tự in ấn: thay vì sắp xếp theo địa chỉ giao hàng, bây giờ phải sắp xếp theo tên của người gửi hàng!

Để đáp ứng yêu cầu mới này, bạn bắt buộc phải sửa đổi trực tiếp lớp `DeliveryDetailsPrinter` (Hình 4.2) bởi vì lớp này đang bị gắn chặt (*strongly coupled*) với kiểu cụ thể `SorterByAddress`.

sorted by their destination address. The object dealing with printing the details needs to delegate to some other object the responsibility of sorting the packages by their delivery addresses (figure 4.1).

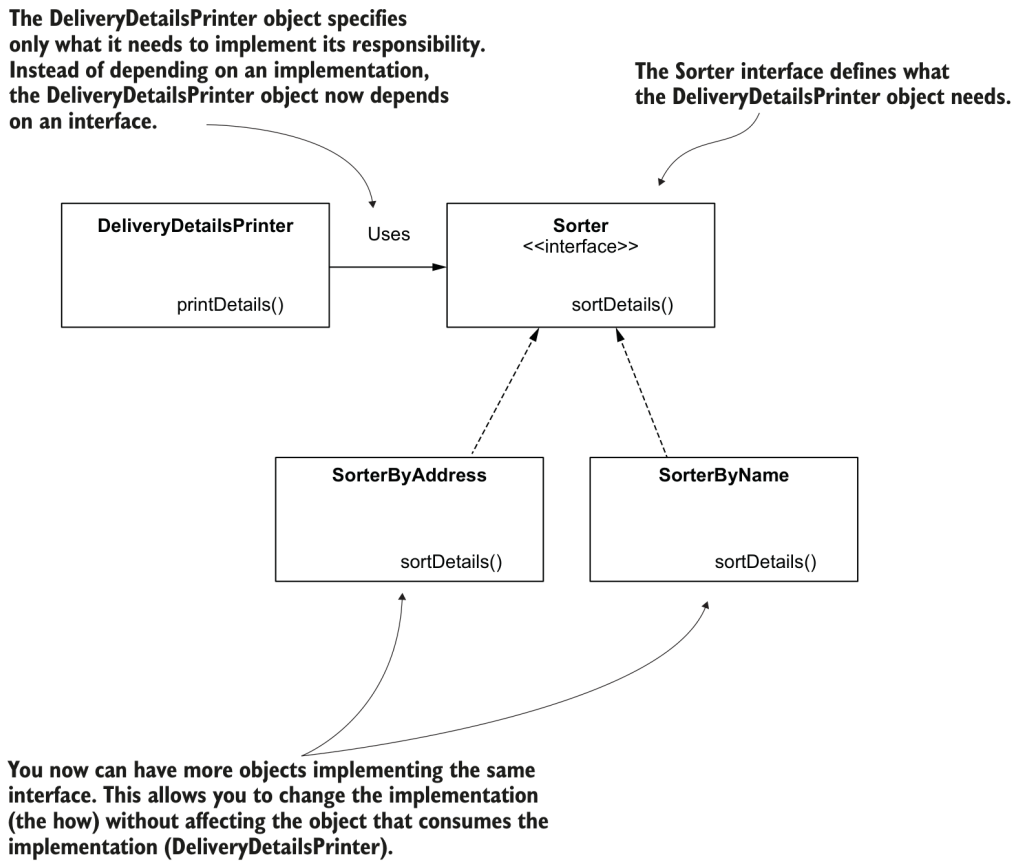
As shown in figure 4.1, the `DeliveryDetailsPrinter` directly delegates the sorting responsibility to the `SorterByAddress` object. If we keep this class design, we may face difficulties later if we need to change this functionality. Let's imagine you need to change the printed details order later, and the new order is by the sender's name. You'd need to replace the `SorterByAddress` object with another one implementing the new responsibility, but you'd also need to change the `DeliveryDetailsPrinter` object that uses the sorting responsibility (figure 4.2).



Hình 4.2: Do hai đối tượng liên kết quá chặt chẽ, nếu bạn muốn thay đổi trách nhiệm sắp xếp, bạn cũng buộc phải sửa đổi chính đối tượng đang tiêu thụ trách nhiệm đó. Một thiết kế tốt hơn phải cho phép bạn thay đổi thuật toán sắp xếp mà không cần chạm vào đối tượng sử dụng nó.

Làm thế nào để chúng ta cải tiến thiết kế này? Khi thay đổi trách nhiệm của một đối tượng, chúng ta mong muốn tránh tối đa việc phải chỉnh sửa các đối tượng khác đang sử dụng trách nhiệm đó. Lỗ hổng trong thiết kế trên xuất phát từ việc `DeliveryDetailsPrinter` vừa chỉ định nó cần **cái gì** (*what*), vừa ràng buộc luôn **cách thức thực hiện** (*how*).

Như đã phân tích, một đối tượng chỉ nên nêu rõ nhu cầu của mình mà không cần quan tâm đến cách thức cài đặt chi tiết phía sau. Giải pháp kinh điển trong lập trình hướng đối tượng chính là sử dụng **interface**. Trên Hình 4.3, tôi đưa vào một giao diện mang tên `Sorter` để tách rời sự phụ thuộc giữa hai đối tượng.



Hình 4.3: Sử dụng interface để tách rời các trách nhiệm. Thay vì phụ thuộc trực tiếp vào một cài đặt cụ thể, đối tượng `DeliveryDetailsPrinter` nay phụ thuộc vào một interface (bản hợp đồng). Giờ đây `DeliveryDetailsPrinter` có thể làm việc với bất kỳ đối tượng nào hiện thực interface này thay vì bị trói chặt vào một lớp đơn lẻ.

Thay vì khai báo biến thành viên thuộc kiểu cụ thể `SorterByAddress`, đối tượng `DeliveryDetailsPrinter` nay chỉ tuyên bố rằng nó cần một đối tượng thỏa mãn hợp đồng `Sorter`. Giờ đây, bạn có thể tạo ra bao nhiêu cài đặt tùy ý để giải quyết nhu cầu mà `DeliveryDetailsPrinter` đòi hỏi. Bất kỳ lớp nào hiện thực interface `Sorter` đều có thể đáp ứng trọn vẹn sự phụ thuộc của `DeliveryDetailsPrinter` tại bất kỳ thời điểm nào.

Định nghĩa của interface `Sorter` vô cùng súc tích:

```
public interface Sorter {
  void sortDetails();
}
```

Hãy quan sát Hình 4.4 và so sánh nó với Hình 4.2. Vì đối tượng `DeliveryDetailsPrinter` chỉ phụ thuộc vào interface thay vì cài đặt cụ thể, bạn hoàn toàn không cần phải sửa đổi lớp này thêm bất kỳ lần nào nữa mỗi khi thuật toán sắp xếp chi tiết giao hàng thay đổi trong tương lai.

In the next code snippet, you find the `Sorter` interface definition:

```
public interface Sorter {
  void sortDetails();
}
```

Look at figure 4.4 and compare it with figure 4.2. Because the `DeliveryDetails-Printer` object depends on the interface instead of the implementation directly, you don't need to change it further if you change the way the delivery details are sorted.

```
public class DeliveryDetailsPrinter {

  private Sorter sorter;

  public DeliveryDetailsPrinter(Sorter sorter) {
    this.sorter = sorter;
  }

  public void printDetails() {
    sorter.sortDetails();
    // printing the delivery details
  }

}
```

Now you can use any implementation of the Sorter interface and no longer need to change the object that uses the responsibility.

Hình 4.4: Đối tượng `DeliveryDetailsPrinter` phụ thuộc vào interface `Sorter`. Bạn có thể tự do thay đổi các cài đặt khác nhau của `Sorter` mà không gây bất kỳ tác động nào lên `DeliveryDetailsPrinter`.

Sau phần giới thiệu lý thuyết nền tảng này, bạn đã hiểu rõ tại sao chúng ta dùng interface để tách rời các đối tượng phụ thuộc lẫn nhau trong sơ đồ thiết kế lớp. Tiếp theo, chúng ta sẽ bắt tay vào hiện thực hóa một kịch bản nghiệp vụ thực tế. Chúng ta sẽ xây dựng tính năng này bằng Java thuần túy trước, chưa dùng tới bất kỳ framework nào, nhằm tập trung cao độ vào việc phân chia trách nhiệm và dùng interface để tách rời các thành phần.

4.1.2 Yêu cầu của kịch bản nghiệp vụ

Cho đến nay, chúng ta mới chỉ làm việc với các ví dụ đơn giản và các đối tượng trực quan (như chú vẹt `Parrot`). Dù chúng chưa thực sự phản ánh đúng những gì xuất hiện trong một ứng dụng doanh nghiệp thực tế, nhưng chúng đã giúp bạn tập trung tối đa vào các quy tắc cú pháp cần nắm vững. Bây giờ là thời điểm thích hợp để bạn tiến thêm một bước: vận dụng những gì đã học vào một bài toán nghiệp vụ sát với thực tế sản xuất.

Giả sử bạn đang xây dựng một ứng dụng nội bộ cho phép một nhóm làm việc quản lý các nhiệm vụ (*tasks*) của họ. Một trong những tính năng quan trọng là cho phép các thành viên để lại bình luận (*comments*) cho từng nhiệm vụ. Khi một người dùng đăng tải một bình luận mới, ứng dụng sẽ thực hiện hai công việc:

1. Lưu trữ bình luận đó vào một nơi lưu trữ bền vững (chẳng hạn như một hệ cơ sở dữ liệu).
2. Gửi một email thông báo nội dung bình luận tới một địa chỉ email cụ thể đã được cấu hình trước trong hệ thống.

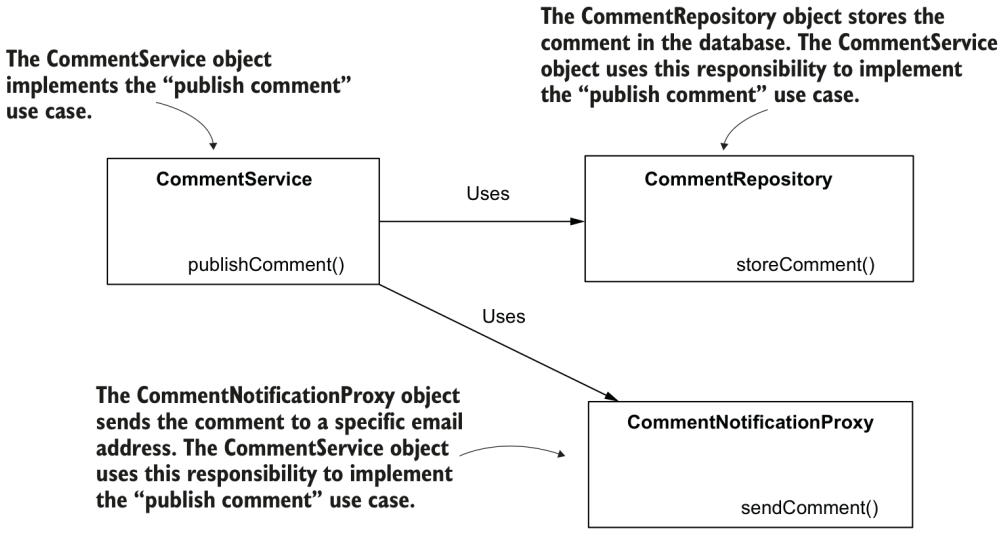
Chúng ta cần phân tích thiết kế các đối tượng, xác định đúng các vai trò trách nhiệm và các tầng trừu tượng cần thiết để hiện thực hóa tính năng này một cách tối ưu.

4.1.3 Hiện thực yêu cầu mà không sử dụng framework

Trong mục này, chúng ta tập trung triển khai yêu cầu nghiệp vụ nêu trên bằng Java thuần túy, ứng dụng những kiến thức về interface vừa ôn lại. Bước đầu tiên là nhận diện các đối tượng (các trách nhiệm) cần phải hiện thực:

- **Lớp dịch vụ (Service):** Trong các ứng dụng doanh nghiệp tiêu chuẩn, đối tượng đảm nhận việc thực thi các trường hợp sử dụng (*use cases*) thường được gọi là **service**. Chúng ta sẽ định nghĩa một đối tượng mang tên `CommentService`. Thói quen đặt tên lớp kết thúc bằng hậu tố "Service" giúp vai trò của lớp trong dự án trở nên rõ ràng và dễ nhận biết. (Để tìm hiểu sâu hơn về quy ước đặt tên chuẩn mực, tôi khuyến nghị bạn đọc Chương 2 cuốn *Clean Code: A Handbook of Agile Software Craftsmanship* của tác giả Robert C. Martin, Nhà xuất bản Pearson, 2008).
- **Kho lưu trữ (Repository):** Khi một đối tượng có nhiệm vụ tương tác trực tiếp với cơ sở dữ liệu để lưu trữ và truy xuất dữ liệu, chúng ta thường gọi đối tượng đó là **repository**. Đôi khi bạn cũng bắt gặp thuật ngữ đối tượng truy cập dữ liệu (*Data Access Object - DAO*). Chúng ta đặt tên cho đối tượng đảm nhận trách nhiệm lưu trữ bình luận là `CommentRepository`.
- **Đối tượng ủy quyền giao tiếp ngoài (Proxy):** Trong ứng dụng thực tế, khi hiện thực các đối tượng có nhiệm vụ thiết lập kết nối giao tiếp với các hệ thống bên ngoài ứng dụng (như máy chủ email, dịch vụ thông báo đẩy, các web service từ bên thứ ba), chúng ta gọi chúng là các **proxy**. Do đó, chúng ta đặt tên cho đối tượng gửi email thông báo là `CommentNotificationProxy`.

Hình 4.5 minh họa mối quan hệ phối hợp giữa ba trách nhiệm này.

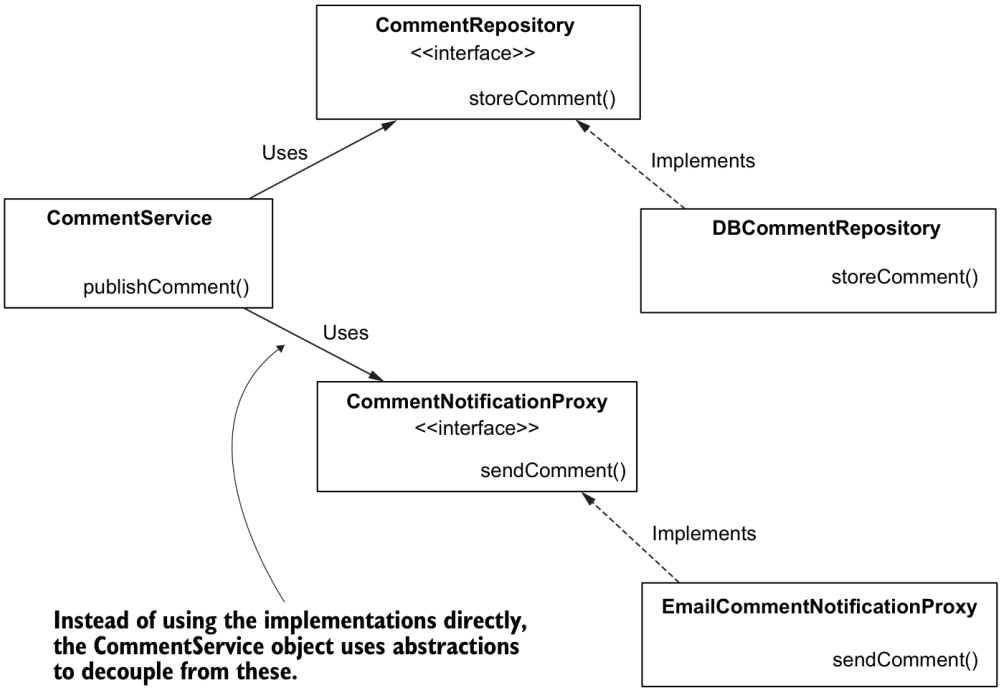


Hình 4.5: Đối tượng `CommentService` hiện thực use case "đăng bình luận". Để hoàn thành việc này, nó ủy thác công việc cho hai đối tượng mang trách nhiệm chuyên biệt: `CommentRepository` và `CommentNotificationProxy`.

Nhưng khoan đã! Chẳng phải chúng ta vừa khẳng định rằng không nên ghép nối trực tiếp giữa các cài đặt cụ thể hay sao? Chúng ta cần tách rời các cài đặt này bằng cách sử dụng các interface. Hiện tại, `CommentRepository` có thể lưu bình luận vào cơ sở dữ liệu quan hệ, nhưng trong tương lai rất có thể yêu cầu sẽ thay đổi sang lưu trữ trên NoSQL hoặc gọi một dịch vụ bên ngoài. Tương tự, `CommentNotificationProxy` hiện đang gửi thông báo qua email, nhưng phiên bản tiếp theo có thể cần gửi qua tin nhắn SMS hoặc thông báo đẩy (*push notification*).

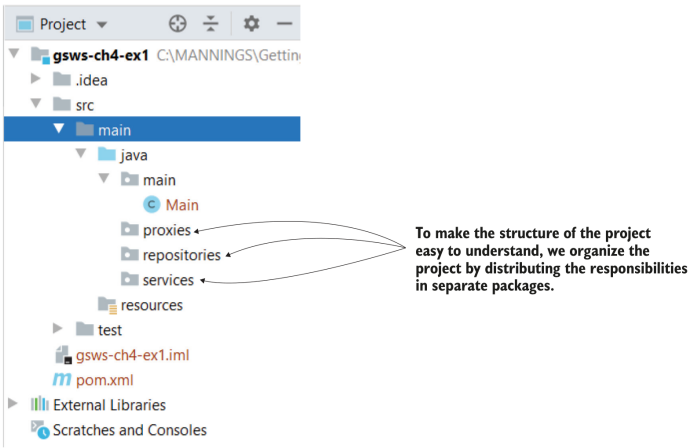
Chúng ta chắc chắn muốn tách rời `CommentService` khỏi các cài đặt chi tiết của các phụ thuộc, để khi cần thay đổi cách lưu trữ hay phương thức gửi thông báo, chúng ta hoàn toàn không cần phải sửa đổi mã nguồn của `CommentService`.

Hình 4.6 thể hiện cách thiết kế lớp tách rời nhờ các tầng trừu tượng: thay vì thiết kế `CommentRepository` và `CommentNotificationProxy` thành các lớp cụ thể, chúng ta thiết kế chúng dưới dạng các interface định nghĩa hợp đồng hành vi.



Hình 4.6: Đối tượng `CommentService` chỉ phụ thuộc vào các tầng trừu tượng do hai interface `CommentRepository` và `CommentNotificationProxy` cung cấp. Các lớp `DBCommentRepository` và `EmailCommentNotificationProxy` lần lượt hiện thực các hợp đồng này. Thiết kế này giải phóng use case "đăng bình luận" khỏi sự ràng buộc vào cài đặt cụ thể, giúp ứng dụng cực kỳ linh hoạt trước các thay đổi.

Bây giờ khi đã có bức tranh rõ ràng về những gì cần làm, hãy bắt đầu viết mã nguồn. Tạm thời, chúng ta tạo một dự án Maven thuần túy, chưa cần thêm bất kỳ thư viện bên ngoài nào vào tệp `pom.xml`. Tôi đặt tên dự án này là `sq-ch4-ex1` và tổ chức cấu trúc gói (*package*) như minh họa trên Hình 4.7, phân tách rạch ròi từng trách nhiệm vào từng package riêng biệt.



Hình 4.7: Cấu trúc dự án `sq-ch4-ex1`. Chúng ta khai báo từng package riêng cho mỗi trách nhiệm nhằm giữ cho dự án luôn trong sáng, khoa học và dễ đọc hiểu.

Một chi tiết nhỏ cần bổ sung là chúng ta cần một đối tượng biểu diễn chính bản thân bình

luận. Chúng ta chỉ cần tạo một lớp POJO (*Plain Old Java Object*) đơn giản tên là `Comment` trong package `model` như trong Đoạn mã 4.1.

```
1  package model;
2
3  public class Comment {
4    private String author;
5    private String text;
6
7    public String getAuthor() {
8      return author;
9    }
10
11   public void setAuthor(String author) {
12     this.author = author;
13   }
14
15   public String getText() {
16     return text;
17   }
18
19   public void setText(String text) {
20     this.text = text;
21   }
22 }
```

Listing 4.1: Định nghĩa lớp Comment biểu diễn dữ liệu bình luận (`model/Comment.java`)

Tiếp theo, chúng ta định nghĩa hợp đồng cho repository và proxy. Đoạn mã 4.2 hiển thị nội dung interface `CommentRepository` trong package `repositories`. Hợp đồng này công bố phương thức `storeComment(Comment comment)` mà `CommentService` cần gọi đến.

```
1  package repositories;
2
3  import model.Comment;
4
5  public interface CommentRepository {
6    void storeComment(Comment comment);
7  }
```

Listing 4.2: Định nghĩa interface CommentRepository (`repositories/CommentRepository.java`)

Interface này chỉ nêu rõ những gì cần làm mà không chứa mã thực thi. Trong Đoạn mã 4.3,

chúng ta tạo một lớp hiện thực cụ thể mang tên `DBCommentRepository` trong cùng package `repositories` để giả lập hành vi lưu trữ vào cơ sở dữ liệu.

```
package repositories;

import model.Comment;

public class DBCommentRepository implements CommentRepository {
  @Override
  public void storeComment(Comment comment) {
    System.out.println("Storing comment: " + comment.getText());
  }
}
```

Listing 4.3: Hiện thực hóa interface CommentRepository (`repositories/DBCommentRepository.java`)

Tiếp theo, chúng ta định nghĩa interface `CommentNotificationProxy` trong package `proxies` (Đoạn mã 4.4). Hợp đồng này quy định phương thức `sendComment(Comment comment)`.

```
package proxies;

import model.Comment;

public interface CommentNotificationProxy {
  void sendComment(Comment comment);
}
```

Listing 4.4: Định nghĩa interface CommentNotificationProxy (`proxies/CommentNotificationProxy.java`)

Đoạn mã 4.5 cung cấp lớp hiện thực `EmailCommentNotificationProxy` gửi thông báo bình luận qua email.

```
package proxies;

import model.Comment;

public class EmailCommentNotificationProxy
  implements CommentNotificationProxy {
  @Override
  public void sendComment(Comment comment) {
    System.out.println("Sending notification for comment: "
                       + comment.getText());
  }
```

```
}
```

Listing 4.5: Hiện thực hóa interface CommentNotificationProxy (`proxies/EmailCommentNotificationProxy.java`)

Bây giờ chúng ta đã sẵn sàng hiện thực đối tượng trung tâm: `CommentService` trong package `services` (Đoạn mã 4.6). Đối tượng này nắm giữ hai phụ thuộc dưới dạng interface (`CommentRepository` và `CommentNotificationProxy`) và nhận các thể hiện cụ thể thông qua hàm khởi tạo.

```
package services;

import model.Comment;
import proxies.CommentNotificationProxy;
import repositories.CommentRepository;

public class CommentService {
  private final CommentRepository commentRepository;
  private final CommentNotificationProxy commentNotificationProxy;

  public CommentService(
           CommentRepository commentRepository,
           CommentNotificationProxy commentNotificationProxy) {
    this.commentRepository = commentRepository;
    this.commentNotificationProxy = commentNotificationProxy;
  }

  public void publishComment(Comment comment) {
    commentRepository.storeComment(comment);
    commentNotificationProxy.sendComment(comment);
  }
}
```

Listing 4.6: Hiện thực đối tượng CommentService (`services/CommentService.java`)

Cuối cùng, chúng ta viết một lớp `Main` trong package `main` (Đoạn mã 4.7) để khởi tạo các đối tượng, kết nối chúng lại với nhau và kiểm thử use case vừa xây dựng.

```
package main;

import model.Comment;
import proxies.EmailCommentNotificationProxy;
import repositories.DBCommentRepository;
import services.CommentService;

```

```
public class Main {
  public static void main(String[] args) {
    var commentRepository =
      new DBCommentRepository();
    var commentNotificationProxy =
      new EmailCommentNotificationProxy();

    var commentService =
      new CommentService(
          commentRepository, commentNotificationProxy);

    var comment = new Comment();
    comment.setAuthor("Laurentiu");
    comment.setText("Demo comment");

    commentService.publishComment(comment);
  }
}
```

Listing 4.7: Gọi trường hợp sử dụng trong lớp Main (`main/Main.java`)

Khi chạy ứng dụng này, bạn sẽ quan sát thấy hai dòng thông báo xuất hiện trên màn hình console, được in ra lần lượt bởi đối tượng `DBCommentRepository` và `EmailCommentNotificationProxy`:

```
Storing comment: Demo comment
Sending notification for comment: Demo comment
```

4.2 Sử dụng tiêm phụ thuộc với các tầng trừu tượng

Trong mục này, chúng ta sẽ đưa framework Spring vào áp dụng cho thiết kế lớp vừa hoàn thành ở Mục 4.1. Thông qua ví dụ này, chúng ta sẽ phân tích tường tận cách thức Spring xử lý việc tiêm phụ thuộc (*Dependency Injection*) khi làm việc với các tầng trừu tượng.

Chủ đề này có tầm quan trọng sống còn, bởi trong đại đa số các dự án thực tế, các mối quan hệ phụ thuộc giữa các đối tượng đều được thiết lập thông qua các interface trừu tượng. Ở Chương 3, khi tìm hiểu về DI, chúng ta đã dùng các lớp cụ thể để khai báo các biến phụ thuộc mà mình muốn Spring gán giá trị từ ngữ cảnh. Nhưng như bạn sắp thấy, Spring hoàn toàn thấu hiểu và xử lý xuất sắc các tầng trừu tượng.

Chúng ta sẽ bắt đầu bằng việc bổ sung thư viện phụ thuộc của Spring vào tệp `pom.xml`, sau đó quyết định xem những đối tượng nào trong ứng dụng cần được Spring quản lý.

4.2.1 Quyết định đối tượng nào nên thuộc ngữ cảnh Spring

Khi thảo luận về Spring ở Chương 2 và Chương 3, chúng ta chủ yếu tập trung vào cú pháp và chưa có một use case mang hơi thở nghiệp vụ thực tế. Đó cũng là lý do vì sao trước đó chúng ta chưa đặt câu hỏi: liệu đối tượng nào nên và không nên đưa vào ngữ cảnh Spring? Từ những ví dụ đơn giản ban đầu, nhiều người có thể lầm tưởng rằng ta phải đưa tất cả mọi đối tượng của ứng dụng vào vùng chứa ngữ cảnh của Spring, nhưng thực tế hoàn toàn không phải như vậy!

Hãy nhớ lại nguyên lý cốt lõi: lý do chính để đưa một đối tượng vào ngữ cảnh Spring là **để trao quyền cho Spring kiểm soát đối tượng đó, và qua đó bồi đắp thêm cho nó những năng lực mạnh mẽ mà framework cung cấp**. Do đó, quyết định này rất giản dị và luôn dựa trên câu hỏi: *«Đối tượng này có cần framework quản lý hay không?»*

Trong kịch bản hiện tại của chúng ta, tính năng duy nhất của Spring mà ta đang sử dụng là Dependency Injection. Do đó, chúng ta chỉ cần thêm một đối tượng vào ngữ cảnh Spring nếu nó rơi vào một trong hai trường hợp:

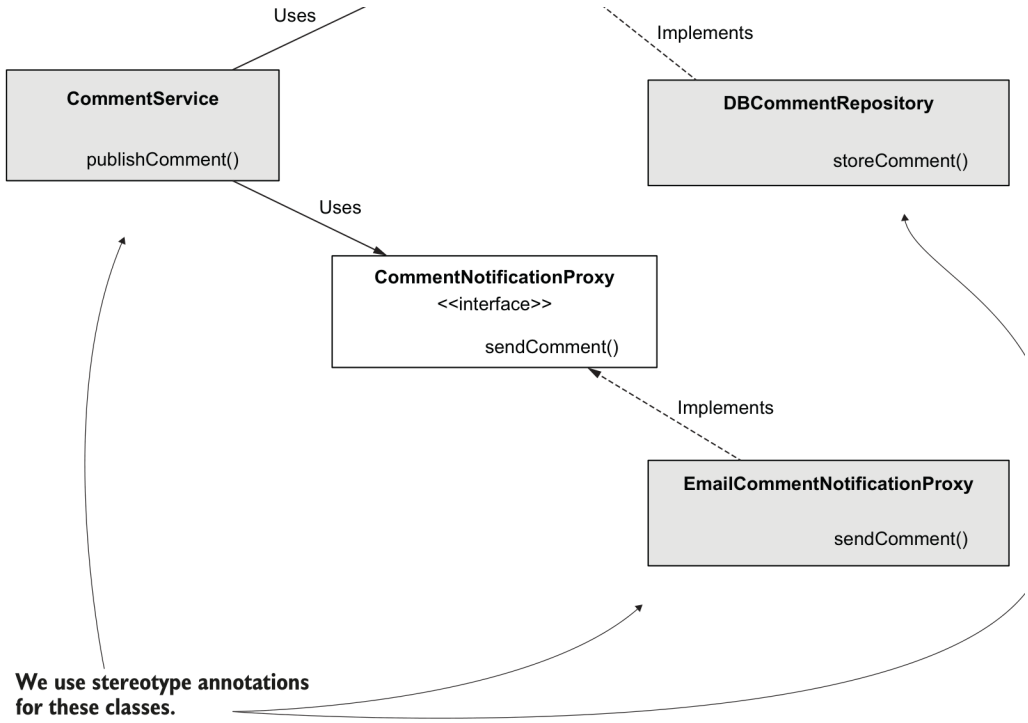
1. Bản thân nó sở hữu các phụ thuộc cần được Spring tiêm vào từ ngữ cảnh.
2. Bản thân nó chính là một đối tượng phụ thuộc cần được tiêm vào đối tượng khác.

Nhìn vào thiết kế lớp của ứng dụng, bạn sẽ nhận thấy đối tượng duy nhất không sở hữu bất kỳ phụ thuộc nào và cũng không phải là phụ thuộc của bất kỳ ai chính là đối tượng `Comment`. Ba đối tượng còn lại đều có vai trò rõ ràng:

- `CommentService`: Nắm giữ hai phụ thuộc là `CommentRepository` và `CommentNotificationProxy`.
- `DBCommentRepository`: Hiện thực interface `CommentRepository` và đóng vai trò là một phụ thuộc của `CommentService`.
- `EmailCommentNotificationProxy`: Hiện thực interface `CommentNotificationProxy` và đóng vai trò là một phụ thuộc của `CommentService`.

Nhưng tại sao chúng ta lại không đưa cả các thể hiện `Comment` vào ngữ cảnh Spring? Khi tôi giảng dạy các khóa học về Spring, học viên rất hay đặt câu hỏi này. Việc thêm những đối tượng không cần sự can thiệp của framework vào ngữ cảnh Spring sẽ tạo ra sự phức tạp dư thừa không đáng có, làm ứng dụng vừa khó bảo trì hơn, vừa tốn kém tài nguyên một cách vô ích. Khi đưa một đối tượng vào ngữ cảnh, bạn trao quyền cho Spring quản lý vòng đời và áp dụng các cơ chế can thiệp. Nếu đối tượng đó chỉ đơn thuần là dữ liệu mang tính thời điểm (*data carrier*) mà không nhận được bất kỳ lợi ích nào từ framework, việc đưa nó vào context chỉ đơn giản là một biểu hiện của thiết kế thái quá (*over-engineering*).

Ở Chương 2, chúng ta đã biết việc dùng các chú thích định hình (`@Component`) là cách thuận tiện nhất để đưa các bean vào ngữ cảnh Spring khi các lớp đó thuộc về mã nguồn dự án của chúng ta. Chúng ta sẽ áp dụng phương pháp này tại đây.



Hình 4.8: Các lớp được đánh dấu bằng chú thích `@Component` được tô màu xám. Khi ngữ cảnh được nạp, Spring sẽ khởi tạo các thể hiện của các lớp này và thêm chúng vào ngữ cảnh của nó. Các interface giữ màu trắng vì chúng ta không bao giờ đánh dấu chú thích định hình lên interface.

Hãy quan sát Hình 4.8: hai interface vẫn giữ màu trắng (chúng ta không đánh dấu `@Component` lên chúng). Rất nhiều lập trình viên mới làm quen thường bối rối về việc nên đặt chú thích định hình ở đâu khi có sự xuất hiện của interface. Nguyên tắc rất rõ ràng: chúng ta chỉ đặt chú thích định hình lên các lớp cụ thể mà ta muốn Spring tạo ra thể hiện và đưa vào ngữ cảnh. Không hề có ý nghĩa gì nếu bạn đặt chú thích định hình lên interface hoặc lớp trừu tượng (*abstract class*), vì bản thân chúng không thể được khởi tạo trực tiếp! Về mặt cú pháp compiler có thể không cấm, nhưng về mặt ngữ nghĩa thì điều đó hoàn toàn vô dụng.

Hãy bắt đầu sửa đổi mã nguồn. Tôi tạo một dự án mới mang tên `sq-ch4-ex2`. Trước tiên, ta thêm chú thích `@Component` vào lớp `DBCommentRepository` như trong Đoạn mã 4.8.

```
1  package repositories;
2
3  import model.Comment;
4  import org.springframework.stereotype.Component;
5
6  @Component
7  public class DBCommentRepository implements CommentRepository {
8    @Override
9    public void storeComment(Comment comment) {
10     System.out.println("Storing comment: " + comment.getText());
```

```
  }
}
```

Listing 4.8: Thêm chú thích @Component vào lớp DBCommentRepository

Tiếp theo, trong Đoạn mã 4.9, chúng ta bổ sung chú thích `@Component` cho lớp `EmailCommentNotification`

```
package proxies;

import model.Comment;
import org.springframework.stereotype.Component;

@Component
public class EmailCommentNotificationProxy
  implements CommentNotificationProxy {
  @Override
  public void sendComment(Comment comment) {
    System.out.println(
      "Sending notification for comment: " +
         comment.getText());
  }
}
```

Listing 4.9: Thêm chú thích @Component vào lớp EmailCommentNotificationProxy

Trong Đoạn mã 4.10, chúng ta cũng đánh dấu lớp `CommentService` bằng chú thích `@Component`. Lớp `CommentService` khai báo hai trường phụ thuộc dưới kiểu interface `CommentRepository` và `CommentNotificationProxy`.

Khi Spring thấy các thuộc tính trong constructor được định nghĩa bằng các kiểu interface, nó đủ thông minh để tự động tìm kiếm trong ngữ cảnh của nó các bean được tạo ra từ những lớp có **hiện thực các interface tương ứng**. Và như đã học ở Chương 2 và Chương 3, vì lớp này chỉ có duy nhất một hàm khởi tạo, chú thích `@Autowired` ở constructor là tùy chọn và có thể lược bỏ.

```
package services;

import model.Comment;
import org.springframework.stereotype.Component;
import proxies.CommentNotificationProxy;
import repositories.CommentRepository;

@Component
public class CommentService {
```

```
  private final CommentRepository commentRepository;
  private final CommentNotificationProxy commentNotificationProxy;

  public CommentService(
           CommentRepository commentRepository,
           CommentNotificationProxy commentNotificationProxy) {
    this.commentRepository = commentRepository;
    this.commentNotificationProxy = commentNotificationProxy;
  }

  public void publishComment(Comment comment) {
    commentRepository.storeComment(comment);
    commentNotificationProxy.sendComment(comment);
  }
}
```

Listing 4.10: Biến lớp CommentService thành một component với constructor injection

Bây giờ chúng ta cần một lớp cấu hình để chỉ dẫn cho Spring biết nơi cần quét các thành phần (*components*). Vì các lớp component nằm rải rác ở các package khác nhau (`services`, `repositories`, `proxies`), chúng ta sử dụng thuộc tính `basePackages` của chú thích `@ComponentScan` để liệt kê các package này như trong Đoạn mã 4.11.

```
package configuration;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan(
  basePackages = {"proxies", "services", "repositories"}
)
public class ProjectConfiguration {
}
```

Listing 4.11: Sử dụng chú thích @ComponentScan trong lớp cấu hình (`configuration/ProjectConfiguration.java`)

Ghi chú: So sánh basePackages và basePackageClasses

Trong ví dụ này, tôi sử dụng thuộc tính `basePackages` của chú thích `@ComponentScan`. Spring cũng cung cấp khả năng chỉ định trực tiếp các lớp tiêu biểu của từng package thông qua thuộc tính `basePackageClasses`.
Ưu điểm của việc định nghĩa tên package là bạn chỉ cần viết tên package một lần: ngay cả khi package đó chứa tới 20 lớp component, bạn cũng chỉ cần đúng một dòng ngắn gọn thay vì phải liệt kê 20 lớp. Tuy nhiên, nhược điểm là nếu sau này lập trình viên đổi tên package (*refactor*), họ có thể quên cập nhật chuỗi ký tự trong cấu hình, dẫn đến lỗi runtime. Việc sử dụng `basePackageClasses` an toàn hơn trước các thao tác đổi tên lớp của IDE, nhưng đòi hỏi phải viết dài dòng hơn nếu có nhiều package độc lập.

Cuối cùng, chúng ta viết lại lớp `Main` (Đoạn mã 4.12) để kiểm thử toàn bộ hệ thống bằng việc khởi tạo vùng chứa Spring IoC container:

```
package main;

import configuration.ProjectConfiguration;
import model.Comment;
import org.springframework.context.annotation.
    AnnotationConfigApplicationContext;
import services.CommentService;

public class Main {
  public static void main(String[] args) {
    var context =
      new AnnotationConfigApplicationContext(
        ProjectConfiguration.class);

    var comment = new Comment();
    comment.setAuthor("Laurentiu");
    comment.setText("Demo comment");

    var commentService = context.getBean(CommentService.class);
    commentService.publishComment(comment);
  }
}
```

Listing 4.12: Lớp Main sử dụng ngữ cảnh Spring (`main/Main.java`)

Khi chạy ứng dụng này, kết quả xuất ra trên cửa sổ console hoàn toàn chính xác:

```
Storing comment: Demo comment
Sending notification for comment: Demo comment
```

Kết quả này chứng minh rằng Spring đã tự động phát hiện các bean phù hợp hiện thực các interface, tiêm chúng vào `CommentService`, và mọi lời gọi nghiệp vụ đều diễn ra trơn tru.

Dù đây là một ví dụ quy mô nhỏ và có vẻ như Spring chưa đem lại sự khác biệt quá lớn, nhưng bạn hãy quan sát kỹ: nhờ tính năng DI, chúng ta không hề phải tự tay viết mã khởi tạo cho `CommentService` và các phụ thuộc của nó, cũng như không phải thiết lập các liên kết thủ công. Trong các hệ thống lớn với hàng trăm lớp, việc trao quyền quản lý đối tượng và liên kết cho Spring sẽ loại bỏ hàng ngàn dòng mã rập khuôn (*boilerplate code*), cho phép bạn dồn toàn tâm toàn ý vào việc hiện thực hóa các logic nghiệp vụ quan trọng. Hơn thế nữa, việc đưa các đối tượng này vào ngữ cảnh là tiền đề bắt buộc để Spring có thể áp dụng các tính năng nâng cao như AOP, quản lý giao dịch và bảo mật mà chúng ta sẽ tìm hiểu ở các chương tiếp theo.

Thanh bên: Các cách khác nhau để tiêm phụ thuộc với tầng trừu tượng

Ở Chương 3, bạn đã học nhiều phong cách nối dây tự động: tiêm qua trường (*field injection*), tiêm qua hàm khởi tạo (*constructor injection*) và tiêm qua setter (*setter injection*), cũng như tiêm qua tham số của các phương thức đánh dấu `@Bean` trong lớp cấu hình.

Trong phần trên, chúng ta dùng phương pháp phổ biến và được khuyến nghị nhất: tiêm qua constructor. Tuy nhiên, DI với abstraction vận hành hoàn toàn nhất quán trên tất cả các phong cách DI khác!

1. Tiêm qua trường (Field Injection) - Dự án `sq-ch4-ex3`: Chúng ta chỉ cần bỏ constructor trong `CommentService` và đánh dấu trực tiếp các trường bằng `@Autowired`:

```
@Component
public class CommentService {
  @Autowired
  private CommentRepository commentRepository;

  @Autowired
  private CommentNotificationProxy commentNotificationProxy;

  public void publishComment(Comment comment) {
    commentRepository.storeComment(comment);
    commentNotificationProxy.sendComment(comment);
  }
}
```

2. Tiêm qua tham số của phương thức `@Bean` - Dự án `sq-ch4-ex4`: Nếu loại bỏ hoàn toàn `@Component` khỏi các lớp và cấu hình tường minh trong lớp cấu hình, Spring vẫn dễ dàng khớp các bean thông qua kiểu interface của tham số:

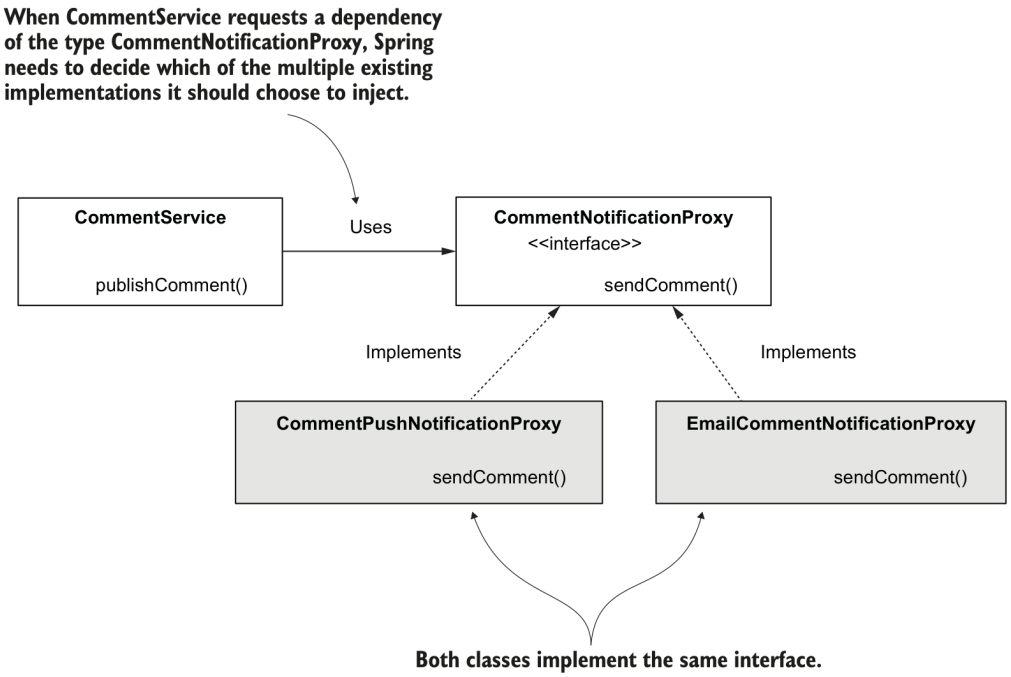
```
@Configuration
public class ProjectConfiguration {
  @Bean
  public CommentRepository commentRepository() {
    return new DBCommentRepository();
  }

  @Bean
  public CommentNotificationProxy commentNotificationProxy() {
    return new EmailCommentNotificationProxy();
  }

  @Bean
  public CommentService commentService(
    CommentRepository commentRepository,
    CommentNotificationProxy commentNotificationProxy) {
    return new CommentService(commentRepository,
```

4.2.2 Lựa chọn bean cần tự động liên kết khi có nhiều cài đặt của một tầng trừu tượng

Cho đến lúc này, chúng ta chỉ mới làm việc với kịch bản mà mỗi interface chỉ có duy nhất một lớp hiện thực trong ngữ cảnh Spring. Nhưng điều gì sẽ xảy ra nếu trong ngữ cảnh có nhiều hơn một bean cùng hiện thực một interface trừu tượng được yêu cầu tiêm vào?

Đây là một tình huống rất đỗi quen thuộc trong các dự án thực tế, và bạn cần biết chính xác cách điều hướng Spring để ứng dụng vận hành đúng như kỳ vọng.

Giả sử ứng dụng của chúng ta có hai lớp cùng hiện thực giao diện `CommentNotificationProxy`: một lớp gửi email (`EmailCommentNotificationProxy`) và một lớp gửi thông báo đẩy (`CommentPushNotifi` (Hình 4.9).



Hình 4.9: Trong các dự án thực tế, một interface thường có nhiều cài đặt khác nhau. Khi sử dụng kỹ thuật tiêm phụ thuộc trên interface, bạn cần chỉ dẫn cho Spring biết nó cần tiêm cài đặt cụ thể nào.

May mắn thay, Spring sử dụng cơ chế xử lý tương tự như những gì chúng ta đã thảo luận ở Chương 3. Khi trong ngữ cảnh tồn tại nhiều hơn một bean tương thích với kiểu được yêu cầu, bạn có hai giải pháp chủ đạo:

1. Sử dụng chú thích `@Primary` để đánh dấu một trong các bean là lựa chọn ưu tiên mặc định.
2. Sử dụng chú thích `@Qualifier` để đặt tên định danh cho từng bean và tham chiếu tường minh theo tên khi tiêm phụ thuộc.

Hãy tạo dự án mới mang tên `sq-ch4-ex5` (xuất phát từ `sq-ch4-ex2`) và bổ sung lớp `CommentPushNotificat` như trong Đoạn mã 4.13.

```
package proxies;

import model.Comment;
import org.springframework.stereotype.Component;

@Component
public class CommentPushNotificationProxy
  implements CommentNotificationProxy {
  @Override
  public void sendComment(Comment comment) {
    System.out.println(
      "Sending push notification for comment: "
          + comment.getText());
  }
}
```

Listing 4.13: Cài đặt mới của interface CommentNotificationProxy (`proxies/CommentPushNotificationProxy.java`)

Nếu bạn để nguyên mã nguồn như vậy và chạy ứng dụng ngay lúc này, ứng dụng sẽ lập tức ném ra ngoại lệ khi khởi động, vì Spring không biết phải chọn bean nào giữa hai bean đang tồn tại:

```
Caused by: org.springframework.beans.factory.
   NoUniqueBeanDefinitionException:
No qualifying bean of type 'proxies.CommentNotificationProxy' available:
expected single matching bean but found 2:
commentPushNotificationProxy,emailCommentNotificationProxy
```

Thông báo ngoại lệ `NoUniqueBeanDefinitionException` chỉ rõ bản chất: Spring mong muốn tìm thấy đúng 1 bean phù hợp, nhưng lại thấy 2 bean (`commentPushNotificationProxy` và `emailCommentNotificationProxy`).

Đánh dấu một cài đặt làm mặc định bằng @Primary

Giải pháp đầu tiên và đơn giản nhất là sử dụng chú thích `@Primary`. Bạn chỉ cần đặt thêm chú thích `@Primary` bên cạnh chú thích `@Component` của lớp mà bạn muốn ưu tiên chọn làm mặc định (Đoạn mã 4.14).

```
package proxies;
```

```
2
3  import model.Comment;
4  import org.springframework.context.annotation.Primary;
5  import org.springframework.stereotype.Component;
6
7  @Component
8  @Primary
9  public class CommentPushNotificationProxy
10   implements CommentNotificationProxy {
11   @Override
12   public void sendComment(Comment comment) {
13     System.out.println(
14       "Sending push notification for comment: "
15           + comment.getText());
16   }
17 }
```

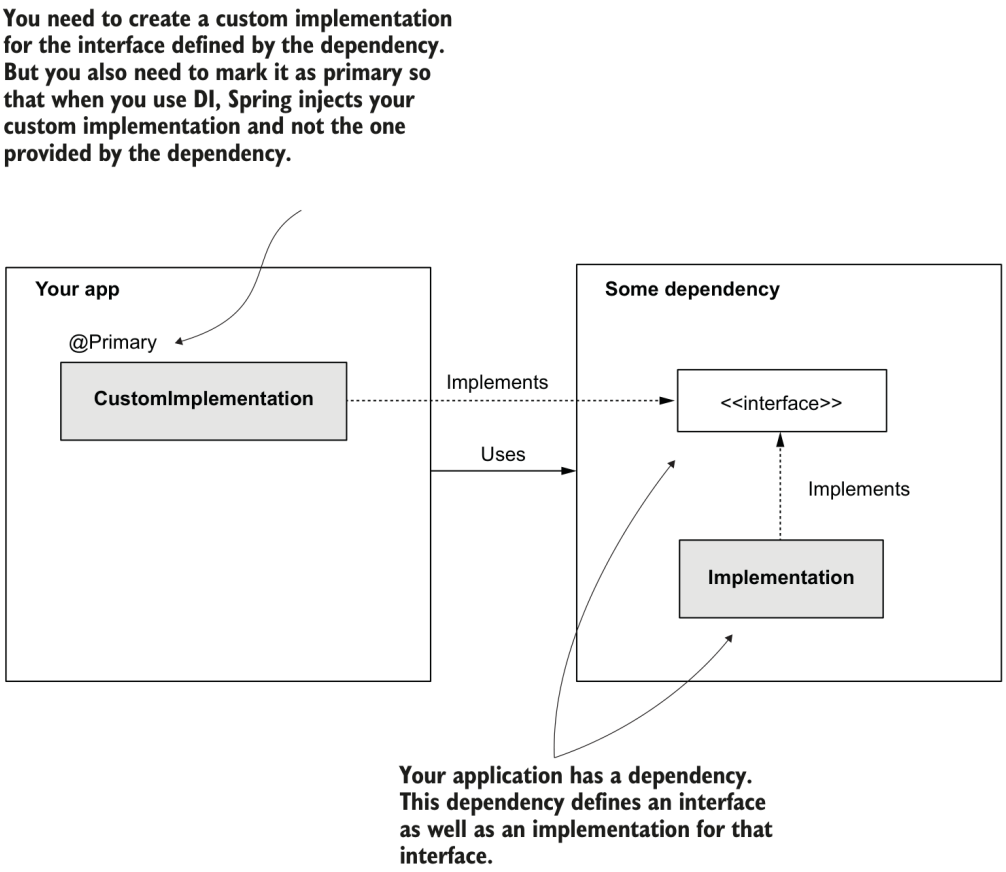
Listing 4.14: Sử dụng chú thích @Primary để chọn cài đặt mặc định khi tiêm phụ thuộc

Chỉ với một thay đổi nhỏ này, ứng dụng sẽ chạy êm thuận và in kết quả ra console:

```
1 Storing comment: Demo comment
2 Sending push notification for comment: Demo comment
```

Lúc này, một câu hỏi rất tự nhiên thường nảy sinh: *«Tại sao chúng ta lại giữ cả hai lớp trong mã nguồn nếu Spring lúc nào cũng chỉ tiêm một cài đặt được đánh dấu `@Primary`? Sao không xóa luôn lớp kia đi?»*

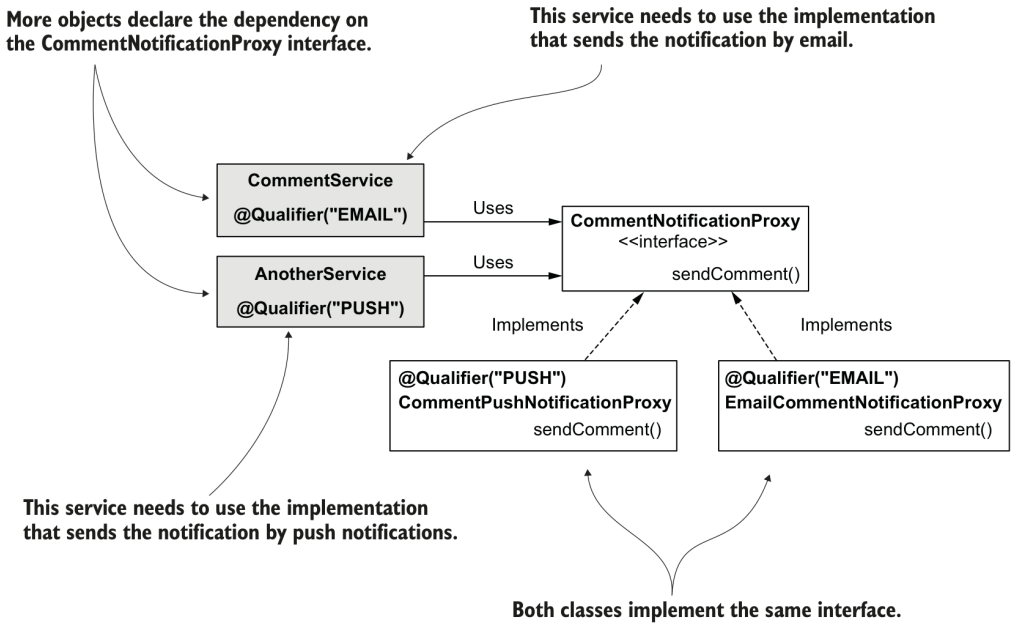
Hãy nhìn nhận kịch bản này dưới góc độ thực tế: các ứng dụng doanh nghiệp thường tích hợp rất nhiều thư viện phụ thuộc bên ngoài. Đôi khi, một thư viện bên thứ ba đã cung cấp sẵn một interface kèm theo một cài đặt mặc định của nó (Hình 4.10). Nhưng cài đặt sẵn có đó không đáp ứng đầy đủ yêu cầu nghiệp vụ của bạn, và bạn muốn viết một cài đặt riêng tùy biến (*custom implementation*). Khi đó, việc đánh dấu `@Primary` lên lớp cài đặt tùy biến của bạn chính là cách đơn giản và thanh lịch nhất để chỉ định Spring ưu tiên sử dụng mã của bạn thay vì lớp của thư viện.



Hình 4.10: Khi sử dụng các thư viện ngoài đã cung cấp sẵn interface và cài đặt mặc định, bạn có thể tạo một cài đặt tùy biến riêng và đánh dấu nó bằng `@Primary`. Nhờ đó, Spring sẽ luôn tiêm lớp của bạn thay vì lớp của thư viện.

Đặt tên định danh cho cài đặt bằng @Qualifier

Đôi khi trong ứng dụng của bạn, nhiều đối tượng khác nhau lại có nhu cầu sử dụng những cài đặt khác nhau của cùng một interface trừu tượng. Ví dụ: một số service cần gửi thông báo qua email, trong khi các service khác lại cần gửi qua thông báo đẩy (Hình 4.11). Cả hai đều cùng hiện thực interface `CommentNotificationProxy`, nhưng mỗi bên phục vụ một mục đích riêng.



Hình 4.11: Nếu các đối tượng khác nhau cần sử dụng những cài đặt khác nhau của cùng một bản hợp đồng, chúng ta sử dụng chú thích `@Qualifier` để đặt tên định danh cho từng cài đặt và chỉ định chính xác cho Spring nơi cần tiêm bean nào.

Chúng ta sẽ điều chỉnh mã nguồn trong dự án `sq-ch4-ex6` để thử nghiệm giải pháp này. Trước hết, chúng ta dùng `@Qualifier` để gán nhãn định danh cho từng lớp component:

```
@Component
@Qualifier("PUSH")
public class CommentPushNotificationProxy
  implements CommentNotificationProxy {
  // Cac phuong thuc giu nguyen...
}
```

```
@Component
@Qualifier("EMAIL")
public class EmailCommentNotificationProxy
  implements CommentNotificationProxy {
  // Cac phuong thuc giu nguyen...
}
```

Khi muốn Spring tiêm cài đặt nào vào đâu, chúng ta chỉ cần dùng chú thích `@Qualifier` tương ứng tại tham số của hàm khởi tạo. Trong Đoạn mã 4.15, chúng ta yêu cầu tiêm cài đặt có định danh `"PUSH"` vào lớp `CommentService`.

```
package services;

import model.Comment;
```

```
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.stereotype.Component;
import proxies.CommentNotificationProxy;
import repositories.CommentRepository;

@Component
public class CommentService {
  private final CommentRepository commentRepository;
  private final CommentNotificationProxy commentNotificationProxy;

  public CommentService(
    CommentRepository commentRepository,
    @Qualifier("PUSH") CommentNotificationProxy commentNotificationProxy
   ) {
    this.commentRepository = commentRepository;
    this.commentNotificationProxy = commentNotificationProxy;
  }

  public void publishComment(Comment comment) {
    commentRepository.storeComment(comment);
    commentNotificationProxy.sendComment(comment);
  }
}
```

Listing 4.15: Chỉ định cài đặt Spring cần tiêm bằng chú thích @Qualifier (`services/CommentService.java`)

Khi khởi chạy ứng dụng, Spring sẽ tiêm chính xác bean `CommentPushNotificationProxy` nhờ nhãn định danh `"PUSH"`:

```
Storing comment: Demo comment
Sending push notification for comment: Demo comment
```

4.3 Làm nổi bật trách nhiệm của đối tượng bằng các chú thích định hình

Cho đến nay, khi thảo luận về các chú thích định hình (*stereotype annotations*), chúng ta mới chỉ sử dụng duy nhất chú thích `@Component`. Tuy nhiên, trong các dự án thực tế, bạn sẽ thấy các kỹ sư phần mềm thường xuyên sử dụng những chú thích chuyên biệt khác cho cùng một mục đích. Trong mục này, tôi sẽ giới thiệu cho bạn hai chú thích định hình vô cùng quan

trọng: `@Service` và `@Repository`.

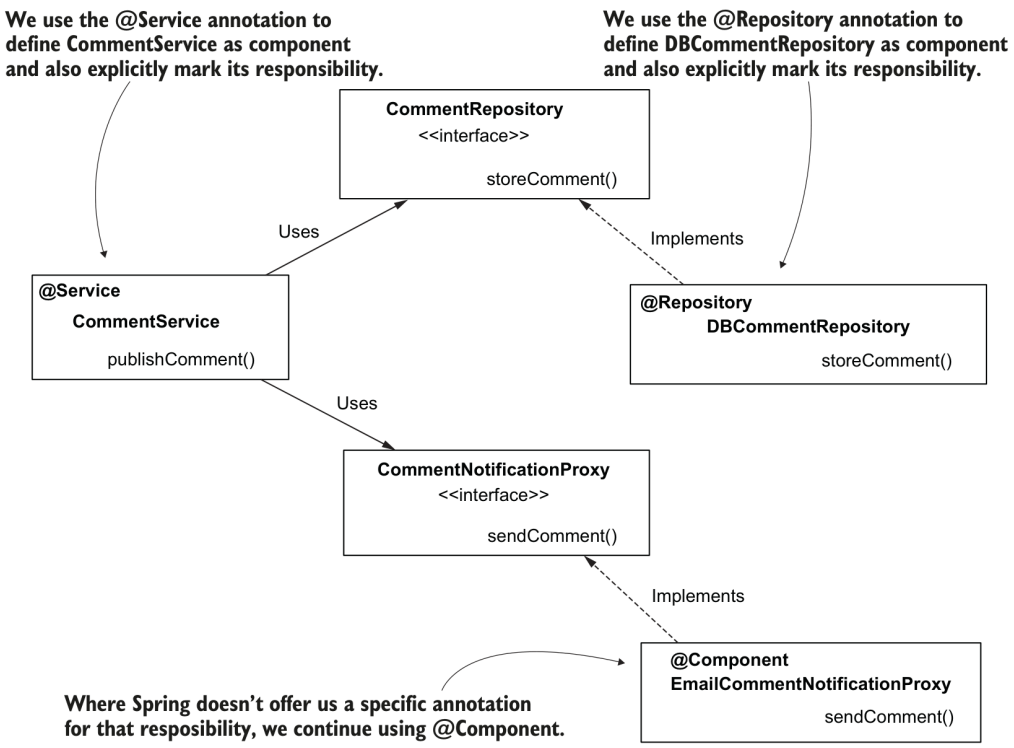
Trong thực tế phát triển phần mềm, việc thể hiện rõ ràng mục đích và trách nhiệm của từng thành phần thông qua các chú thích định hình là một thói quen chuẩn mực rất được khuyến khích. Việc sử dụng chú thích `@Component` mang tính chất quá chung chung và không cung cấp bất kỳ thông tin nào về vai trò cụ thể của đối tượng mà bạn đang hiện thực. Nhưng trong thiết kế hệ thống, các đối tượng hầu như luôn thuộc về một vài nhóm trách nhiệm quen thuộc. Hai trong số các nhóm trách nhiệm tiêu biểu nhất mà chúng ta đã thảo luận ở Mục 4.1 chính là:

- **Lớp dịch vụ (Service):** Các đối tượng mang trọng trách hiện thực hóa các trường hợp sử dụng (use cases) và quy tắc nghiệp vụ.
- **Kho lưu trữ (Repository):** Các đối tượng chuyên trách quản lý việc lưu trữ và truy xuất dữ liệu bền vững.

Vì hai trách nhiệm này xuất hiện dày đặc và đóng vai trò rường cột trong cấu trúc ứng dụng, việc sở hữu một cách thức đánh dấu đặc thù sẽ giúp lập trình viên nhanh chóng nắm bắt bức tranh kiến trúc khi đọc mã nguồn.

Spring cung cấp cho chúng ta:

- Chú thích `@Service`: Dùng để đánh dấu một component đảm nhận vai trò của một dịch vụ nghiệp vụ.
- Chú thích `@Repository`: Dùng để đánh dấu một component hiện thực trách nhiệm kho lưu trữ dữ liệu (Hình 4.12).



Hình 4.12: Chúng ta sử dụng các chú thích `@Service` và `@Repository` để làm nổi bật trách nhiệm của các thành phần trong thiết kế lớp. Với những thành phần mà Spring không cung cấp chú thích đặc thù riêng, chúng ta tiếp tục sử dụng chú thích chung `@Component`.

Cả ba chú thích (`@Component`, `@Service`, và `@Repository`) đều là các chú thích định hình (*stereotype annotations*). Về mặt kỹ thuật, chúng đều chỉ thị cho Spring tạo một thể hiện của lớp được đánh dấu và thêm vào ngữ cảnh của nó. Trên thực tế, `@Service` và `@Repository` đều là các chú thích mở rộng thừa kế từ `@Component` (chúng được chú thích bằng chính `@Component` bên trong mã nguồn nội bộ của Spring).

Trong ví dụ của chương này (dự án `sq-ch4-ex7`), chúng ta đánh dấu lớp `CommentService` bằng `@Service` thay vì `@Component`:

```
@Service
public class CommentService {
  // Cac phuong thuc va thuoc tinh giu nguyen...
}
```

Tương tự, chúng ta đánh dấu lớp kho lưu trữ `DBCommentRepository` bằng chú thích `@Repository`:

```
@Repository
public class DBCommentRepository implements CommentRepository {
  // Cac phuong thuc giu nguyen...
}
```

Đối với lớp proxy (`EmailCommentNotificationProxy`), vì Spring không cung cấp một chú thích định hình riêng biệt cho proxy, chúng ta tiếp tục giữ nguyên chú thích chung `@Component`.

4.4 Tóm tắt chương

- **Tách rời cài đặt thông qua tầng trừu tượng** là một thực hành thiết kế phần mềm cốt lõi. Việc giảm độ phụ thuộc giữa các đối tượng giúp mã nguồn dễ dàng thay đổi mà không gây xáo trộn diện rộng trong toàn bộ ứng dụng, từ đó nâng cao tính mở rộng và khả năng bảo trì của hệ thống.
- Trong ngôn ngữ Java, chúng ta sử dụng các **interface** để tách rời các cài đặt cụ thể. Nói cách khác, chúng ta thiết lập các bản hợp đồng (*contracts*) giữa các thành phần thông qua interface.
- Khi kết hợp tính trừu tượng với kỹ thuật **tiêm phụ thuộc (Dependency Injection)**, Spring đủ thông minh để tự động tìm kiếm trong ngữ cảnh một bean tương thích hiện thực interface trừu tượng được yêu cầu.
- Bạn chỉ sử dụng các **chú thích định hình** (*stereotype annotations*) trên các lớp cụ thể mà bạn muốn Spring khởi tạo và quản lý trong ngữ cảnh. **Tuyệt đối không bao giờ** đặt các chú thích định hình lên interface hoặc lớp trừu tượng.
- Khi ngữ cảnh Spring chứa nhiều hơn một bean cùng hiện thực một tầng trừu tượng, bạn có thể điều hướng Spring chọn đúng bean cần tiêm bằng hai cách:
  - Sử dụng chú thích `@Primary` để đánh dấu một bean làm lựa chọn ưu tiên mặc định.
  - Sử dụng chú thích `@Qualifier` để gán nhãn định danh cho từng bean và chỉ định tường minh bean cần tiêm theo tên định danh.
- Đối với các thành phần mang trách nhiệm dịch vụ nghiệp vụ (*service*), chúng ta sử dụng chú thích `@Service` thay cho `@Component`. Tương tự, đối với các thành phần quản lý lưu trữ dữ liệu (*repository*), chúng ta dùng chú thích `@Repository`. Cách tiếp cận này giúp mã nguồn tường minh hơn về mặt ngữ nghĩa kiến trúc và giúp dự án trở nên trực quan, dễ hiểu đối với toàn bộ đội ngũ phát triển.

Chương 5

Ngữ cảnh Spring: Phạm vi và vòng đời của bean

Cho đến thời điểm hiện tại, chúng ta đã cùng nhau thảo luận về nhiều khía cạnh trọng yếu liên quan đến các thể hiện đối tượng do Spring quản lý (*các bean*). Chúng ta đã nắm vững các cú pháp quan trọng để khởi tạo bean, cũng như cách thiết lập mối liên kết giữa các bean với nhau (bao gồm cả sự cần thiết của việc áp dụng các tầng trừu tượng). Tuy nhiên, chúng ta vẫn chưa đi sâu vào việc Spring tạo ra các bean **như thế nào** và **vào thời điểm nào**. Từ góc nhìn này, chúng ta mới chỉ hoàn toàn dựa vào các hành vi mặc định của framework.

Tôi chủ ý không đề cập đến khía cạnh này sớm hơn trong sách vì muốn bạn tập trung tối đa vào các cú pháp thực hành mà bạn cần ngay lập tức trong dự án. Dẫu vậy, các bài toán trong ứng dụng thực tế trên môi trường sản xuất (*production*) thường rất phức tạp, và đôi khi việc phụ thuộc vào hành vi mặc định của framework là chưa đủ. Vì lý do đó, trong chương này, chúng ta cần đào sâu hơn cách thức Spring quản lý các bean bên trong ngữ cảnh của nó.

Nội dung trọng tâm của chương này

- Sử dụng phạm vi bean đơn thể (*singleton bean scope*).
- Sử dụng cơ chế khởi tạo tức thì (*eager*) và khởi tạo trễ (*lazy*) đối với các singleton bean.
- Sử dụng phạm vi bean nguyên mẫu (*prototype bean scope*).

Spring cung cấp nhiều phương thức tiếp cận khác nhau để tạo bean và quản lý vòng đời của chúng. Trong thế giới Spring, các phương thức tiếp cận này được gọi là các **phạm vi** (*scopes*). Trong chương này, chúng ta sẽ tập trung phân tích hai phạm vi phổ biến nhất mà bạn sẽ thường xuyên bắt gặp trong các ứng dụng Spring: **singleton** và **prototype**.

Ghi chú về các phạm vi web

Sau này, trong Chương 9, chúng ta sẽ tiếp tục tìm hiểu thêm 3 phạm vi bean chuyên biệt khác dành riêng cho các ứng dụng web: *request*, *session*, và *application*.

Singleton là phạm vi mặc định của mọi bean trong Spring, và đó cũng chính là cơ chế mà chúng ta đã âm thầm sử dụng xuyên suốt từ đầu cuốn sách đến giờ. Trong Mục 5.1, chúng ta sẽ bàn luận chi tiết về phạm vi singleton. Chúng ta sẽ làm rõ cách Spring quản lý các singleton bean, sau đó điểm qua những lưu ý sống còn khi sử dụng phạm vi này trong các ứng dụng thực tế.

Trong Mục 5.2, chúng ta sẽ tiếp tục khám phá phạm vi **prototype**. Trọng tâm của mục này là làm rõ sự khác biệt bản chất giữa phạm vi prototype và singleton, cũng như chỉ ra những tình huống thực tế đòi hỏi bạn phải chuyển sang sử dụng prototype thay vì singleton.

5.1 Sử dụng phạm vi bean singleton

Phạm vi bean singleton định nghĩa phương thức tiếp cận mặc định của Spring trong việc quản lý các bean bên trong ngữ cảnh. Đây cũng là phạm vi xuất hiện nhiều nhất trong các dự án phần mềm doanh nghiệp.

Trong Mục 5.1.1, chúng ta sẽ bắt đầu bằng việc tìm hiểu cách Spring khởi tạo và quản lý các singleton bean — điều cốt yếu để bạn biết chính xác khi nào và ở đâu nên sử dụng chúng. Chúng ta sẽ cùng xem xét hai ví dụ áp dụng các cách định nghĩa bean khác nhau (đã học ở Chương 2) và phân tích hành vi của Spring. Kế đó, trong Mục 5.1.2, chúng ta sẽ thảo luận về những khía cạnh trọng yếu khi làm việc với singleton bean trong môi trường thực tế (đặc biệt là vấn đề an toàn đa luồng). Cuối cùng, trong Mục 5.1.3, chúng ta sẽ so sánh hai chiến lược khởi tạo singleton bean: khởi tạo tức thì (*eager*) và khởi tạo trễ (*lazy*).

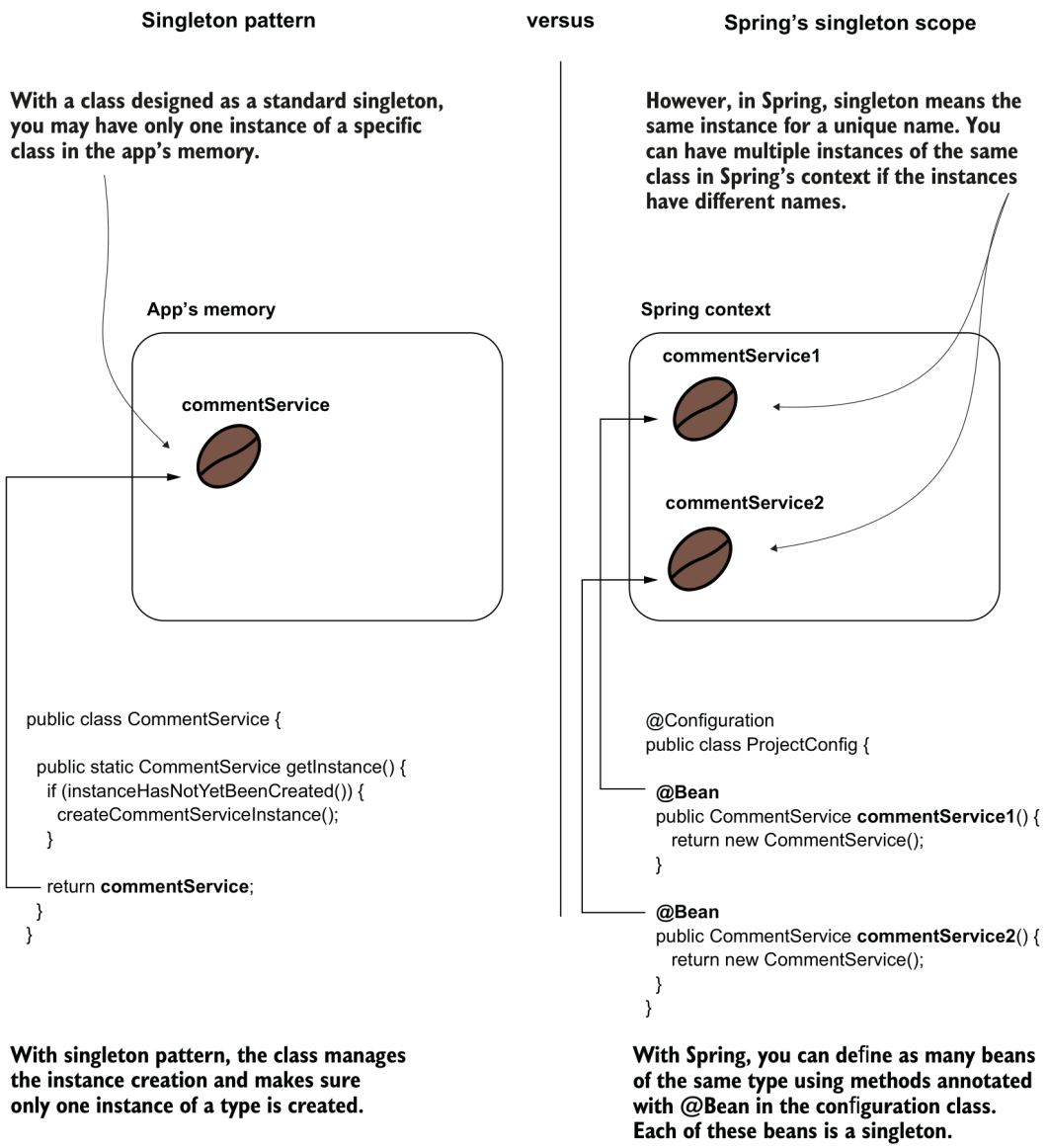
5.1.1 Cách thức hoạt động của singleton bean

Hãy bắt đầu với hành vi của Spring khi quản lý các bean có phạm vi singleton. Bạn cần hiểu rõ điều gì thực sự diễn ra phía sau hậu trường, bởi singleton vừa là phạm vi mặc định vừa là phạm vi được sử dụng rộng rãi nhất trong Spring. Tôi sẽ làm rõ mối liên kết giữa đoạn mã bạn viết và vùng chứa ngữ cảnh Spring, sau đó chúng ta sẽ kiểm chứng hành vi này qua các ví dụ thực hành cụ thể.

Spring tạo một singleton bean khi nó nạp ngữ cảnh (*context*) và gán cho bean đó một cái tên duy nhất (đôi khi còn gọi là *bean ID*). Chúng ta gọi phạm vi này là **singleton** bởi vì mỗi khi bạn tham chiếu tới một bean cụ thể qua tên gọi đó, bạn **luôn luôn nhận về cùng một thể hiện duy nhất**.

Tuy nhiên, hãy hết sức lưu ý! **Bạn hoàn toàn có thể có nhiều thể hiện của cùng một kiểu dữ liệu trong ngữ cảnh Spring, miễn là chúng mang tên gọi khác nhau.** Tôi đặc biệt nhấn mạnh điểm này bởi vì có thể bạn đã từng biết hoặc sử dụng mẫu thiết kế (*design pattern*) Singleton kinh điển của Gang of Four (GoF) trong quá khứ. Nếu bạn chưa từng biết đến mẫu thiết kế Singleton này, bạn sẽ không bị nhầm lẫn và có thể bỏ qua đoạn tiếp theo.

Ngược lại, nếu bạn đã quen thuộc với mẫu thiết kế Singleton trong sách giáo khoa mẫu thiết kế, cách hoạt động của singleton trong Spring thoạt nhìn có vẻ hơi lạ lẫm. Với mẫu thiết kế Singleton truyền thống, bạn chỉ được phép có duy nhất một thể hiện của một lớp trong toàn bộ bộ nhớ của ứng dụng (*application's memory*). Nhưng đối với Spring, khái niệm singleton lại cho phép tồn tại nhiều thể hiện của cùng một lớp: **singleton ở đây có nghĩa là duy nhất trên mỗi tên gọi (unique per name), chứ không phải duy nhất trên toàn ứng dụng (unique per app)**, như được minh họa trên Hình 5.1.



Hình 5.1: So sánh giữa mẫu thiết kế Singleton truyền thống và phạm vi Singleton trong Spring. Với mẫu thiết kế Singleton truyền thống, chỉ có đúng một thể hiện của lớp tồn tại trong bộ nhớ ứng dụng. Trong khi đó, với Spring, singleton có nghĩa là cùng một thể hiện đối với một tên định danh duy nhất; bạn hoàn toàn có thể định nghĩa nhiều bean cùng kiểu nếu chúng có tên khác nhau.

Khai báo singleton bean bằng chú thích @Bean

Hãy cùng chứng minh hành vi của một singleton bean thông qua một ví dụ cụ thể sử dụng chú thích `@Bean` để thêm thể hiện vào ngữ cảnh Spring, sau đó tham chiếu đến bean đó nhiều lần trong phương thức `main()`. Mục đích là chứng minh rằng chúng ta nhận về cùng một tham chiếu đối tượng mỗi khi lấy bean từ context.

Bạn có thể tìm thấy ví dụ này trong dự án mẫu mang tên `sq-ch5-ex1`. Đầu tiên, chúng ta định nghĩa một lớp `CommentService` đơn giản không chứa thuộc tính hay phương thức nào:

```
public class CommentService {
}
```

Tiếp theo, trong Listing 5.1, chúng ta định nghĩa lớp cấu hình `ProjectConfig`, sử dụng một phương thức được đánh dấu bằng `@Bean` để đưa một thể hiện của lớp `CommentService` vào ngữ cảnh Spring.

```
@Configuration
public class ProjectConfig {

  @Bean
  public CommentService commentService() {
    return new CommentService();
  }
}
```

Listing 5.1: Khai báo bean trong lớp cấu hình (dự án `sq-ch5-ex1`)

Bây giờ, chúng ta viết lớp `Main` (Listing 5.2) để kiểm tra hành vi của Spring. Chúng ta lấy tham chiếu đến bean `commentService` hai lần từ ngữ cảnh và so sánh địa chỉ tham chiếu của chúng bằng toán tử `==`.

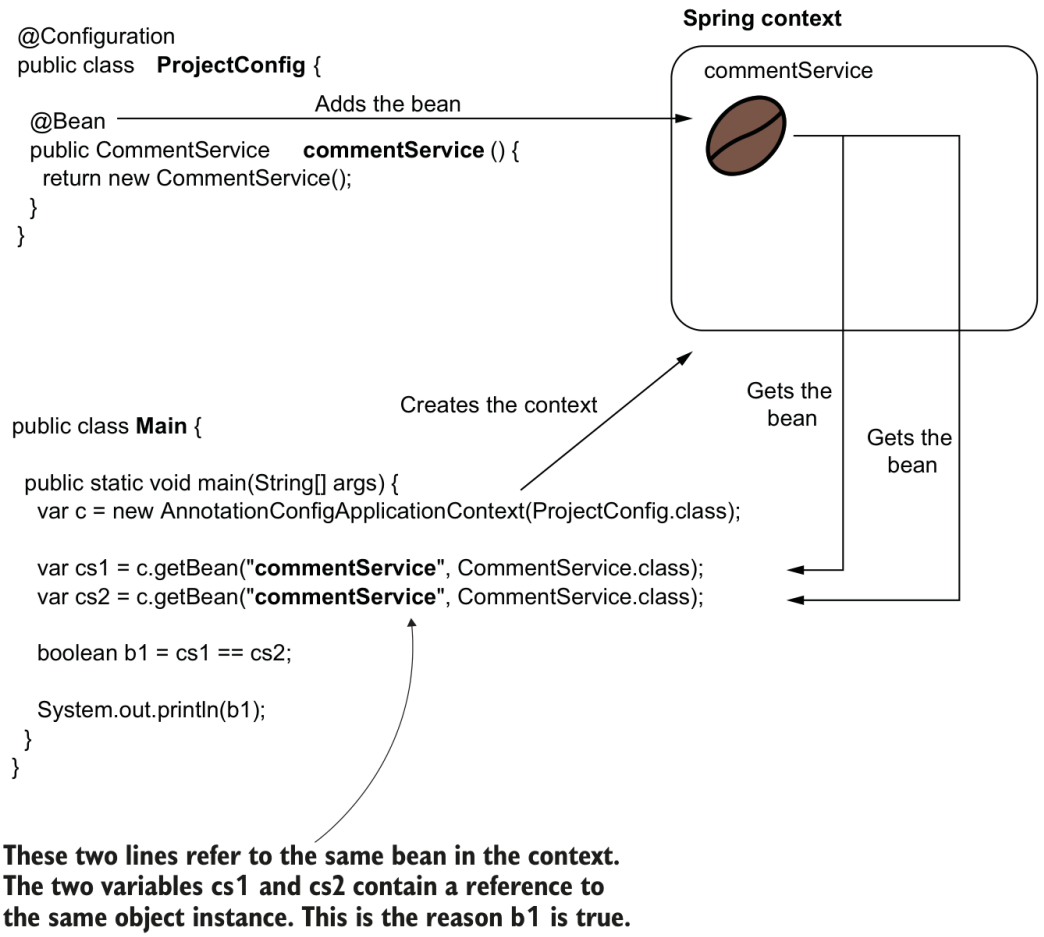
```
public class Main {
  public static void main(String[] args) {
    var c = new AnnotationConfigApplicationContext(ProjectConfig.class);

    var cs1 = c.getBean("commentService", CommentService.class);
    var cs2 = c.getBean("commentService", CommentService.class);

    boolean b1 = cs1 == cs2;
    System.out.println(b1);
  }
}
```

Listing 5.2: Lớp `Main` kiểm tra tính duy nhất của thể hiện singleton bean

Khi chạy ứng dụng, bạn sẽ thấy kết quả in ra màn hình console là:

```
true
```

Biến `b1` có giá trị `true` bởi vì là một singleton bean, Spring luôn trả về chính xác cùng một tham chiếu đối tượng mỗi khi được gọi (Hình 5.2).



Hình 5.2: Một singleton bean trong ngữ cảnh Spring. Ứng dụng khởi tạo ngữ cảnh khi khởi động và thêm bean vào vùng chứa. Tên của phương thức `@Bean` trở thành tên định danh của bean. Bất kỳ nơi nào bạn sử dụng tên định danh đó, bạn đều nhận về cùng một thể hiện đối tượng duy nhất.

Trong ví dụ này, tôi đã truyền tường minh tên của bean (`"commentService"`) khi gọi phương thức `getBean()`. Như bạn đã biết từ Chương 2, khi trong ngữ cảnh chỉ có duy nhất một bean thuộc kiểu dữ liệu đó, bạn thậm chí không cần phải chỉ định tên mà chỉ cần truyền vào kiểu dữ liệu (`CommentService.class`). Ở đây tôi dùng tên nhằm làm nổi bật bản chất: chúng ta đang yêu cầu lấy đúng bean mang tên định danh đó.

Khai báo singleton bean bằng các chú thích định hình (Stereotype Annotations)

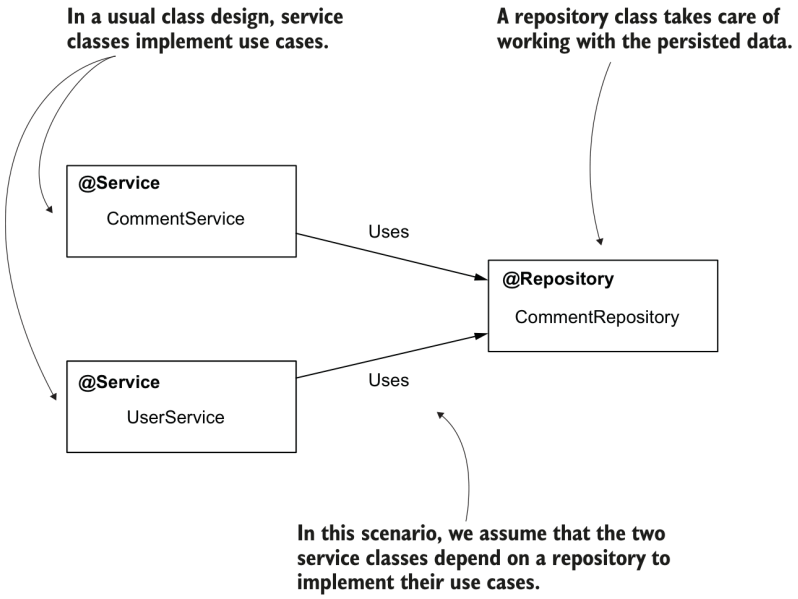
Như đã đề cập trước đó, hành vi của Spring đối với các singleton bean hoàn toàn không có sự khác biệt dù bạn khai báo bằng chú thích `@Bean` hay sử dụng các chú thích định hình (*stereotype annotations*) như `@Component`, `@Service`, hay `@Repository`.

Hãy xem xét một kịch bản thiết kế lớp gồm hai lớp dịch vụ cùng phụ thuộc vào một lớp kho lưu trữ dữ liệu (*repository*). Cụ thể, cả `CommentService` và `UserService` đều cần sử dụng `CommentRepository` để thực hiện các trường hợp sử dụng (*use cases*) của chúng (Hình 5.3).

DECLARING SINGLETON BEANS USING STEREOTYPE ANNOTATIONS

As mentioned earlier, Spring's behavior for singleton beans isn't any different when using stereotype annotations than when you declared them with the `@Bean` annotation. But in this section, I'd like to enforce this statement with an example.

Consider a class design scenario where two service classes depend on a repository. Say we have both `CommentService` and `UserService` depending on a repository named `CommentRepository`, as presented in figure 5.3.



Hình 5.3: Kịch bản thiết kế lớp: Hai lớp dịch vụ (`CommentService` và `UserService`) cùng phụ thuộc vào một kho lưu trữ (`CommentRepository`). Khi được thiết kế dưới dạng các singleton bean, ngữ cảnh Spring sẽ chỉ chứa duy nhất một thể hiện cho mỗi lớp này.

Hãy cùng xây dựng ba lớp này trong dự án mẫu mang tên `sq-ch5-ex2` để kiểm chứng xem liệu Spring có tiêm cùng một thể hiện `CommentRepository` vào cả hai lớp dịch vụ hay không.

Đầu tiên là lớp kho lưu trữ `CommentRepository`, được đánh dấu bằng chú thích `@Repository`:

```
@Repository
public class CommentRepository {
}
```

Tiếp theo là lớp dịch vụ `CommentService`. Chúng ta sử dụng chú thích `@Autowired` để yêu cầu Spring tiêm thể hiện của `CommentRepository` vào thuộc tính của lớp, đồng thời bổ sung một phương thức getter để lấy tham chiếu này ra kiểm tra:

```
@Service
public class CommentService {

  @Autowired
  private CommentRepository commentRepository;

```

```
  public CommentRepository getCommentRepository() {
    return commentRepository;
  }
}
```

Tương tự, lớp dịch vụ thứ hai là `UserService` cũng tiêm kho lưu trữ `CommentRepository`:

```
@Service
public class UserService {

  @Autowired
  private CommentRepository commentRepository;

  public CommentRepository getCommentRepository() {
    return commentRepository;
  }
}
```

Khác với ví dụ trước, lớp cấu hình `ProjectConfig` ở ví dụ này hoàn toàn trống rỗng phần thân; chúng ta chỉ cần dùng chú thích `@ComponentScan` để chỉ cho Spring vị trí các gói mã nguồn cần quét:

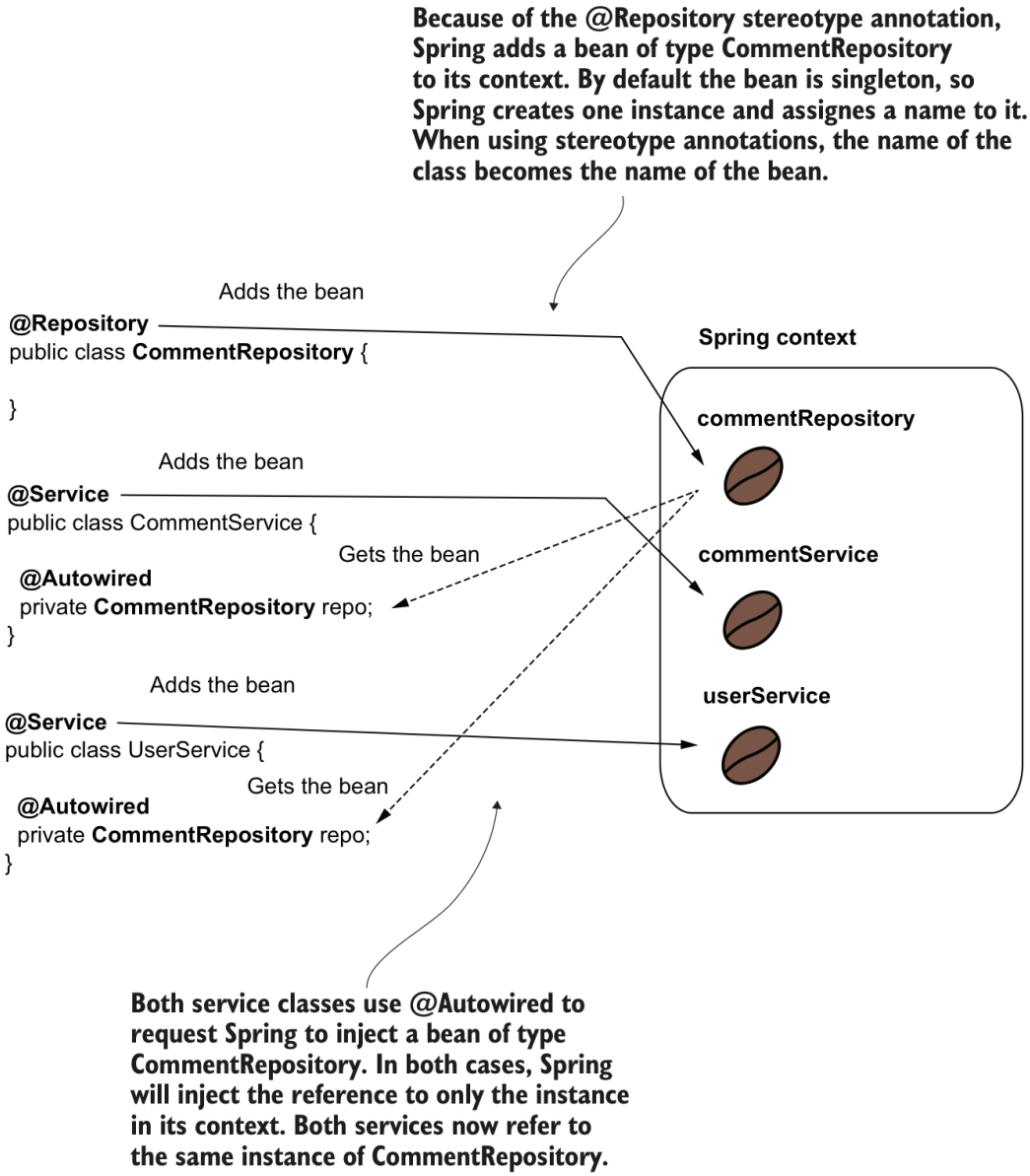
```
@Configuration
@ComponentScan(basePackages = {"services", "repositories"})
public class ProjectConfig {
}
```

Trong lớp `Main` (Listing 5.3), chúng ta lấy tham chiếu của cả hai bean dịch vụ ra từ ngữ cảnh, rồi so sánh thuộc tính `commentRepository` bên trong chúng xem có trỏ tới cùng một đối tượng hay không.

```
public class Main {
  public static void main(String[] args) {
    var c = new AnnotationConfigApplicationContext(ProjectConfig.class);

    var s1 = c.getBean(CommentService.class);
    var s2 = c.getBean(UserService.class);

    boolean b = s1.getCommentRepository() == s2.getCommentRepository();
    System.out.println(b);
  }
}
```

Listing 5.3: Kiểm tra việc tiêm singleton bean vào nhiều vị trí phụ thuộc khác nhau (dự án `sq-ch5-ex2`)

Khi chạy chương trình, kết quả hiển thị trên console chắc chắn là `true`. Hình 5.4 mô tả trực quan cấu trúc của ngữ cảnh Spring trong kịch bản này.



Hình 5.4: Các bean tạo ra bằng chú thích định hình cũng mặc định có phạm vi singleton. Khi sử dụng `@Autowired` để yêu cầu tiêm phụ thuộc, Spring sẽ tiêm cùng một tham chiếu đến thể hiện singleton duy nhất vào tất cả các vị trí được yêu cầu.

Do `CommentRepository` là một singleton bean, Spring chỉ tạo ra đúng một thể hiện của lớp này trong ngữ cảnh. Khi khởi tạo `CommentService` và `UserService`, framework đã tiêm chính xác địa chỉ tham chiếu của thể hiện duy nhất đó vào cả hai đối tượng dịch vụ.

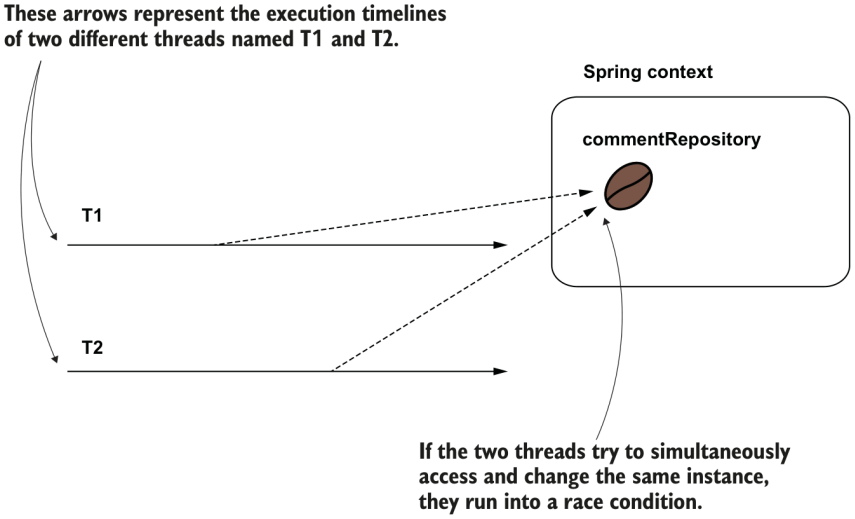
5.1.2 Singleton bean trong các kịch bản thực tế

Chúng ta vừa nắm được cách Spring quản lý các singleton bean. Bây giờ là lúc thảo luận về những quy tắc quan trọng bạn bắt buộc phải khắc cốt ghi tâm khi làm việc với singleton bean trong môi trường sản xuất thực tế.

Bởi vì phạm vi singleton giả định rằng nhiều thành phần trong toàn bộ ứng dụng có thể cùng chia sẻ một thể hiện đối tượng, điều kiện tiên quyết và quan trọng nhất là: **các singleton bean phải có tính chất bất biến (*immutable*)**.

Trong hầu hết các bài toán doanh nghiệp thực tế, ứng dụng của bạn luôn thực thi các thao tác trên nhiều luồng (*multithreaded*) chạy song song (chẳng hạn như bất kỳ ứng dụng web nào tiếp nhận nhiều yêu cầu HTTP từ các người dùng khác nhau cùng một thời điểm). Trong môi trường đa luồng như vậy, nhiều luồng sẽ cùng truy cập vào một thể hiện bean duy nhất. Nếu các luồng này cùng thay đổi trạng thái (thuộc tính) của thể hiện đó, bạn sẽ ngay lập tức đối mặt với hiện tượng **điều kiện tranh chấp** (*race condition*), như mô tả trên Hình 5.5.

If you want mutable singleton beans (whose attributes change), you need to make these beans concurrent by yourself (mainly by employing thread synchronization). But singleton beans aren't designed to be synchronized. They're commonly used to define an app's backbone class design and delegate responsibilities one to another. Technically, synchronization is possible, but it's not a good practice. Synchronizing the thread



Hình 5.5: Khi nhiều luồng truy cập đồng thời vào một singleton bean, chúng dùng chung một thể hiện duy nhất. Nếu các luồng cố gắng thay đổi trạng thái của thể hiện này cùng lúc, chúng sẽ rơi vào điều kiện tranh chấp (*race condition*). Hiện tượng này dẫn đến sai lệch dữ liệu khó lường hoặc phát sinh ngoại lệ runtime nếu bean không được thiết kế an toàn đồng thời.

Điều kiện tranh chấp (*race condition*) là tình huống xuất hiện trong kiến trúc đa luồng khi nhiều luồng cùng cố gắng can thiệp và sửa đổi một tài nguyên dùng chung. Khi điều kiện tranh chấp xảy ra, lập trình viên bắt buộc phải thực hiện cơ chế đồng bộ hóa (*thread*

synchronization) một cách chuẩn xác để tránh phát sinh dữ liệu rác hoặc ngoại lệ trong lúc vận hành.

Nếu bạn muốn tạo ra các singleton bean có trạng thái thay đổi (*mutable singleton beans*), bạn buộc phải tự tay xử lý tính an toàn đồng thời (*concurrency*) bằng các cơ chế đồng bộ hóa luồng (như từ khóa `synchronized`, các cấu trúc khóa `ReentrantLock`, hay các biến `Atomic`). Tuy nhiên, **singleton bean không hề được thiết kế ra cho mục đích đồng bộ hóa luồng**. Trong kiến trúc ứng dụng tiêu chuẩn, các singleton bean đóng vai trò là bộ khung xương sống (*backbone*) điều phối và ủy thác trách nhiệm giữa các thành phần. Về mặt kỹ thuật, việc đồng bộ hóa luồng trên một bean singleton hoàn toàn khả thi, nhưng đó là một **thực hành tồi (*bad practice*)**. Việc bắt các luồng phải xếp hàng chờ đợi nhau trên một thể hiện dùng chung sẽ làm sụt giảm nghiêm trọng hiệu năng và thông lượng xử lý của toàn bộ hệ thống. Trong hầu hết các trường hợp, bạn luôn có những giải pháp thanh thoát hơn nhiều để giải quyết bài toán nghiệp vụ mà không cần dùng đến việc đồng bộ hóa luồng.

Bạn còn nhớ lời khuyên của tôi ở Chương 3 chứ: *Kỹ thuật tiêm phụ thuộc qua hàm khởi tạo (constructor injection) luôn là thực hành tốt nhất được khuyến nghị, vượt trội hơn hẳn so với tiêm qua thuộc tính (field injection)*. Một trong những lợi ích to lớn nhất của tiêm qua hàm khởi tạo là nó cho phép bạn thiết kế thể hiện đối tượng có **tính chất bất biến hoàn toàn** bằng cách khai báo các trường thuộc tính với từ khóa `final`.

Trong ví dụ trước, chúng ta có thể nâng cấp định nghĩa lớp `CommentService` bằng cách thay thế tiêm thuộc tính bằng tiêm hàm khởi tạo:

```
@Service
public class CommentService {

  private final CommentRepository commentRepository;

  public CommentService(CommentRepository commentRepository) {
    this.commentRepository = commentRepository;
  }

  public CommentRepository getCommentRepository() {
    return commentRepository;
  }
}
```

Việc đặt từ khóa `final` cho thuộc tính `commentRepository` nhấn mạnh một thông điệp thiết kế rõ ràng: giá trị của thuộc tính này được cố định mãi mãi ngay từ thời điểm tạo đối tượng và không ai có thể can thiệp thay đổi nó sau đó.

Ba nguyên tắc vàng khi làm việc với Bean

Quy tắc sử dụng bean trong thực tế có thể tóm gọn trong 3 điểm cốt tử sau:

1. **Chỉ biến một đối tượng thành bean trong ngữ cảnh Spring nếu bạn cần Spring quản lý nó** nhằm mục đích tận dụng các tính năng mà framework cung cấp (như DI, AOP, quản lý giao dịch, v.v.). Nếu đối tượng chỉ thuần túy chứa dữ liệu hoặc không cần bất kỳ sự can thiệp nào từ framework, đừng đưa nó vào ngữ cảnh.
2. **Nếu bạn đưa một đối tượng vào ngữ cảnh, nó chỉ nên mang phạm vi singleton khi nó có tính chất bất biến (*immutable*)**. Tuyệt đối tránh thiết kế các singleton bean có trạng thái thay đổi (*mutable*).
3. **Nếu một đối tượng bắt buộc phải có trạng thái thay đổi (*mutable*)**, giải pháp hợp lý là sử dụng **phạm vi prototype**, chủ đề mà chúng ta sẽ khám phá trong Mục 5.2.

5.1.3 Sử dụng cơ chế khởi tạo tức thì (Eager) và khởi tạo trễ (Lazy)

Trong phần lớn các trường hợp, Spring tạo ra toàn bộ các singleton bean ngay tại thời điểm nó nạp và khởi dựng ngữ cảnh ứng dụng — đây chính là hành vi mặc định của Spring. Chúng ta gọi cơ chế mặc định này là **khởi tạo tức thì** (*eager instantiation*).

Tuy nhiên, framework còn cung cấp một cơ chế thứ hai: **khởi tạo trễ** (*lazy instantiation*). Với cơ chế khởi tạo trễ, Spring sẽ không tạo sẵn thể hiện của singleton bean khi ngữ cảnh khởi động. Thay vào đó, framework chỉ thực sự tạo thể hiện của bean vào thời điểm đầu tiên có một thành phần nào đó trong ứng dụng tham chiếu hoặc yêu cầu sử dụng bean đó.

Hãy cùng phân tích một ví dụ để thấy rõ sự khác biệt giữa hai cách tiếp cận này, từ đó so sánh ưu và nhược điểm của chúng trong các ứng dụng thực tế.

Trong kịch bản ban đầu (dự án mẫu `sq-ch5-ex3`), chúng ta kiểm tra hành vi mặc định (khởi tạo tức thì - eager). Chúng ta định nghĩa lớp `CommentService` và in một dòng thông báo ra màn hình console ngay bên trong hàm khởi tạo của nó:

```
@Service
public class CommentService {

  public CommentService() {
    System.out.println("CommentService instance created!");
  }
}
```

Lớp cấu hình `ProjectConfig` chỉ định quét gói `services`:

```
@Configuration
@ComponentScan(basePackages = {"services"})
```

```
public class ProjectConfig {
}
```

Trong lớp `Main`, chúng ta chỉ làm duy nhất một việc: khởi tạo ngữ cảnh Spring mà không hề gọi phương thức `getBean()` hay sử dụng bean `CommentService` ở bất kỳ đâu:

```
public class Main {
  public static void main(String[] args) {
    var c = new AnnotationConfigApplicationContext(ProjectConfig.class);
  }
}
```

Dù ứng dụng hoàn toàn không đụng tới bean, khi chạy chương trình, bạn vẫn sẽ thấy dòng chữ sau xuất hiện trên console:

```
CommentService instance created!
```

Điều này khẳng định rằng theo mặc định (*eager*), Spring chủ động tạo sẵn thể hiện của tất cả các singleton bean và cất vào ngữ cảnh ngay khi ứng dụng khởi động.

Bây giờ, chúng ta chuyển sang kịch bản thứ hai (dự án mẫu `sq-ch5-ex4`). Chúng ta bổ sung chú thích `@Lazy` lên trên khai báo của lớp `CommentService` (nếu dùng phương thức `@Bean`, bạn đặt `@Lazy` ngay phía trên phương thức đó):

```
@Service
@Lazy
public class CommentService {

  public CommentService() {
    System.out.println("CommentService instance created!");
  }
}
```

Chú thích `@Lazy` đóng vai trò là một chỉ thị yêu cầu Spring: *Hãy tạm thời hoãn việc khởi tạo đối tượng này cho tới khi có ai đó thực sự cần dùng đến nó lần đầu tiên.*

Nếu bạn giữ nguyên lớp `Main` như ở ví dụ trước và chạy lại ứng dụng, bạn sẽ nhận thấy dòng thông báo trong constructor hoàn toàn không xuất hiện trên console nữa! Vì không có ai yêu cầu bean `CommentService`, Spring đã không tạo ra thể hiện của nó.

Bây giờ, hãy chỉnh sửa lớp `Main` để yêu cầu Spring cung cấp bean `CommentService`:

```
public class Main {
  public static void main(String[] args) {
    var c = new AnnotationConfigApplicationContext(ProjectConfig.class);

```

```
    System.out.println("Truoc khi goi bean CommentService");
    var service = c.getBean(CommentService.class);
    System.out.println("Sau khi goi bean CommentService");
  }
}
```

Chạy lại chương trình, bạn sẽ thấy kết quả hiển thị trên màn hình:

```
Truoc khi goi bean CommentService
CommentService instance created!
Sau khi goi bean CommentService
```

Thể hiện của bean chỉ được khởi tạo chính xác tại thời điểm phương thức `getBean()` được thực thi, nằm giữa hai dòng thông báo trước và sau!

Khi nào nên dùng Eager và khi nào nên dùng Lazy? Trong đa số trường hợp, để framework tạo sẵn toàn bộ các thể hiện ngay từ đầu khi ngữ cảnh khởi tạo (*eager*) là phương án tối ưu và thuận tiện hơn nhiều:

- **Hiệu năng thực thi cao hơn:** Khi một bean ủy thác công việc cho bean khác trong quá trình xử lý nghiệp vụ, đối tượng phụ thuộc đã có sẵn trong bộ nhớ. Framework không cần phải tốn thêm các thao tác kiểm tra xem thể hiện đã tồn tại hay chưa, từ đó giảm thiểu độ trễ xử lý.
- **Phát hiện lỗi sớm (*Fail-fast*):** Đây là lợi thế lớn nhất của cơ chế *eager*. Nếu lớp cấu hình bị sai sót, thiếu phụ thuộc, hoặc xảy ra ngoại lệ trong hàm khởi tạo của bean, ứng dụng sẽ báo lỗi và dừng ngay tại thời điểm khởi động. Lập trình viên có thể phát hiện và sửa chữa lỗi ngay lập tức. Ngược lại, với cơ chế *lazy*, lỗi tiềm ẩn này chỉ bộc phát khi ứng dụng đang chạy thực tế trên môi trường sản xuất và có người dùng tình cờ kích hoạt chức năng đó, gây ảnh hưởng tiêu cực tới trải nghiệm người dùng.

Dẫu vậy, cơ chế khởi tạo trễ (*lazy*) không phải là hoàn toàn vô dụng. Hãy cùng lắng nghe một câu chuyện thực tế từ kinh nghiệm của tác giả.

Bài học từ dự án monolithic khổng lồ

Cách đây một thời gian, tôi từng tham gia bảo trì một hệ thống ứng dụng nguyên khối (*monolith*) đồ sộ. Ứng dụng này được triển khai cài đặt cho nhiều khách hàng ở các địa điểm và phân hệ nghiệp vụ rất khác nhau.

Ở mỗi phân hệ, một khách hàng cụ thể thường chỉ sử dụng một phần nhỏ các tính năng của toàn bộ hệ thống. Nếu khởi tạo tất cả các bean cùng lúc khi ứng dụng khởi động, ngữ cảnh Spring sẽ tiêu tốn một lượng dung lượng RAM khổng lồ hoàn toàn lãng phí. Để giải quyết bài toán tài nguyên, đội ngũ kỹ sư khi đó đã cấu hình hầu hết các bean sang chế độ khởi tạo trễ (*lazy*) để ứng dụng chỉ cấp phát bộ nhớ cho những thể hiện thực sự có yêu cầu phát sinh.

Tuy nhiên, lời khuyên chân thành của tôi là: **hãy luôn ưu tiên giữ nguyên cơ chế mặc định là khởi tạo tức thì (*eager*)**. Nhu cầu bắt buộc phải chuyển sang *lazy* thường là một tín hiệu cảnh báo ngầm rằng kiến trúc của ứng dụng đang gặp vấn đề (chẳng hạn như quá cồng kềnh, phân chia trách nhiệm chưa hợp lý). Thay vì lạm dụng *lazy*, giải pháp căn cơ là tái cấu trúc ứng dụng thành các mô-đun độc lập hoặc chuyển dịch sang kiến trúc microservices để chỉ triển khai những gì khách hàng cần. Dẫu sao trong thực tế, nếu vì rào cản thời gian và chi phí mà bạn chưa thể giải quyết triệt để nguyên nhân gốc rễ, thì cơ chế *lazy* vẫn là một giải pháp tình thế cứu cánh để chữa trị bớt triệu chứng trước mắt.

5.2 Sử dụng phạm vi bean prototype

Trong mục này, chúng ta sẽ chuyển sang khám phá phạm vi bean thứ hai do Spring hỗ trợ: **nguyên mẫu (*prototype*)**.

Trong Mục 5.2.1, chúng ta sẽ tìm hiểu hành vi của framework đối với các bean được cấu hình phạm vi prototype, nắm bắt cách sử dụng chú thích `@Scope` và chạy thử các ví dụ minh chứng. Kế đó, trong Mục 5.2.2, chúng ta sẽ đi sâu vào các trường hợp sử dụng trong thực tế mà bạn bắt buộc phải cân nhắc dùng phạm vi prototype thay vì singleton.

5.2.1 Cách thức hoạt động của prototype bean

Hãy cùng làm rõ cơ chế quản lý của Spring đối với các prototype bean trước khi bàn về bài toán ứng dụng của chúng. Ý tưởng cốt lõi của phạm vi này vô cùng trực quan: **Mỗi lần bạn yêu cầu một tham chiếu tới một prototype bean, Spring sẽ tạo ra một thể hiện đối tượng hoàn toàn mới**.

Đối với các bean mang phạm vi prototype, Spring không trực tiếp lưu trữ và quản lý thể hiện đối tượng cố định trong ngữ cảnh như singleton. Thay vào đó, framework chỉ quản lý **kiểu**

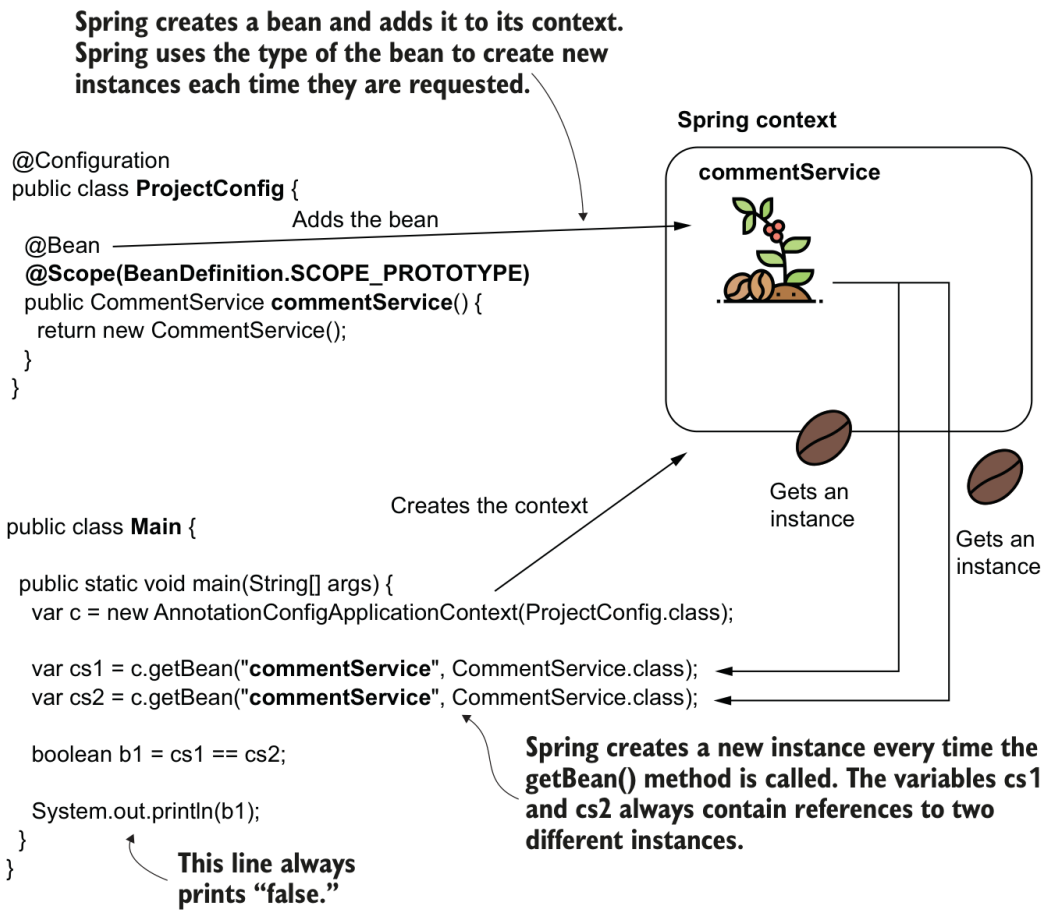
dữ liệu (*type*) của đối tượng và đóng vai trò như một nhà máy tạo mới thể hiện đối tượng mỗi khi có lời gọi lấy bean.

Trên Hình 5.6, tôi ví von bean prototype giống như một **cây cà phê** (*coffee plant*): mỗi lần bạn yêu cầu một hạt cà phê (một bean), cái cây lại sinh ra một hạt mới tinh cho bạn. Tôi dùng hình ảnh cây cà phê để giúp bạn ghi nhớ sâu sắc sự khác biệt giữa phạm vi prototype và hạt cà phê đơn lẻ của singleton.

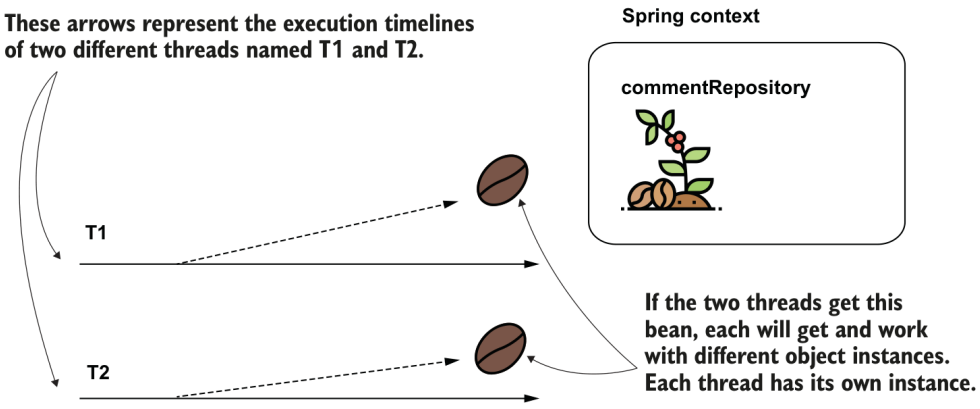
Để thay đổi phạm vi của một bean sang prototype, chúng ta sử dụng chú thích `@Scope`:

- Khi khai báo bean bằng phương thức `@Bean`, chúng ta đặt chú thích `@Scope` song hành cùng với `@Bean` ngay phía trên phương thức cấu hình.
- Khi khai báo bean bằng các chú thích định hình (`@Component`, `@Service`, `@Repository`), chúng ta đặt `@Scope` trực tiếp phía trên định nghĩa của lớp.

Một ưu điểm tuyệt vời của prototype bean là: **chúng ta hoàn toàn không phải bận tâm về các vấn đề tương tranh đa luồng (*concurrency*)**, bởi vì mỗi luồng khi yêu cầu bean đều được trao cho một thể hiện độc lập riêng biệt. Nhờ vậy, việc thiết kế các prototype bean có trạng thái thay đổi (*mutable*) là hoàn toàn an toàn (Hình 5.7).



Hình 5.6: Sử dụng chú thích `@Scope` để chuyển phạm vi của bean sang prototype. Bean lúc này giống như một cây cà phê, liên tục sinh ra các thể hiện mới mỗi khi được gọi. Hai biến `cs1` và `cs2` trỏ tới hai vùng nhớ khác nhau, vì vậy phép so sánh luôn cho ra giá trị `false`.



Hình 5.7: Khi nhiều luồng cùng yêu cầu một prototype bean, mỗi luồng sẽ nhận được một thể hiện đối tượng hoàn toàn riêng biệt. Nhờ vậy, các luồng không bao giờ tranh chấp tài nguyên của nhau và tránh được hiện tượng *race condition*.

Khai báo prototype bean bằng chú thích @Bean

Để kiểm chứng lý thuyết này, chúng ta xây dựng dự án mẫu `sq-ch5-ex5`. Chúng ta khai báo một bean mang tên `CommentService` có phạm vi prototype để chứng minh rằng mỗi lần gọi hàm lấy bean, chúng ta đều nhận về một thể hiện mới.

Lớp `CommentService` vẫn giữ định nghĩa rỗng đơn giản:

```
public class CommentService {
}
```

Trong lớp cấu hình `ProjectConfig` (Listing 5.4), chúng ta sử dụng chú thích `@Scope(BeanDefinition.SCO` để thiết lập phạm vi nguyên mẫu cho bean:

```
@Configuration
public class ProjectConfig {

  @Bean
  @Scope(BeanDefinition.SCOPE_PROTOTYPE)
  public CommentService commentService() {
    return new CommentService();
  }
}
```

Listing 5.4: Khai báo prototype bean trong lớp cấu hình (dự án `sq-ch5-ex5`)

Hằng số định nghĩa Scope

Thay vì gõ thủ công chuỗi ký tự `"prototype"`, bạn nên sử dụng hằng số chuẩn `BeanDefinition.SCOPE_PROTOTYPE` (thuộc gói `org.springframework.beans.factory.config`) để hạn chế lỗi gõ chính tả (*typo*) và tận dụng tính năng tự động gợi ý mã nguồn của IDE.

Tiếp theo, trong lớp `Main` (Listing 5.5), chúng ta gọi phương thức `getBean()` hai lần và so sánh hai tham chiếu nhận được:

```
public class Main {
  public static void main(String[] args) {
    var c = new AnnotationConfigApplicationContext(ProjectConfig.class);

    var cs1 = c.getBean("commentService", CommentService.class);
    var cs2 = c.getBean("commentService", CommentService.class);

    boolean b1 = cs1 == cs2;
    System.out.println(b1);
```

```
  }
}
```

Listing 5.5: Kiểm tra hành vi của prototype bean trong lớp `Main`

Khi chạy ứng dụng, màn hình console sẽ luôn in ra giá trị:

```
false
```

Kết quả `false` chứng minh dứt khoát rằng hai biến `cs1` và `cs2` đang chứa hai vùng nhớ của hai thể hiện hoàn toàn độc lập với nhau.

Khai báo prototype bean bằng các chú thích định hình

Hãy tiếp tục kiểm chứng hành vi này khi kết hợp prototype bean với cơ chế tự động tiêm phụ thuộc (`@Autowired`) thông qua dự án mẫu `sq-ch5-ex6`.

Chúng ta sẽ định nghĩa bean `CommentRepository` có phạm vi prototype, sau đó tiêm bean này vào hai lớp dịch vụ khác nhau (`CommentService` và `UserService`). Chúng ta sẽ chứng minh rằng mỗi lớp dịch vụ sẽ được tiêm một thể hiện riêng biệt của `CommentRepository` (Hình 5.8).

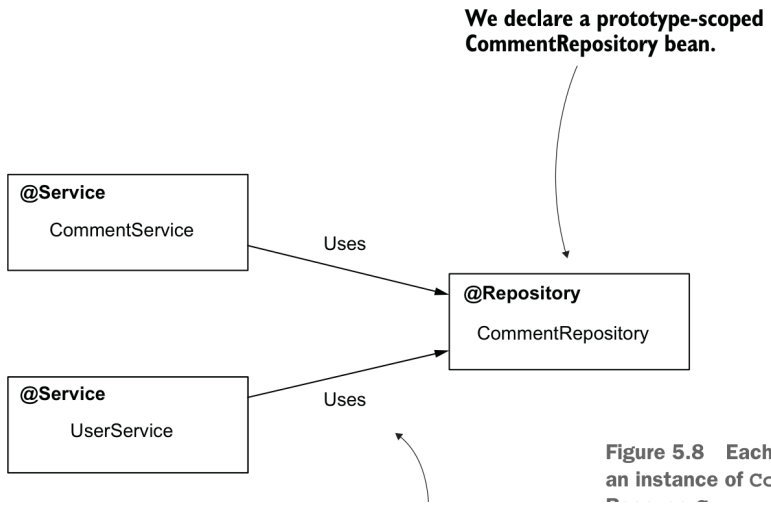


Figure 5.8 Each service class requests an instance of `CommentRepository`.

Hình 5.8: Mỗi lớp dịch vụ yêu cầu một thể hiện của `CommentRepository`. Vì `CommentRepository` được cấu hình là prototype bean, mỗi lớp dịch vụ sẽ nhận về một thể hiện hoàn toàn riêng biệt.

Lớp kho lưu trữ `CommentRepository` được chú thích bằng `@Repository` và `@Scope(BeanDefinition.SCO`

```
@Repository
@Scope(BeanDefinition.SCOPE_PROTOTYPE)
public class CommentRepository {
}
```

Hai lớp dịch vụ `CommentService` và `UserService` yêu cầu tiêm `CommentRepository` thông qua chú thích `@Autowired`:

```
@Service
public class CommentService {

  @Autowired
  private CommentRepository commentRepository;

  public CommentRepository getCommentRepository() {
    return commentRepository;
  }
}
```

Lớp `UserService` có cấu trúc tương tự:

```
@Service
public class UserService {

  @Autowired
  private CommentRepository commentRepository;

  public CommentRepository getCommentRepository() {
    return commentRepository;
  }
}
```

Lớp cấu hình `ProjectConfig` kích hoạt quét thành phần:

```
@Configuration
@ComponentScan(basePackages = {"services", "repositories"})
public class ProjectConfig {
}
```

Bây giờ, chúng ta chạy kiểm thử trong lớp `Main` (Listing 5.6) bằng cách lấy hai bean dịch vụ và so sánh thể hiện kho lưu trữ được tiêm bên trong chúng:

```
public class Main {
  public static void main(String[] args) {
    var c = new AnnotationConfigApplicationContext(ProjectConfig.class);

    var s1 = c.getBean(CommentService.class);
    var s2 = c.getBean(UserService.class);

```

```
    boolean b = s1.getCommentRepository() == s2.getCommentRepository();
    System.out.println(b);
  }
}
```

Listing 5.6: Kiểm tra tiêm phụ thuộc đối với prototype bean (dự án `sq-ch5-ex6`)

Kết quả hiển thị trên màn hình console là:

```
false
```

Do `CommentRepository` là prototype bean, mỗi lần Spring phát hiện một vị trí cần tiêm phụ thuộc (`@Autowired`), nó lại khởi tạo một thể hiện mới toanh để đưa vào. Nhờ vậy, `s1` và `s2` sở hữu hai thể hiện kho lưu trữ hoàn toàn tách biệt.

5.2.2 Prototype bean trong các kịch bản thực tế

Chúng ta vừa khám phá hành vi kỹ thuật của prototype bean. Bây giờ, hãy tập trung vào các trường hợp sử dụng thực tế: **khi nào bạn nên dùng prototype bean và khi nào bạn nên tránh xa chúng?**

Trong thực tế phát triển phần mềm, số lượng prototype bean mà bạn gặp sẽ ít hơn rất nhiều so với singleton bean. Tuy nhiên, có một quy tắc thiết kế kinh điển giúp bạn nhận diện chính xác trường hợp cần đến prototype.

Như chúng ta đã nhấn mạnh trong Mục 5.1.2, singleton bean tuyệt đối không được chứa trạng thái thay đổi (*mutable state*). Giả sử bạn thiết kế một lớp mang tên `CommentProcessor` chuyên đảm nhận nhiệm vụ xử lý và thẩm định tính hợp lệ của các bình luận. Một lớp dịch vụ sẽ sử dụng đối tượng `CommentProcessor` để thực hiện trường hợp sử dụng. Đối tượng `CommentProcessor` lưu trữ chính đối tượng bình luận cần xử lý dưới dạng một thuộc tính, và các phương thức nội bộ của nó sẽ trực tiếp sửa đổi thuộc tính này (Hình 5.9).

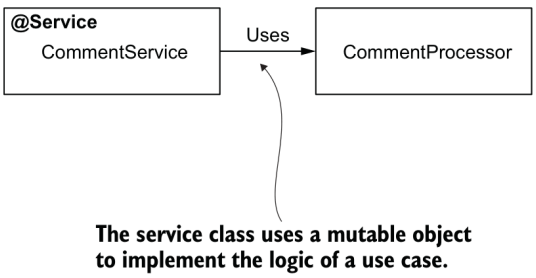


Figure 5.9 A service class uses a mutable object to implement the logic of a use case.

The next listing shows the implementation of the `CommentProcessor` bean.

Hình 5.9: Một lớp dịch vụ sử dụng một đối tượng có trạng thái biến đổi (*mutable*) để hiện thực hóa logic của một trường hợp sử dụng.

Listing 5.7 mô tả mã nguồn cài đặt của lớp `CommentProcessor`.

```
public class CommentProcessor {

  private Comment comment;

  public void setComment(Comment comment) {
    this.comment = comment;
  }

  public Comment getComment() {
    return this.comment;
  }

  public void processComment(Comment comment) {
    // Thay doi thuoc tinh comment...
  }

  public void validateComment(Comment comment) {
    // Tham dinh va thay doi thuoc tinh comment...
  }
}
```

Listing 5.7: Một đối tượng có trạng thái biến đổi: ứng viên tiềm năng cho phạm vi prototype

Trong Listing 5.8, lớp dịch vụ `CommentService` tạo mới một thể hiện của `CommentProcessor` bằng toán tử `new` thông thường bên trong thân phương thức và sử dụng nó:

```
@Service
public class CommentService {

  public void sendComment(Comment c) {
    CommentProcessor p = new CommentProcessor();
    p.setComment(c);
    p.processComment(c);
    p.validateComment(c);

    c = p.getComment();
    // Thuc hien tiep cac logic nghiep vu khac...
  }
}
```

Listing 5.8: Lớp dịch vụ sử dụng trực tiếp toán tử `new` để tạo đối tượng biến đổi

Ở đoạn mã trên, đối tượng `CommentProcessor` thậm chí còn chưa phải là một bean trong ngữ cảnh Spring! Liệu nó có cần thiết phải trở thành một bean không?

Đây là câu hỏi sống còn bạn luôn phải tự đặt ra trước khi quyết định đưa bất kỳ đối tượng nào vào ngữ cảnh. Hãy nhớ rằng: **một đối tượng chỉ cần trở thành bean khi bạn muốn Spring quản lý nó nhằm trang bị thêm một khả năng cụ thể nào đó của framework**. Nếu dừng lại ở kịch bản trên, đối tượng `CommentProcessor` hoàn toàn **không cần** trở thành một bean; việc khởi tạo bằng từ khóa `new` bên trong phương thức là hoàn hảo.

Tuy nhiên, hãy giả định thêm một bước nữa: lớp `CommentProcessor` nay cần sử dụng một đối tượng `CommentRepository` để lưu trữ dữ liệu xuống cơ sở dữ liệu, và `CommentRepository` này vốn đã là một bean trong ngữ cảnh Spring (Hình 5.10).

```
    c = p.getComment();
    // do something further
  }
}
```

Gets the modified Comment instance and uses it further

The `CommentProcessor` object is not even a bean in the Spring context. Does it need to be a bean? It's critical you ask yourself this question before deciding to make any object a bean. Remember that an object needs to be a bean in the context only if Spring needs to manage it to augment the object with some capability the framework offers. If we leave our scenario like this, the `CommentProcessor` object doesn't need to be a bean at all.

But let's suppose further that the `CommentProcessor` bean needs to use an object `CommentRepository` to persist some data, and `CommentRepository` is a bean in the Spring context (figure 5.10).

The CommentProcessor object needs a bean from the Spring context. The easiest way to get an instance of CommentRepository is to request a DI.

Hình 5.10: Nếu đối tượng `CommentProcessor` cần sử dụng một thể hiện `CommentRepository`, con đường thuận tiện nhất là nhờ cậy vào cơ chế tiêm phụ thuộc (DI). Nhưng để làm được điều đó, Spring bắt buộc phải biết đến `CommentProcessor`, tức đối tượng này phải trở thành một bean trong ngữ cảnh.

Trong kịch bản mới này, để được Spring tự động tiêm `CommentRepository` vào, `CommentProcessor` bắt buộc phải được đăng ký làm một bean trong ngữ cảnh.

Thế nhưng, liệu `CommentProcessor` có thể mang phạm vi singleton được không? **Tuyệt đối không!** Nếu bạn khai báo nó là singleton, khi có nhiều luồng cùng gọi phương thức `sendComment()` đồng thời, tất cả các luồng sẽ cùng ghi đè lên thuộc tính `comment` của một

thể hiện duy nhất, gây ra lỗi xung đột dữ liệu tai hại (*race condition*). Bạn sẽ không thể nào biết được bình luận của luồng nào đang được xử lý và liệu dữ liệu cuối cùng có bị ghi đè nhầm hay không!

Trong bài toán này, mỗi lần phương thức `sendComment()` được thực thi, chúng ta bắt buộc phải có một thể hiện `CommentProcessor` mới hoàn toàn. Vì vậy, chúng ta cấu hình lớp `CommentProcessor` thành một **prototype bean**:

```
@Component
@Scope(BeanDefinition.SCOPE_PROTOTYPE)
public class CommentProcessor {

  @Autowired
  private CommentRepository commentRepository;

  // Cac phuong thuc giu nguyen nhu truoc...
}
```

Bây giờ, làm thế nào để `CommentService` lấy được một thể hiện mới của `CommentProcessor` mỗi lần phương thức `sendComment()` được gọi?

Giải pháp trực tiếp và chuẩn xác là: chúng ta tiêm trực tiếp ngữ cảnh Spring (`ApplicationContext`) vào lớp dịch vụ `CommentService`, sau đó gọi phương thức `context.getBean()` bên trong thân phương thức `sendComment()` (Listing 5.9).

```
@Service
public class CommentService {

  @Autowired
  private ApplicationContext context;

  public void sendComment(Comment c) {
    // Moi lan goi ham nay se tao ra mot the hien CommentProcessor moi
   hoan toan
    CommentProcessor p = context.getBean(CommentProcessor.class);

    p.setComment(c);
    p.processComment(c);
    p.validateComment(c);
    c = p.getComment();
    // Tiep tuc xu ly...
  }
}
```

Listing 5.9: Sử dụng `ApplicationContext` để lấy thể hiện prototype mới trong mỗi lần thực thi phương thức

Cạm bẫy tai hại: Tiêm trực tiếp prototype bean vào singleton bean!

Tuyệt đối không mắc phải sai lầm tiêm trực tiếp prototype bean làm thuộc tính của một singleton bean như trong Listing 5.10:

```
@Service
public class CommentService {

  @Autowired
  private CommentProcessor p; // SAI LAM THIET KE!

  public void sendComment(Comment c) {
    p.setComment(c);
    p.processComment(c);
    p.validateComment(c);
    c = p.getComment();
  }
}
```

Listing 5.10: Sai lầm nghiêm trọng: Tiêm một prototype bean trực tiếp vào một singleton bean

Tại sao đây lại là một sai lầm chết người? Bởi vì `CommentService` là một **singleton bean**, Spring chỉ khởi tạo nó đúng một lần duy nhất khi ứng dụng nạp ngữ cảnh. Tại thời điểm khởi tạo đó, Spring cũng chỉ tiêm phụ thuộc vào trường `p` **đúng một lần duy nhất**. Kết quả là dù `CommentProcessor` có được đánh dấu là prototype, ứng dụng của bạn vẫn sẽ tái sử dụng chính xác một thể hiện `p` duy nhất đó xuyên suốt tất cả các lần gọi hàm `sendComment()`! Khi ứng dụng chạy đa luồng, bạn sẽ lại rơi vào đúng thảm họa điều kiện tranh chấp (*race condition*) mà bạn đang cố né tránh.

Tôi xin kết thúc mục này bằng quan điểm thực chiến cá nhân về việc sử dụng prototype bean. Trong quá trình phát triển các dự án mới từ đầu, tôi luôn chủ động tránh việc sử dụng prototype bean, và nói chung là tránh các đối tượng có trạng thái biến đổi (*mutable instances*). Tuy nhiên, khi bạn phải tiếp quản hoặc tái cấu trúc (*refactor*) các hệ thống phần mềm kế thừa (*legacy applications*) lâu đời, việc xóa bỏ toàn bộ các đối tượng biến đổi trong thời gian ngắn là bất khả thi. Khi tôi từng tham gia một dự án đưa Spring vào một hệ thống legacy cũ kỹ ngập tràn các đối tượng mutable, giải pháp sử dụng prototype bean đã giúp đội ngũ duy trì hệ thống chạy ổn định và an toàn, đồng thời tạo tiền đề để tái cấu trúc từng phân hệ một cách

tuần tự và bài bản.

Để giúp bạn ghi nhớ và đối chiếu tổng quan, Bảng 5.1 tóm lược các đặc tính so sánh chi tiết giữa phạm vi singleton và prototype.

Bảng 5.1: So sánh đối chiếu chi tiết giữa phạm vi bean Singleton và Prototype

Tiêu chí so sánh	Phạm vi Singleton	Phạm vi Prototype
Cơ chế liên kết	Framework liên kết một tên định danh với một thể hiện đối tượng thực tế cụ thể.	Framework liên kết một tên định danh với một kiểu dữ liệu (*type*).
Tính duy nhất	Mỗi khi tham chiếu đến tên bean, bạn luôn nhận về cùng một thể hiện đối tượng duy nhất.	Mỗi khi tham chiếu đến tên bean, bạn luôn nhận được một thể hiện đối tượng mới hoàn toàn.
Thời điểm khởi tạo	Có thể cấu hình tạo ngay khi nạp context (*eager*) hoặc khi được gọi lần đầu (*lazy*).	Luôn luôn được tạo ra tại thời điểm có yêu cầu phát sinh.
Tính chất mặc định	Là phạm vi mặc định của mọi bean trong Spring.	Phải được chỉ định tường minh thông qua chú thích `@Scope`.
Trạng thái đối tượng	Khuyến nghị đối tượng phải có tính bất biến (*immutable*) để an toàn đa luồng.	Đối tượng có thể chứa trạng thái biến đổi (*mutable*) mà không sợ xung đột đa luồng.

5.3 Tóm tắt chương

- Trong Spring, **phạm vi của bean (*bean scope*)** quy định cách thức framework tạo dựng và quản lý các thể hiện đối tượng trong suốt vòng đời của chúng.
- Spring cung cấp hai phạm vi cốt lõi cơ bản: **singleton** và **prototype**.
  - Với **singleton**, Spring trực tiếp quản lý các thể hiện đối tượng trong ngữ cảnh. Mỗi thể hiện gắn liền với một tên định danh duy nhất, và bạn luôn nhận về cùng một thể hiện khi gọi tên đó. Singleton là phạm vi mặc định trong Spring.
  - Với **prototype**, Spring chỉ quản lý kiểu đối tượng. Framework sẽ sinh ra một thể hiện đối tượng mới mỗi khi có yêu cầu truy xuất bean.
- Bạn có thể cấu hình Spring khởi tạo singleton bean ngay khi ngữ cảnh khởi động (**khởi tạo tức thì - *eager***) hoặc chỉ khởi tạo khi có thành phần đầu tiên yêu cầu đến nó (**khởi tạo trễ - *lazy*** bằng chú thích `@Lazy`). Theo mặc định, các bean luôn được khởi tạo tức thì.

- Trong các ứng dụng thực tế, singleton bean là lựa chọn phổ biến nhất. Vì nhiều luồng có thể cùng truy cập vào một thể hiện duy nhất, việc thiết kế bean có **tính bất biến (*immutable*)** là nguyên tắc sống còn. Nếu bạn cố tình tạo bean có trạng thái thay đổi, bạn phải tự chịu trách nhiệm xử lý cơ chế đồng bộ hóa luồng.
- Nếu bạn bắt buộc phải đưa một đối tượng có trạng thái biến đổi (*mutable*) vào làm bean trong ngữ cảnh, phạm vi **prototype** là giải pháp hợp lý để tránh điều kiện tranh chấp (*race condition*).
- **Cảnh giác cao độ với việc tiêm một prototype bean trực tiếp vào một singleton bean**. Khi làm như vậy, singleton bean sẽ chỉ được tiêm đúng một thể hiện prototype duy nhất tại thời điểm nó được tạo ra, làm mất đi hoàn toàn ý nghĩa tạo mới của prototype và khiến mã nguồn đối mặt với nguy cơ xung đột luồng nghiêm trọng.

Chương 6

Sử dụng các khía cạnh với Spring AOP

Cho đến nay, chúng ta đã thảo luận chi tiết về ngữ cảnh Spring (*Spring context*), và khả năng duy nhất của framework mà chúng ta khai thác là tiêm phụ thuộc (*Dependency Injection - DI*) dựa trên nguyên lý Đảo ngược quyền điều khiển (*Inversion of Control - IoC*). Với DI, framework chịu trách nhiệm quản lý các đối tượng mà bạn định nghĩa, và bạn có thể yêu cầu framework cấp phát những đối tượng này tới bất kỳ nơi nào cần thiết. Như chúng ta đã thực hành từ Chương 2 đến Chương 5, để lấy tham chiếu tới một bean, phương thức phổ biến nhất là sử dụng chú thích `@Autowired`. Khi bạn yêu cầu một đối tượng như vậy từ ngữ cảnh, chúng ta nói rằng Spring "tiêm" (*injects*) đối tượng vào vị trí được yêu cầu.

Trong chương này, bạn sẽ được học một kỹ thuật mạnh mẽ khác cũng được nâng đỡ bởi chính nguyên lý IoC: **các khía cạnh** (*aspects*).

Nội dung trọng tâm của chương này

- Lập trình hướng khía cạnh (*Aspect-Oriented Programming - AOP*).
- Cách thức khai báo và hiện thực hóa các khía cạnh (*aspects*) trong Spring.
- Quản lý chuỗi thực thi các khía cạnh (*aspect execution chain*) bằng chú thích `@Order`.

Các khía cạnh là giải pháp mà framework sử dụng để **đánh chặn (*intercept*) các lời gọi phương thức và có thể can thiệp làm thay đổi quá trình thực thi của các phương thức đó**. Bạn có quyền can thiệp vào quá trình thực thi của những lời gọi phương thức cụ thể mà bạn chủ động lựa chọn. Kỹ thuật này giúp bạn tách rời một phần logic kỹ thuật phụ trợ ra khỏi phần logic nghiệp vụ cốt lõi của phương thức đang chạy.

Trong nhiều kịch bản, việc tách rời một phần mã nguồn ra riêng sẽ giúp phương thức nghiệp vụ trở nên trong sáng, ngắn gọn và dễ hiểu hơn rất nhiều (Hình 6.1). Nó cho phép lập trình viên chỉ cần tập trung vào các chi tiết nghiệp vụ then chốt khi đọc mã nguồn của phương thức.

Trong chương này, chúng ta sẽ học cách triển khai các khía cạnh và xác định thời điểm phù

hợp để áp dụng chúng. Khía cạnh là một vũ khí cực kỳ sắc bén, và đúng như lời dặn nổi tiếng của người bác dành cho Peter Parker (Người Nhện): *Sức mạnh càng lớn, trách nhiệm càng cao!* ("With great power comes great responsibility!"). Nếu bạn lạm dụng khía cạnh mà thiếu đi sự cẩn trọng, bạn có thể biến ứng dụng của mình thành một hệ thống mù mờ, khó theo dõi và cực kỳ khó bảo trì — hoàn toàn trái ngược với mục tiêu ban đầu bạn muốn hướng tới. Mô thức lập trình này được gọi là **Lập trình hướng khía cạnh** (*Aspect-Oriented Programming - AOP*).

Hình 6.1: Đôi khi việc viết các đoạn mã phụ trợ cùng một chỗ với mã nghiệp vụ cốt lõi là không hợp lý vì nó làm ứng dụng trở nên rối rắm và khó hiểu. Giải pháp là tách phần mã phụ trợ ra khỏi mã nghiệp vụ bằng cách sử dụng các khía cạnh (*aspects*). Trong bức tranh minh họa, nữ lập trình viên Jane cảm thấy nản lòng trước hàng loạt dòng mã ghi log xen lẫn mã nghiệp vụ. Bá tước Dracula đã chỉ cho cô thấy phép màu của khía cạnh trong việc tách biệt các dòng log sang một aspect riêng.

Một lý do đặc biệt quan trọng khác để bạn phải học kỹ khía cạnh là: **Spring sử dụng khía cạnh để triển khai rất nhiều tính năng nền tảng sống còn của nó**. Nắm vững cơ chế vận hành bên dưới của framework sẽ giúp bạn tiết kiệm hàng chục giờ đồng hồ gỡ lỗi (*debug*) sau này khi đối mặt với các sự cố thực tế. Một ví dụ điển hình về tính năng của Spring dựa trên khía cạnh là **quản lý giao dịch** (*transactionality*), chủ đề chúng ta sẽ thảo luận chi tiết ở Chương 13. Giao dịch là một trong những tính năng cốt lõi mà hầu như mọi ứng dụng doanh nghiệp ngày nay đều sử dụng để bảo toàn tính nhất quán của dữ liệu lưu trữ. Một tính năng trọng yếu khác cũng dựa trên khía cạnh là **bảo mật** (*security*), giúp ứng dụng phân quyền và bảo vệ dữ liệu trước các truy cập trái phép. Để thực sự làm chủ những tính năng cao cấp này,

bạn bắt buộc phải hiểu vững về các khía cạnh.

Chúng ta sẽ mở đầu bằng phần lý thuyết nền tảng về cơ chế hoạt động của khía cạnh trong Mục 6.1. Sau khi đã thấu hiểu các khái niệm cơ bản này, trong Mục 6.2, bạn sẽ bắt tay vào việc cài đặt khía cạnh qua các ví dụ thực tiễn, từ cơ bản đến nâng cao. Cuối cùng, trong Mục 6.3, chúng ta sẽ tìm hiểu điều gì xảy ra khi có nhiều khía cạnh cùng can thiệp vào một phương thức duy nhất và cách làm chủ chuỗi thực thi đó.

6.1 Cách thức hoạt động của các khía cạnh trong Spring

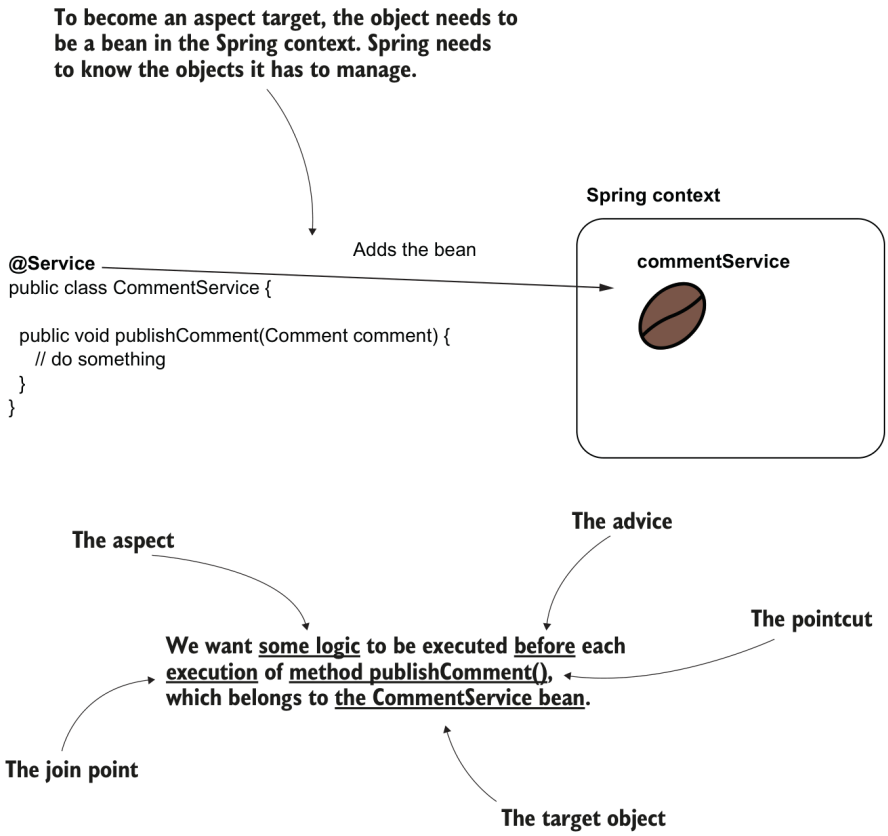
Trong mục này, bạn sẽ tìm hiểu cách các khía cạnh vận hành và những thuật ngữ chuyên ngành bắt buộc phải nắm vững khi tiếp cận AOP. Việc hiểu rõ cơ chế này không chỉ giúp bạn làm cho mã nguồn dễ bảo trì hơn, mà còn vén bức màn bí ẩn về cách Spring cắm các tính năng tự động vào ứng dụng của bạn.

Một **khía cạnh** (*aspect*) đơn giản là một đoạn mã logic mà framework sẽ thực thi mỗi khi bạn gọi một phương thức cụ thể nào đó do bạn chỉ định. Khi thiết kế một aspect, bạn cần xác định rõ ba yếu tố then chốt sau:

- **Aspect (Khía cạnh):** Đoạn mã bạn muốn Spring thực thi khi các phương thức cụ thể được gọi.
- **Advice (Chỉ dẫn / Hành động can thiệp):** *Thời điểm* ứng dụng nên thực thi logic của khía cạnh (chẳng hạn: trước khi phương thức chạy, sau khi phương thức hoàn tất, hoặc thay thế hoàn toàn lời gọi phương thức).
- **Pointcut (Điểm cắt):** *Những phương thức nào* mà framework cần đánh chặn để áp dụng khía cạnh.

Trong thuật ngữ của AOP, bạn còn gặp khái niệm **Điểm nối** (*Join point*). Điểm nối đại diện cho sự kiện kích hoạt việc thực thi của một khía cạnh. Tuy nhiên, trong phạm vi của Spring AOP, sự kiện này **luôn luôn là một lời gọi phương thức** (*method call*).

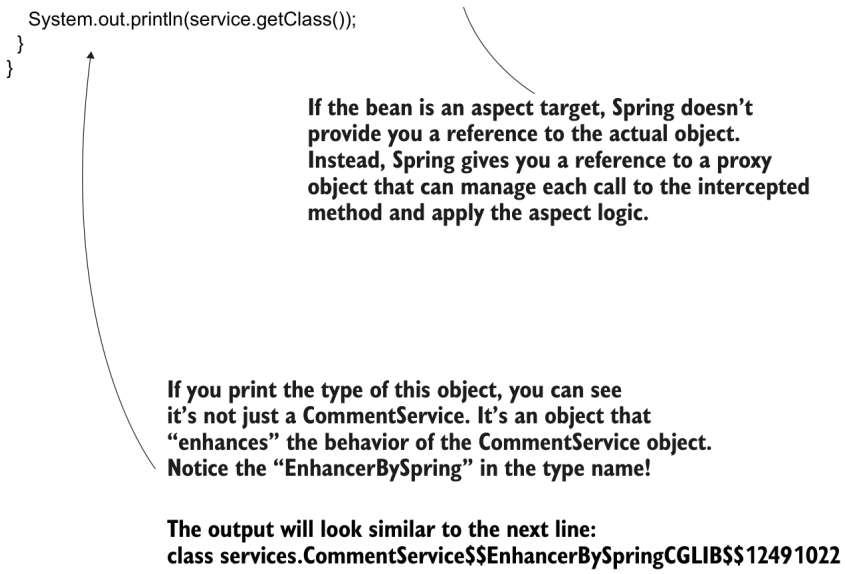
Tương tự như trường hợp tiêm phụ thuộc, để có thể áp dụng khía cạnh, bạn bắt buộc phải để framework quản lý đối tượng chứa phương thức đó. Bạn sử dụng các phương thức đã học ở Chương 2 để thêm các bean vào ngữ cảnh Spring, từ đó trao quyền cho framework kiểm soát và áp dụng các khía cạnh bạn định nghĩa. Bean khai báo phương thức bị đánh chặn được gọi là **đối tượng mục tiêu** (*target object*). Hình 6.2 tổng kết mối quan hệ giữa các thuật ngữ này.



Hình 6.2: Các thuật ngữ cốt lõi trong AOP: Spring thực thi một đoạn logic (**Aspect**) khi ai đó gọi một phương thức xác định (**Pointcut**). Chúng ta cần chỉ định thời điểm đoạn logic đó chạy so với điểm cắt (ví dụ: chạy trước) — đó là **Advice**. Để Spring có thể đánh chặn phương thức, đối tượng chứa phương thức đó bắt buộc phải là một bean trong ngữ cảnh, và đối tượng đó đóng vai trò là đối tượng mục tiêu (**Target object**). Lời gọi phương thức chính là điểm nối (**Join point**).

Thế nhưng, làm cách nào Spring có thể đánh chặn từng lời gọi phương thức và áp dụng logic của khía cạnh? Như đã thảo luận, đối tượng mục tiêu bắt buộc phải là một bean trong ngữ cảnh Spring. Nhưng bởi vì bạn đã biến đối tượng đó thành mục tiêu của một khía cạnh, Spring sẽ **không trực tiếp trả về cho bạn tham chiếu tới thể hiện bean gốc** khi bạn yêu cầu từ ngữ cảnh. Thay vào đó, Spring cung cấp cho bạn một đối tượng trung gian chuyên làm nhiệm vụ gọi logic của khía cạnh trước khi chuyển tiếp tới phương thức thực tế. Chúng ta gọi đối tượng trung gian này là **đối tượng ủy quyền** (*proxy object*).

Kể từ lúc này, bất cứ khi nào bạn lấy bean ra khỏi ngữ cảnh — dù bằng phương thức `getBean()` hay thông qua kỹ thuật tiêm phụ thuộc DI — bạn đều nhận về đối tượng proxy thay vì bean gốc (Hình 6.3). Cơ chế cài cắm khía cạnh thông qua proxy này được gọi là **đan dệt** (*weaving*).



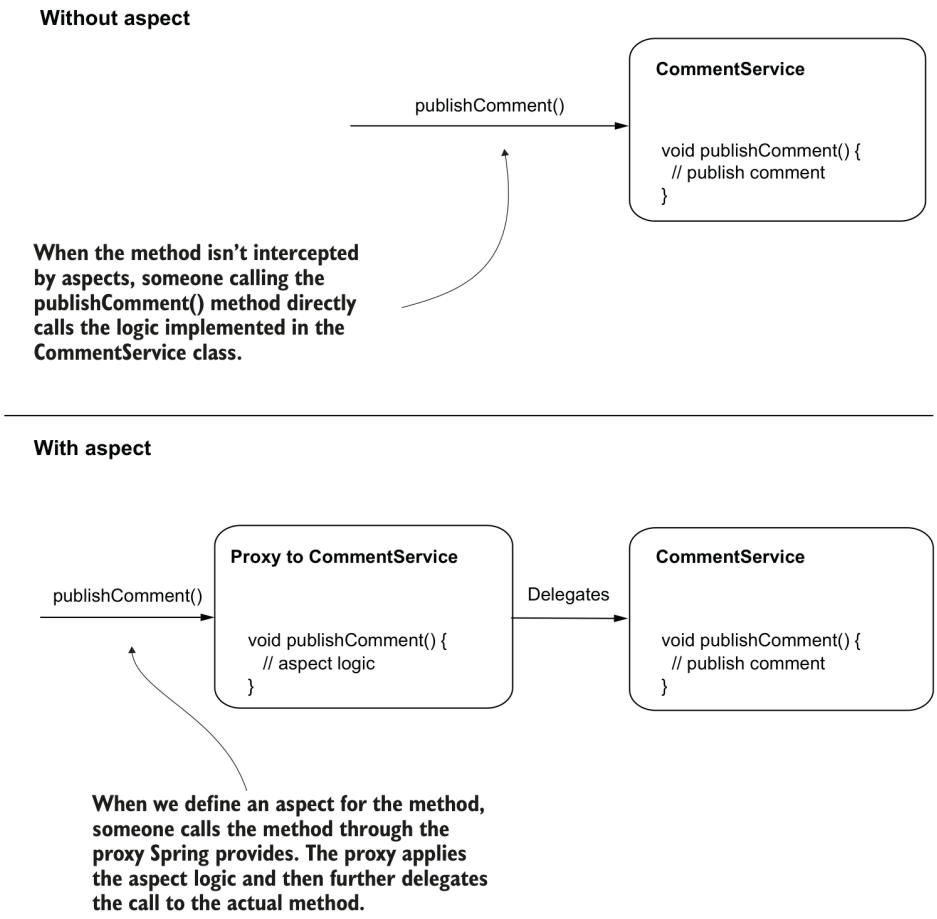
Hình 6.3: Quá trình đan dệt (*weaving*) một khía cạnh. Thay vì trao cho bạn tham chiếu tới bean thực tế, Spring cung cấp cho bạn một tham chiếu tới đối tượng ủy quyền (*proxy object*) để đánh chặn các lời gọi phương thức và điều phối logic của khía cạnh.

Nếu bạn thử in ra kiểu lớp (*class type*) của đối tượng nhận được từ ngữ cảnh bằng lệnh `System.out.println(service.getClass())`, bạn sẽ thấy rằng nó không đơn thuần là `CommentService`, mà tên lớp của nó sẽ kèm theo các hậu tố do thư viện CGLIB của Spring sinh động, tương tự như:

```
class services.CommentService$$EnhancerBySpringCGLIB$$12491022
```

Từ khóa `EnhancerBySpring` xuất hiện trong tên lớp là bằng chứng rõ ràng cho thấy đây là một đối tượng proxy được Spring tự động sinh ra để "tăng cường" khả năng cho bean gốc.

Trên Hình 6.4, bạn có thể so sánh trực quan sự khác biệt giữa hai trường hợp: gọi phương thức thông thường không có khía cạnh và gọi phương thức đã được đan dệt khía cạnh. Bạn sẽ thấy rằng việc gọi một phương thức có khía cạnh đồng nghĩa với việc bạn gửi lời gọi xuyên qua đối tượng proxy do Spring cung cấp. Proxy sẽ thực thi logic của khía cạnh rồi mới ủy thác lời gọi tới phương thức thực sự bên trong đối tượng gốc.



Hình 6.4: So sánh giữa việc gọi phương thức không có khía cạnh và có khía cạnh. Khi không có khía cạnh, lời gọi đi thẳng vào phương thức nghiệp vụ của lớp `CommentService`. Khi có khía cạnh can thiệp, lời gọi phải đi qua đối tượng proxy; proxy áp dụng logic của aspect rồi mới chuyển tiếp tới phương thức thực tế.

6.2 Triển khai các khía cạnh với Spring AOP

Bây giờ bạn đã nắm được bức tranh tổng quan về khía cạnh và cách Spring điều phối chúng thông qua đối tượng ủy quyền (proxy). Tiếp theo, chúng ta sẽ bước vào phần thực hành các cú pháp để triển khai khía cạnh trong các ứng dụng thực tế.

Hãy xem xét kịch bản nghiệp vụ sau: Giả sử bạn đang phát triển một hệ thống gồm nhiều trường hợp sử dụng (*use cases*) trong các lớp dịch vụ. Do một quy định bảo mật và kiểm toán mới được ban hành, hệ thống bắt buộc phải ghi lại nhật ký (*log*) chính xác thời điểm bắt đầu và kết thúc của từng trường hợp sử dụng khi thực thi. Trong nhóm phát triển, bạn nhận trách nhiệm xây dựng tính năng ghi log cho toàn bộ các sự kiện bắt đầu và kết thúc này.

Trong Mục 6.2.1, chúng ta sẽ sử dụng một khía cạnh để giải quyết kịch bản này theo cách thức đơn giản và trực tiếp nhất. Tiếp theo, trong Mục 6.2.2, chúng ta sẽ tìm hiểu cách một khía cạnh truy cập và thậm chí can thiệp sửa đổi các tham số đầu vào cũng như giá trị trả

về của phương thức bị đánh chặn. Trong Mục 6.2.3, bạn sẽ học cách dùng các chú thích tùy biến (*custom annotations*) để đánh dấu các phương thức cần can thiệp — đây là mẫu thiết kế được các lập trình viên chuyên nghiệp áp dụng phổ biến nhất. Cuối cùng, Mục 6.2.4 sẽ giới thiệu các loại chú thích advice khác mà Spring cung cấp.

6.2.1 Triển khai một khía cạnh đơn giản

Chúng ta sẽ khởi tạo một dự án mới mang tên `sq-ch6-ex1`. Bên cạnh phụ thuộc cơ bản `spring-context`, dự án này cần thêm phụ thuộc `spring-aspects` để kích hoạt cơ chế AOP. Hãy bổ sung hai khối phụ thuộc sau vào tệp `pom.xml` của bạn:

```
<dependency>
   <groupId>org.springframework</groupId>
   <artifactId>spring-context</artifactId>
   <version>5.2.8.RELEASE</version>
</dependency>
<dependency>
   <groupId>org.springframework</groupId>
   <artifactId>spring-aspects</artifactId>
   <version>5.2.8.RELEASE</version>
</dependency>
```

Để giữ cho ví dụ gọn gàng và giúp bạn tập trung tối đa vào cú pháp khía cạnh, chúng ta chỉ cần một đối tượng mô hình đại diện cho dữ liệu bình luận mang tên `Comment`:

```
public class Comment {
  private String text;
  private String author;

  // Cac phuong thuc getter va setter...
}
```

Trong Listing 6.1, chúng ta định nghĩa lớp dịch vụ `CommentService`. Chúng ta đánh dấu lớp này bằng chú thích định hình `@Service` để đưa nó vào làm bean trong ngữ cảnh Spring. Phương thức `publishComment(Comment comment)` đại diện cho trường hợp sử dụng mà chúng ta cần ghi log.

```
@Service
public class CommentService {

  private Logger logger =
    Logger.getLogger(CommentService.class.getName());

```

```
  public void publishComment(Comment comment) {
    logger.info("Dang xuat ban binh luan: " + comment.getText());
  }
}
```

Listing 6.1: Lớp dịch vụ `CommentService` đại diện cho trường hợp sử dụng nghiệp vụ

Ghi chú về thư viện Logging trong ứng dụng thực tế

Trong ví dụ này, thay vì sử dụng lệnh in thông thường `System.out.println`, tôi dùng đối tượng `Logger` thuộc gói `java.util.logging` của JDK. Trong các ứng dụng thực tế trên môi trường sản xuất, bạn không bao giờ dùng `System.out` để in nhật ký ra console. Thay vào đó, bạn sẽ sử dụng các bộ khung ghi log chuyên nghiệp như **Logback**, **Log4j2**, hoặc **SLF4J** để có khả năng tùy biến định dạng ngày giờ, cấp độ log (*INFO, WARN, ERROR*) và chuyển hướng log ra tệp lưu trữ.

Lớp cấu hình `ProjectConfig` được thiết lập để quét gói `"services"`:

```
@Configuration
@ComponentScan(basePackages = "services")
public class ProjectConfig {
}
```

Lớp `Main` (Listing 6.2) khởi tạo ngữ cảnh, lấy bean `CommentService` và gọi phương thức nghiệp vụ để kiểm tra trạng thái ban đầu của ứng dụng khi chưa có khía cạnh can thiệp:

```
public class Main {
  public static void main(String[] args) {
    var c = new AnnotationConfigApplicationContext(ProjectConfig.class);
    var service = c.getBean(CommentService.class);

    Comment comment = new Comment();
    comment.setText("Binh luan thu nghiem");
    comment.setAuthor("Natasha");

    service.publishComment(comment);
  }
}
```

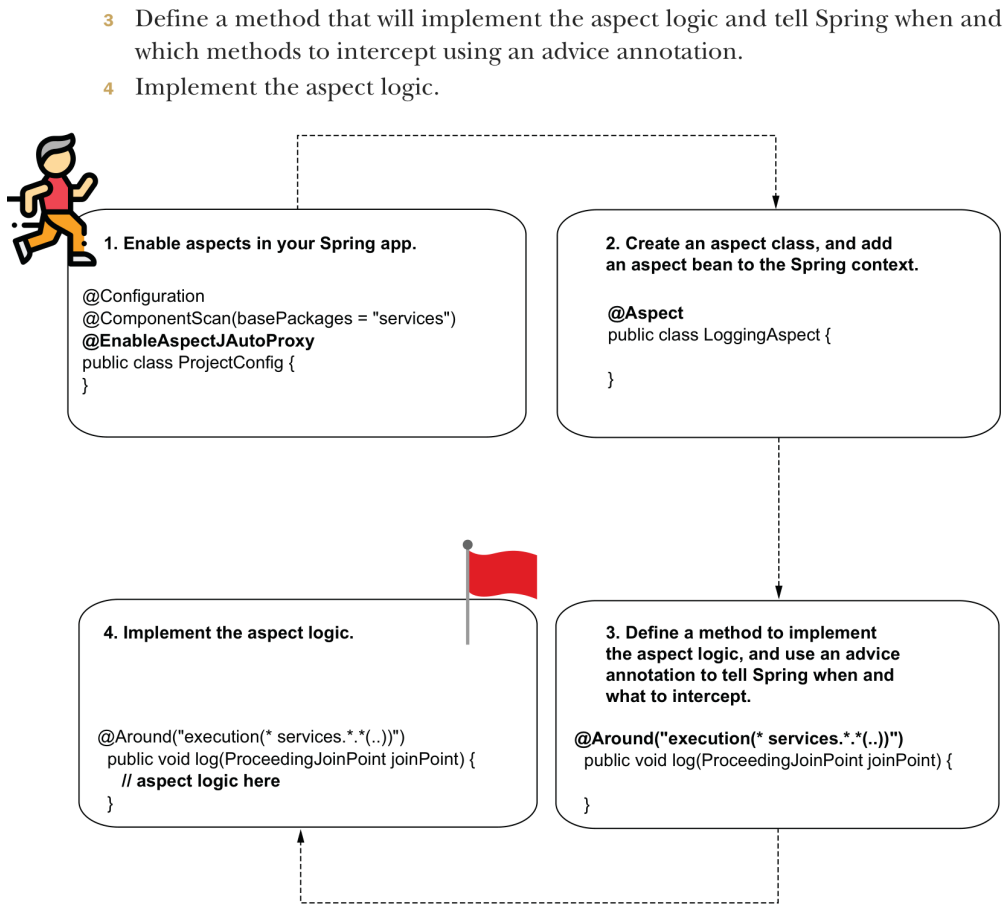
Listing 6.2: Lớp `Main` kiểm tra hành vi của ứng dụng trước khi áp dụng khía cạnh

Khi chạy chương trình, trên màn hình console chỉ xuất hiện đúng một dòng thông báo phát ra từ chính phương thức `publishComment()`:

```
INFO: Dang xuat ban binh luan: Binh luan thu nghiem
```

Bây giờ, chúng ta sẽ bắt đầu xây dựng một khía cạnh nhằm tự động chèn thêm thông báo trước và sau khi phương thức này chạy mà không cần sửa đổi bất kỳ dòng mã nào bên trong `CommentService`. Để tạo một khía cạnh trong Spring, bạn thực hiện theo 4 bước chuẩn hóa (Hình 6.5):

1. **Bước 1:** Kích hoạt cơ chế AOP trong ứng dụng Spring bằng cách gắn chú thích `@EnableAspectJAutoPr` lên lớp cấu hình.
2. **Bước 2:** Tạo một lớp Java mới, đánh dấu bằng chú thích `@Aspect`, đồng thời đăng ký lớp này làm một bean trong ngữ cảnh Spring.
3. **Bước 3:** Định nghĩa một phương thức bên trong lớp khía cạnh để chứa logic can thiệp, sử dụng một chú thích advice để thông báo cho Spring biết thời điểm và phương thức cần đánh chặn.
4. **Bước 4:** Hiện thực hóa logic của khía cạnh.



Hình 6.5: Quy trình 4 bước để triển khai một khía cạnh trong Spring: (1) Bật tính năng AOP bằng `@EnableAspectJAutoProxy`, (2) Tạo lớp có `@Aspect` và đăng ký bean, (3) Định nghĩa phương thức can thiệp kèm chú thích advice chỉ định điểm cắt, (4) Viết mã logic của khía cạnh.

Bước 1: Kích hoạt cơ chế AOP trong ứng dụng Bất cứ khi nào bạn muốn sử dụng một tính năng bổ trợ của Spring, bạn luôn phải bật tính năng đó tường minh trên lớp cấu hình.

Các chú thích bật tính năng trong Spring hầu hết đều bắt đầu bằng tiền tố `@Enable`. Ở đây, chúng ta sử dụng `@EnableAspectJAutoProxy` (Listing 6.3):

```
@Configuration
@ComponentScan(basePackages = "services")
@EnableAspectJAutoProxy
public class ProjectConfig {
}
```

Listing 6.3: Kích hoạt cơ chế AOP trên lớp cấu hình bằng `@EnableAspectJAutoProxy`

Bước 2: Tạo lớp định nghĩa khía cạnh và đăng ký bean Chúng ta tạo lớp `LoggingAspect` và đánh dấu bằng chú thích `@Aspect` (Listing 6.4):

```
@Aspect
public class LoggingAspect {

  public void log() {
    // Se cai dat logic o cac buoc tiep theo
  }
}
```

Listing 6.4: Định nghĩa lớp khía cạnh với chú thích `@Aspect`

Cạm bẫy hay gặp: @Aspect không phải là một Stereotype Annotation!

Chú thích `@Aspect` chỉ đóng vai trò khai báo với Spring rằng lớp này chứa các định nghĩa của một khía cạnh. **Bản thân chú thích `@Aspect` không tự động biến lớp đó thành một bean trong ngữ cảnh!** Bạn bắt buộc phải tự tay đăng ký nó thành một bean, bằng cách dùng chú thích `@Component` trên lớp hoặc dùng phương thức `@Bean` trong lớp cấu hình `ProjectConfig`:

```
@Configuration
@ComponentScan(basePackages = "services")
@EnableAspectJAutoProxy
public class ProjectConfig {

  @Bean
  public LoggingAspect loggingAspect() {
    return new LoggingAspect();
  }
}
```

Quên đăng ký bean cho lớp khía cạnh là một trong những lỗi sơ đẳng phổ biến nhất khiến khía cạnh không bao giờ được kích hoạt!

Bước 3: Sử dụng chú thích advice để chỉ định thời điểm và phương thức cần đánh chặn

Chúng ta gắn chú thích `@Around` lên phương thức `log()` bên trong lớp khía cạnh (Listing 6.5):

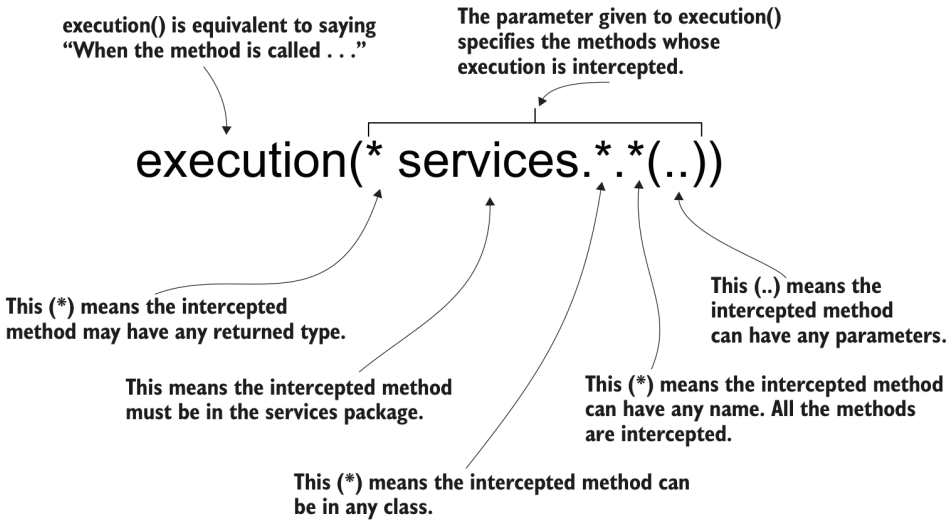
```
@Aspect
public class LoggingAspect {

  @Around("execution(* services.*.*(..))")
  public void log(ProceedingJoinPoint joinPoint) throws Throwable {
    joinPoint.proceed();
  }
}
```

Listing 6.5: Sử dụng chú thích advice `@Around` và biểu thức điểm cắt AspectJ

Ở đoạn mã trên, chuỗi ký tự bên trong `@Around("execution(* services.*.*(..))")` được gọi là **ngôn ngữ biểu thức điểm cắt AspectJ** (*AspectJ pointcut expression*). Biểu thức này thông báo cho Spring biết chính xác những phương thức nào cần bị đánh chặn. Hãy quan sát Hình 6.6 để hiểu rõ cấu trúc của biểu thức này:

- `execution(...)`: Chỉ thị đánh chặn khi phương thức được thực thi.

- Dấu hoa thị thứ nhất (*): Phương thức bị đánh chặn có thể có **bất kỳ kiểu giá trị trả về nào**.
- `services.*`: Phương thức phải thuộc về **bất kỳ lớp nào nằm trong gói `services`**.
- Dấu hoa thị thứ hai (`.*`): Phương thức có thể mang **bất kỳ tên gọi nào**.
- Dấu hai chấm trong ngoặc (`..`): Phương thức có thể tiếp nhận **bất kỳ danh sách tham số nào**.



Hình 6.6: Cấu trúc biểu thức điểm cắt AspectJ: Chỉ thị cho Spring đánh chặn tất cả các phương thức nằm trong gói `services`, không phân biệt kiểu trả về, tên lớp, tên phương thức hay kiểu tham số đầu vào.

Phần tử thứ hai cần lưu ý là tham số `ProceedingJoinPoint` được truyền vào phương thức `log()`. Tham số này đại diện cho chính phương thức đang bị đánh chặn. Mục đích chính của việc sử dụng đối tượng này là để khía cạnh ra lệnh cho Spring tiếp tục chuyển tiếp lời gọi tới phương thức thực tế thông qua phương thức `joinPoint.proceed()`.

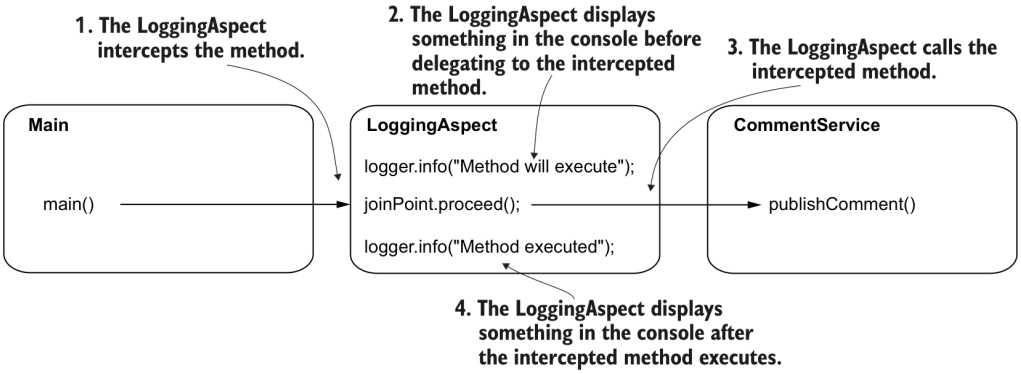
Bước 4: Cài đặt logic của khía cạnh Trong Listing 6.6, chúng ta bổ sung logic hoàn chỉnh: in một dòng log trước khi gọi `proceed()`, gọi phương thức thực tế, rồi in tiếp một dòng log sau khi phương thức thực tế kết thúc.

```
@Aspect
public class LoggingAspect {

  private Logger logger = Logger.getLogger(LoggingAspect.class.getName()
   );

  @Around("execution(* services.*.*(..))")
  public void log(ProceedingJoinPoint joinPoint) throws Throwable {
    logger.info("Phuong thuc chuan bi thuc thi");
```

```
9       joinPoint.proceed();
10      logger.info("Phuong thuc da thuc thi xong");
11    }
12  }
```

Listing 6.6: Cài đặt logic hoàn chỉnh của khía cạnh ghi nhật ký (dự án `sq-ch6-ex1`)

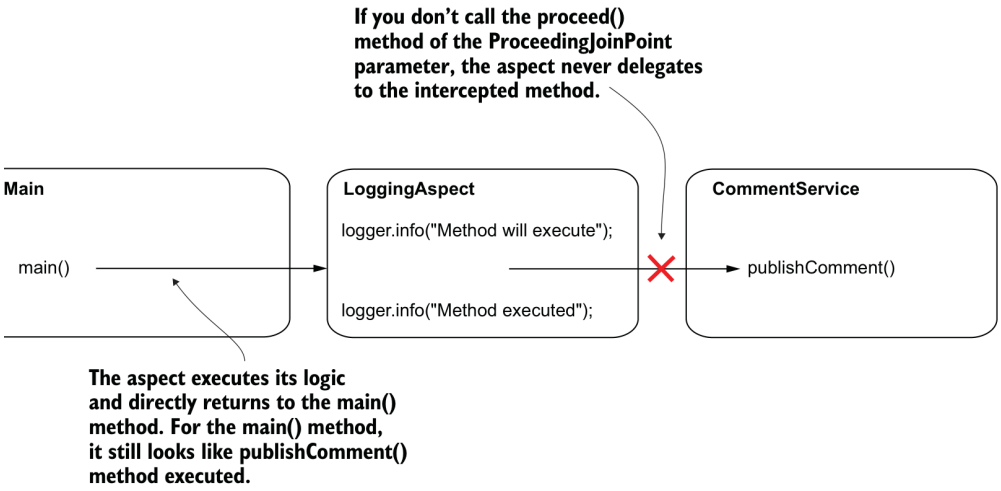
Hình 6.7 minh họa toàn bộ luồng điều khiển khi khía cạnh này vận hành.



Hình 6.7: Hành vi can thiệp của `LoggingAspect`: Khía cạnh bọc ngoài lời gọi phương thức `publishComment()`, tự động chèn thêm thông báo trước và sau khi ủy thác lời gọi cho phương thức thực tế.

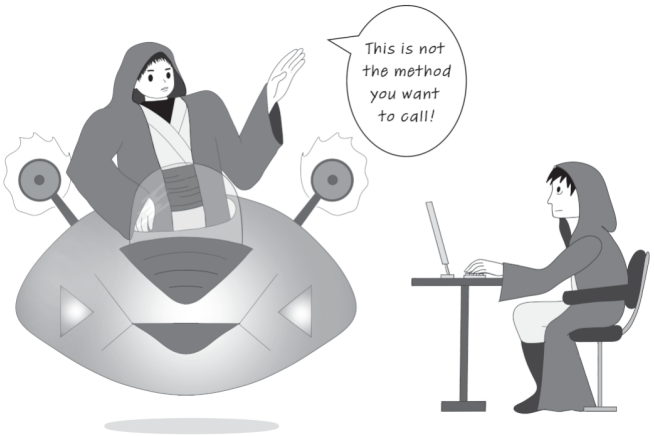
Lưu ý rằng phương thức `proceed()` có thể ném ra một ngoại lệ kiểu `Throwable` — đây là cơ chế giúp ném lại bất kỳ ngoại lệ nào phát sinh từ chính phương thức nghiệp vụ thực tế. Ở đây chúng ta chọn cách khai báo `throws Throwable` ở chữ ký phương thức để lan truyền ngoại lệ ra ngoài.

Chuyện gì sẽ xảy ra nếu bạn **không gọi** `joinPoint.proceed()`? Nếu bạn bỏ qua lệnh gọi này, khía cạnh sẽ không bao giờ chuyển tiếp lời gọi tới phương thức nghiệp vụ thực tế! Trong trường hợp đó, phương thức gốc sẽ hoàn toàn bị vô hiệu hóa, và đối tượng gọi bên ngoài (như phương thức `main()`) hoàn toàn không hay biết gì về sự việc này (Hình 6.8).



Hình 6.8: Nếu khía cạnh không gọi `joinPoint.proceed()`, phương thức nghiệp vụ thực tế sẽ không bao giờ được thực thi. Khía cạnh chạy xong logic của nó rồi lập tức trả kết quả về cho bên gọi.

Khả năng này thoạt nghe có vẻ nguy hiểm, nhưng trong thực tế nó lại là một tính năng cực kỳ đắc lực. Ví dụ kinh điển là một khía cạnh kiểm tra quyền truy cập (*authorization*): nếu người dùng không đủ quyền hạn, khía cạnh sẽ chặn đứng lời gọi ngay lập tức mà không cho phép chạm tới phương thức bảo mật phía sau (Hình 6.9).



Hình 6.9: Khía cạnh có thể quyết định không ủy thác lời gọi cho phương thức bị can thiệp. Hành vi này giống như một "thuật thôi miên" đối với bên gọi, khiến bên gọi tưởng rằng phương thức đã chạy trong khi thực tế một logic hoàn toàn khác đã được áp dụng.

Bây giờ, hãy chạy lại ứng dụng `Main`. Kết quả hiển thị trên màn hình console sẽ phản ánh đầy đủ cả ba giai đoạn:

```
INFO: Phuong thuc chuan bi thuc thi
INFO: Dang xuat ban binh luan: Binh luan thu nghiem
INFO: Phuong thuc da thuc thi xong
```

Chúng ta đã giải quyết trọn vẹn yêu cầu bài toán ghi log mà không cần chỉnh sửa hay làm vấy bẩn bất kỳ dòng mã nghiệp vụ nào trong lớp `CommentService`!

6.2.2 Thay đổi tham số và giá trị trả về của phương thức bị chặn

Khía cạnh không chỉ dừng lại ở việc quan sát và ghi chép. Chúng còn sở hữu sức mạnh can thiệp sâu hơn: **đọc và thay đổi các tham số truyền vào, cũng như đón bắt và thay đổi giá trị trả về của phương thức bị đánh chặn**.

Hãy xem xét kịch bản trong dự án mẫu `sq-ch6-ex2`. Chúng ta nâng cấp phương thức `publishComment()` để nó trả về một chuỗi kết quả:

```
@Service
public class CommentService {
  private Logger logger = Logger.getLogger(CommentService.class.getName
    ());

  public String publishComment(Comment comment) {
    logger.info("Dang xuat ban binh luan: " + comment.getText());
    return "SUCCESS";
  }
}
```

Thông qua đối tượng `ProceedingJoinPoint`, khía cạnh có thể dễ dàng trích xuất tên phương thức và mảng các tham số đầu vào:

```
String methodName = joinPoint.getSignature().getName();
Object[] arguments = joinPoint.getArgs();
```

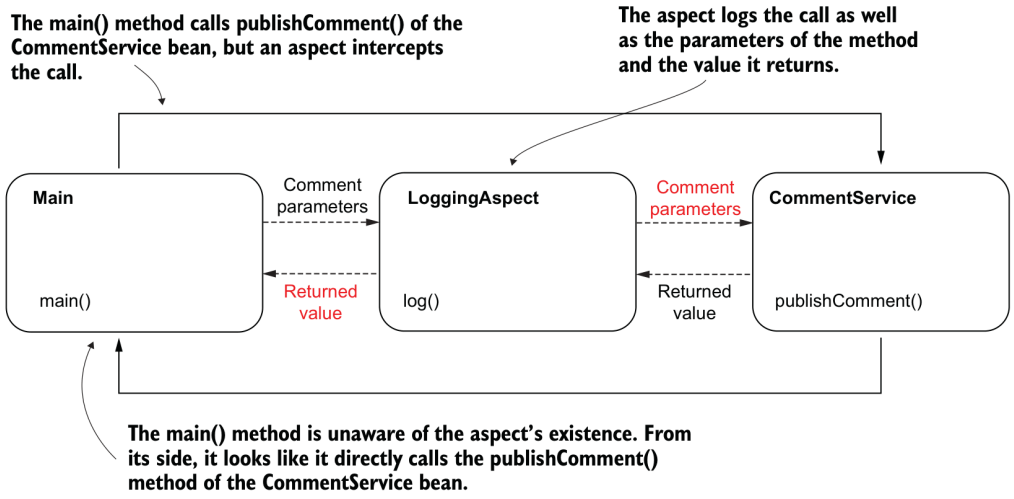
Trong Listing 6.7, chúng ta nâng cấp phương thức của `LoggingAspect` để vừa ghi log tên phương thức và danh sách tham số, vừa nhận giá trị trả về từ lời gọi `joinPoint.proceed()` và trả lại kết quả đó:

```
@Aspect
public class LoggingAspect {

  private Logger logger = Logger.getLogger(LoggingAspect.class.getName()
    );

  @Around("execution(* services.*.*(..))")
  public Object log(ProceedingJoinPoint joinPoint) throws Throwable {
    String methodName = joinPoint.getSignature().getName();
    Object[] arguments = joinPoint.getArgs();
```

```
10
11      logger.info("Phuong thuc " + methodName +
12          " voi tham so " + Arrays.asList(arguments) + " chuan bi chay");
13
14      Object returnedByMethod = joinPoint.proceed();
15
16      logger.info("Phuong thuc da chay xong va tra ve ket qua: " +
        returnedByMethod);
17
18      return returnedByMethod;
19    }
20  }
```

Listing 6.7: Đọc tên phương thức, tham số và giá trị trả về trong khía cạnh (dự án `sq-ch6-ex2`)

Hình 6.10 mô tả trực quan luồng dữ liệu luân chuyển giữa `Main`, `LoggingAspect` và `CommentService`.



Hình 6.10: Khía cạnh chặn lời gọi phương thức, truy cập vào tham số truyền vào và đón nhận giá trị trả về sau khi phương thức thực tế kết thúc. Đối với phương thức `main()`, nó hoàn toàn không hay biết có một khía cạnh đang đứng ở giữa.

Trong lớp `Main` (Listing 6.8), chúng ta ghi log giá trị mà phương thức `publishComment()` trả về:

```
1  public class Main {
2    private static Logger logger = Logger.getLogger(Main.class.getName());
3
4    public static void main(String[] args) {
5      var c = new AnnotationConfigApplicationContext(ProjectConfig.class);
6      var service = c.getBean(CommentService.class);
7
```

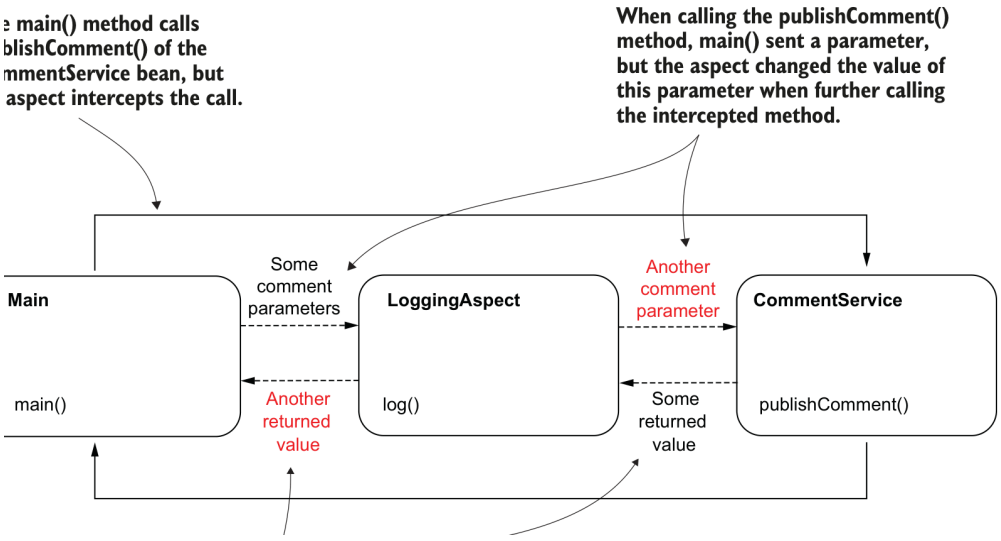
```
    Comment comment = new Comment();
    comment.setText("Binh luan thu nghiem");
    comment.setAuthor("Natasha");

    String value = service.publishComment(comment);
    logger.info("Ket qua nhan duoc tai Main: " + value);
  }
}
```

Listing 6.8: Lớp `Main` nhận giá trị trả về từ phương thức nghiệp vụ

Chạy ứng dụng, bạn sẽ thấy toàn bộ thông tin chi tiết về tham số và kết quả trả về xuất hiện trên console:

```
INFO: Phuong thuc publishComment voi tham so [Comment{text='Binh luan
    thu nghiem', author='Natasha'}] chuan bi chay
INFO: Dang xuat ban binh luan: Binh luan thu nghiem
INFO: Phuong thuc da chay xong va tra ve ket qua: SUCCESS
INFO: Ket qua nhan duoc tai Main: SUCCESS
```

Chưa dừng lại ở đó, khía cạnh thậm chí còn có thể **thay thế hoàn toàn tham số truyền vào bằng một đối tượng khác**, hoặc **thay thế giá trị trả về thành một giá trị khác** (Hình 6.11).



Hình 6.11: Khía cạnh có thể đánh tráo tham số gửi tới phương thức thực tế, đồng thời biến đổi giá trị trả về trước khi trao lại cho bên gọi. Khả năng này mang lại sự linh hoạt tuyệt đối trong việc kiểm soát hành vi ứng dụng.

Hãy quan sát dự án mẫu `sq-ch6-ex3` trong Listing 6.9. Khi gọi `joinPoint.proceed(newArguments)`, chúng ta truyền vào một mảng tham số mới chứa một đối tượng `Comment` hoàn toàn khác. Đồng thời, thay vì trả về `SUCCESS`, khía cạnh chủ động trả về chuỗi `"FAILED"`:

```java
@Aspect
public class LoggingAspect {

  private Logger logger = Logger.getLogger(LoggingAspect.class.getName()
    );

  @Around("execution(* services.*.*(..))")
  public Object log(ProceedingJoinPoint joinPoint) throws Throwable {
    String methodName = joinPoint.getSignature().getName();
    Object[] arguments = joinPoint.getArgs();

    logger.info("Phuong thuc " + methodName + " chuan bi chay");

    // Tao mot doi tuong binh luan moi de danh trao
    Comment comment = new Comment();
    comment.setText("Noi dung binh luan da bi thay doi boi Aspect!");
    Object[] newArguments = {comment};

    // Chuyen tiep tham so moi vao phuong thuc thuc te
    Object returnedByMethod = joinPoint.proceed(newArguments);

    logger.info("Phuong thuc thuc te tra ve: " + returnedByMethod);

    // Tra ve ket qua gia mao cho ben goi
    return "FAILED";
  }
}
```

Listing 6.9: Đánh tráo tham số và thay đổi giá trị trả về trong khía cạnh (dự án `sq-ch6-ex3`)

Khi chạy chương trình, phương thức `publishComment()` sẽ in ra nội dung đã bị sửa đổi:

```
INFO: Dang xuat ban binh luan: Noi dung binh luan da bi thay doi boi
   Aspect!
INFO: Phuong thuc thuc te tra ve: SUCCESS
INFO: Ket qua nhan duoc tai Main: FAILED
```

Lời cảnh báo nghiêm túc về việc can thiệp logic qua Aspect

Tôi phải nhắc lại điều này vì nó tối quan trọng: **Hãy sử dụng khía cạnh một cách có trách nhiệm!** Bạn chỉ nên dùng aspect để tách rời những dòng mã phụ trợ mang tính thủ tục kỹ thuật (như đo thời gian thực thi, kiểm tra bảo mật, ghi nhật ký) mà bất kỳ ai đọc mã nguồn cũng có thể dễ dàng suy đoán được. Tuyệt đối không dùng khía cạnh để thay đổi các logic nghiệp vụ quan trọng một cách âm thầm, bởi điều đó sẽ khiến ứng dụng trở thành một "hộp đen ma thuật", cực kỳ khó khăn cho các đồng nghiệp khác khi đọc và bảo trì hệ thống.

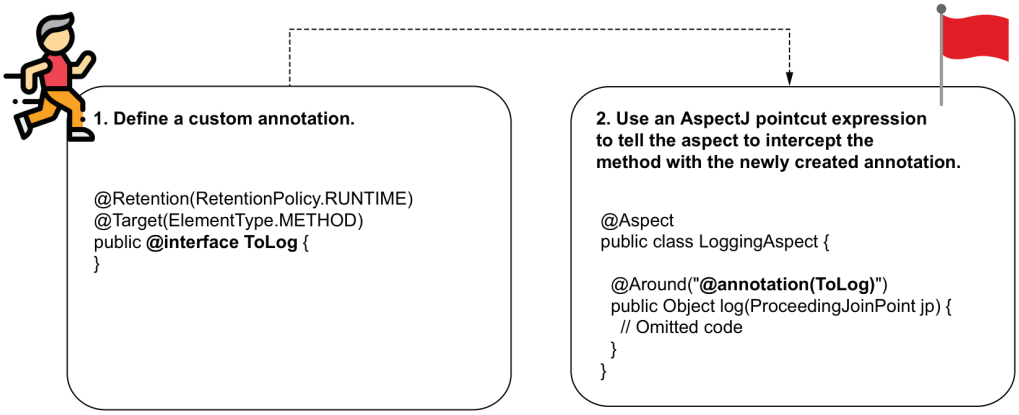
6.2.3 Đan dệt khía cạnh vào các phương thức được gắn chú thích

Trong các dự án thực tế, thay vì phải viết các biểu thức điểm cắt AspectJ dài dòng và dễ vỡ theo tên gói mã nguồn, phương thức ưa chuộng nhất của các lập trình viên Spring là: **Sử dụng một chú thích tùy biến (*custom annotation*) để đánh dấu các phương thức cần can thiệp**.

Giả sử trong lớp `CommentService`, chúng ta có 3 phương thức: `publishComment()`, `deleteComment()`, và `editComment()` (dự án mẫu `sq-ch6-ex4`). Yêu cầu đặt ra là chúng ta chỉ muốn ghi log duy nhất phương thức `deleteComment()`.

Để làm được điều đó, chúng ta thực hiện theo 2 bước tinh gọn (Hình 6.12):

1. Định nghĩa một chú thích tùy biến mang tên `@ToLog`, thiết lập chính sách lưu giữ ở thời điểm chạy (*runtime*).
2. Sử dụng biểu thức điểm cắt `@annotation(ToLog)` bên trong lớp khía cạnh để chỉ thị cho Spring đánh chặn bất kỳ phương thức nào có gắn chú thích này.



Hình 6.12: Quy trình đánh chặn các phương thức được gắn chú thích: (1) Tạo chú thích tùy biến với chính sách lưu giữ `RUNTIME`, (2) Cấu hình biểu thức điểm cắt AspectJ dạng `@annotation(...)` để chỉ can thiệp vào các phương thức có chú thích tương ứng.

Định nghĩa của chú thích tùy biến `ToLog` vô cùng đơn giản:

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface ToLog {
}
```

Hai chú thích siêu dữ liệu (*meta-annotations*) ở trên có ý nghĩa sống còn:

- `@Retention(RetentionPolicy.RUNTIME)`: Bắt buộc phải có để chú thích được lưu giữ trong mã byte và có thể truy xuất được tại thời điểm ứng dụng đang chạy. Nếu thiếu dòng này, chú thích sẽ bị trình biên dịch loại bỏ sau khi dịch mã nguồn.
- `@Target(ElementType.METHOD)`: Giới hạn phạm vi sử dụng của chú thích chỉ được phép đặt trên các phương thức.

Trong Listing 6.10, chúng ta chỉ gắn chú thích `@ToLog` duy nhất lên phương thức `deleteComment()`:

```
@Service
public class CommentService {
  private Logger logger = Logger.getLogger(CommentService.class.getName
    ());

  public void publishComment(Comment comment) {
    logger.info("Xuat ban binh luan: " + comment.getText());
  }

  @ToLog
  public void deleteComment(Comment comment) {
    logger.info("Xoa binh luan: " + comment.getText());
  }

  public void editComment(Comment comment) {
    logger.info("Chinh sua binh luan: " + comment.getText());
  }
}
```

Listing 6.10: Đánh dấu phương thức cần can thiệp bằng chú thích tùy biến `@ToLog`

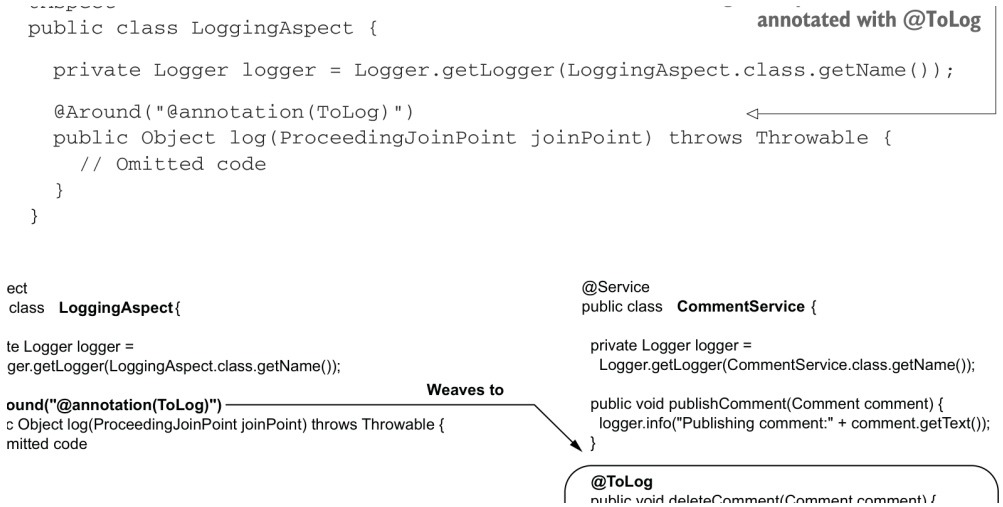
Tiếp theo, trong lớp `LoggingAspect` (Listing 6.11), chúng ta thay đổi giá trị của biểu thức điểm cắt thành `@annotation(ToLog)`:

```
@Aspect
public class LoggingAspect {

  private Logger logger = Logger.getLogger(LoggingAspect.class.getName()
    );
```

```
5
6   @Around("@annotation(ToLog)")
7   public Object log(ProceedingJoinPoint joinPoint) throws Throwable {
8     logger.info("Logging Aspect: Phuong thuc danh dau @ToLog chuan bi
      thuc thi");
9     Object returnedValue = joinPoint.proceed();
10    logger.info("Logging Aspect: Phuong thuc danh dau @ToLog da thuc thi
      xong");
11    return returnedValue;
12  }
13 }
```

Listing 6.11: Sử dụng biểu thức `@annotation(ToLog)` để đan dệt khía cạnh

Hình 6.13 minh họa cơ chế đan dệt này: khía cạnh sẽ tự động bỏ qua hai phương thức `publishComment` và `editComment`, chỉ tập trung đánh chặn duy nhất phương thức `deleteComment`.



Hình 6.13: Sử dụng biểu thức `@annotation(ToLog)`, chúng ta đan dệt logic của khía cạnh vào bất kỳ phương thức nào được đánh dấu bằng chú thích tùy biến. Đây là cách làm thanh thoát và an toàn nhất trong kiến trúc ứng dụng Spring hiện đại.

Khi bạn chạy thử nghiệm gọi cả 3 phương thức trong hàm `main()`, màn hình console sẽ xác nhận rằng chỉ có duy nhất phương thức `deleteComment()` được khía cạnh ghi log bọc ngoài!

6.2.4 Các chú thích can thiệp (Advice) khác trong Spring

Xuyên suốt từ đầu chương đến giờ, chúng ta chủ yếu sử dụng chú thích can thiệp `@Around`. Đây quả thực là chú thích phổ biến và quyền năng nhất trong Spring AOP vì nó bao quát mọi kịch bản: bạn có thể chạy mã trước, sau, hoặc thậm chí hủy bỏ lời gọi phương thức thực tế.

Tuy nhiên, trong các bài toán đơn giản, việc sử dụng `@Around` đôi khi lại quá cồng kềnh. Để giữ cho mã nguồn trong sáng và súc tích, Spring cung cấp thêm 4 chú thích advice chuyên biệt hơn:

- `@Before`: Thực thi logic của khía cạnh **ngay trước** khi phương thức bị can thiệp được gọi.
- `@AfterReturning`: Thực thi logic của khía cạnh **sau khi phương thức bị can thiệp kết thúc thành công** (trả về kết quả bình thường mà không ném ra ngoại lệ). Bạn có thể đón nhận giá trị trả về này thông qua thuộc tính `returning`.
- `@AfterThrowing`: Thực thi logic của khía cạnh **khi phương thức bị can thiệp ném ra một ngoại lệ**. Bạn có thể đón nhận thể hiện ngoại lệ thông qua thuộc tính `throwing`.
- `@After`: Thực thi logic của khía cạnh **ngay sau khi phương thức kết thúc**, bất kể phương thức đó kết thúc thành công hay gặp ngoại lệ (tương tự như khối lệnh `finally` trong cấu trúc `try-catch-finally` của Java).

Khác với `@Around`, các phương thức được đánh dấu bằng 4 chú thích này **không tiếp nhận tham số `ProceedingJoinPoint`** và không có quyền tự quyết định thời điểm ủy thác cho phương thức gốc, bởi vì thời điểm ủy thác đã được mặc định gắn chặt với ý nghĩa của từng chú thích.

Trong dự án mẫu `sq-ch6-ex5`, chúng ta minh họa việc sử dụng `@AfterReturning` để đón nhận kết quả trả về:

```
@Aspect
public class LoggingAspect {

  private Logger logger = Logger.getLogger(LoggingAspect.class.getName()
   );

  @AfterReturning(value = "@annotation(ToLog)", returning = "
   returnedValue")
  public void log(Object returnedValue) {
    logger.info("Phuong thuc da ket thuc thanh cong va tra ve: " +
   returnedValue);
  }
}
```

Thuộc tính `returning = "returnedValue"` chỉ thị cho Spring ánh xạ giá trị trả về của phương thức nghiệp vụ trực tiếp vào tham số có tên tương ứng `returnedValue` của phương thức `log()`.

6.3 Chuỗi thực thi các khía cạnh (Aspect Execution Chain)

Trong các ví dụ trước, chúng ta mới chỉ xem xét trường hợp một phương thức bị can thiệp bởi một khía cạnh duy nhất. Nhưng trong một dự án doanh nghiệp thực tế, một phương thức thường xuyên phải chịu sự can thiệp của **nhiều khía cạnh cùng lúc**.

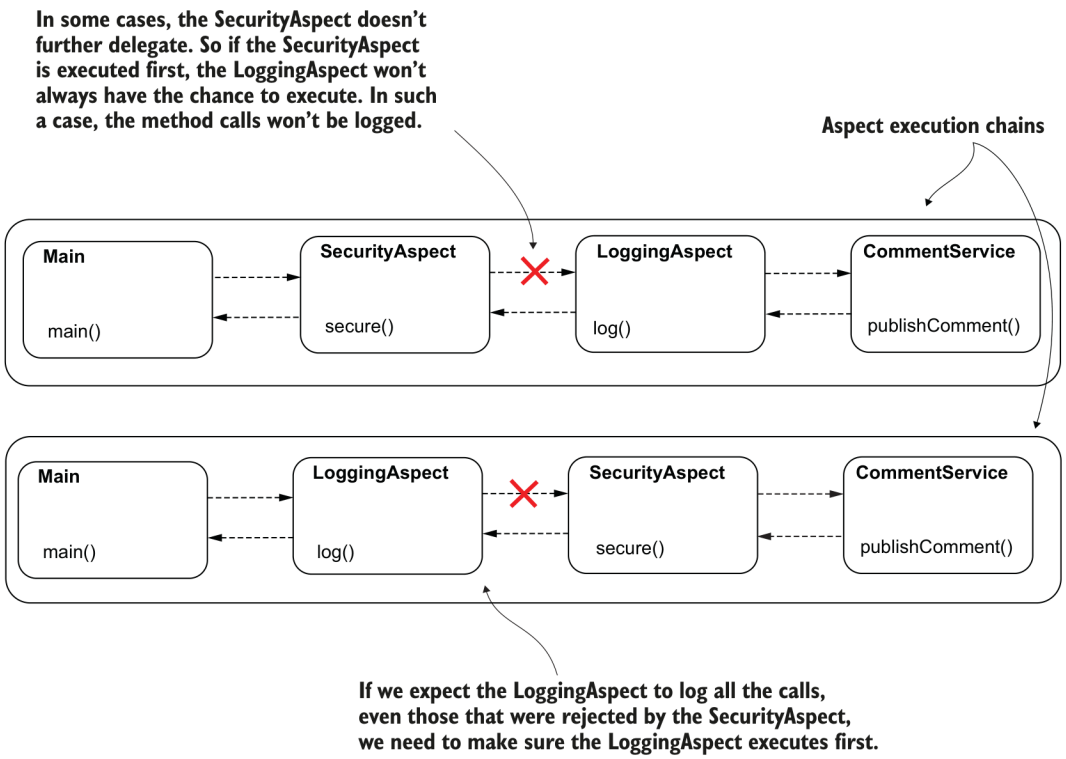
Chẳng hạn, đối với cùng một phương thức nghiệp vụ, chúng ta vừa muốn áp dụng khía cạnh kiểm tra bảo mật (`SecurityAspect`), vừa muốn áp dụng khía cạnh ghi nhật ký (`LoggingAspect`). Khi xuất hiện nhiều khía cạnh trên cùng một điểm cắt, hai câu hỏi cốt tử được đặt ra là:

1. **Spring thực thi các khía cạnh này theo thứ tự nào?**
2. **Thứ tự thực thi giữa các khía cạnh có thực sự quan trọng không?**

Thứ tự thực thi có thực sự quan trọng? Câu trả lời là: **Cực kỳ quan trọng!** Việc thực thi các khía cạnh theo những thứ tự khác nhau có thể mang lại kết quả hoàn toàn trái ngược nhau.

Hãy phân tích kịch bản trên Hình 6.14: Chúng ta biết rằng `SecurityAspect` có thể từ chối và chặn đứng lời gọi nếu vi phạm bảo mật.

- Nếu `SecurityAspect` chạy trước, khi xảy ra vi phạm bảo mật, nó sẽ dừng cuộc gọi ngay lập tức, và `LoggingAspect` sẽ không bao giờ có cơ hội được thực thi. Kết quả là sự cố vi phạm bảo mật không hề được ghi lại trong nhật ký hệ thống!
- Ngược lại, nếu `LoggingAspect` được chạy trước, nó sẽ ghi nhận lại mọi lượt truy cập, bất kể lời gọi đó sau này có bị `SecurityAspect` từ chối hay không.



Hình 6.14: Thứ tự thực thi các khía cạnh mang tính quyết định: Tùy thuộc vào yêu cầu nghiệp vụ của ứng dụng, bạn cần phải lựa chọn một thứ tự chính xác. Nếu `SecurityAspect` chạy trước và từ chối lời gọi, `LoggingAspect` sẽ mất cơ hội ghi log.

Quy định thứ tự thực thi bằng chú thích @Order Theo mặc định, Spring **không đưa ra bất kỳ sự đảm bảo nào** về thứ tự thực thi giữa hai khía cạnh cùng can thiệp vào một phương thức. Để kiểm soát thứ tự này một cách tường minh, Spring cung cấp chú thích `@Order`.

Chú thích `@Order(int)` tiếp nhận một giá trị số nguyên đại diện cho mức độ ưu tiên trong chuỗi thực thi: **Số nguyên càng nhỏ thì khía cạnh đó càng được ưu tiên thực thi sớm hơn**.

Hãy cùng kiểm chứng cơ chế này trong hai dự án mẫu `sq-ch6-ex6` và `sq-ch6-ex7`.

Đầu tiên, trong dự án `sq-ch6-ex6`, chúng ta định nghĩa hai khía cạnh: `LoggingAspect` (Listing 6.12) và `SecurityAspect` (Listing 6.13). Cả hai đều đánh chặn các phương thức có gắn chú thích `@ToLog`. Lưu ý rằng lệnh gọi `joinPoint.proceed()` trong chuỗi thực thi sẽ chuyển tiếp tới khía cạnh tiếp theo trong chuỗi hoặc tới phương thức thực tế nếu nó là mắt xích cuối cùng.

```
@Aspect
public class LoggingAspect {

  private Logger logger = Logger.getLogger(LoggingAspect.class.getName()
   );

  @Around("@annotation(ToLog)")
```

```
  public Object log(ProceedingJoinPoint joinPoint) throws Throwable {
    logger.info("Logging Aspect: Dang chuyen tiep loi goi...");
    Object returnedValue = joinPoint.proceed();
    logger.info("Logging Aspect: Phuong thuc hoan tat va tra ve: " +
   returnedValue);
    return returnedValue;
  }
}
```

Listing 6.12: Định nghĩa `LoggingAspect` trong chuỗi thực thi

```
@Aspect
public class SecurityAspect {

  private Logger logger = Logger.getLogger(SecurityAspect.class.getName
    ());

  @Around("@annotation(ToLog)")
  public Object secure(ProceedingJoinPoint joinPoint) throws Throwable {
    logger.info("Security Aspect: Dang kiem tra bao mat va chuyen tiep
   ...");
    Object returnedValue = joinPoint.proceed();
    logger.info("Security Aspect: Phuong thuc hoan tat va tra ve: " +
   returnedValue);
    return returnedValue;
  }
}
```

Listing 6.13: Định nghĩa `SecurityAspect` trong chuỗi thực thi

Lớp dịch vụ `CommentService` (Listing 6.14) và lớp cấu hình `ProjectConfig` (Listing 6.15) đăng ký cả hai bean khía cạnh vào ngữ cảnh Spring:

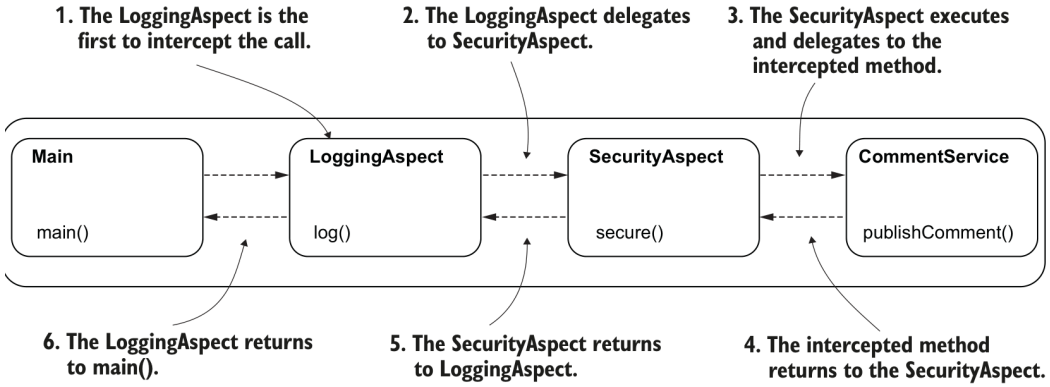
```
@Service
public class CommentService {
  private Logger logger = Logger.getLogger(CommentService.class.getName
    ());

  @ToLog
  public String publishComment(Comment comment) {
    logger.info("Publishing comment: " + comment.getText());
    return "SUCCESS";
  }
}
```

Listing 6.14: Lớp `CommentService` gắn chú thích `@ToLog`

```
@Configuration
@ComponentScan(basePackages = "services")
@EnableAspectJAutoProxy
public class ProjectConfig {

  @Bean
  public LoggingAspect loggingAspect() {
    return new LoggingAspect();
  }

  @Bean
  public SecurityAspect securityAspect() {
    return new SecurityAspect();
  }
}
```

Listing 6.15: Đăng ký cả hai bean khía cạnh trong lớp cấu hình `ProjectConfig`

Khi chạy chương trình ở trạng thái mặc định chưa gán thứ tự, giả sử Spring chạy `LoggingAspect` trước rồi mới tới `SecurityAspect` (Hình 6.15).



Hình 6.15: Luồng thực thi khi `LoggingAspect` chạy trước: `LoggingAspect` chặn lời gọi đầu tiên, chuyển tiếp tới `SecurityAspect`; `SecurityAspect` chuyển tiếp tới phương thức thực tế. Kết quả trả về lần lượt theo chiều ngược lại.

Bây giờ, chúng ta chuyển sang dự án `sq-ch6-ex7`: chúng ta muốn **đảo ngược thứ tự**, buộc `SecurityAspect` phải chạy trước `LoggingAspect`. Chúng ta chỉ cần đặt chú thích `@Order` lên trên định nghĩa của hai lớp khía cạnh:

```
@Aspect
```

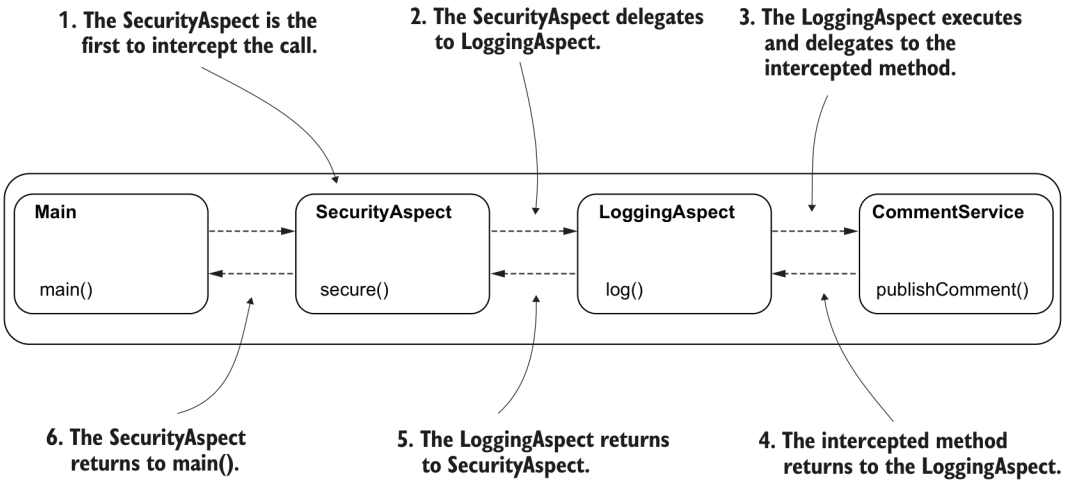
```
@Order(1)
public class SecurityAspect {
  // Cac logic giu nguyen nhu truoc...
}
```

Đối với `LoggingAspect`, chúng ta gán mức ưu tiên số 2:

```
@Aspect
@Order(2)
public class LoggingAspect {
  // Cac logic giu nguyen nhu truoc...
}
```

Chạy lại ứng dụng, bạn sẽ thấy thứ tự trên màn hình console đã được đảo ngược hoàn toàn theo đúng ý muốn (Hình 6.16):

```
INFO: Security Aspect: Dang kiem tra bao mat va chuyen tiep...
INFO: Logging Aspect: Dang chuyen tiep loi goi...
INFO: Publishing comment: Demo comment
INFO: Logging Aspect: Phuong thuc hoan tat va tra ve: SUCCESS
INFO: Security Aspect: Phuong thuc hoan tat va tra ve: SUCCESS
```



Hình 6.16: Luồng thực thi sau khi thiết lập thứ tự bằng chú thích `@Order`: `SecurityAspect` (`@Order(1)`) trở thành chốt chặn đầu tiên tiếp nhận lời gọi, sau đó mới ủy thác tới `LoggingAspect` (`@Order(2)`).

Nhờ chú thích `@Order`, bạn có thể dễ dàng thiết lập và kiểm soát các chuỗi xử lý phức tạp gồm nhiều khía cạnh mà không lo sợ xung đột hoặc sai lệch trình tự nghiệp vụ.

6.4 Tóm tắt chương

- **Khía cạnh (Aspect)** là một thành phần kỹ thuật đánh chặn các lời gọi phương thức và có thể thực thi logic bổ trợ trước, sau, hoặc thay thế phương thức bị đánh chặn. Khía cạnh giúp tách rời các mối quan tâm xuyên suốt (*cross-cutting concerns*) ra khỏi mã nghiệp vụ cốt lõi, nâng cao tính bảo trì của hệ thống.
- Việc sử dụng khía cạnh giúp người đọc mã nguồn chỉ cần tập trung vào logic nghiệp vụ của phương thức mà không bị phân tâm bởi các dòng mã kỹ thuật lặp dư thừa.
- Tuy nhiên, **khía cạnh là một công cụ tiềm ẩn rủi ro nếu lạm dụng**. Việc thiết kế quá đà (*overengineering*) với khía cạnh sẽ biến mã nguồn thành một hệ thống mù mờ, khó lần vết và phản tác dụng đối với mục tiêu bảo trì. Hãy chỉ sử dụng khía cạnh khi nó thực sự đem lại giá trị rõ ràng.
- Rất nhiều tính năng cốt lõi của Spring như **quản lý giao dịch (*transactions*)** và **phân quyền bảo mật (*security*)** đều được xây dựng dựa trên nền tảng của khía cạnh.
- Để định nghĩa một khía cạnh trong Spring, bạn đánh dấu lớp bằng chú thích `@Aspect`. Nhưng hãy nhớ rằng: **`@Aspect` không tự động tạo bean trong ngữ cảnh**. Bạn bắt buộc phải tự tay đăng ký bean đó (bằng `@Component` hoặc `@Bean`).
- Để chỉ thị cho Spring biết phương thức nào cần bị đánh chặn, bạn sử dụng các **biểu thức điểm cắt AspectJ** (*AspectJ pointcut expressions*) đặt trong các chú thích advice. Spring cung cấp 5 chú thích can thiệp: `@Around`, `@Before`, `@After`, `@AfterReturning`, và `@AfterThrowing`. Trong đó `@Around` là chú thích phổ biến và quyền năng nhất.
- Thay vì dùng biểu thức điểm cắt phức tạp theo tên gói, giải pháp thanh thoát nhất là **tạo chú thích tùy biến** và sử dụng biểu thức `@annotation(...)` để đánh dấu các phương thức cần can thiệp.
- Khi có nhiều khía cạnh cùng can thiệp vào một phương thức, chúng tạo thành một **chuỗi thực thi (*execution chain*)**. Thứ tự thực thi giữa các khía cạnh có thể quyết định kết quả của chương trình. Bạn sử dụng chú thích `@Order(int)` để ấn định thứ tự ưu tiên chuẩn xác giữa các khía cạnh.

Phần II

Triển khai thực tế (Implementation)

Chương 7

Tìm hiểu Spring Boot và Spring MVC

Chào mừng bạn bước sang **Phần 2: Triển khai thực tế (*Implementation*)** của cuốn sách! Trong 6 chương đầu tiên của Phần 1, bạn đã làm chủ nền móng vững chắc của hệ sinh thái Spring: vùng chứa ngữ cảnh Spring (*Spring context*), cơ chế tiêm phụ thuộc (*Dependency Injection*) và lập trình hướng khía cạnh (*Spring AOP*). Từ chương này trở đi, chúng ta sẽ bắt đầu vận dụng toàn bộ những kiến thức nền tảng đó để xây dựng các ứng dụng phục vụ các bài toán thực tiễn trong doanh nghiệp.

Chủ đề đầu tiên và cũng là chủ đề quan trọng nhất trong phát triển phần mềm hiện đại chính là: **Xây dựng các ứng dụng web (*web applications*)**.

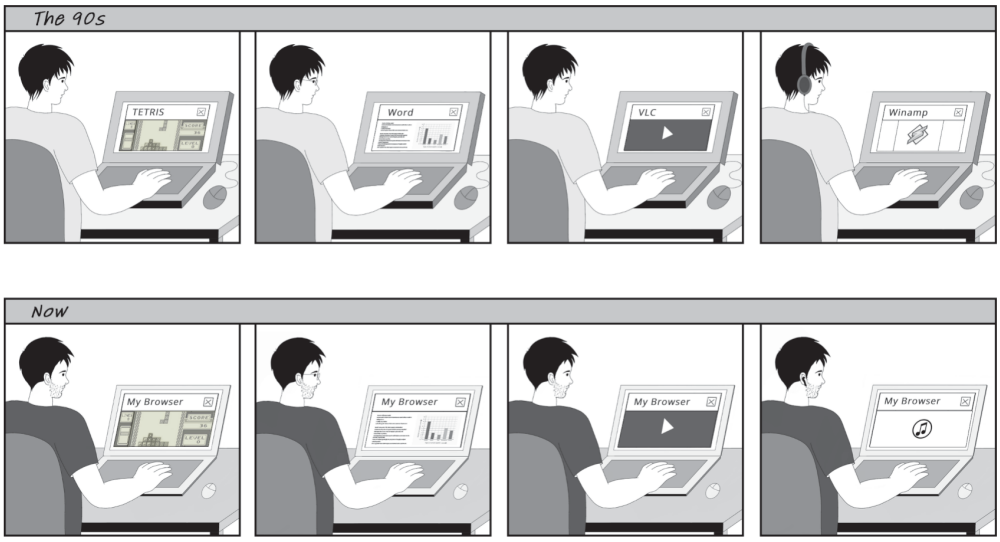
Nội dung trọng tâm của chương này

- Khái niệm kiến trúc tổng quan của một ứng dụng web và vai trò của *servlet container*.
- Tìm hiểu phép màu của **Spring Boot**: dịch vụ khởi tạo dự án, các gói phụ thuộc khởi đầu (*starters*), và nguyên lý cấu hình theo quy ước (*convention-over-configuration*).
- Hiện thực hóa trang web đầu tiên với **Spring MVC**: tạo lớp điều khiển (*controller*), ánh xạ yêu cầu (*request mapping*) và phân tích luồng điều phối của *DispatcherServlet*.

Thời thế đã thay đổi rất nhiều (Hình 7.1). Nếu như vào những năm 1990, chúng ta cài đặt và sử dụng ứng dụng màn hình máy tính bàn (*desktop app*) cho mọi công việc, thì ngày nay đại đa số người dùng tương tác với các hệ thống thông qua trình duyệt web trên máy tính hoặc ứng dụng di động. Do đó, việc hiểu rõ cách thức vận hành và nắm chắc kỹ năng xây dựng ứng dụng web là điều kiện tiên quyết đối với mọi lập trình viên phần mềm chuyên nghiệp.

Maybe you don't even read this book on paper and use the Manning liveBook web app for this.

Any app you access through your web browser is a web app. Years ago, we used desktop apps installed on our computers for almost anything we were doing (figure 7.1). With time, most of these apps became accessible via a web browser. Accessing an app in a browser makes it more comfortable to use. You don't have to install anything, and you can use it from any device that has access to the internet, such as a tablet or smartphone.



Hình 7.1: Thời thế thay đổi: Trong thập niên 1990, mọi phần mềm đều được cài đặt dưới dạng ứng dụng desktop chạy cục bộ. Ngày nay, hầu hết các hệ thống doanh nghiệp đều chuyển dịch lên nền tảng web, nơi người dùng chỉ cần một trình duyệt để truy cập dịch vụ.

Trong Mục 7.1, chúng ta sẽ bắt đầu bằng việc tìm hiểu bản chất của một ứng dụng web: kiến trúc phân tầng client-server, giao thức truyền thông HTTP, và thành phần thùng chứa servlet (*servlet container*) trong hệ sinh thái Java.

Kế đó, trong Mục 7.2, chúng ta sẽ khám phá sức mạnh đột phá của **Spring Boot** — dự án đã làm thay đổi hoàn toàn diện mạo của hệ sinh thái Spring bằng cách loại bỏ hàng núi mã cấu hình rườm rà. Cuối cùng, trong Mục 7.3, chúng ta sẽ cùng nhau xây dựng ứng dụng web Spring MVC đầu tiên và phân tích sâu cơ chế điều phối đằng sau hậu trường của nó.

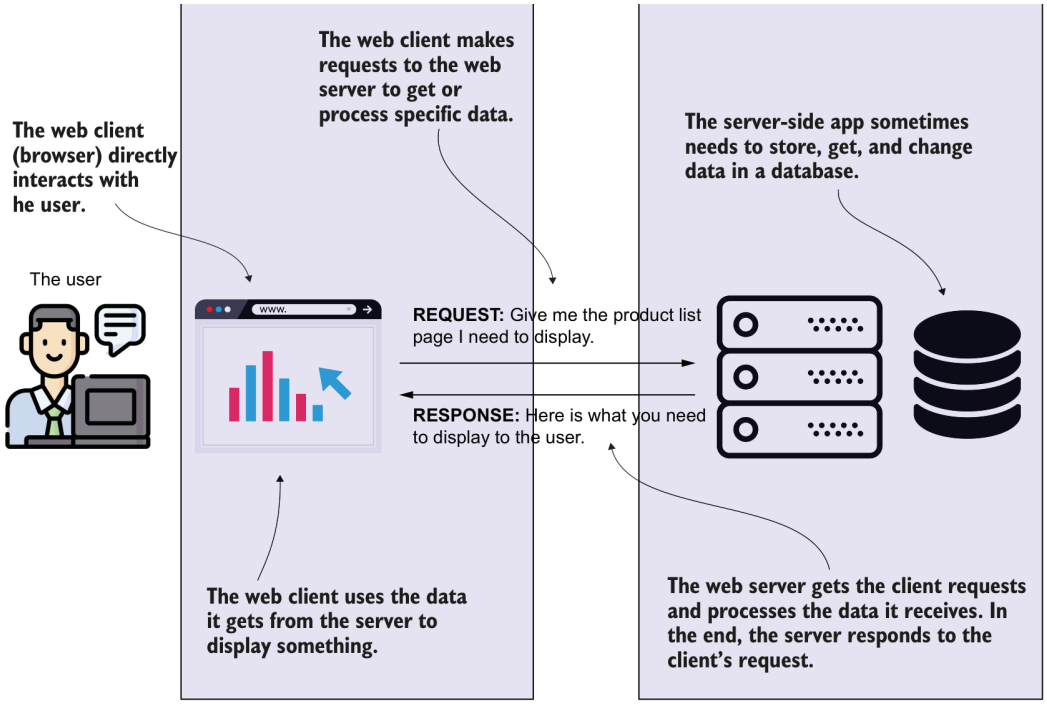
7.1 Ứng dụng web là gì?

Trước khi viết bất kỳ dòng mã nào, chúng ta cần thống nhất với nhau về bức tranh toàn cảnh của một ứng dụng web. Một ứng dụng web là một ứng dụng phần mềm mà người dùng tương tác thông qua một trình duyệt web (*web browser*).

7.1.1 Bức tranh tổng quan về ứng dụng web

Về mặt kiến trúc, ứng dụng web luôn được chia tách thành hai phía rõ rệt (Hình 7.2):

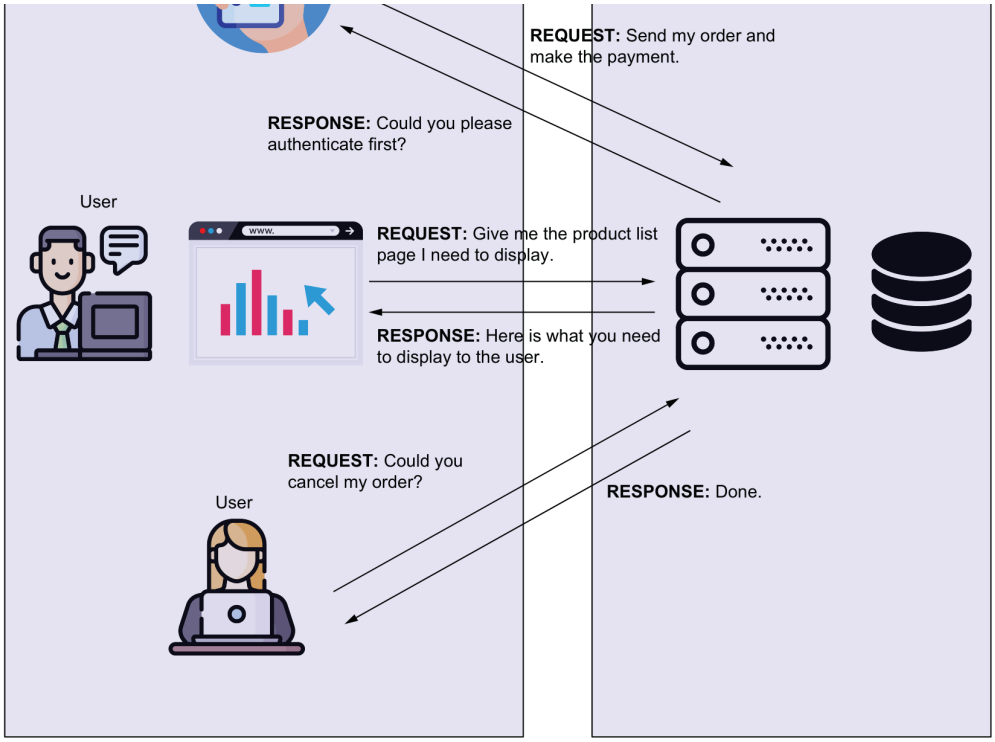
1. **Phía khách (*Client side / Frontend*):** Thành phần chạy trực tiếp trên thiết bị của người dùng (trong trình duyệt web hoặc ứng dụng di động). Nó chịu trách nhiệm hiển thị giao diện đồ họa và thu nhận các tương tác bấm chuột, gõ phím từ người dùng.
2. **Phía máy chủ (*Server side / Backend*):** Thành phần chạy trên các máy chủ từ xa, nơi toàn bộ dữ liệu được xử lý tập trung, áp dụng các quy tắc nghiệp vụ và lưu trữ bền vững vào các hệ cơ sở dữ liệu.



Hình 7.2: Bức tranh tổng quan của một ứng dụng web: Người dùng tương tác với giao diện hiển thị trên trình duyệt (client). Trình duyệt gửi các gói tin yêu cầu HTTP (*HTTP request*) qua mạng Internet tới máy chủ ứng dụng (backend). Máy chủ xử lý nghiệp vụ và gửi lại gói tin phản hồi HTTP (*HTTP response*).

Để hai phía này có thể trò chuyện được với nhau, chúng sử dụng một ngôn ngữ giao tiếp chung chuẩn mực trên Internet: **giao thức truyền siêu văn bản (HTTP - Hypertext Transfer Protocol)**. Mỗi lần người dùng nhấp chuột vào một nút bấm hoặc gõ một địa chỉ liên kết, trình duyệt sẽ đóng gói một thông điệp yêu cầu (*HTTP request*) và gửi tới backend. Backend tiếp nhận gói tin này, thực thi logic nghiệp vụ tương ứng và gửi trả về một thông điệp phản hồi (*HTTP response*).

Trong kiến trúc web, một thể hiện backend có thể phục vụ cùng lúc hàng ngàn hoặc hàng triệu thể hiện client khác nhau (Hình 7.3). Các client này có thể là các tab trình duyệt độc lập hoặc các ứng dụng chạy trên điện thoại thông minh.

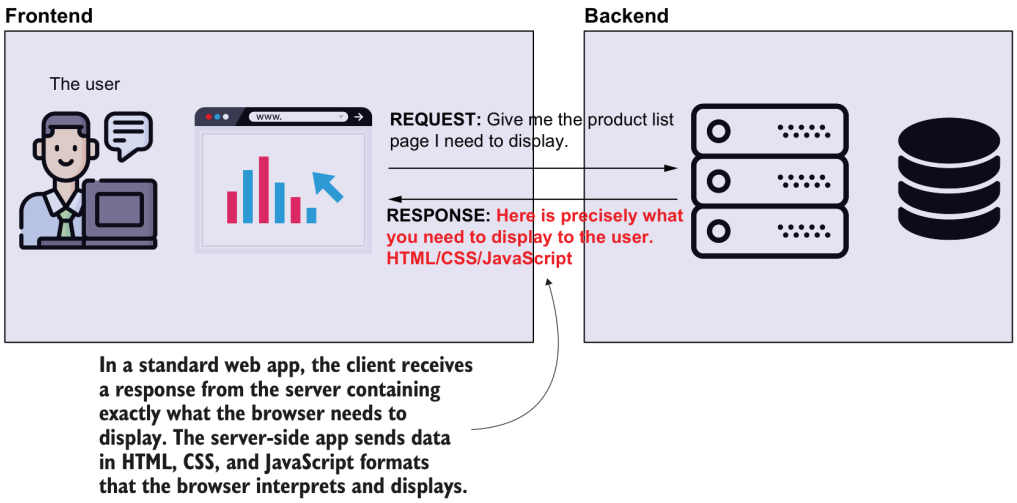


Hình 7.3: Khái niệm phía khách (*client*): Trong kiến trúc web, mỗi thể hiện tab trình duyệt hoặc một ứng dụng di động được xem là một client độc lập. Tất cả các client này đều gửi yêu cầu HTTP tới cùng một hệ thống backend để chia sẻ dữ liệu chung.

7.1.2 Các cách tiếp cận xây dựng ứng dụng web với Spring

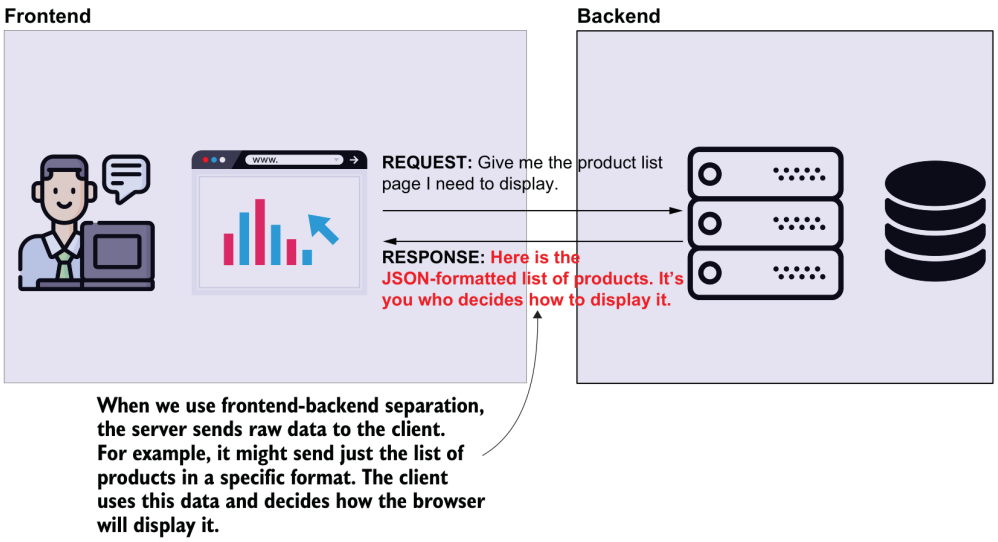
Khi sử dụng Spring để xây dựng backend, lập trình viên có hai trường phái kiến trúc chính để lựa chọn tùy thuộc vào đặc thù dự án:

Trường phái 1: Không tách biệt Frontend - Backend (Server-Side Rendering) Trong cách tiếp cận này (Hình 7.4), máy chủ backend kiêm nhiệm luôn cả việc tạo dựng giao diện người dùng. Khi trình duyệt gửi yêu cầu, backend sẽ xử lý dữ liệu và kết hợp với các mẫu giao diện để sinh ra trực tiếp mã HTML hoàn chỉnh, rồi gửi tệp HTML này trong thân phản hồi HTTP. Trình duyệt phía người dùng chỉ việc đọc và kết xuất (*render*) mã HTML đó ra màn hình. Đây là cách tiếp cận kinh điển mà chúng ta sẽ tìm hiểu trong Chương 7 và Chương 8 thông qua công cụ động cơ mẫu giao diện **Thymeleaf**.



Hình 7.4: Ứng dụng web không tách rời frontend-backend: Trình duyệt gửi yêu cầu tới máy chủ; máy chủ xử lý dữ liệu, trực tiếp kết xuất ra tệp HTML hoàn chỉnh và trả về. Trình duyệt chỉ việc hiển thị tài liệu HTML nhận được.

Trường phái 2: Tách biệt hoàn toàn Frontend - Backend (RESTful Web API) Trong cách tiếp cận hiện đại này (Hình 7.5), phía frontend và backend là hai dự án phần mềm hoàn toàn độc lập với nhau. Frontend thường được xây dựng bằng các framework JavaScript hiện đại (như React, Angular, Vue) hoặc ứng dụng di động native (iOS, Android). Phía backend (xây dựng bằng Spring) **tuyệt đối không can thiệp vào giao diện HTML**, mà chỉ đóng vai trò là một dịch vụ cung cấp dữ liệu thuần túy (thường ở định dạng chuẩn **JSON**). Frontend sẽ chủ động gọi tới các điểm cuối (*endpoints*) của backend để lấy dữ liệu JSON, sau đó tự tay cập nhật lại giao diện người dùng trên máy khách. Chúng ta sẽ làm chủ toàn diện trường phái này từ Chương 10 đến Chương 12.

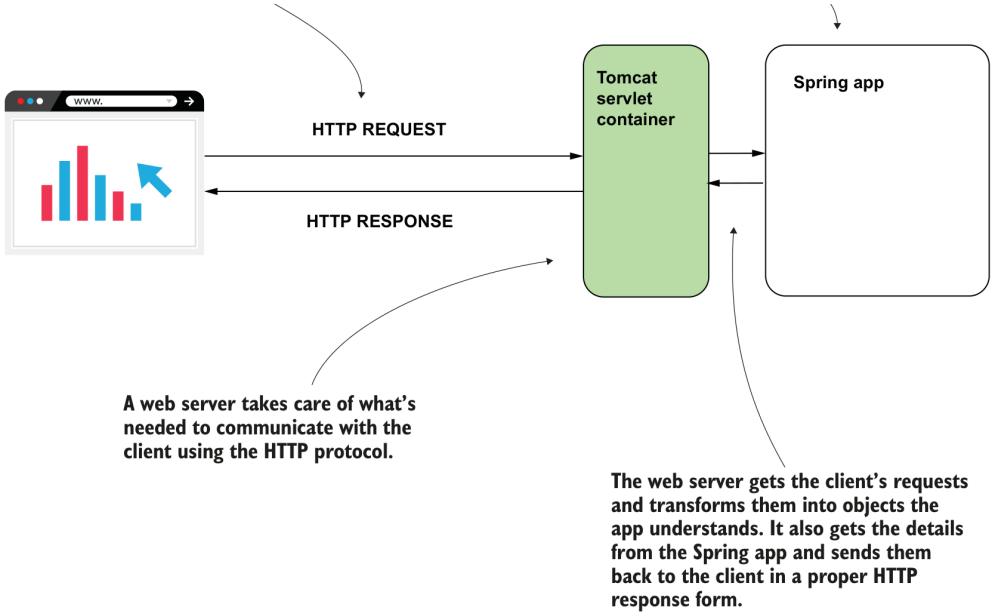


Hình 7.5: Ứng dụng web tách rời frontend-backend: Máy chủ backend không trả về giao diện HTML mà chỉ phản hồi dữ liệu thô (thường là JSON). Ứng dụng phía frontend (viết bằng React, Angular, iOS/Android) nhận dữ liệu này và tự kết xuất giao diện cục bộ trên thiết bị.

7.1.3 Sử dụng Servlet Container trong phát triển ứng dụng web Java

Làm thế nào để ứng dụng Java của chúng ta có thể tiếp nhận và xử lý các gói tin HTTP gửi qua mạng Internet?

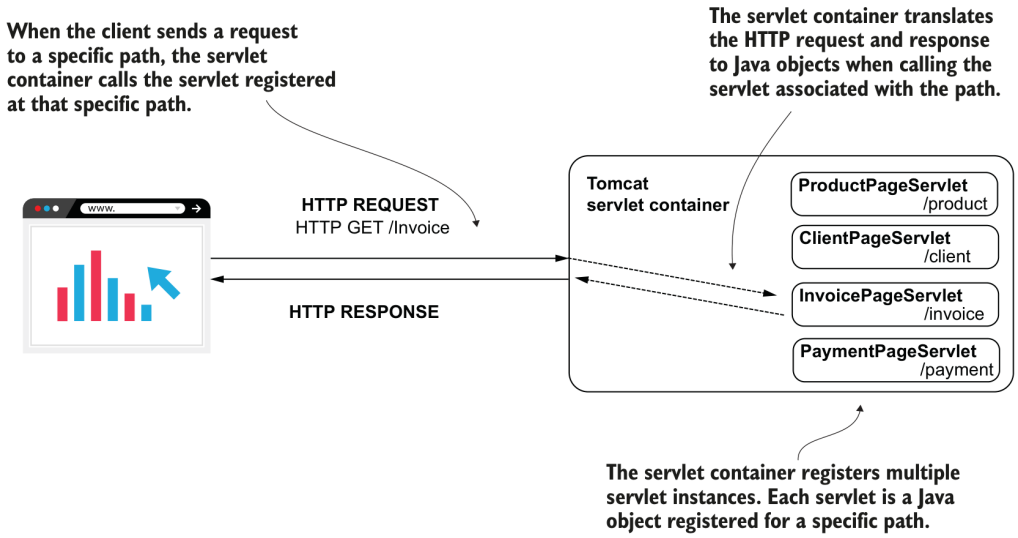
Nếu bạn phải tự tay viết mã bằng socket mạng để lắng nghe cổng TCP, bóc tách các dòng tiêu đề (*headers*) của HTTP request, quản lý các luồng kết nối đồng thời và định dạng lại gói tin HTTP response, bạn sẽ tốn hàng ngàn dòng mã phức tạp và cực kỳ dễ gặp lỗi bảo mật. Rất may mắn, các nhà phát triển Java không cần phải làm việc này nhờ vào một thành phần phần mềm chuyên trách mang tên: **Servlet Container** (tiêu biểu nhất là **Apache Tomcat**).



Hình 7.6: Thùng chứa Servlet (*Servlet container* như Tomcat) là thành phần "nói tiếng HTTP". Nó chịu trách nhiệm tiếp nhận gói tin HTTP từ mạng, dịch thành các đối tượng Java (`HttpServletRequest`) cho ứng dụng, và chuyển đổi kết quả từ ứng dụng thành gói tin HTTP response gửi ngược lại client.

Servlet container đóng vai trò như một thông dịch viên HTTP (Hình 7.6). Nó đứng ở tầng biên mạng, mở cổng giao tiếp (mặc định là cổng `8080`), tiếp nhận các gói tin HTTP, phân tích cú pháp và chuyển đổi chúng thành các đối tượng Java thân thiện.

Bên trong ứng dụng Java, thành phần đứng ra tiếp nhận yêu cầu từ servlet container được gọi là một **Servlet**. Một servlet container có thể đăng ký nhiều thể hiện servlet khác nhau để xử lý các phân vùng đường dẫn khác nhau (Hình 7.7).

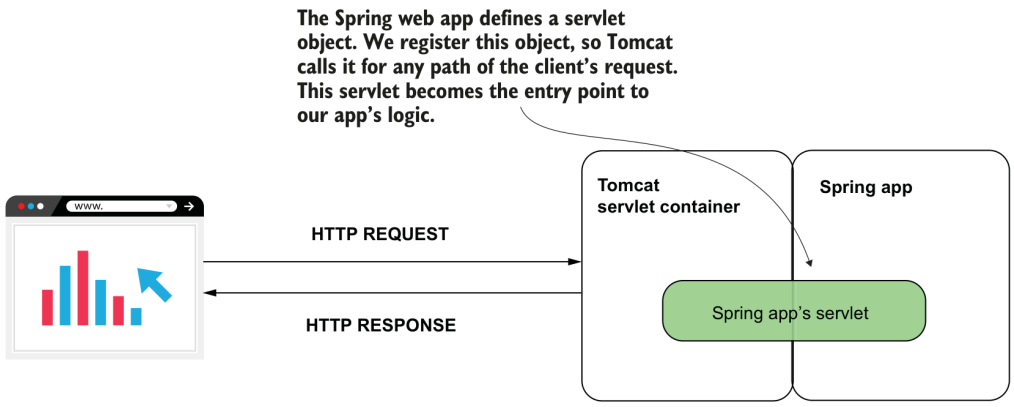


Hình 7.7: Servlet container quản lý và đăng ký nhiều thể hiện servlet bên trong nó. Khi một yêu cầu HTTP gửi đến, container sẽ căn cứ vào đường dẫn để phân luồng yêu cầu tới servlet phụ trách tương ứng.

Trong thế giới Spring MVC, thay vì bắt lập trình viên phải tự viết hàng chục servlet riêng lẻ, Spring áp dụng một mẫu thiết kế cực kỳ thông minh: **Front Controller Pattern**.

Spring chỉ định nghĩa đúng **MỘT servlet trung tâm duy nhất** mang tên `DispatcherServlet` và đăng ký nó với Tomcat (Hình 7.8). Mọi yêu cầu HTTP gửi tới ứng dụng đều sẽ đi qua cửa ngõ của `DispatcherServlet`, và chính servlet trung tâm này sẽ tự động tìm kiếm phương thức nghiệp vụ thích hợp bên trong ứng dụng Spring để thực thi!

need to remember the servlet is the entry point to your app's logic. It's the component the servlet container (Tomcat, in our case) directly interacts with. It's how the request data enters your app and how the response goes through Tomcat back to the client (figure 7.8).



Hình 7.8: Kiến trúc Spring MVC: Ứng dụng Spring đăng ký một thể hiện servlet trung tâm duy nhất (`DispatcherServlet`) vào trong servlet container. Mọi yêu cầu HTTP gửi đến đều đi qua servlet này để được định tuyến tới các thành phần bên trong ngữ cảnh Spring.

7.2 Phép màu của Spring Boot

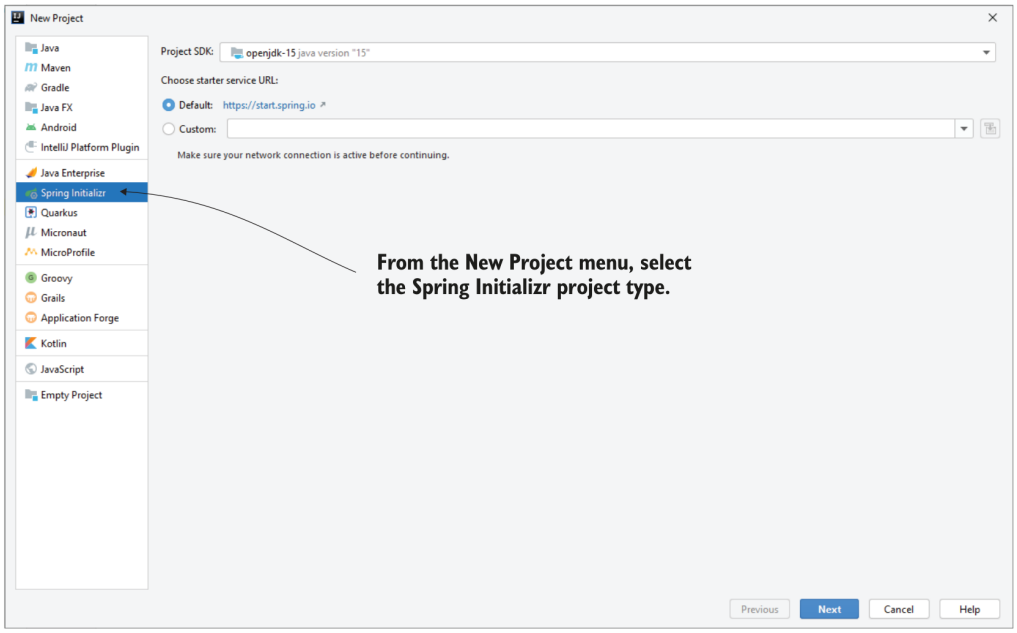
Trước khi Spring Boot ra đời, việc cấu hình một ứng dụng web Spring với Tomcat là một cơn ác mộng đối với hầu hết các lập trình viên. Bạn phải tự tay cài đặt máy chủ Tomcat độc lập, cấu hình các tệp XML rắc rối (‘web.xml‘), quản lý thủ công hàng chục thư viện phụ thuộc trong ‘pom.xml‘, đồng thời giải quyết các xung đột phiên bản nhức đầu giữa các thư viện JAR.

Spring Boot xuất hiện và thay đổi hoàn toàn cuộc chơi. Nó mang lại ba giá trị cốt lõi mang tính cách mạng:

1. **Dịch vụ khởi tạo dự án nhanh chóng** (*Project Initializer*).
2. **Các gói phụ thuộc khởi đầu** (*Dependency Starters*).
3. **Cơ chế tự động cấu hình theo quy ước** (*Autoconfiguration by Convention*).

7.2.1 Khởi tạo dự án Spring Boot bằng dịch vụ Spring Initializr

Cách thuận tiện và chuẩn mực nhất để bắt đầu một dự án Spring Boot mới là sử dụng công cụ chính thức **Spring Initializr** thông qua trang web: `https://start.spring.io`. Hầu hết các môi trường phát triển tích hợp (IDE) hiện đại như IntelliJ IDEA, Eclipse (với Spring Tools), hay VS Code đều tích hợp sẵn dịch vụ này trực tiếp vào menu tạo dự án mới (Hình 7.9).

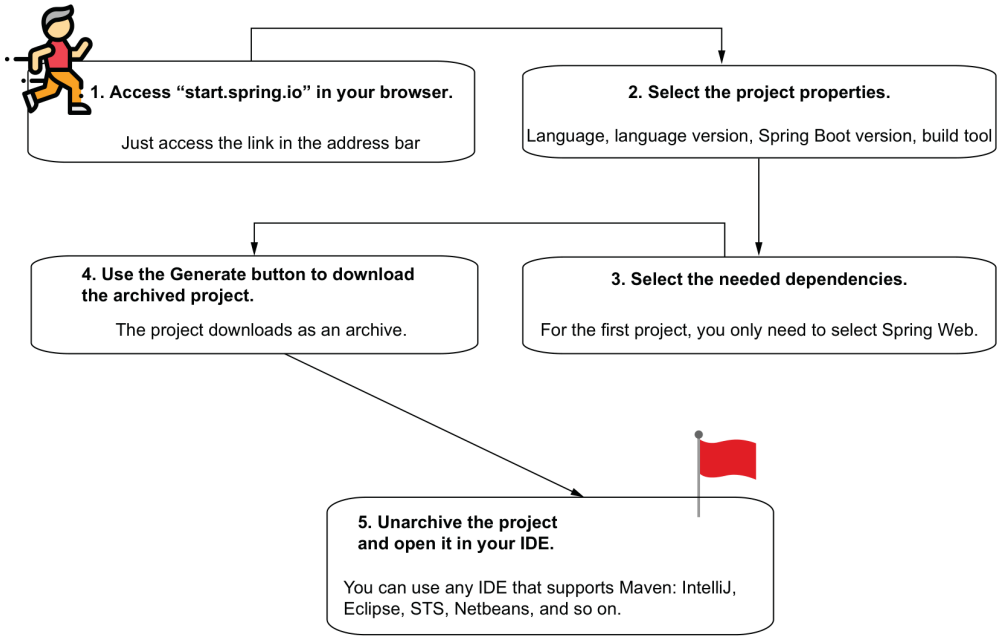


Hình 7.9: Nhiều IDE tích hợp sẵn dịch vụ khởi tạo dự án Spring Boot. Bạn có thể chọn menu tạo New Project, chọn Spring Initializr để IDE tự động tải mã nguồn khởi tạo mà không cần mở trình duyệt web.

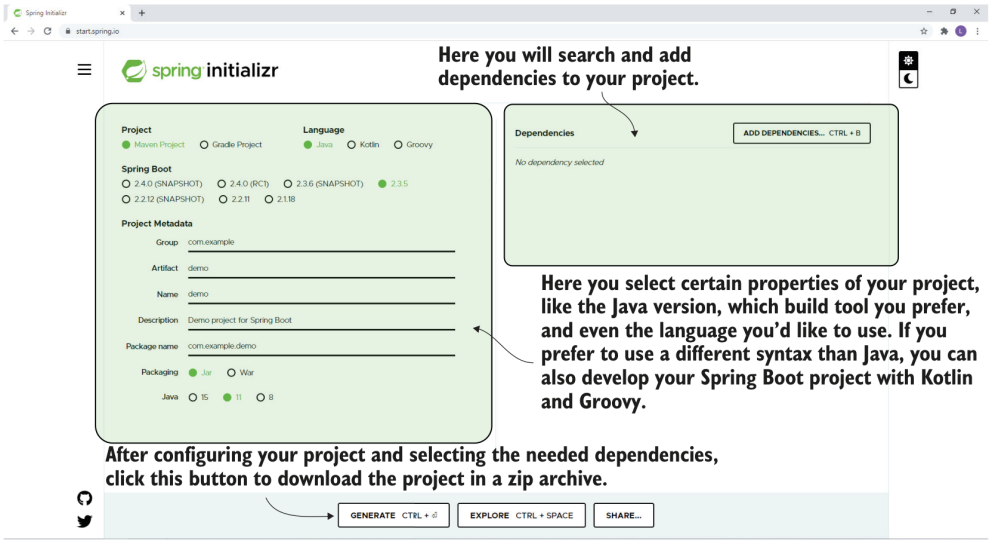
Quy trình tạo dự án trên giao diện web `start.spring.io` gồm 3 bước đơn giản (Hình 7.10

và Hình 7.11):

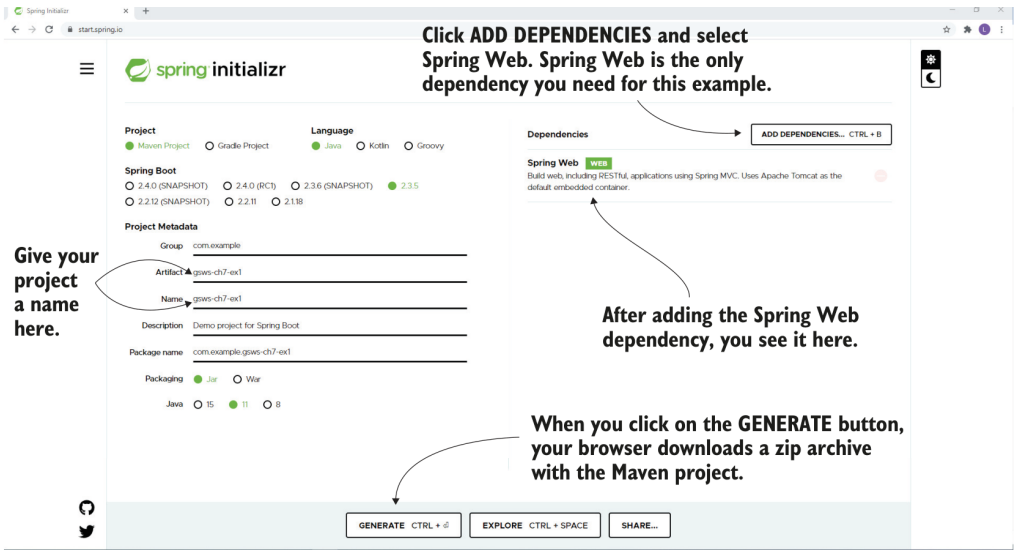
1. Truy cập vào trang web `https://start.spring.io`.
2. Thiết lập các thông số dự án: Chọn loại dự án (Maven), ngôn ngữ (Java), phiên bản Spring Boot ổn định, GroupId, ArtifactId và phiên bản Java (Java 11).
3. Thêm các gói phụ thuộc cần thiết: Bấm nút **Add Dependencies** và chọn **Spring Web** (Hình 7.12), sau đó nhấn nút **Generate** để tải về tệp tin `.zip`.



Hình 7.10: Các bước tạo dự án Spring Boot bằng dịch vụ `start.spring.io`: Truy cập web, cấu hình thông tin dự án, bổ sung các phụ thuộc theo nhu cầu rồi nhấn Generate.

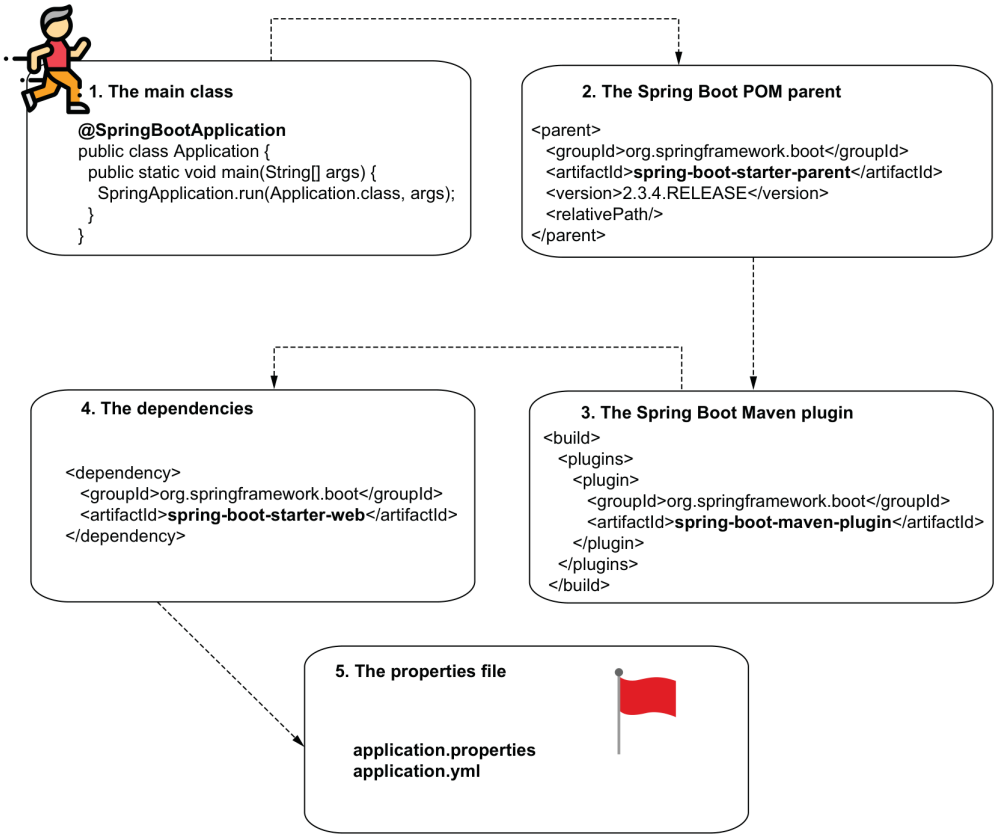


Hình 7.11: Giao diện trực quan của `start.spring.io`: Nơi bạn lựa chọn các thông số công nghệ nền tảng và tìm kiếm các thành phần phụ thuộc của hệ sinh thái Spring.



Hình 7.12: Tìm kiếm và thêm phụ thuộc **Spring Web**. Phụ thuộc này sẽ cung cấp trọn bộ khả năng xây dựng ứng dụng web, bao gồm Spring MVC và máy chủ Tomcat nhúng.

Khi giải nén tệp tin dự án vừa tải về và mở trong IDE, bạn sẽ thấy cấu trúc thư mục chuẩn mực như mô tả trên Hình 7.13.



Hình 7.13: Cấu trúc thư mục của một dự án Spring Boot tiêu chuẩn: Gồm thư mục mã nguồn `src/main/java`, thư mục tài nguyên `src/main/resources` (chứa `static`, `templates` và tệp `application.properties`), cùng tệp quản lý `pom.xml`.

Hãy mở tệp `pom.xml` của dự án. Bạn sẽ nhận thấy những phần tử đặc biệt mà Spring Initializr đã thiết lập sẵn cho bạn:

- **Khối `<parent>`:**

```
<parent>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-parent</artifactId>
   <version>2.3.4.RELEASE</version>
   <relativePath/>
</parent>
```

  Phần tử cha `spring-boot-starter-parent` đóng vai trò là cơ quan quản lý phiên bản tập trung. Nó quy định trước phiên bản tương thích của toàn bộ hàng trăm thư viện phổ biến trong hệ sinh thái Java. Nhờ đó, trong tất cả các thẻ phụ thuộc con bên dưới, **bạn hoàn toàn không cần phải ghi số phiên bản `<version>`**! Spring Boot sẽ tự động chọn phiên bản chuẩn xác và an toàn nhất.

- **Phụ thuộc `spring-boot-starter-web`:**

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

- **Plugin `spring-boot-maven-plugin`:**

```
<build>
   <plugins>
      <plugin>
         <groupId>org.springframework.boot</groupId>
         <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
   </plugins>
</build>
```

  Plugin này cho phép đóng gói toàn bộ ứng dụng cùng với máy chủ Tomcat nhúng vào **một tệp tin JAR duy nhất** (gọi là *Fat JAR / Uber JAR*). Bạn có thể triển khai chạy ứng dụng trên bất kỳ máy chủ nào chỉ bằng một lệnh đơn giản: `java -jar app.jar`!

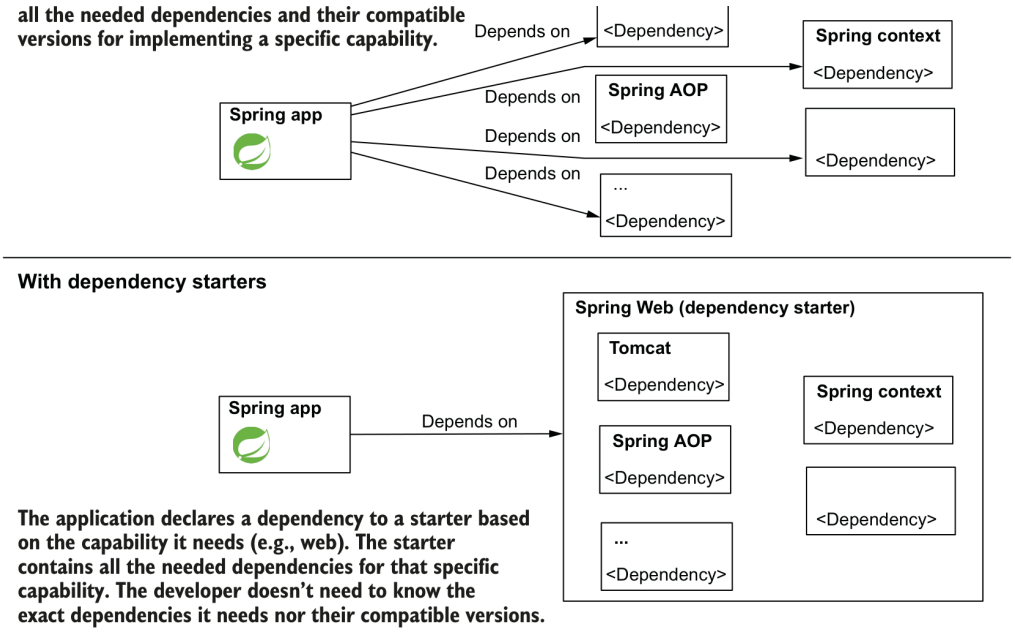
- **Tệp cấu hình `application.properties`:** Nằm trong thư mục `src/main/resources`, là nơi bạn thiết lập các tham số môi trường như cổng cổng mạng, thông tin kết nối cơ sở dữ liệu, v.v.

7.2.2 Sử dụng Dependency Starters để đơn giản hóa quản lý phụ thuộc

Ưu điểm đột phá thứ hai của Spring Boot chính là các gói phụ thuộc khởi đầu (**Dependency Starters**).

Trong các dự án Java truyền thống, để xây dựng một tính năng web hoàn chỉnh, bạn phải tự tra cứu và liệt kê thủ công hàng loạt thư viện riêng rẽ: ‘spring-web‘, ‘spring-webmvc‘, ‘spring-core‘, ‘spring-beans‘, ‘tomcat-embed-core‘, ‘jackson-databind‘ (để xử lý JSON), ‘hibernate-validator‘, v.v. Việc duy trì tính tương thích giữa các phiên bản của từng thư viện này là một công việc cực kỳ mệt mỏi và dễ gây đổ vỡ hệ thống.

Với Spring Boot, **chúng ta không yêu cầu từng thư viện cụ thể nữa — chúng ta yêu cầu các khả năng (*capabilities*)! (Hình 7.14).**



Hình 7.14: Sự vượt trội của Dependency Starters: Thay vì ứng dụng phải tự khai báo hàng chục thư viện đơn lẻ kèm phiên bản chi tiết, ứng dụng chỉ cần khai báo một gói khởi đầu duy nhất (`spring-boot-starter-web`). Gói starter này sẽ tự động kéo theo toàn bộ các thư viện tương thích cần thiết để đáp ứng trọn vẹn khả năng web.

Bạn chỉ cần thêm một phụ thuộc duy nhất mang tên `spring-boot-starter-web`. Khi bạn mở mục *External Libraries* trong IDE, bạn sẽ thấy Spring Boot đã âm thầm tải về đầy đủ toàn bộ các tệp JAR của Spring Core, Spring Context, Spring AOP, Jackson JSON, và cả toàn bộ máy chủ Apache Tomcat với phiên bản tương thích hoàn hảo 100%!

7.2.3 Cơ chế tự động cấu hình theo quy ước (Convention-over-Configuration)

Tính năng được yêu thích và ngưỡng mộ nhất của Spring Boot chính là **Tự động cấu hình** (*Autoconfiguration*) dựa trên nguyên lý **Quy ước hơn Cấu hình** (*Convention over Configuration*).

Hãy thử chạy phương thức `main()` trong lớp khởi động được sinh sẵn của ứng dụng:

```
@SpringBootApplication
public class Main {
  public static void main(String[] args) {
    SpringApplication.run(Main.class, args);
  }
}
```

Dù bạn chưa viết bất kỳ dòng mã nào, bạn sẽ thấy trên màn hình console xuất hiện nhật ký khởi động rực rỡ kèm thông báo:

```
Tomcat started on port(s): 8080 (http) with context path ''
Started Main in 1.684 seconds (JVM running for 2.306)
```

Làm thế nào mà ứng dụng lại tự động bật một máy chủ web Tomcat trên cổng 8080? Đó chính là phép màu của cơ chế tự động cấu hình! Khi phát hiện thấy gói phụ thuộc `spring-boot-starter-web` có mặt trong classpath, Spring Boot tự động suy luận: *"Lập trình viên này đang muốn xây dựng một ứng dụng web. Theo quy ước chuẩn mực nhất, một ứng dụng web Java cần một servlet container, và Tomcat là lựa chọn phổ biến nhất. Do đó, tôi sẽ tự động khởi tạo một thể hiện Tomcat nhúng và lắng nghe trên cổng mặc định 8080!"*

Nhờ nguyên lý này, 90% các cấu hình kỹ thuật thông thường được framework thiết lập sẵn theo các quy ước chuẩn tốt nhất. Bạn chỉ cần viết mã cấu hình bổ sung khi ứng dụng của bạn có những đòi hỏi mang tính đặc thù riêng biệt.

7.3 Triển khai ứng dụng web với Spring MVC

Ứng dụng của chúng ta hiện đã có một máy chủ Tomcat đang chạy sẵn sàng trên cổng 8080. Tuy nhiên, nó vẫn chưa phải là một ứng dụng web thực thụ bởi vì chúng ta chưa có trang web nào để người dùng truy cập.

Để đưa một trang web vào ứng dụng Spring Boot, bạn chỉ cần thực hiện đúng **2 bước đơn giản** (Hình 7.15):

1. **Bước 1:** Tạo một tài liệu HTML chứa nội dung thông tin bạn muốn hiển thị trên trình duyệt.
2. **Bước 2:** Viết một lớp điều khiển (*controller*) chứa một phương thức hành động (*action*) để liên kết đường dẫn yêu cầu với tài liệu HTML đó.

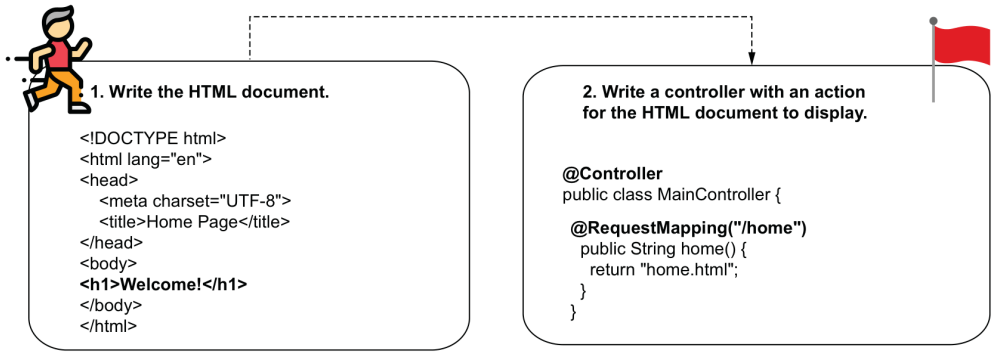
to change those places where your app needs a more particular configuration. With this approach, you'll write less code for configuration (if any).

.3 Implementing a web app with Spring MVC

In this section, we'll implement our first web page in a Spring web app. It's true we already have a Spring Boot project with the default configurations, but this app only starts a Tomcat server. These configurations don't make our app a web app yet! We still have to implement the pages that someone can access using a web browser. We continue implementing the "sq-ch7-ex1" project to add a web page with static content. With these changes, you'll learn to implement a web page and how your Spring app works behind the scenes.

To add a web page to your app, you follow two steps (figure 7.15):

1. Write an HTML document with the content you want to be displayed by the browser.
2. Write a controller with an action for the web page created at point 1.



Hình 7.15: Quy trình 2 bước để thêm một trang web vào ứng dụng Spring MVC: (1) Tạo tài liệu HTML trong thư mục tài nguyên, (2) Tạo lớp `@Controller` với phương thức gắn `@RequestMapping` để tiếp nhận và trả về trang HTML tương ứng.

7.3.1 Bước 1: Viết tài liệu HTML

Chúng ta tạo một tệp tin HTML tĩnh mang tên `home.html` và đặt nó vào thư mục **`src/main/resources/stat`** trong dự án Maven (Listing 7.1). Thư mục `static` là vị trí mặc định theo quy ước mà Spring Boot luôn tìm kiếm các tài nguyên tĩnh (HTML, CSS, hình ảnh, JavaScript) để phục vụ cho người dùng.

```
<!DOCTYPE html>
<html lang="vi">
<head>
    <meta charset="UTF-8">
    <title>Trang chu</title>
</head>
<body>
    <h1>Chao mung ban den voi Spring MVC!</h1>
</body>
</html>
```

Listing 7.1: Nội dung của tệp `home.html` trong thư mục `resources/static`

7.3.2 Bước 2: Định nghĩa lớp Controller

Bộ điều khiển (Controller) là một thành phần trọng yếu của ứng dụng web, chứa các phương thức (thường được gọi là các *action*) chịu trách nhiệm tiếp nhận và xử lý từng yêu cầu HTTP cụ thể.

Để định nghĩa một lớp controller trong Spring MVC, bạn thực hiện như sau (Listing 7.2):

- Đánh dấu lớp bằng chú thích định hình **`@Controller`**. Chú thích này báo hiệu cho Spring biết đây là một bộ điều khiển web, đồng thời đăng ký lớp làm một bean trong ngữ cảnh Spring.
- Đánh dấu phương thức hành động bằng chú thích **`@RequestMapping("/home")`** để gắn kết phương thức này với đường dẫn URL `/home`.
- Phương thức trả về một chuỗi ký tự chứa đúng tên tệp tin HTML cần hiển thị: `"home.html"`.

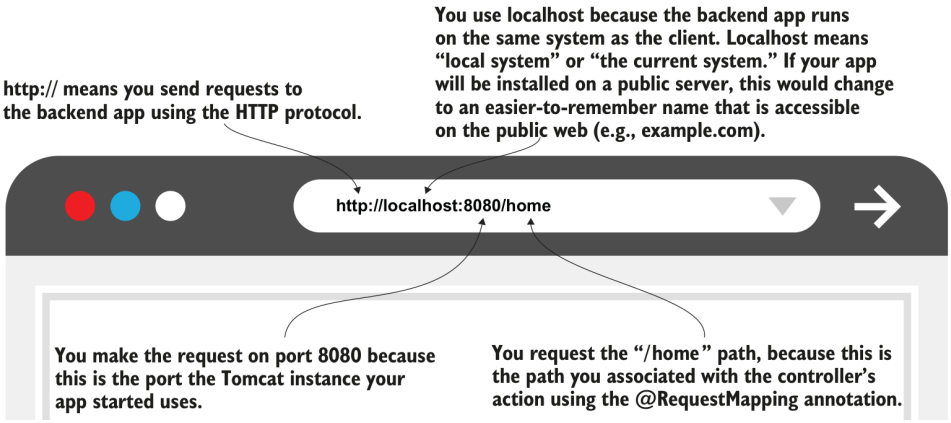
```
@Controller
public class MainController {

  @RequestMapping("/home")
  public String home() {
    return "home.html";
  }
}
```

Listing 7.2: Định nghĩa lớp điều khiển `MainController` trong Spring MVC

7.3.3 Chạy và kiểm thử ứng dụng trên trình duyệt

Bây giờ, hãy khởi động ứng dụng Spring Boot. Mở một trình duyệt web bất kỳ trên máy tính của bạn và nhập vào thanh địa chỉ đường dẫn sau: **`http://localhost:8080/home`** (Hình 7.16).

Open a browser window on the same computer where you run the app and write the following address in the address bar: http://localhost:8080/home (figure 7.16). Do not forget to write the path /home you mapped with the controller's action; otherwise, you'll get an error and an HTTP response with the status "404 Not Found."

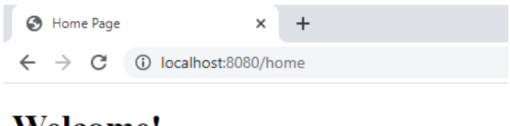


Hình 7.16: Kiểm thử ứng dụng trên trình duyệt: Gửi yêu cầu HTTP tới địa chỉ `http://localhost:8080/home`, trong đó `localhost` trỏ tới máy cục bộ, cổng `8080` là cổng Tomcat đang mở, và `/home` là đường dẫn đã gán với action của controller.

Hãy phân tích ý nghĩa của địa chỉ URL này:

- `http://`: Bạn đang gửi yêu cầu qua giao thức truyền tải siêu văn bản HTTP.
- `localhost`: Ứng dụng backend đang chạy ngay trên chính máy tính cục bộ của bạn. Khi đưa lên môi trường sản xuất, nó sẽ được thay bằng tên miền chính thức (ví dụ: `mywebsite.com`).
- `:8080`: Cổng mạng mà máy chủ Tomcat đang lắng nghe.
- `/home`: Đường dẫn tài nguyên mà chúng ta đã ánh xạ với phương thức `home()` thông qua chú thích `@RequestMapping("/home")`. Nếu bạn gõ sai đường dẫn (chẳng hạn chỉ gõ `http://localhost:8080/`), ứng dụng sẽ trả về mã lỗi **404 Not Found**.

Khi nhấn phím Enter, bạn sẽ thấy dòng chữ tiêu đề *"Chao mung ban den voi Spring MVC!"* xuất hiện rõ ràng trên cửa sổ trình duyệt (Hình 7.17). Trình duyệt đã tiếp nhận tệp HTML trả về từ backend và kết xuất thành công giao diện trực quan cho người dùng!



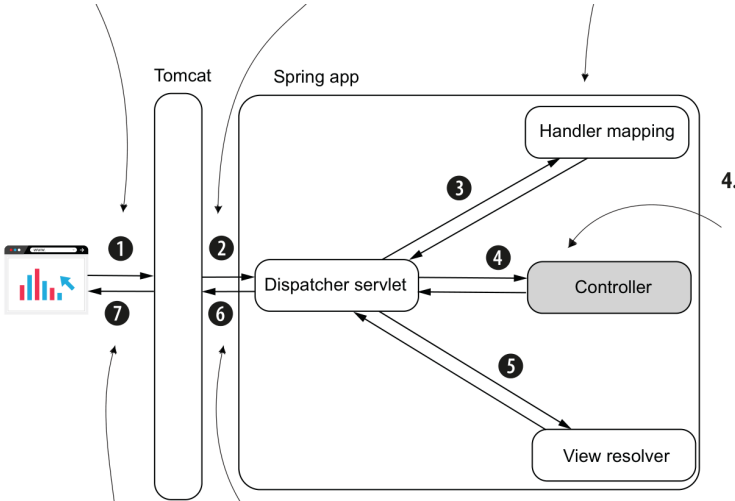
When you access the page exposed by the app, the browser gets the content of the home.html HTML document in response. The browser interprets the HTML content and displays the data. For this app, your will see the heading text "Welcome!"

Hình 7.17: Kết quả truy cập trang web trên trình duyệt: Trình duyệt nhận được mã HTML từ backend, bóc tách và hiển thị dòng chữ tiêu đề chào mừng.

7.3.4 Cơ chế kiến trúc Spring MVC vận hành đằng sau hậu trường

Làm thế nào mà Spring có thể biến một yêu cầu HTTP thô trên cổng 8080 thành phương thức Java `home()` rồi trả về trang HTML chuẩn xác như vậy?

Toàn bộ quy trình này được điều phối nhịp nhàng bởi kiến trúc **Spring MVC** thông qua chuỗi 7 bước tuần tự (Hình 7.18):



4. Once it knows what controller method to call, the dispatcher servlet calls the method. After execution, the controller method provides the view name and data that needs to be rendered in the view.

5. The dispatcher servlet finds the view and renders it with the data provided by the controller. To find the view to render, the dispatcher servlet uses a component named view resolver.

7. The browser interprets the HTTP response and displays the data.

6. Through Tomcat, the HTTP response is returned to the client.

Hình 7.18: Kiến trúc nội bộ của Spring MVC: Luồng phối hợp chặt chẽ giữa các thành phần cốt lõi (`DispatcherServlet`, `HandlerMapping`, `Controller`, `ViewResolver`) để xử lý một yêu cầu HTTP và trả về giao diện cho người dùng.

1. **Khách gửi yêu cầu:** Người dùng kích hoạt hành động trên trình duyệt, gửi gói tin HTTP request tới cổng 8080.
2. **Tomcat tiếp nhận và chuyển giao:** Máy chủ Tomcat đón nhận yêu cầu qua mạng và chuyển giao toàn bộ gói tin cho servlet trung tâm của Spring: **`DispatcherServlet`**.
3. **Tìm kiếm hành động điều khiển (*Handler Mapping*):** `DispatcherServlet` là cửa ngõ đầu vào nhưng nó chưa biết phải gọi phương thức Java nào. Nó liền hỏi thành phần bản đồ điều hướng mang tên **`HandlerMapping`**. Dựa vào đường dẫn `/home` và phương thức HTTP (GET), `HandlerMapping` tra cứu bảng ánh xạ tạo bởi `@RequestMapping` và chỉ ra phương thức `home()` trong lớp `MainController`.
4. **Thực thi phương thức Controller:** `DispatcherServlet` gọi phương thức `home()` của `MainController`. Phương thức này thực thi các logic nghiệp vụ cần thiết và trả về tên của khung nhìn (*view name*): `"home.html"`.
5. **Phân giải khung nhìn (*View Resolver*):** Nhận được chuỗi ký tự `"home.html"`, `DispatcherServlet` không thể gửi một chuỗi văn bản thuần túy cho khách; nó cần tệp tài liệu thực tế. Nó ủy thác công việc này cho thành phần **`ViewResolver`**. `ViewResolver` sẽ quét thư mục tài nguyên (`resources/static`), tìm thấy tệp `home.html` và nạp nội dung của nó.
6. **Đóng gói phản hồi HTTP:** `DispatcherServlet` nhận nội dung HTML đã được kết xuất, đưa vào phần thân của thông điệp phản hồi HTTP (*HTTP response*) và gửi lại cho Tomcat.
7. **Hiển thị trên trình duyệt:** Tomcat chuyển gói tin phản hồi HTTP qua mạng Internet về cho trình duyệt. Trình duyệt giải mã HTML và hiển thị trang web hoàn chỉnh cho người dùng.

Thành phần duy nhất bạn cần quan tâm trong Spring MVC

Hãy quan sát lại Hình 7.18: Ngoại trừ lớp **`Controller`** là phần mã nguồn nghiệp vụ do bạn tự tay viết, toàn bộ các thành phần phức tạp còn lại (`DispatcherServlet`, `HandlerMapping`, `ViewResolver`, và máy chủ Tomcat) đều đã được **Spring Boot tự động cấu hình và kết nối sẵn hoàn toàn**! Đó chính là lý do vì sao phát triển ứng dụng web với Spring Boot lại nhanh chóng và năng suất vượt trội đến như vậy.

7.4 Tóm tắt chương

- **Ứng dụng web** là mô hình ứng dụng phổ biến nhất hiện nay, gồm hai phần tách biệt: phía khách (*frontend*) chạy trên trình duyệt người dùng và phía máy chủ (*backend*) xử lý nghiệp vụ và lưu trữ dữ liệu. Hai phía giao tiếp với nhau thông qua **giao thức HTTP**.
- Trong hệ sinh thái Java, backend cần một **servlet container** (như Apache Tomcat) để đứng ra làm cầu nối chuyển đổi các gói tin mạng HTTP thành các đối tượng Java thân thiện.

- **Spring Boot** giúp đơn giản hóa triệt để việc xây dựng ứng dụng web thông qua nguyên lý **quy ước hơn cấu hình (*convention-over-configuration*)**: nó tự động cung cấp các thiết lập mặc định tối ưu nhất dựa trên các thư viện phụ thuộc có mặt trong dự án.
- **Dependency Starters** là các gói phụ thuộc hướng tính năng (như `spring-boot-starter-web`), tự động tập hợp các thư viện tương thích cần thiết mà không yêu cầu bạn phải tự chỉ định chi tiết từng phiên bản.
- **Spring Initializr** (`https://start.spring.io`) là công cụ chính thức giúp tạo nhanh bộ khung dự án Spring Boot với cấu trúc tệp `pom.xml` chuẩn mực.
- Để phục vụ một trang web tĩnh trong Spring MVC, bạn chỉ cần tạo tệp HTML trong thư mục `src/main/resources/static/` và định nghĩa một lớp điều khiển `@Controller` có phương thức trả về tên tệp HTML gắn với chú thích `@RequestMapping`.
- Trái tim điều phối của Spring MVC là servlet trung tâm **`DispatcherServlet`**. Khi có yêu cầu gửi đến, nó phối hợp với **`HandlerMapping`** để tìm controller phù hợp, gọi action thực thi, và nhờ **`ViewResolver`** nạp nội dung giao diện HTML trả về cho khách.

Chương 8

Triển khai ứng dụng web với Spring Boot và Spring MVC

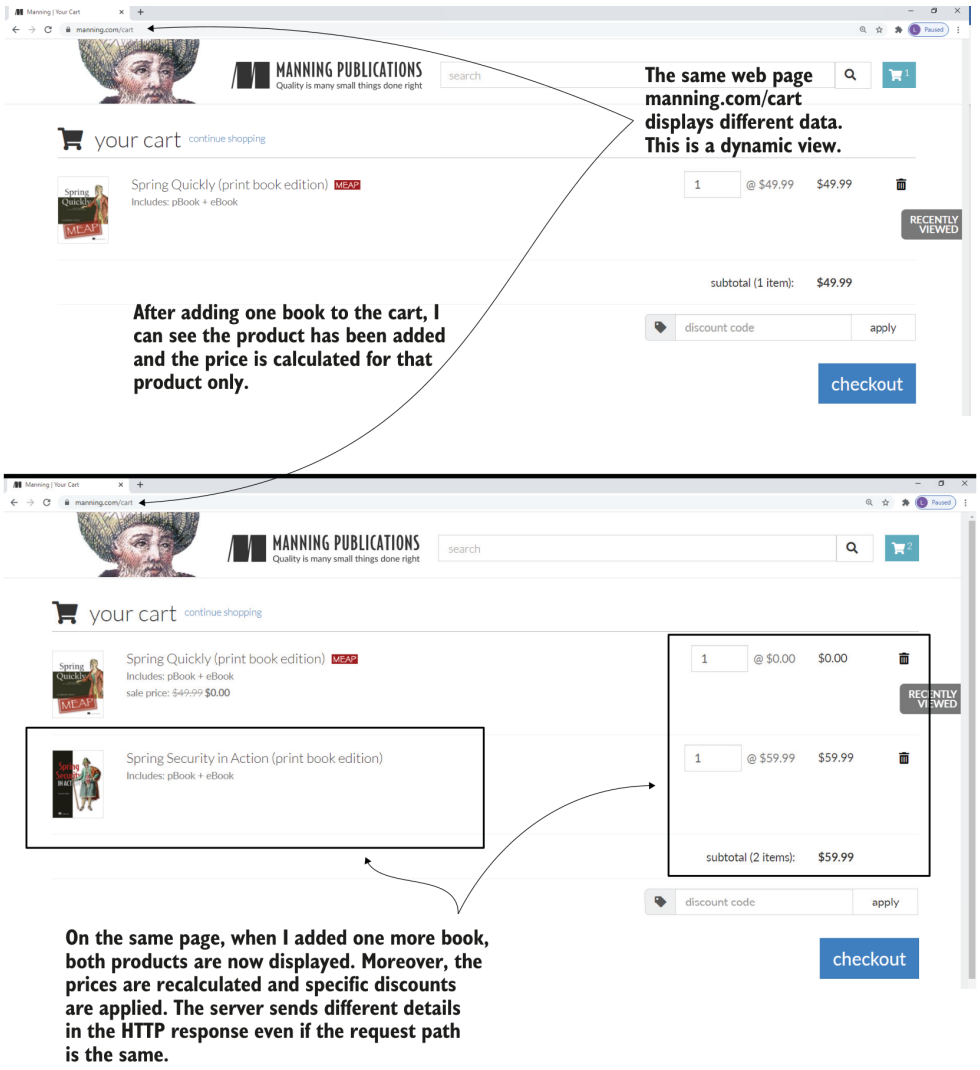
Trong chương này, chúng ta sẽ xây dựng một ứng dụng web có khung nhìn động (*dynamic view*). Hầu hết các ứng dụng phần mềm ngày nay đều cần hiển thị dữ liệu động cho người dùng. Giờ đây, đối với mỗi yêu cầu của người dùng được thể hiện qua một yêu cầu HTTP (*HTTP request*) do trình duyệt gửi đi, ứng dụng web sẽ tiếp nhận một số dữ liệu, xử lý chúng và gửi phản hồi HTTP (*HTTP response*) trở lại để trình duyệt hiển thị (Hình 8.2).

Nội dung trọng tâm của chương này

- Hiện thực hóa các khung nhìn động (*dynamic views*) trong ứng dụng web bằng việc kết hợp Spring MVC với công cụ kết xuất mẫu Thymeleaf.
- Tiếp nhận dữ liệu do client gửi lên thông qua tham số yêu cầu (*request parameters*) với chú thích `@RequestParam`.
- Trích xuất dữ liệu từ đường dẫn thông qua biến đường dẫn (*path variables*) với chú thích `@PathVariable`.
- Phân biệt và ứng dụng linh hoạt các phương thức HTTP: `GET` (truy xuất dữ liệu) và `POST` (tạo mới/thay đổi dữ liệu) cùng các chú thích ánh xạ tương ứng (`@GetMapping`, `@PostMapping`).
- Tích hợp biểu mẫu HTML (*HTML form*) để thu thập dữ liệu nhập từ người dùng và tự động liên kết dữ liệu vào mô hình (*model binding*).

Hãy quan sát một ví dụ thực tế về khung nhìn động trên trang giỏ hàng của Manning (Hình 8.1). Cùng là một địa chỉ web `manning.com/cart`, nhưng trang web lại hiển thị dữ liệu khác nhau tùy theo trạng thái mua sắm của người dùng. Khi thêm một cuốn sách, giỏ hàng chỉ hiển thị cuốn sách đó cùng mức giá tương ứng. Khi thêm cuốn sách thứ hai, cả hai sản phẩm cùng xuất hiện, tổng giá trị được tính toán lại và các chiết khấu đặc thù được áp dụng. Máy chủ gửi về các chi tiết khác nhau trong phản hồi HTTP dù cho đường dẫn yêu cầu hoàn toàn

không thay đổi!



Hình 8.1: Khung nhìn động minh họa qua chức năng giỏ hàng của Manning: Dù trang yêu cầu giữ nguyên đường dẫn, nội dung trang vẫn thay đổi linh hoạt. Phía backend trả về dữ liệu phản hồi khác nhau trước và sau khi thêm một sản phẩm vào giỏ.

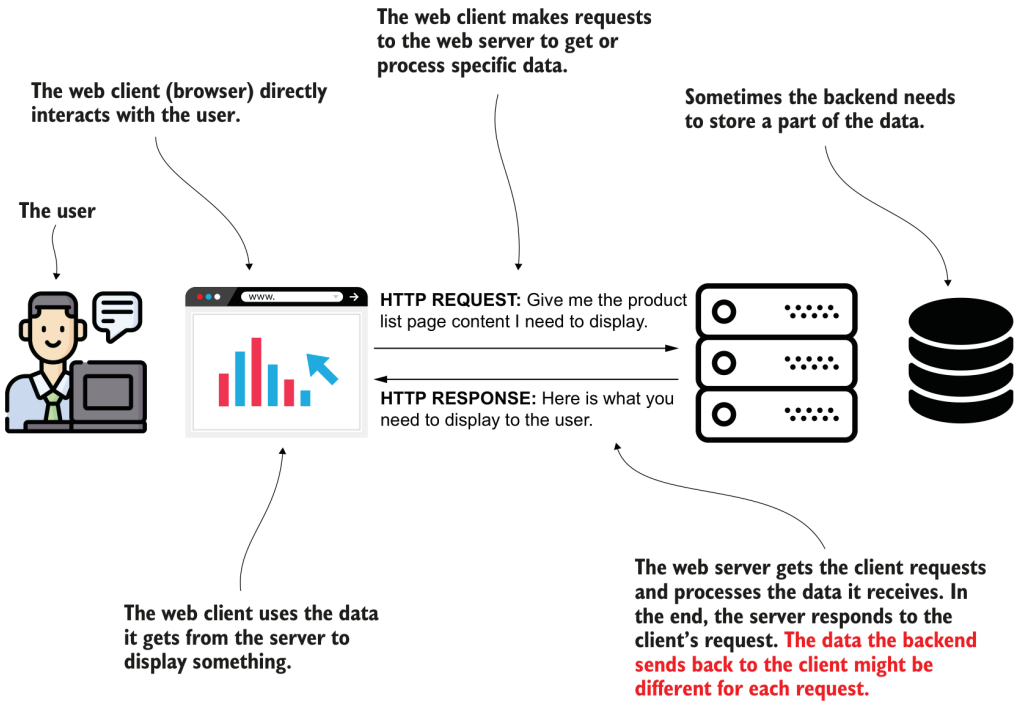
Trong Mục 8.1, chúng ta sẽ bắt đầu tìm hiểu cách thức chuyển dữ liệu từ lớp điều khiển (*controller*) sang khung nhìn (*view*) thông qua công cụ kết xuất mẫu Thymeleaf, cũng như cách thức client gửi dữ liệu lên server qua tham số truy vấn (*request parameters*) và biến đường dẫn (*path variables*). Kế đó, trong Mục 8.2, chúng ta sẽ đi sâu vào hai phương thức HTTP quan trọng nhất là GET và POST, xây dựng một ứng dụng hoàn chỉnh quản lý danh sách sản phẩm với biểu mẫu nhập liệu HTML.

8.1 Triển khai ứng dụng web với khung nhìn động

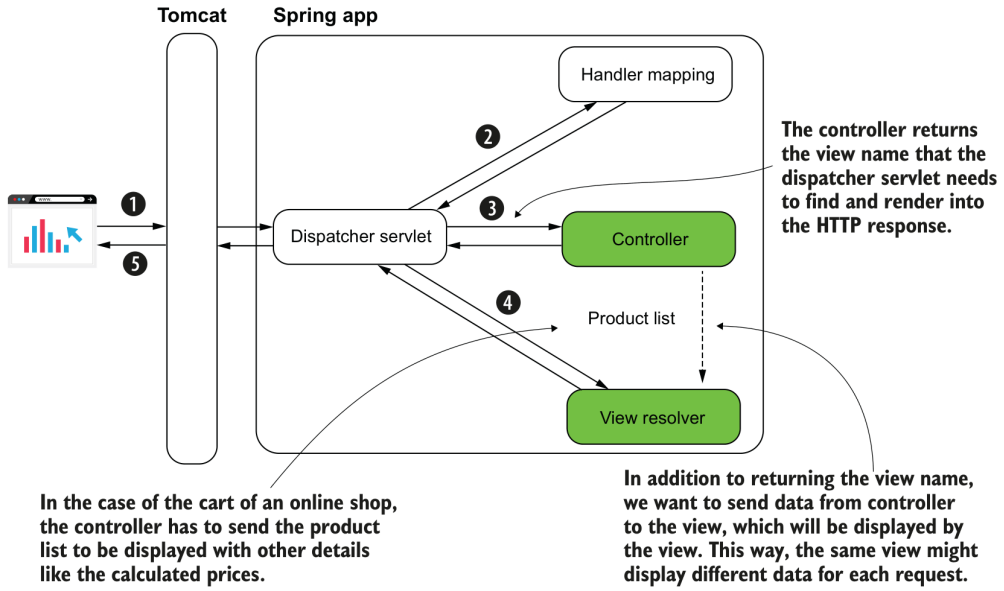
Chúng ta sẽ ôn lại luồng điều phối của Spring MVC và cùng thực hiện một ví dụ minh họa cách khung nhìn nhận các giá trị động từ controller.

Trong ví dụ chúng ta đã triển khai ở cuối Chương 7, nội dung hiển thị trên trình duyệt là tĩnh và giống hệt nhau trong mọi yêu cầu HTTP gửi tới trang. Hãy nhớ lại 5 bước trong luồng xử lý của Spring MVC (Hình 8.3):

1. Khách (*client*) gửi một yêu cầu HTTP tới máy chủ web.
2. Khối điều phối `DispatcherServlet` sử dụng bộ ánh xạ *Handler Mapping* để xác định hành động của controller cần gọi.
3. Khối `DispatcherServlet` kích hoạt thực thi hành động của controller.
4. Sau khi thực thi hành động tương ứng, controller trả về tên của khung nhìn để `DispatcherServlet` kết xuất vào phản hồi HTTP.
5. Phản hồi HTTP được gửi ngược trở lại cho client.



Hình 8.2: Client gửi dữ liệu qua yêu cầu HTTP. Phía backend tiếp nhận, xử lý và tạo phản hồi gửi lại client. Tùy thuộc vào kết quả xử lý dữ liệu của backend, các yêu cầu khác nhau sẽ mang lại dữ liệu hiển thị khác nhau cho người dùng.



Hình 8.3: Luồng xử lý Spring MVC với khung nhìn động: Để định nghĩa một khung nhìn động, controller không chỉ trả về tên view mà còn gửi kèm dữ liệu vào view. Công cụ giải quyết khung nhìn (*view resolver*) sẽ hòa trộn dữ liệu này vào tệp mẫu để tạo ra nội dung phản hồi cuối cùng.

Bước số 4 chính là nơi chúng ta cần tạo ra bước ngoặt. Chúng ta muốn controller không chỉ đơn thuần trả về tên của khung nhìn, mà còn phải gửi kèm dữ liệu sang cho khung nhìn đó. Khung nhìn sẽ lồng ghép dữ liệu này để định hình nên phản hồi HTTP cuối cùng. Nhờ đó, nếu server gửi danh sách có 1 sản phẩm, trang web sẽ hiển thị 1 sản phẩm; nếu controller gửi danh sách có 2 sản phẩm cho cùng khung nhìn đó, trang web sẽ hiển thị 2 sản phẩm (đúng như hành vi bạn quan sát thấy trong Hình 8.1).

Bây giờ, hãy cùng tạo một dự án mang tên `sq-ch8-ex1` để tìm hiểu cú pháp truyền dữ liệu từ controller sang view.

Giả sử mục tiêu ban đầu là gửi một tên người dùng và in nó ra màn hình với một màu sắc nhất định. Trong thực tế, bạn thường xuyên phải in tên người dùng đang đăng nhập lên góc trang web. Để làm được điều này, chúng ta cần một **công cụ kết xuất mẫu** (*template engine*).

Công cụ mẫu là một thư viện cho phép chúng ta dễ dàng đẩy dữ liệu từ controller sang view và hiển thị chúng theo định dạng mong muốn. Chúng ta lựa chọn **Thymeleaf** vì nó đơn giản, trực quan và các tệp mẫu Thymeleaf thực chất chỉ là các tệp HTML tĩnh tiêu chuẩn.

Trong tệp `pom.xml`, chúng ta khai báo thêm gói phụ thuộc starter cho Thymeleaf cùng với starter web:

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Listing 8.1: Khai báo phụ thuộc Thymeleaf starter trong tệp pom.xml

Trong Listing 8.2, chúng ta định nghĩa lớp controller. Ta đánh dấu phương thức bằng chú thích `@RequestMapping` để ánh xạ hành động tới đường dẫn yêu cầu `/home`. Điểm mới ở đây là chúng ta khai báo thêm một tham số kiểu `Model` cho phương thức. Tham số `Model` này là nơi lưu trữ dữ liệu mà controller muốn chuyển giao cho khung nhìn.

Để đưa một giá trị vào `Model`, chúng ta gọi phương thức `addAttribute()`. Tham số thứ nhất của `addAttribute()` là khóa định danh (*key*) duy nhất dạng chuỗi ký tự, và tham số thứ hai là giá trị (*value*) tương ứng.

```
@Controller
public class MainController {

  @RequestMapping("/home")
  public String home(Model page) {
    page.addAttribute("username", "Katy");
    page.addAttribute("color", "red");
    return "home.html";
  }
}
```

Listing 8.2: Định nghĩa hành động của trang trong lớp controller (dự án sq-ch8-ex1)

Lưu ý về lỗi Whitelabel Error Page khi gõ thiếu đường dẫn

Nhiều bạn học viên thắc mắc tại sao khi gõ trực tiếp `localhost:8080` lên thanh địa chỉ của trình duyệt mà không kèm theo đường dẫn con (như `/home`) thì lại gặp lỗi 404. Điều này hoàn toàn tự nhiên: nếu bạn chưa cấu hình bất kỳ controller nào lắng nghe đường dẫn gốc `/`, máy chủ không tìm thấy tài nguyên phù hợp và Spring Boot sẽ trả về trang báo lỗi mặc định mang tên *Whitelabel Error Page*. Để xem trang web, hãy luôn gõ đúng đường dẫn đầy đủ `http://localhost:8080/home`.

Tiếp theo, trong thư mục `src/main/resources/templates/`, chúng ta tạo tệp tin `home.html` như trong Listing 8.3.

Điểm mấu chốt đầu tiên trong tệp tin là thẻ `<html>` có thêm thuộc tính `xmlns:th="http://www.thymeleaf.`. Khai báo này tương tự như một câu lệnh `import` trong Java. Nó cho phép chúng ta sử dụng tiền tố `th:` để gọi các tính năng kết xuất do Thymeleaf cung cấp.

Bên trong phần thân trang, chúng ta sử dụng cú pháp `${attribute_key}` để truy xuất dữ liệu từ đối tượng `Model`:

- `${username}` sẽ lấy giá trị được gắn với khóa `"username"` (ở đây là `"Katy"`).
- `${color}` sẽ lấy giá trị được gắn với khóa `"color"` (ở đây là `"red"`).
- Thuộc tính `th:style` sẽ gán mã CSS động vào phần tử HTML, và `th:text` sẽ thay thế nội dung văn bản bên trong thẻ.

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
  <head>
    <meta charset="UTF-8">
    <title>Home Page</title>
  </head>
  <body>
    <h1>Welcome
    <span th:style="'color:' + ${color}"
          th:text="${username}"></span>!</h1>
  </body>
</html>
```

Listing 8.3: Tệp home.html đại diện cho khung nhìn động với Thymeleaf

Khởi chạy ứng dụng và truy cập địa chỉ `http://localhost:8080/home` trên trình duyệt web, bạn sẽ thấy kết quả hiển thị sinh động như trong Hình 8.4: từ **Katy** hiển thị bằng màu đỏ đúng như dữ liệu controller đã đẩy vào model!

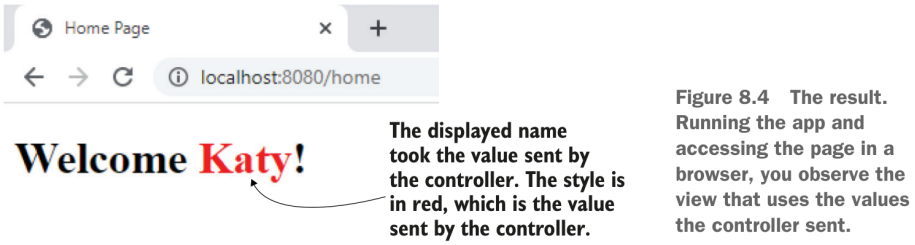


Figure 8.4 The result. Running the app and accessing the page in a browser, you observe the view that uses the values the controller sent.

Hình 8.4: Kết quả kiểm thử trên trình duyệt: Tên hiển thị tiếp nhận giá trị do controller gửi sang, với màu chữ đỏ tương ứng.

8.1.1 Tiếp nhận dữ liệu từ yêu cầu HTTP

Ở phần trước, dữ liệu gửi sang khung nhìn vẫn đang được gán cứng (*hardcoded*) bên trong mã nguồn của controller. Nhưng trong các ứng dụng thực tế, dữ liệu cần hiển thị thường bắt nguồn từ chính các tương tác của người dùng phía client:

- Khi người dùng đặt hàng trên sàn thương mại điện tử, client gửi danh sách các mã sản phẩm và số lượng lên server.
- Khi người dùng đăng bài trên diễn đàn, client gửi tiêu đề và nội dung bài viết lên server để lưu vào cơ sở dữ liệu.
- Khi người dùng đăng nhập, client gửi thông tin định danh và mật khẩu để xác thực.
- Khi người dùng gửi biểu mẫu liên hệ, client gửi họ tên, email và nội dung phản hồi lên server.

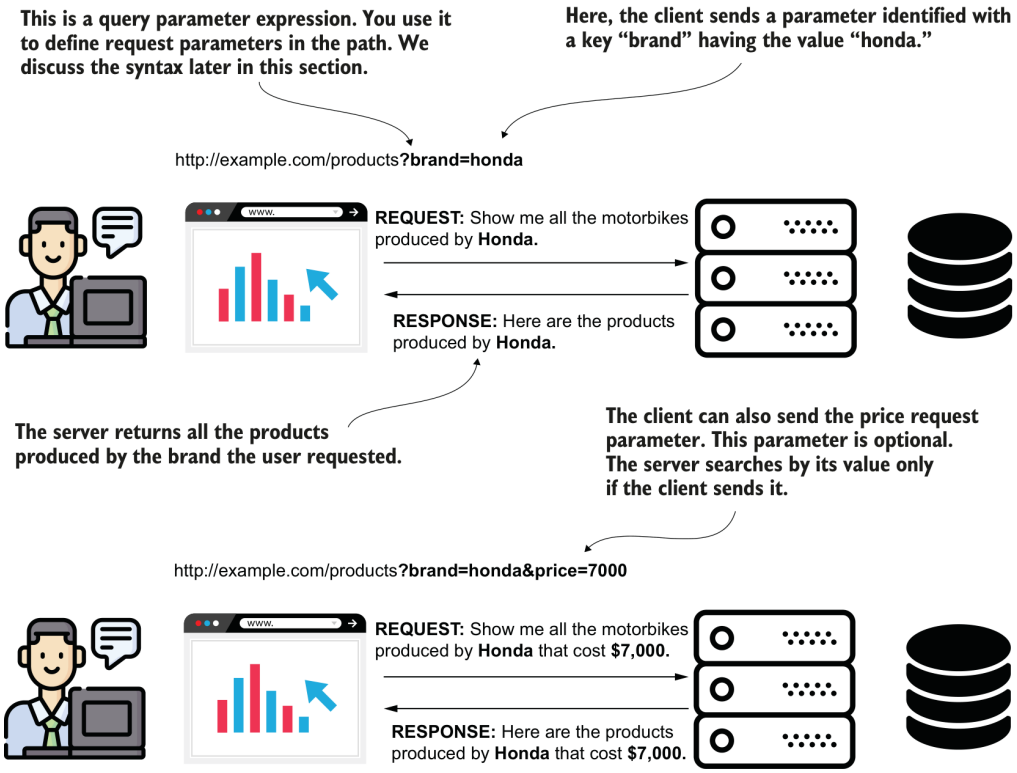
Trong giao thức HTTP, client thường truyền dữ liệu lên máy chủ thông qua các hình thức phổ biến sau:

1. **Tham số yêu cầu (*HTTP Request Parameters* / *Query Parameters*):** Dữ liệu được đính kèm trực tiếp vào chuỗi định danh URI dưới dạng các cặp khóa-giá trị (*key-value pairs*). Cách tiếp cận này rất gọn nhẹ và phù hợp cho các khối dữ liệu nhỏ.
2. **Tiêu đề yêu cầu (*HTTP Request Headers*):** Dữ liệu được gửi trong phần header của yêu cầu HTTP (thường dùng để truyền thông tin xác thực, token, kiểu dữ liệu mong muốn, v.v.).
3. **Thân yêu cầu (*HTTP Request Body*):** Dữ liệu được đặt trong phần thân của gói tin HTTP (thường dùng khi truyền tải khối lượng dữ liệu lớn hoặc phức tạp như biểu mẫu form, đối tượng JSON, tải tệp tin).
4. **Biến đường dẫn (*Path Variables*):** Giá trị dữ liệu được đặt trực tiếp như một phân đoạn con nằm trong cấu trúc đường dẫn URI.

Chúng ta sẽ lần lượt tìm hiểu chi tiết cách xử lý tham số yêu cầu và biến đường dẫn ngay sau đây.

8.1.2 Sử dụng tham số yêu cầu (Request Parameters) để truyền dữ liệu

Một kịch bản cực kỳ kinh điển của tham số yêu cầu là triển khai tính năng **tìm kiếm và lọc dữ liệu** (Hình 8.5). Giả sử hệ thống hiển thị danh sách xe máy có các thuộc tính: tên, giá tiền, và thương hiệu. Người dùng có thể lọc theo giá, lọc theo thương hiệu, hoặc kết hợp cả hai. Khi đó, client sẽ gửi các tiêu chí tìm kiếm này dưới dạng các tham số truy vấn tùy chọn (*optional request parameters*) gắn sau dấu chấm hỏi `?` trên URL.



Hình 8.5: Tham số yêu cầu có thể mang tính tùy chọn: Kịch bản phổ biến nhất là triển khai chức năng lọc và tìm kiếm, nơi các tiêu chí lọc có thể được cung cấp hoặc để trống tùy ý người dùng.

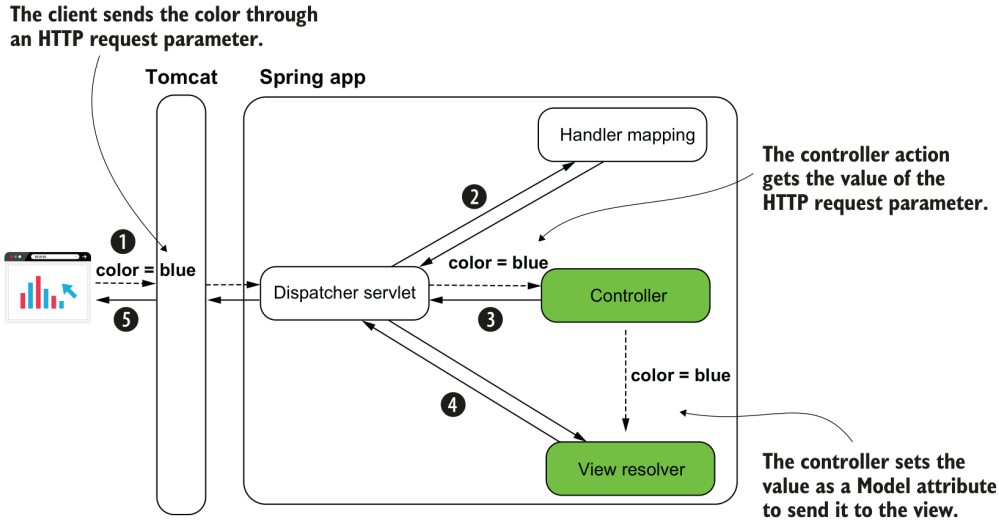
Bây giờ, chúng ta hãy điều chỉnh ứng dụng mẫu sang dự án `sq-ch8-ex2` để cho phép client tự chỉ định màu sắc hiển thị thông qua một request parameter.

Để đón nhận giá trị từ một tham số yêu cầu, ta chỉ cần khai báo một tham số đầu vào trong phương thức hành động của controller và gắn chú thích `@RequestParam` lên tham số đó (Listing 8.4). Chú thích `@RequestParam` báo hiệu cho Spring biết rằng nó cần tự động trích xuất giá trị từ tham số HTTP có tên trùng khớp với tên biến để truyền vào phương thức.

```
@Controller
public class MainController {

  @RequestMapping("/home")
  public String home(
      @RequestParam String color,
      Model page) {
    page.addAttribute("username", "Katy");
    page.addAttribute("color", color);
    return "home.html";
  }
}
```

Listing 8.4: Đón nhận giá trị tham số yêu cầu bằng chú thích @RequestParam (dự án sq-ch8-ex2)

Hình 8.6 phác họa lại hành trình di chuyển của tham số `color` từ trình duyệt xuyên qua các tầng của Spring MVC cho đến khi được hiển thị lên khung nhìn.



Hình 8.6: Đường đi của giá trị tham số trong Spring MVC: Client gửi tham số `color=blue` qua yêu cầu HTTP. Lớp điều khiển tiếp nhận giá trị thông qua `@RequestParam`, đưa vào `Model`, và chuyển tiếp tới khung nhìn Thymeleaf để hiển thị.

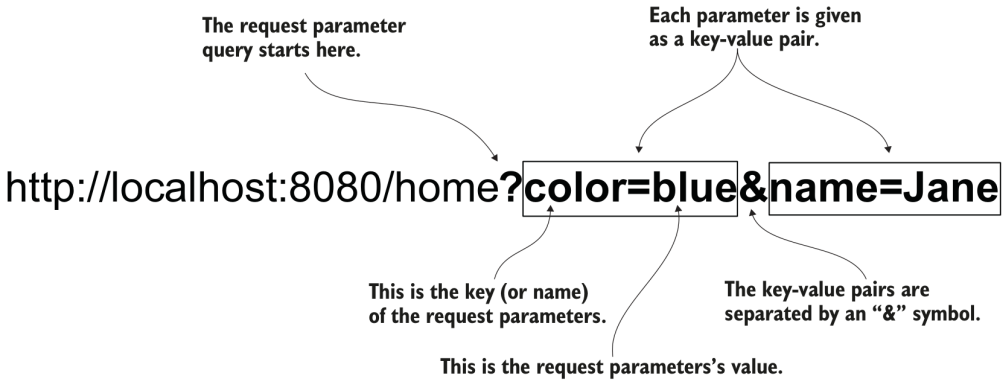
Khởi chạy ứng dụng và kiểm thử bằng cách truy cập địa chỉ kèm theo chuỗi truy vấn:

```
http://localhost:8080/home?color=blue
```

Bạn sẽ thấy tên **Katy** lập tức đổi sang màu xanh da trời! Nếu đổi thành `color=green`, chữ sẽ chuyển sang màu xanh lá cây.

Cú pháp tổng quát khi truyền tham số yêu cầu qua URL được tổng kết trong Hình 8.7:

- Dấu hỏi chấm `?` đánh dấu điểm bắt đầu của chuỗi truy vấn tham số.
- Mỗi tham số được biểu diễn dưới dạng cặp `khóa=giá_trị` (*key=value*).
- Nếu có nhiều tham số, các cặp khóa-giá trị được phân tách với nhau bằng ký tự dấu và `&`.



Hình 8.7: Cấu trúc cú pháp của tham số yêu cầu trên URL: Bắt đầu bằng dấu `?`, các cặp `khóa=giá_trị` được kết nối với nhau thông qua ký tự `&`.

Mặc định tham số @RequestParam là bắt buộc (mandatory)

Theo mặc định trong Spring MVC, một tham số được gắn chú thích `@RequestParam` là bắt buộc phải có mặt trong yêu cầu. Nếu client gửi yêu cầu mà không truyền tham số đó (ví dụ truy cập thẳng `http://localhost:8080/home`), máy chủ Spring sẽ lập tức từ chối và phản hồi mã lỗi HTTP **400 Bad Request**.

Nếu bạn muốn tham số đó trở thành tùy chọn (*optional*), bạn có thể cấu hình thuộc tính `required = false` trong chú thích:

```
@RequestParam(required = false) String color
```

Khi client không truyền giá trị, biến `color` sẽ nhận giá trị `null` mà không gây ra lỗi.

Nếu muốn nhận nhiều tham số (ví dụ cả `color` và `name` như trong dự án `sq-ch8-ex3`), chúng ta chỉ cần khai báo thêm tham số tương ứng trong phương thức controller:

```
@RequestMapping("/home")
public String home(
    @RequestParam(required = false) String color,
    @RequestParam(required = false) String name,
    Model page) {
  page.addAttribute("username", name);
  page.addAttribute("color", color);
  return "home.html";
}
```

Khi đó, client có thể gọi yêu cầu với URL:

```
http://localhost:8080/home?color=blue&name=Jane
```

8.1.3 Sử dụng biến đường dẫn (Path Variables) để truyền dữ liệu

Một kỹ thuật truyền dữ liệu phổ biến khác từ client lên server là sử dụng **biến đường dẫn** (*path variables*). Thay vì đặt dữ liệu sau dấu hỏi chấm `?`, chúng ta gắn trực tiếp giá trị vào một phân đoạn cụ thể của đường dẫn URL:

- **Truyền qua tham số yêu cầu:** `http://localhost:8080/home?color=blue`
- **Truyền qua biến đường dẫn:** `http://localhost:8080/home/blue`

Với biến đường dẫn, giá trị không còn đi kèm với tên khóa trên URL nữa. Máy chủ backend sẽ tự động bóc tách giá trị dựa trên vị trí chính xác của nó trên mẫu đường dẫn được định nghĩa trước.

Bảng 8.1 so sánh sự khác biệt then chốt giữa hai phương pháp này.

Tiêu chí	Tham số yêu cầu (Request Parameters)	Biến đường dẫn (Path Variables)
Hình thức trên URL	`/home?color=blue&name=Jane`	`/home/blue`
Tính bắt buộc	Mặc định bắt buộc, nhưng dễ dàng cấu hình tùy chọn (`required=false`).	Luôn nên dùng cho dữ liệu bắt buộc.
Số lượng tham số	Thích hợp khi truyền nhiều tham số kết hợp (tìm kiếm, phân trang, lọc nâng cao).	Chỉ nên dùng cho 1 hoặc tối đa 2 giá trị định danh chính.
Trường hợp sử dụng điển hình	Bộ lọc tìm kiếm, sắp xếp, tiêu chí tùy ý.	Định danh tài nguyên cụ thể (ví dụ ID người dùng, ID bài viết: `/users/123`).

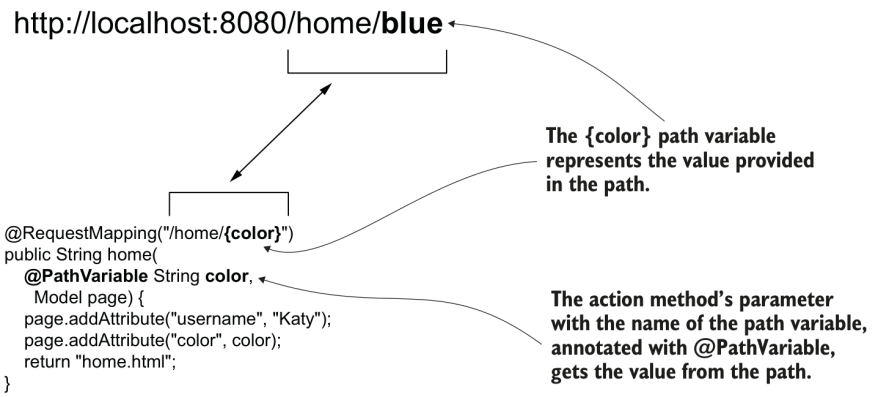
Bảng 8.1: So sánh giữa tham số yêu cầu (*Request Parameters*) và biến đường dẫn (*Path Variables*)

Để định nghĩa một biến đường dẫn trong controller (dự án `sq-ch8-ex4`), ta đặt tên biến bên trong cặp ngoặc nhọn `{...}` ở chuỗi mẫu đường dẫn của `@RequestMapping`, sau đó gắn chú thích `@PathVariable` lên tham số của phương thức (Listing 8.5).

```
@Controller
public class MainController {

  @RequestMapping("/home/{color}")
  public String home(
      @PathVariable String color,
      Model page) {
```

```
8       page.addAttribute("username", "Katy");
9       page.addAttribute("color", color);
10      return "home.html";
11    }
12  }
```

Listing 8.5: Sử dụng biến đường dẫn với chú thích @PathVariable (dự án sq-ch8-ex4)



Hình 8.8: Cơ chế hoạt động của biến đường dẫn: Phân đoạn `/blue` trên URL khớp với mẫu `/{color}`, và giá trị `"blue"` được Spring tự động tiêm vào biến `color` thông qua `@PathVariable`.

Khởi chạy ứng dụng và thử truy cập các URL khác nhau:

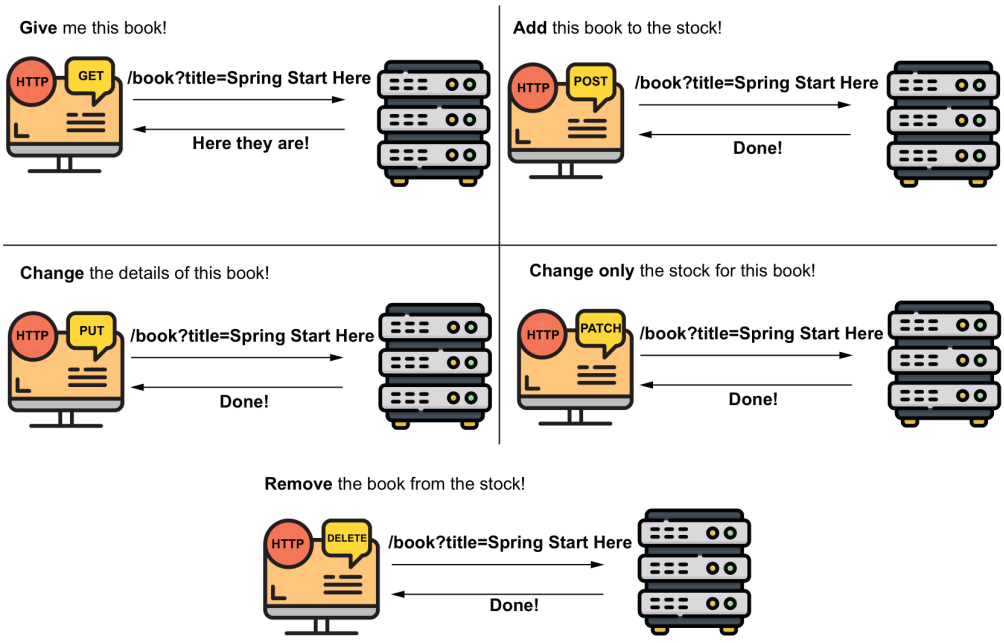
- `http://localhost:8080/home/blue` → Tên Katy hiển thị màu xanh da trời.
- `http://localhost:8080/home/red` → Tên Katy hiển thị màu đỏ.
- `http://localhost:8080/home/green` → Tên Katy hiển thị màu xanh lá cây.

8.2 Sử dụng các phương thức HTTP GET và POST

Trong các ví dụ từ đầu sách tới nay, chúng ta mới chỉ ngầm sử dụng phương thức **HTTP GET**. Mỗi yêu cầu HTTP trong thực tế luôn được định danh bởi một cặp: **đường dẫn (*path*) + phương thức (*HTTP method / verb*)**. Phương thức HTTP đóng vai trò là một động từ thể hiện **ý đồ hành động** của client đối với tài nguyên máy chủ.

Phương thức	Ý đồ hành vi của Client	Tính chất an toàn / Bền vững
`GET`	Yêu cầu truy xuất, đọc dữ liệu từ máy chủ.	An toàn (*Safe*): Không làm thay đổi trạng thái dữ liệu trên server.
`POST`	Yêu cầu tạo mới một bản ghi hoặc gửi dữ liệu để máy chủ xử lý.	Không an toàn: Làm thay đổi trạng thái dữ liệu máy chủ.
`PUT`	Cập nhật/thay thế toàn bộ một bản ghi dữ liệu hiện có.	Có tính lũy biến (*Idempotent*).
`PATCH`	Cập nhật một phần thuộc tính của bản ghi dữ liệu hiện có.	Không bắt buộc lũy biến.
`DELETE`	Xóa bỏ một bản ghi dữ liệu khỏi hệ thống.	Có tính lũy biến (*Idempotent*).

Bảng 8.2: Các phương thức HTTP cơ bản thường gặp trong các ứng dụng web



Hình 8.9: Các phương thức HTTP cơ bản: `GET` để lấy dữ liệu, `POST` để tạo mới, `PUT` để cập nhật toàn bộ, `PATCH` để cập nhật cục bộ một phần, và `DELETE` để xóa dữ liệu.

Không bao giờ sử dụng sai mục đích thiết kế của phương thức HTTP!

Về mặt kỹ thuật, bạn hoàn toàn có thể viết một phương thức `GET` làm thay đổi hoặc xóa sạch dữ liệu trong cơ sở dữ liệu. Nhưng làm như vậy là một sai lầm kiến trúc cực kỳ nghiêm trọng! Các trình thu thập thông tin web (*web crawlers*), bộ nhớ đệm trình duyệt (*browser cache*) hay tính năng tải trước liên kết (*link prefetching*) đều mặc định coi `GET` là thao tác an toàn không gây tác dụng phụ. Nếu bạn dùng `GET` để xóa dữ liệu, một bộ tìm kiếm có thể vô tình xóa sạch tài nguyên hệ thống của bạn khi quét liên kết! Hãy luôn tuân thủ nghiêm ngặt chuẩn mực thiết kế của từng phương thức HTTP.

8.2.1 Xây dựng ứng dụng quản lý sản phẩm với GET và POST

Để thấy rõ sức mạnh của việc kết hợp các phương thức HTTP, chúng ta sẽ xây dựng dự án `sq-ch8-ex5`: một ứng dụng web quản lý danh sách sản phẩm gồm hai ca sử dụng (*use cases*):

1. **Xem danh sách toàn bộ sản phẩm:** Sử dụng phương thức `HTTP GET`.
2. **Thêm một sản phẩm mới vào danh sách:** Sử dụng phương thức `HTTP POST` thông qua biểu mẫu nhập liệu HTML (*HTML form*).

Trước hết, chúng ta tạo lớp mô hình `Product` nằm trong gói `model` để đại diện cho một sản phẩm (Listing 8.6).

```
1  package com.example.model;
2
3  public class Product {
4    private String name;
5    private double price;
6
7    public String getName() {
8      return name;
9    }
10
11   public void setName(String name) {
12     this.name = name;
13   }
14
15   public double getPrice() {
16     return price;
17   }
18
19   public void setPrice(double price) {
20     this.price = price;
```

```
  }
}
```

Listing 8.6: Lớp Product mô tả thuộc tính tên và giá của sản phẩm

Kế đó, chúng ta xây dựng lớp dịch vụ `ProductService` trong gói `service` để quản lý danh sách lưu trữ trong bộ nhớ và thực thi các ca nghiệp vụ (Listing 8.7).

```
package com.example.service;

import java.util.ArrayList;
import java.util.List;
import org.springframework.stereotype.Service;
import com.example.model.Product;

@Service
public class ProductService {

  private List<Product> products = new ArrayList<>();

  public void addProduct(Product p) {
    products.add(p);
  }

  public List<Product> findAll() {
    return products;
  }
}
```

Listing 8.7: Lớp ProductService triển khai các ca sử dụng của ứng dụng

Lưu ý về thiết kế danh sách trong bộ nhớ

Cách tổ chức lưu trữ danh sách trong thuộc tính của bean singleton ở đây là một sự đơn giản hóa có chủ đích để bạn tập trung trọn vẹn vào việc nắm bắt cơ chế của các phương thức HTTP. Trong môi trường doanh nghiệp thực tế, các yêu cầu web chạy đa luồng đồng thời, việc thay đổi trực tiếp một `ArrayList` thông thường có thể dẫn đến tranh chấp tài nguyên (*race condition*). Ở các chương sau, chúng ta sẽ thay thế danh sách tạm thời này bằng cơ sở dữ liệu quan hệ hoàn chỉnh.

Bây giờ, chúng ta hiện thực hóa ca sử dụng thứ nhất: hiển thị danh sách sản phẩm qua phương thức `HTTP GET` trên đường dẫn `/products`. Trước hết, chúng ta tạo lớp `ProductsController` tiêm phụ thuộc `ProductService` (Listing 8.8).

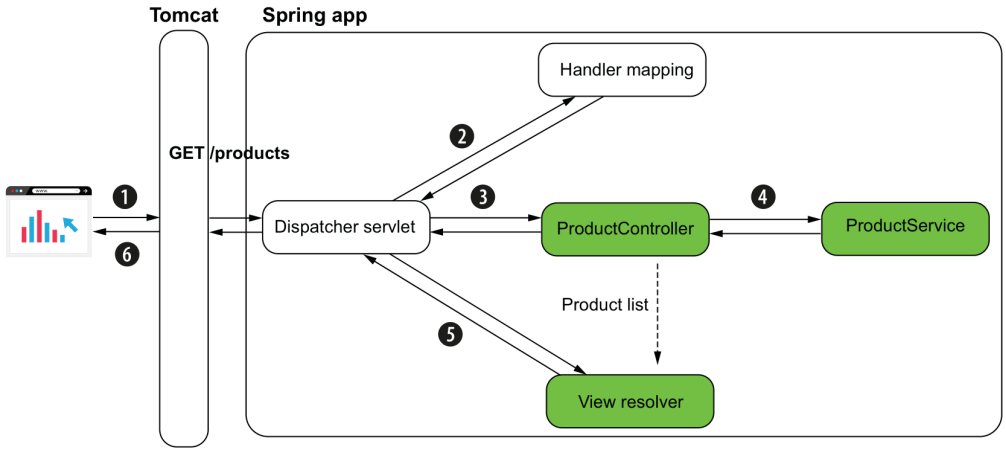
```
@Controller
public class ProductsController {

  private final ProductService productService;

  public ProductsController(ProductService productService) {
    this.productService = productService;
  }
  // ươPhng ứthc hành động ẽs được ổb sung ở listing ếtip theo
}
```

Listing 8.8: Lớp ProductsController sử dụng dịch vụ để thực thi các ca sử dụng

Tiếp theo, chúng ta định nghĩa phương thức hành động gửi danh sách sản phẩm sang khung nhìn (Listing 8.9). Lớp `ProductsController` nhận yêu cầu từ `DispatcherServlet`, truy vấn danh sách từ `ProductService`, đưa danh sách này vào `Model` với khóa `"products"`, và yêu cầu khung nhìn `products.html` kết xuất thành bảng HTML (Hình 8.10).

```
  @GetMapping("/products")
  public String viewProducts(Model model) {
    var products = productService.findAll();
    model.addAttribute("products", products);
    return "products.html";
  }
```

Listing 8.9: Gửi danh sách sản phẩm sang cho khung nhìn



Hình 8.10: Luồng thực thi khi gọi `GET /products`: Lớp điều khiển lấy danh sách từ dịch vụ và đưa vào model. Phản hồi HTTP chứa bảng HTML kết xuất danh sách sản phẩm.

Trong tệp khung nhìn `products.html`, chúng ta sử dụng thuộc tính `th:each` của Thymeleaf

để lặp qua từng phần tử trong danh sách `products` và hiển thị vào các hàng của bảng HTML (Listing 8.10).

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Danh sách ảsn ẩphm</title>
</head>
<body>
    <h1>Danh sách ảsn ẩphm</h1>
    <h2>ấTt ả̉c ảsn ẩphm ệhin có</h2>
    <table>
        <tr>
            <th>Tên ảsn ẩphm</th>
            <th>Đơn giá</th>
        </tr>
        <tr th:each="p : ${products}">
            <td th:text="${p.name}"></td>
            <td th:text="${p.price}"></td>
        </tr>
    </table>
</body>
</html>
```

Listing 8.10: Khung nhìn products.html hiển thị danh sách sản phẩm trên trang

8.2.2 Xử lý thêm sản phẩm với phương thức HTTP POST

Bây giờ chúng ta sẽ bổ sung ca sử dụng thứ hai: thêm một sản phẩm mới. Để thêm mới dữ liệu, chúng ta sử dụng phương thức `HTTP POST`.

Tại sao lại có thể dùng chung đường dẫn `/products` cho cả hai hành động? Bởi vì Spring MVC phân biệt các hành động không chỉ bằng đường dẫn URL mà còn bằng chính phương thức HTTP!

- Gọi `GET /products` → Kích hoạt xem danh sách sản phẩm.
- Gọi `POST /products` → Kích hoạt thêm mới sản phẩm.

Trong `@RequestMapping`, bạn có thể chỉ định phương thức thông qua thuộc tính `method = RequestMethod.POST` (Listing 8.11). Tuy nhiên, cách làm ngắn gọn và phổ biến nhất trong giới lập trình Spring chuyên nghiệp là sử dụng các chú thích chuyên biệt:

- `@GetMapping("/products")` thay cho `@RequestMapping(path="/products", method=RequestMe`

- `@PostMapping("/products")` thay cho `@RequestMapping(path="/products", method=RequestM`

```
@PostMapping("/products")
public String addProduct(
    @RequestParam String name,
    @RequestParam double price,
    Model model
) {
  Product p = new Product();
  p.setName(name);
  p.setPrice(price);
  productService.addProduct(p);

  var products = productService.findAll();
  model.addAttribute("products", products);
  return "products.html";
}
```

Listing 8.11: Triển khai phương thức hành động để thêm sản phẩm mới

Listing 8.12 tổng hợp toàn bộ lớp điều khiển `ProductsController` hoàn chỉnh, xử lý đồng thời cả hai phương thức `GET` và `POST`.

```
package com.example.controllers;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;
import com.example.model.Product;
import com.example.service.ProductService;

@Controller
public class ProductsController {

  private final ProductService productService;

  public ProductsController(ProductService productService) {
    this.productService = productService;
  }

  @GetMapping("/products")
  public String viewProducts(Model model) {
```

```
    var products = productService.findAll();
    model.addAttribute("products", products);
    return "products.html";
  }

  @PostMapping("/products")
  public String addProduct(
      @RequestParam String name,
      @RequestParam double price,
      Model model
  ) {
    Product p = new Product();
    p.setName(name);
    p.setPrice(price);
    productService.addProduct(p);

    var products = productService.findAll();
    model.addAttribute("products", products);
    return "products.html";
  }
}
```

Listing 8.12: Lớp ProductsController hoàn chỉnh xử lý cả GET và POST

Để cho phép người dùng nhập thông tin và phát yêu cầu `HTTP POST`, chúng ta bổ sung một biểu mẫu HTML (*HTML form*) vào tệp `products.html` (Listing 8.13). Biểu mẫu này có thuộc tính `action="/products"` và `method="post"`.

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>åQun lý åsn ẩphm</title>
</head>
<body>
    <h1>åQun lý åsn ẩphm</h1>
    <h2>ấTt ảc åsn ẩphm ệhin có</h2>
    <table>
        <tr>
            <th>Tên</th>
            <th>Giá</th>
        </tr>
        <tr th:each="p : ${products}">
```

```
            <td th:text="${p.name}"></td>
            <td th:text="${p.price}"></td>
        </tr>
    </table>

    <h2>Thêm ảsn ẩphm ớmi</h2>
    <form action="/products" method="post">
        Tên: <input type="text" name="name"><br />
        Giá: <input type="number" step="any" name="price"><br />
        <button type="submit">Thêm ảsn ẩphm</button>
    </form>
</body>
</html>
```

Listing 8.13: Bổ sung biểu mẫu HTML form vào products.html để gửi yêu cầu POST

Hình 8.11: Giao diện hoàn chỉnh của ứng dụng quản lý sản phẩm: Người dùng xem danh sách sản phẩm trong bảng và có thể nhập tên, đơn giá rồi bấm nút gửi biểu mẫu để thêm sản phẩm mới.

Khởi chạy ứng dụng và truy cập `http://localhost:8080/products`. Ban đầu bảng sản phẩm trống trơn. Khi bạn nhập *"Kẹo"* và giá *"5.0"* rồi bấm *"Thêm sản phẩm"*, trình duyệt sẽ gửi gói tin `POST` chứa hai tham số `name` và `price`. Controller xử lý, lưu sản phẩm vào danh sách, và tải lại trang với bảng sản phẩm đã được bổ sung bản ghi mới (Hình 8.11)!

8.2.3 Rút gọn mã nguồn bằng cách liên kết trực tiếp đối tượng Model

Trong Listing 8.12, chúng ta đã sử dụng từng chú thích `@RequestParam` đơn lẻ để đón nhận từng trường dữ liệu, sau đó khởi tạo đối tượng `Product` rồi gán thủ công từng giá trị thông

qua hàm setter. Cách làm này rất trực quan cho người mới bắt đầu.

Tuy nhiên, Spring Framework còn hỗ trợ một cơ chế mạnh mẽ hơn rất nhiều mang tên **tự động liên kết mô hình** (*Model Binding* / *Command Object*). Khi tên các trường trong biểu mẫu form (hoặc tên các tham số HTTP) trùng khớp hoàn toàn với tên các thuộc tính trong lớp đối tượng mô hình `Product`, Spring có khả năng tự động khởi tạo đối tượng `Product` và gán toàn bộ giá trị vào đó mà bạn không cần phải viết bất kỳ dòng mã gán thủ công nào (Listing 8.14, dự án `sq-ch8-ex6`)!

```
@Controller
public class ProductsController {

  private final ProductService productService;

  public ProductsController(ProductService productService) {
    this.productService = productService;
  }

  // Các ươphng ứthc khác ữgi nguyên...

  @PostMapping("/products")
  public String addProduct(
      Product p,
      Model model
  ) {
    productService.addProduct(p);

    var products = productService.findAll();
    model.addAttribute("products", products);
    return "products.html";
  }
}
```

Listing 8.14: Sử dụng trực tiếp đối tượng Product làm tham số của hành động controller (dự án sq-ch8-ex6)

Lời khuyên dành cho lập trình viên mới làm quen với Spring

Khi đọc các bài viết hay mã nguồn dự án thực tế trên mạng, bạn sẽ bắt gặp rất nhiều cú pháp rút gọn kỳ diệu như việc đặt trực tiếp đối tượng `Product p` vào tham số phương thức như trên. Nếu chưa hiểu rõ cơ chế đằng sau, bạn sẽ rất băn khoăn không biết đối tượng này từ đâu rơi xuống. Hãy luôn ghi nhớ rằng Spring có vô vàn quy ước ngầm định nhằm tối giản hóa mã nguồn cho lập trình viên. Mỗi khi gặp một cú pháp lạ mắt, hãy tìm hiểu tài liệu đặc tả của framework để hiểu rõ nguyên lý vận hành bên dưới thay vì chỉ sao chép thụ động.

8.3 Tóm tắt chương

- Một ứng dụng web có **khung nhìn động** (*dynamic view*) hiển thị nội dung tùy biến dựa trên dữ liệu do controller gửi sang thay vì chỉ trả về các tệp tin HTML tĩnh.
- Công cụ kết xuất mẫu (*template engine*) như **Thymeleaf** giúp dễ dàng lồng ghép dữ liệu động từ controller vào các tệp giao diện HTML thông qua đối tượng `Model` và các thuộc tính tiền tố `th:`.
- Client có thể gửi dữ liệu lên server thông qua hai kỹ thuật chính:
  - **Tham số yêu cầu (*Request Parameters*):** Đính kèm vào URI sau dấu `?`, tiếp nhận trong controller bằng chú thích `@RequestParam`. Mặc định là bắt buộc nhưng có thể đặt `required=false` cho các tham số tùy chọn (như bộ lọc tìm kiếm).
  - **Biến đường dẫn (*Path Variables*):** Đặt trực tiếp vào phân đoạn đường dẫn URL (ví dụ `/home/{color}`), tiếp nhận trong controller bằng chú thích `@PathVariable`. Phù hợp cho các định danh tài nguyên mang tính bắt buộc.
- Mỗi yêu cầu HTTP được xác định bởi sự kết hợp giữa **đường dẫn (*path*)** và **phương thức HTTP (*HTTP method*)**.
- Các phương thức HTTP căn bản bao gồm:
  - **`GET`:** Truy xuất dữ liệu mà không làm thay đổi trạng thái của hệ thống.
  - **`POST`:** Tạo mới dữ liệu hoặc kích hoạt xử lý nghiệp vụ trên máy chủ.
  - **`PUT`:** Cập nhật hoặc thay thế toàn bộ một bản ghi tài nguyên.
  - **`PATCH`:** Cập nhật cục bộ một phần thuộc tính của bản ghi tài nguyên.
  - **`DELETE`:** Xóa bỏ tài nguyên khỏi hệ sinh thái máy chủ.
- Thông qua biểu mẫu HTML tiêu chuẩn của trình duyệt, bạn chỉ có thể gửi trực tiếp các phương thức `GET` và `POST`. Để gửi các phương thức khác như `PUT` hay `DELETE`, bạn cần sử dụng mã kịch bản phía khách (như JavaScript/AJAX).
- Trong Spring MVC, lập trình viên thường sử dụng các chú thích chuyên biệt ngắn gọn như `@GetMapping` và `@PostMapping` thay cho cú pháp dài dòng của `@RequestMapping`.
- Cơ chế **Model Binding** của Spring cho phép tự động liên kết các tham số HTTP vào các

thuộc tính của đối tượng mô hình nếu tên của chúng trùng khớp nhau, giúp loại bỏ đáng kể lượng mã boilerplate thừa thãi.

Chương 9

Sử dụng các phạm vi web của Spring

Trong Chương 5, bạn đã tìm hiểu về các phạm vi bean (*bean scopes*) căn bản: phạm vi đơn thể (*singleton*) và phạm vi nguyên mẫu (*prototype*). Nhưng đối với một ứng dụng web, vòng đời của dữ liệu còn gắn bó mật thiết với các khái niệm của giao thức HTTP như yêu cầu (*request*) và phiên làm việc (*session*). Để giải quyết nhu cầu đó, Spring cung cấp thêm một nhóm phạm vi chuyên biệt mang tên **các phạm vi web** (*web scopes*) (Hình 9.1).

Nội dung trọng tâm của chương này

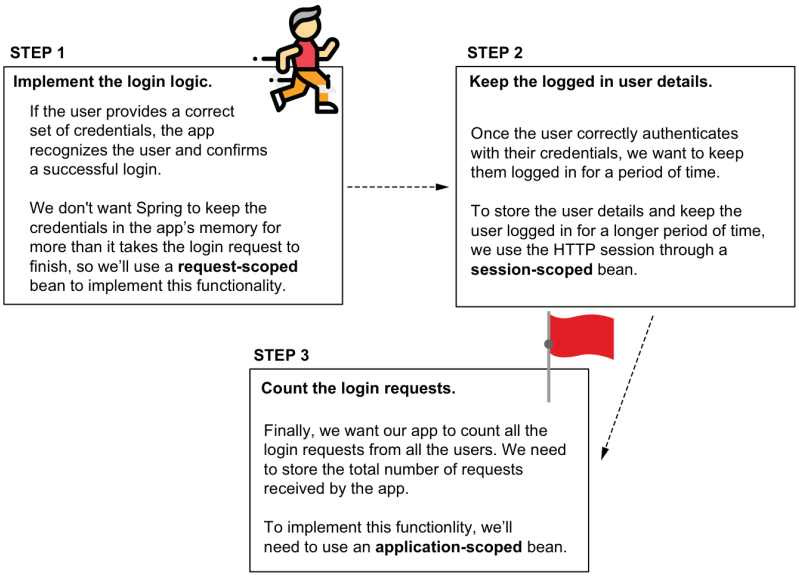
- Hiểu rõ bản chất và thời điểm sử dụng **phạm vi yêu cầu** (*request scope*) thông qua chú thích `@RequestScope`.
- Quản lý trạng thái xuyên suốt nhiều yêu cầu của cùng một người dùng bằng **phạm vi phiên** (*session scope*) với chú thích `@SessionScope`.
- Chia sẻ dữ liệu cho toàn bộ ứng dụng và tất cả người dùng bằng **phạm vi ứng dụng** (*application scope*) với chú thích `@ApplicationScope`.
- Khám phá cơ chế đối tượng ủy nhiệm (*Spring scoped proxy*) giúp kết nối an toàn giữa các bean có phạm vi vòng đời khác nhau (ví dụ tiêm một bean request-scope hoặc session-scope vào một controller singleton).
- Xây dựng hoàn chỉnh luồng nghiệp vụ xác thực đăng nhập (*login*), điều hướng bảo vệ trang (*redirection*), và đăng xuất (*logout*).

In this chapter, you'll learn that in web apps you can use other bean scopes that are relevant only to web applications. We call them *web scopes*:

- *Request scope*—Spring creates an instance of the bean class for every HTTP request. The instance exists only for that specific HTTP request.
- *Session scope*—Spring creates an instance and keeps the instance in the server's memory for the full HTTP session. Spring links the instance in the context with the client's session.
- *Application scope*—The instance is unique in the app's context, and it's available while the app is running.

To teach you how these web scopes work in a Spring application, we'll work on an example in which we implement a login functionality. Most of the web apps today offer their users the possibility to log in and access an account, so the example is also relevant from a real-world perspective.

In section 9.1, we'll use a request-scoped bean to take the user's credentials for login and make sure the app uses them only for the login request. Then, in section 9.2, we'll use a session-scoped bean to store all the relevant details we need to keep for the logged-in user as long as the user remains logged in. In section 9.3, we'll use the application-scoped bean to add a capability to count logins. Figure 9.1 shows you the steps we take to implement this app.



Hình 9.1: Các phạm vi web của Spring: Chúng ta sẽ hiện thực hóa tính năng đăng nhập bằng cách phối hợp linh hoạt phạm vi yêu cầu (để xử lý dữ liệu của từng lần đăng nhập) và phạm vi phiên (để duy trì thông tin người dùng đã đăng nhập thành công).

Trong Mục 9.1, chúng ta sẽ bắt đầu với phạm vi yêu cầu (`@RequestScope`). Kế tiếp, trong Mục 9.2, chúng ta sẽ nghiên cứu phạm vi phiên (`@SessionScope`) để giải quyết bài toán duy trì trạng thái đăng nhập. Cuối cùng, trong Mục 9.3, chúng ta sẽ tìm hiểu phạm vi ứng dụng (`@ApplicationScope`) và so sánh nó với phạm vi singleton truyền thống.

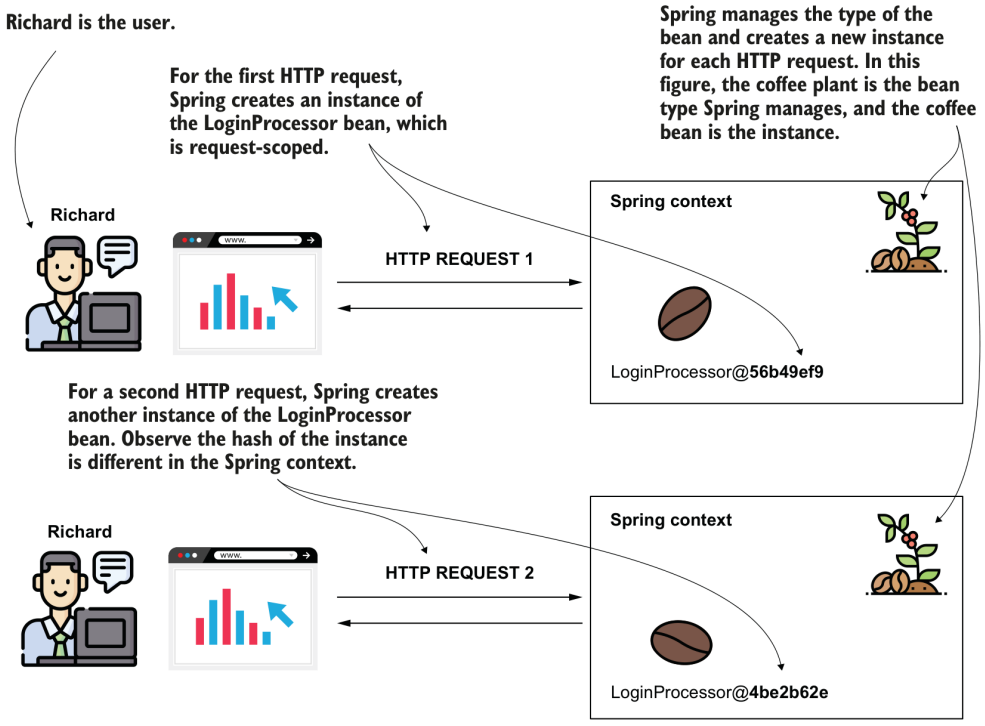
9.1 Sử dụng phạm vi yêu cầu (Request scope) trong ứng dụng Spring

Khi một bean được định nghĩa với **phạm vi yêu cầu** (*request scope*), Spring sẽ khởi tạo một thực thể bean mới cho **mỗi yêu cầu HTTP riêng biệt** do client gửi đến máy chủ. Thực thể bean này chỉ tồn tại trong suốt vòng đời xử lý yêu cầu đó và sẽ bị bộ thu gom rác (*Garbage Collector*) dọn dẹp ngay sau khi phản hồi HTTP được gửi đi (Hình 9.2).

9.1 Using the request scope in a Spring web app

In this section, you'll learn how to use request-scoped beans in Spring web apps. As you learned in chapters 7 and 8, web apps are focused on HTTP requests and responses. For this reason, and often in web apps, certain functionalities are easier to manage if Spring offers you a way to manage the bean life cycle in relationship with the HTTP request.

A request-scoped bean is an object managed by Spring, for which the framework creates a new instance for every HTTP request. The app can use the instance only for the request that created it. Any new HTTP request (from the same or other clients) creates and uses a different instance of the same class (figure 9.2).



Hình 9.2: Mô hình hoạt động của phạm vi yêu cầu: Với mỗi yêu cầu HTTP gửi đến, Spring tạo ra một thực thể bean hoàn toàn mới trong ngữ cảnh. Khi yêu cầu hoàn tất và phản hồi được gửi đi, thực thể bean đó sẽ bị giải phóng.

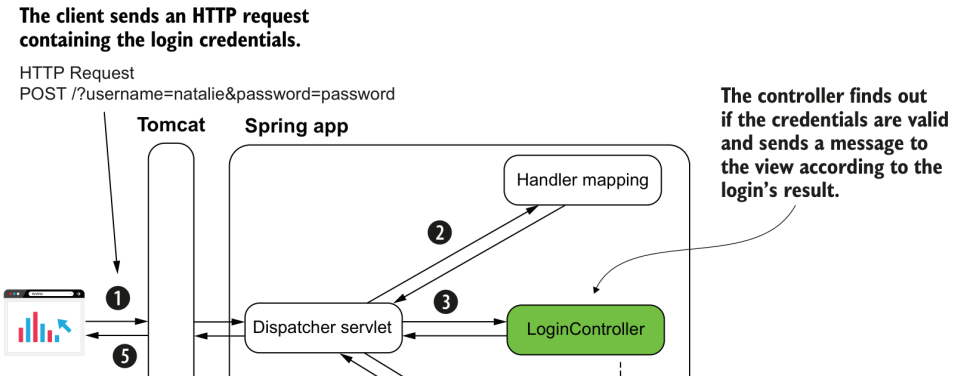
Tại sao chúng ta lại cần phạm vi yêu cầu? Hãy nhớ lại rằng trong ứng dụng web đa luồng, mỗi yêu cầu HTTP được phục vụ bởi một luồng (*thread*) riêng. Nếu chúng ta lưu trữ dữ liệu xử lý của yêu cầu trong một bean singleton, các luồng chạy song song sẽ tranh chấp và ghi

đè dữ liệu lên nhau (*race condition*). Ngược lại, nếu sử dụng một bean có phạm vi request, mỗi luồng sẽ thao tác trên một bản thể riêng biệt, loại bỏ hoàn toàn nguy cơ rò rỉ dữ liệu giữa các yêu cầu mà không cần phải đồng bộ hóa (*synchronization*) phức tạp.

9.1.1 Hiện thực hóa chức năng đăng nhập với Request Scope

Chúng ta sẽ cùng xây dựng dự án `sq-ch8-ex1` (trong mã nguồn mẫu là `sq-ch9-ex1`) để làm rõ cơ chế này qua bài toán xác thực đăng nhập (Hình 9.3).

> Let's create a Spring Boot project and add the needed dependencies. You will find this example in the project "sq-ch9-ex1." You can add the dependencies directly when creating the project (for example, using start.spring.io) or afterward in your pom.xml. For this example, we will use the web dependency and Thymeleaf as a templating engine (like we did in chapter 8). The next code snippet shows the dependencies you need to have in your pom.xml file:
>
> ```
> <dependency>
>    <groupId>org.springframework.boot</groupId>
>    <artifactId>spring-boot-starter-thymeleaf</artifactId>
> </dependency>
> <dependency>
>    <groupId>org.springframework.boot</groupId>
>    <artifactId>spring-boot-starter-web</artifactId>
> </dependency>
> ```
>
> We'll create a page that contains a login form asking for a user's name and password. The app compares the username and the password with a set of credentials it knows (in my case, user "natalie" with password "password"). If we provide correct credentials (they match with the credentials the app knows), then the page displays a message "You are now logged in" under the login form. If the credentials we provide are not correct, then the app displays a message: "Login failed."
>
> As you learned in chapters 7 and 8, we need to implement a page (representing our view) and a controller class. The controller sends the message it needs to display to the view according to the login's result (figure 9.3).



Hình 9.3: Kiến trúc chức năng đăng nhập: Người dùng gửi thông tin xác thực qua form HTML, controller tiếp nhận yêu cầu và ủy thác nghiệp vụ kiểm tra cho lớp xử lý đăng nhập (*LoginProcessor*).

Trước tiên, chúng ta xây dựng giao diện đăng nhập `login.html` đặt trong thư mục `src/main/resources/te` (Listing 9.1). Trang web chứa một biểu mẫu HTML gửi yêu cầu `POST` tới đường dẫn gốc `/`

kèm theo hai trường nhập liệu `username` và `password`, cùng một đoạn văn bản hiển thị thông báo kết quả `${message}`.

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Trang đăng ậnhp</title>
</head>
<body>
    <h1>Đăng ậnhp</h1>
    <form action="/" method="post">
        Tài ảkhon: <input type="text" name="username"><br />
        ậMt ẩkhu: <input type="password" name="password"><br />
        <button type="submit">Đăng ậnhp</button>
    </form>
    <p th:text="${message}"></p>
</body>
</html>
```

Listing 9.1: Định nghĩa trang đăng nhập login.html (Listing 9.1)

Tiếp theo, chúng ta tạo lớp `LoginController` để điều phối các yêu cầu. Listing 9.2 ánh xạ phương thức `HTTP GET` trên đường dẫn gốc `/` để hiển thị trang đăng nhập.

```
package com.example.controllers;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class LoginController {

  @GetMapping("/")
  public String loginGet() {
    return "login.html";
  }
}
```

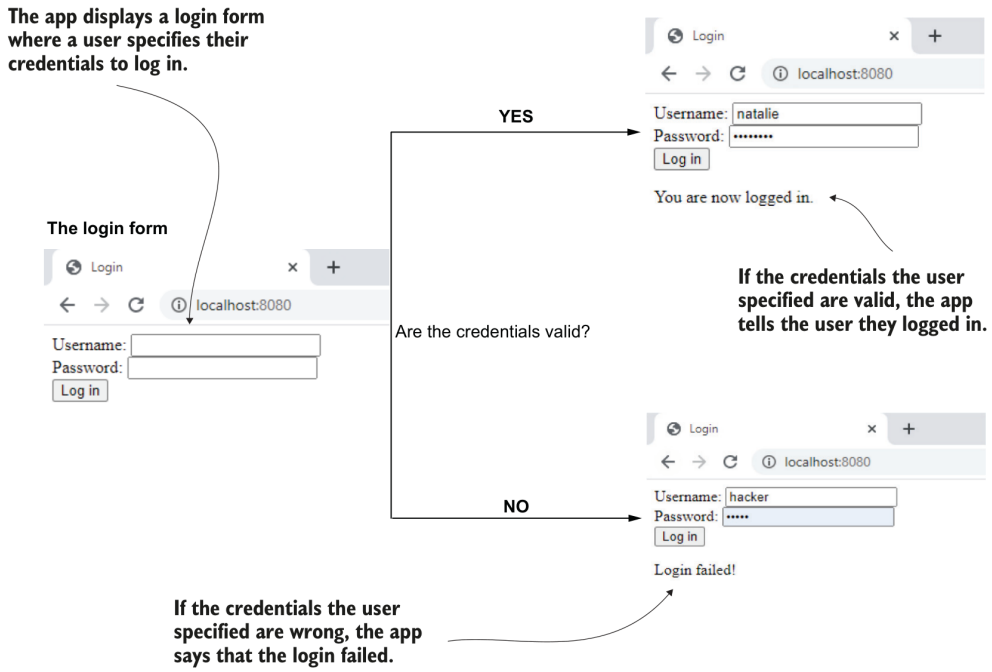
Listing 9.2: Ánh xạ hành động hiển thị trang đăng nhập bằng GET (Listing 9.2)

Để xử lý yêu cầu đăng nhập khi người dùng nhấn nút gửi biểu mẫu, chúng ta bổ sung phương thức `loginPost()` xử lý `HTTP POST` (Listing 9.3, Hình 9.4).

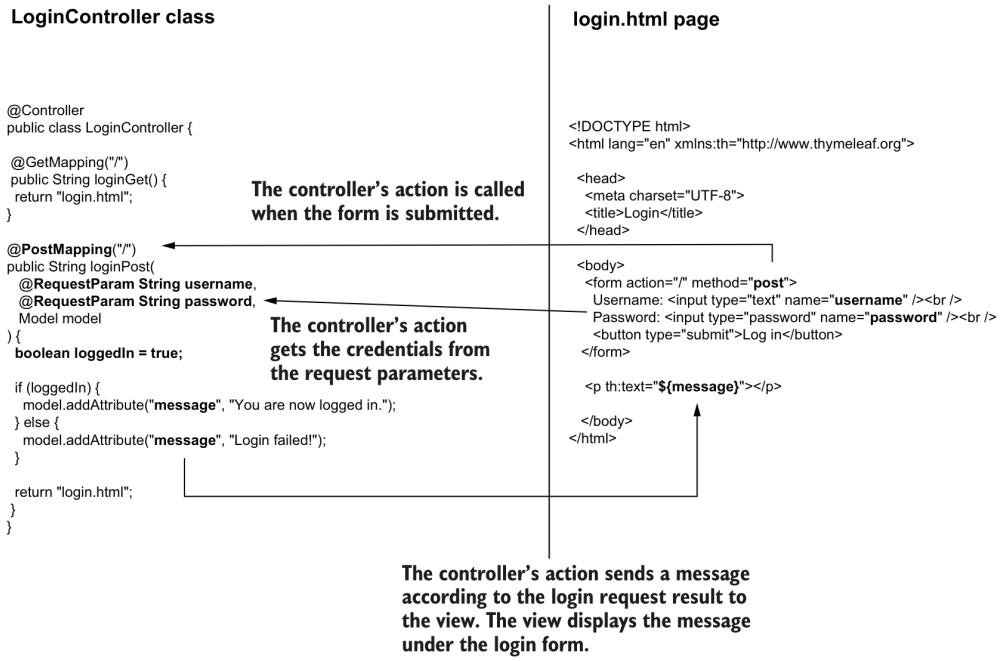
```
  @PostMapping("/")
```

```
  public String loginPost(
      @RequestParam String username,
      @RequestParam String password,
      Model model
  ) {
    boolean loggedIn = false; // logic xác ựthc ẽs được ổb sung

    if (loggedIn) {
      model.addAttribute("message", "ạBn đã đăng ậnhp thành công!");
    } else {
      model.addAttribute("message", "Đăng ậnhp ấtht ạbi!");
    }

    return "login.html";
  }
```

Listing 9.3: Đón nhận yêu cầu đăng nhập bằng HTTP POST (Listing 9.3)



Hình 9.4: Chức năng đăng nhập ban đầu: Khi người dùng bấm nút, yêu cầu POST được gửi lên, controller kiểm tra thông tin và đặt thông báo kết quả vào model để hiển thị lại trên trang.



Hình 9.5: Mối liên hệ giữa controller và khung nhìn: Các tham số từ form HTML được tiêm vào phương thức thông qua `@RequestParam`, sau đó kết quả được chuyển giao vào view thông qua `Model`.

Bây giờ, chúng ta tách biệt logic xử lý đăng nhập vào một lớp riêng biệt mang tên `LoginProcessor` có phạm vi yêu cầu (Listing 9.4). Để chỉ định phạm vi này, chúng ta sử dụng chú thích `@RequestScope`.

```
1  package com.example.processor;
2
3  import org.springframework.stereotype.Component;
4  import org.springframework.web.context.annotation.RequestScope;
5
6  @Component
7  @RequestScope
8  public class LoginProcessor {
9
10   private String username;
11   private String password;
12
13   public boolean login() {
14     String username = this.getUsername();
15     String password = this.getPassword();
16
17     if ("natalie".equals(username) && "password".equals(password)) {
18       return true;
```

```
    } else {
      return false;
    }
  }

  public String getUsername() {
    return username;
  }

  public void setUsername(String username) {
    this.username = username;
  }

  public String getPassword() {
    return password;
  }

  public void setPassword(String password) {
    this.password = password;
  }
}
```

Listing 9.4: Lớp LoginProcessor có phạm vi yêu cầu thực thi logic xác thực (Listing 9.4)

9.1.2 Cơ chế Scoped Proxy của Spring đằng sau Request Scope

Hãy dừng lại một chút để suy ngẫm về một vấn đề kiến trúc thú vị: Lớp `LoginController` là một bean có phạm vi **singleton** (được khởi tạo duy nhất một lần khi ứng dụng khởi động). Trong khi đó, lớp `LoginProcessor` lại có phạm vi **request** (được khởi tạo mới ở mỗi yêu cầu HTTP).

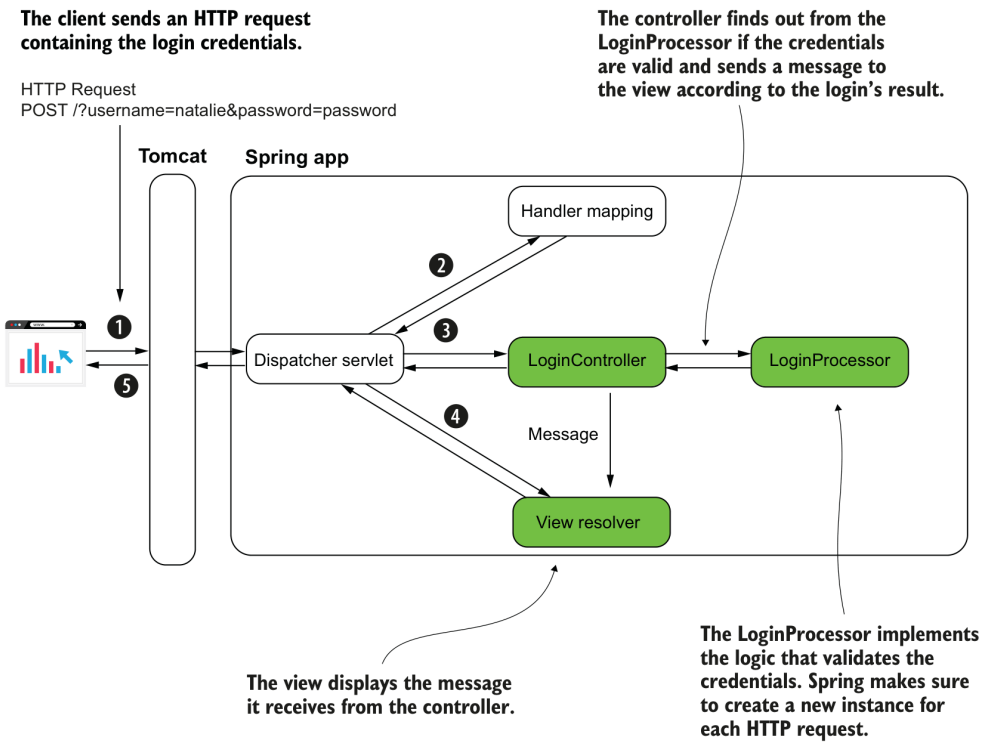
Làm thế nào một bean sống vĩnh viễn (singleton) lại có thể tiêm một bean có vòng đời ngắn ngủi từng giây từng phút (request-scoped)?

Câu trả lời nằm ở cơ chế **đối tượng ủy nhiệm theo phạm vi** (*Scoped Proxy*) (Hình 9.6). Spring không tiêm trực tiếp thực thể thật của `LoginProcessor` vào controller. Thay vào đó, Spring tiêm một đối tượng proxy đại diện. Khi phương thức của controller gọi một hàm trên proxy này, proxy sẽ kiểm tra ngữ cảnh yêu cầu HTTP hiện tại của luồng thực thi, tìm kiếm (hoặc khởi tạo mới) bean `LoginProcessor` tương ứng với yêu cầu đó, rồi chuyển tiếp lệnh gọi hàm sang thực thể thật!

Now we have a controller and a view, but where is the request scope in all of this? The only class we wrote is the `LoginController`, and we left it a singleton, which is the default Spring scope. We don't need to change the scope for `LoginController` as long as it doesn't store any detail in its attributes. But remember, we need to implement the login logic. The login logic depends on the user's credentials, and we have to take into consideration two things about these credentials:

1. The credentials are sensitive details, and you don't want to store them in the app's memory for longer than the login request.
2. More users with different credentials might attempt to log in simultaneously.

Considering these two points, we need to make sure that if we use a bean for implementing the login logic, each instance is unique for each HTTP request. We need to use a request-scoped bean. We'll extend the app as presented in figure 9.5. We add a request-scoped bean `LoginProcessor`, which takes the credentials on the request and validates them (figure 9.6).



Hình 9.6: Cơ chế Scoped Proxy: Spring tiêm một proxy vào controller singleton. Mỗi khi phương thức được gọi, proxy đóng vai trò trung gian chuyển hướng cuộc gọi đến đúng thực thể bean được tạo riêng cho yêu cầu HTTP đó.

Chúng ta cập nhật lại `LoginController` để sử dụng `LoginProcessor`:

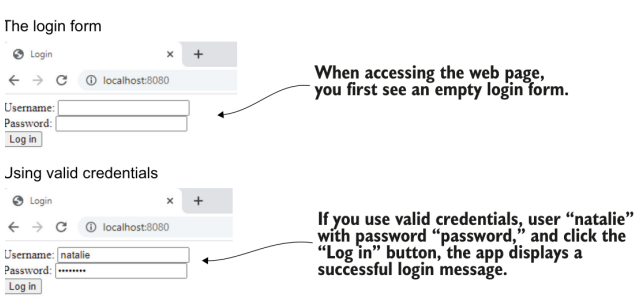
```
@Controller
public class LoginController {

  private final LoginProcessor loginProcessor;

  public LoginController(LoginProcessor loginProcessor) {
    this.loginProcessor = loginProcessor;
  }
```

```
9
10    @GetMapping("/")
11    public String loginGet() {
12      return "login.html";
13    }
14
15    @PostMapping("/")
16    public String loginPost(
17        @RequestParam String username,
18        @RequestParam String password,
19        Model model
20    ) {
21      loginProcessor.setUsername(username);
22      loginProcessor.setPassword(password);
23      boolean loggedIn = loginProcessor.login();
24
25      if (loggedIn) {
26        model.addAttribute("message", "ạBn đã đăng ậnhp thành công!");
27      } else {
28        model.addAttribute("message", "Đăng ậnhp ấtht ạbi!");
29      }
30
31      return "login.html";
32    }
33  }
```

Listing 9.5: Tiêm LoginProcessor vào LoginController để xử lý đăng nhập

Khi chạy ứng dụng và thử nghiệm trên trình duyệt (Hình 9.7):

- Nếu nhập đúng tài khoản `natalie` và mật khẩu `password`, màn hình báo thông điệp *"Bạn đã đăng nhập thành công!"*.
- Nếu nhập sai, màn hình báo *"Đăng nhập thất bại!"*.



Hình 9.7: Kết quả kiểm thử trên trình duyệt: Ứng dụng thông báo kết quả đăng nhập chính xác theo thông tin tài khoản người dùng gửi lên.

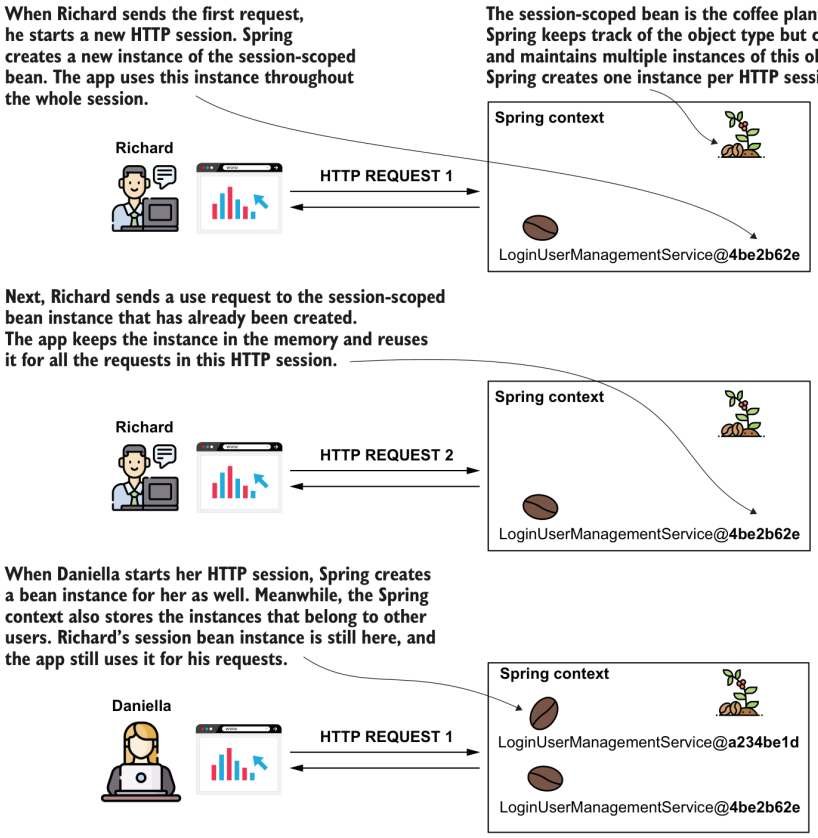
9.2 Sử dụng phạm vi phiên (Session scope) trong ứng dụng Spring

Mặc dù ứng dụng ở Mục 9.1 đã kiểm tra được thông tin đăng nhập, nó vẫn gặp phải một thiếu sót lớn: ngay sau khi yêu cầu kết thúc, server sẽ ”quên sạch” việc người dùng đã đăng nhập thành công! Nếu người dùng chuyển sang một trang khác trong hệ thống, họ lại bị coi như một khách vãng lai chưa xác thực.

Để giải quyết vấn đề này, chúng ta cần một cơ chế lưu giữ trạng thái xuyên suốt nhiều yêu cầu liên tiếp từ cùng một người dùng. Đó chính là sứ mệnh của **phạm vi phiên** (*session scope*) (Hình 9.8).

9.2 Using the session scope in a Spring web app

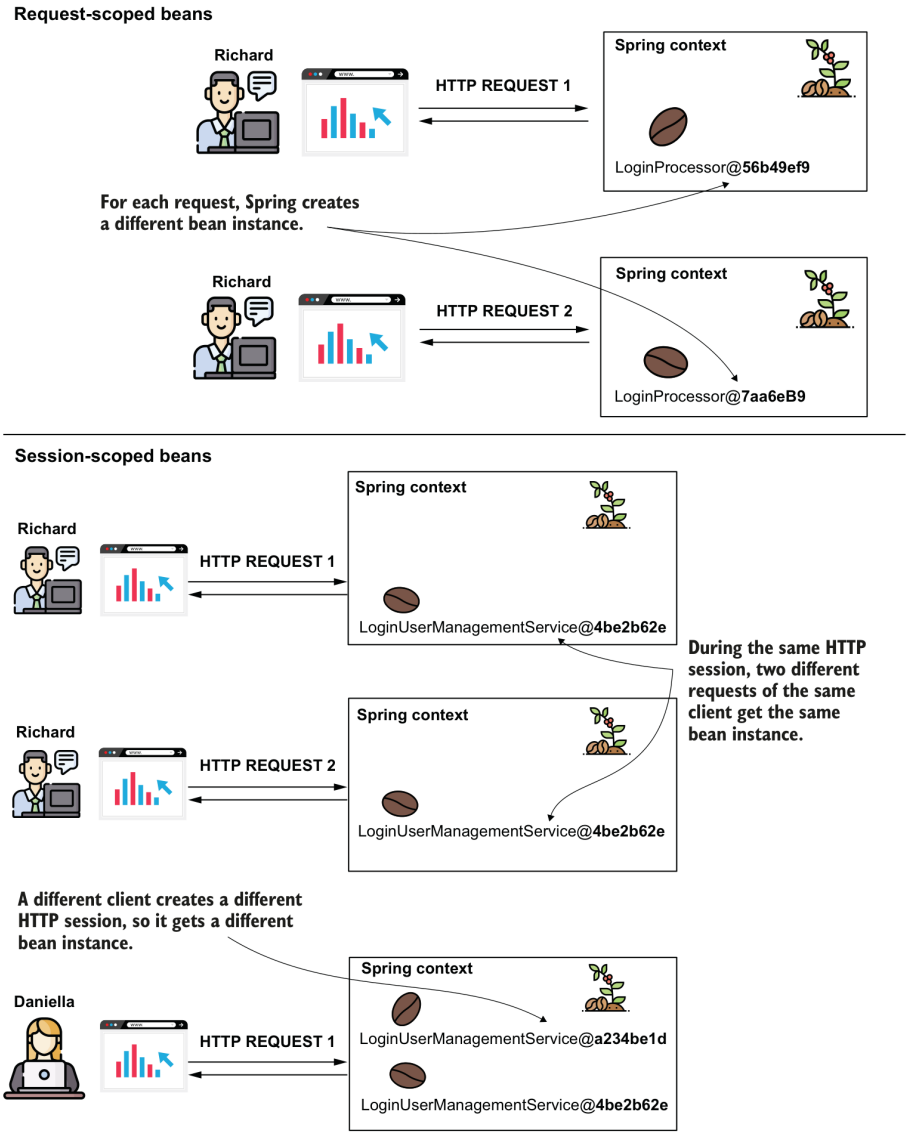
In this section, we discuss session-scoped beans. When you enter a web app and log in, you expect to then surf through that app's pages, and the app still remembers you've logged in. A session-scoped bean is an object managed by Spring, for which Spring creates an instance and links it to the HTTP session. Once a client sends a request to the server, the server reserves a place in the memory for this request, for the whole duration of their session. Spring creates an instance of a session-scoped bean when the HTTP session is created for a specific client. That instance can be reused for the same client while it still has the HTTP session active. The data you store in the session-scoped bean attribute is available for all the client's requests throughout an HTTP session. This approach of storing the data allows you to store information about what users do while they're surfing through the pages of your app.



Hình 9.8: Phạm vi phiên làm việc: Bean có phạm vi phiên được liên kết với một client duy nhất và tồn tại xuyên suốt nhiều yêu cầu HTTP từ client đó. Khi client gửi yêu cầu tiếp theo, Spring tái sử dụng chính xác thực thể bean của phiên đó.

Bản chất bên dưới của phiên làm việc trong ứng dụng web là cơ chế **HTTP Session**. Khi một client gửi yêu cầu lần đầu tiên, máy chủ tạo ra một phiên làm việc và gửi về cho trình duyệt một mã định danh phiên duy nhất (thường thông qua cookie mang tên `JSESSIONID`). Ở tất cả các yêu cầu tiếp theo, trình duyệt tự động gửi kèm cookie `JSESSIONID` này, giúp máy chủ nhận diện chính xác phiên tương ứng để tải lại dữ liệu đã lưu.

Hình 9.9 minh họa sự khác biệt trực quan giữa bean có phạm vi yêu cầu (`@RequestScope`) và bean có phạm vi phiên (`@SessionScope`).



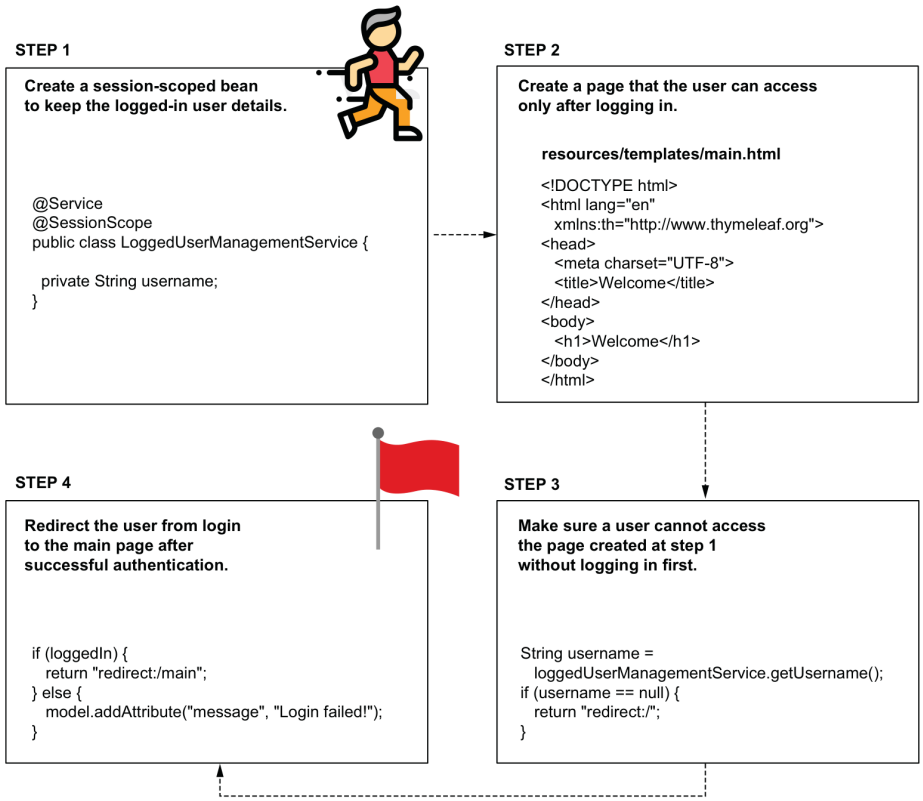
Hình 9.9: So sánh Request Scope vs Session Scope: Với Request Scope, mỗi yêu cầu tạo một thực thể bean mới (dù là cùng một người dùng). Với Session Scope, nhiều yêu cầu từ cùng một người dùng sẽ chia sẻ chung một thực thể bean duy nhất; chỉ khi có một client khác xuất hiện thì một thực thể session mới được sinh ra.

9.2.1 Xây dựng dịch vụ quản lý người dùng đăng nhập với Session Scope

Chúng ta tạo dự án `sq-ch9-ex2` để hoàn thiện luồng đăng nhập và quản lý phiên theo 4 bước chiến lược (Hình 9.10):

1. Tạo một bean session-scope để ghi nhớ tên người dùng đã đăng nhập.
2. Tạo trang chủ (*main page*) chỉ dành riêng cho người dùng đã đăng nhập hợp lệ.
3. Ngăn chặn người dùng chưa đăng nhập truy cập trang chủ (tự động điều hướng về trang login).

4. Tự động chuyển hướng (*redirect*) người dùng từ trang login sang trang chủ sau khi xác thực thành công.



Hình 9.10: Bốn bước triển khai luồng phiên đăng nhập: Sử dụng bean session-scope làm trung tâm kết nối trạng thái giữa trang đăng nhập và trang chủ.

Trước tiên (Bước 1), chúng ta định nghĩa lớp dịch vụ `LoggedUserManagementService` và gắn chú thích `@SessionScope` (Listing 9.6).

```
1  package com.example.service;
2
3  import org.springframework.stereotype.Service;
4  import org.springframework.web.context.annotation.SessionScope;
5
6  @Service
7  @SessionScope
8  public class LoggedUserManagementService {
9
10   private String username;
11
12   public String getUsername() {
13     return username;
14   }
```

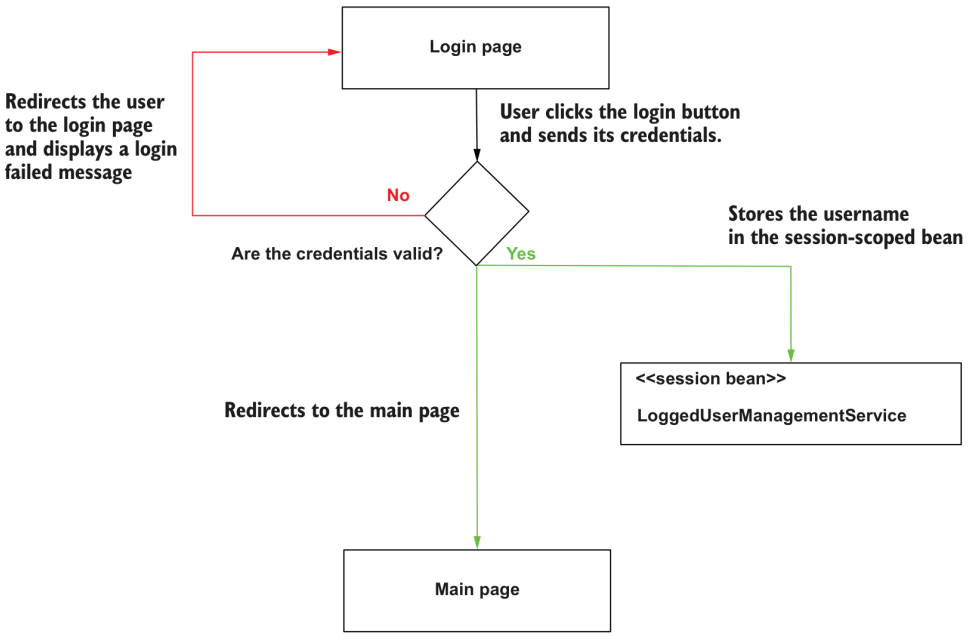
```
15
16    public void setUsername(String username) {
17      this.username = username;
18    }
19  }
```

Listing 9.6: Định nghĩa bean session-scoped lưu trữ chi tiết người dùng đã đăng nhập (Listing 9.5)

Tiếp theo, chúng ta cập nhật lớp `LoginProcessor` để tiêm dịch vụ `LoggedUserManagementService`. Khi đăng nhập thành công, chúng ta lưu tên tài khoản vào bean session này (Listing 9.7, Hình 9.11).

```
1  @Component
2  @RequestScope
3  public class LoginProcessor {
4
5    private final LoggedUserManagementService loggedUserManagementService;
6
7    private String username;
8    private String password;
9
10   public LoginProcessor(LoggedUserManagementService
       loggedUserManagementService) {
11     this.loggedUserManagementService = loggedUserManagementService;
12   }
13
14   public boolean login() {
15     String username = this.getUsername();
16     String password = this.getPassword();
17
18     boolean loginResult = false;
19     if ("natalie".equals(username) && "password".equals(password)) {
20       loginResult = true;
21       loggedUserManagementService.setUsername(username);
22     }
23
24     return loginResult;
25   }
26
27   // Getters và Setters cho username, password...
28 }
```

Listing 9.7: Cập nhật LoginProcessor để lưu thông tin vào bean session khi đăng nhập thành công (Listing 9.6)

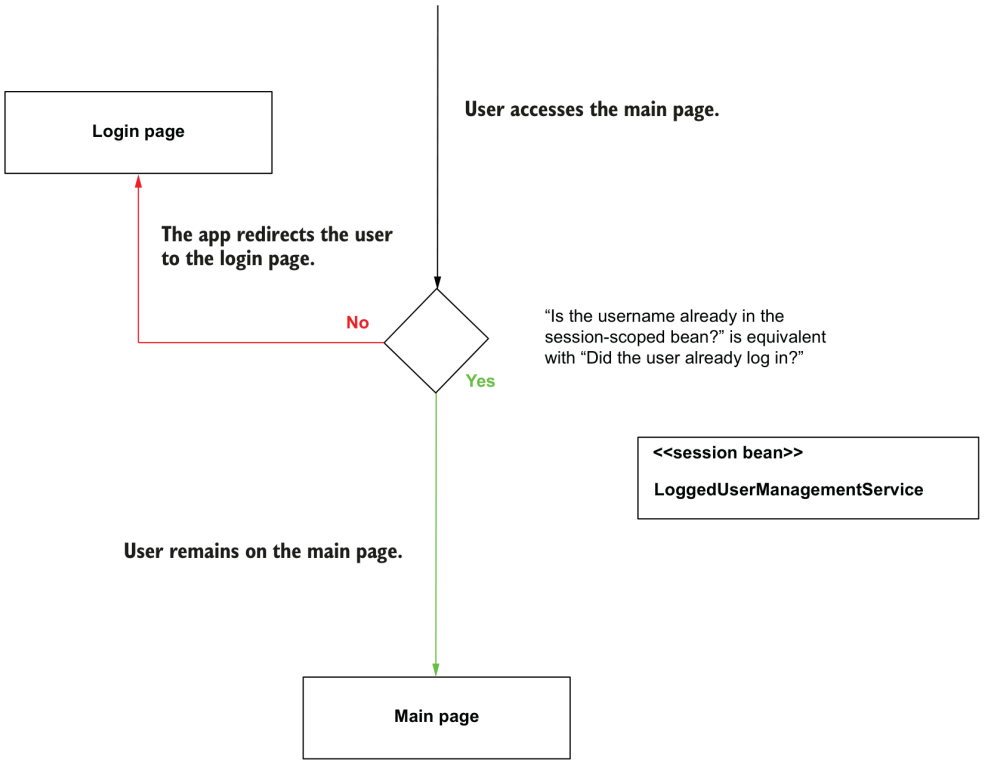


Hình 9.11: Luồng đăng nhập hoàn chỉnh: Người dùng gửi thông tin xác thực; nếu hợp lệ, tên đăng nhập được lưu vào bean session `LoggedUserManagementService` và người dùng được chuyển hướng sang trang chủ.

9.2.2 Bảo vệ trang chủ và điều hướng người dùng

Bây giờ chúng ta tạo lớp `MainController` để phục vụ trang chủ tại đường dẫn `/main` (Listing 9.8, Hình 9.12). Trong controller này, chúng ta kiểm tra xem `username` trong bean session có tồn tại hay không:

- Nếu chưa đăng nhập (`username == null`), chúng ta lập tức điều hướng người dùng quay trở lại trang đăng nhập bằng cú pháp `"redirect:/"`.
- Nếu đã đăng nhập, chúng ta gửi tên tài khoản vào `Model` và hiển thị trang `main.html`.



Hình 9.12: Cơ chế bảo vệ trang chủ: Khi client truy cập `/main`, hệ thống kiểm tra bean session. Nếu chưa đăng nhập, người dùng bị chuyển hướng về `login.html`; nếu đã đăng nhập, trang chủ được hiển thị.

```
package com.example.controllers;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import com.example.service.LoggedUserManagementService;

@Controller
public class MainController {

  private final LoggedUserManagementService loggedUserManagementService;

  public MainController(LoggedUserManagementService
    loggedUserManagementService) {
    this.loggedUserManagementService = loggedUserManagementService;
  }

  @GetMapping("/main")
  public String home(Model model) {
    String username = loggedUserManagementService.getUsername();
```

```
20
21      if (username == null) {
22        return "redirect:/";
23      }
24
25      model.addAttribute("username", username);
26      return "main.html";
27    }
28  }
```

Listing 9.8: Lớp MainController kiểm tra phiên đăng nhập (Listing 9.7)

Nội dung trang `main.html` được định nghĩa trong Listing 9.9.

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Trang ủch</title>
</head>
<body>
    <h1>Xin chào, <span th:text="${username}"></span>!</h1>
</body>
</html>
```

Listing 9.9: Nội dung tệp main.html chào đón người dùng (Listing 9.8)

9.2.3 Tính năng đăng xuất (Logout) và kỹ thuật chuyển hướng (Redirection)

Một ứng dụng hoàn chỉnh không thể thiếu tính năng **đăng xuất**. Để đăng xuất, người dùng chỉ cần hủy bỏ thông tin đã lưu trong phiên làm việc.

Chúng ta bổ sung thêm một đường liên kết đăng xuất vào tệp `main.html` (Listing 9.10):

```
<a href="/main?logout">Đăng ấxut</a>
```

Listing 9.10: Bổ sung liên kết đăng xuất vào main.html (Listing 9.9)

Sau đó, trong `MainController`, chúng ta bổ sung tham số tùy chọn `@RequestParam(required = false) String logout`. Khi tham số này xuất hiện trong yêu cầu, chúng ta gán `username` về `null` và chuyển hướng người dùng về trang đăng nhập (Listing 9.11).

```
  @GetMapping("/main")
  public String home(
      @RequestParam(required = false) String logout,
```

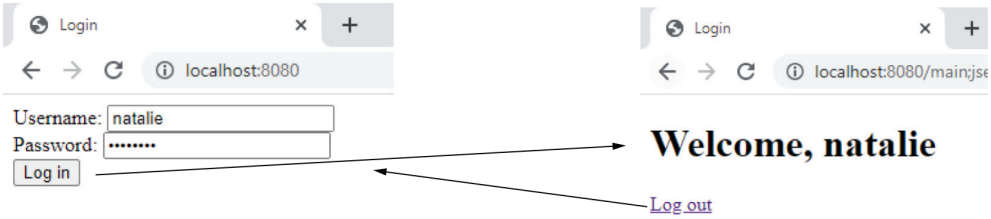
```
    Model model
) {
  if (logout != null) {
    loggedUserManagementService.setUsername(null);
  }

  String username = loggedUserManagementService.getUsername();

  if (username == null) {
    return "redirect:/";
  }

  model.addAttribute("username", username);
  return "main.html";
}
```

Listing 9.11: Xử lý đăng xuất khi có tham số logout (Listing 9.10)

Cuối cùng, trong phương thức `loginPost()` của `LoginController`, sau khi người dùng đăng nhập thành công, thay vì giữ nguyên trang `login.html`, chúng ta trả về chuỗi chuyển hướng `"redirect:/main"` (Listing 9.12, Hình 9.13).

```
@PostMapping("/")
public String loginPost(
    @RequestParam String username,
    @RequestParam String password,
    Model model
) {
  loginProcessor.setUsername(username);
  loginProcessor.setPassword(password);
  boolean loggedIn = loginProcessor.login();

  if (loggedIn) {
    return "redirect:/main";
  }

  model.addAttribute("message", "Đăng ậnhp ấtht ạbi!");
  return "login.html";
}
```

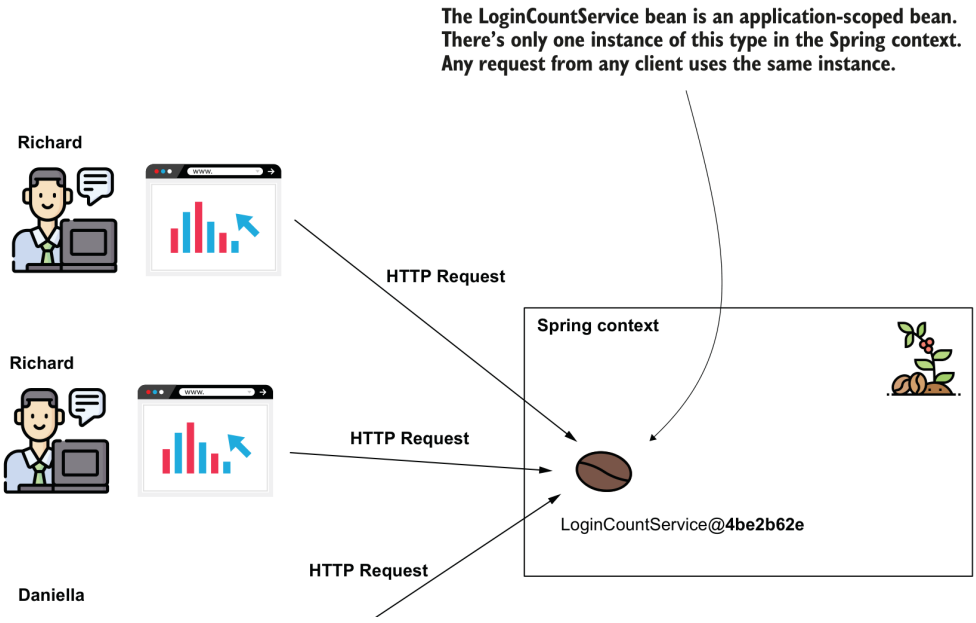
Listing 9.12: Chuyển hướng người dùng sang trang chủ sau khi đăng nhập thành công (Listing 9.11)



Hình 9.13: Luồng chuyển hướng giữa hai trang: Đăng nhập thành công sẽ kích hoạt HTTP 302 Redirect đưa người dùng tới trang chủ. Khi bấm đăng xuất, người dùng được đưa ngược trở lại trang đăng nhập.

9.3 Sử dụng phạm vi ứng dụng (Application scope) trong ứng dụng Spring

Phạm vi web thứ ba của Spring là **phạm vi ứng dụng** (*application scope*). Một bean được đánh dấu bằng chú thích `@ApplicationScope` sẽ được khởi tạo một lần duy nhất và tồn tại trong suốt vòng đời hoạt động của ứng dụng web (Hình 9.14).



Hình 9.14: Phạm vi ứng dụng: Thực thể bean `LoginCountService` được chia sẻ chung cho toàn bộ các yêu cầu HTTP từ mọi người dùng trong hệ thống.

So sánh Application Scope với Singleton Scope

Về mặt hình thức, cả `@ApplicationScope` và phạm vi mặc định `singleton` đều chỉ tạo đúng 1 thực thể bean trong Spring context. Tuy nhiên, điểm khác biệt căn bản là:

- **Singleton:** Thuộc về vòng đời của Spring `ApplicationContext`. Một ứng dụng có thể có nhiều ngữ cảnh Spring độc lập, mỗi ngữ cảnh sở hữu một bản thể singleton riêng.
- **Application Scope:** Được gắn chặt trực tiếp vào `ServletContext` của thùng chứa máy chủ web (như Tomcat). Mọi thành phần trong cùng web servlet context đều chia sẻ chung một thực thể này.

Cẩn trọng khi sử dụng Application Scope trong ứng dụng thực tế

Bởi vì một bean application-scoped được chia sẻ chung cho hàng ngàn yêu cầu từ mọi người dùng đồng thời, bất kỳ trạng thái biến đổi (*mutable state*) nào lưu trong bean này cũng tiềm ẩn nguy cơ xung đột luồng dữ liệu (*race condition*). Nếu cần thay đổi dữ liệu trong bean này, bạn phải sử dụng các cơ chế đồng bộ hóa an toàn luồng (như `AtomicInteger`, `synchronized`, hoặc lưu vào cơ sở dữ liệu).

9.3.1 Đếm tổng số lượt đăng nhập thành công với Application Scope

Để trải nghiệm phạm vi ứng dụng, chúng ta xây dựng dự án `sq-ch9-ex3`: bổ sung chức năng đếm tổng số lần đăng nhập thành công của toàn bộ người dùng trong hệ thống.

Chúng ta tạo lớp `LoginCountService` và đánh dấu bằng chú thích `@ApplicationScope` (Listing 9.13).

```
1  package com.example.service;
2
3  import org.springframework.stereotype.Service;
4  import org.springframework.web.context.annotation.ApplicationScope;
5
6  @Service
7  @ApplicationScope
8  public class LoginCountService {
9
10   private int count;
11
12   public void increment() {
13     count++;
14   }
15
```

```
  public int getCount() {
    return count;
  }
}
```

Listing 9.13: Lớp LoginCountService có phạm vi ứng dụng đếm số lần đăng nhập (Listing 9.12)

Tiếp theo, chúng ta tiêm `LoginCountService` vào `LoginProcessor`. Mỗi khi một người dùng đăng nhập thành công, chúng ta gọi phương thức `increment()` để tăng biến đếm toàn cục (Listing 9.14).

```
@Component
@RequestScope
public class LoginProcessor {

  private final LoggedUserManagementService loggedUserManagementService;
  private final LoginCountService loginCountService;

  public LoginProcessor(
      LoggedUserManagementService loggedUserManagementService,
      LoginCountService loginCountService) {
    this.loggedUserManagementService = loggedUserManagementService;
    this.loginCountService = loginCountService;
  }

  public boolean login() {
    String username = this.getUsername();
    String password = this.getPassword();

    boolean loginResult = false;
    if ("natalie".equals(username) && "password".equals(password)) {
      loginResult = true;
      loggedUserManagementService.setUsername(username);
      loginCountService.increment();
    }

    return loginResult;
  }
  // Code khác ữgi nguyên...
}
```

Listing 9.14: Tăng biến đếm đăng nhập trong LoginProcessor (Listing 9.13)

Trong `MainController`, chúng ta tiêm `LoginCountService` và đẩy giá trị biến đếm vào `Model` với khóa `"count"` để hiển thị lên trang chủ (Listing 9.15).

```
@Controller
public class MainController {

  private final LoggedUserManagementService loggedUserManagementService;
  private final LoginCountService loginCountService;

  public MainController(
      LoggedUserManagementService loggedUserManagementService,
      LoginCountService loginCountService) {
    this.loggedUserManagementService = loggedUserManagementService;
    this.loginCountService = loginCountService;
  }

  @GetMapping("/main")
  public String home(
      @RequestParam(required = false) String logout,
      Model model
  ) {
    if (logout != null) {
      loggedUserManagementService.setUsername(null);
    }

    String username = loggedUserManagementService.getUsername();

    if (username == null) {
      return "redirect:/";
    }

    int count = loginCountService.getCount();

    model.addAttribute("username", username);
    model.addAttribute("count", count);

    return "main.html";
  }
}
```

Listing 9.15: Gửi giá trị số lượt đăng nhập sang khung nhìn trong MainController (Listing 9.14)

Cuối cùng, chúng ta cập nhật tệp `main.html` để hiển thị số lượt đăng nhập (Listing 9.16).

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Trang ủch</title>
</head>
<body>
    <h1>Xin chào, <span th:text="${username}"></span>!</h1>
    <p>ốS ượlt đăng ậnhp thành công ủca toàn ệh ốthng: <span th:text="${
    count}"></span></p>
    <a href="/main?logout">Đăng ấxut</a>
</body>
</html>
```

Listing 9.16: Hiển thị số lượt đăng nhập trên trang main.html (Listing 9.15)

Khởi chạy ứng dụng và thử nghiệm: Khi đăng nhập tài khoản lần đầu, biến đếm hiển thị là 1. Nếu bạn đăng xuất rồi đăng nhập lại bằng một trình duyệt khác (hoặc chế độ ẩn danh giả lập một người dùng thứ hai), biến đếm sẽ tăng lên 2 (Hình 9.15)! Giá trị này được lưu giữ và tích lũy chung cho toàn bộ ứng dụng.

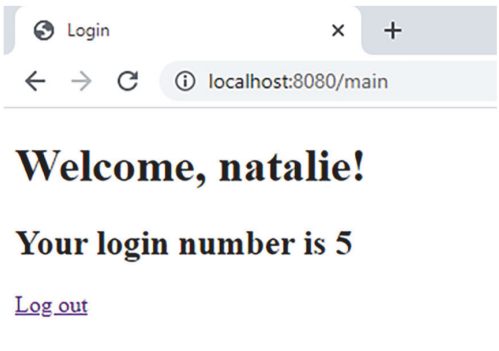


Figure 9.15 The result of the application is a web page that displays the total number of logins for all the users. This main page displays the total number of login attempts.

Hình 9.15: Kết quả kiểm thử trên trình duyệt: Trang chủ hiển thị số lượt đăng nhập toàn cục tăng dần sau mỗi lần người dùng đăng nhập thành công.

9.4 Tóm tắt chương

- Bên cạnh hai phạm vi cốt lõi là **singleton** và **prototype**, Spring cung cấp các phạm vi web chuyên dụng: **request scope**, **session scope**, và **application scope**.
- **Phạm vi yêu cầu (`@RequestScope`):** Spring tạo một thực thể bean riêng biệt cho mỗi yêu cầu HTTP và tự động giải phóng bean đó ngay khi hoàn tất gửi phản hồi. Giúp các luồng xử lý độc lập, tránh tranh chấp dữ liệu.

- **Phạm vi phiên (`@SessionScope`):** Spring liên kết thực thể bean với một phiên làm việc HTTP duy nhất của client. Tất cả các yêu cầu tiếp theo từ cùng client đó sẽ dùng chung một thực thể bean, giúp duy trì trạng thái người dùng (như tài khoản đăng nhập hay giỏ hàng).
- **Cơ chế Scoped Proxy:** Cho phép tiêm an toàn một bean có vòng đời ngắn (như request hay session) vào một bean sống vĩnh viễn (như controller singleton). Proxy hoạt động như một cầu nối trung gian chuyển cuộc gọi đến đúng thực thể bean tương ứng trong luồng hiện tại.
- **Kỹ thuật chuyển hướng (*Redirection*):** Sử dụng tiền tố `"redirect:path"` trong chuỗi trả về của phương thức controller để hướng dẫn trình duyệt thực hiện một yêu cầu mới tới địa chỉ khác sau khi xử lý xong (chuẩn thiết kế *Post/Redirect/Get*).
- **Phạm vi ứng dụng (`@ApplicationScope`):** Thực thể bean được tạo một lần duy nhất và tồn tại gắn liền với `ServletContext` của máy chủ web, được chia sẻ giữa mọi người dùng và mọi phiên. Cần hết sức thận trọng khi lưu trữ trạng thái khả biến để tránh nguy cơ tương tranh dữ liệu.

Chương 10

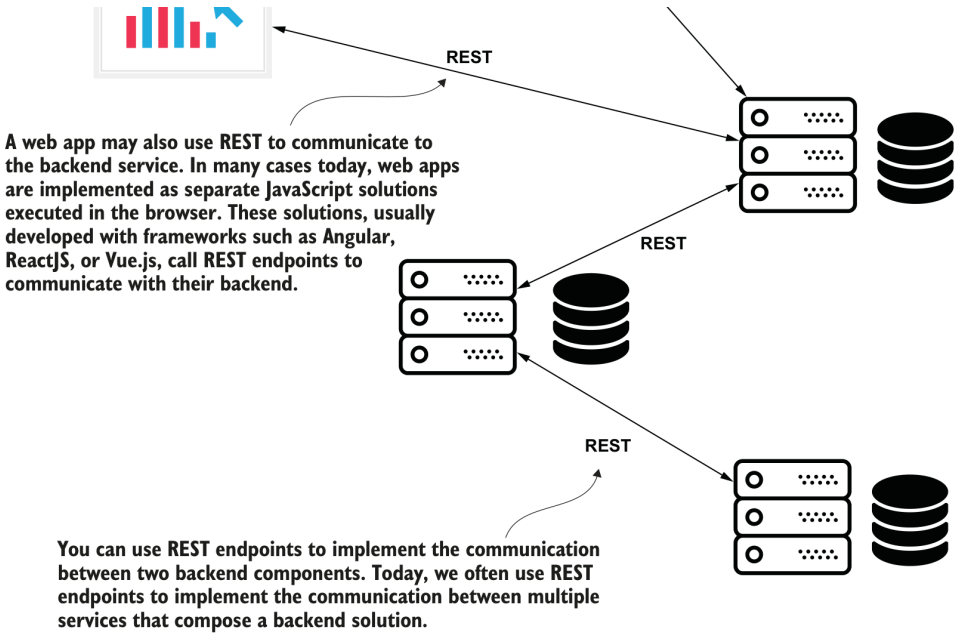
Xây dựng các dịch vụ REST

Trong các Chương 7 đến 9, chúng ta đã nhắc đến thuật ngữ **dịch vụ REST** (*REST services* — viết tắt của *Representational State Transfer*) khi thảo luận về các ứng dụng web. Trong chương này, chúng ta sẽ mở rộng và đào sâu chủ đề này, và bạn sẽ nhận thấy rằng REST không chỉ giới hạn trong phạm vi các trang web truyền thống.

Dịch vụ REST là một trong những giải pháp phổ biến nhất hiện nay để hiện thực hóa sự **giao tiếp giữa hai ứng dụng phần mềm** (*app-to-app communication*). REST cung cấp khả năng truy cập tài nguyên thông qua các phương thức chuẩn mực của giao thức HTTP. Vì hầu hết các dự án Spring thực tế trong doanh nghiệp đều sử dụng dịch vụ REST, tôi coi đây là một kỹ năng bắt buộc phải nắm vững đối với mọi lập trình viên Spring chuyên nghiệp.

Nội dung trọng tâm của chương này

- Hiểu rõ bản chất của kiến trúc REST và vai trò của các điểm cuối REST (*REST endpoints*) trong truyền thông giữa các hệ thống phần mềm.
- Sử dụng chú thích `@ResponseBody` và `@RestController` để trả về trực tiếp dữ liệu thô trong phần thân phản hồi HTTP (*HTTP response body*).
- Tự động chuyển đổi giữa đối tượng Java và định dạng JSON thông qua thư viện Jackson tích hợp sẵn trong Spring Boot.
- Tùy biến mã trạng thái HTTP, phần thân và các tiêu đề tùy biến bằng đối tượng `ResponseEntity`.
- Quản lý và xử lý ngoại lệ tập trung chuyên nghiệp bằng chú thích `@RestControllerAdvice` và `@ExceptionHandler`.
- Đón nhận các đối tượng dữ liệu phức tạp do client gửi lên trong phần thân yêu cầu bằng chú thích `@RequestBody`.



Hình 10.1: Dịch vụ REST là phương thức truyền thông giữa hai ứng dụng phần mềm: Ứng dụng di động gọi backend qua REST; ứng dụng web viết bằng React/Angular/Vue chạy trên trình duyệt gọi REST; và các dịch vụ vi mô (*microservices*) ở phía backend cũng liên lạc với nhau qua các cuộc gọi REST.

Trong Mục 10.1, chúng ta sẽ tìm hiểu khái niệm cốt lõi của dịch vụ REST và cơ chế điều phối của Spring MVC đằng sau nó. Trong Mục 10.2, chúng ta sẽ xây dựng các điểm cuối REST đầu tiên và kiểm thử với Postman. Trong Mục 10.3, chúng ta sẽ làm chủ `ResponseEntity`. Kế đó, Mục 10.4 sẽ hướng dẫn kỹ thuật tách biệt xử lý ngoại lệ bằng controller advice. Cuối cùng, Mục 10.5 sẽ giải thích cách tiếp nhận dữ liệu qua request body.

10.1 Dịch vụ REST là gì?

Trong các chương trước, khi xây dựng ứng dụng web MVC với khung nhìn HTML, mục tiêu của controller là trả về một chuỗi ký tự đại diện cho **tên của khung nhìn** (ví dụ `"home.html"`). Khối `DispatcherServlet` sau đó sẽ phối hợp với bộ giải quyết khung nhìn (*view resolver*) để lồng ghép dữ liệu từ `Model` vào tệp mẫu và sinh ra mã HTML gửi về cho trình duyệt.

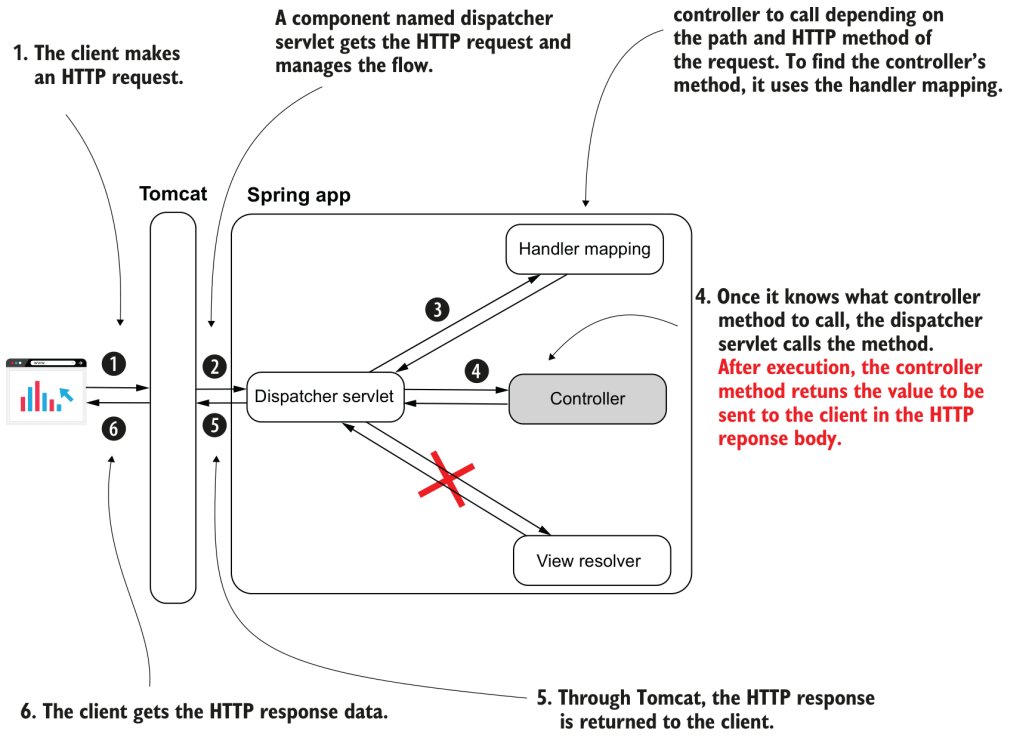
Tuy nhiên, trong kiến trúc hiện đại ngày nay:

- Các ứng dụng di động (Android, iOS) không cần máy chủ trả về mã HTML cồng kềnh; chúng chỉ cần dữ liệu thô (thường là định dạng JSON) để tự kết xuất giao diện native.
- Các ứng dụng web đơn trang (*Single Page Applications* - *SPA*) xây dựng bằng các khung làm việc JavaScript hiện đại như React, Angular hay Vue.js tự đảm nhiệm toàn bộ phần hiển thị trên trình duyệt. Phía backend Spring chỉ đóng vai trò cung cấp các API dữ liệu.

- Các dịch vụ phía backend trong kiến trúc vi dịch vụ (*microservices*) trao đổi dữ liệu có cấu trúc với nhau thông qua mạng nội bộ hoặc internet.

Một điểm cuối REST (*REST endpoint*) chính là một thành phần có thể truy cập qua giao thức HTTP nhằm mục đích truyền tải **dữ liệu thuần túy** giữa hai ứng dụng (Hình 10.1).

Điểm khác biệt kiến trúc mấu chốt giữa ứng dụng web MVC truyền thống và dịch vụ REST trong Spring được phác họa trong Hình 10.2: **Trong dịch vụ REST, controller gửi dữ liệu trực tiếp vào phần thân của phản hồi HTTP (*HTTP response body*), bỏ qua hoàn toàn công đoạn phân giải khung nhìn (*View Resolver*)!**



Hình 10.2: Cơ chế Spring MVC khi triển khai REST endpoint: Thay vì trả về tên view cho DispatcherServlet để tìm tệp HTML, controller ghi dữ liệu trực tiếp vào thân phản hồi HTTP. Nhờ đó, công đoạn kết xuất khung nhìn được bỏ qua hoàn toàn.

10.2 Triển khai các điểm cuối REST trong Spring

Làm thế nào để báo hiệu cho Spring biết rằng giá trị trả về của phương thức controller không phải là một tên view mà là dữ liệu cần đưa thẳng vào HTTP response body? Giải pháp chính là chú thích `@ResponseBody`!

10.2.1 Điểm cuối REST đầu tiên với chú thích @ResponseBody

Chúng ta tạo dự án `sq-ch10-ex1`. Trong tệp `pom.xml`, chúng ta chỉ cần khai báo gói starter duy nhất: `spring-boot-starter-web` (chúng ta không cần starter Thymeleaf nữa vì không kết xuất tệp HTML).

Trong Listing 10.1, chúng ta định nghĩa lớp `HelloController`. Bên cạnh chú thích quen thuộc `@GetMapping("/hello")`, chúng ta gắn thêm chú thích `@ResponseBody` lên phương thức.

```
package com.example.controllers;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ResponseBody;

@Controller
public class HelloController {

  @GetMapping("/hello")
  @ResponseBody
  public String hello() {
    return "Hello!";
  }
}
```

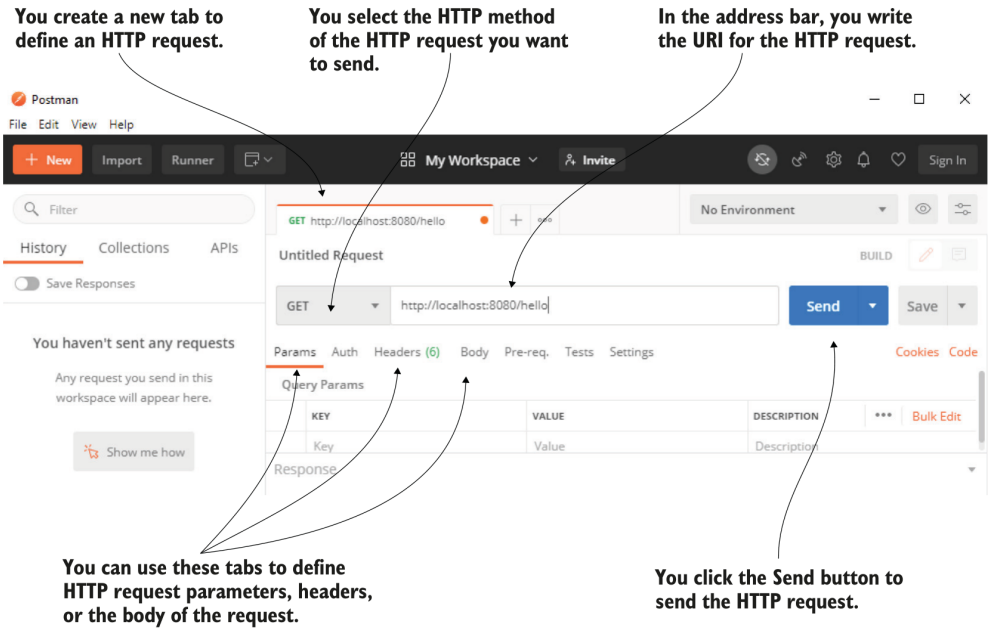
Listing 10.1: Triển khai hành động của điểm cuối REST với chú thích @ResponseBody (Listing 10.1)

Để kiểm thử điểm cuối REST, các lập trình viên chuyên nghiệp thường sử dụng các công cụ chuyên dụng như **Postman** hoặc `cURL` thay vì chỉ dựa vào thanh địa chỉ của trình duyệt. Postman cho phép bạn tùy chỉnh mọi phương thức HTTP, thêm tiêu đề, và xem chi tiết cấu trúc phản hồi.

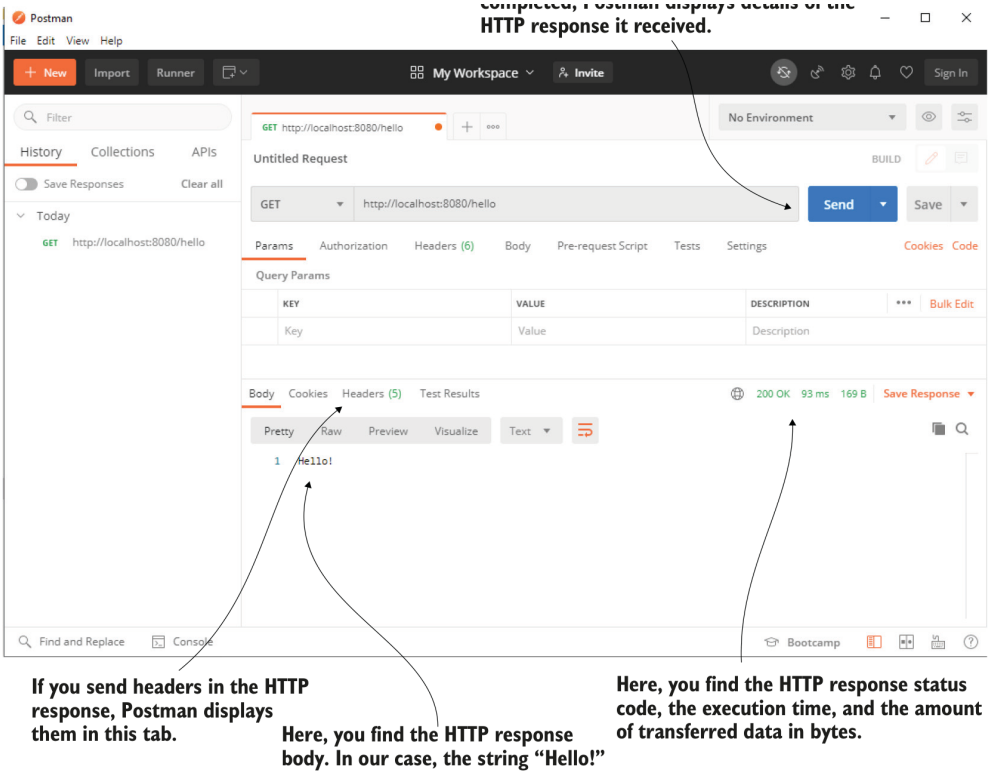
Hình 10.3 và 10.4 minh họa giao diện gửi yêu cầu `GET http://localhost:8080/hello` trong Postman và kết quả nhận về chuỗi ký tự `"Hello!"` nguyên vẹn trong phần thân phản hồi.

figures a Tomcat servlet container to be accessible on port 8080.

Let's discuss Postman first. You need to install the tool on your system as presented on their official website: https://www.postman.com/. Once you have Postman installed, when you open it, you'll find it has an interface like the one presented in figure 10.3.



Hình 10.3: Giao diện Postman để cấu hình và phát yêu cầu HTTP tới điểm cuối REST.



Hình 10.4: Kết quả phản hồi trong Postman: Sau khi yêu cầu hoàn tất, phần thân phản hồi hiển thị đúng chuỗi ký tự `"Hello!"` cùng mã trạng thái 200 OK.

10.2.2 Tối ưu hóa mã nguồn với chú thích @RestController

Nếu một lớp controller có 10 hành động REST khác nhau, việc lặp đi lặp lại chú thích `@ResponseBody` trên từng phương thức sẽ khiến mã nguồn bị trùng lặp rườm rà (Listing 10.2).

```
@Controller
public class HelloController {

  @GetMapping("/hello")
  @ResponseBody
  public String hello() {
    return "Hello!";
  }

  @GetMapping("/ciao")
  @ResponseBody
  public String ciao() {
    return "Ciao!";
  }
}
```

Listing 10.2: Chú thích @ResponseBody bị lặp lại ở mọi phương thức (Listing 10.2)

Để giải quyết vấn đề này, Spring cung cấp chú thích kết hợp **`@RestController`** (Listing 10.3). Khi bạn gắn `@RestController` lên cấp độ lớp, Spring sẽ tự động áp dụng hành vi của `@ResponseBody` cho **tất cả các phương thức** bên trong controller đó mà không cần khai báo riêng lẻ.

```
@RestController
public class HelloController {

  @GetMapping("/hello")
  public String hello() {
    return "Hello!";
  }

  @GetMapping("/ciao")
  public String ciao() {
    return "Ciao!";
  }
}
```

Listing 10.3: Sử dụng @RestController để loại bỏ trùng lặp mã (Listing 10.3)

10.2.3 Trả về đối tượng Java và cơ chế chuyển đổi JSON tự động

Trong thực tế, hiếm khi chúng ta chỉ trả về một chuỗi ký tự đơn giản. Thay vào đó, chúng ta thường trả về các đối tượng dữ liệu phức tạp biểu diễn thực thể nghiệp vụ (như thông tin khách hàng, đơn hàng, quốc gia).

Trong dự án `sq-ch10-ex2`, chúng ta tạo lớp mô hình `Country` trong gói `model` (Listing 10.4).

```
package com.example.model;

public class Country {

  private String name;
  private int population;

  public static Country of(String name, int population) {
    Country country = new Country();
    country.setName(name);
    country.setPopulation(population);
    return country;
  }

  public String getName() {
    return name;
  }

  public void setName(String name) {
    this.name = name;
  }

  public int getPopulation() {
    return population;
  }

  public void setPopulation(int population) {
    this.population = population;
  }
}
```

Listing 10.4: Lớp mô hình Country đại diện cho dữ liệu trả về (Listing 10.4)

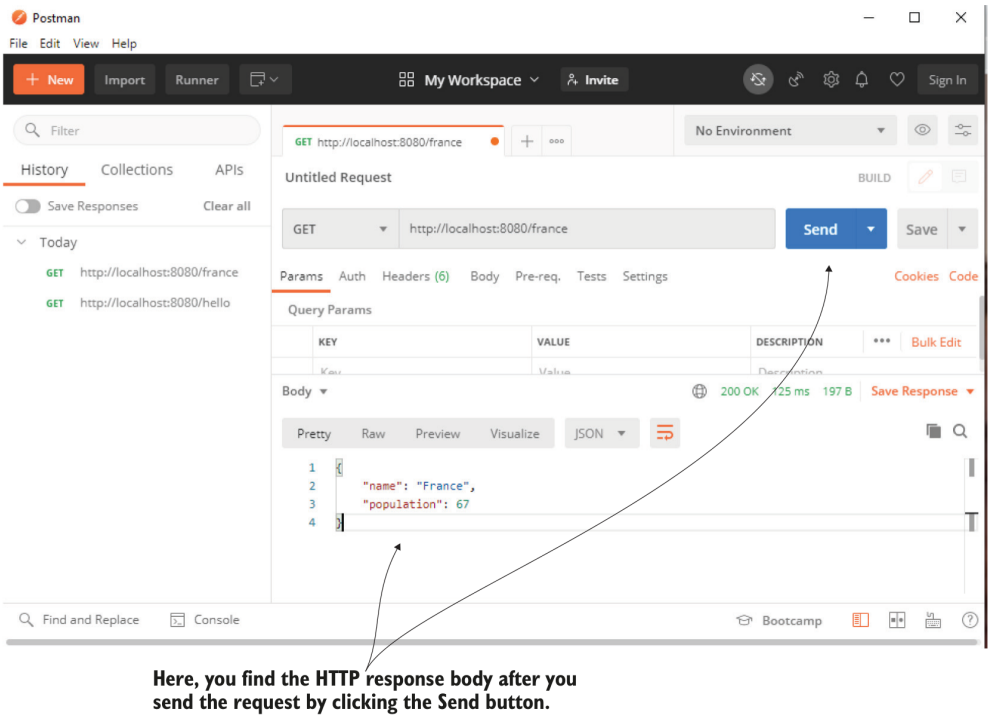
Trong lớp `CountryController`, phương thức hành động trả về trực tiếp một thực thể `Country` (Listing 10.5).

```java
@RestController
public class CountryController {

  @GetMapping("/france")
  public Country france() {
    Country c = Country.of("France", 67);
    return c;
  }
}
```

Listing 10.5: Phương thức controller trả về trực tiếp một đối tượng Country (Listing 10.5)

Điều kỳ diệu xảy ra khi bạn gọi điểm cuối này trong Postman (Hình 10.5): Thay vì trả về một đối tượng Java nhị phân hay gọi hàm `toString()`, Spring tự động chuyển đổi đối tượng `Country` thành một chuỗi **JSON** hoàn chỉnh:

```
{
    "name": "France",
    "population": 67
}
```



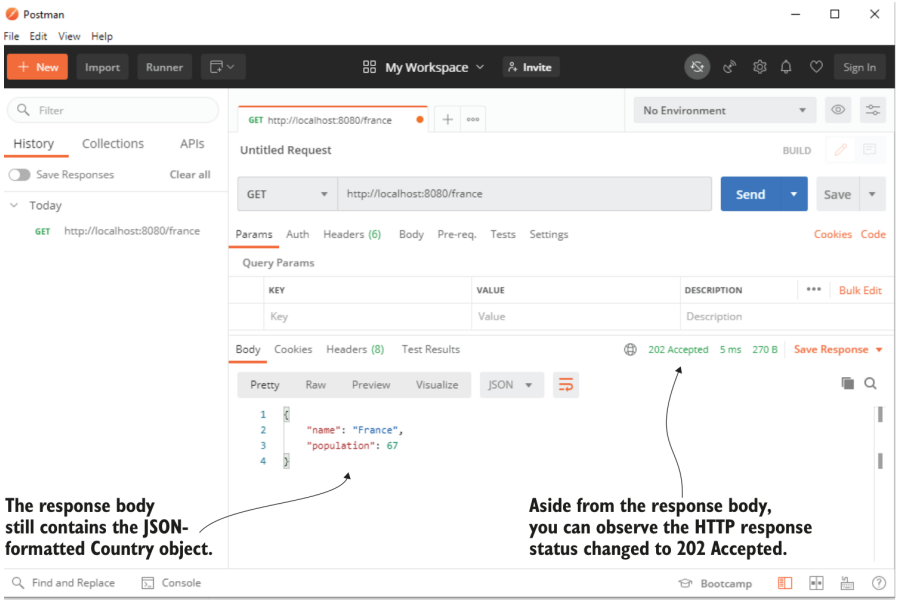
Hình 10.5: Phản hồi JSON trong Postman: Khi phương thức trả về một đối tượng Java, Spring tự động tuần tự hóa thuộc tính của đối tượng thành định dạng JSON chuẩn trong phần thân HTTP.

Làm sao Spring có thể làm được điều này? Khi bạn thêm gói `spring-boot-starter-web`, Spring Boot đã tự động kéo theo thư viện **Jackson** và cấu hình các bộ chuyển đổi thông điệp HTTP (*HttpMessageConverter*). Mặc định, Jackson sẽ đọc các phương thức getter của đối tượng Java và chuyển đổi các thuộc tính thành các cặp khóa-giá trị tương ứng trong định dạng JSON.

Hoàn toàn tương tự, nếu phương thức trả về một danh sách các đối tượng (`List<Country>` như trong dự án `sq-ch10-ex3`, Listing 10.6), Jackson sẽ tự động chuyển đổi danh sách đó thành một mảng JSON (*JSON array*) (Hình 10.6):

```
@RestController
public class CountryController {

  @GetMapping("/all")
  public List<Country> countries() {
    Country c1 = Country.of("France", 67);
    Country c2 = Country.of("Spain", 47);

    return List.of(c1, c2);
  }
}
```

Listing 10.6: Trả về một tập hợp các đối tượng trong phần thân phản hồi (Listing 10.6)



Hình 10.6: Kết quả trả về danh sách trong Postman: Tập hợp `List<Country>` được tự động định dạng thành mảng JSON gồm nhiều phần tử trong dấu ngoặc vuông `[...]`.

10.3 Tùy biến phản hồi HTTP với ResponseEntity

Trong các ví dụ trên, Spring mặc định gán mã trạng thái HTTP là **200 OK** và các tiêu đề tiêu chuẩn. Nhưng trong các bài toán thực tế, bạn thường cần:

- Trả về mã trạng thái chính xác phản ánh kết quả xử lý (ví dụ: **201 Created** khi tạo mới tài nguyên, **204 No Content** khi xử lý thành công không có dữ liệu trả về, hoặc **400 Bad Request** khi tham số sai).
- Đính kèm các tiêu đề HTTP tùy biến (*custom headers*) vào phản hồi.

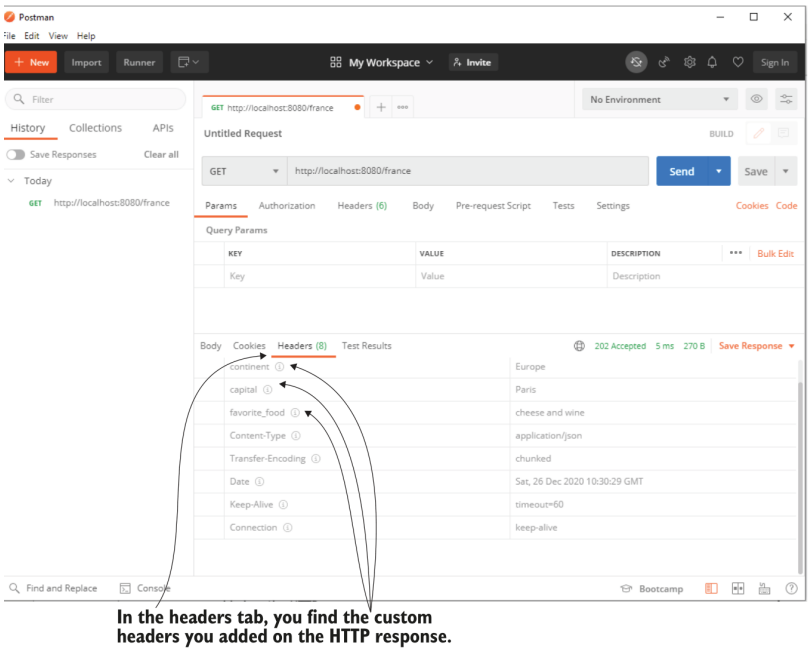
Để toàn quyền kiểm soát phản hồi HTTP, Spring cung cấp lớp bao bọc **`ResponseEntity<T>`**.

Trong dự án `sq-ch10-ex4`, chúng ta sử dụng `ResponseEntity` để cấu hình mã trạng thái 202 Accepted, thêm tiêu đề tùy biến `code: test`, và đặt đối tượng `Country` vào phần thân (Listing 10.7).

```
@RestController
public class CountryController {

  @GetMapping("/korea")
  public ResponseEntity<Country> korea() {
    Country c = Country.of("Korea", 51);
    return ResponseEntity
        .status(HttpStatus.ACCEPTED)
        .header("continent", "Asia")
        .header("capital", "Seoul")
        .header("favorite_food", "kimchi")
        .body(c);
  }
}
```

Listing 10.7: Tùy biến tiêu đề và mã trạng thái HTTP bằng ResponseEntity (Listing 10.7)

Hình 10.7 cho thấy trong Postman, bạn có thể dễ dàng kiểm tra các tiêu đề tùy biến vừa được thêm vào trong tab *Headers* của kết quả phản hồi.



Hình 10.7: Kiểm tra các tiêu đề tùy biến trong Postman: Các tiêu đề bổ sung như `continent`, `capital` xuất hiện đầy đủ trong phản hồi HTTP.

10.4 Quản lý và xử lý ngoại lệ trong dịch vụ REST

Một hệ thống dịch vụ REST chuẩn mực phải có khả năng xử lý ngoại lệ duyên dáng: khi có lỗi xảy ra, hệ thống không nên để lộ các vết ngăn xếp (*stack traces*) nội bộ cho phía client, mà cần phản hồi mã trạng thái lỗi thích hợp (như 400 hoặc 404) kèm theo một cấu trúc thông điệp lỗi rõ ràng.

10.4.1 Xử lý ngoại lệ cục bộ bên trong Controller

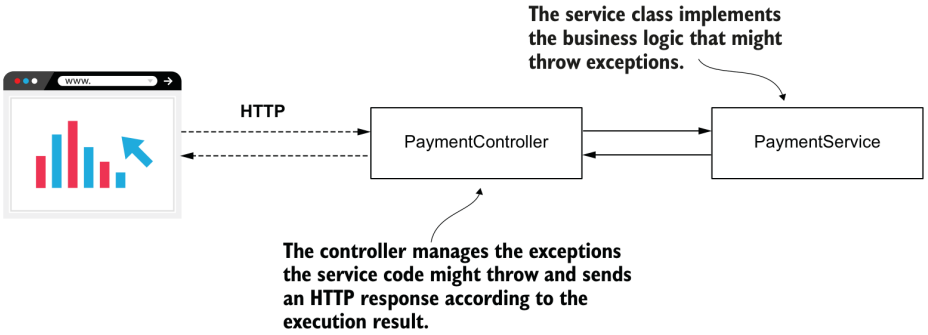
Hãy xem xét kịch bản thanh toán trong dự án `sq-ch10-ex5` (Hình 10.8):

- Lớp `PaymentService` thực hiện nghiệp vụ thanh toán. Nếu số dư tài khoản không đủ, nó sẽ ném ra ngoại lệ kiểm tra `NotEnoughMoneyException`.
- Lớp `ErrorDetails` chứa thông điệp lỗi gửi về cho client.

and only defines the error `message` as an attribute. The next code snippet presents the `ErrorDetails` model class:

```
public class ErrorDetails {

  private String message;

  // Omitted getters and setters
}
```

How could the controller decide what object to send back depending on how the flow executed? When there's no exception (the app successfully completes the payment), we want to return an HTTP response with the status "Accepted" of type `PaymentDetails`. Suppose the app encountered an exception during the execution flow. In that case, the controller's action returns an HTTP response with the status "400 Bad Request" and an `ErrorDetails` instance containing a message that describes the issue. Figure 10.8 visually presents the relationship between the components and their responsibilities.



Hình 10.8: Kiến trúc dịch vụ thanh toán: Lớp `PaymentService` thực hiện logic nghiệp vụ và ném ngoại lệ `NotEnoughMoneyException` khi tài khoản không đủ tiền thanh toán.

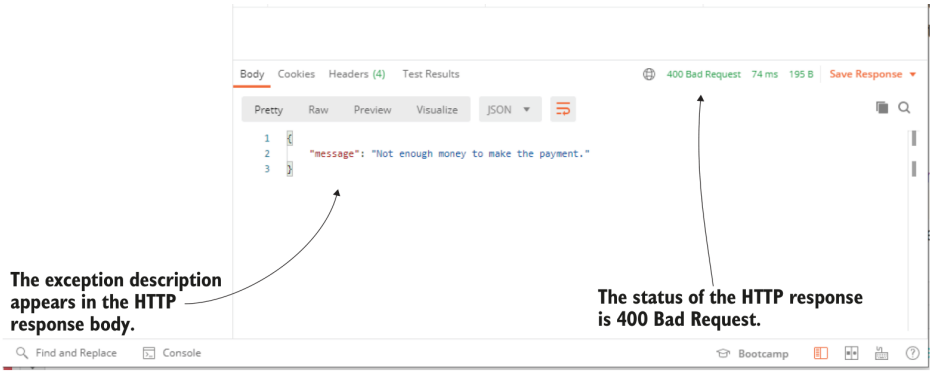
Listing 10.8 cho thấy cách xử lý ngoại lệ truyền thống bằng khối `try-catch` ngay bên trong phương thức của controller.

```
@RestController
public class PaymentController {

  private final PaymentService paymentService;

  public PaymentController(PaymentService paymentService) {
    this.paymentService = paymentService;
  }

  @PostMapping("/payment")
  public ResponseEntity<?> makePayment() {
    try {
      PaymentDetails paymentDetails = paymentService.processPayment();
      return ResponseEntity
          .status(HttpStatus.ACCEPTED)
          .body(paymentDetails);
```

```
    } catch (NotEnoughMoneyException e) {
      ErrorDetails errorDetails = new ErrorDetails();
      errorDetails.setMessage("Không đủ ềtin để ựthc ệhin giao ịdch!");
      return ResponseEntity
          .badRequest()
          .body(errorDetails);
    }
  }
}
```

Listing 10.8: Quản lý ngoại lệ trực tiếp trong hành động controller bằng try-catch (Listing 10.8)

Khi gọi `POST /payment` trong Postman, nếu xảy ra ngoại lệ, client nhận được mã phản hồi **400 Bad Request** kèm theo nội dung lỗi từ đối tượng `ErrorDetails` (Hình 10.9).



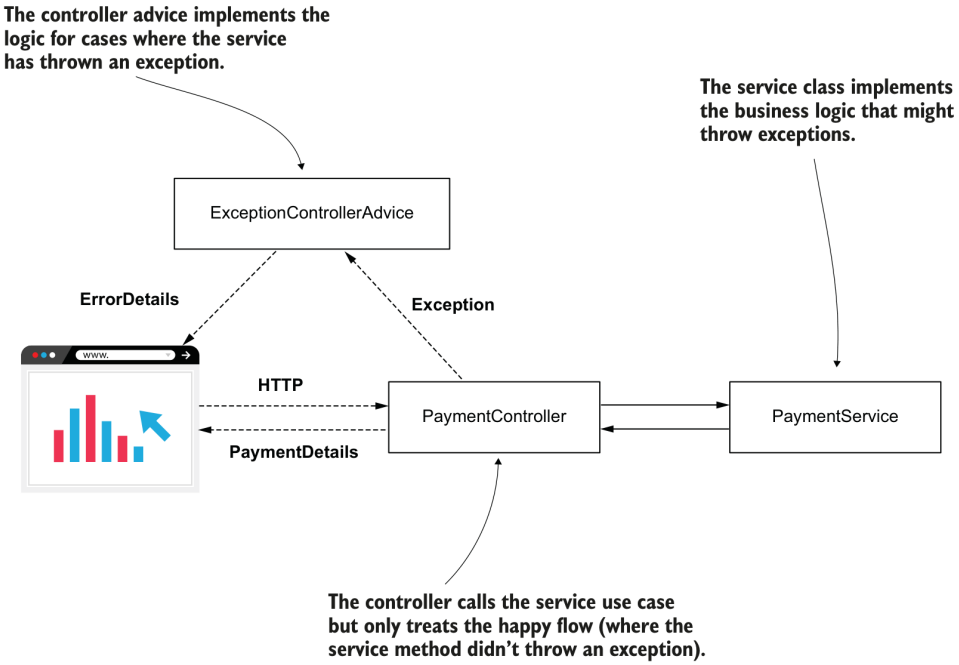
Hình 10.9: Kết quả phản hồi khi xảy ra ngoại lệ: Client nhận được mã HTTP 400 Bad Request cùng thông điệp lỗi dạng JSON rõ ràng.

10.4.2 Tách biệt xử lý ngoại lệ tập trung với @RestControllerAdvice

Mặc dù giải pháp trong Listing 10.8 hoạt động tốt, nó nhanh chóng bộc lộ điểm yếu khi ứng dụng phát triển lớn:

- Các phương thức controller bị bôi bẩn bởi hàng loạt khối `try-catch`, làm lu mờ luồng nghiệp vụ chính (”happy path”).
- Nếu nhiều controller cùng gặp một loại ngoại lệ, mã xử lý lỗi sẽ bị sao chép ở nhiều nơi.

Để giải quyết triệt để vấn đề này, Spring cung cấp cơ chế chặn lỗi tập trung bằng chú thích **`@RestControllerAdvice`** và **`@ExceptionHandler`** (dự án `sq-ch10-ex6`, Hình 10.10).



Hình 10.10: Mô hình phân tách trách nhiệm: Thay vì tự bắt lỗi bằng try-catch, controller để ngoại lệ nổi lên tự do. Lớp `ExceptionControllerAdvice` sẽ chặn bắt ngoại lệ từ xa và chuyển hóa thành phản hồi lỗi thích hợp.

Giờ đây, phương thức trong `PaymentController` trở nên cực kỳ trong sáng và ngắn gọn (Listing 10.9):

```
@RestController
public class PaymentController {

  private final PaymentService paymentService;

  public PaymentController(PaymentService paymentService) {
    this.paymentService = paymentService;
  }

  @PostMapping("/payment")
  public ResponseEntity<PaymentDetails> makePayment() {
    PaymentDetails paymentDetails = paymentService.processPayment();
    return ResponseEntity
        .status(HttpStatus.ACCEPTED)
        .body(paymentDetails);
  }
}
```

Listing 10.9: Hành động controller chỉ tập trung vào luồng thành công (Listing 10.9)

Chúng ta tạo một lớp cố vấn lỗi riêng biệt mang tên `ExceptionControllerAdvice` và gắn chú thích `@RestControllerAdvice` (Listing 10.10). Khi bất kỳ controller nào trong ứng dụng ném ngoại lệ `NotEnoughMoneyException`, phương thức có chú thích `@ExceptionHandler(NotEnoug` sẽ được tự động kích hoạt để sinh ra phản hồi lỗi!

```
package com.example.advice;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import com.example.exceptions.NotEnoughMoneyException;
import com.example.model.ErrorDetails;

@RestControllerAdvice
public class ExceptionControllerAdvice {

  @ExceptionHandler(NotEnoughMoneyException.class)
  public ResponseEntity<ErrorDetails> exceptionNotEnoughMoneyHandler() {
    ErrorDetails errorDetails = new ErrorDetails();
    errorDetails.setMessage("Không đủ ềtin để ựthc ệhin giao ịdch!");
    return ResponseEntity
        .badRequest()
        .body(errorDetails);
  }
}
```

Listing 10.10: Tách biệt logic xử lý ngoại lệ bằng REST controller advice (Listing 10.10)

10.5 Nhận dữ liệu từ Client qua phần thân yêu cầu (Request Body)

Ở Chương 8, bạn đã học cách đón nhận dữ liệu từ client qua tham số truy vấn (`@RequestParam`) và biến đường dẫn (`@PathVariable`). Tuy nhiên, khi client cần truyền một cấu trúc dữ liệu đồ sộ hoặc đối tượng phức tạp, việc nhồi nhét lên URL là không khả thi.

Trong các dịch vụ REST, giải pháp chuẩn mực để gửi dữ liệu đối tượng từ client lên server là đặt chuỗi JSON vào **phần thân yêu cầu** (*HTTP request body*) đi kèm với phương thức `POST` hoặc `PUT`.

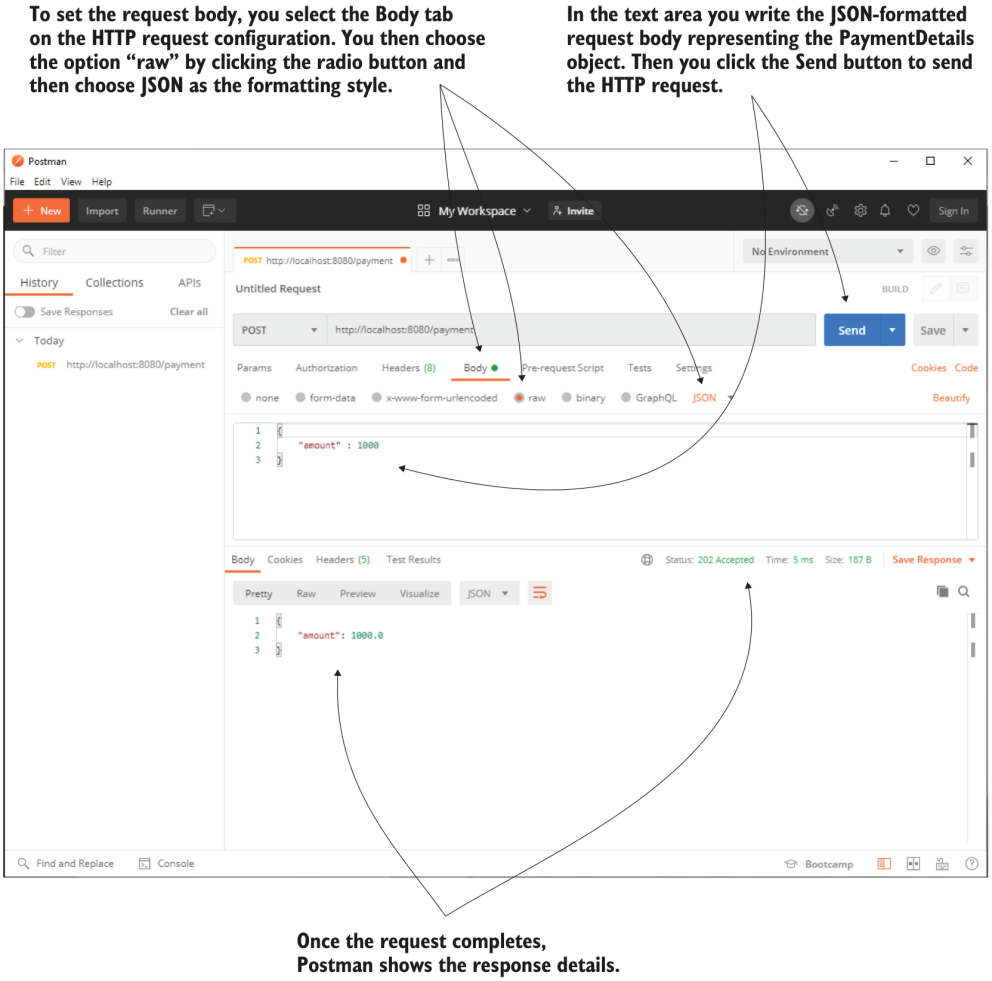
Để tiếp nhận đối tượng này, chúng ta gắn chú thích **`@RequestBody`** lên tham số của

phương thức trong controller (dự án `sq-ch10-ex7`, Listing 10.11). Spring sẽ tự động kích hoạt Jackson để giải tuần tự hóa (*deserialize*) chuỗi JSON trong body thành một thực thể Java tương ứng!

```
@RestController
public class PaymentController {

  private static Logger logger =
      Logger.getLogger(PaymentController.class.getName());

  @PostMapping("/payment")
  public ResponseEntity<PaymentDetails> makePayment(
      @RequestBody PaymentDetails paymentDetails) {

    logger.info("Đã ếtip ậnhn giao ịdch thanh toán: " +
        paymentDetails.getAmount());

    return ResponseEntity
        .status(HttpStatus.ACCEPTED)
        .body(paymentDetails);
  }
}
```

Listing 10.11: Đón nhận dữ liệu từ client trong phần thân yêu cầu với @RequestBody (Listing 10.11)

Hình 10.11 minh họa cách cấu hình yêu cầu trong Postman: chọn tab *Body*, định dạng *raw* với kiểu *JSON*, và nhập nội dung JSON gửi lên. Khi nhấn Send, controller sẽ nhận được đối tượng đầy đủ dữ liệu và trả về phản hồi tương ứng.



Hình 10.11: Sử dụng Postman để truyền dữ liệu trong phần thân yêu cầu: Chọn Body → raw → JSON và cung cấp đối tượng JSON cần gửi tới điểm cuối.

10.6 Tóm tắt chương

- Dịch vụ REST (*REST services*) là chuẩn mực truyền thông hàng đầu hiện nay giữa các ứng dụng phần mềm (giữa web app SPA, mobile app và backend, hoặc giữa các microservices).
- Điểm khác biệt căn bản giữa điểm cuối REST và ứng dụng web MVC truyền thống là điểm cuối REST trả về trực tiếp dữ liệu thô trong **thân phản hồi HTTP** (*HTTP response body*) mà không qua tầng phân giải khung nhìn HTML (*view resolver*).
- Chú thích `@ResponseBody` chỉ thị cho Spring ghi trực tiếp giá trị trả về của phương thức vào HTTP response body.
- Chú thích `@RestController` là sự kết hợp tiện lợi của `@Controller` và `@ResponseBody`, tự động áp dụng hành vi REST cho toàn bộ các phương thức trong lớp.
- Khi phương thức trả về một đối tượng hoặc tập hợp Java, Spring Boot (thông qua thư

viện Jackson tích hợp sẵn) sẽ tự động chuyển đổi đối tượng đó thành định dạng **JSON**.

- Đối tượng `ResponseEntity<T>` cho phép lập trình viên toàn quyền tùy biến mã trạng thái HTTP, các tiêu đề tùy biến và phần thân phản hồi một cách linh hoạt.
- Sử dụng `@RestControllerAdvice` kết hợp `@ExceptionHandler` là mô hình chuẩn mực để xử lý ngoại lệ tập trung, giúp tách biệt hoàn toàn mã bắt lỗi ra khỏi luồng nghiệp vụ chính của controller.
- Chú thích `@RequestBody` cho phép controller đón nhận các cấu trúc dữ liệu JSON phức tạp do client gửi lên trong phần thân của yêu cầu HTTP.

Chương 11

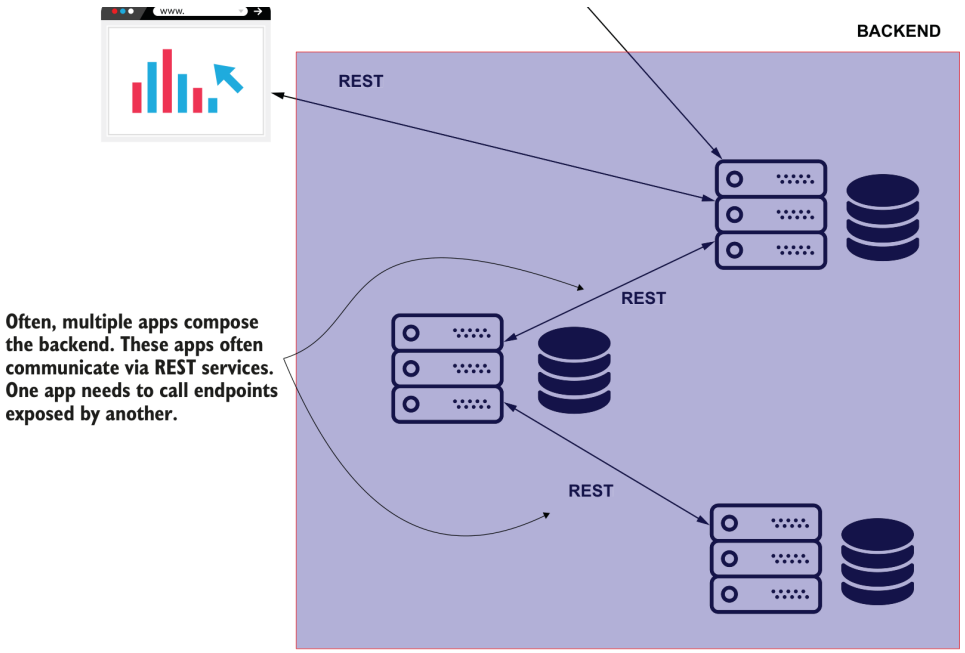
Tiêu thụ các điểm cuối REST

Trong Chương 10, bạn đã học cách biến ứng dụng Spring thành một nhà cung cấp dịch vụ REST (*REST service provider*) phục vụ cho các ứng dụng khách khác. Tuy nhiên, trong các hệ thống phần mềm doanh nghiệp hiện đại, ứng dụng của bạn rất hiếm khi đứng độc lập một mình. Thay vào đó, nó thường xuyên đóng vai trò là một **khách tiêu thụ** (*client*), liên tục gọi tới các dịch vụ web bên ngoài (Hình 11.1):

- Gọi sang các cổng thanh toán của bên thứ ba (như Stripe, PayPal) để xử lý giao dịch tài chính.
- Gọi sang dịch vụ tin nhắn hoặc email (như Twilio, SendGrid) để gửi mã OTP.
- Trong kiến trúc vi dịch vụ (*microservices*), các dịch vụ nghiệp vụ nhỏ liên tục gọi chéo các điểm cuối REST của nhau để hoàn thành một quy trình nghiệp vụ tổng thể.

Nội dung trọng tâm của chương này

- Hiểu rõ ba cách tiếp cận chính để gọi một điểm cuối REST trong hệ sinh thái Spring: **Spring Cloud OpenFeign**, **RestTemplate**, và **Spring WebClient**.
- Gọi điểm cuối REST theo phong cách khai báo (*declarative*) cực kỳ tinh gọn với **OpenFeign**.
- Sử dụng công cụ truyền thống **RestTemplate** cho các lời gọi đồng bộ chặn (*synchronous blocking calls*).
- Nắm vững bản chất của mô hình lập trình phản ứng (*reactive programming*) và thực thi các cuộc gọi phi ngăn chặn (*non-blocking calls*) với **WebClient** trong Spring WebFlux.



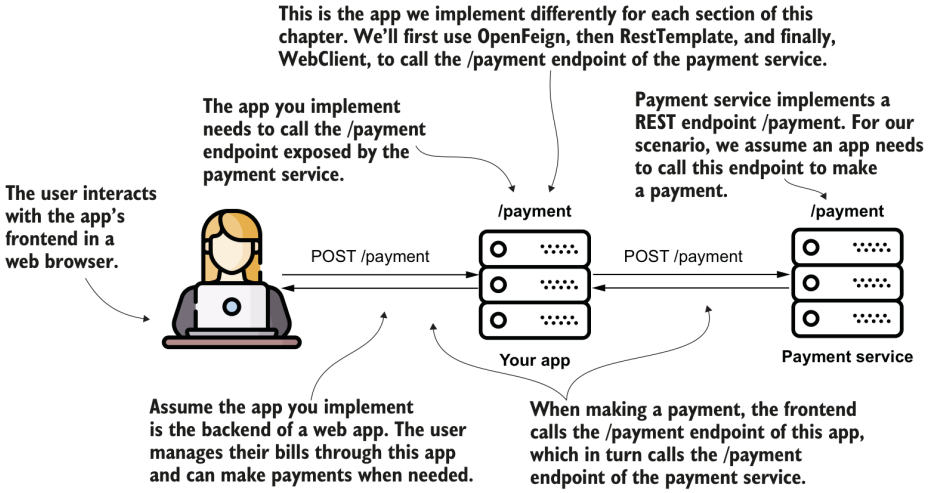
Hình 11.1: Ứng dụng backend đóng vai trò là client: Một ứng dụng backend thường xuyên cần tiêu thụ các điểm cuối REST được cung cấp bởi các dịch vụ khác (như cổng thanh toán, dịch vụ đối tác, hoặc các microservices nội bộ).

Trong Mục 11.1, chúng ta sẽ bắt đầu với công cụ hiện đại và tiện lợi nhất: **Spring Cloud OpenFeign**. Kế tiếp, trong Mục 11.2, chúng ta sẽ nghiên cứu công cụ kinh điển **RestTemplate**. Cuối cùng, trong Mục 11.3, chúng ta sẽ khám phá giải pháp phản ứng phi ngăn chặn hiện đại với **WebClient**.

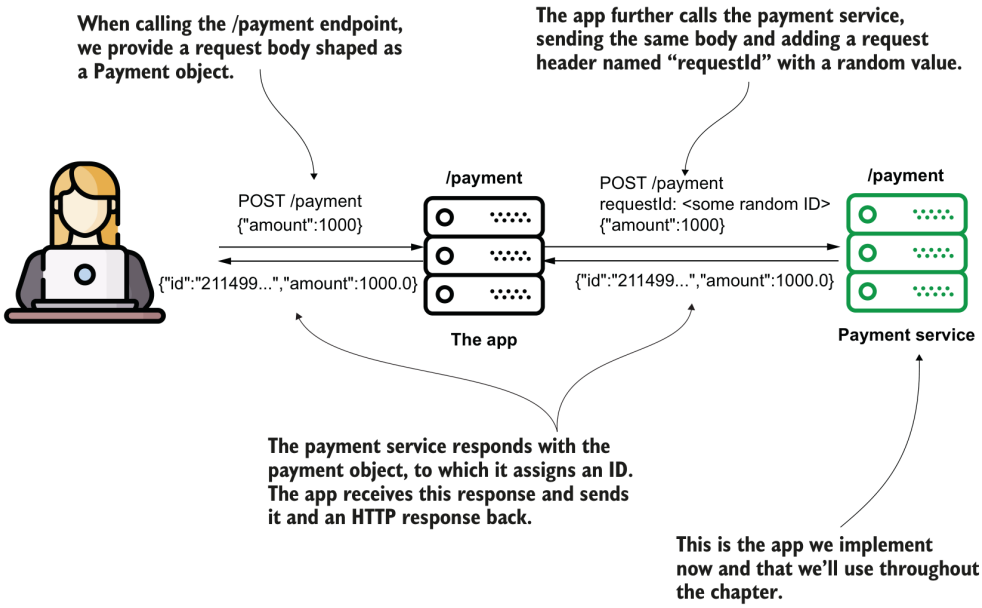
11.1 Gọi các điểm cuối REST bằng Spring Cloud OpenFeign

Để có một mục tiêu thực tế để gọi tới, chúng ta sẽ bắt đầu bằng việc thiết lập một dịch vụ thanh toán mô phỏng (*mock payment service*) cung cấp điểm cuối `POST /payment` (Hình 11.2 và 11.3).

details.



Hình 11.2: Kịch bản thực hành: Chúng ta thiết lập một dịch vụ thanh toán cung cấp điểm cuối REST, và sau đó xây dựng ứng dụng độc lập thứ hai để tiêu thụ điểm cuối này.



Hình 11.3: Chi tiết giao thức của điểm cuối `POST /payment`: Yêu cầu cần một tiêu đề `requestId` và phần thân chứa số tiền thanh toán; dịch vụ phản hồi đối tượng thanh toán đã được cấp phát mã định danh duy nhất (`id`).

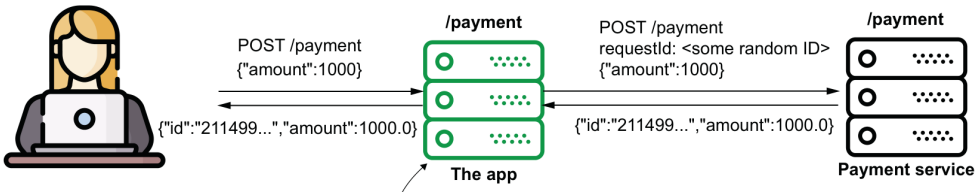
Listing 11.1 trình bày mã nguồn của điểm cuối thanh toán trong dịch vụ mẫu (dự án `sq-ch11-payments`). Điểm cuối yêu cầu một tiêu đề `requestId` (thường dùng để chống trùng lặp giao dịch — *idempotency*) và nhận dữ liệu thanh toán từ `@RequestBody`.

```
@RestController
public class PaymentController {

```

```
  private static Logger logger =
      Logger.getLogger(PaymentController.class.getName());

  @PostMapping("/payment")
  public ResponseEntity<Payment> createPayment(
      @RequestHeader String requestId,
      @RequestBody Payment payment) {

    logger.info("Đã ậnhn yêu ầcu ID: " + requestId +
                "; ốS ềtin: " + payment.getAmount());

    payment.setId(UUID.randomUUID().toString());

    return ResponseEntity
        .status(HttpStatus.OK)
        .header("requestId", requestId)
        .body(payment);
  }
}
```

Listing 11.1: Hiện thực hóa điểm cuối /payment trong dịch vụ thanh toán mẫu (Listing 11.1)

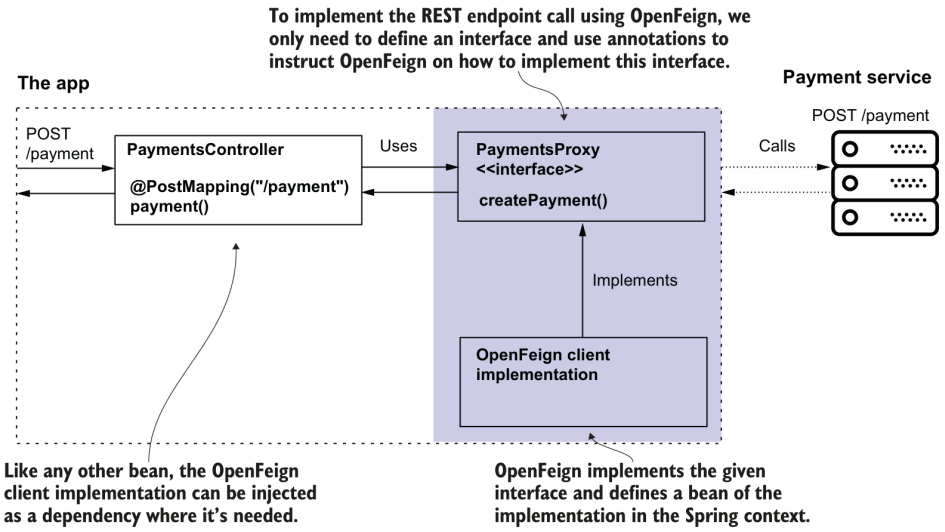
Bây giờ, chúng ta sẽ xây dựng ứng dụng thứ hai để gọi tới dịch vụ này (Hình 11.4).



Hình 11.4: Ứng dụng tiêu thụ điểm cuối: Ứng dụng này sẽ gửi yêu cầu HTTP sang dịch vụ thanh toán và đón nhận kết quả trả về.

11.1.1 Sức mạnh của OpenFeign: Lập trình client dạng khai báo

Thay vì phải tự mở kết nối HTTP, cấu hình URL, tuần tự hóa chuỗi JSON và đọc luồng dữ liệu nhị phân, **OpenFeign** cho phép bạn gọi dịch vụ REST chỉ bằng cách khai báo một **giao diện (*interface*) Java** (Hình 11.5)!



Hình 11.5: Thiết kế với OpenFeign: Bạn chỉ cần định nghĩa một interface với các chú thích Spring MVC. Spring Cloud OpenFeign sẽ tự động tạo một thực thể ủy nhiệm ngầm (*proxy*) để thực hiện toàn bộ công việc kết nối HTTP bên dưới.

Trong tệp `pom.xml` của dự án `sq-ch11-ex1`, chúng ta khai báo thêm gói phụ thuộc:

```
<dependency>
   <groupId>org.springframework.cloud</groupId>
   <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
```

Trong Listing 11.2, chúng ta khai báo interface `PaymentsProxy`. Chúng ta gắn chú thích `@FeignClient` kèm theo tên đại diện và địa chỉ URL gốc của dịch vụ mục tiêu. Bên trong interface, chúng ta định nghĩa chữ ký phương thức với các chú thích hoàn toàn tương tự như khi viết controller: `@PostMapping`, `@RequestHeader`, và `@RequestBody`.

```
package com.example.proxy;

import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestHeader;
import com.example.model.Payment;

@FeignClient(name = "payments", url = "${name.service.url}")
public interface PaymentsProxy {

  @PostMapping("/payment")
  Payment createPayment(
      @RequestHeader("requestId") String requestId,
```

```
      @RequestBody Payment payment);
}
```

Listing 11.2: Khai báo giao diện client OpenFeign (Listing 11.2)

Để Spring quét và tự động sinh mã proxy cho các interface này, chúng ta cần bổ sung chú thích **`@EnableFeignClients`** vào lớp cấu hình hoặc lớp chính của ứng dụng (Listing 11.3).

```
package com.example.config;

import org.springframework.cloud.openfeign.EnableFeignClients;
import org.springframework.context.annotation.Configuration;

@Configuration
@EnableFeignClients(basePackages = "com.example.proxy")
public class ProjectConfig {
}
```

Listing 11.3: Kích hoạt tính năng OpenFeign trong lớp cấu hình (Listing 11.3)

Giờ đây, bạn có thể tiêm `PaymentsProxy` vào bất kỳ controller hoặc service nào như một Spring bean thông thường và gọi nó như một phương thức Java cục bộ (Listing 11.4)!

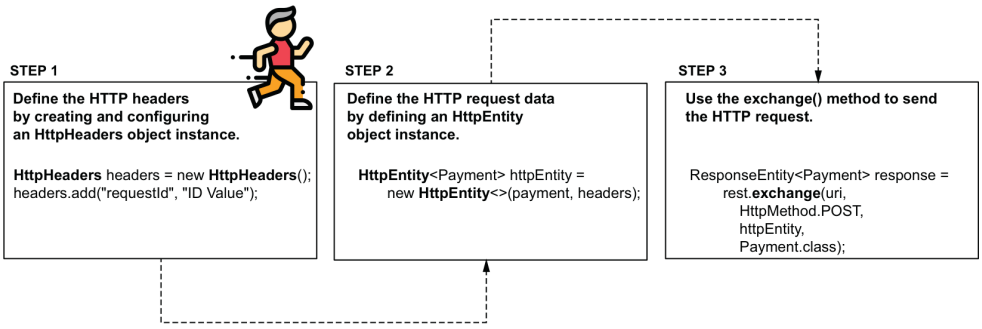
```
@RestController
public class PaymentController {

  private final PaymentsProxy paymentsProxy;

  public PaymentController(PaymentsProxy paymentsProxy) {
    this.paymentsProxy = paymentsProxy;
  }

  @PostMapping("/payment")
  public Payment createPayment(@RequestBody Payment payment) {
    String requestId = UUID.randomUUID().toString();
    return paymentsProxy.createPayment(requestId, payment);
  }
}
```

Listing 11.4: Tiêm và sử dụng OpenFeign client trong controller (Listing 11.4)

11.2 Gọi các điểm cuối REST bằng RestTemplate

Trước khi OpenFeign trở nên phổ biến, **`RestTemplate`** là công cụ tiêu chuẩn được hàng triệu nhà phát triển Spring sử dụng để gọi các dịch vụ web. `RestTemplate` cung cấp một tập hợp phong phú các phương thức tiện ích để cấu hình và thực thi các yêu cầu HTTP đồng bộ (Hình 11.6).

1. Define the HTTP headers by creating and configuring an `HttpHeaders` instance.
2. Create an `HttpEntity` instance that represents the request data (headers and body).
3. Send the HTTP call using the `exchange()` method and get the HTTP response.



Hình 11.6: Cấu trúc gọi với RestTemplate: Để gửi một yêu cầu phức tạp có đầy đủ tiêu đề và phần thân, lập trình viên tạo đối tượng `HttpEntity` và sử dụng phương thức `exchange()` của `RestTemplate`.

Trong dự án `sq-ch11-ex2`, trước tiên chúng ta định nghĩa một bean `RestTemplate` trong lớp cấu hình:

```
@Configuration
public class ProjectConfig {

  @Bean
  public RestTemplate restTemplate() {
    return new RestTemplate();
  }
}
```

Tiếp theo, chúng ta tạo lớp proxy `PaymentsProxy` để đóng gói logic gọi HTTP thông qua phương thức `exchange()` (Listing 11.5).

```
@Component
public class PaymentsProxy {

  private final RestTemplate rest;
```

```
5
6   @Value("${name.service.url}")
7   private String paymentsServiceUrl;
8
9   public PaymentsProxy(RestTemplate rest) {
10    this.rest = rest;
11  }
12
13  public Payment createPayment(Payment payment) {
14    String uri = paymentsServiceUrl + "/payment";
15
16    HttpHeaders headers = new HttpHeaders();
17    headers.add("requestId", UUID.randomUUID().toString());
18
19    HttpEntity<Payment> httpEntity = new HttpEntity<>(payment, headers);
20
21    ResponseEntity<Payment> response =
22        rest.exchange(uri,
23                      HttpMethod.POST,
24                      httpEntity,
25                      Payment.class);
26
27    return response.getBody();
28  }
29 }
```

Listing 11.5: Lớp PaymentsProxy sử dụng RestTemplate để gọi điểm cuối REST (Listing 11.5)

Listing 11.6 định nghĩa controller để kiểm thử việc tiêu thụ điểm cuối qua proxy.

```
1  @RestController
2  public class PaymentController {
3
4   private final PaymentsProxy paymentsProxy;
5
6   public PaymentController(PaymentsProxy paymentsProxy) {
7     this.paymentsProxy = paymentsProxy;
8   }
9
10  @PostMapping("/payment")
11  public Payment createPayment(@RequestBody Payment payment) {
12    return paymentsProxy.createPayment(payment);
```

```
13    }
14 }
```

Listing 11.6: Lớp controller kiểm thử việc gọi điểm cuối (Listing 11.6)

So sánh OpenFeign và RestTemplate

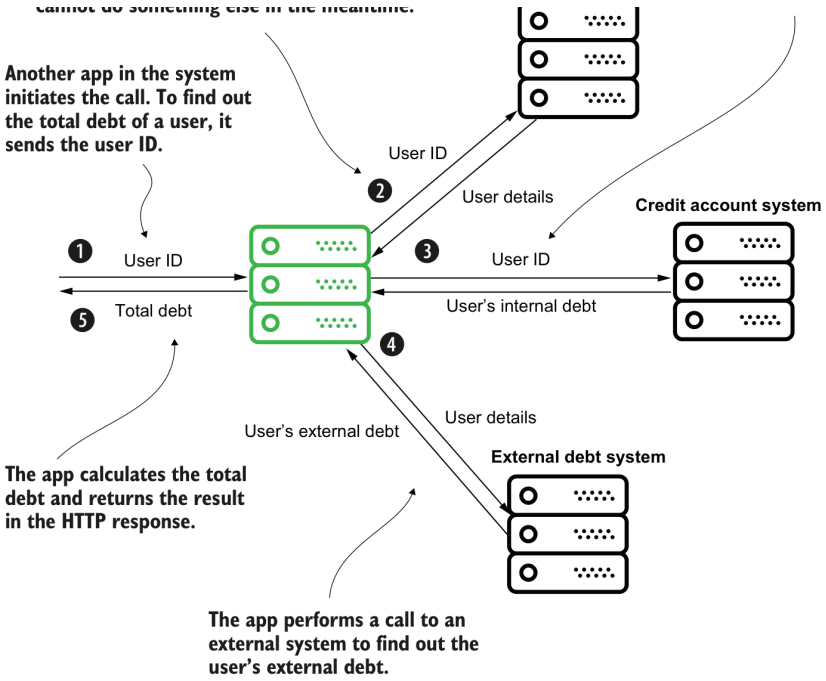
Khi so sánh Listing 11.2 (OpenFeign) và Listing 11.5 (RestTemplate), bạn sẽ thấy sự khác biệt rõ rệt. Với OpenFeign, bạn chỉ cần 1 dòng khai báo phương thức trong interface. Với RestTemplate, bạn phải tự tạo `HttpHeaders`, đóng gói vào `HttpEntity`, gọi `exchange()` và bóc tách `getBody()`. Do đó, đối với các ứng dụng mới, OpenFeign luôn là sự lựa chọn ưu tiên hàng đầu. Tuy nhiên, việc nắm vững RestTemplate là vô cùng cần thiết vì bạn sẽ bắt gặp nó ở hầu hết các dự án Spring hiện hữu trong doanh nghiệp.

11.3 Gọi các điểm cuối REST bằng WebClient (Mô hình phản ứng)

Cả OpenFeign và RestTemplate đều hoạt động theo mô hình **đồng bộ và ngăn chặn** (*synchronous and blocking*). Để hiểu rõ hạn chế của mô hình này và lý do ra đời của **Spring WebClient**, chúng ta hãy cùng phân tích bài toán hiệu năng dưới đây.

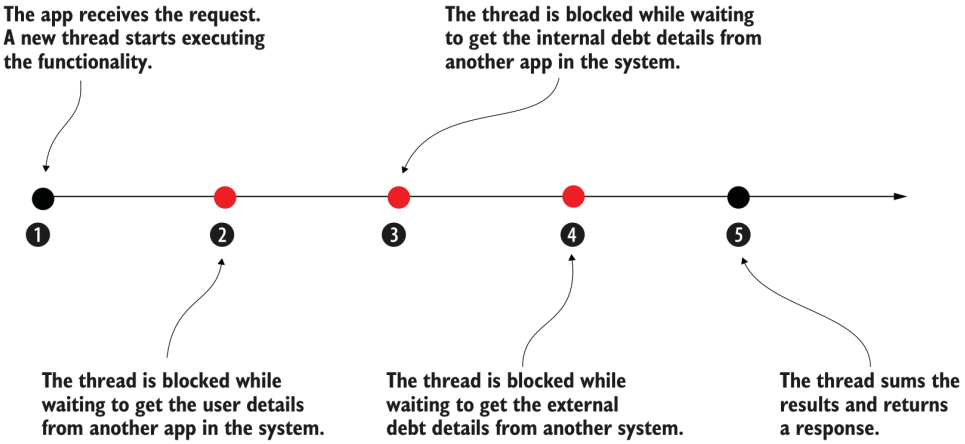
11.3.1 Hạn chế của mô hình ngăn chặn và sức mạnh của lập trình phản ứng

Hãy tưởng tượng một kịch bản thực tế (Hình 11.7 và 11.8): Ứng dụng của bạn nhận một yêu cầu từ người dùng, sau đó phải gọi sang một dịch vụ đối tác mất 3 giây để chờ kết quả phản hồi.



Hình 11.7: Kịch bản thực tế: Ứng dụng nhận yêu cầu và phải thực hiện một cuộc gọi REST sang dịch vụ bên ngoài trước khi có thể phản hồi cho người dùng.

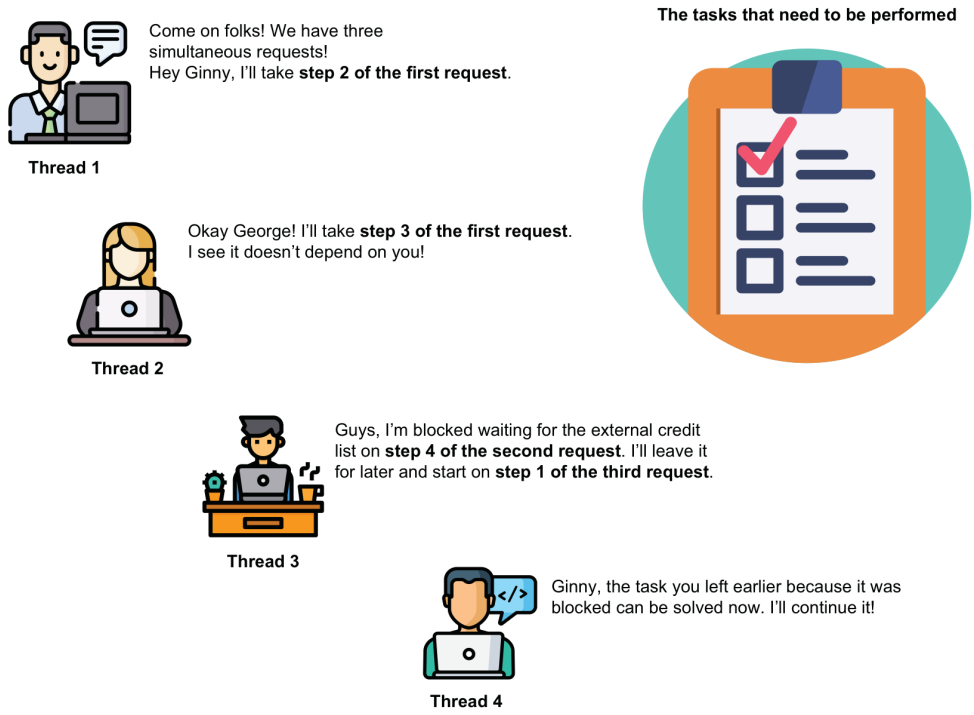
Trong mô hình truyền thống (Hình 11.8): Luồng thực thi (*thread*) của máy chủ bị **khóa cứng hoàn toàn (bị chặn - blocked)** trong suốt 3 giây đó chỉ để ngồi chờ gói tin mạng quay về! Một máy chủ Tomcat thường chỉ có một lượng luồng hữu hạn (mặc định khoảng 200 luồng). Nếu có 200 người dùng đồng thời gửi yêu cầu, toàn bộ nhóm luồng (*thread pool*) sẽ cạn kiệt, và ứng dụng của bạn sẽ lập tức bị tê liệt, từ chối mọi yêu cầu mới dù cho CPU và RAM của máy chủ vẫn đang rảnh rỗi!



Hình 11.8: Hạn chế của mô hình chặn: Luồng xử lý bị đóng băng trong suốt thời gian chờ phản hồi mạng. Khi lượng yêu cầu đồng thời tăng cao, nhóm luồng bị cạn kiệt nhanh chóng.

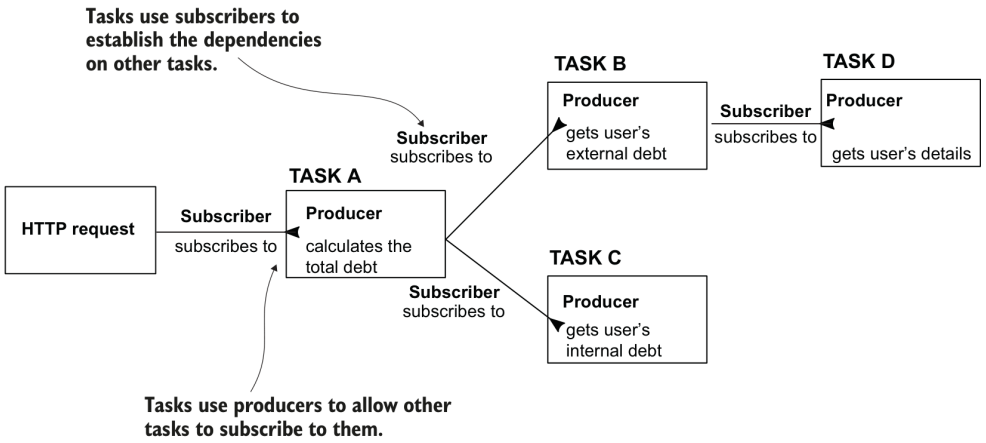
Để giải quyết vấn đề nghẽn tài nguyên này, **mô hình phản ứng (*reactive programming*)** đã ra đời.

Hãy liên tưởng tới ví dụ nhà hàng trong Hình 11.9: Nếu mỗi người bồi bàn nhận đơn của khách rồi phải đứng bất động đợi đầu bếp nấu xong 30 phút mới đi phục vụ bàn khác, nhà hàng sẽ phá sản. Trong thực tế, bồi bàn chuyển đơn xuống bếp rồi **ngay lập tức quay đi phục vụ các bàn khác**! Khi món ăn nấu xong, chuông báo reo lên (*event*), một bồi bàn rảnh rỗi sẽ mang món ra cho khách.



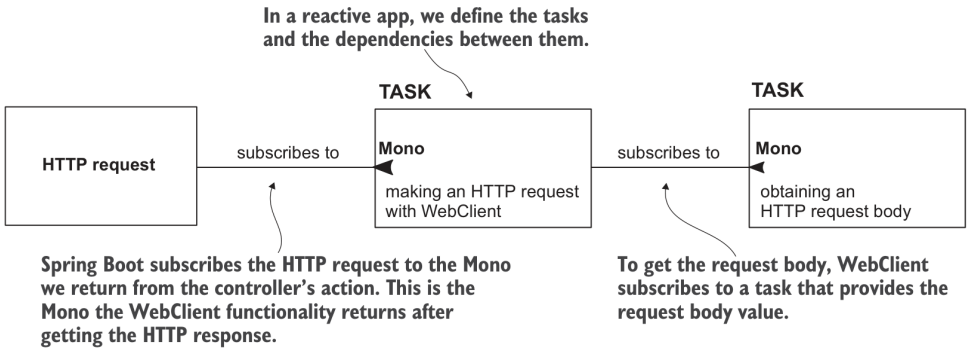
Hình 11.9: Phép ẩn dụ về mô hình phản ứng: Thay vì ngồi chờ đợi lãng phí, một luồng giải phóng chính mình ngay khi phát lệnh gọi mạng để đi phục vụ công việc khác, và chỉ quay lại xử lý tiếp khi có thông báo dữ liệu đã về.

Trong ứng dụng phản ứng (Hình 11.10 và 11.11): Quy trình xử lý được chia thành các tác vụ (*tasks*). Luồng thứ nhất nhận yêu cầu và kích hoạt cuộc gọi REST, sau đó **ngay lập tức được trả về pool** để đón nhận yêu cầu từ người dùng khác. Cuộc gọi mạng chạy bất đồng bộ ở tầng hạt nhân hệ điều hành. Khi phản hồi mạng về tới nơi, một luồng khác sẽ đón lấy kết quả và hoàn tất tác vụ phản hồi cho khách!



Hình 11.10: Mô hình phản ứng trong phần mềm: Các bước trở thành các tác vụ độc lập. Luồng phát lệnh gọi HTTP rồi lập tức quay lại phục vụ việc khác; dữ liệu về sẽ được xử lý bởi bất kỳ luồng rảnh rỗi nào.

The method also doesn't return a value directly. Instead, it returns a `Mono`, allowing another functionality to subscribe to it. This way, the app builds the flow, not by chaining them on a thread, but by linking the dependencies between tasks through producers and consumers (figure 11.11).



Hình 11.11: Chuỗi tác vụ trong ứng dụng phản ứng: Luồng xử lý được liên kết thành các ống dẫn (*pipelines*) dữ liệu luân chuyển mượt mà.

11.3.2 Sử dụng WebClient trong dự án Spring WebFlux

Để sử dụng công cụ client phản ứng, chúng ta khai báo gói phụ thuộc **Spring WebFlux** trong tệp `pom.xml` của dự án `sq-ch11-ex3`:

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
```

Trong lớp `ProjectConfig`, chúng ta cấu hình một bean **`WebClient`** (Listing 11.7).

```
@Configuration
public class ProjectConfig {

  @Bean
  public WebClient webClient() {
    return WebClient.builder().build();
  }
}
```

Listing 11.7: Khai báo bean WebClient trong lớp cấu hình (Listing 11.7)

Trong lớp `PaymentsProxy`, chúng ta sử dụng `WebClient` để tạo chuỗi gọi phản ứng (Listing 11.8). Lưu ý rằng kiểu trả về của phương thức ở đây không phải là đối tượng `Payment` thuần túy, mà là **`Mono<Payment>`**! Trong thư viện Project Reactor của Spring, `Mono` là một kiểu đại diện cho một luồng dữ liệu phản ứng phát ra tối đa 1 phần tử (hoặc không có phần tử nào) trong tương lai.

```
@Component
public class PaymentsProxy {

  private final WebClient webClient;

  @Value("${name.service.url}")
  private String url;

  public PaymentsProxy(WebClient webClient) {
    this.webClient = webClient;
  }

  public Mono<Payment> createPayment(
      String requestId,
      Payment payment) {

    return webClient.post()
        .uri(url + "/payment")
        .header("requestId", requestId)
        .body(Mono.just(payment), Payment.class)
        .retrieve()
        .bodyToMono(Payment.class);
  }
}
```

Listing 11.8: Triển khai lớp proxy gọi điểm cuối REST bằng WebClient (Listing 11.8)

Cuối cùng, trong lớp `PaymentController`, phương thức của controller cũng trả về trực tiếp đối tượng `Mono<Payment>` (Listing 11.9). Spring sẽ tự động lắng nghe và giải phóng luồng xử lý cho đến khi dữ liệu của `Mono` sẵn sàng kết xuất ra phản hồi HTTP!

```
@RestController
public class PaymentController {

  private final PaymentsProxy paymentsProxy;

  public PaymentController(PaymentsProxy paymentsProxy) {
    this.paymentsProxy = paymentsProxy;
  }

  @PostMapping("/payment")
  public Mono<Payment> createPayment(
      @RequestBody Payment payment) {

    String requestId = UUID.randomUUID().toString();
    return paymentsProxy.createPayment(requestId, payment);
  }
}
```

Listing 11.9: Controller phơi bày điểm cuối và trả về luồng dữ liệu Mono (Listing 11.9)

11.4 Tóm tắt chương

- Ứng dụng Spring thường xuyên đóng vai trò là một client tiêu thụ các điểm cuối REST từ bên ngoài (cổng thanh toán, dịch vụ nhắn tin, hoặc giữa các microservices).
- Spring cung cấp ba công cụ chính để tiêu thụ REST: **Spring Cloud OpenFeign**, **RestTemplate**, và **Spring WebClient**.
- **Spring Cloud OpenFeign:** Cung cấp phương pháp tiếp cận khai báo (*declarative*). Lập trình viên chỉ cần viết một interface với các chú thích Spring MVC quen thuộc, Spring Cloud sẽ tự động sinh mã proxy kết nối HTTP bên dưới. Đây là giải pháp tinh gọn và thanh lịch nhất cho các ứng dụng đồng bộ.
- **RestTemplate:** Công cụ truyền thống của Spring dành cho các cuộc gọi HTTP đồng bộ và ngăn chặn (*blocking*). Mặc dù đòi hỏi nhiều mã chuẩn bị (*boilerplate code*) hơn OpenFeign, nó vẫn rất phổ biến trong các dự án hiện hữu.
- **Mô hình ngăn chặn (*Blocking*):** Luồng thực thi bị đóng băng trong suốt thời gian chờ phản hồi mạng, dễ dẫn đến cạn kiệt tài nguyên nhóm luồng khi có lượng truy cập lớn.

- **Mô hình phản ứng (*Reactive*):** Phân rã luồng xử lý thành các tác vụ phi ngăn chặn (*non-blocking*). Luồng phát lệnh gọi mạng rồi ngay lập tức được giải phóng để đi phục vụ yêu cầu khác, giúp tối ưu hóa tối đa hiệu suất tài nguyên máy chủ.
- **Spring WebClient:** Công cụ hiện đại trong module Spring WebFlux, hỗ trợ hoàn hảo cả hai phong cách gọi chặn và phản ứng phi ngăn chặn thông qua các kiểu dữ liệu luồng như `Mono` và `Flux`.

Chương 12

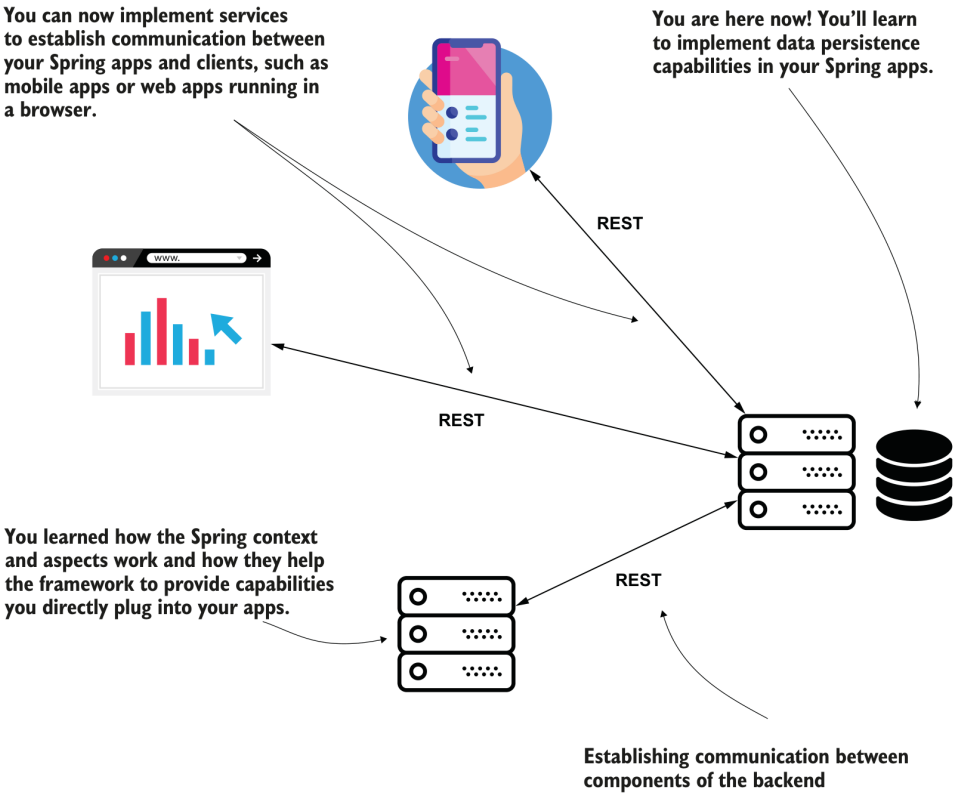
Sử dụng nguồn dữ liệu trong ứng dụng Spring

Hầu như mọi ứng dụng phần mềm thực tế ngày nay đều cần lưu trữ dữ liệu bền vững (*persistent data*), và cơ sở dữ liệu quan hệ (*relational databases*) vẫn là giải pháp thống trị hàng đầu trong ngành công nghiệp suốt nhiều thập kỷ qua. Trong các chương trước, để đơn giản hóa bài toán và tập trung vào các khái niệm web, chúng ta đã tạm thời lưu trữ dữ liệu trong các danh sách trong bộ nhớ (*in-memory lists*). Nhưng trong môi trường doanh nghiệp, dữ liệu phải được ghi nhận an toàn vào các hệ quản trị cơ sở dữ liệu như PostgreSQL, MySQL, Oracle hoặc SQL Server.

Nội dung trọng tâm của chương này

- Hiểu rõ khái niệm và vai trò then chốt của thành phần **nguồn dữ liệu** (*Data Source*) trong việc quản lý kết nối cơ sở dữ liệu.
- Khám phá cơ chế hoạt động của **bể chứa kết nối** (*connection pool*) giúp tối ưu hóa hiệu năng và ngăn chặn lãng phí tài nguyên mạng.
- Sử dụng công cụ `JdbcTemplate` của Spring để thực thi các truy vấn SQL, loại bỏ hoàn toàn hàng núi mã chuẩn bị rườm rà (*boilerplate code*) của JDBC truyền thống.
- Ánh xạ các tập kết quả truy vấn (*result sets*) thành các đối tượng Java bằng cơ chế `RowMapper`.
- Cấu hình nguồn dữ liệu linh hoạt thông qua tệp thuộc tính `application.properties` hoặc định nghĩa tường minh bean `DataSource`.

Hình 12.1 tổng kết lại chặng đường chúng ta đã đi qua: bạn đã làm chủ vùng chứa Spring context (Chương 2–6), xây dựng ứng dụng web MVC động (Chương 7–9), và thiết kế các điểm cuối REST (Chương 10–11). Giờ đây, chúng ta sẽ hoàn thiện mảnh ghép quan trọng tiếp theo: kết nối ứng dụng với tầng lưu trữ dữ liệu.

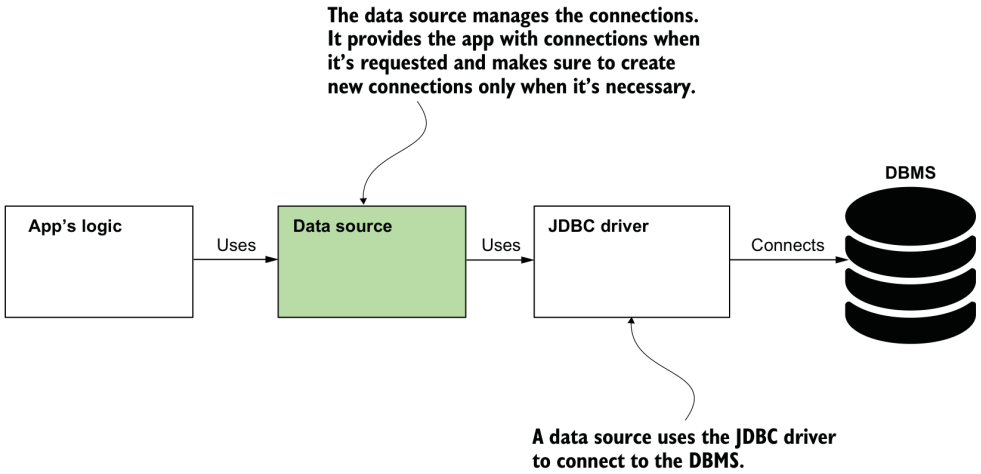


Hình 12.1: Lược đồ tiến trình học tập: Sau khi nắm vững ngữ cảnh Spring và tầng giao tiếp web/REST, chúng ta bước vào tầng lưu trữ dữ liệu cơ sở dữ liệu.

Trong Mục 12.1, chúng ta sẽ tìm hiểu bản chất của một nguồn dữ liệu (*Data Source*) và kỹ thuật quản lý bể chứa kết nối. Kế đó, trong Mục 12.2, chúng ta sẽ làm quen với `JdbcTemplate` để thực thi thêm mới và truy vấn dữ liệu. Cuối cùng, trong Mục 12.3, chúng ta sẽ tìm hiểu cách tùy biến cấu hình nguồn dữ liệu cho ứng dụng.

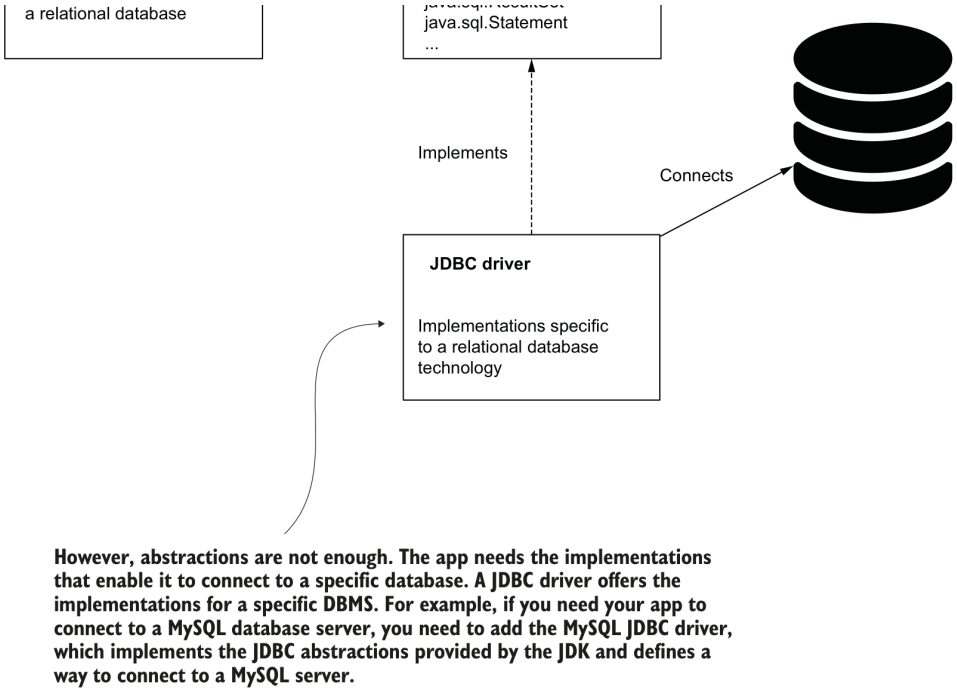
12.1 Nguồn dữ liệu (Data Source) là gì?

Trong hệ sinh thái Java, **nguồn dữ liệu** (*Data Source*) là một thành phần đại diện cho kết nối tới cơ sở dữ liệu. Về mặt kỹ thuật, nó là một thực thể triển khai giao diện chuẩn `javax.sql.DataSource` của bộ công cụ JDK (Hình 12.2).



Hình 12.2: Nguồn dữ liệu là thành phần quản lý kết nối tới cơ sở dữ liệu: Nó đóng vai trò nhà máy sản xuất và điều phối các kết nối mạng phục vụ ứng dụng.

Để hiểu vì sao chúng ta cần một `DataSource`, hãy nhìn lại cách các ứng dụng Java truyền thống kết nối tới cơ sở dữ liệu thông qua thư viện JDBC thuần túy (*Java Database Connectivity*) (Hình 12.3):



Hình 12.3: Mô hình kết nối JDBC truyền thống: Ứng dụng sử dụng `DriverManager` để thiết lập một kết nối mạng socket mới tới cơ sở dữ liệu mỗi khi cần thực thi truy vấn.

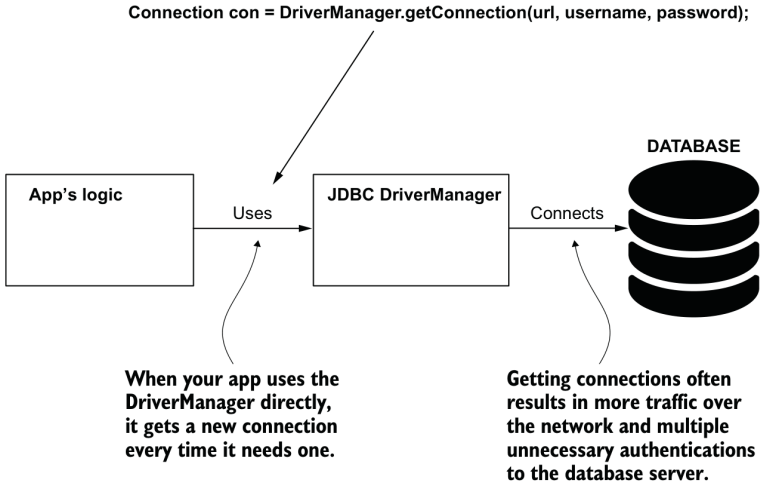
Trong mô hình này, mỗi lần ứng dụng cần gửi một câu lệnh SQL, nó gọi `DriverManager.getConnection()` để bắt tay mở một kết nối mạng mới tới máy chủ cơ sở dữ liệu. Việc bắt tay mạng (*network handshake*), xác thực người dùng và cấp phát bộ nhớ cho một kết nối cơ sở dữ liệu là một thao

tác **cực kỳ tốn kém về mặt thời gian và tài nguyên CPU**. Nếu có hàng trăm người dùng đồng thời, việc liên tục mở và đóng kết nối sẽ khiến hệ thống trở nên chậm chạp nghiêm trọng!

12.1.1 Cơ chế Bể chứa kết nối (Connection Pool)

Để giải quyết triệt để vấn đề này, các ứng dụng doanh nghiệp hiện đại luôn sử dụng **bể chứa kết nối** (*connection pool*) (Hình 12.4):

bar and ask for a beer; you look young, so the barman asks for your ID. This is fine, but it would become tedious if the barman asked you for the ID again when you ordered the second and the third beer (hypothetically, of course).



Hình 12.4: Cơ chế tái sử dụng kết nối qua Connection Pool: Thay vì mở kết nối mới rồi đóng bỏ, ứng dụng duy trì một bể kết nối được mở sẵn. Khi cần, ứng dụng mượn kết nối từ bể, thực thi truy vấn, rồi trả kết nối lại cho bể.

Một `DataSource` hiện đại đóng vai trò là bộ quản lý bể chứa kết nối. Khi ứng dụng khởi động, `DataSource` mở sẵn một số lượng kết nối nhất định. Khi một luồng nghiệp vụ cần truy vấn dữ liệu, nó mượn một kết nối rảnh rỗi từ bể. Sau khi thực thi câu lệnh SQL xong, luồng đó gọi hàm `close()` trên kết nối — nhưng thay vì ngắt kết nối vật lý, kết nối đó chỉ đơn giản là được **trả về bể chứa** để sẵn sàng phục vụ cho các yêu cầu tiếp theo!

Hình 12.5 minh họa kiến trúc phân tầng chuẩn mực khi đưa `DataSource` vào ứng dụng Spring: Tầng logic nghiệp vụ chỉ giao tiếp với tầng kho lưu trữ (*Repository*), tầng kho lưu trữ sử dụng `DataSource` để nhận kết nối và làm việc với cơ sở dữ liệu.

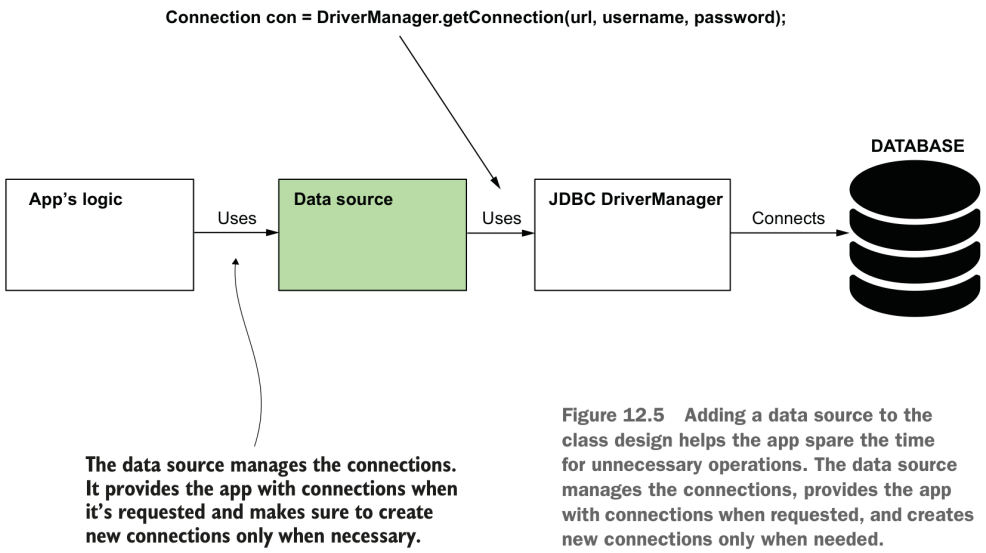


Figure 12.5 Adding a data source to the class design helps the app spare the time for unnecessary operations. The data source manages the connections, provides the app with connections when requested, and creates new connections only when needed.

Hình 12.5: Kiến trúc phân tầng với Data Source: Logic ứng dụng gọi Repository, Repository ủy thác việc quản lý kết nối cơ sở dữ liệu cho Data Source.

12.2 Sử dụng JdbcTemplate để thao tác với dữ liệu

Ngay cả khi đã có `DataSource`, việc sử dụng API JDBC tiêu chuẩn của Java vẫn là một cơn ác mộng đối với lập trình viên vì lượng mã chuẩn bị (*boilerplate code*) khổng lồ: bạn phải tự tạo `PreparedStatement`, lặp qua `ResultSet`, bắt các ngoại lệ kiểm tra `SQLException`, và đảm bảo đóng kết nối trong khối `finally`.

Để giải phóng lập trình viên khỏi gánh nặng đó, Spring Framework cung cấp công cụ kinh điển mang tên **`JdbcTemplate`**. `JdbcTemplate` tự động quản lý kết nối, bắt và chuyển hóa các ngoại lệ JDBC thành các ngoại lệ không kiểm tra (*unchecked runtime exceptions*) thân thiện của Spring, và dọn dẹp tài nguyên an toàn 100%.

12.2.1 Xây dựng ứng dụng quản lý đơn hàng với JdbcTemplate

Chúng ta sẽ cùng xây dựng dự án `sq-ch12-ex1` để quản lý các giao dịch mua hàng (*purchase*).

Trong tệp `pom.xml`, chúng ta khai báo hai gói phụ thuộc chính:

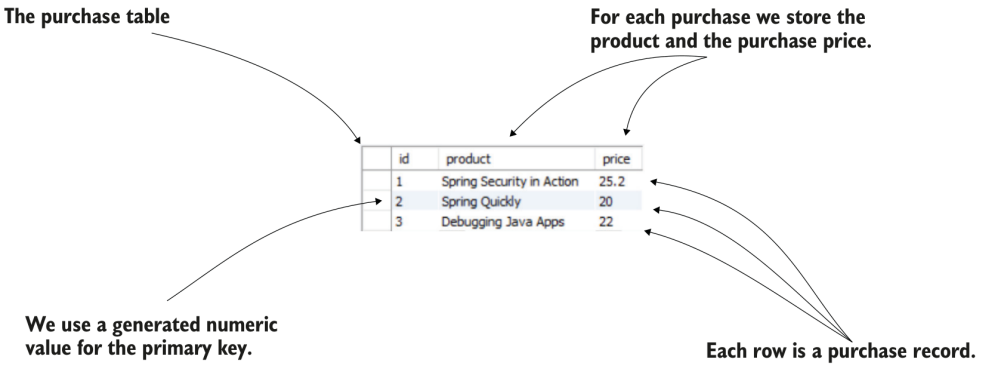
```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>
<dependency>
   <groupId>com.h2database</groupId>
   <artifactId>h2</artifactId>
```

```
    <scope>runtime</scope>
</dependency>
```

Ở đây, chúng ta sử dụng cơ sở dữ liệu **H2** chạy trực tiếp trong bộ nhớ (*in-memory database*). H2 rất nhẹ và hoàn hảo cho việc học tập, thử nghiệm và kiểm thử vì bạn không cần phải cài đặt máy chủ cơ sở dữ liệu cồng kềnh.

Bảng cơ sở dữ liệu mang tên `purchase` được thiết kế gồm ba cột (Hình 12.6):

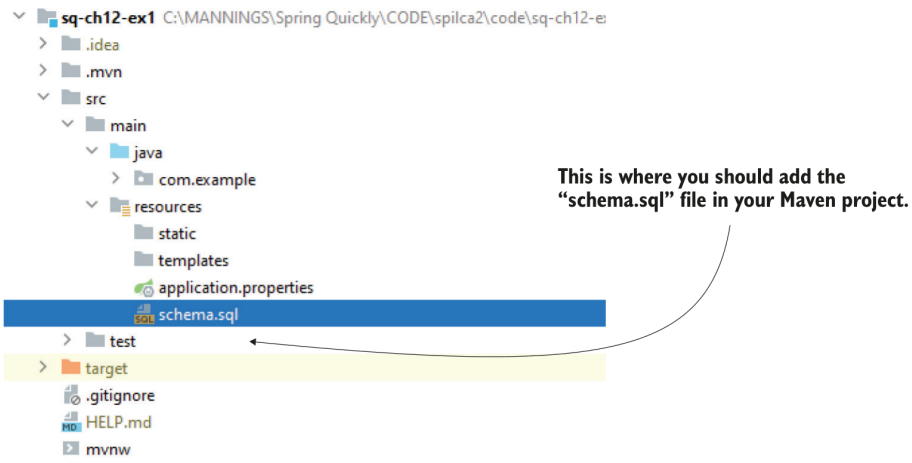
- `id`: Khóa chính kiểu số nguyên tự tăng (`INT AUTO_INCREMENT PRIMARY KEY`).
- `product`: Tên sản phẩm kiểu chuỗi ký tự (`VARCHAR(50)`).
- `price`: Đơn giá sản phẩm kiểu số thực (`NUMERIC(10, 2)`).



Hình 12.6: Cấu trúc bảng `purchase`: Mỗi giao dịch mua hàng được lưu thành một dòng gồm khóa chính `id`, tên sản phẩm và mức giá.

Chúng ta tạo tệp khởi tạo lược đồ mang tên `schema.sql` đặt trong thư mục `src/main/resources/` (Hình 12.7):

```
CREATE TABLE IF NOT EXISTS purchase (
    id INT AUTO_INCREMENT PRIMARY KEY,
    product VARCHAR(50) NOT NULL,
    price DOUBLE NOT NULL
);
```



Hình 12.7: Vị trí tệp tin `schema.sql`: Đặt trong thư mục `src/main/resources/` của dự án Maven để Spring Boot tự động thực thi khi khởi động.

Cơ chế tự động thực thi schema.sql của Spring Boot

Khi phát hiện một cơ sở dữ liệu nhúng (như H2) trên đường dẫn classpath, Spring Boot sẽ tự động tìm kiếm tệp `schema.sql` trong thư mục tài nguyên và thực thi toàn bộ các câu lệnh DDL trong đó để khởi tạo bảng trước khi ứng dụng bắt đầu phục vụ các yêu cầu!

Tiếp theo, chúng ta tạo lớp mô hình `Purchase`:

```
public class Purchase {
  private int id;
  private String product;
  private BigDecimal price;

  // Getters và Setters...
}
```

Bây giờ, chúng ta xây dựng lớp kho lưu trữ `PurchaseRepository` (Listing 12.1 và **??**). Chúng ta đánh dấu lớp bằng chú thích `@Repository` và tiêm bean `JdbcTemplate` do Spring Boot tự động cấu hình.

```
package com.example.repositories;

import java.util.List;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Repository;
import com.example.model.Purchase;

@Repository
```

```
public class PurchaseRepository {

  private final JdbcTemplate jdbc;

  public PurchaseRepository(JdbcTemplate jdbc) {
    this.jdbc = jdbc;
  }
}
```

Listing 12.1: Định nghĩa lớp PurchaseRepository và tiêm JdbcTemplate (Listing 12.1 và 12.2)

12.2.2 Thêm mới dữ liệu với phương thức update()

Để thực thi các câu lệnh SQL thay đổi dữ liệu (`INSERT`, `UPDATE`, hoặc `DELETE`), chúng ta sử dụng phương thức `jdbc.update()` của `JdbcTemplate` (Listing 12.2). Phương thức này nhận câu lệnh SQL chứa các dấu hỏi chấm `?` làm tham số giữ chỗ (*positional parameters*), kèm theo danh sách các giá trị tương ứng.

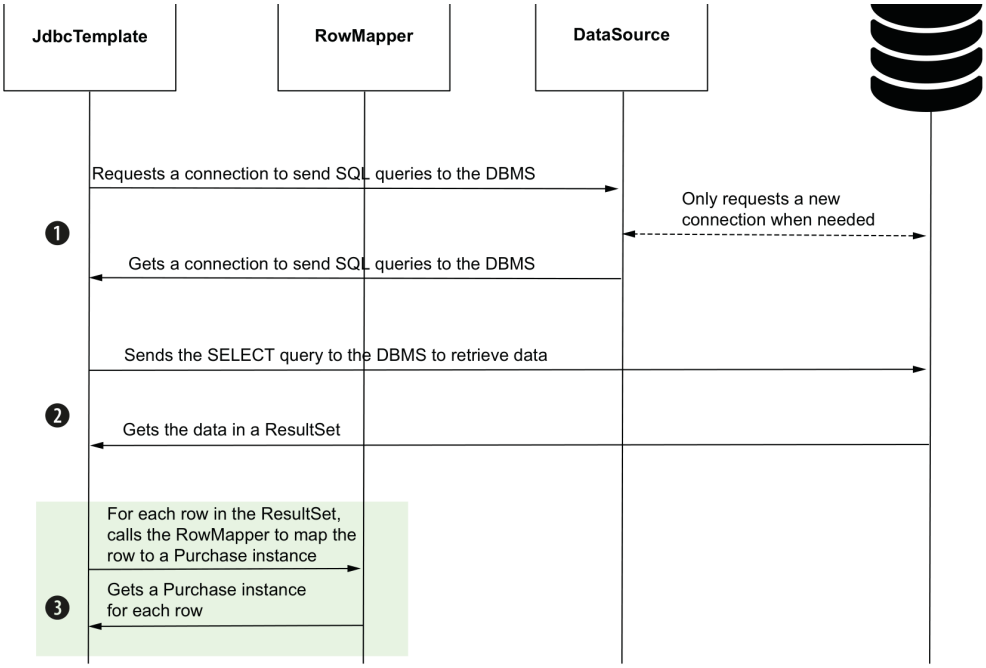
```
  public void storePurchase(Purchase purchase) {
    String sql =
        "INSERT INTO purchase (product, price) VALUES (?, ?)";

    jdbc.update(sql,
                purchase.getProduct(),
                purchase.getPrice());
  }
```

Listing 12.2: Sử dụng JdbcTemplate để thêm bản ghi mới vào cơ sở dữ liệu (Listing 12.3)

12.2.3 Truy vấn dữ liệu và ánh xạ bằng RowMapper

Để truy vấn dữ liệu (`SELECT`), chúng ta sử dụng phương thức `jdbc.query()`. Tuy nhiên, làm thế nào để `JdbcTemplate` biết cách biến đổi các cột dữ liệu trong `ResultSet` thành các thuộc tính của đối tượng Java `Purchase`?

Câu trả lời là giao diện **`RowMapper<T>`** (Hình 12.8). `RowMapper` đóng vai trò là một hàm chuyển đổi: với mỗi dòng trong tập kết quả, Spring gọi phương thức `mapRow(ResultSet r, int rowNum)` để bạn tự thiết lập đối tượng Java tương ứng.



Hình 12.8: Cơ chế hoạt động của RowMapper: `JdbcTemplate` duyệt qua từng dòng của `ResultSet` và sử dụng logic của `RowMapper` để chuyển hóa dòng dữ liệu thành một đối tượng Java.

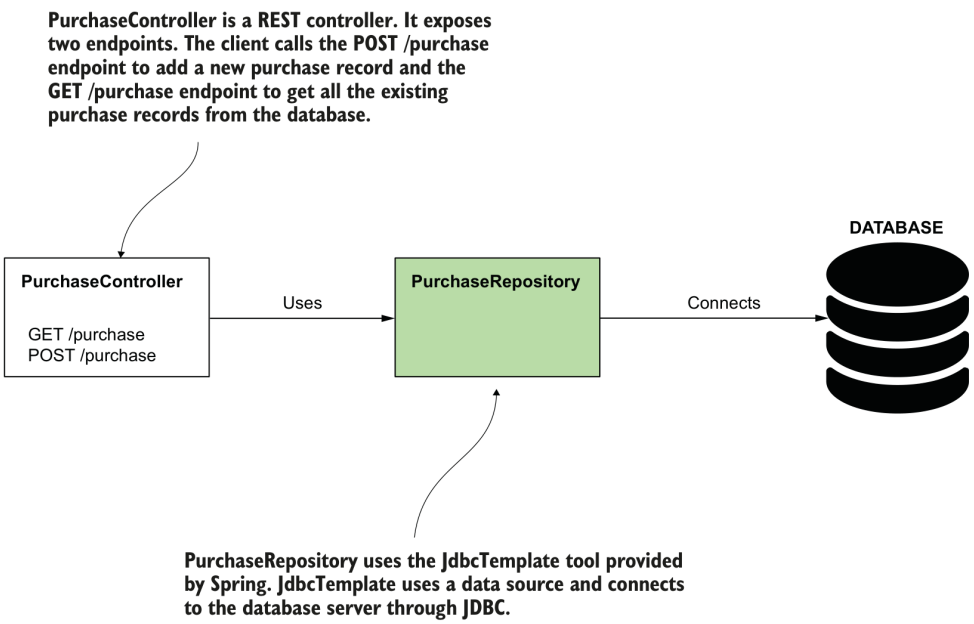
Listing 12.3 minh họa phương thức truy vấn danh sách mua hàng sử dụng biểu thức lambda triển khai `RowMapper`.

```
public List<Purchase> findAllPurchases() {
  String sql = "SELECT * FROM purchase";

  RowMapper<Purchase> purchaseRowMapper = (r, i) -> {
    Purchase rowObject = new Purchase();
    rowObject.setId(r.getInt("id"));
    rowObject.setProduct(r.getString("product"));
    rowObject.setPrice(r.getBigDecimal("price"));
    return rowObject;
  };

  return jdbc.query(sql, purchaseRowMapper);
}
```

Listing 12.3: Sử dụng JdbcTemplate để truy vấn dữ liệu từ cơ sở dữ liệu (Listing 12.4)

Cuối cùng, chúng ta xây dựng lớp điều khiển `PurchaseController` (Listing 12.4) cung cấp hai điểm cuối REST (Hình 12.9):

- `POST /purchase`: Tiếp nhận đơn mua hàng và lưu vào cơ sở dữ liệu.
- `GET /purchase`: Lấy danh sách toàn bộ các đơn mua hàng hiện có.



Hình 12.9: Hai điểm cuối REST của PurchaseController: Một điểm cuối để thêm đơn mua hàng và một điểm cuối để xem danh sách.

```java
package com.example.controllers;

import java.util.List;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import com.example.model.Purchase;
import com.example.repositories.PurchaseRepository;

@RestController
@RequestMapping("/purchase")
public class PurchaseController {

  private final PurchaseRepository purchaseRepository;

  public PurchaseController(PurchaseRepository purchaseRepository) {
    this.purchaseRepository = purchaseRepository;
  }

  @PostMapping
  public void storePurchase(@RequestBody Purchase purchase) {
    purchaseRepository.storePurchase(purchase);
```

```
  }

  @GetMapping
  public List<Purchase> findPurchases() {
    return purchaseRepository.findAllPurchases();
  }
}
```

Listing 12.4: Lớp PurchaseController sử dụng kho lưu trữ để phục vụ các yêu cầu (Listing 12.5)

12.3 Tùy biến cấu hình DataSource

Trong dự án `sq-ch12-ex1`, Spring Boot đã tự động tạo bean `DataSource` cho chúng ta dựa trên các quy ước ngầm định đối với cơ sở dữ liệu H2. Tuy nhiên, khi kết nối tới các hệ thống cơ sở dữ liệu thực tế ngoài đời (như PostgreSQL hay MySQL), bạn sẽ cần cấu hình nguồn dữ liệu một cách tường minh.

Có hai cách cấu hình phổ biến:

1. **Cấu hình qua tệp `application.properties`:** Đây là cách làm chuẩn mực và được khuyến nghị nhất trong các dự án Spring Boot. Bạn chỉ cần cung cấp các thuộc tính URL, tên tài khoản, mật khẩu và driver:

```
spring.datasource.url=jdbc:mysql://localhost:3306/demo_db
spring.datasource.username=root
spring.datasource.password=secret123
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

2. **Cấu hình tường minh bằng bean `DataSource`:** Đôi khi bạn cần sử dụng một thư viện bể chứa kết nối tùy biến đặc thù (như **HikariCP**) hoặc cần tinh chỉnh sâu các tham số kỹ thuật của pool. Khi đó, bạn có thể định nghĩa một phương thức gắn chú thích `@Bean` trả về thực thể `DataSource` trong lớp cấu hình (dự án `sq-ch12-ex2`, Listing 12.5).

```
package com.example.config;

import javax.sql.DataSource;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import com.zaxxer.hikari.HikariDataSource;

@Configuration
```

```
10 public class ProjectConfig {
11
12   @Value("${custom.datasource.url}")
13   private String datasourceUrl;
14
15   @Value("${custom.datasource.username}")
16   private String datasourceUsername;
17
18   @Value("${custom.datasource.password}")
19   private String datasourcePassword;
20
21   @Bean
22   public DataSource dataSource() {
23     HikariDataSource dataSource = new HikariDataSource();
24     dataSource.setJdbcUrl(datasourceUrl);
25     dataSource.setUsername(datasourceUsername);
26     dataSource.setPassword(datasourcePassword);
27     dataSource.setConnectionTimeout(1000);
28     return dataSource;
29   }
30 }
```

Listing 12.5: Định nghĩa tường minh bean DataSource trong lớp cấu hình (Listing 12.6)

12.4 Tóm tắt chương

- **Nguồn dữ liệu (*Data Source*):** Là thành phần cốt lõi quản lý kết nối giữa ứng dụng Java và máy chủ cơ sở dữ liệu quan hệ, triển khai giao diện chuẩn `javax.sql.DataSource`.
- **Bể chứa kết nối (*Connection Pool*):** Giúp tối ưu hóa hiệu năng bằng cách duy trì một nhóm các kết nối mở sẵn, cho phép các luồng mượn và tái sử dụng kết nối thay vì phải liên tục tạo mới và đóng kết nối vật lý.
- **`JdbcTemplate`:** Là công cụ tiện ích của Spring giúp đơn giản hóa triệt để việc tương tác với cơ sở dữ liệu qua JDBC, tự động quản lý vòng đời tài nguyên và chuyển hóa các ngoại lệ SQL thành các ngoại lệ runtime thân thiện.
- Phương thức `update()` của `JdbcTemplate` được dùng cho các câu lệnh SQL thay đổi dữ liệu (`INSERT`, `UPDATE`, `DELETE`).
- Giao diện **`RowMapper<T>`** cho phép lập trình viên định nghĩa logic ánh xạ từng dòng kết quả trong `ResultSet` thành một thực thể đối tượng Java tương ứng khi thực thi phương thức `query()`.

- Tệp `schema.sql` đặt trong thư mục tài nguyên `src/main/resources/` là giải pháp tiện lợi để tự động khởi tạo cấu trúc bảng khi ứng dụng Spring Boot khởi động.
- Bạn có thể dễ dàng cấu hình nguồn dữ liệu thông qua tệp thuộc tính `application.properties` hoặc khai báo tường minh bean `DataSource` khi cần tùy biến các thông số nâng cao của bể chứa kết nối (như HikariCP).

Chương 13

Sử dụng giao dịch trong ứng dụng Spring

Ở Chương 12, bạn đã học cách kết nối ứng dụng Spring với cơ sở dữ liệu và thực thi các câu lệnh SQL riêng lẻ bằng `JdbcTemplate`. Tuy nhiên, trong các bài toán kinh doanh thực tế, một ca sử dụng (*use case*) hiếm khi chỉ bao gồm một câu lệnh SQL duy nhất. Thay vào đó, nó thường là một chuỗi các thao tác phụ thuộc lẫn nhau. Nếu một thao tác ở giữa chuỗi thất bại mà các thao tác trước đó đã ghi vào cơ sở dữ liệu, toàn bộ dữ liệu của bạn sẽ rơi vào trạng thái mất nhất quán nghiêm trọng.

Để giải quyết bài toán sống còn này, chúng ta cần đến khái niệm **giao dịch** (*transactions*).

Nội dung trọng tâm của chương này

- Hiểu rõ bản chất của giao dịch và bốn đặc tính cốt lõi **ACID** (Nguyên tử, Nhất quán, Cô lập, Bền vững) trong hệ quản trị cơ sở dữ liệu.
- Cơ chế quản lý giao dịch khai báo (*declarative transaction management*) của Spring hoạt động như thế nào đằng sau hậu trường với Spring AOP.
- Ứng dụng chú thích `@Transactional` để bao bọc các phương thức nghiệp vụ.
- Phân tích và kiểm chứng cơ chế tự động **hoàn nguyên** (*rollback*) khi xảy ra lỗi hoặc ngoại lệ runtime.
- Cấu hình hành vi hoàn nguyên cho các ngoại lệ có kiểm tra (*checked exceptions*) với thuộc tính `rollbackFor`.

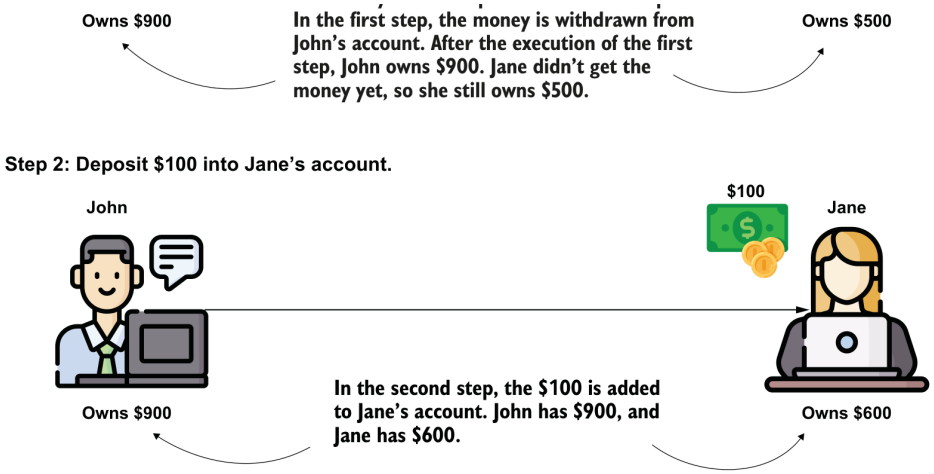
Trong Mục 13.1, chúng ta sẽ làm rõ khái niệm giao dịch và lý do tại sao mọi ứng dụng doanh nghiệp đều phụ thuộc vào nó. Kế đó, trong Mục 13.2, chúng ta sẽ khám phá cơ chế ủy nhiệm mà Spring sử dụng để điều phối giao dịch. Trong Mục 13.3, chúng ta sẽ xây dựng ứng dụng chuyển tiền ngân hàng hoàn chỉnh. Cuối cùng, Mục 13.4 sẽ kiểm chứng cơ chế hoàn nguyên khi xảy ra sự cố.

13.1 Giao dịch (Transactions) là gì?

Một **giao dịch** (*transaction*) là một tập hợp các thao tác phải được thực thi như một **khối thống nhất không thể tách rời**: hoặc là **tất cả cùng thành công**, hoặc là **tất cả cùng thất bại** mà không để lại bất kỳ dấu vết dở dang nào (*all-or-nothing*).

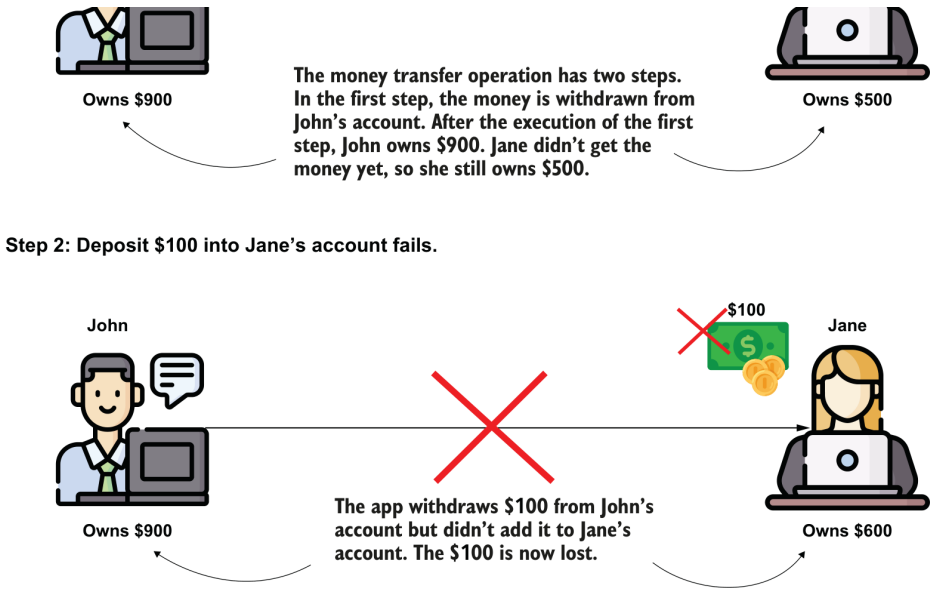
Hãy xem xét kịch bản chuyển tiền kinh điển giữa hai tài khoản ngân hàng (Hình 13.1): Người dùng muốn chuyển 100 đô-la từ tài khoản của Jane sang tài khoản của John. Ca sử dụng này bắt buộc phải trải qua hai bước tuần tự:

1. **Bước 1:** Trừ 100 đô-la từ số dư của tài khoản Jane.
2. **Bước 2:** Cộng 100 đô-la vào số dư của tài khoản John.



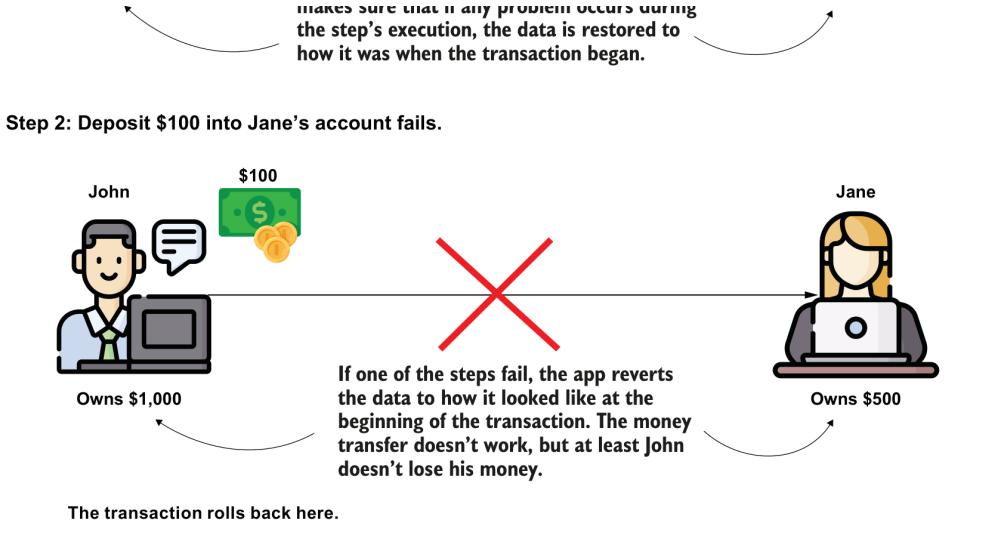
Hình 13.1: Kịch bản chuyển tiền ngân hàng: Quy trình bắt buộc phải gồm hai bước liên tiếp (trừ tiền người gửi và cộng tiền người nhận).

Điều gì sẽ xảy ra nếu hệ thống **không sử dụng giao dịch**? (Hình 13.2). Giả sử Bước 1 thực thi thành công (tiền của Jane đã bị trừ), nhưng ngay trước khi Bước 2 được thực thi, máy chủ gặp sự cố mất điện, đứt cáp mạng, hoặc cơ sở dữ liệu ném ra lỗi. Kết quả là Jane bị mất 100 đô-la nhưng John không nhận được đồng nào! Tiền đã bốc hơi khỏi hệ thống, và dữ liệu tài chính rơi vào trạng thái hư hỏng mất nhất quán. Đây là thảm họa không thể chấp nhận được trong bất kỳ phần mềm chuyên nghiệp nào.



Hình 13.2: Hậu quả khi không có giao dịch: Nếu bước thứ hai gặp sự cố, dữ liệu bị sai lệch nghiêm trọng vì bước thứ nhất đã bị ghi vĩnh viễn vào cơ sở dữ liệu.

Giao dịch giải quyết triệt để vấn đề này bằng cơ chế **hoàn nguyên** (*rollback*) (Hình 13.3): Khi có sự cố xảy ra ở bất kỳ bước nào trong chuỗi, hệ thống sẽ tự động hủy bỏ mọi thay đổi đã thực hiện trước đó trong phạm vi giao dịch, trả toàn bộ dữ liệu về trạng thái nguyên vẹn ban đầu như thể quy trình chưa từng bắt đầu!



Hình 13.3: Giao dịch bảo vệ tính nhất quán của dữ liệu: Nếu một bước thất bại, toàn bộ các bước trước đó được hoàn nguyên (*rollback*), đảm bảo dữ liệu không bao giờ bị sai lệch dở dang.

Trong lý thuyết cơ sở dữ liệu, một giao dịch phải thỏa mãn 4 tính chất vàng mang tên **ACID**:

- **A (Atomicity - Tính nguyên tử):** Toàn bộ các thao tác được xem như một đơn vị nguyên tử duy nhất; không bao giờ có trạng thái hoàn thành một nửa.
- **C (Consistency - Tính nhất quán):** Giao dịch chỉ đưa cơ sở dữ liệu từ một trạng thái hợp lệ này sang một trạng thái hợp lệ khác, tuân thủ mọi ràng buộc toàn vẹn.
- **I (Isolation - Tính cô lập):** Các giao dịch chạy đồng thời không được can thiệp hoặc nhìn thấy dữ liệu trung gian chưa hoàn tất của nhau.
- **D (Durability - Tính bền vững):** Một khi giao dịch đã thành công (*committed*), dữ liệu được lưu vĩnh viễn và không bị mất ngay cả khi hệ thống sập nguồn.

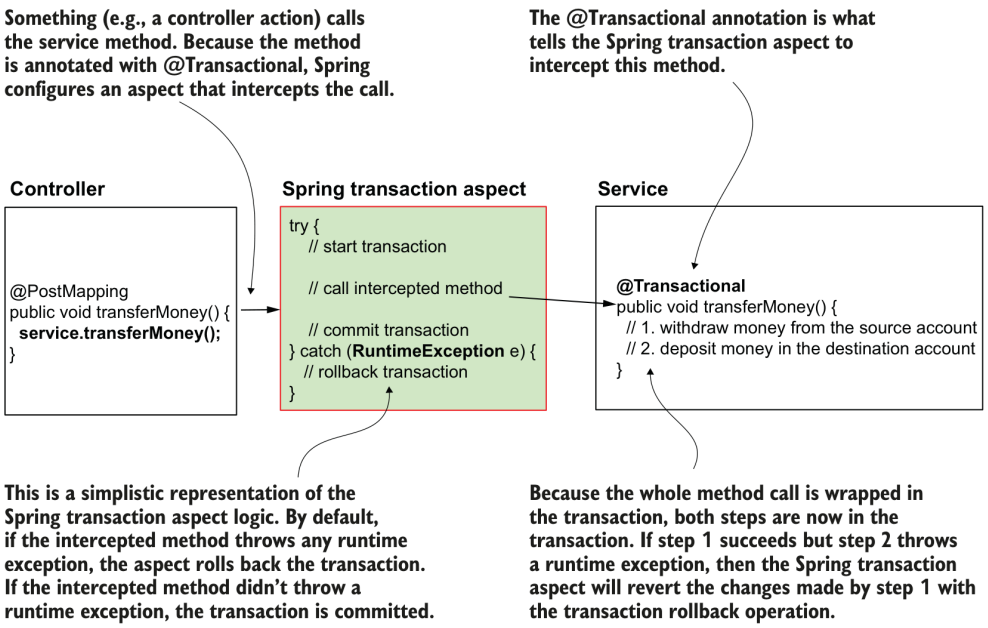
13.2 Cơ chế hoạt động của giao dịch trong Spring

Spring Framework cung cấp một cơ chế quản lý giao dịch vô cùng thanh lịch: thay vì bạn phải tự viết mã Java để mở giao dịch, commit hay rollback một cách thủ công, bạn chỉ cần gắn chú thích **`@Transactional`** lên phương thức cần bảo vệ!

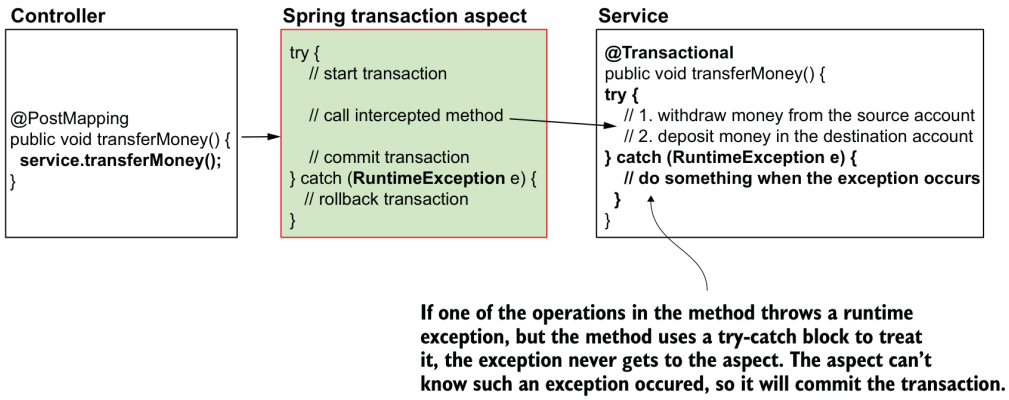
Làm thế nào Spring có thể điều khiển giao dịch chỉ với một chú thích? Hãy nhớ lại kiến thức về **Spring AOP (Lập trình hướng khía cạnh)** mà bạn đã làm chủ ở Chương 6!

Khi bạn gắn `@Transactional` lên một phương thức của service, Spring sẽ tự động tạo một đối tượng ủy nhiệm (*proxy*) bọc quanh bean đó (Hình 13.4):

1. Khi có lời gọi tới phương thức, proxy sẽ đánh chặn cuộc gọi trước tiên.
2. Proxy kết nối tới `DataSource` và gửi lệnh bắt đầu một giao dịch mới tới cơ sở dữ liệu (`BEGIN TRANSACTION`).
3. Proxy chuyển tiếp cuộc gọi tới phương thức nghiệp vụ thật của service.
4. Nếu phương thức thực thi thành công và trả về kết quả bình thường, proxy sẽ gửi lệnh xác nhận giao dịch (`COMMIT`) (Hình 13.4).
5. Nếu phương thức ném ra một ngoại lệ runtime (*RuntimeException*) hoặc lỗi hệ thống (*Error*), proxy sẽ lập tức chặn lại và gửi lệnh hủy bỏ hoàn nguyên (`ROLLBACK`) tới cơ sở dữ liệu (Hình 13.5)!



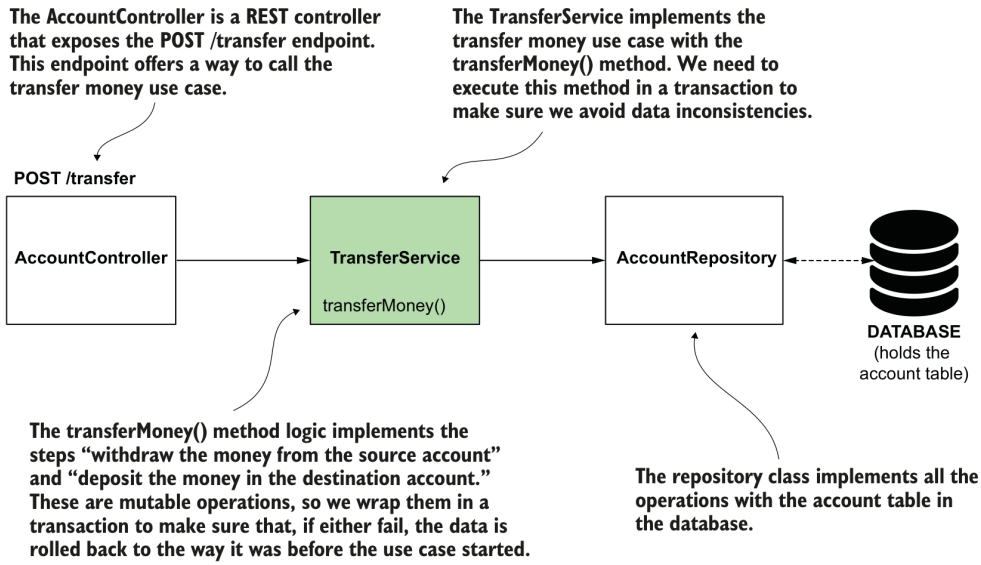
Hình 13.4: Cơ chế ủy nhiệm của chú thích @Transactional khi thành công: Proxy bắt đầu giao dịch trước khi gọi phương thức service, và xác nhận (*commit*) giao dịch sau khi phương thức hoàn thành mà không có lỗi.



Hình 13.5: Cơ chế ủy nhiệm của chú thích @Transactional khi thất bại: Khi một ngoại lệ runtime bị ném ra, proxy lập tức bắt lấy và phát lệnh hoàn nguyên (*rollback*) để khôi phục dữ liệu ban đầu.

13.3 Triển khai giao dịch trong ứng dụng Spring

Chúng ta sẽ xây dựng dự án `sq-ch13-ex1` để hiện thực hóa quy trình chuyển tiền ngân hàng giữa hai tài khoản (Hình 13.6).



Hình 13.6: Thiết kế hệ thống chuyển tiền: Lớp controller tiếp nhận yêu cầu REST, ủy thác ca sử dụng cho TransferService có gắn @Transactional, TransferService phối hợp với AccountRepository để cập nhật số dư trong cơ sở dữ liệu.

Trước hết, chúng ta tạo bảng `account` trong tệp `schema.sql`:

```
CREATE TABLE IF NOT EXISTS account (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    amount DOUBLE NOT NULL
);
```

Kèm theo tệp chèn dữ liệu mẫu ban đầu mang tên `data.sql` đặt trong thư mục `src/main/resources/`:

```
INSERT INTO account VALUES (1, 'Jane Down', 1000);
INSERT INTO account VALUES (2, 'John Read', 1000);
```

Listing 13.1 định nghĩa lớp mô hình `Account`.

```
package com.example.model;

import java.math.BigDecimal;

public class Account {

  private long id;
  private String name;
  private BigDecimal amount;

  public long getId() {
```

```
    return id;
  }

  public void setId(long id) {
    this.id = id;
  }

  public String getName() {
    return name;
  }

  public void setName(String name) {
    this.name = name;
  }

  public BigDecimal getAmount() {
    return amount;
  }

  public void setAmount(BigDecimal amount) {
    this.amount = amount;
  }
}
```

Listing 13.1: Lớp Account mô hình hóa bảng tài khoản (Listing 13.1)

Trong Listing 13.2, chúng ta xây dựng lớp kho lưu trữ `AccountRepository` với các phương thức truy vấn và cập nhật số dư tài khoản.

```
package com.example.repositories;

import java.math.BigDecimal;
import java.util.List;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Repository;
import com.example.model.Account;

@Repository
public class AccountRepository {

  private final JdbcTemplate jdbc;

  public AccountRepository(JdbcTemplate jdbc) {
```

```
    this.jdbc = jdbc;
  }

  public Account findAccountById(long id) {
    String sql = "SELECT * FROM account WHERE id = ?";
    return jdbc.queryForObject(sql, new AccountRowMapper(), id);
  }

  public void changeAmount(long id, BigDecimal amount) {
    String sql = "UPDATE account SET amount = ? WHERE id = ?";
    jdbc.update(sql, amount, id);
  }

  public List<Account> findAllAccounts() {
    String sql = "SELECT * FROM account";
    return jdbc.query(sql, new AccountRowMapper());
  }
}
```

Listing 13.2: Lớp AccountRepository cung cấp các thao tác bền vững (Listing 13.2, 13.3, 13.4)

Lớp `AccountRowMapper` ánh xạ dữ liệu từ `ResultSet` sang đối tượng `Account`:

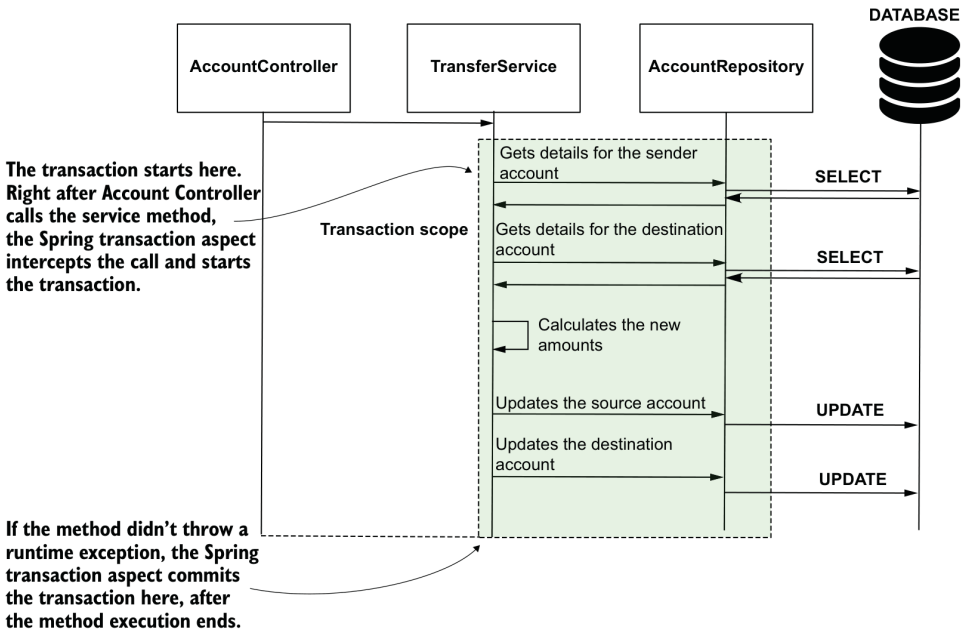
```
public class AccountRowMapper implements RowMapper<Account> {

  @Override
  public Account mapRow(ResultSet rs, int rowNum) throws SQLException {
    Account a = new Account();
    a.setId(rs.getInt("id"));
    a.setName(rs.getString("name"));
    a.setAmount(rs.getBigDecimal("amount"));
    return a;
  }
}
```

Bây giờ là trái tim của ứng dụng: lớp dịch vụ `TransferService` (Listing 13.3). Chúng ta gắn chú thích **`@Transactional`** lên phương thức `transferMoney()`. Toàn bộ chuỗi thao tác: đọc số dư người gửi, trừ tiền, đọc số dư người nhận, cộng tiền đều được bao bọc trong một phạm vi giao dịch duy nhất (Hình 13.7)!

```
package com.example.services;

```

```
import java.math.BigDecimal;
import java.util.List;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import com.example.model.Account;
import com.example.repositories.AccountRepository;

@Service
public class TransferService {

  private final AccountRepository accountRepository;

  public TransferService(AccountRepository accountRepository) {
    this.accountRepository = accountRepository;
  }

  @Transactional
  public void transferMoney(
      long idSender,
      long idReceiver,
      BigDecimal amount) {

    Account sender = accountRepository.findAccountById(idSender);
    Account receiver = accountRepository.findAccountById(idReceiver);

    BigDecimal senderNewAmount = sender.getAmount().subtract(amount);
    BigDecimal receiverNewAmount = receiver.getAmount().add(amount);

    accountRepository.changeAmount(idSender, senderNewAmount);
    accountRepository.changeAmount(idReceiver, receiverNewAmount);
  }

  public List<Account> getAllAccounts() {
    return accountRepository.findAllAccounts();
  }
}
```

Listing 13.3: Triển khai ca sử dụng chuyển tiền có gắn chú thích @Transactional (Listing 13.5 và 13.6)



Hình 13.7: Phạm vi giao dịch của phương thức transferMoney(): Giao dịch được mở ra trước khi bắt đầu đọc tài khoản và chỉ được đóng lại sau khi cả hai câu lệnh cập nhật số dư hoàn tất.

Cuối cùng, chúng ta tạo lớp `AccountController` (Listing 13.4) để phơi bày các điểm cuối REST. Lớp `TransferRequest` (Listing **??**) là DTO đón nhận phần thân JSON chứa thông tin giao dịch.

```
@RestController
public class AccountController {

  private final TransferService transferService;

  public AccountController(TransferService transferService) {
    this.transferService = transferService;
  }

  @PostMapping("/transfer")
  public void transferMoney(
      @RequestBody TransferRequest request) {

    transferService.transferMoney(
        request.getSenderAccountId(),
        request.getReceiverAccountId(),
        request.getAmount());
  }

```

```
  @GetMapping("/accounts")
  public List<Account> getAllAccounts() {
    return transferService.getAllAccounts();
  }
}
```

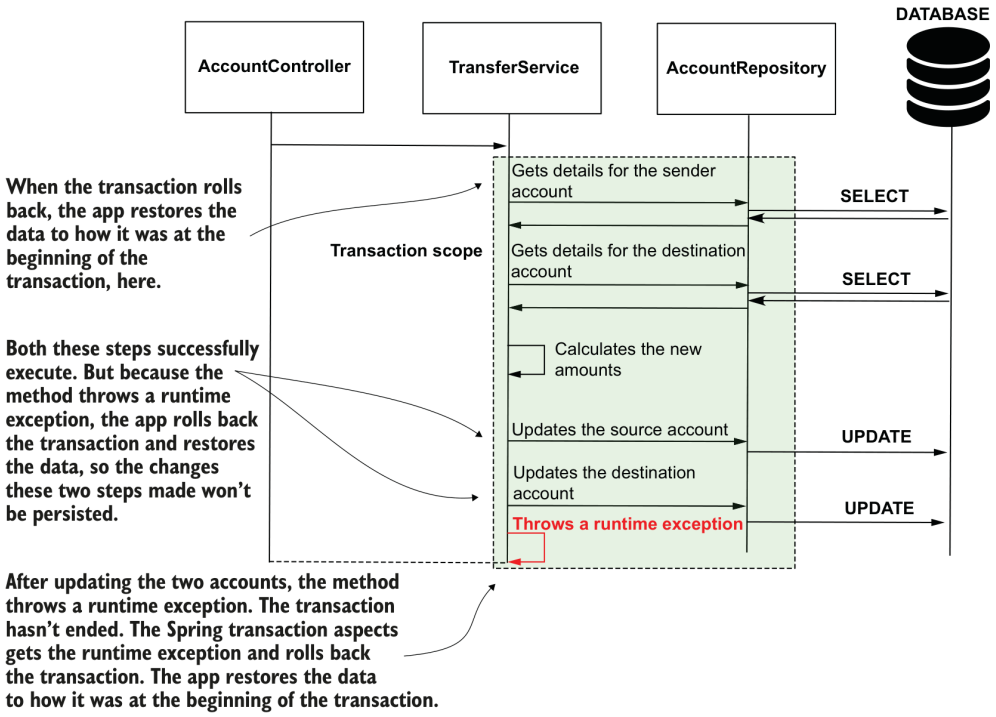
Listing 13.4: Lớp AccountController phơi bày các điểm cuối REST (Listing 13.7 và 13.8)

13.4 Kiểm thử cơ chế Rollback khi xảy ra lỗi

Để chứng minh sức mạnh của giao dịch, chúng ta xây dựng dự án `sq-ch13-ex2`: chúng ta cố tình chèn một dòng lệnh ném ngoại lệ `RuntimeException` ngay sau khi trừ tiền tài khoản người gửi (Listing 13.5, Hình 13.8).

```
  @Transactional
  public void transferMoney(
      long idSender,
      long idReceiver,
      BigDecimal amount) {

    Account sender = accountRepository.findAccountById(idSender);
    Account receiver = accountRepository.findAccountById(idReceiver);

    BigDecimal senderNewAmount = sender.getAmount().subtract(amount);
    BigDecimal receiverNewAmount = receiver.getAmount().add(amount);

    accountRepository.changeAmount(idSender, senderNewAmount);

    // ốC tình ném ạngoi ệl để ảgi ậlp ựs ốc ệh ốthng!
    throw new RuntimeException("Có ỗli ấbt ờng ảxy ra trong quá trình
   ểchuyn ềtin!");

    // Dòng ệlnh ướdi đây ẽs không bao ờgi được ạchm ớti:
    // accountRepository.changeAmount(idReceiver, receiverNewAmount);
  }
```

Listing 13.5: Mô phỏng sự cố phát sinh ngoại lệ giữa chừng trong quy trình chuyển tiền (Listing 13.9)



Hình 13.8: Kiểm thử Rollback khi gặp lỗi: Mặc dù câu lệnh cập nhật số dư người gửi đã được gửi tới cơ sở dữ liệu, nhưng ngoại lệ runtime khiến Spring Transaction Aspect hủy bỏ giao dịch và hoàn nguyên số dư về nguyên trạng.

Khi bạn gọi điểm cuối `POST /transfer` trong Postman, máy chủ trả về mã lỗi 500 do ngoại lệ ném ra. Tiếp theo, hãy gọi `GET /accounts` để kiểm tra số dư: Bạn sẽ thấy số dư của cả Jane và John vẫn là **1000 đô-la nguyên vẹn**! Câu lệnh trừ tiền đã bị cơ sở dữ liệu hủy bỏ hoàn toàn nhờ cơ chế hoàn nguyên của `@Transactional`. Dữ liệu của bạn được bảo vệ an toàn tuyệt đối.

Lưu ý sống còn về Ngoại lệ kiểm tra (Checked Exceptions) và Rollback

Theo mặc định trong Spring, chú thích `@Transactional` **chỉ tự động rollback đối với các ngoại lệ không kiểm tra** (*Unchecked Exceptions* — kế thừa từ `RuntimeException`) và các lỗi nghiêm trọng (`Error`). Nếu phương thức của bạn ném ra một ngoại lệ kiểm tra (*Checked Exception* — kế thừa trực tiếp từ `Exception`), Spring mặc định vẫn xem đó là kết quả xử lý bình thường và sẽ **COMMIT** giao dịch!

Để bắt buộc Spring phải hoàn nguyên ngay cả khi gặp ngoại lệ kiểm tra, bạn phải chỉ định tường minh thuộc tính `rollbackFor` trong chú thích:

```
@Transactional(rollbackFor = Exception.class)
```

Đây là một quy tắc tối quan trọng cần luôn ghi nhớ trong các dự án thực tế.

13.5 Tóm tắt chương

- **Giao dịch (*Transaction*):** Là một tập hợp các thao tác phải cùng thành công hoặc cùng thất bại như một khối nguyên tử duy nhất (*all-or-nothing*), ngăn chặn tình trạng dữ liệu bị sai lệch dở dang khi có sự cố phát sinh.
- Bốn thuộc tính cốt lõi của giao dịch là **ACID**: **A**tomicity (Nguyên tử), **C**onsistency (Nhất quán), **I**solation (Cô lập), và **D**urability (Bền vững).
- Chú thích `@Transactional` của Spring cung cấp giải pháp quản lý giao dịch khai báo thanh lịch, sử dụng cơ chế Spring AOP Proxy để tự động mở, xác nhận (*commit*), hoặc hoàn nguyên (*rollback*) giao dịch.
- Mặc định, Spring sẽ tự động kích hoạt hoàn nguyên khi phương thức ném ra một ngoại lệ kế thừa từ `RuntimeException` hoặc `Error`.
- Đối với các ngoại lệ có kiểm tra (*checked exceptions*), Spring mặc định vẫn commit giao dịch. Để yêu cầu rollback đối với mọi loại ngoại lệ, lập trình viên cần cấu hình thuộc tính `@Transactional(rollbackFor = Exception.class)`.

Chương 14

Triển khai lưu trữ dữ liệu với Spring Data (Implementing data persistence with Spring Data)

Trong chương này, bạn sẽ học cách sử dụng **Spring Data** — một dự án thuộc hệ sinh thái Spring giúp bạn triển khai tầng lưu trữ dữ liệu (*persistence layer*) của ứng dụng với nỗ lực tối thiểu. Như bạn đã biết, vai trò cốt lõi của một khung ứng dụng (*framework*) là cung cấp sẵn các tính năng dựng sẵn mà bạn có thể cắm trực tiếp vào phần mềm. Khung làm việc giúp chúng ta tiết kiệm thời gian, đồng thời làm cho cấu trúc thiết kế ứng dụng trở nên sáng sủa, dễ hiểu hơn rất nhiều.

Bạn sẽ học cách kiến tạo các kho lưu trữ (*repositories*) của ứng dụng chỉ bằng việc khai báo các giao diện (*interfaces*). Bạn sẽ để mặc cho framework tự động cung cấp các lớp triển khai cho những giao diện này. Bạn hoàn toàn có thể cho phép ứng dụng làm việc với cơ sở dữ liệu mà không cần phải tự tay viết mã triển khai cho repository, tiết kiệm tối đa công sức lập trình.

Nội dung trọng tâm của Chương 14

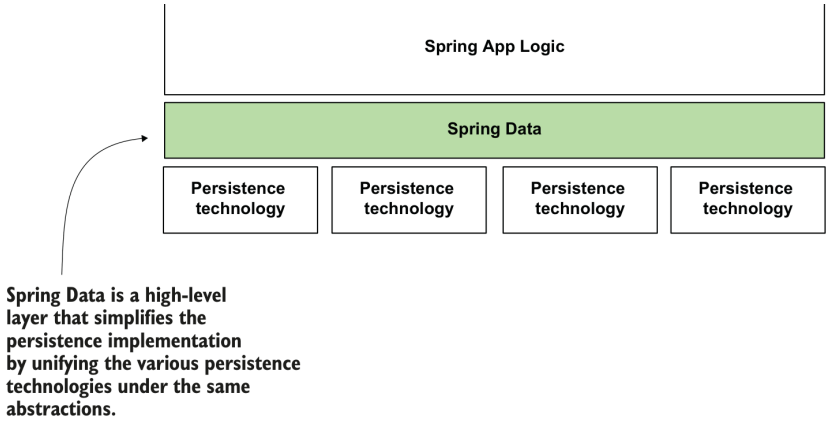
- Cách thức hoạt động của Spring Data.
- Định nghĩa các kho lưu trữ Spring Data (*Spring Data repositories*).
- Ứng dụng Spring Data JDBC để triển khai tầng lưu trữ dữ liệu cho ứng dụng Spring.

Chúng ta sẽ mở đầu chương bằng việc phân tích nguyên lý vận hành của Spring Data, và trong Mục 14.2, bạn sẽ hiểu cách Spring Data tích hợp liền mạch vào ứng dụng Spring. Kế tiếp, ở Mục 14.3, chúng ta sẽ bắt tay vào một bài toán thực tế sinh động: sử dụng Spring Data JDBC để hiện thực hóa tầng lưu trữ dữ liệu cho ứng dụng chuyển tiền điện tử.

14.1 Spring Data là gì? (What Spring Data is)

Trong mục này, chúng ta cùng bàn luận Spring Data thực chất là gì và tại sao chúng ta nên tận dụng dự án này để hiện thực hóa khả năng lưu trữ dữ liệu trong ứng dụng Spring.

Spring Data là một dự án lớn trong hệ sinh thái Spring, có sứ mệnh đơn giản hóa quá trình phát triển tầng lưu trữ dữ liệu bằng cách tự động cung cấp các lớp hiện thực tương ứng với từng công nghệ lưu trữ dữ liệu cụ thể mà chúng ta chọn. Nhờ đó, chúng ta chỉ cần viết vài dòng mã ngắn gọn là đã định nghĩa xong các repository cho ứng dụng Spring. Hình 14.1 minh họa trực quan vị trí và vai trò của Spring Data dưới góc nhìn kiến trúc ứng dụng.

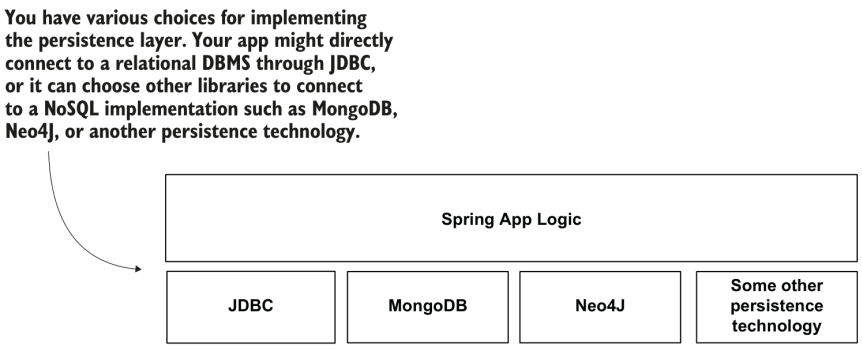


Hình 14.1: Hệ sinh thái Java cung cấp phong phú các công nghệ lưu trữ dữ liệu khác nhau. Mỗi công nghệ lại có giao diện, cơ chế và lớp thiết kế riêng. Spring Data cung cấp một tầng trừu tượng chung bao bọc toàn bộ các công nghệ này để thống nhất cách sử dụng.

Hãy xem xét vị trí của Spring Data trong một ứng dụng Spring. Trong thực tế, bạn có vô vàn công nghệ khác nhau để thao tác với dữ liệu được lưu trữ. Ở Chương 12 và Chương 13, chúng ta đã dùng JDBC — công cụ kết nối trực tiếp đến hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) thông qua bộ quản lý trình điều khiển (*driver manager*). Tuy nhiên, JDBC không phải là con đường duy nhất để chạm tới cơ sở dữ liệu quan hệ. Một cách tiếp cận cực kỳ phổ biến khác là sử dụng khung ánh xạ quan hệ - đối tượng (*Object-Relational Mapping - ORM*), tiêu biểu là Hibernate. Mặt khác, cơ sở dữ liệu quan hệ cũng không phải loại hình duy nhất: ứng dụng hoàn toàn có thể chọn lưu trữ dữ liệu trên các cơ sở dữ liệu NoSQL như MongoDB, Cassandra, hay Neo4J.

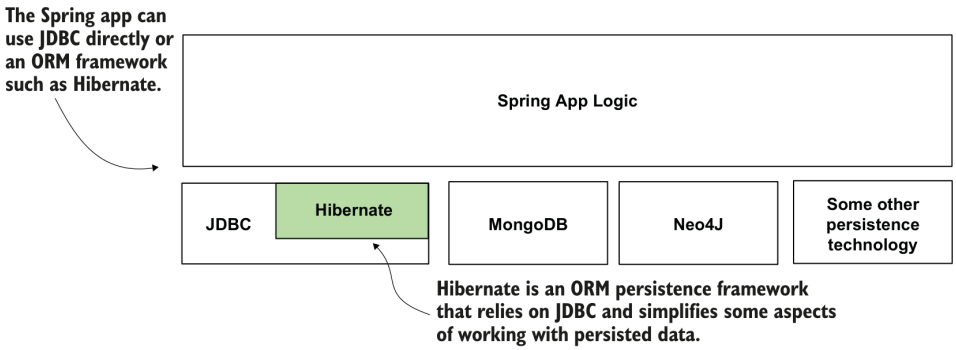
Hình 14.2 chỉ ra các lựa chọn thay thế khác nhau để lưu trữ dữ liệu. Mỗi lựa chọn lại đòi hỏi một cách triển khai repository riêng biệt. Đôi khi, ngay cả với một công nghệ duy nhất như JDBC, bạn cũng có nhiều cách tiếp cận: dùng `JdbcTemplate` (như đã học ở Chương 12) hoặc làm việc trực tiếp với các giao diện cấp thấp của JDK (`Statement`, `PreparedStatement`, `ResultSet`, v.v.). Sự phân mảnh và đa dạng quá mức này làm gia tăng đáng kể độ phức tạp của dự án.

Bức tranh kiến trúc còn phức tạp hơn nếu bổ sung thêm các khung ORM như Hibernate.



Hình 14.2: Sử dụng JDBC để kết nối RDBMS không phải là lựa chọn duy nhất. Trong thực tế, bạn có thể chọn các giải pháp NoSQL (MongoDB, Neo4J, v.v.). Mỗi giải pháp lại có thư viện và tập API riêng biệt mà lập trình viên phải học, làm tăng độ phức tạp.

Hình 14.3 mô tả vị trí của Hibernate: ứng dụng có thể dùng JDBC trực tiếp, hoặc ủy thác qua một khung làm việc trung gian dựng trên nền JDBC.



Hình 14.3: Ứng dụng đôi khi dựa vào các framework xây trên nền JDBC như Hibernate. Sự đa dạng các giải pháp khiến việc viết mã tầng lưu trữ trở nên rườm rà. Spring Data ra đời nhằm giải phóng lập trình viên khỏi gánh nặng này.

Bạn đừng quá lo lắng! Bạn không cần phải học tất cả các công nghệ này cùng một lúc, và cũng không cần biết hết chúng mới học được Spring Data. May mắn thay, những nền tảng về JDBC mà chúng ta đã tích lũy ở Chương 12 và Chương 13 là hoàn toàn đủ vững chắc để bắt đầu chinh phục Spring Data.

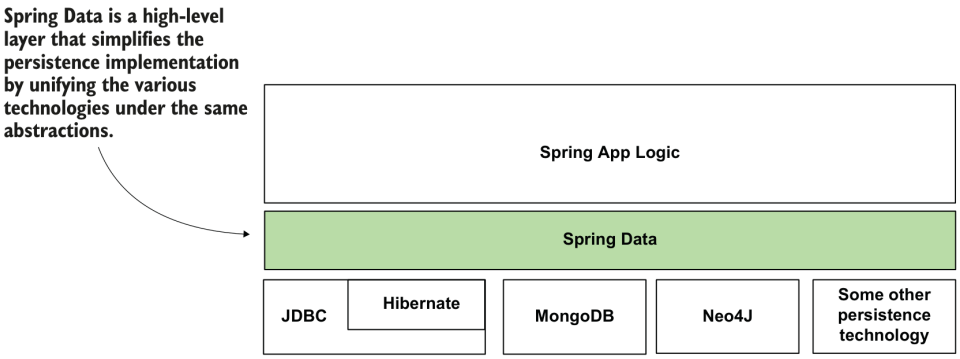
Lý do tôi liệt kê bức tranh tổng thể này là để bạn thấy rõ giá trị to lớn của Spring Data. Bạn hẳn đã từng tự hỏi: *"Liệu có cách nào để chúng ta lập trình lưu trữ dữ liệu cho mọi công nghệ này theo một phong cách thống nhất, thay vì phải học thuộc lòng từng thư viện riêng lẻ?"* Câu trả lời là **Có**, và Spring Data chính là chìa khóa để đạt được mục tiêu ấy.

Spring Data tinh giản tầng lưu trữ dữ liệu thông qua hai điểm đột phá cốt lõi:

1. **Cung cấp một tập hợp các trừu tượng (giao diện) dùng chung** cho mọi công nghệ lưu trữ dữ liệu khác nhau. Nhờ đó, bạn tiếp cận việc đọc/ghi dữ liệu theo một phương thức tương tự, bất kể công nghệ bên dưới là gì.

2. **Cho phép lập trình viên chỉ cần thao tác thông qua các hợp đồng trừu tượng**, và Spring Data sẽ chịu trách nhiệm sinh mã triển khai tự động. Bạn viết ít mã hơn, bàn giao tính năng nhanh hơn, đồng thời hệ thống trở nên ngắn gọn, dễ hiểu và dễ bảo trì hơn gấp bội.

Hình 14.4 trình bày vị thế của Spring Data trong ứng dụng. Như bạn thấy, Spring Data là một tầng trừu tượng bậc cao đứng trên tất cả các cách thức lưu trữ dữ liệu. Dù bạn chọn công nghệ nào, khi có Spring Data, bạn đều viết mã truy vấn dữ liệu theo một cách tương đồng.

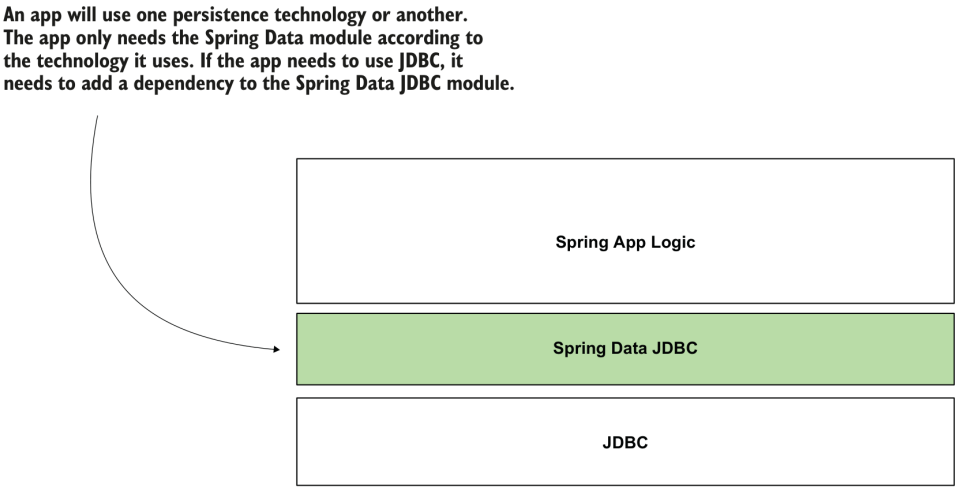


Hình 14.4: Spring Data tinh giản hóa việc triển khai tầng lưu trữ dữ liệu bằng cách cung cấp một bộ trừu tượng chung duy nhất cho nhiều công nghệ khác nhau.

14.2 Cơ chế hoạt động của Spring Data (How Spring Data works)

Trong mục này, chúng ta đi sâu vào cơ chế hoạt động của Spring Data và cách bạn sẽ áp dụng nó trong ứng dụng của mình. Khi các lập trình viên nhắc tới thuật ngữ "Spring Data", họ thường ám chỉ toàn bộ năng lực mà dự án này đem lại để kết nối ứng dụng với các công nghệ lưu trữ. Trong một ứng dụng cụ thể, thông thường bạn sẽ sử dụng một giải pháp nhất định: JDBC, Hibernate, MongoDB hoặc Neo4j.

Dự án Spring Data cung cấp nhiều mô-đun độc lập dành riêng cho từng công nghệ. Các mô-đun này tách rời nhau và được thêm vào dự án qua các phụ thuộc Maven (*dependencies*) khác nhau. Vì vậy, khi phát triển, bạn **không** thêm một phụ thuộc chung chung mang tên "Spring Data" (thực tế không tồn tại một gói phụ thuộc gộp chung như vậy). Thay vào đó, bạn thêm mô-đun chuyên biệt tương ứng với công nghệ bạn chọn. Ví dụ: bạn thêm `spring-boot-starter-data-jdbc` nếu muốn dùng JDBC, hoặc `spring-boot-starter-data-mongodb` nếu dùng MongoDB. Hình 14.5 minh họa cách mạng lưới Spring Data hoạt động khi ứng dụng chọn giải pháp JDBC.



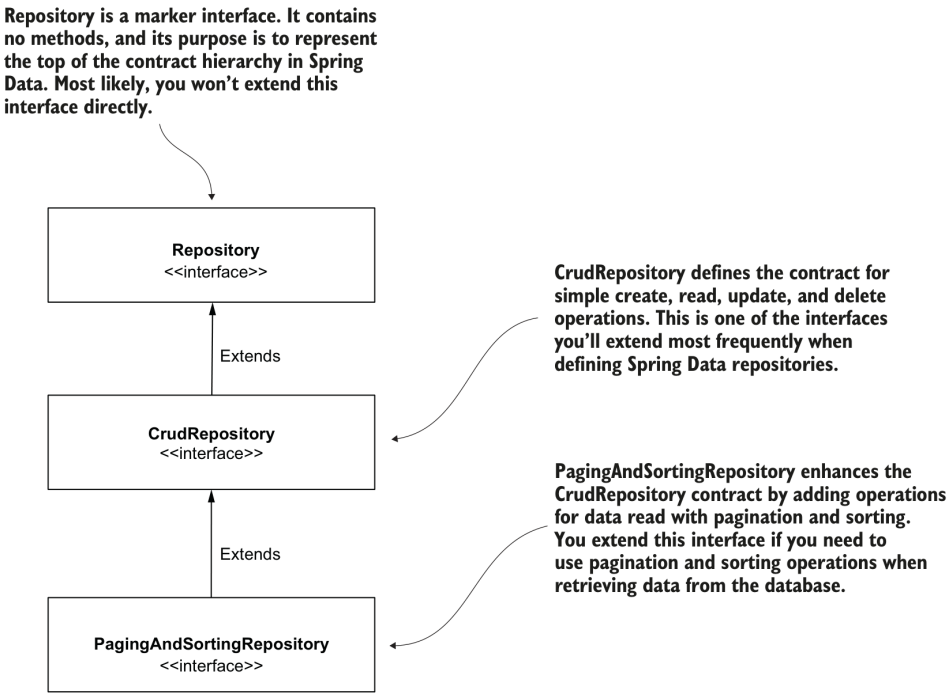
Hình 14.5: Nếu ứng dụng sử dụng JDBC, nó chỉ cần mô-đun quản lý lưu trữ qua JDBC của Spring Data, mang tên Spring Data JDBC. Mô-đun này được đưa vào dự án qua gói phụ thuộc riêng của nó.

Danh mục mô-đun Spring Data

Bạn có thể tra cứu danh sách đầy đủ các mô-đun chính thức và cộng đồng trên trang chủ của Spring Data: `https://spring.io/projects/spring-data`.

Dù ứng dụng chọn công nghệ lưu trữ nào, Spring Data đều cung cấp một hệ thống giao diện chuẩn (*contracts*) phân tầng để bạn kế thừa và khai báo năng lực lưu trữ cho ứng dụng. Hình 14.6 mô tả phả hệ ba giao diện căn bản:

- `Repository`: Là hợp đồng trừu tượng cao nhất. Nếu bạn kế thừa hợp đồng này, ứng dụng sẽ nhận biết giao diện của bạn là một kho lưu trữ Spring Data. Tuy nhiên, bạn sẽ chưa thừa hưởng bất kỳ hàm định sẵn nào (như thêm bản ghi, lấy tất cả bản ghi, hay tìm kiếm theo khóa chính). Bản thân `Repository` là một giao diện đánh dấu (*marker interface*), không chứa bất kỳ phương thức nào.
- `CrudRepository`: Là hợp đồng cơ bản nhất cung cấp sẵn các thao tác lưu trữ. Kế thừa giao diện này, bạn lập tức sở hữu các phương thức kinh điển: Tạo (*Create*), Đọc (*Read*), Cập nhật (*Update*), và Xóa (*Delete*) dữ liệu (CRUD).
- `PagingAndSortingRepository`: Kế thừa từ `CrudRepository` và bổ sung thêm các phương thức hỗ trợ sắp xếp dữ liệu (*sorting*) và phân trang (*pagination* — lấy dữ liệu theo từng trang với số lượng phần tử cố định).



Hình 14.6: Để tạo repository trong Spring Data, bạn mở rộng các giao diện chuẩn. Ba giao diện nòng cốt là `Repository`, `CrudRepository`, và `PagingAndSortingRepository`.

Phân biệt chú thích @Repository và giao diện Repository

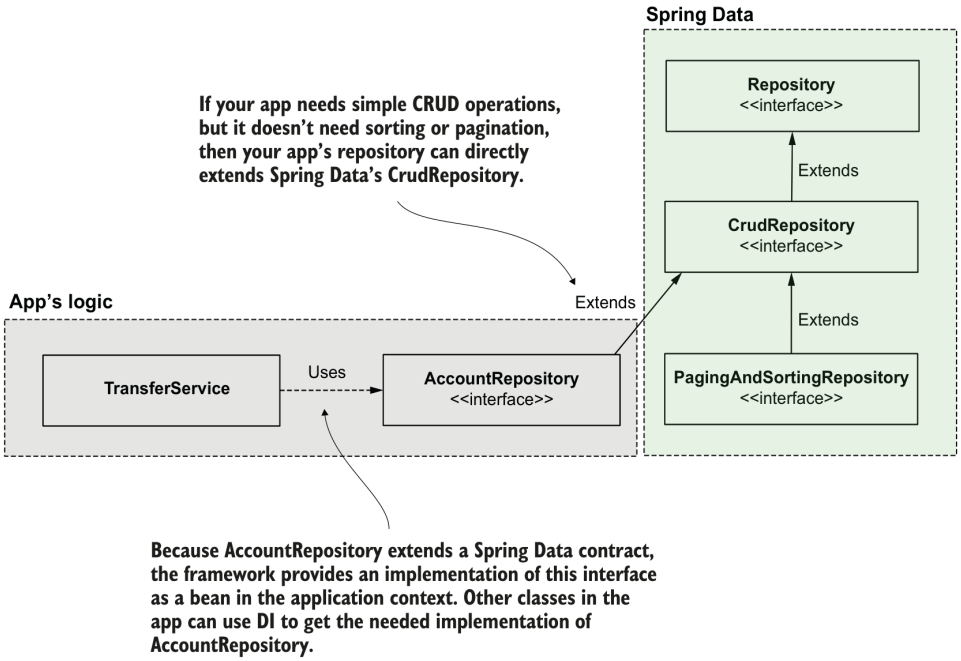
Đừng nhầm lẫn giữa chú thích `@Repository` (đã học ở Chương 4) và giao diện `Repository` của Spring Data! Chú thích `@Repository` là một chú thích định kiểu (*stereotype annotation*) dùng để đánh dấu một lớp cụ thể nhằm báo cho Spring đăng ký một bean vào context. Còn giao diện `Repository` trong chương này là một hợp đồng trừu tượng của Spring Data để bạn mở rộng (*extends*) khi xây dựng kho dữ liệu.

Có thể bạn sẽ thắc mắc: Tại sao Spring Data lại chia thành nhiều giao diện kế thừa nhau thay vì gom tất cả vào một giao diện đồ sộ duy nhất? Bằng cách tách biệt thành nhiều hợp đồng nhỏ gọn thay vì tạo một giao diện "béo phì" (*fat interface*), Spring Data cho phép ứng dụng chỉ tiếp nhận đúng những hành vi mà nó thực sự cần. Đây chính là nguyên lý phân tách giao diện (*Interface Segregation Principle - ISP*) trứ danh trong thiết kế phần mềm hướng đối tượng (SOLID).

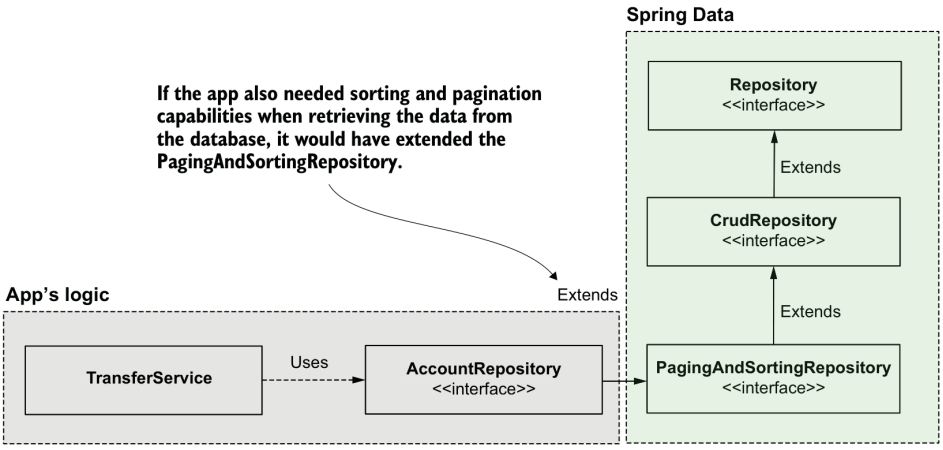
Ví dụ: Nếu ứng dụng chỉ cần các tác vụ CRUD đơn giản, nó chỉ việc kế thừa `CrudRepository`. Ứng dụng sẽ không phải bận tâm đến các hàm phân trang hay sắp xếp phức tạp, giúp thiết kế gọn gàng và tinh tế hơn (Hình 14.7).

Nếu ứng dụng cần thêm tính năng phân trang và sắp xếp kết quả bên cạnh các hàm CRUD, bạn chỉ việc cho interface kế thừa `PagingAndSortingRepository` (Hình 14.8).

Một số mô-đun Spring Data đặc thù còn cung cấp thêm các hợp đồng chuyên biệt dành



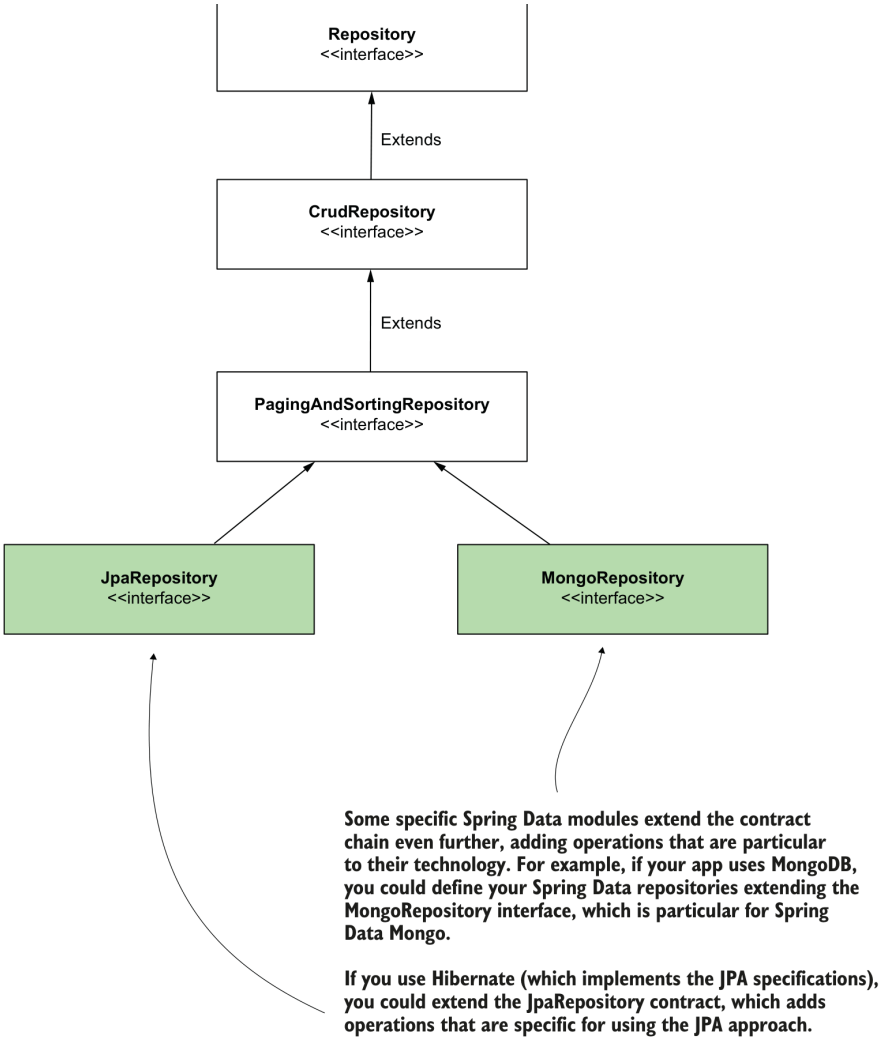
Hình 14.7: Để tạo repository, bạn định nghĩa interface mở rộng từ một hợp đồng của Spring Data. Nếu chỉ cần các hàm CRUD, interface của bạn chỉ cần kế thừa `CrudRepository`. Spring Data sẽ tự động tạo một bean triển khai interface này trong context, cho phép các thành phần khác tiêm phụ thuộc (*inject*) dễ dàng.



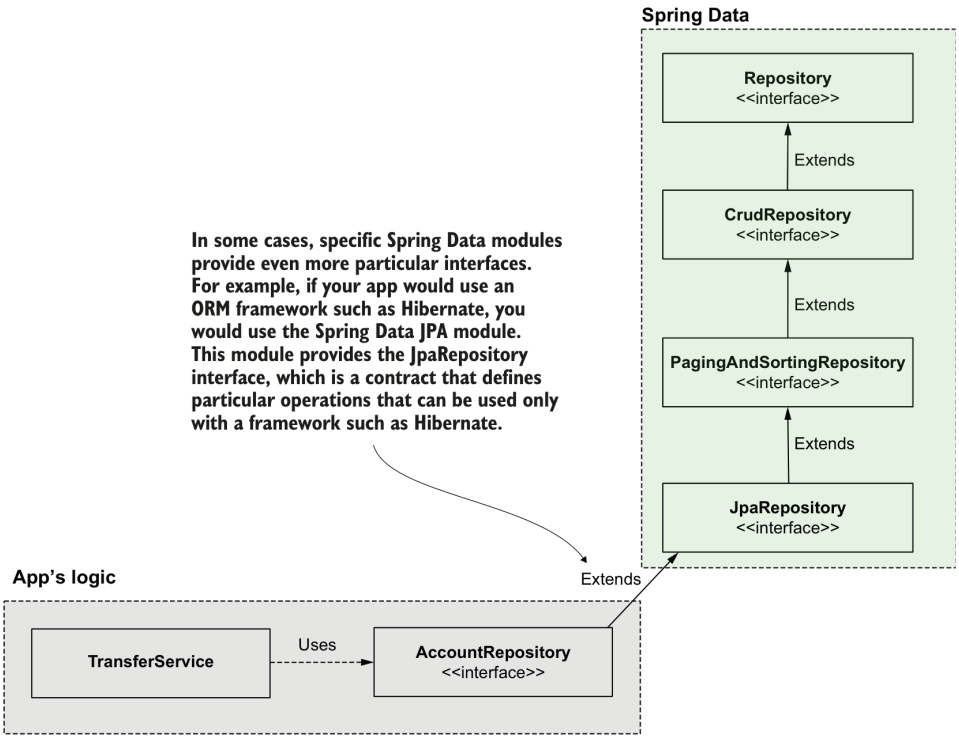
Hình 14.8: Khi cần sắp xếp và phân trang, interface mở rộng hợp đồng chuyên biệt hơn là `PagingAndSortingRepository`.

riêng cho công nghệ mà chúng đại diện. Chẳng hạn, khi sử dụng Spring Data JPA, bạn có thể kế thừa trực tiếp giao diện `JpaRepository` (Hình 14.9). `JpaRepository` là hợp đồng chi tiết hơn `PagingAndSortingRepository`, bổ sung các tác vụ đặc thù của chuẩn Jakarta Persistence API (JPA) và Hibernate (như xóa hàng loạt *deleteInBatch*, ép xả bộ đệm *flush*, v.v.).

Tương tự, với cơ sở dữ liệu MongoDB, mô-đun Spring Data Mongo mang tới hợp đồng `MongoRepository` chứa các hàm thao tác tài liệu JSON/BSON đặc thù. Hình 14.10 minh họa cấu trúc mở rộng linh hoạt này.



Hình 14.9: Các mô-đun Spring Data chuyên biệt có thể cung cấp thêm các hợp đồng cấp dưới chứa các tác vụ riêng cho từng công nghệ (ví dụ `JpaRepository` cho JPA/Hibernate hay `MongoRepository` cho MongoDB).



Hình 14.10: Sự phân nhánh của các hợp đồng Spring Data giúp lập trình viên vừa tận dụng được tính đồng nhất vừa khai thác tối đa sức mạnh của từng công nghệ chuyên biệt.

14.3 Sử dụng Spring Data JDBC trong ứng dụng thực tế

Trong mục này, chúng ta sẽ bắt tay sử dụng Spring Data JDBC để hiện thực hóa tầng lưu trữ dữ liệu cho ứng dụng Spring. Chúng ta đã bàn rằng tất cả những gì bạn cần làm là kế thừa một hợp đồng của Spring Data, nhưng bây giờ hãy chiêm ngưỡng điều đó hoạt động trên mã nguồn thực tế. Bên cạnh việc sử dụng các hàm CRUD có sẵn, bạn cũng sẽ học cách xây dựng các tác vụ truy vấn tùy biến (*custom repository operations*).

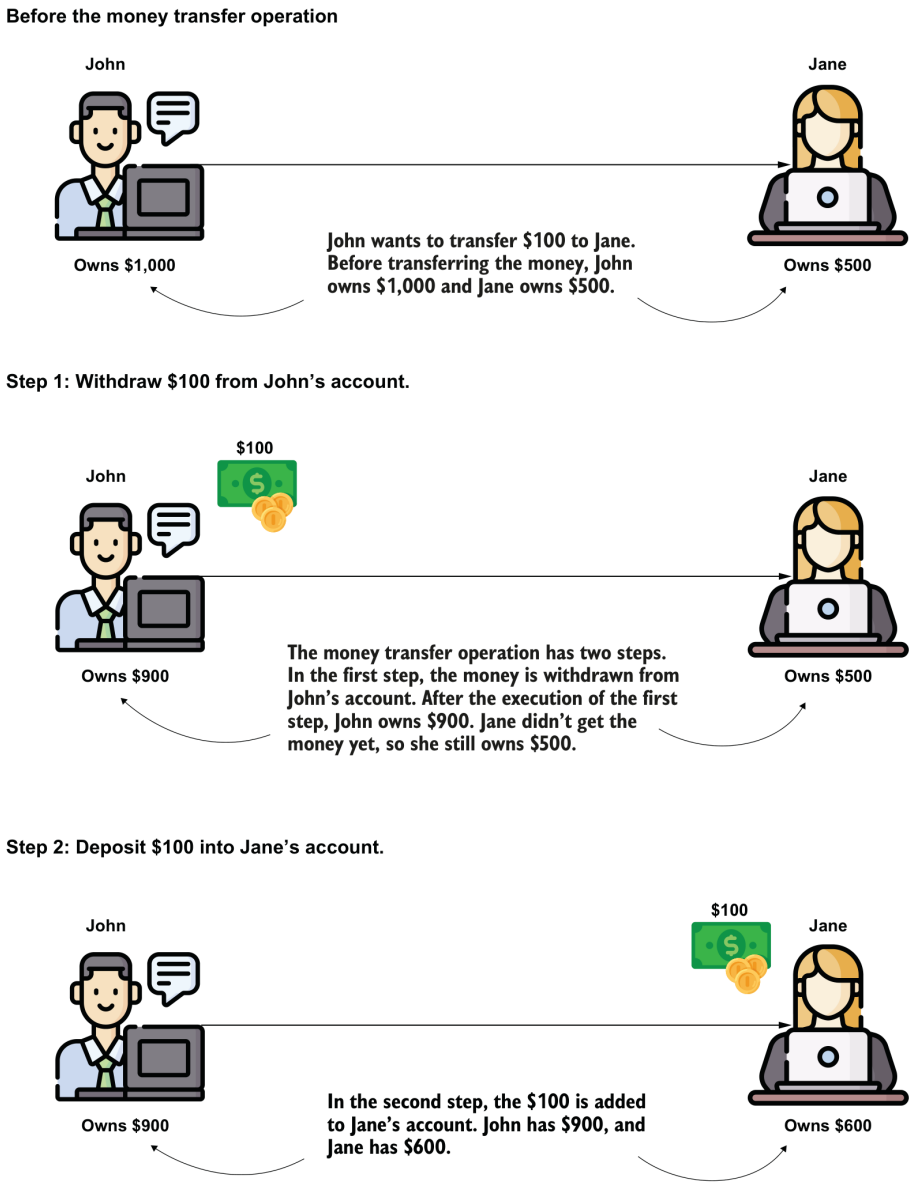
Chúng ta sẽ tiếp tục xây dựng bài toán quen thuộc từ Chương 13: ứng dụng ví điện tử quản lý tài khoản người dùng, cho phép chuyển tiền từ tài khoản này sang tài khoản khác. Quy trình chuyển tiền gồm hai bước bắt buộc (Hình 14.11):

1. Rút một khoản tiền chỉ định từ tài khoản người gửi.
2. Nạp khoản tiền đó vào tài khoản người nhận.

Chúng ta sẽ lưu thông tin tài khoản vào một bảng trong cơ sở dữ liệu H2 in-memory. Bảng `account` có các trường:

- `id`: Khóa chính, kiểu số nguyên tự tăng (`INT AUTO_INCREMENT`).
- `name`: Tên chủ tài khoản (`VARCHAR(50)`).
- `amount`: Số dư tài khoản (`DOUBLE` hoặc `DECIMAL`).

Ví dụ này có trong dự án mẫu mang tên `sq-ch14-ex1`. Các phụ thuộc Maven cần khai báo



Hình 14.11: Kịch bản chuyển tiền gồm 2 bước: Bước 1 rút tiền từ tài khoản người gửi (John); Bước 2 nạp tiền vào tài khoản người nhận (Jane).

trong tệp `pom.xml` như sau:

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-data-jdbc</artifactId>
</dependency>
<dependency>
   <groupId>com.h2database</groupId>
   <artifactId>h2</artifactId>
   <scope>runtime</scope>
</dependency>
```

Listing 14.1: Khai báo phụ thuộc trong pom.xml

Tiếp theo, ta tạo tệp `schema.sql` trong thư mục tài nguyên `src/main/resources/` để tự động khởi tạo cấu trúc bảng khi ứng dụng khởi động:

```
create table account (
    id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    amount DOUBLE NOT NULL
);
```

Listing 14.2: Tệp schema.sql khởi tạo bảng account

Để có dữ liệu ban đầu phục vụ kiểm thử, ta tạo tệp `data.sql` trong cùng thư mục `src/main/resources/`:

```
INSERT INTO account VALUES (NULL, 'Jane Down', 1000);
INSERT INTO account VALUES (NULL, 'John Read', 1000);
```

Listing 14.3: Tệp data.sql nạp dữ liệu khởi tạo

Chúng ta sẽ mô hình hóa các bản ghi trong bảng `account` bằng lớp đối tượng miền mang tên `Account`. Mỗi thuộc tính của lớp ánh xạ trực tiếp đến một cột trong bảng cơ sở dữ liệu.

Dùng BigDecimal cho dữ liệu tiền tệ

Trong thực tế, với các giá trị số thập phân đại diện cho tiền tệ, bạn nên ưu tiên sử dụng `BigDecimal` thay vì `double` hay `float` để tránh sai số dấu phẩy động trong các phép tính toán học tài chính.

Để thực hiện các tác vụ lưu trữ (đặc biệt là tìm kiếm theo khóa chính), Spring Data cần biết trường nào đóng vai trò là khóa chính của bảng. Bạn sử dụng chú thích `@Id` (thuộc gói `org.springframework.data.annotation.Id`) để đánh dấu trường này, như trong Listing 14.4.

```
package com.example.model;

import org.springframework.data.annotation.Id;
import java.math.BigDecimal;

public class Account {

  @Id
  private long id;
  private String name;
  private BigDecimal amount;

  public long getId() {
    return id;
  }

  public void setId(long id) {
    this.id = id;
  }

  public String getName() {
    return name;
  }

  public void setName(String name) {
    this.name = name;
  }

  public BigDecimal getAmount() {
    return amount;
  }

  public void setAmount(BigDecimal amount) {
    this.amount = amount;
  }
}
```

Listing 14.4: Lớp mô hình Account đại diện cho bản ghi bảng account

Khi đã có lớp mô hình, ta tiến hành định nghĩa kho lưu trữ Spring Data (Listing 14.5). Vì ứng dụng này chỉ cần các tác vụ CRUD cơ bản, ta cho giao diện kế thừa từ `CrudRepository`. Mọi giao diện của Spring Data đều yêu cầu cung cấp hai kiểu tham số generic:

1. **Kiểu thực thể miền**: Lớp mô hình đại diện cho bảng dữ liệu (ở đây là `Account`).
2. **Kiểu khóa chính**: Kiểu dữ liệu của trường được đánh dấu `@Id` (ở đây là `Long`).

```
package com.example.repository;

import com.example.model.Account;
import org.springframework.data.repository.CrudRepository;

public interface AccountRepository
    extends CrudRepository<Account, Long> {
}
```

Listing 14.5: Định nghĩa kho lưu trữ Spring Data AccountRepository

Khi bạn mở rộng giao diện `CrudRepository`, Spring Data sẽ tự động cung cấp các hàm quen thuộc: tìm theo khóa chính (`findById`), lấy tất cả bản ghi (`findAll`), lưu bản ghi (`save`), xóa bản ghi (`delete`), v.v. Nhưng nó không thể bao quát toàn bộ các nhu cầu nghiệp vụ thực tế. Làm thế nào để bổ sung các truy vấn tùy biến vào Spring Data repository?

14.3.1 Tự động sinh câu lệnh truy vấn từ tên phương thức

Spring Data hỗ trợ một tính năng vô cùng độc đáo: tự động thông dịch tên phương thức thành câu lệnh SQL dựa trên quy ước đặt tên chuẩn.

Ví dụ: Bạn muốn lấy danh sách tất cả các tài khoản khớp với một cái tên chỉ định. Trong Spring Data, bạn chỉ cần khai báo một phương thức có tên:

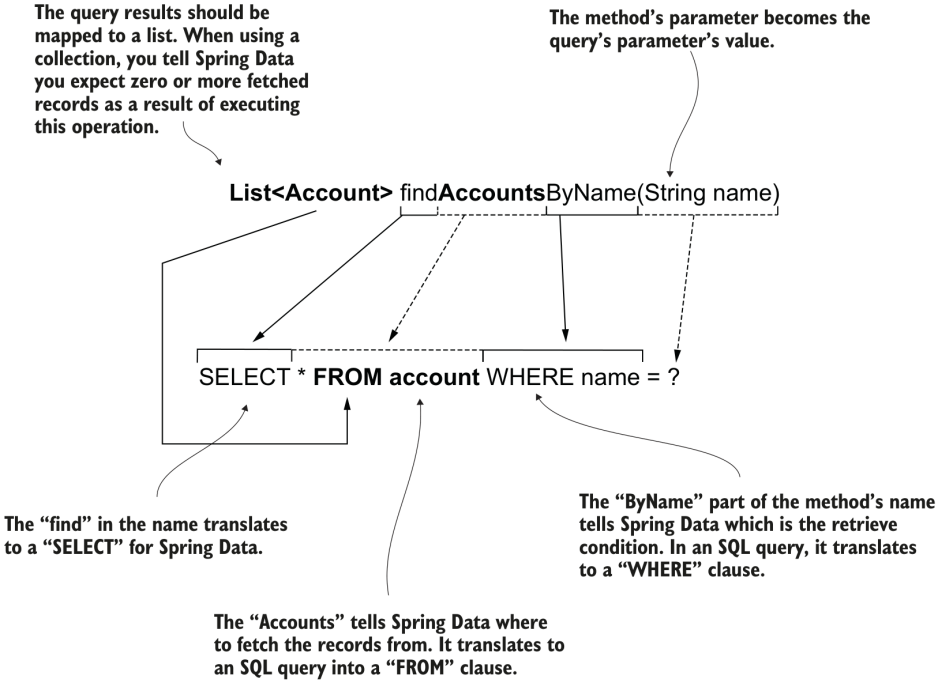
```
List<Account> findAccountsByName(String name);
```

Khi phương thức bắt đầu bằng tiền tố `find`, Spring Data hiểu rằng bạn muốn thực hiện lệnh `SELECT`. Từ tiếp theo `Accounts` chỉ định đối tượng lấy dữ liệu (thậm chí nếu bạn chỉ đặt là `findByName`, Spring Data vẫn tự hiểu vì interface này gắn với thực thể `Account`). Từ khóa `By` báo hiệu điều kiện lọc (`WHERE`), và `Name` xác định cột điều kiện là `name = ?`. Tham số truyền vào phương thức sẽ được gán tự động vào dấu hỏi chấm tương ứng trong câu lệnh SQL!

Hình 14.12 minh họa trực quan sự ánh xạ kỳ diệu giữa tên phương thức và câu lệnh SQL được Spring Data tạo ra ngầm bên dưới.

Listing 14.6 trình bày việc bổ sung phương thức này vào `AccountRepository`.

```
package com.example.repository;
```



Hình 14.12: Mối tương quan giữa tên phương thức trong repository và câu truy vấn SQL được Spring Data tạo ra phía sau hậu trường.

```
2
3  import com.example.model.Account;
4  import org.springframework.data.repository.CrudRepository;
5  import java.util.List;
6
7  public interface AccountRepository
8      extends CrudRepository<Account, Long> {
9
10   List<Account> findAccountsByName(String name);
11 }
```

Listing 14.6: Thêm thao tác tìm tài khoản theo tên vào AccountRepository

14.3.2 Khai báo tường minh truy vấn với chú thích @Query

Sự kỳ diệu của việc suy diễn truy vấn từ tên hàm thoạt nhìn rất hấp dẫn. Tuy nhiên, qua thực tế dự án, bạn sẽ nhận ra đây không phải viên đạn bạc. Nó bộc lộ một số nhược điểm lớn:

- Đối với các truy vấn phức tạp (nhiều điều kiện, nối bảng, lồng ghép), tên phương thức sẽ trở nên dài dòng thảm họa và cực kỳ khó đọc.
- Nếu một lập trình viên vô tình đổi tên phương thức khi tái cấu trúc mã nguồn (*refactoring*), hành vi truy vấn của ứng dụng có thể bị biến đổi sai lệch mà không hề có cảnh báo tại

thời điểm biên dịch.

- Bạn buộc phải học và ghi nhớ quy ước đặt tên riêng của Spring Data. Trong khi bạn đã thành thạo SQL, việc học thêm một bộ quy tắc đặt tên là điều không mấy dễ chịu.
- Tốc độ khởi động ứng dụng bị ảnh hưởng vì Spring Data phải phân tích ngữ pháp cú pháp tên hàm và dịch sang SQL khi ứng dụng nạp context lúc khởi động.

Giải pháp tối ưu và an toàn nhất được khuyến nghị là sử dụng chú thích `@Query` để chỉ định rõ ràng câu lệnh SQL bạn muốn chạy. Khi đã gắn `@Query`, tên phương thức trở nên tự do, bạn có thể đặt bất cứ tên nào dễ đọc nhất (Listing 14.7).

```
package com.example.repository;

import com.example.model.Account;
import org.springframework.data.jdbc.repository.query.Query;
import org.springframework.data.repository.CrudRepository;
import java.util.List;

public interface AccountRepository
    extends CrudRepository<Account, Long> {

  @Query("SELECT * FROM account WHERE name = :name")
  List<Account> findAccountsByName(String name);
}
```

Listing 14.7: Sử dụng chú thích @Query để chỉ định truy vấn SQL tường minh

Quy ước tham số trong @Query

Lưu ý rằng tên tham số trong câu SQL (đặt sau dấu hai chấm `:name`) phải trùng khớp chính xác với tên của tham số truyền vào phương thức Java (`String name`). Giữa dấu hai chấm và tên biến tuyệt đối không có khoảng trắng.

Đối với các câu lệnh làm biến đổi dữ liệu (`INSERT`, `UPDATE`, `DELETE`), ngoài chú thích `@Query`, bạn bắt buộc phải gắn thêm chú thích `@Modifying` để thông báo cho Spring Data biết đây là tác vụ cập nhật chứ không phải truy vấn đọc (Listing 14.8).

```
package com.example.repository;

import com.example.model.Account;
import org.springframework.data.jdbc.repository.query.Modifying;
import org.springframework.data.jdbc.repository.query.Query;
import org.springframework.data.repository.CrudRepository;
import java.math.BigDecimal;
import java.util.List;
```

```
9
10 public interface AccountRepository
11     extends CrudRepository<Account, Long> {
12
13   @Query("SELECT * FROM account WHERE name = :name")
14   List<Account> findAccountsByName(String name);
15
16   @Modifying
17   @Query("UPDATE account SET amount = :amount WHERE id = :id")
18   void changeAmount(long id, BigDecimal amount);
19 }
```

Listing 14.8: Định nghĩa thao tác cập nhật dữ liệu với chú thích @Modifying

14.3.3 Triển khai tầng dịch vụ (Service) và quản lý giao dịch

Giờ đây, bạn có thể sử dụng cơ chế tiêm phụ thuộc (DI) để đưa bean triển khai `AccountRepository` vào bất cứ lớp nào cần nó. Bạn hoàn toàn không cần bận tâm về việc mình chỉ khai báo giao diện: Spring Data sẽ tự động tạo một đối tượng ủy quyền động (*dynamic proxy*) cài đặt interface này và nạp vào Application Context lúc chạy.

Listing 14.9 minh họa lớp dịch vụ `TransferService` tiêm `AccountRepository` thông qua constructor.

```
1  package com.example.service;
2
3  import com.example.repository.AccountRepository;
4  import org.springframework.stereotype.Service;
5
6  @Service
7  public class TransferService {
8
9    private final AccountRepository accountRepository;
10
11   public TransferService(AccountRepository accountRepository) {
12     this.accountRepository = accountRepository;
13   }
14 }
```

Listing 14.9: Tiêm repository vào lớp dịch vụ TransferService

Tiếp theo, chúng ta hoàn thiện phương thức chuyển tiền `transferMoney`. Chúng ta gắn chú thích `@Transactional` (như đã học ở Chương 13) để đảm bảo hai thao tác rút tiền và nạp

tiền diễn ra trong cùng một giao dịch nguyên tử (Listing 14.10).

```
package com.example.service;

import com.example.model.Account;
import com.example.repository.AccountRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.math.BigDecimal;

@Service
public class TransferService {

  private final AccountRepository accountRepository;

  public TransferService(AccountRepository accountRepository) {
    this.accountRepository = accountRepository;
  }

  @Transactional
  public void transferMoney(
      long idSender,
      long idReceiver,
      BigDecimal amount) {

    Account sender = accountRepository.findById(idSender)
        .orElseThrow(() -> new AccountNotFoundException());

    Account receiver = accountRepository.findById(idReceiver)
        .orElseThrow(() -> new AccountNotFoundException());

    BigDecimal senderNewAmount = sender.getAmount().subtract(amount);
    BigDecimal receiverNewAmount = receiver.getAmount().add(amount);

    accountRepository.changeAmount(idSender, senderNewAmount);
    accountRepository.changeAmount(idReceiver, receiverNewAmount);
  }
}
```

Listing 14.10: Triển khai nghiệp vụ chuyển tiền an toàn với @Transactional

Trong đoạn mã trên, nếu không tìm thấy tài khoản, ứng dụng sẽ ném ngoại lệ tùy biến `AccountNotFoundException`:

```
package com.example.service;

public class AccountNotFoundException extends RuntimeException {
}
```

Listing 14.11: Lớp ngoại lệ AccountNotFoundException

Để phục vụ việc kiểm thử, ta bổ sung thêm hai phương thức tra cứu dữ liệu vào `TransferService` (Listing 14.12). Lưu ý rằng phương thức `findAll()` được thừa hưởng trực tiếp từ `CrudRepository`.

```
package com.example.service;

import com.example.model.Account;
import com.example.repository.AccountRepository;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class TransferService {

  // Cac doan ma truoc do giu nguyen...

  public Iterable<Account> getAllAccounts() {
    return accountRepository.findAll();
  }

  public List<Account> findAccountsByName(String name) {
    return accountRepository.findAccountsByName(name);
  }
}
```

Listing 14.12: Bổ sung các phương thức tra cứu tài khoản trong TransferService

14.3.4 Phơi bày các điểm cuối REST Controller và kiểm thử

Bước cuối cùng là xây dựng `AccountController` để phơi bày các điểm cuối REST cho người dùng hoặc ứng dụng client gọi vào (Listing 14.13 và Listing 14.15).

```
package com.example.controller;

import com.example.dto.TransferRequest;
import com.example.service.TransferService;
```

```
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class AccountController {

  private final TransferService transferService;

  public AccountController(TransferService transferService) {
    this.transferService = transferService;
  }

  @PostMapping("/transfer")
  public void transferMoney(@RequestBody TransferRequest request) {
    transferService.transferMoney(
        request.getSenderAccountId(),
        request.getReceiverAccountId(),
        request.getAmount());
  }
}
```

Listing 14.13: Phơi bày điểm cuối chuyển tiền qua AccountController

Lớp đối tượng truyền dữ liệu (*DTO*) `TransferRequest` được định nghĩa như sau:

```
package com.example.dto;

import java.math.BigDecimal;

public class TransferRequest {
  private long senderAccountId;
  private long receiverAccountId;
  private BigDecimal amount;

  public long getSenderAccountId() {
    return senderAccountId;
  }

  public void setSenderAccountId(long senderAccountId) {
    this.senderAccountId = senderAccountId;
  }

```

```
  public long getReceiverAccountId() {
    return receiverAccountId;
  }

  public void setReceiverAccountId(long receiverAccountId) {
    this.receiverAccountId = receiverAccountId;
  }

  public BigDecimal getAmount() {
    return amount;
  }

  public void setAmount(BigDecimal amount) {
    this.amount = amount;
  }
}
```

Listing 14.14: Lớp DTO TransferRequest ánh xạ thân yêu cầu HTTP

Listing 14.15 bổ sung điểm cuối `GET /accounts` để tra cứu toàn bộ danh sách tài khoản hoặc lọc theo tên thông qua tham số truy vấn (`@RequestParam`).

```
package com.example.controller;

import com.example.model.Account;
import com.example.service.TransferService;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class AccountController {

  // Ma nguon truoc do giu nguyen...

  @GetMapping("/accounts")
  public Iterable<Account> getAllAccounts(
      @RequestParam(required = false) String name) {
    if (name == null) {
      return transferService.getAllAccounts();
    } else {
      return transferService.findAccountsByName(name);
    }
```

```
  }
}
```

Listing 14.15: Điểm cuối lấy danh sách tài khoản trong AccountController

Khởi động ứng dụng Spring Boot và gửi yêu cầu kiểm tra danh sách tài khoản ban đầu bằng lệnh cURL:

```
curl http://localhost:8080/accounts
```

Kết quả phản hồi JSON hiển thị hai tài khoản với số dư khởi tạo $1000:

```
[
 {"id":1,"name":"Jane Down","amount":1000.0},
 {"id":2,"name":"John Read","amount":1000.0}
]
```

Bây giờ, ta gọi điểm cuối `/transfer` để thực hiện chuyển $100 từ tài khoản Jane (id = 1) sang John (id = 2):

```
curl -XPOST -H "content-type:application/json" \
  -d '{"senderAccountId":1, "receiverAccountId":2, "amount":100}' \
  http://localhost:8080/transfer
```

Sau khi lệnh chuyển tiền hoàn tất thành công, ta gọi lại điểm cuối `/accounts`:

```
curl http://localhost:8080/accounts
```

Kết quả phản hồi xác nhận số dư đã được cập nhật chính xác: Jane còn $900, và John tăng lên $1100:

```
[
 {"id":1,"name":"Jane Down","amount":900.0},
 {"id":2,"name":"John Read","amount":1100.0}
]
```

Cuối cùng, ta có thể thử nghiệm tính năng lọc theo tên chủ tài khoản qua tham số truy vấn:

```
curl http://localhost:8080/accounts?name=Jane+Down
```

Kết quả chỉ trả về thông tin tài khoản của riêng Jane Down:

```
[
  {
    "id": 1,
    "name": "Jane Down",
    "amount": 900.0
```

```
  }
]
```

14.4 Tóm tắt nội dung chương (Summary)

- **Spring Data** là một dự án lớn thuộc hệ sinh thái Spring giúp chúng ta hiện thực hóa tầng lưu trữ dữ liệu của ứng dụng một cách dễ dàng và nhanh chóng nhất. Spring Data cung cấp một tầng trừu tượng thống nhất cho đa dạng các công nghệ lưu trữ dữ liệu khác nhau thông qua một hệ thống hợp đồng chung.
- Với Spring Data, chúng ta xây dựng các repository dưới dạng các giao diện (*interfaces*) mở rộng từ các hợp đồng chuẩn của framework:
  - `Repository`: Giao diện đánh dấu cấp cao nhất, không chứa phương thức sẵn có nào.
  - `CrudRepository`: Cung cấp các thao tác CRUD cơ bản (thêm, đọc, sửa, xóa).
  - `PagingAndSortingRepository`: Mở rộng `CrudRepository`, bổ sung các phương thức sắp xếp và phân trang dữ liệu.
- Mỗi công nghệ cơ sở dữ liệu sẽ tương ứng với một mô-đun Spring Data độc lập. Chẳng hạn, ứng dụng kết nối RDBMS qua JDBC sẽ dùng `spring-boot-starter-data-jdbc`, còn ứng dụng dùng MongoDB sẽ khai báo `spring-boot-starter-data-mongodb`.
- Khi mở rộng một hợp đồng của Spring Data, ứng dụng sẽ thừa hưởng toàn bộ các thao tác được khai báo trong hợp đồng đó. Ngoài ra, bạn hoàn toàn có thể định nghĩa các phương thức nghiệp vụ tùy biến bổ sung.
- Bạn có thể dùng chú thích `@Query` trên các phương thức của repository để chỉ định tường minh câu lệnh truy vấn SQL mà ứng dụng sẽ thực thi.
- Nếu không dùng chú thích `@Query`, Spring Data sẽ cố gắng thông dịch tên phương thức thành câu lệnh SQL dựa trên quy tắc đặt tên. Tuy nhiên, nếu tên phương thức quá phức tạp hoặc bị viết sai quy ước, ứng dụng sẽ vấp phải lỗi ngoại lệ ngay khi khởi động.
- Việc sử dụng chú thích `@Query` tường minh luôn được khuyến nghị mạnh mẽ hơn so với dựa dẫm vào tên hàm, bởi vì:
  - Tránh được việc sinh ra những cái tên phương thức dài dòng, phức tạp, khó đọc.
  - Giảm thiểu rủi ro biến đổi hành vi ngoài ý muốn khi tái cấu trúc (*refactor*) tên phương thức.
  - Tiết kiệm thời gian vì không phải học thuộc toàn bộ bảng quy tắc đặt tên riêng của Spring Data.
  - Cải thiện hiệu năng khởi động ứng dụng vì không phải phân tích cú pháp chuỗi tên hàm sang SQL khi nạp context.
- Mọi tác vụ làm thay đổi dữ liệu (`INSERT`, `UPDATE`, `DELETE`) khi dùng với `@Query` bắt buộc phải được gắn thêm chú thích `@Modifying` để Spring Data nhận biết và thực thi

câu lệnh dưới dạng cập nhật dữ liệu.

Chương 15

Kiểm thử ứng dụng Spring (Testing your Spring app)

Trong chương này, bạn sẽ học cách triển khai kiểm thử (*testing*) cho ứng dụng Spring. Một bài kiểm thử là một đoạn mã logic nhỏ gọn có mục đích xác nhận xem một tính năng cụ thể mà ứng dụng xây dựng có hoạt động đúng như mong đợi hay không. Chúng ta phân loại kiểm thử thành hai nhóm chính:

- **Kiểm thử đơn vị (Unit tests)**: Chỉ tập trung kiểm chứng một khối logic độc lập, cô lập hoàn toàn khỏi mọi phụ thuộc bên ngoài.
- **Kiểm thử tích hợp (Integration tests)**: Tập trung xác thực sự tương tác, phối hợp nhịp nhàng giữa nhiều thành phần khác nhau trong hệ thống.

Khi tôi dùng thuật ngữ "kiểm thử" (*test*) nói chung, tôi đang muốn ám chỉ đến cả hai thể loại này.

Kiểm thử đóng vai trò sống còn đối với bất kỳ ứng dụng phần mềm chuyên nghiệp nào. Chúng đảm bảo rằng những thay đổi chúng ta thực hiện trong quá trình phát triển không làm gãy đổ (*break*) các tính năng hiện có (hoặc ít nhất là giảm thiểu tối đa xác suất xảy ra lỗi), đồng thời các ca kiểm thử còn đóng vai trò như một tài liệu đặc tả sống động cho mã nguồn. Đáng tiếc thay, nhiều lập trình viên thường xem nhẹ việc viết kiểm thử vì chúng không trực tiếp cấu thành nghiệp vụ hiển hiện cho người dùng cuối và việc viết chúng cũng tiêu tốn một lượng thời gian nhất định. Do đó, các bài test thoạt nhìn có vẻ không đem lại tác động rõ rệt trước mắt. Nhưng hãy tin tôi, kiểm thử có giá trị vô giá trong dài hạn. Tôi không thể nhấn mạnh hết tầm quan trọng của việc kiểm thử thấu đáo logic ứng dụng của bạn.

Nội dung trọng tâm của Chương 15

- Vì sao kiểm thử phần mềm lại có ý nghĩa sống còn?
- Cơ chế vận hành của kiểm thử tự động.
- Triển khai kiểm thử đơn vị (*unit tests*) cho ứng dụng Spring với JUnit 5 và Mockito.
- Triển khai kiểm thử tích hợp Spring (*Spring integration tests*) với `@SpringBootTest` và `@MockBean`.

Tại sao bạn nên viết kiểm thử tự động thay vì chỉ kiểm thử thủ công (*manual testing*)?

1. **Tự động hóa lặp đi lặp lại**: Bạn có thể chạy bài kiểm thử đó hàng nghìn lần để xác nhận hệ thống vận hành trơn tru với nỗ lực tối thiểu (đảm bảo tính đúng đắn liên tục).
2. **Tài liệu kỹ thuật sống**: Khi đọc các bước trong ca kiểm thử, bất kỳ ai cũng có thể nắm bắt mục đích và quy chuẩn của ca nghiệp vụ một cách dễ dàng.
3. **Phản hồi sớm (*Early feedback*)**: Kiểm thử cung cấp cảnh báo lập tức ngay khi một lỗi mới vừa phát sinh trong chu trình lập trình.

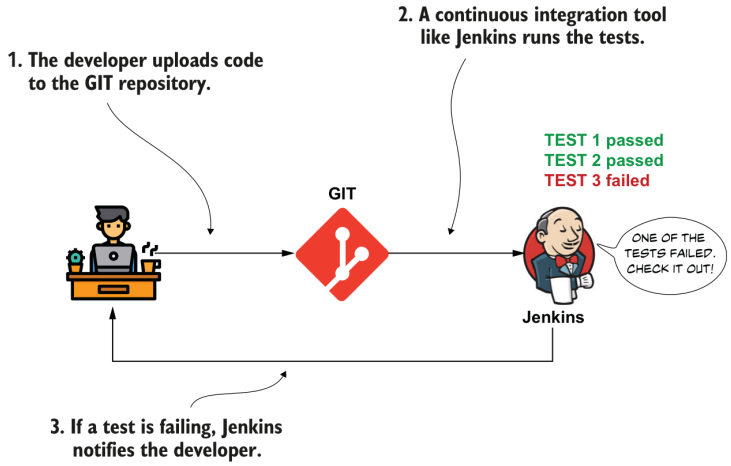
Vì sao một tính năng ban đầu chạy tốt nhưng sau đó lại có nguy cơ bị hỏng? Bởi vì trong thực tế, chúng ta liên tục sửa đổi mã nguồn để vá lỗi (*bug fixes*) hoặc bổ sung tính năng mới. Khi sửa mã, bạn rất dễ vô tình làm hỏng các tính năng cũ đã xây dựng trước đó. Nếu có hệ thống kiểm thử tự động bảo hộ, bạn có thể chạy lại bộ test bất cứ lúc nào để kiểm tra xem mã cũ có bị ảnh hưởng hay không. Kỹ thuật này được gọi là **kiểm thử hồi quy** (*regression testing*).

Ngày nay, chúng ta không chỉ trông chờ vào việc lập trình viên tự chạy test bằng tay trên máy cá nhân, mà biến việc thực thi kiểm thử thành một mắt xích tự động trong quy trình đóng gói phần mềm. Các đội ngũ phát triển áp dụng mô hình **Tích hợp liên tục** (*Continuous Integration - CI*): họ cấu hình các công cụ tự động hóa như Jenkins, GitHub Actions hoặc TeamCity để kích hoạt tiến trình dựng ứng dụng (*build*) mỗi khi lập trình viên đẩy mã nguồn lên kho lưu trữ Git. Công cụ CI sẽ tự động chạy toàn bộ các bài kiểm thử và bắn thông báo ngay lập tức nếu phát hiện ca kiểm thử nào bị thất bại (Hình 15.1).

Ở Mục 15.1, chúng ta sẽ phác họa bức tranh tổng thể về cách thiết kế ứng dụng để dễ kiểm thử. Sau đó, ở Mục 15.2, chúng ta sẽ đi sâu vào thực hành hai loại kiểm thử quan trọng nhất: kiểm thử đơn vị và kiểm thử tích hợp trên các ca nghiệp vụ cụ thể.

15.1 Vị trí đúng đắn của kiểm thử trong thiết kế phần mềm

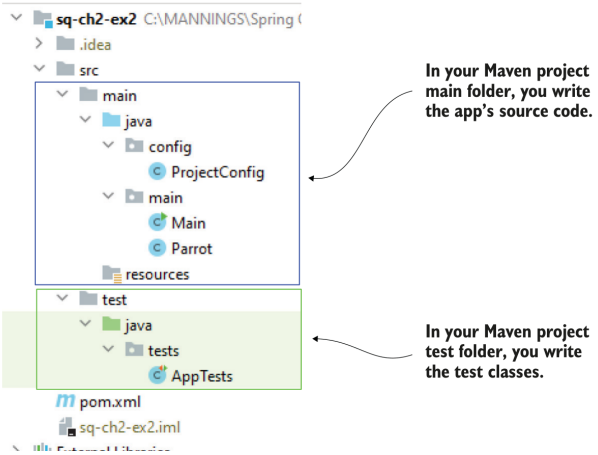
Trong mục này, chúng ta bàn luận về nguyên lý kiểm thử và thế nào là một bài kiểm thử được thiết kế đúng đắn. Bạn sẽ nhận ra một chân lý quan trọng trong kỹ nghệ phần mềm: **có một mối liên kết mật thiết giữa tính dễ kiểm thử (*testability*) và tính dễ bảo trì (*maintainability*)**.



Hình 15.1: Công cụ CI (như Jenkins hay TeamCity) tự động chạy bộ kiểm thử mỗi khi lập trình viên cập nhật mã nguồn lên Git. Nếu có bất kỳ bài test nào thất bại, hệ thống CI sẽ thông báo để đội ngũ kịp thời khoanh vùng và khắc phục lỗi.

Khi bạn thiết kế mã nguồn sao cho dễ viết kiểm thử, bạn đồng thời làm cho cấu trúc ứng dụng trở nên tách bạch, mô-đun hóa cao và cực kỳ dễ bảo trì.

Chúng ta viết kiểm thử để chứng minh logic của một phương thức cụ thể hoạt động đúng theo kỳ vọng. Khi kiểm thử một phương thức, thông thường bạn phải bao quát nhiều kịch bản khác nhau (*test scenarios*) tùy thuộc vào dữ liệu đầu vào. Với mỗi kịch bản, bạn viết một phương thức kiểm thử nằm trong một lớp kiểm thử (*test class*). Trong dự án Maven tiêu chuẩn, toàn bộ các lớp kiểm thử được đặt trong thư mục `src/test/java/`, hoàn toàn tách biệt với mã nguồn chính nằm ở `src/main/java/` (Hình 15.2).



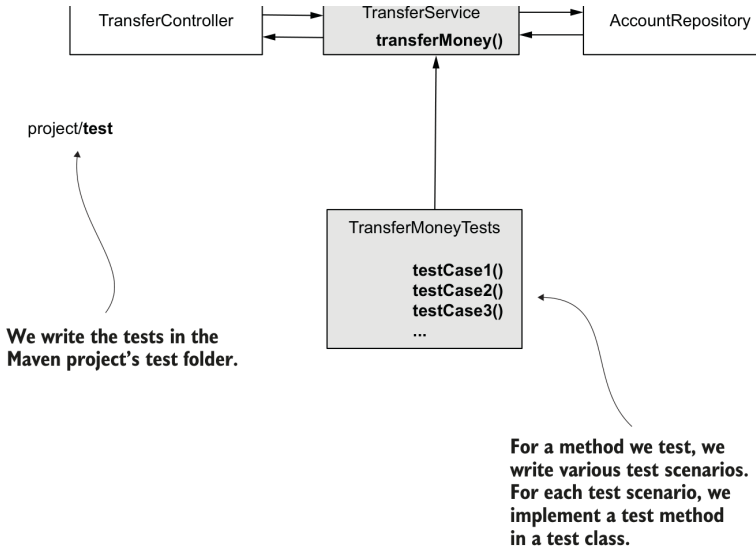
Hình 15.2: Trong dự án Maven, mã nguồn chính của ứng dụng nằm trong thư mục `src/main/`, còn toàn bộ các lớp kiểm thử được tổ chức trong thư mục `src/test/`.

Một lớp kiểm thử nên tập trung vào một lớp hoặc một phương thức nghiệp vụ nhất định. Ngay cả một đoạn logic ngắn cũng có thể sản sinh ra nhiều kịch bản thực thi. Hãy nhớ lại ca nghiệp vụ chuyển tiền từ Chương 13 và Chương 14 gồm bốn bước:

1. Tìm thông tin tài khoản người gửi trong cơ sở dữ liệu.
2. Tìm thông tin tài khoản người nhận trong cơ sở dữ liệu.
3. Tính toán số dư mới cho cả hai tài khoản sau khi chuyển.
4. Cập nhật số dư mới vào cơ sở dữ liệu.

Chỉ với bốn bước đơn giản trên, chúng ta đã có thể liệt kê hàng loạt kịch bản cần kiểm thử:

- Kịch bản 1: Không tìm thấy tài khoản người gửi trong hệ thống.
- Kịch bản 2: Không tìm thấy tài khoản người nhận trong hệ thống.
- Kịch bản 3: Tài khoản người gửi không đủ số dư để chuyển tiền.
- Kịch bản 4: Quá trình cập nhật cơ sở dữ liệu gặp sự cố.
- Kịch bản 5: Tất cả các bước diễn ra hoàn hảo và số dư được cập nhật chuẩn xác.



Hình 15.3: Với mỗi khối logic nghiệp vụ, bạn liệt kê các kịch bản kiểm thử tương ứng. Mỗi kịch bản được cụ thể hóa thành một phương thức test bên trong lớp kiểm thử nằm tại thư mục `test` của dự án Maven.

Hình 15.3 minh họa mối tương quan giữa lớp dịch vụ `TransferService` và lớp kiểm thử `TransferMoneyTests`. Nhận xét cốt lõi ở đây là: ngay cả một hàm nhỏ cũng tiềm ẩn nhiều kịch bản kiểm thử. Đây chính là lý do bạn **luôn phải giữ cho các phương thức trong chương trình thật ngắn gọn và đơn nhiệm**! Nếu bạn viết những hàm "khổng lồ" dài hàng trăm dòng, đảm đương quá nhiều trách nhiệm cùng lúc, việc liệt kê và kiểm thử toàn bộ các nhánh rẽ logic sẽ trở thành một cơn ác mộng.

15.2 Triển khai kiểm thử trong ứng dụng Spring

Trong mục này, chúng ta sẽ bắt tay triển khai hai kỹ thuật kiểm thử kinh điển trong môi trường Spring:

- **Kiểm thử đơn vị (*Unit tests*)**: Ngắn gọn, tốc độ thực thi tính bằng mili-giây, tập trung duy nhất vào một luồng logic bằng cách loại bỏ và cô lập triệt để mọi phụ thuộc bên ngoài.
- **Kiểm thử tích hợp Spring (*Spring integration tests*)**: Xác thực sự tương tác giữa logic nghiệp vụ với các cơ chế cốt lõi do Spring cung cấp (tiêm phụ thuộc, giao dịch, bảo mật).

15.2.1 Triển khai kiểm thử đơn vị (Unit tests)

Kiểm thử đơn vị là phương thức gọi một ca nghiệp vụ trong những điều kiện xác định để kiểm tra tính đúng đắn của hành vi. Điểm mấu chốt của unit test là: **khi một ca test thất bại, bạn biết chính xác dòng mã nào hoặc thành phần nào đang bị lỗi**.

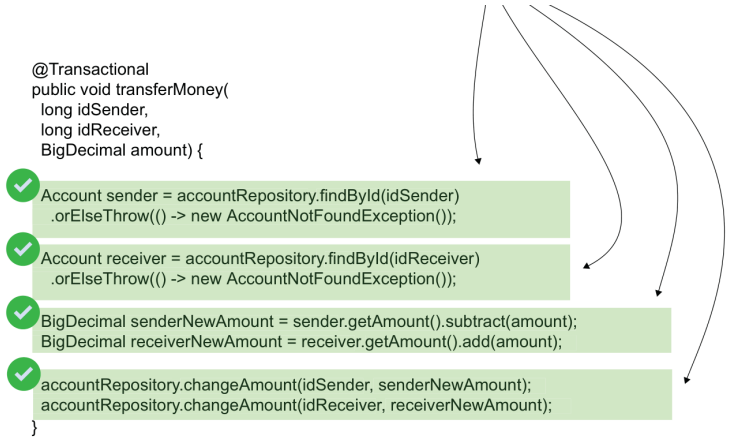
Hãy liên tưởng tới bảng đồng hồ điều khiển trên xe ô tô: Nếu xe không nổ máy được, có thể do hết xăng hoặc hỏng ắc-quy. Xe hơi là một hệ thống phức tạp, và bạn sẽ không biết nguyên nhân ở đâu nếu thiếu các đèn chỉ báo. Nếu đèn báo xăng sáng lên, bạn lập tức xác định được ngay sự cố! Kiểm thử đơn vị chính là những chiếc đèn chỉ báo đó: chúng cô lập và định vị chính xác trục trặc trong từng khối mã riêng rẽ.

Hãy xem lại mã nguồn phương thức chuyển tiền `transferMoney` trong `TransferService` từ dự án `sq-ch14-ex1` (Listing 15.1).

```
@Transactional
public void transferMoney(
    long idSender,
    long idReceiver,
    BigDecimal amount) {

  Account sender = accountRepository.findById(idSender)
    .orElseThrow(() -> new AccountNotFoundException());

  Account receiver = accountRepository.findById(idReceiver)
    .orElseThrow(() -> new AccountNotFoundException());

  BigDecimal senderNewAmount = sender.getAmount().subtract(amount);
  BigDecimal receiverNewAmount = receiver.getAmount().add(amount);

  accountRepository.changeAmount(idSender, senderNewAmount);
  accountRepository.changeAmount(idReceiver, receiverNewAmount);
}
```

Listing 15.1: Mã nguồn nghiệp vụ chuyển tiền cần kiểm thử

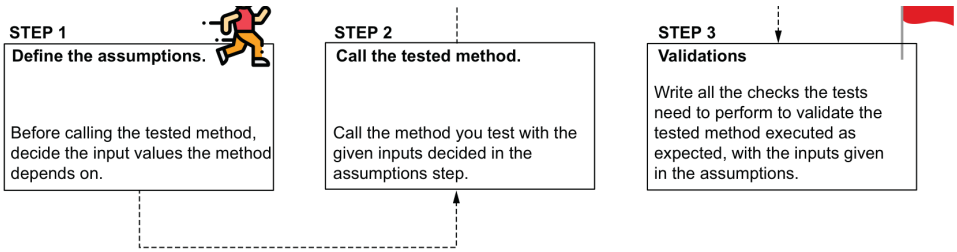
Thông thường, kịch bản đầu tiên và hiển nhiên nhất chúng ta viết test là luồng thuận lợi (*happy flow*) — luồng thực thi trong điều kiện lý tưởng, không gặp bất kỳ lỗi hay ngoại lệ nào (Hình 15.4).



Hình 15.4: Luồng thuận lợi (happy flow): Kịch bản thực thi hoàn hảo mà không gặp phải bất kỳ sự cố hay ngoại lệ nào.

Bất kỳ một bài kiểm thử đơn vị tiêu chuẩn nào cũng được cấu thành từ ba phần chính (Hình 15.5):

1. **Giả định / Thiết lập (Assumptions / Arrange / Given)**: Xác định các tham số đầu vào và điều khiển mọi phụ thuộc bên ngoài để tạo tiền đề cho kịch bản mong muốn.
2. **Thực thi (Call / Act / When)**: Kích hoạt gọi phương thức cần kiểm thử với các điều kiện đã thiết lập.
3. **Kiểm chứng (Validations / Assert / Then)**: Kiểm tra xem kết quả trả về hoặc các hành vi phát sinh có đúng với thiết kế hay không.



Hình 15.5: Ba bước cấu thành của một bài kiểm thử đơn vị: (1) Thiết lập giả định; (2) Kích hoạt thực thi; (3) Xác thực kết quả.

Trong bài toán chuyển tiền, các phụ thuộc gồm có các tham số truyền vào (`idSender`, `idReceiver`, `amount`) và đối tượng kho dữ liệu `AccountRepository` (Hình 15.6). Tham số thì ta dễ dàng truyền giá trị bất kỳ, nhưng làm sao kiểm soát được `AccountRepository`?

Để giải quyết vấn đề này, chúng ta sử dụng đối tượng giả lập (**Mock object**) — một đối tượng "đóng giả" mà ta có thể hoàn toàn điều khiển hành vi của nó (Hình 15.7). Thay vì kết nối tới cơ sở dữ liệu thật, phương thức test sẽ tiêm đối tượng mock này vào `TransferService`.

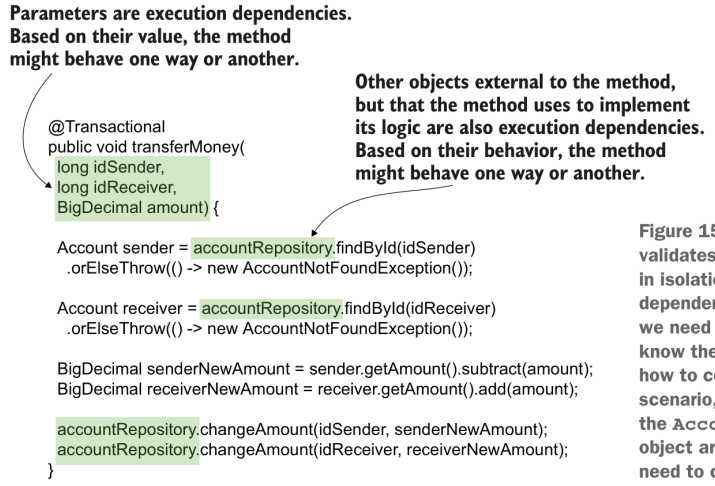
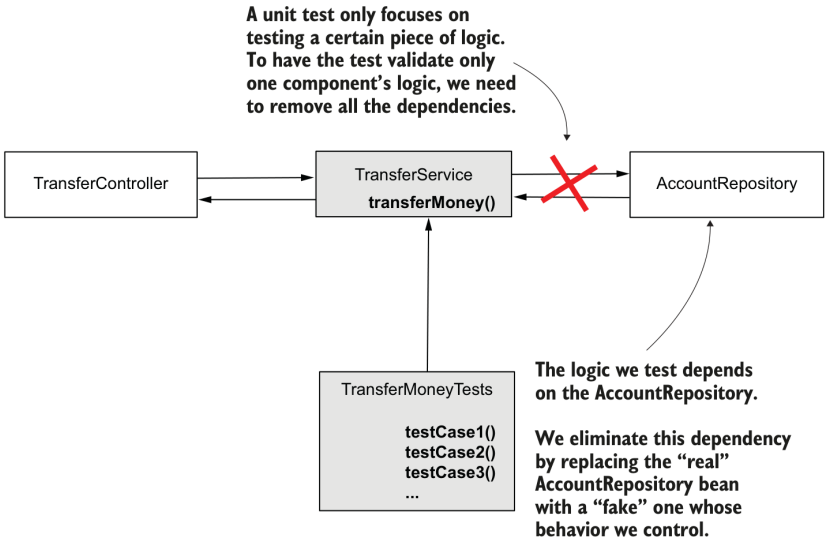


Figure 15.6 A unit test validates a use case logic in isolation from any dependency. To write the test, we need to make sure we know the dependencies and how to control them. For our scenario, the parameters and the `AccountRepository` object are dependencies we need to control for the test.

Hình 15.6: Các phụ thuộc của phương thức cần kiểm thử: bao gồm tham số truyền vào và đối tượng ngoài (`AccountRepository`). Để kiểm thử đơn vị, ta phải cô lập và làm chủ hoàn toàn các phụ thuộc này.



Hình 15.7: Thay thế `AccountRepository` thật bằng một đối tượng Mock do ta kiểm soát hoàn toàn, giúp bài kiểm thử chỉ tập trung xác thực thuật toán của `transferMoney`.

Chúng ta sử dụng thư viện kiểm thử **JUnit 5** kết hợp với **Mockito**. Listing 15.2 khởi tạo lớp kiểm thử `TransferServiceUnitTests` và tạo mock object thông qua phương thức tĩnh `Mockito.mock()`.

```
package com.example.service;

import com.example.repository.AccountRepository;
import org.junit.jupiter.api.Test;
import static org.mockito.Mockito.mock;

public class TransferServiceUnitTests {

  @Test
  public void moneyTransferHappyFlow() {
    AccountRepository accountRepository =
        mock(AccountRepository.class);

    TransferService transferService =
        new TransferService(accountRepository);
  }
}
```

Listing 15.2: Tạo đối tượng kiểm thử và giả lập phụ thuộc với mock()

Để quy định hành vi cho mock object, ta sử cú pháp `given(...).willReturn(...)` của Mockito (Listing 15.3). Câu lệnh này có nghĩa: *"Nếu ai đó gọi hàm findById() với ID của người gửi, hãy trả về đối tượng sender chỉ định"*.

```
package com.example.service;

import com.example.model.Account;
import com.example.repository.AccountRepository;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import java.math.BigDecimal;
import java.util.Optional;
import static org.mockito.BDDMockito.given;
import static org.mockito.Mockito.mock;

public class TransferServiceUnitTests {

  @Test
  @DisplayName("Kiem tra chuyen tien thanh cong giua 2 tai khoan neu
    khong co loi")
```

```
  public void moneyTransferHappyFlow() {
    AccountRepository accountRepository = mock(AccountRepository.class);
    TransferService transferService = new TransferService(
   accountRepository);

    Account sender = new Account();
    sender.setId(1);
    sender.setAmount(new BigDecimal(1000));

    Account destination = new Account();
    destination.setId(2);
    destination.setAmount(new BigDecimal(1000));

    given(accountRepository.findById(sender.getId()))
        .willReturn(Optional.of(sender));

    given(accountRepository.findById(destination.getId()))
        .willReturn(Optional.of(destination));

    transferService.transferMoney(
        sender.getId(),
        destination.getId(),
        new BigDecimal(100)
    );
  }
}
```

Listing 15.3: Thiết lập giả định hành vi cho Mock object với given()

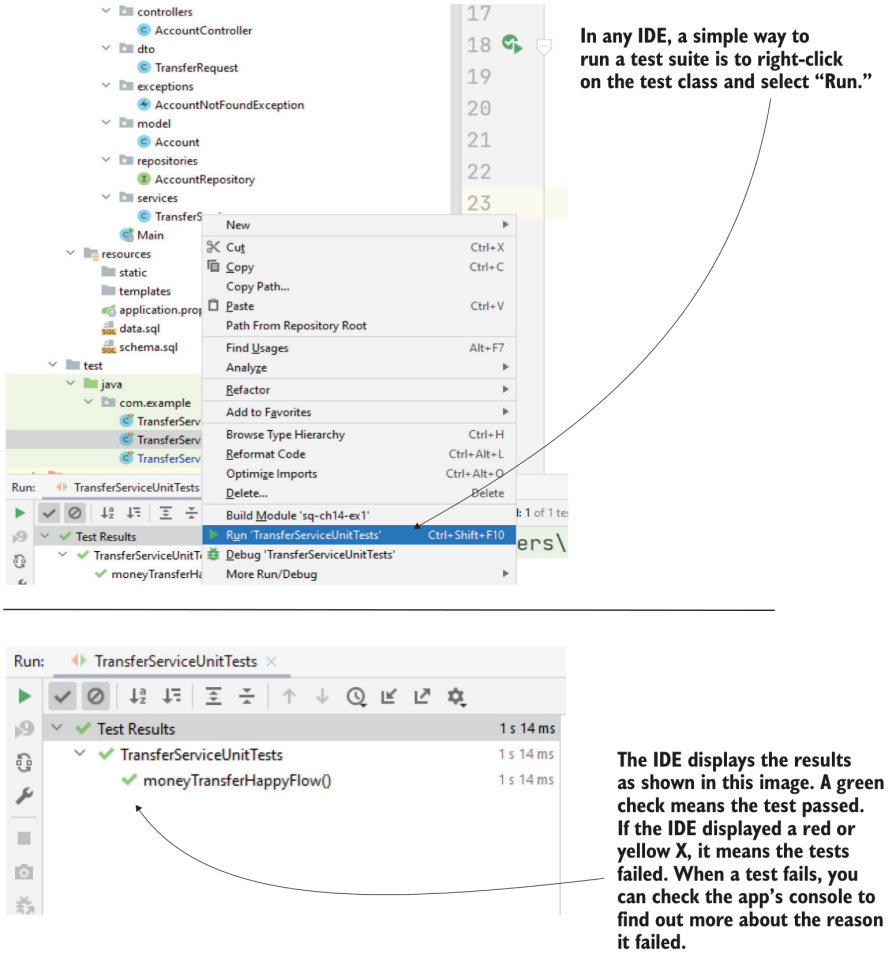
Bước cuối cùng trong bài test là kiểm chứng xem phương thức đã thực sự gọi xuống repository để cập nhật số dư mới hay chưa. Ta dùng phương thức `verify()` của Mockito để xác nhận điều này (Listing 15.4).

```
    // ... cac dong truoc do giu nguyen ...
    transferService.transferMoney(
        sender.getId(),
        destination.getId(),
        new BigDecimal(100)
    );

    verify(accountRepository)
        .changeAmount(1, new BigDecimal(900));

```

```
    verify(accountRepository)
        .changeAmount(2, new BigDecimal(1100));
  }
}
```

Listing 15.4: Hoàn thiện bài kiểm thử luồng thuận lợi với verify()

Khi chạy kiểm thử trong IDE, một biểu tượng tích xanh (*green checkmark*) sẽ hiện ra báo hiệu bài test đã vượt qua thành công rực rỡ (Hình 15.8).



Hình 15.8: Chạy kiểm thử trong môi trường phát triển (IDE). Màu xanh biểu thị ca kiểm thử thành công hoàn toàn; màu đỏ hoặc vàng cảnh báo ca test thất bại.

15.2.2 Tối ưu hóa mã kiểm thử bằng các chú thích Mockito

Thay vì phải tự tay gọi hàm `mock()` và truyền tham số thủ công vào constructor trong từng phương thức kiểm thử, Mockito cung cấp các chú thích thanh lịch:

- `@Mock`: Tự động tạo đối tượng giả lập cho trường được đánh dấu.

- `@InjectMocks`: Tạo đối tượng cần kiểm thử và tự động tiêm tất cả các mock đã tạo vào đối tượng đó.
- `@ExtendWith(MockitoExtension.class)`: Gắn trên đầu lớp kiểm thử để kích hoạt cơ chế xử lý các chú thích của Mockito trong JUnit 5.

Listing 15.5 minh họa phong cách viết bài test chuyên nghiệp này. Hình 15.9 tổng hợp cấu trúc ba bước rõ nét của bài kiểm thử.

```
package com.example.service;

import com.example.model.Account;
import com.example.repository.AccountRepository;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import java.math.BigDecimal;
import java.util.Optional;
import static org.mockito.BDDMockito.given;
import static org.mockito.Mockito.verify;

@ExtendWith(MockitoExtension.class)
public class TransferServiceWithAnnotationsUnitTests {

  @Mock
  private AccountRepository accountRepository;

  @InjectMocks
  private TransferService transferService;

  @Test
  public void moneyTransferHappyFlow() {
    Account sender = new Account();
    sender.setId(1);
    sender.setAmount(new BigDecimal(1000));

    Account destination = new Account();
    destination.setId(2);
    destination.setAmount(new BigDecimal(1000));

    given(accountRepository.findById(sender.getId()))
        .willReturn(Optional.of(sender));
```

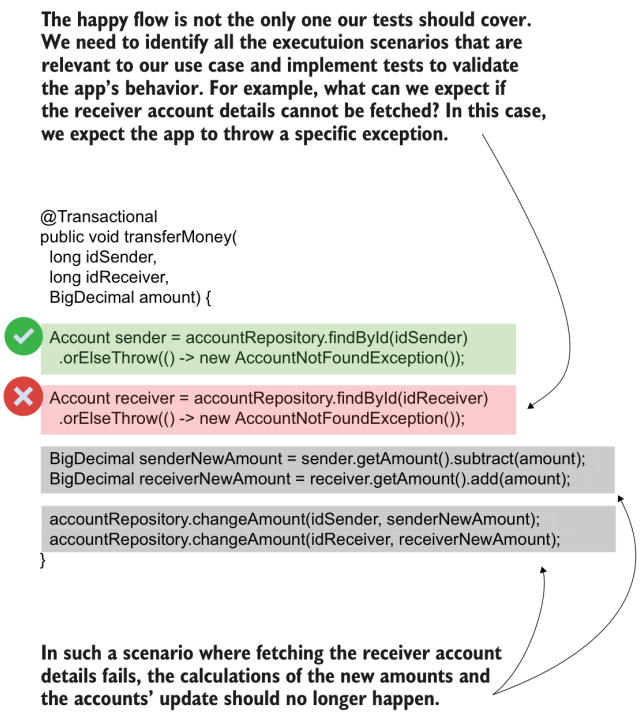
```
36
37      given(accountRepository.findById(destination.getId()))
38          .willReturn(Optional.of(destination));
39
40      transferService.transferMoney(1, 2, new BigDecimal(100));
41
42      verify(accountRepository)
43          .changeAmount(1, new BigDecimal(900));
44
45      verify(accountRepository)
46          .changeAmount(2, new BigDecimal(1100));
47    }
48  }
```

Listing 15.5: Sử dụng chú thích Mockito @Mock và @InjectMocks

Hình 15.9: Ba giai đoạn tường minh của bài test: (1) Thiết lập giả định; (2) Kích hoạt hàm kiểm thử; (3) Xác thực kết quả.

15.2.3 Kiểm thử luồng phát sinh ngoại lệ (Exception Flows)

Bên cạnh luồng thuận lợi, bạn bắt buộc phải kiểm thử các tình huống phát sinh lỗi (*exception flow*) — ví dụ: không tìm thấy tài khoản người nhận trong cơ sở dữ liệu (Hình 15.10). Trong tình huống này, ta kỳ vọng ứng dụng phải ném ngoại lệ `AccountNotFoundException` và tuyệt đối không được phép gọi lệnh trừ hay nạp tiền.



Hình 15.10: Luồng ngoại lệ: Nếu không tìm thấy người nhận, phương thức phải ném ngoại lệ và dừng ngay lập tức, không được phép gọi lệnh cập nhật số dư.

Listing 15.6 minh họa cách dùng phương thức `assertThrows()` của JUnit 5 và `verify(..., never())` của Mockito để kiểm chứng luồng này.

```
@Test
public void moneyTransferDestinationAccountNotFoundFlow() {
  Account sender = new Account();
  sender.setId(1);
  sender.setAmount(new BigDecimal(1000));

  given(accountRepository.findById(1L))
      .willReturn(Optional.of(sender));

  given(accountRepository.findById(2L))
      .willReturn(Optional.empty());

  assertThrows(
      AccountNotFoundException.class,
      () -> transferService.transferMoney(1, 2, new BigDecimal(100))
  );

  verify(accountRepository, never())
      .changeAmount(anyLong(), any());
```

```
  }
```

Listing 15.6: Kiểm thử luồng ngoại lệ với assertThrows và verify(..., never())

15.2.4 Kiểm thử giá trị trả về của phương thức Controller

Một trường hợp phổ biến khác là kiểm tra giá trị trả về của một hàm điều khiển (Controller action) mà chúng ta từng xây dựng ở Chương 9 (Listing 15.7).

```
@PostMapping("/")
public String loginPost(
    @RequestParam String username,
    @RequestParam String password,
    Model model) {

  loginProcessor.setUsername(username);
  loginProcessor.setPassword(password);

  boolean loggedIn = loginProcessor.login();

  if (loggedIn) {
    model.addAttribute("message", "You are now logged in.");
  } else {
    model.addAttribute("message", "Login failed!");
  }
  return "login.html";
}
```

Listing 15.7: Phương thức xử lý đăng nhập trong LoginController

Để kiểm thử đơn vị cho phương thức này, ta giả lập đối tượng `LoginProcessor` và `Model`. Ta dùng `assertEquals()` để kiểm tra chuỗi trả về có đúng là `"login.html"` hay không (Listing 15.8 và Listing 15.9).

```
@ExtendWith(MockitoExtension.class)
class LoginControllerUnitTests {

  @Mock
  private Model model;

  @Mock
  private LoginProcessor loginProcessor;

```

```
  @InjectMocks
  private LoginController loginController;

  @Test
  public void loginPostLoginSucceedsTest() {
    given(loginProcessor.login())
        .willReturn(true);

    String result = loginController.loginPost("username", "password",
    model);

    assertEquals("login.html", result);
    verify(model).addAttribute("message", "You are now logged in.");
  }
}
```

Listing 15.8: Kiểm thử giá trị trả về khi đăng nhập thành công

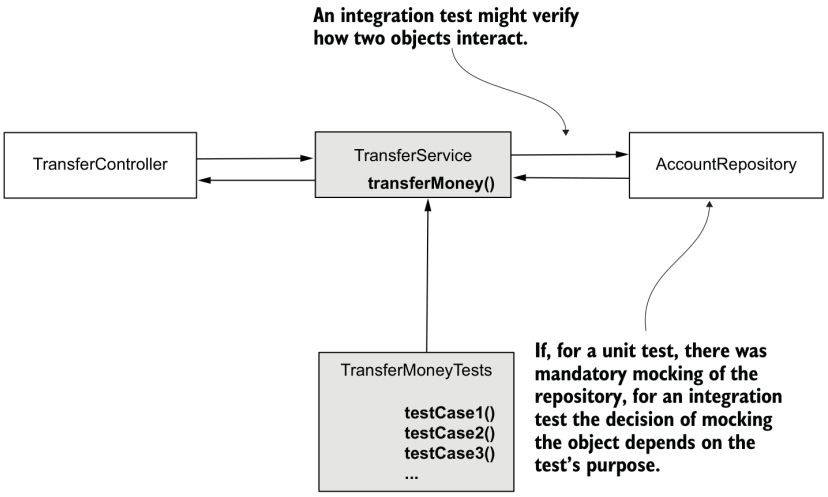
```
  @Test
  public void loginPostLoginFailsTest() {
    given(loginProcessor.login())
        .willReturn(false);

    String result = loginController.loginPost("username", "password",
    model);

    assertEquals("login.html", result);
    verify(model).addAttribute("message", "Login failed!");
  }
```

Listing 15.9: Kiểm thử trường hợp đăng nhập thất bại

15.2.5 Triển khai kiểm thử tích hợp (Spring Integration Tests)

Trong khi kiểm thử đơn vị chỉ quan tâm đến một khối logic cô lập, **kiểm thử tích hợp** tập trung kiểm tra xem hai hoặc nhiều thành phần có kết nối và phối hợp với nhau đúng cách hay không. Một chiếc xe dù bình xăng đầy ắp nhưng đường ống dẫn xăng đến động cơ bị tắc thì xe vẫn không thể nổ máy. Kiểm thử tích hợp giúp chúng ta phòng ngừa những thảm họa phát sinh khi từng linh kiện chạy tốt riêng lẻ nhưng khi ghép nối lại thì không thể giao tiếp với nhau.

Trong ứng dụng Spring, kiểm thử tích hợp cho phép Spring thực sự khởi tạo Application



Hình 15.11: Trong unit test, mọi phụ thuộc đều bắt buộc phải bị làm giả (mock). Trong kiểm thử tích hợp, tùy theo mục đích kiểm tra tương tác, bạn có thể cho phép gọi trực tiếp tới đối tượng thực tế.

Context và quản lý vòng đời bean. Spring Boot hỗ trợ kiểm thử tích hợp cực kỳ tiện lợi thông qua hai chú thích then chốt:

- `@SpringBootTest`: Khởi động toàn bộ ngữ cảnh Spring Boot để chạy test y hệt môi trường thực tế.
- `@MockBean`: Tạo một đối tượng mock do Mockito quản lý nhưng **tự động đưa mock này vào làm bean trong Application Context**, thay thế cho bean thật cùng loại!

Listing 15.10 trình bày cách xây dựng một bài kiểm thử tích hợp Spring.

```
package com.example.service;

import com.example.model.Account;
import com.example.repository.AccountRepository;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import java.math.BigDecimal;
import java.util.Optional;
import static org.mockito.Mockito.verify;
import static org.mockito.Mockito.when;

@SpringBootTest
class TransferServiceSpringIntegrationTests {

  @MockBean
```

```
  private AccountRepository accountRepository;

  @Autowired
  private TransferService transferService;

  @Test
  void transferServiceTransferAmountTest() {
    Account sender = new Account();
    sender.setId(1);
    sender.setAmount(new BigDecimal(1000));

    Account receiver = new Account();
    receiver.setId(2);
    receiver.setAmount(new BigDecimal(1000));

    when(accountRepository.findById(1L))
        .thenReturn(Optional.of(sender));

    when(accountRepository.findById(2L))
        .thenReturn(Optional.of(receiver));

    transferService.transferMoney(1, 2, new BigDecimal(100));

    verify(accountRepository)
        .changeAmount(1, new BigDecimal(900));

    verify(accountRepository)
        .changeAmount(2, new BigDecimal(1100));
  }
}
```

Listing 15.10: Triển khai kiểm thử tích hợp với @SpringBootTest và @MockBean

Chiến lược kết hợp Unit Test và Integration Test

Trong các dự án thực tế, kiểm thử tích hợp khởi động Spring Context nên thường tiêu tốn thời gian hơn nhiều so với kiểm thử đơn vị. Vì vậy, nguyên tắc vàng là: **Dùng Unit Test để kiểm tra phần lớn các kịch bản logic nghiệp vụ ngóc ngách, và dùng Integration Test có chọn lọc để xác nhận sự tích hợp giữa các thành phần với nhau và với framework.**

15.3 Tóm tắt nội dung chương (Summary)

- **Kiểm thử** là một đoạn mã logic được viết ra để xác minh tính đúng đắn của hành vi trong ứng dụng. Kiểm thử tự động giúp đảm bảo những thay đổi trong tương lai không làm gãy đổ các tính năng sẵn có, đồng thời đóng vai trò như một tài liệu đặc tả sống động cho dự án.
- Kiểm thử được chia thành hai nhánh lớn:
  - **Kiểm thử đơn vị (Unit tests)**: Chỉ tập trung xác thực một khối logic độc lập, tốc độ chạy siêu nhanh, loại trừ mọi phụ thuộc bên ngoài và chỉ điểm chính xác vị trí phát sinh lỗi.
  - **Kiểm thử tích hợp (Integration tests)**: Tập trung kiểm tra sự phối hợp tương tác giữa hai hoặc nhiều đối tượng, đảm bảo các thành phần kết nối đồng bộ khi ráp lại với nhau.
- Trong kiểm thử, chúng ta sử dụng các đối tượng giả lập (**Mock objects**) để thay thế các thành phần phụ thuộc mà ta không muốn kiểm tra, cho phép ta toàn quyền kiểm soát dữ liệu đầu vào và đầu ra của chúng.
- Cấu trúc ba giai đoạn kinh điển của một bài kiểm thử bao gồm:
  - **Giả định (Assumptions / Arrange / Given)**: Thiết lập giá trị tham số và định nghĩa hành vi giả lập cho các đối tượng mock.
  - **Kích hoạt (Call / Act / When)**: Thực thi phương thức cần kiểm thử.
  - **Kiểm chứng (Validations / Assert / Then)**: Kiểm tra tính đúng đắn của giá trị trả về và xác nhận các lời gọi phương thức mong đợi đã được kích hoạt.
- Spring Boot cung cấp chú thích `@SpringBootTest` để nạp context đầy đủ cho kiểm thử tích hợp và chú thích `@MockBean` để đăng ký đối tượng mock trực tiếp vào Application Context của Spring.

Phụ lục A

Các phương pháp tiếp cận kiến trúc (Architectural approaches)

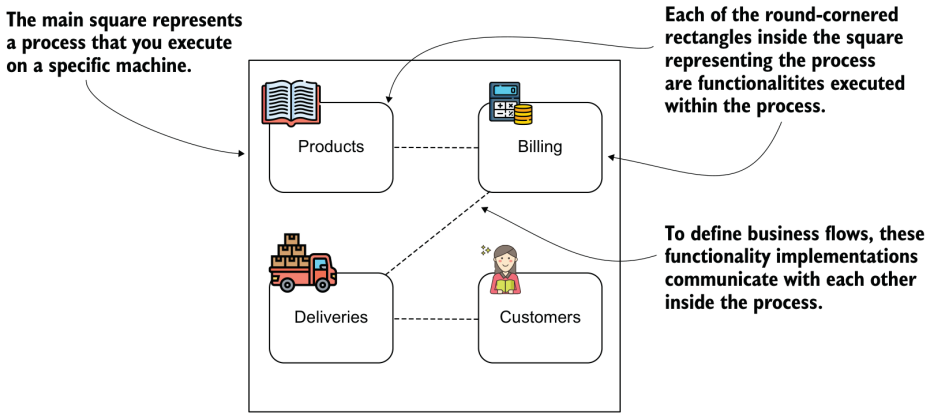
Trong phụ lục này, chúng ta thảo luận về một số khái niệm kiến trúc phần mềm cốt lõi mà bạn sẽ thường xuyên bắt gặp. Để thấu hiểu tường tận mọi điều được trình bày trong cuốn sách này, bạn cần nắm vững bức tranh tổng quan ở cấp độ cao về các phong cách kiến trúc. Tôi sẽ dẫn dắt bạn qua các mô hình từ **Kiến trúc khối thống nhất (Monolith)**, **Kiến trúc hướng dịch vụ (SOA)**, cho đến **Kiến trúc vi dịch vụ (Microservices)** và **Điện toán không máy chủ (Serverless)**.

Những chủ đề này vốn rất sâu rộng và có hàng chục đầu sách chuyên khảo riêng. Mục tiêu của phụ lục này không phải biến bạn thành chuyên gia kiến trúc ngay lập tức, mà giúp bạn thấu hiểu *vì sao* chúng ta lại ứng dụng Spring theo những cách thức cụ thể xuyên suốt cuốn sách.

A.1 Phương pháp tiếp cận khối thống nhất (The monolithic approach)

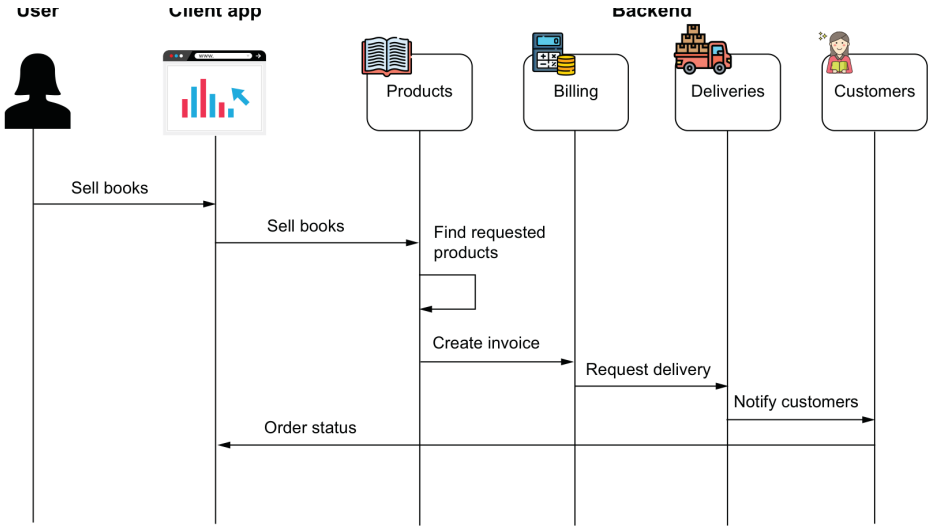
Khi các kỹ sư mô tả một ứng dụng là "khối thống nhất" (*monolithic* hay *monolith*), điều đó có nghĩa là toàn bộ hệ thống được đóng gói thành một đơn vị triển khai duy nhất chạy trong một tiến trình (*process*) hệ điều hành. Thành phần duy nhất này tự thân gánh vác toàn bộ mọi tính năng nghiệp vụ.

Ví dụ: hãy xem xét hệ thống quản lý một hiệu sách. Người dùng cần quản lý sản phẩm sách (*Products*), hóa đơn bán hàng (*Billing*), giao nhận vận chuyển (*Deliveries*), và thông tin khách hàng (*Customers*). Trong Hình A.1, toàn bộ các tính năng này cùng tồn tại và thực thi bên trong duy nhất một tiến trình.



Hình A.1: Ứng dụng khối thống nhất (Monolith). Toàn bộ các chức năng nghiệp vụ được triển khai và vận hành gói gọn trong duy nhất một tiến trình hệ thống.

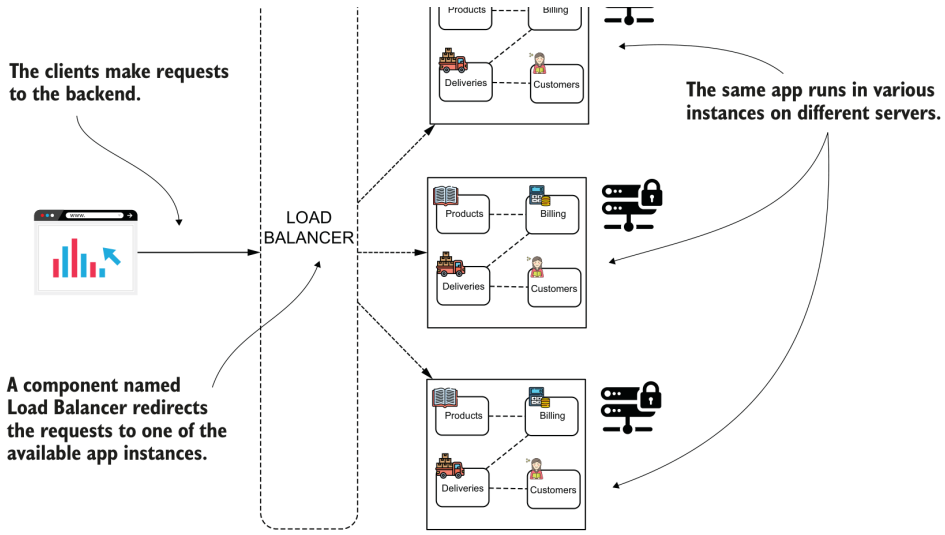
Một luồng nghiệp vụ (*business flow*) là chuỗi hành động mà người dùng mong đợi hệ thống thực hiện. Khi chủ tiệm bán sách, luồng xử lý diễn ra như sau: chức năng Products kiểm tra và giữ hàng trong kho; chức năng Billing xuất hóa đơn thanh toán; chức năng Deliveries lên lịch giao hàng và chức năng Customers thông báo tới khách (Hình A.2).



Hình A.2: Ví dụ về luồng nghiệp vụ bán sách trong ứng dụng Monolith. Các mô-đun chức năng trao đổi trực tiếp với nhau thông qua các lời gọi phương thức nội bộ trong cùng tiến trình.

Ban đầu, hầu hết mọi phần mềm đều được kiến trúc theo dạng monolith, và mô hình này hoạt động cực kỳ hoàn hảo. Tuy nhiên, sự bùng nổ của Internet và số lượng người dùng tăng vọt từ hàng nghìn lên hàng triệu đã đặt ra bài toán mở rộng tài nguyên (*scalability*).

Phương thức mở rộng tự nhiên nhất của monolith là **mở rộng theo chiều ngang (Horizontal scaling)**: chạy đồng thời nhiều bản sao (*instances*) của cùng ứng dụng trên nhiều máy chủ khác nhau, kết hợp cùng một bộ cân bằng tải (*Load Balancer*) để điều phối các yêu cầu (Hình A.3).



Hình A.3: Mở rộng theo chiều ngang (Horizontal scaling). Nhân bản cùng một ứng dụng monolith lên nhiều máy chủ giúp phân tải và phục vụ số lượng yêu cầu đồng thời lớn hơn.

Minh oan cho Kiến trúc Monolith

Nhiều lập trình viên ngày nay thường gắn định kiến tiêu cực cho monolith, than phiền rằng ứng dụng monolith rất khó bảo trì. Nhưng sự thật là: **nguyên nhân khiến phần mềm khó bảo trì bắt nguồn từ mã nguồn bừa bãi, thiếu tách bạch trách nhiệm, chứ không phải do bản thân kiến trúc monolith**! Một ứng dụng monolith được thiết kế theo mô-đun hóa sạch sẽ (*modular monolith*) vẫn là lựa chọn hàng đầu cho vô số bài toán thực tế.

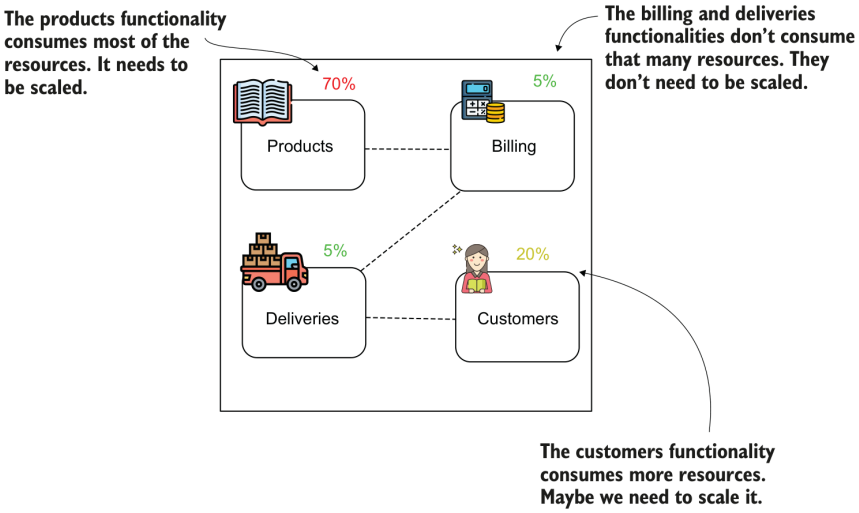
A.2 Kiến trúc hướng dịch vụ (Service-Oriented Architecture - SOA)

Hạn chế lớn nhất của monolith nằm ở chỗ: **các chức năng không tiêu thụ tài nguyên đồng đều nhau**. Trong thực tế, tính năng xem sản phẩm có thể chiếm 70% lượng tải, trong khi thanh toán và giao hàng chỉ chiếm dưới 5% (Hình A.4).

Với monolith, bạn buộc phải nhân bản toàn bộ ứng dụng (kéo theo cả những phần không cần thiết). Để quản lý tài nguyên tối ưu, ta tách monolith thành nhiều dịch vụ độc lập — chuyển dịch sang **Kiến trúc hướng dịch vụ (SOA)**. Mỗi chức năng trở thành một tiến trình riêng biệt và ta có thể mở rộng độc lập từng dịch vụ khi cần (Hình A.5).

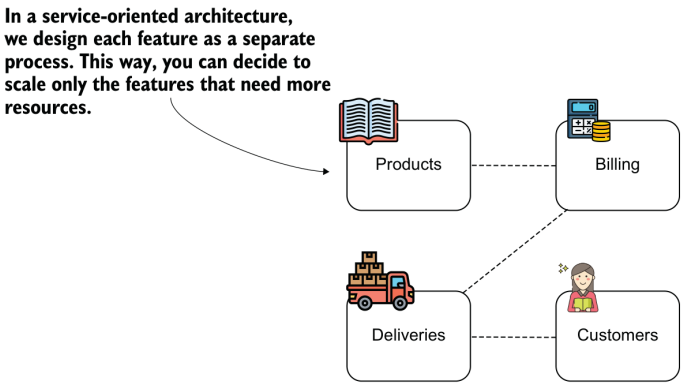
SOA cũng giúp phân chia công việc cho các đội ngũ phát triển độc lập dễ dàng hơn mà không dẫm chân lên nhau (Hình A.6).

Tuy nhiên, SOA đem lại những thách thức kỹ thuật lớn:



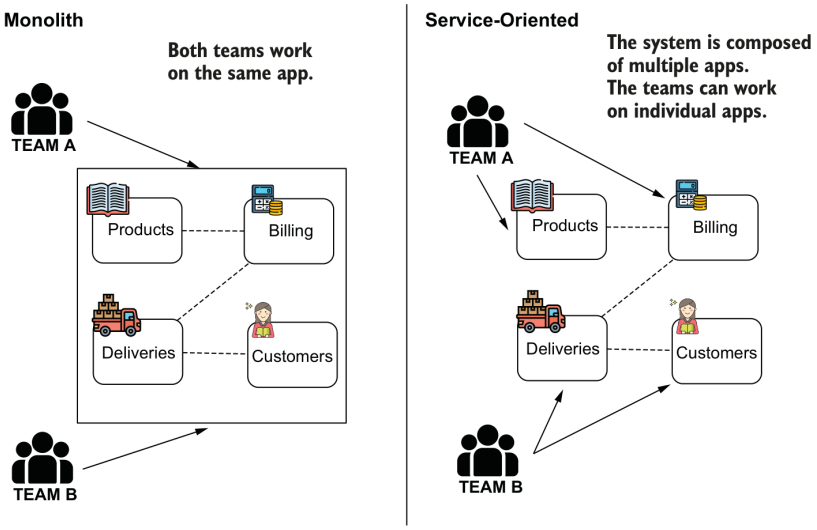
Hình A.4: Các chức năng trong ứng dụng tiêu thụ tài nguyên bất đối xứng. Tính năng Products chịu tải lớn nhất và cần mở rộng, trong khi Billing và Deliveries tiêu tốn rất ít tài nguyên.

the feature that needs more resources (figure A.5).



Hình A.5: Trong kiến trúc SOA, mỗi chức năng là một tiến trình độc lập. Ta có thể chủ động cấp phát tài nguyên và mở rộng riêng cho dịch vụ Products mà không lãng phí tài nguyên cho các dịch vụ khác.

easier to manage the teams working on the system because you can offer different teams specific services on which they work, instead of having multiple teams working on the same app (figure A.6).



Hình A.6: Với monolith, mọi đội ngũ cùng chỉnh sửa một kho mã duy nhất, đòi hỏi sự phối hợp phức tạp. Với SOA, mỗi đội có thể toàn quyền làm chủ một dịch vụ độc lập.

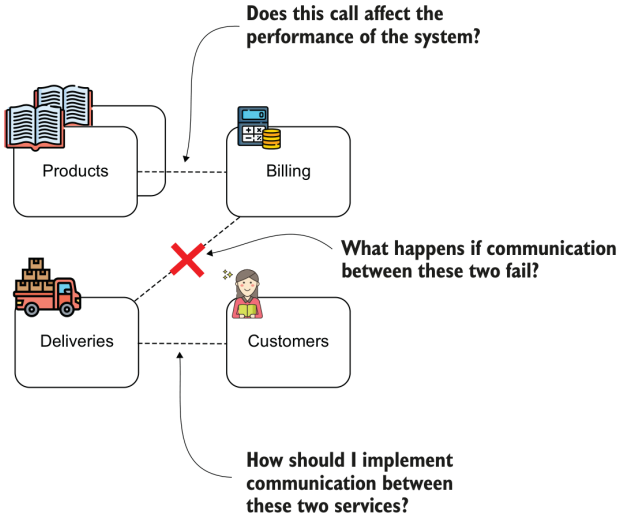
1. **Giao tiếp qua mạng**: Các dịch vụ phải gọi nhau qua mạng (REST, gRPC, tin nhắn bất đồng bộ qua Kafka/RabbitMQ). Mạng máy tính vốn không bao giờ tin cậy tuyệt đối, đòi hỏi phải xử lý lỗi mạng, độ trễ và cơ chế ngắt mạch (*circuit breaker*) (Hình A.7).
2. **Bảo mật**: Dữ liệu truyền giữa các dịch vụ cần được mã hóa và xác thực chặt chẽ để phòng chống nghe lén hoặc giả mạo (Hình A.8).
3. **Lưu trữ và tính nhất quán dữ liệu**: Dùng chung một cơ sở dữ liệu (*shared database*) hay mỗi dịch vụ sở hữu cơ sở dữ liệu riêng (*database-per-service*)? (Hình A.9 và Hình A.10). Việc duy trì giao dịch phân tán giữa nhiều cơ sở dữ liệu là bài toán vô cùng nan giải.
4. **Độ phức tạp khi triển khai và vận hành**: Số lượng thành phần tăng đột biến đòi hỏi tự động hóa CI/CD và giám sát phức tạp.

A.3 Từ Microservices đến Serverless

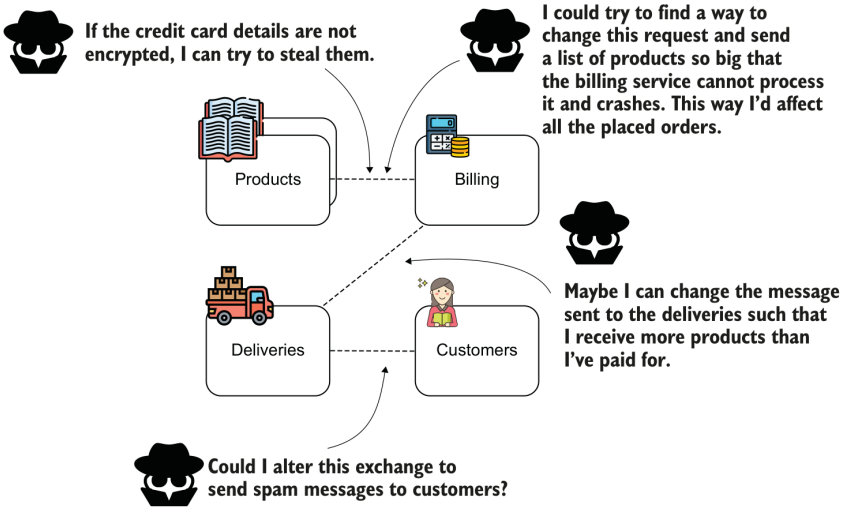
Microservices là một biến thể đặc thù và tinh gọn hóa cao độ của SOA. Một vi dịch vụ thường tuân theo nguyên lý đơn trách nhiệm (*Single Responsibility Principle*), sở hữu cơ sở dữ liệu riêng biệt và được đóng gói trong các vùng chứa (*containers* như Docker) để triển khai trên đám mây (Kubernetes).

Tiến trình thu nhỏ dịch vụ đạt đến đỉnh cao với mô hình **Điện toán không máy chủ (Serverless Functions)**: bạn chỉ viết vài dòng mã xử lý logic nghiệp vụ và đẩy lên đám mây (AWS Lambda, Google Cloud Functions). Khi có sự kiện (yêu cầu HTTP, hàng đợi tin nhắn, bộ

tion among the services (figure A.7). You could use REST services, GraphQL, SOAP, gRPC, JMS message brokers, Kafka, and so on. Which is the best approach? Of course, in any situation one or more of these approaches is fine. You'll find long debates and discussions in many books on how to choose the right fit for typical scenarios.



Hình A.7: Giao tiếp phân tán giữa các dịch vụ làm gia tăng đáng kể độ phức tạp kiến trúc và tiềm ẩn nguy cơ đứt gãy kết nối mạng.



Hình A.8: Các bề mặt tấn công bảo mật gia tăng khi dữ liệu luân chuyển qua hạ tầng mạng giữa các tiến trình phân tán.

In most cases, an app needs a way to store data. Databases are a popular way persistence is implemented in apps. With a monolithic approach, an app had one database to store the data, as presented in figure A.9. We called this a *three-tier architecture* because it consists of three tiers: the client, the backend, and the database used for persistence.

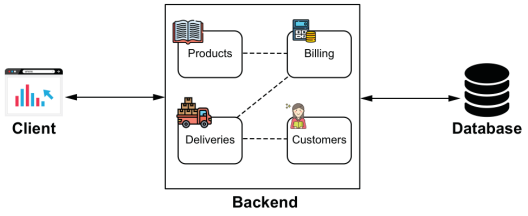
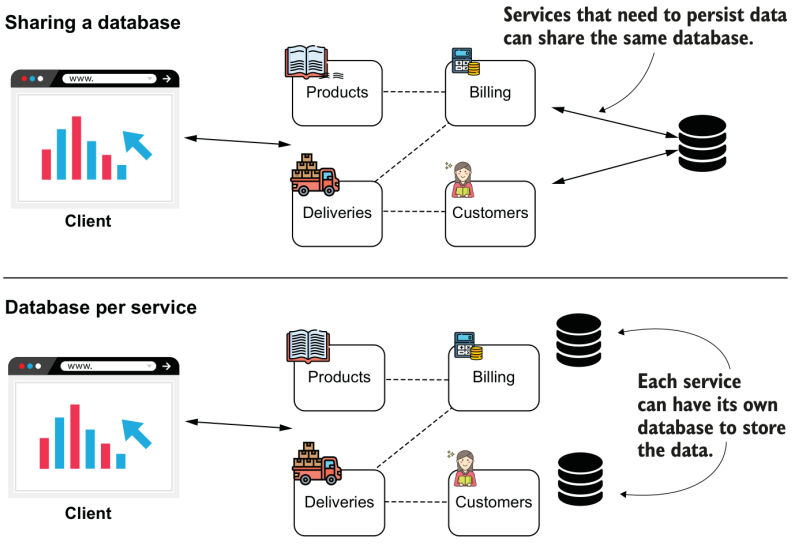


Figure A.9 With a monolithic approach, you only have one application and usually one database. The system is simple and can be easily visualized and understood.

Hình A.9: Kiến trúc monolith 3 tầng truyền thống: Khách hàng - Backend - Cơ sở dữ liệu duy nhất. Cấu trúc đơn giản, trực quan và dễ duy trì tính toàn vẹn giao dịch.



Hình A.10: Hai mô hình lưu trữ trong kiến trúc hướng dịch vụ: Chia sẻ chung một cơ sở dữ liệu (*Shared database*) hoặc mỗi dịch vụ có cơ sở dữ liệu riêng (*Database per service*).

định thời), hàm sẽ tự động được kích hoạt và chạy tức thì mà lập trình viên không cần quản lý bất kỳ máy chủ nào bên dưới.

A.4 Tài liệu đọc thêm (Further reading)

- *Microservices in Action* (Morgan Bruce & Paulo A. Pereira, Manning 2018).
- *Microservices Patterns* (Chris Richardson, Manning 2018).
- *Spring Microservices in Action* (John Carnell & Illary Huaylupo Sánchez, Manning 2020).
- *Microservices Security in Action* (Prabath Siriwardena & Nuwan Dias, Manning 2020).
- *Monolith to Microservices* (Sam Newman, O'Reilly Media 2020).

Phụ lục B

Cấu hình Spring Context bằng XML (Using XML for the context configuration)

Nhiều năm về trước, khi tôi bắt đầu học và làm việc với Spring, các kỹ sư chủ yếu sử dụng tệp XML để cấu hình Application Context và toàn bộ hệ sinh thái Spring nói chung. Ngày nay, bạn sẽ chỉ còn bắt gặp cấu hình XML trong các hệ thống phần mềm cũ (*legacy systems*) được duy trì từ quá khứ. Cộng đồng Spring đã rời bỏ cấu hình XML từ nhiều năm trước và thay thế hoàn toàn bằng các chú thích (*annotations*) bởi sự tường minh và dễ đọc của chúng. Dù XML vẫn có những điểm mạnh riêng, việc sử dụng chú thích trong mã Java giúp cấu trúc mã nguồn gọn gàng, tăng cường mạnh mẽ tính dễ bảo trì. Vì lý do đó, cuốn sách này tập trung hoàn toàn vào cấu hình dựa trên chú thích hiện đại.

Lời khuyên chân thành của tôi dành cho bạn: **chỉ học cấu hình XML khi bạn bắt buộc phải bảo trì một dự án kế thừa lâu đời**. Nền tảng kiến thức về Spring bean, vòng đời và phụ thuộc mà bạn tích lũy từ các chương trước hoàn toàn áp dụng được cho cấu hình XML — điểm khác biệt duy nhất chỉ là cú pháp khai báo.

Để bạn hình dung cấu hình XML trông như thế nào, chúng ta cùng xem ví dụ thêm một bean vào context bằng phong cách cổ điển này (dự án mẫu `sq-app2-ex1`).

Ta tạo tệp `config.xml` đặt trong thư mục tài nguyên `src/main/resources/`:

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
  http://www.springframework.org/schema/beans/spring-beans.xsd">

  <bean id="parrot1" class="main.Parrot">
```

```
    <property name="name" value="Kiki" />
  </bean>

</beans>
```

Listing B.1: Tệp cấu hình config.xml

Thẻ gốc `<beans>` bao bọc toàn bộ định nghĩa bean. Thẻ `<bean>` khai báo một bean kiểu `main.Parrot` với định danh (*id*) là `"parrot1"`. Thẻ con `<property>` thiết lập giá trị cho thuộc tính `name` là `"Kiki"`.

Để nạp cấu hình XML này, trong hàm `main()`, chúng ta sử dụng lớp `ClassPathXmlApplicationContext`:

```
package main;

import org.springframework.context.support.
    ClassPathXmlApplicationContext;

public class Main {
  public static void main(String[] args) {
    var context = new ClassPathXmlApplicationContext("config.xml");

    Parrot p = context.getBean(Parrot.class);
    System.out.println(p.getName());
  }
}
```

Listing B.2: Khởi tạo ApplicationContext từ tệp XML

Khi chạy chương trình, tên chú vẹt `Kiki` khai báo trong tệp XML sẽ được in ra màn hình điều khiển.

Phụ lục C

Giới thiệu nhanh về giao thức HTTP (A quick introduction to HTTP)

Trong phụ lục này, chúng ta thảo luận về các khía cạnh cốt lõi của giao thức **HTTP** mà mọi lập trình viên web đều cần nắm vững. May mắn thay, bạn không cần phải là chuyên gia giao thức mạng hay thuộc lòng hàng trăm trang tài liệu RFC để có thể xây dựng nên những ứng dụng web tuyệt vời với Spring. Tuy nhiên, việc nắm chắc nguyên lý của HTTP là điều kiện tiên quyết để hiểu rõ cách thức Spring MVC và REST controller vận hành (từ Chương 7 đến Chương 11).

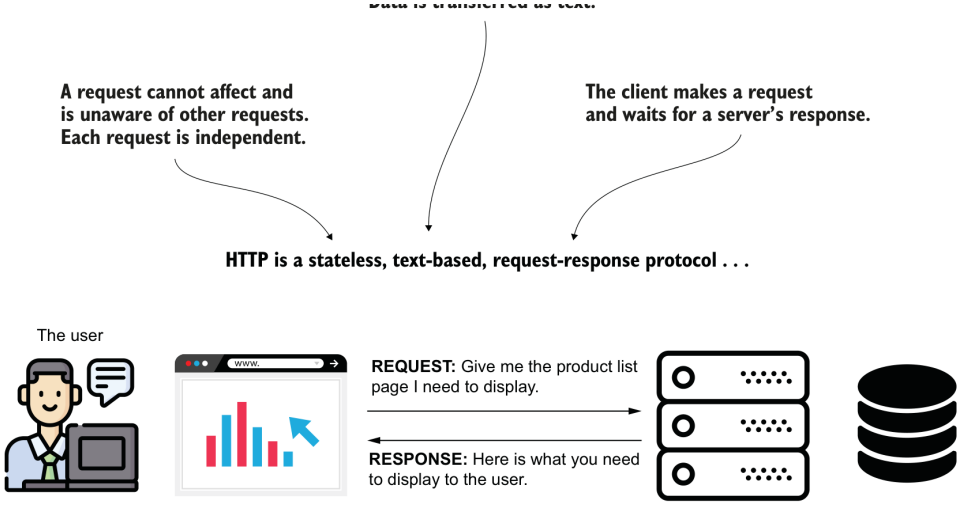
Tại sao chúng ta lại học về HTTP trong một cuốn sách viết về Spring? Bởi vì ngày nay, phần lớn các hệ thống được xây dựng bằng Spring đều là các ứng dụng web hoặc dịch vụ web — và web thì vận hành trên nền tảng HTTP!

C.1 HTTP là gì? (What is HTTP?)

Tôi luôn ưu tiên những định nghĩa súc tích: **HTTP là phương thức giao tiếp giữa client (máy khách) và server (máy chủ) trong ứng dụng web**. Các phần mềm cần có những quy tắc nghiêm ngặt để "trò chuyện" cùng nhau, và giao thức chính là tập hợp các luật lệ chuẩn hóa để trao đổi thông tin đó (Hình C.1).

Ba đặc tính nền tảng của HTTP:

- **Mô hình yêu cầu - phản hồi (Request-Response)**: Client chủ động gửi yêu cầu (*request*) và chờ đợi phản hồi (*response*) từ server.
- **Dựa trên văn bản thuần (Text-based)**: Dữ liệu tiêu đề và cấu trúc gói tin được biểu diễn dưới dạng chuỗi ký tự ASCII dễ đọc.
- **Không lưu trạng thái (Stateless)**: Mỗi yêu cầu là hoàn toàn độc lập, server không tự động lưu giữ thông tin hay ngữ cảnh của các yêu cầu trước đó.



Hình C.1: HTTP là giao thức dạng văn bản (text-based), không lưu trạng thái (stateless), theo mô hình yêu cầu - phản hồi (request-response) giữa client và server.

C.2 Yêu cầu HTTP (The HTTP request)

Một thông điệp yêu cầu HTTP chuẩn gồm các thành phần sau:

1. **Định danh tài nguyên (Request URI)**: Địa chỉ đường dẫn giúp client thông báo cho server biết mình muốn thao tác với tài nguyên nào.
2. **Phương thức HTTP (HTTP Method / Verb)**: Động từ chỉ định hành động muốn thực hiện (GET, POST, PUT, DELETE, v.v.).
3. **Tham số yêu cầu (Request Parameters / Query Parameters)**: Dữ liệu ngắn dạng cặp `key=value` gắn trực tiếp trên chuỗi URL sau dấu chấm hỏi (`?`).
4. **Tiêu đề yêu cầu (Request Headers)**: Các cặp siêu dữ liệu (*metadata*) bổ sung như `Content-Type`, `User-Agent`, `Authorization`.
5. **Thân yêu cầu (Request Body)**: Khối dữ liệu chính (thường là JSON hoặc XML) gửi kèm trong các yêu cầu tạo mới hoặc sửa đổi dữ liệu.

Hình C.2 phân tích cấu trúc giải phẫu của một URI yêu cầu HTTP.

C.3 Phản hồi HTTP (The HTTP response)

Sau khi xử lý yêu cầu của client, server sẽ gửi về một gói phản hồi gồm:

- **Mã trạng thái phản hồi (Status Code)**: Một số nguyên 3 chữ số thể hiện kết quả xử lý.
- **Tiêu đề phản hồi (Response Headers)**: Siêu dữ liệu bổ trợ (như `Content-Type: application/json`).

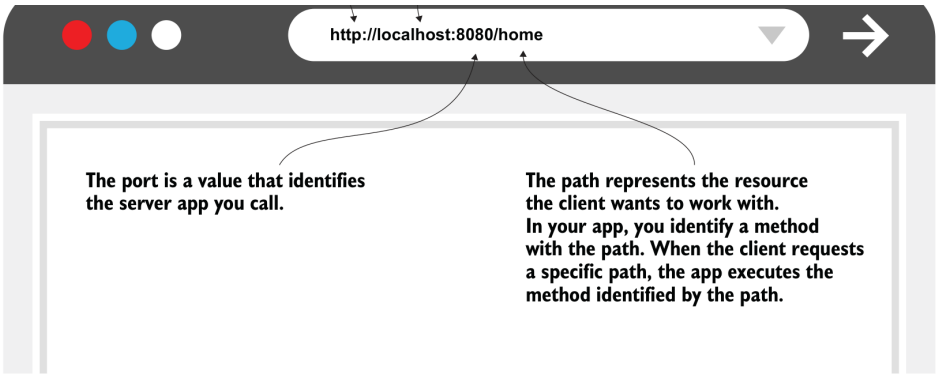


Figure C.2 The HTTP request URI identifies the resource the client requests to work with. The first part of the URI identifies the protocol and the server that runs the server app. The path identifies a resource exposed by the server.

Figure C.2 analyzes the format of an HTTP request URI.

NOTE A uniform resource identifier (URI) includes a uniform resource locator (URL) and a path. We can say the formula is URI = URL + path. But in many cases, you'll find people confusing the URI with the URL or considering them to be the same thing. You need to remember that the URL identifies

Hình C.2: Cấu trúc một HTTP URI: gồm giao thức (`http://`), địa chỉ máy chủ (`localhost`), cổng dịch vụ (`8080`) và đường dẫn tài nguyên (`/home`).

Bảng C.1: Các phương thức HTTP (HTTP Methods) phổ biến nhất

Phương thức	Ý nghĩa nghiệp vụ	Ví dụ mục đích
`GET`	Lấy dữ liệu	Truy xuất thông tin sản phẩm, hiển thị trang web.
`POST`	Thêm mới dữ liệu	Gửi biểu mẫu đăng ký, tạo mới đơn hàng.
`PUT`	Thay thế / Cập nhật toàn bộ	Ghi đè toàn bộ thông tin một tài khoản.
`PATCH`	Cập nhật một phần	Chỉ sửa đổi số dư hoặc email của tài khoản.
`DELETE`	Xóa dữ liệu	Xóa bỏ một mặt hàng khỏi kho.
`OPTIONS`	Truy vấn năng lực server	Trình duyệt thăm dò cơ chế CORS trước khi gọi API.

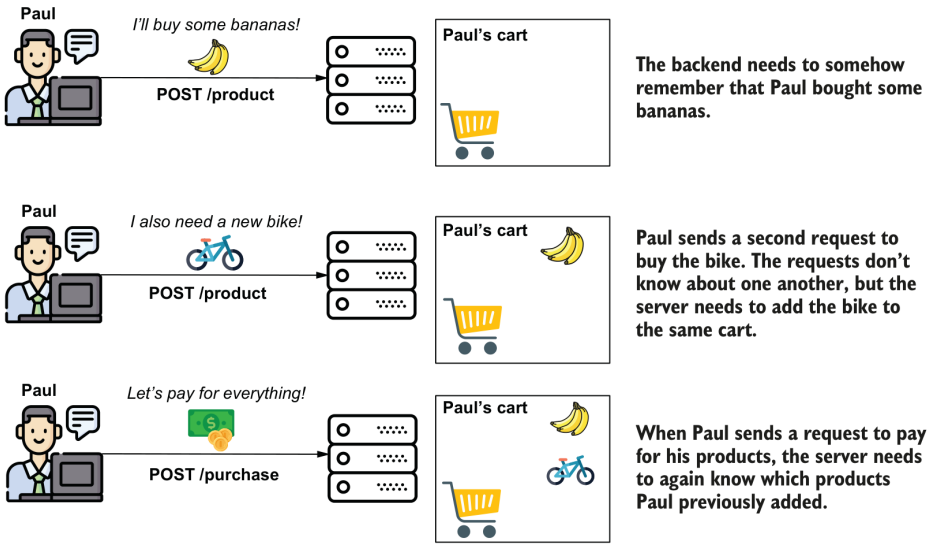
- **Thân phản hồi (Response Body)**: Dữ liệu thực sự trả về (HTML, JSON, hình ảnh, v.v.).

Các nhóm mã trạng thái quan trọng:

- **2xx (Thành công)**:
  - `200 OK`: Yêu cầu đã được xử lý hoàn hảo.
  - `201 Created`: Đã tạo mới tài nguyên thành công (thường dùng sau lệnh POST).
  - `204 No Content`: Xử lý thành công nhưng không có thân phản hồi trả về (thường dùng sau DELETE).
- **4xx (Lỗi từ phía client)**:
  - `400 Bad Request`: Yêu cầu không hợp lệ, sai cú pháp dữ liệu.
  - `401 Unauthorized`: Chưa xác thực danh tính (cần đăng nhập).
  - `403 Forbidden`: Đã đăng nhập nhưng không đủ quyền truy cập tài nguyên.
  - `404 Not Found`: Tài nguyên được yêu cầu không tồn tại trên hệ thống.
- **5xx (Lỗi từ phía máy chủ)**:
  - `500 Internal Server Error`: Lỗi bất ngờ phát sinh trong mã nguồn server (ngoại lệ chưa được bắt).

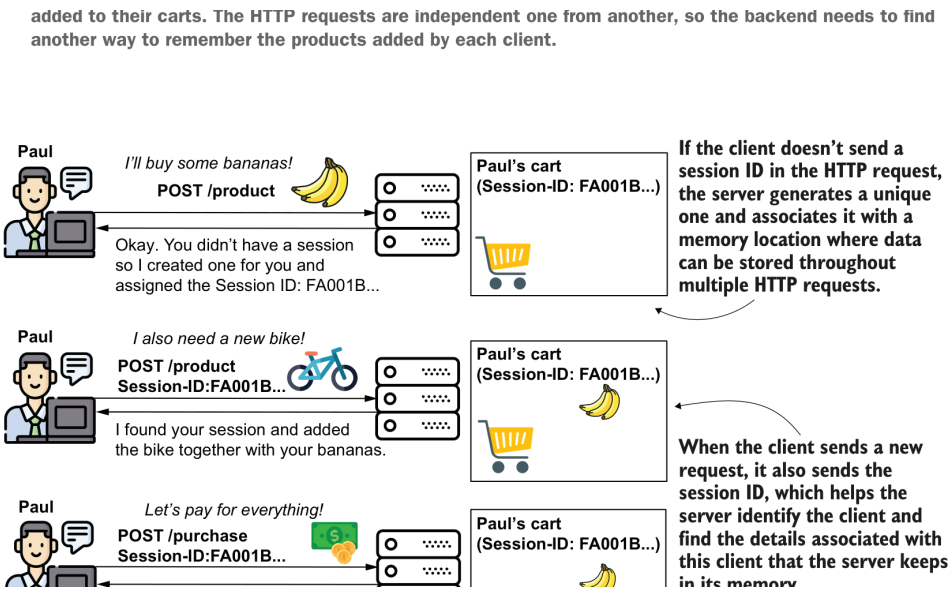
C.4 Phiên làm việc HTTP (The HTTP session)

Vì HTTP là giao thức không lưu trạng thái (*stateless*), mỗi yêu cầu hoàn toàn độc lập và không nhớ gì về các yêu cầu trước. Tuy nhiên, có những bài toán đòi hỏi server phải liên kết các yêu cầu của cùng một người dùng — ví dụ: giỏ hàng trong sàn thương mại điện tử (Hình C.3).



Hình C.3: Bài toán giỏ hàng trong thương mại điện tử đòi hỏi server phải ghi nhớ danh sách các mặt hàng người dùng đã chọn qua nhiều lượt yêu cầu độc lập.

Giải pháp kinh điển cho bài toán này là **HTTP Session**. Khi client gửi yêu cầu đầu tiên, server tạo ra một vùng nhớ riêng và cấp phát một mã định danh duy nhất gọi là **Session ID**. Client lưu mã này (thường qua Cookie) và gửi kèm trong tiêu đề của các yêu cầu tiếp theo để server nhận diện phiên (Hình C.4).



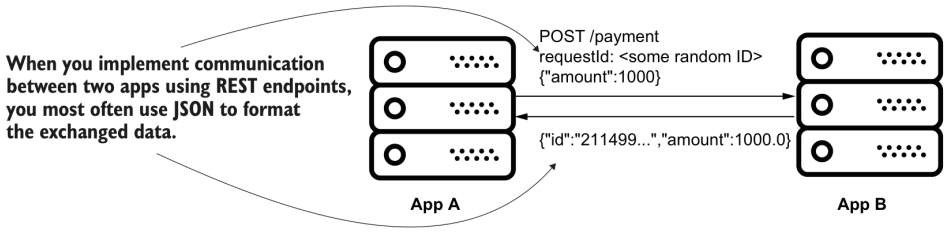
Hình C.4: Cơ chế hoạt động của HTTP Session: Server sinh mã Session ID duy nhất và ánh xạ tới vùng nhớ lưu trữ trạng thái người dùng trong suốt các phiên làm việc.

Phụ lục D

Định dạng dữ liệu JSON (Using JSON formatting)

Trong phụ lục này, chúng ta thảo luận về định dạng **JSON** (*JavaScript Object Notation*). JSON là tiêu chuẩn phổ biến nhất hiện nay để định dạng dữ liệu trao đổi giữa các ứng dụng qua yêu cầu và phản hồi HTTP khi giao tiếp bằng các điểm cuối REST (Hình D.1).

In this appendix, we discuss JavaScript Object Notation (JSON). JSON is an often-used way to format the data exchanged by apps in the HTTP request and response when using REST endpoints to communicate (figure D.1). Because REST endpoints are one of the most encountered ways to establish communication between apps, and JSON is the main way to format the data exchanged, understanding and knowing how to use JSON formatting is essential.



Hình D.1: JSON là định dạng dữ liệu chuẩn mực nhất trong kiến trúc trao đổi thông tin giữa các dịch vụ RESTful hiện đại.

May mắn thay, cú pháp JSON cực kỳ trực quan và chỉ tuân theo một vài quy tắc nền tảng:

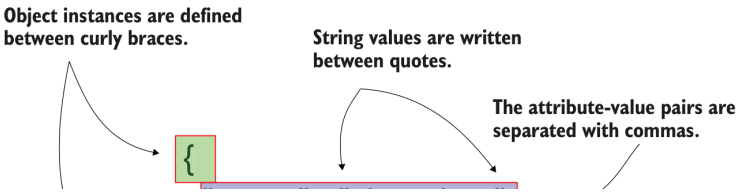
1. Một đối tượng (*object*) được bao bọc trong cặp dấu ngoặc nhọn `{ }`.
2. Bên trong cặp ngoặc nhọn là danh sách các cặp thuộc tính - giá trị (*attribute-value pairs*), phân tách nhau bằng dấu phẩy `,`.
3. Tên thuộc tính luôn được đặt trong dấu ngoặc kép `"name"`.
4. Tên thuộc tính và giá trị được phân tách bằng dấu hai chấm `:`.

5. Giá trị chuỗi ký tự đặt trong dấu ngoặc kép `"chocolate"`.
6. Giá trị số nguyên hoặc số thực viết trực tiếp không có dấu ngoặc kép (ví dụ: `5` hoặc `100.5`).
7. Giá trị logic viết là `true` hoặc `false`.

Hình D.2 minh họa chi tiết cấu trúc giải phẫu của một đối tượng JSON.

- To define an object instance in JSON, we use curly braces.
- Between the curly braces, we enumerate the attribute-value pairs, separating them with commas.
- The attribute names are written between double quotes.
- The string values are written between double quotes. (Any double quote the string contains needs to have a backslash "\" in front of it.)
- The numerical values are written without quotes.
- The attribute name and its value are separated by a colon.

Figure D.2 presents the JSON-formatted product instance with the attribute name "chocolate" and the price "5."



Hình D.2: Cấu trúc một đối tượng JSON biểu diễn sản phẩm có tên "chocolate" và đơn giá là 5.

Một đối tượng JSON có thể chứa các đối tượng lồng nhau (*nested objects*):

```
{
   "name" : "chocolate",
   "price" : 5,
   "pack" : {
       "color" : "blue"
   }
}
```

Để biểu diễn một danh sách hoặc tập hợp (*array / collection*), JSON sử dụng cặp dấu ngoặc vuông `[ ]`, phân tách các phần tử bằng dấu phẩy:

```
[
  {
    "name" : "chocolate",
    "price" : 5
  },
  {
    "name" : "candy",
    "price" : 2
```

```
  }
]
```

Phụ lục E

Cài đặt MySQL và tạo cơ sở dữ liệu (Installing MySQL and creating a database)

Trong phụ lục này, tôi sẽ hướng dẫn bạn cách cài đặt hệ quản trị cơ sở dữ liệu MySQL và tạo cơ sở dữ liệu phục vụ các bài thực hành trong Chương 12 đến Chương 14.

Trong hệ sinh thái Java và Spring, bạn có toàn quyền lựa chọn bất kỳ hệ quản trị cơ sở dữ liệu nào: MySQL, PostgreSQL, Oracle, hay Microsoft SQL Server. Về bản chất, mã nguồn Spring Boot, JDBC hay Spring Data đều hoàn toàn như nhau bất kể bạn dùng RDBMS nào. Trong cuốn sách này, tôi lựa chọn MySQL vì nó phổ biến, mã nguồn mở, nhẹ nhàng và dễ dàng cài đặt trên mọi hệ điều hành.

Quy trình thiết lập gồm 4 bước:

1. Cài đặt MySQL Server trên máy tính cục bộ.
2. Cài đặt phần mềm giao diện đồ họa MySQL Workbench.
3. Kết nối MySQL Workbench với máy chủ MySQL cục bộ.
4. Tạo cơ sở dữ liệu `spring_quickly`.

E.1 Các bước thực hiện

Bước 1: Cài đặt MySQL Server

Tải bộ cài đặt chính thức tại: `https://dev.mysql.com/downloads/mysql/` và làm theo hướng dẫn cài đặt. Trong quá trình cài đặt, hãy ghi nhớ mật khẩu tài khoản quản trị (`root`) mà bạn thiết lập.

Bước 2: Cài đặt MySQL Workbench

Tải công cụ quản trị trực quan MySQL Workbench tại: `https://dev.mysql.com/downloads/`

workbench/.

Bước 3: Kết nối tới máy chủ MySQL cục bộ

Khởi động MySQL Workbench. Nhấp chuột vào biểu tượng dấu cộng (+) cạnh mục "MySQL Connections" (Hình E.1).

account. Remember these credentials because you'll need them in step 3.

Step 2: Install a client application for your DBMS

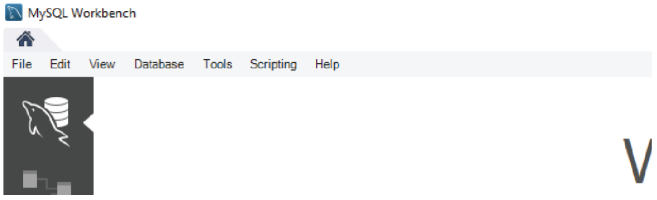
To work with the DBMS, you need a client app, which is used to create the database and sometimes alter its structure and validate your app. You need to install a client app according to the DBMS technology of your choice. If you work with MySQL, you can use MySQL Workbench, one of the most used MySQL clients.

Download MySQL Workbench installer according to your operating system (https://dev.mysql.com/downloads/workbench/) and install it as described in the installation manual: https://dev.mysql.com/doc/workbench/en/wb-installing.html.

Step 3: Connecting to the local DBMS

Once you install the DBMS on your local system and a client for your DBMS, you need to connect the client to the DBMS. To set up the connection, you need the credentials you configured in step 1. If the DBMS installation didn't request you set up a username and password, you could use the username "root" with an empty password.

I show you how to add the connection in MySQL Workbench. As presented in figure E.1, once you open MySQL Workbench, click the plus icon near "MySQL Connections."



Hình E.1: Nhấp vào biểu tượng dấu cộng cạnh "MySQL Connections" để khởi tạo kết nối mới.

Đặt tên kết nối (ví dụ: `local DBMS`), nhập tên người dùng `root` và mật khẩu bạn đã đặt (Hình E.2). Bạn có thể nhấn nút "Test Connection" để kiểm tra đường truyền kết nối.

Nếu xuất hiện thông báo cảnh báo về phiên bản (Hình E.3), bạn chỉ cần nhấn "Continue Anyway".

Khi kết nối thành công, hộp thoại xác nhận sẽ hiện ra (Hình E.4). Nhấp "OK" để lưu lại kết nối (Hình E.5).

Bước 4: Tạo cơ sở dữ liệu mới

Mở kết nối vừa tạo. Nhấp vào biểu tượng chiếc bình hình trụ nhỏ trên thanh công cụ để tạo schema mới (Hình E.6).

Đặt tên cơ sở dữ liệu là `spring_quickly` và nhấn nút "Apply" (Hình E.7).

Xác nhận câu lệnh SQL `CREATE SCHEMA` `` `spring_quickly` `` và nhấn "Apply" (Hình E.8).

Khi hoàn tất, cơ sở dữ liệu `spring_quickly` sẽ xuất hiện tại ngăn điều hướng bên trái và sẵn sàng để ứng dụng kết nối (Hình E.9).

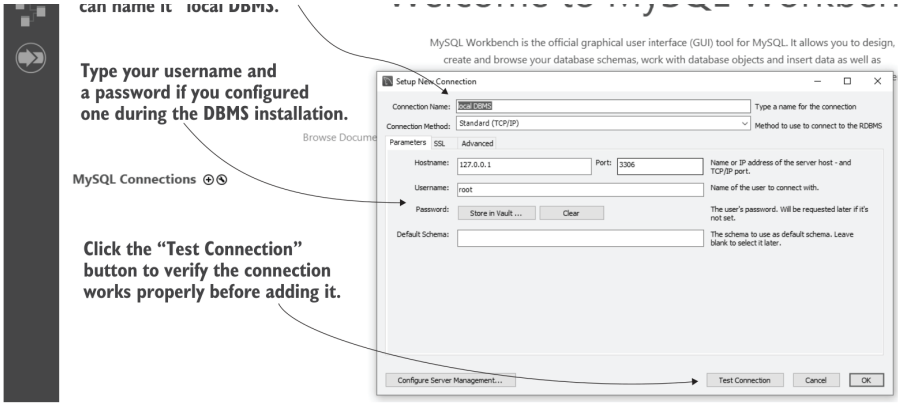
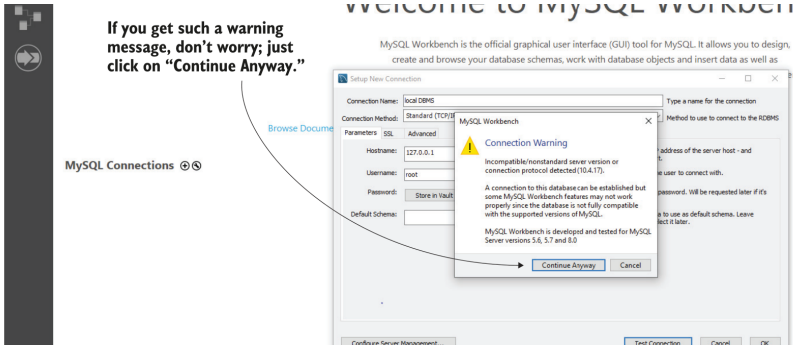


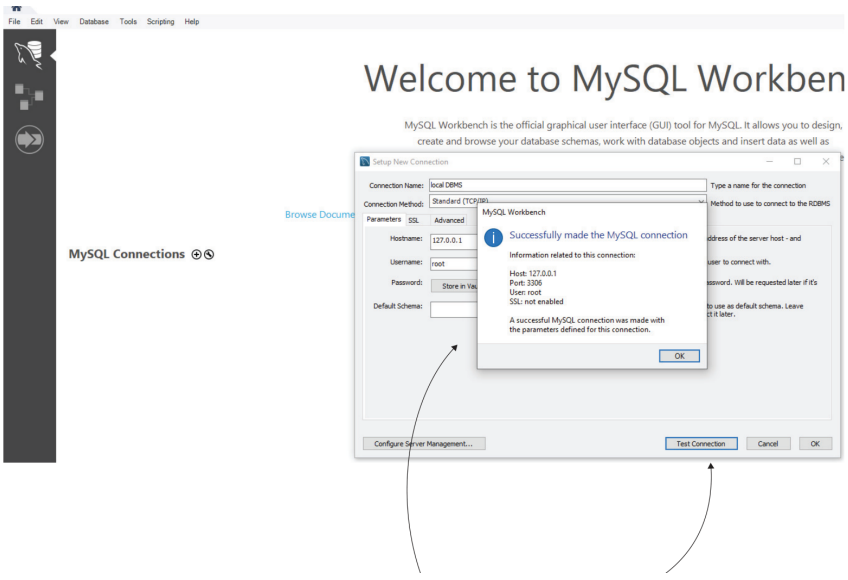
Figure E.2 Give the connection a name and fill in the credentials you configured when installing the DBMS. Then, click on the "Test Connection" button to verify that MySQL Workbench can connect to the database.

For some MySQL versions, MySQL Workbench might display a warning message. For implementing the examples in this book, this won't affect you, so click the "Continue Anyway" button if you get such a message (figure E.3).

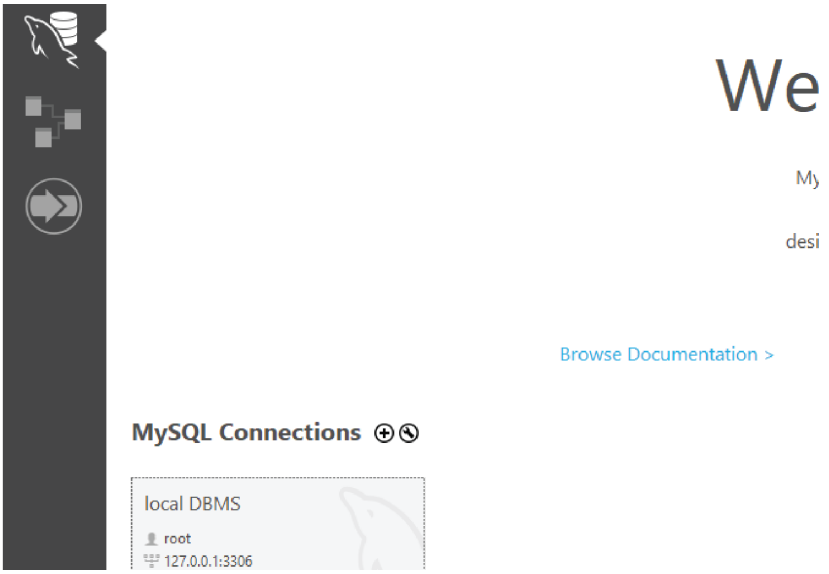
Hình E.2: Cấu hình thông số kết nối và kiểm tra kết nối với cơ sở dữ liệu.



Hình E.3: Nhấn "Continue Anyway" nếu gặp hộp thoại cảnh báo tương thích.



Hình E.4: Thông báo kiểm tra kết nối cơ sở dữ liệu thành công.



Hình E.5: Kết nối mới hiển thị trên màn hình chính của MySQL Workbench.

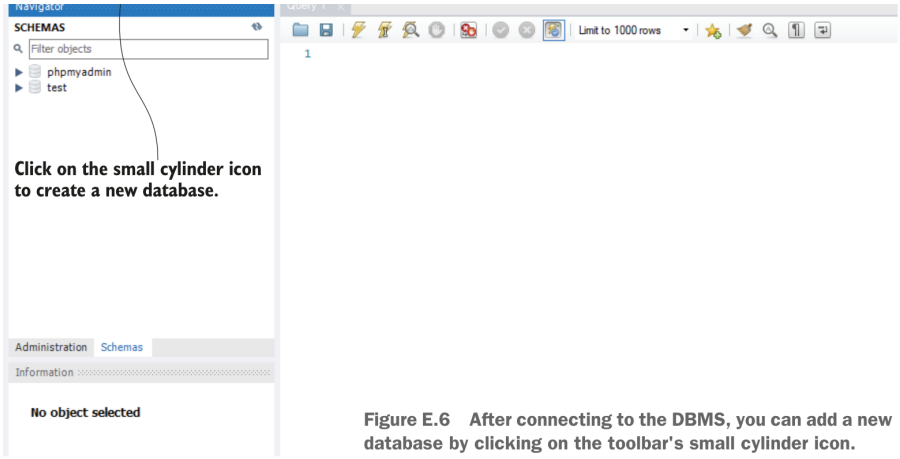
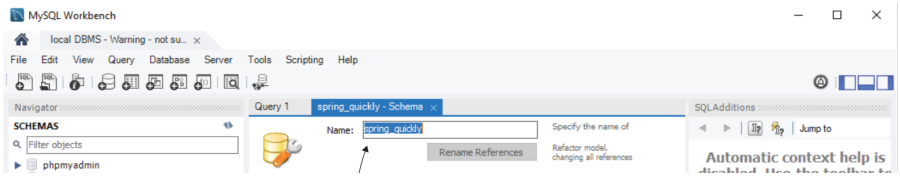
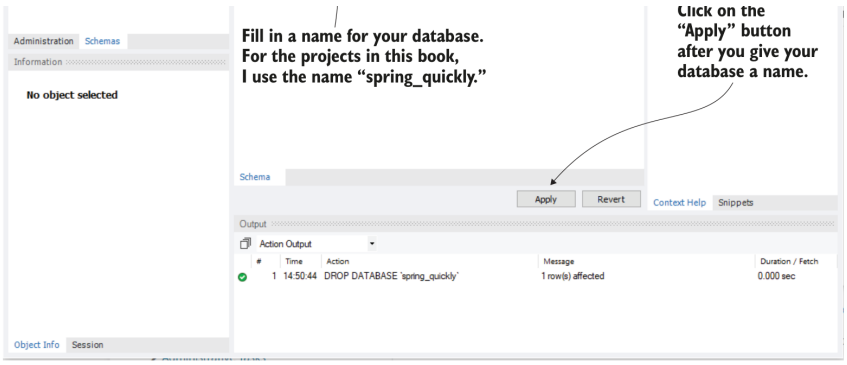


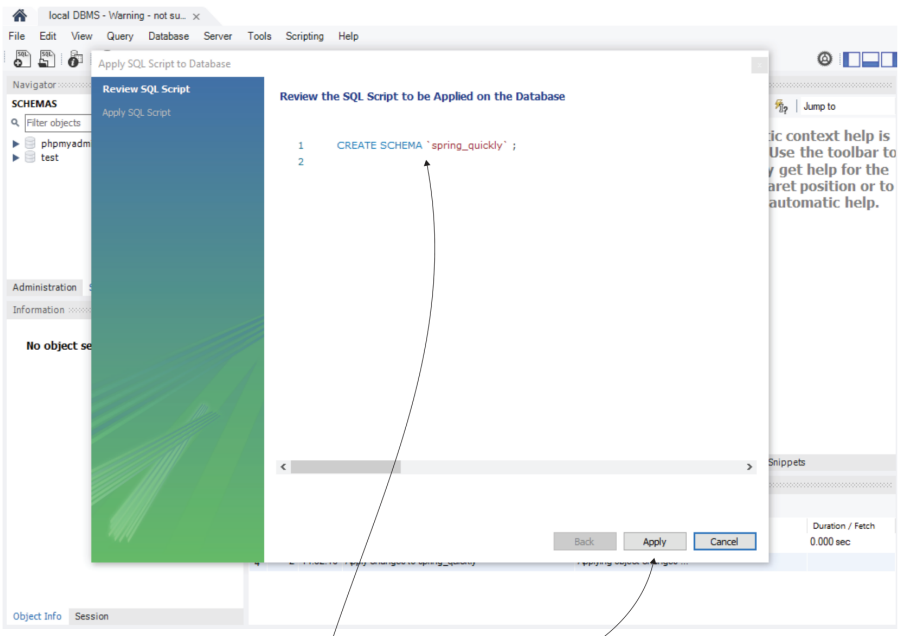
Figure E.6 After connecting to the DBMS, you can add a new database by clicking on the toolbar's small cylinder icon.



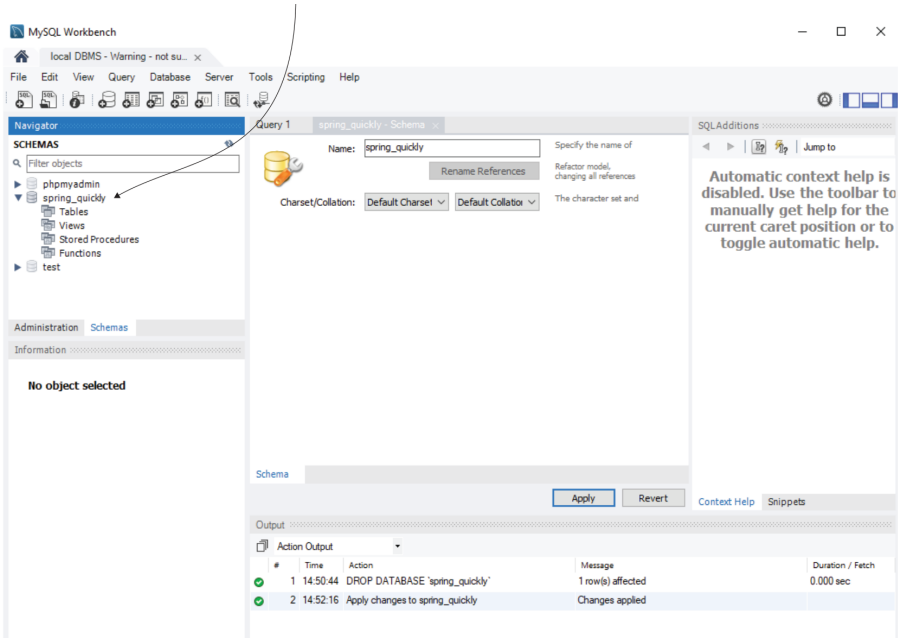
Hình E.6: Nhấp vào biểu tượng hình trụ để tạo cơ sở dữ liệu mới.



Hình E.7: Đặt tên cơ sở dữ liệu là spring_quickly và nhấn Apply.



Hình E.8: Xác nhận thực thi câu lệnh tạo schema.



Hình E.9: Cơ sở dữ liệu spring_quickly xuất hiện tại danh sách Schemas bên trái.

Phụ lục F

Các công cụ khuyên dùng (Recommended tools)

Trong phụ lục này, tôi liệt kê các công cụ lập trình và phát triển được sử dụng xuyên suốt cuốn sách, cùng các giải pháp thay thế phổ biến trong cộng đồng.

F.1 Môi trường phát triển tích hợp (IDEs)

- **JetBrains IntelliJ IDEA**: Công cụ chính được sử dụng để xây dựng toàn bộ ví dụ trong cuốn sách. Bạn có thể sử dụng phiên bản miễn phí *IntelliJ IDEA Community Edition* để thực hành. Phiên bản thương mại *Ultimate* cung cấp thêm nhiều tính năng chuyên sâu cho Spring. (`https://www.jetbrains.com/idea/`)
- **Eclipse IDE**: Môi trường mã nguồn mở kinh điển. Để có trải nghiệm tốt nhất với Spring trên Eclipse, bạn nên cài đặt bộ công cụ **Spring Tools**. (`https://www.eclipse.org/downloads/`)
- **Spring Tools (STS)**: Bộ công cụ mở rộng tích hợp sẵn cho Eclipse hoặc VS Code giúp tăng tốc lập trình ứng dụng Spring Boot. (`https://spring.io/tools`)

F.2 Công cụ kiểm thử RESTful API

- **Postman**: Công cụ đồ họa phổ biến hàng đầu để gửi yêu cầu và kiểm thử các điểm cuối REST, hỗ trợ quản lý bộ sưu tập và tự động hóa kiểm thử. (`https://www.postman.com/`)
- **cURL**: Tiện ích dòng lệnh gọn nhẹ, mạnh mẽ, có sẵn trên hầu hết các hệ điều hành hiện đại để tương tác nhanh với các API HTTP. (`https://curl.se/`)

F.3 Hệ quản trị cơ sở dữ liệu và công cụ quản trị

- **MySQL Server & MySQL Workbench**: Hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở nhẹ nhàng cùng bộ giao diện trực quan trực tiếp. (https://dev.mysql.com/downloads/)
- **SQLyog**: Giao diện đồ họa thay thế xuất sắc cho MySQL Workbench. (https://webyog.com/product/sqlyog/)
- **PostgreSQL & pgAdmin**: Lựa chọn cơ sở dữ liệu quan hệ mạnh mẽ chuẩn doanh nghiệp thay thế cho MySQL. (https://www.postgresql.org/, https://www.pgadmin.org/)

Phụ lục G

Tài liệu học tập khuyên đọc để nghiên cứu sâu hơn (Recommended learning materials for further study)

Phụ lục này tổng hợp những nguồn tài liệu và sách chuyên khảo chất lượng cao mà tôi khuyên bạn nên tiếp tục đọc sau khi hoàn thành cuốn sách này.

1. **Spring in Action (Tái bản lần thứ 6)**, tác giả Craig Walls (Manning, 2021): Tác phẩm kinh điển tiếp nối hoàn hảo cho những gì bạn đã học trong *Spring Start Here*, mở rộng sang Spring Security, truyền thông bất đồng bộ, Project Reactor, RSocket và Spring Boot Actuator.
2. **Spring Security in Action**, tác giả Laurențiu Spilcă (Manning, 2020): Cuốn sách chuyên sâu của chính tác giả về bảo mật ứng dụng với Spring Security, phòng chống các lỗ hổng bảo mật phổ biến, kiểm soát xác thực (*Authentication*) và phân quyền (*Authorization*).
3. **Spring Boot: Up and Running**, tác giả Mark Heckler (O'Reilly Media, 2021): Tài liệu tuyệt vời giúp bạn làm chủ sâu sắc các ngóc ngách của Spring Boot trong môi trường sản xuất.
4. **Reactive Spring**, tác giả Josh Long (Tự xuất bản, 2020): Khám phá lập trình phản ứng (*reactive programming*) với Spring WebFlux và Project Reactor.
5. **JUnit in Action (Tái bản lần thứ 3)**, tác giả Cătălin Tudose (Manning, 2020): Nâng tầm kỹ năng viết kiểm thử đơn vị và kiểm thử tích hợp chuyên sâu cho các ứng dụng Java doanh nghiệp.
6. **Learning SQL (Tái bản lần thứ 3)**, tác giả Alan Beaulieu (O'Reilly Media, 2020): Ôn tập và củng cố toàn diện kỹ năng viết câu lệnh truy vấn SQL từ cơ bản đến nâng cao.
7. **OCP Oracle Certified Professional Java SE 11 Developer Complete Study Guide**,

tác giả Jeanne Boyarsky & Scott Selikoff (Sybex, 2020): Cẩm nang chuẩn mực củng cố nền tảng cốt lõi về cú pháp và cấu trúc dữ liệu Java.

8. **Kênh YouTube của tác giả**: Tra cứu danh sách phát *Spring Framework playlist* trên kênh YouTube cá nhân của Laurențiu Spilcă: `https://youtube.com/c/laurentiuspilca`.
9. **Blog kỹ thuật của tác giả**: Các bài viết chuyên sâu về Java và Spring tại `https://laurspilca.com/blog`.

Tài liệu tham khảo

[1] JAXEnter. Jaxenter java trends survey. http://mng.bz/DK9a, 2020. Khảo sát xu hướng framework Java.

[2] JRebel by Perforce. Java technology report 2020. http://mng.bz/N4V7, 2020. Báo cáo khảo sát thị phần các công nghệ Java.